LLD Handbook — Vol 2: Design Patterns and Concurrency

OOP · UML · SOLID · Clean Code · Refactoring · Design Patterns · Concurrency · 45+ case studies in Java

6 parts · one place to study LLD: DesignGurus courses + classic books + GitHub problem sets, combined

what it is | problem | how it works | trade-offs | real world | interview | self-check | TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 8 — Design Patterns: Introduction & Creational Patterns**
2. **Part A — Design patterns: the big picture**
   - 8.1 What is a design pattern?
   - 8.2 A short history: from buildings to the "Gang of Four"
   - 8.3 The three categories and all 23 GoF patterns
   - 8.4 The OO principles behind every pattern
   - 8.5 How to pick a pattern
   - 8.6 Anti-patterns: when patterns go wrong
3. **Part B — Creational patterns**
   - 8.7 Creational patterns at a glance
   - 8.8 Singleton
   - 8.9 Simple Factory (an idiom, not a GoF pattern)
   - 8.10 Factory Method
   - 8.11 Abstract Factory
   - 8.12 Builder
   - 8.13 Prototype
   - 8.14 Bonus — Object Pool
   - 8.15 Creational patterns compared
   - 8.16 Chapter self-check
   - 8.17 Chapter TL;DR
4. **Chapter 9 — Structural Design Patterns**
   - 9.0 What structural patterns are
   - 9.1 Adapter Pattern
   - 9.2 Bridge Pattern
   - 9.3 Composite Pattern
   - 9.4 Decorator Pattern
   - 9.5 Facade Pattern
   - 9.6 Flyweight Pattern
   - 9.7 Proxy Pattern
   - 9.8 Putting it all together — comparison
5. **Chapter 10 — Behavioral Design Patterns (Part 1)**
   - 10.0 Behavioral patterns: the big picture
   - 10.1 Chain of Responsibility
   - 10.2 Command
   - 10.3 Iterator
   - 10.4 Mediator
   - 10.5 Memento
   - 10.6 Interpreter
   - 10.7 Chapter recap: the six patterns side by side
6. **Chapter 11 — Behavioral Design Patterns (Part 2) & Beyond GoF**
   - 11.1 Observer
   - 11.2 State
   - 11.3 Strategy
   - 11.4 Template Method
   - 11.5 Visitor
   - 11.6 GoF behavioral patterns: quick-reference summary
   - 11.7 Beyond GoF: patterns every LLD interview expects
   - 11.8 Choose the right behavioral pattern
7. **Chapter 12 — Concurrency & Multithreading for LLD (Java)**
8. **Part A — Foundations**
   - 12.1 Concurrency vs parallelism
   - 12.2 Processes vs threads
   - 12.3 Thread lifecycle & states
   - 12.4 Creating threads: `Thread`, `Runnable`, `Callable`, `ExecutorService`, virtual threads
9. **Part B — What goes wrong**
   - 12.5 Race conditions & critical sections
   - 12.6 Java Memory Model basics: visibility, happens-before, `volatile`
10. **Part C — Locks**
    - 12.7 Mutex (mutual exclusion) and `synchronized`
    - 12.8 `ReentrantLock` and reentrancy
    - 12.9 Try-lock & timed locking
    - 12.10 Read-write locks: `ReadWriteLock` and `StampedLock`
    - 12.11 Coarse-grained vs fine-grained locking
11. **Part D — Coordination**
    - 12.12 Semaphores
    - 12.13 Condition variables: `wait/notify` and `Condition`
    - 12.14 `CountDownLatch` and `CyclicBarrier`
12. **Part E — Lock-free**
    - 12.15 Compare-and-swap (CAS) & atomics
    - 12.16 Concurrent collections
13. **Part F — Liveness failures**
    - 12.17 Deadlock
    - 12.18 Livelock
    - 12.19 Starvation (and fairness)
14. **Part G — Concurrency patterns**
    - 12.20 Signaling pattern
    - 12.21 Thread pool pattern
    - 12.22 Producer-consumer pattern
    - 12.23 Reader-writer pattern
    - 12.24 Immutability & thread confinement
15. **Part H — Concurrency in LLD interviews**

- 12.25 How concurrency shows up in LLD interviews (parking, seat booking, inventory)
- 12.26 Choosing the right tool (cheat sheet)
- 12.27 Chapter self-check
- 12.28 Chapter TL;DR

16. **Chapter 13 — Classic Concurrency Problems**
    - 13.1 Print FooBar Alternately (LeetCode 1115)
    - 13.2 Print Zero Even Odd (LeetCode 1116)
    - 13.3 Fizz Buzz Multithreaded (LeetCode 1195)
    - 13.4 Building H2O (LeetCode 1117)
    - 13.5 Dining Philosophers (LeetCode 1226)
    - 13.6 Thread-safe cache with TTL
    - 13.7 Design a (simplified) Concurrent HashMap — lock striping
    - 13.8 Thread-safe bounded Blocking Queue (two ways)
    - 13.9 Concurrent Bloom Filter
    - 13.10 Multi-threaded Merge Sort (Fork/Join)
    - 13.11 Rate limiter — token bucket (thread-safe)
    - 13.12 Chapter summary
    - 13.13 Chapter self-check
    - 13.14 Chapter TL;DR

Chapter 8 — Design Patterns: Introduction & Creational Patterns

What you will learn. What a design pattern is and where the idea came from, the full Gang of Four catalogue of 23 patterns, the object-oriented principles that sit underneath all of them, how to choose a pattern (and how to *not* choose one), the anti-patterns that come from overusing them, and then — in depth — the five creational patterns (Singleton, Factory Method, Abstract Factory, Builder, Prototype), the Simple Factory idiom, and the Object Pool bonus pattern.

How to read it. Part A is the map. Part B walks through each creational pattern with the same template: *what it is → the problem → how it works → Java code (with real output) → trade-offs → when not to use it → common mistakes → real-world examples → interview answers → self-check → TL;DR*. The chapter ends with one big comparison table.

Code. Everything lives in `chapters/08-patterns-creational/code/lld/ch08/<pattern>/`. Each package (a named folder of related Java classes that also acts as a namespace) has a `Main` (the class whose `main` method the program starts from). Compile and run all of them with:

```
cd chapters/08-patterns-creational/code
javac --release 21 -d /tmp/lld_08 $(find . -name '*.java')
java -cp /tmp/lld_08 lld.ch08.singleton.Main        # or .simplefactory / .factorymethod / .abstractfactory /
.builder / .prototype / .objectpool
```

Part A — Design patterns: the big picture

8.1 What is a design pattern?

What it is

A **design pattern** is a named, reusable description of a good solution to a problem that keeps showing up in software design. It is not a library and not a piece of code you paste in. It is closer to a *recipe* you adapt to your kitchen.

Everyday analogy. Think about building houses. Every builder has to decide where to put the front door so people can move through the house easily. Over hundreds of houses, builders learned which placements work. If someone wrote that experience down as a guideline — "put the door here, for these reasons, and this is what you give up" — that guideline would be a pattern. A design pattern is the same thing for code: the combined experience of many developers, written down so you do not have to rediscover it.

Two words you will see everywhere:

- **Intent** — the one-sentence purpose of a pattern ("ensure a class has only one instance").
- **Participants** — the roles (classes/interfaces — an **interface** is a type that lists method signatures without bodies, which classes then implement) that take part in the pattern, such as *Creator*, *Product*, *Client*.

The problem it solves

Without patterns, every team solves the same recurring problems (how to create objects without hard-coding classes, how to add behaviour without editing old code, how to notify many listeners) from scratch, each in a slightly different, slightly buggy way. Patterns fix that in seven ways:

Benefit	What it means in practice
Solve common design problems	You start from a proven solution that has been refined over years.
Avoid reinventing the wheel	Less time designing, fewer surprises.
Readability and maintainability	Code that follows a known shape is easier for the next person to understand.
Shared vocabulary	Saying "this is a Factory" or "use an Observer" replaces a paragraph of explanation. This is the biggest win in interviews.
Best practices built in	Patterns embody good principles (loose coupling — parts know little about each other; open/closed — extend without editing), so code tends to be more testable.
Adaptability and scalability	Many patterns exist precisely so code can absorb change later.
Faster development	Tested paradigms speed up design decisions.

How it works

Every classic pattern description has four essential parts (this is how the Gang of Four book frames them):

1. **Name** — a word or two that becomes part of the team's vocabulary.
2. **Problem** — when to apply it: the situation and the forces in tension.
3. **Solution** — the participants, their responsibilities and how they collaborate. It is abstract: a *template*, not concrete code.
4. **Consequences** — the costs and benefits. A pattern without its trade-offs is only half a pattern.

You then **adapt** the template to your language and your problem. Two correct implementations of the same pattern can look quite different.

When not to use it

Patterns must be used with judgement. Not every problem needs one. Forcing a pattern where it is not needed adds classes, indirection (extra layers you must step through to reach the code that does the real work) and confusion — which is worse than having no pattern at all. We come back to this in 8.6 Anti-patterns.

TL;DR

A design pattern = **name + problem + solution template + consequences**. It is shared experience, not copy-paste code. Its biggest value is vocabulary and proven trade-offs; its biggest risk is overuse.

8.2 A short history: from buildings to the "Gang of Four"

Year	Event	Why it matters
1977	Architect **Christopher Alexander** publishes *A Pattern Language*, a catalogue of solutions for towns and buildings.	The idea of a "pattern" as a named, reusable solution comes from architecture, not software.
1987	**Kent Beck** and **Ward Cunningham** experiment with applying pattern languages to programming (user-interface design in Smalltalk, an early object-oriented language).	First bridge from architecture to software.
1994	**Erich Gamma, Richard Helm, Ralph Johnson and John Vlissides** publish *Design Patterns: Elements of Reusable Object-Oriented Software*. The four authors are nicknamed the **Gang of Four (GoF)**.	Catalogues **23 patterns** in three categories, with C++ and Smalltalk examples. This is the book people mean when they say "GoF patterns".
2004	*Head First Design Patterns* (Freeman & Robson, with Sierra and Bates) teaches the same patterns in Java with pictures and stories (the pizza store for factories, the chocolate boiler for Singleton, the Starbuzz coffee shop for Decorator).	Made the GoF catalogue approachable, and put the **OO principles** behind patterns front and centre.

The GoF book states two principles that run through the whole catalogue:

1. **Program to an interface, not an implementation.** Variables and parameters should be typed by abstractions (interfaces or abstract classes that say *what* something does, not *how*), so callers do not care which concrete class (an ordinary, fully implemented class you can create with `new`) they get.
2. **Favour object composition over class inheritance.** Build behaviour by *holding* other objects ("has-a"), not only by extending classes ("is-a"). Composition is "black-box" reuse; inheritance exposes a parent's internals to its children.

Creational patterns are what make principle 1 practical: if code must not name concrete classes, *someone* still has to create those objects — and creational patterns decide who and how.

8.3 The three categories and all 23 GoF patterns

What it is

The GoF grouped patterns by **purpose** — what problem family they address:

Category	Question it answers	In one line
Creational	*How do objects get made?*	Control object creation so the system does not depend on concrete classes or on the messy details of construction.
Structural	*How are objects put together?*	Compose classes and objects into larger structures while keeping them flexible — simple ways to realise relationships between entities.
Behavioral	*How do objects talk and share work?*	Assign responsibilities and design communication between objects, so you think about how objects are connected instead of about raw control flow.

The GoF also used a second axis, **scope**:

- **Class patterns** use inheritance, fixed at compile time (when the compiler translates the source code, before the program runs) (Factory Method, class Adapter, Template Method, Interpreter).
- **Object patterns** use composition, changeable at run time (while the program is running) (almost everything else).

The full catalogue

#	Pattern	Category	Intent in one line	Chapter
1	**Abstract Factory**	Creational	Create *families* of related objects without naming their concrete classes.	8
2	**Builder**	Creational	Build a complex object step by step, separating construction from the final representation.	8
3	**Factory Method**	Creational	Let subclasses decide which concrete class to instantiate through an overridable creation method.	8
4	**Prototype**	Creational	Create new objects by copying a pre-configured instance.	8
5	**Singleton**	Creational	Ensure a class has exactly one instance and give a global access point to it.	8
6	**Adapter** (also called Wrapper)	Structural	Convert one interface into another that clients expect.	9
7	**Bridge**	Structural	Split an abstraction from its implementation so both can vary independently.	9
8	**Composite**	Structural	Treat individual objects and trees of objects uniformly.	9
9	**Decorator**	Structural	Attach extra responsibilities to an object dynamically by wrapping it.	9
10	**Facade**	Structural	Give a simple, unified entry point to a complex subsystem.	9
11	**Flyweight**	Structural	Share common (intrinsic) state to support huge numbers of fine-grained objects cheaply.	9
12	**Proxy**	Structural	Provide a stand-in that controls access to another object (lazy, remote, protective…).	9
13	**Chain of Responsibility**	Behavioral	Pass a request along a chain of handlers until one handles it.	10/11
14	**Command**	Behavioral	Turn a request into an object so it can be queued, logged or undone.	10/11
15	**Interpreter**	Behavioral	Represent a small language's grammar as classes and evaluate sentences in it.	10/11
16	**Iterator**	Behavioral	Traverse a collection without exposing how it is stored.	10/11
17	**Mediator**	Behavioral	Centralise complex communication between objects in one mediator object.	10/11
18	**Memento**	Behavioral	Capture and restore an object's state without breaking encapsulation (keeping internals private).	10/11
19	**Observer**	Behavioral	Notify many dependents automatically when one object changes.	10/11
20	**State**	Behavioral	Let an object change behaviour when its internal state changes, as if it changed class.	10/11
21	**Strategy**	Behavioral	Make a family of algorithms interchangeable behind one interface.	10/11
22	**Template Method**	Behavioral	Fix an algorithm's skeleton in a base class and let subclasses fill in steps.	10/11
23	**Visitor**	Behavioral	Add new operations to a set of classes without changing those classes.	10/11

Count check: 5 creational + 7 structural + 11 behavioral = **23**. (Some course lists show ten behavioral patterns and leave out Interpreter, which is rarely used in practice — it is still one of the 23.)

Not in the GoF 23 but you will meet them: *Simple Factory* (an idiom), *Object Pool* (covered at the end of this chapter), *Dependency Injection* (handing an object the things it needs from outside instead of letting it create them), *Null Object*, *Repository*, *MVC* (Model–View–Controller, which splits an app into data, display and input handling; Head First calls it a "compound pattern" built from Observer, Strategy and Composite).

Self-check

- **Q:** Which creational pattern is a *class* pattern (uses inheritance) rather than an object pattern? **A:** Factory Method — subclasses override the creation method.
- **Q:** How many behavioral GoF patterns are there? **A:** Eleven.

8.4 The OO principles behind every pattern

Patterns are not magic; they are principles applied to recurring situations. *Head First Design Patterns* collects these principles as it goes; together with SOLID (Chapter 4 — five object-oriented design rules: Single responsibility, Open–closed, Liskov substitution, Interface segregation, Dependency inversion) they explain *why* each pattern is shaped the way it is.

Principle (plain words)	What it means	Patterns that apply it (examples)
Encapsulate what varies	Find the part of the code that changes and put it behind its own abstraction, separate from what stays the same.	Factory Method (the created type varies), Strategy, State, Builder
Favour composition over inheritance	Get behaviour by holding objects, not only by subclassing.	Abstract Factory (client *holds* a factory), Strategy, Decorator
Program to interfaces, not implementations	Depend on the supertype. Callers should not care which concrete class they get.	Every creational pattern returns an interface type
Strive for loosely coupled designs between objects that interact	Objects should know as little about each other as possible.	Observer, Mediator, factories
Open–Closed Principle — open for extension, closed for modification	Add new behaviour by adding code, not by editing tested code.	Factory Method (new product = new subclass), Decorator
Dependency Inversion — depend on abstractions, not concrete classes	High-level code and low-level code both depend on interfaces.	The core of all factory patterns: the client never calls `new ConcreteThing()`
Least Knowledge ("only talk to your friends")	A method should call only its own object, its parameters, objects it creates and its fields.	Facade
Hollywood Principle ("don't call us, we'll call you")	High-level components decide when low-level ones run.	Template Method, Factory Method (the base class calls the subclass's hook), frameworks/IoC (inversion of control)
Single Responsibility — one reason to change	Each class does one job.	Builder (construction separated from the product), Iterator

Why creational patterns exist, in one sentence: Dependency Inversion says "do not depend on concrete classes", but `new PdfDocument()` *is* a dependency on a concrete class — so creational patterns move that `new` somewhere controlled (a factory, a subclass, a builder, a prototype, a container).

8.5 How to pick a pattern

The problem it solves

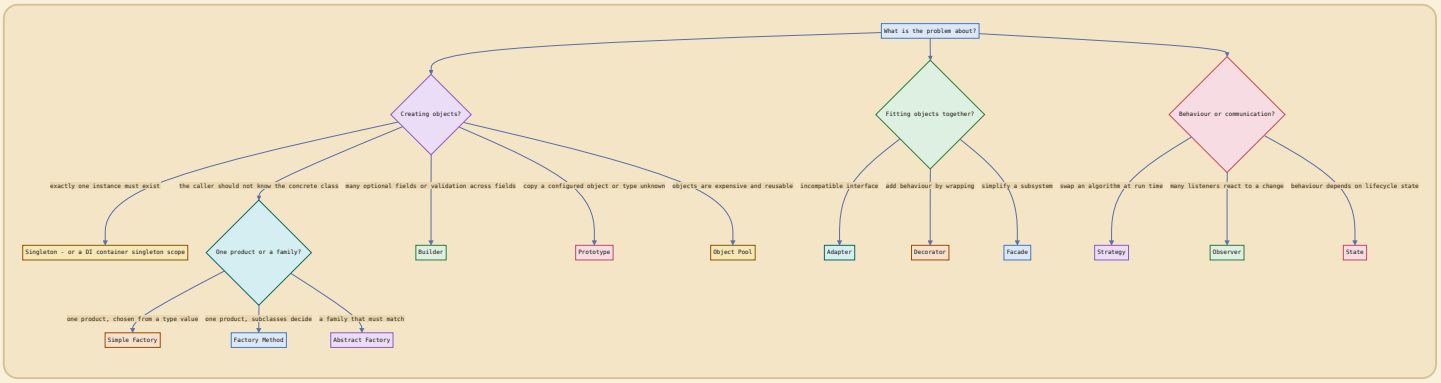
Beginners often learn 23 patterns and then look for places to use them. That is backwards. You should start from a **signal** in the requirements and only then reach for the pattern that answers it.

How it works

1. **Name the problem first**, in plain words: "the pricing rule will change", "there must be exactly one of these", "the constructor has ten arguments".
2. **Decide the category**: is it about *creating*, *structuring*, or *communicating/behaving*?
3. **Match the signal** to a pattern using the tables below.

4. **Check the cost**: how many classes does it add? Is there a simpler option (an enum — a type with a fixed list of named constants; a lambda — a short unnamed function; a constructor; a DI container — a framework that creates objects and hands them their dependencies)?
5. **Say what you rejected and why.** In an interview this is worth as much as the pattern you picked.

The flowchart below is a decision guide for the creational family plus the most common neighbours.



The six patterns that come up in almost every LLD interview

Six patterns turn up again and again in an object-oriented design round. Knowing their names pays off twice: a single word replaces a long explanation, and it signals to the interviewer that you have met this kind of problem before. The flip side matters more — dropping a pattern into a place that does not need it costs you more than never having heard of it. So learn each one together with its **signal**: the clue in the problem statement that says it fits.

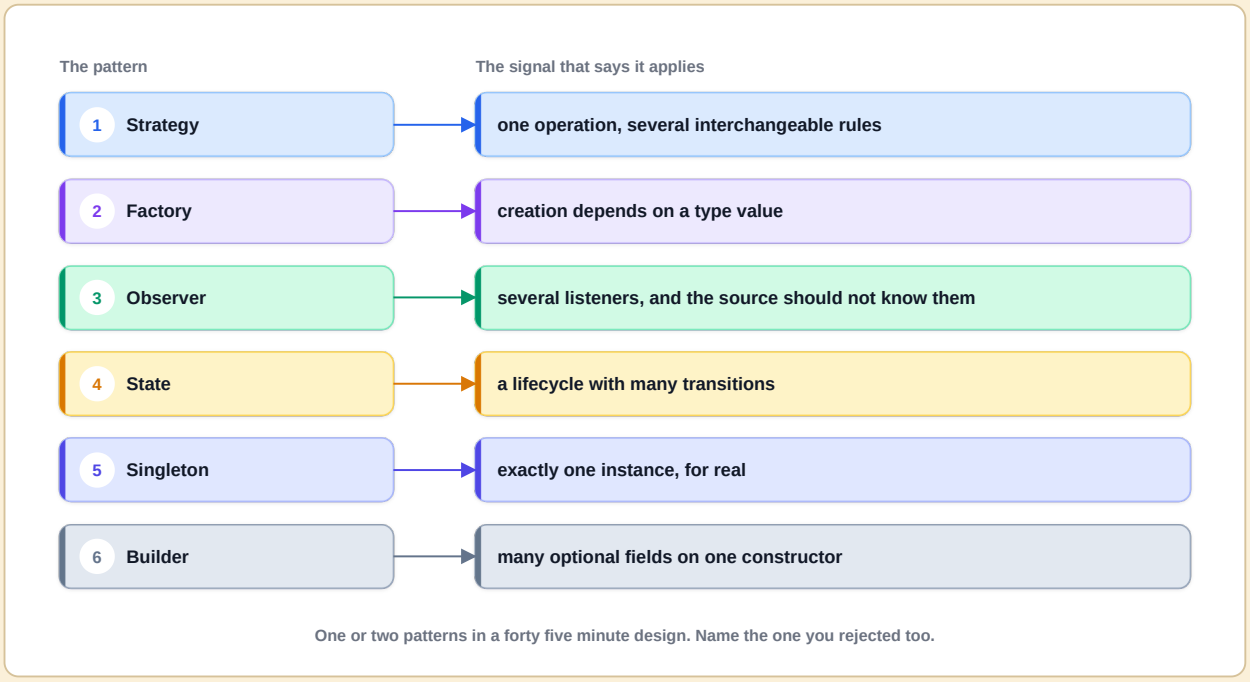


Figure — the six interview patterns and the signal that tells you each one applies (course diagram)

As the figure above shows, each pattern is paired with a trigger, not a place to force it in:

Pattern	The signal	Where it typically applies	Use it when	Do not use it when
Strategy	A single operation comes in several swappable versions, and which one runs is decided at run time	Pricing, matching, sorting, ranking, routing	There is one operation with several interchangeable rules	Only one rule exists and nothing hints at a second
Factory	Something has to be created, and the caller should not care which concrete class it gets	Turning a type value into a vehicle, spot, payment or notification	The class to create depends on a type value	Only one concrete class exists
Observer	One object changes and several others must react, without the changing object knowing who they are	Display boards, notifications, live updates, event logs	There are several listeners the source should stay ignorant of	There is a single listener the source already has a reference to
State	What an object does depends on its current status, and the allowed moves between statuses are fixed	Tickets, orders, bookings, loans — anything with a lifecycle	The lifecycle has many states and transitions	Three or four states can be handled by one guarded method
Singleton	The program genuinely needs one and only one instance	The top-level object, e.g. `ParkingLot`, `Library`	There really must be exactly one	All you want is an easy global handle
Builder	Many fields are optional and the constructor keeps getting longer	A book copy, a booking, a search query	Lots of optional fields	A handful of required fields

How each signal shows up in a parking-lot or library design:

- **Strategy.** At first the price is simply "per hour". Later the business wants a weekend tariff and a monthly pass too. With no pattern, the fee method grows a `switch` that someone has to edit whenever a tariff appears. Strategy instead defines an **interface** (a type that lists method signatures without bodies) named `ParkingRate`, whose one method turns hours into an amount; `HourlyRate`, `WeekendRate` and `MonthlyPass` implement it, and the lot holds a reference to the one currently in force. Adding a tariff means writing one class and editing nothing. Use this when the interviewer asks near the end, "what if the pricing rules change?" — the follow-up you will hear more than any other.
- **Factory.** At some point a string or enum value has to turn into a `Vehicle` object. If that conversion lives inside the entrance panel, the panel must know every vehicle type and changes whenever a type is added. Hand the job to a factory instead: it receives the type and gives back the right object, so the panel simply asks for "a vehicle". The factory becomes the single registration point for new types — the Open–Closed Principle in practice. Keep it lean: in a 45-minute round a creation method on a class you already have is normally enough; one factory class per vehicle type is more structure than the time allows.
- **Observer.** When a spot is taken, every display board on that floor must refresh. The naive version has the floor recompute its counts and push them to its single board. With Observer the spot is the *subject*, and boards, entrance panels and floor portals *subscribe* to it; on each status change the spot notifies all subscribers without knowing what they are. Adding another board needs no change to the spot. Contrast the library: when a reserved book is returned, exactly one member must be told — a direct method call is enough, because a single listener does not justify Observer.
- **State.** A parking ticket goes active → paid → closed; a book copy can be available, reserved, on loan or lost. Without the pattern every method opens with "if status is…", and the rules about which move is legal end up scattered through the class. State gives each status its own small class that knows the legal next steps, so an illegal move cannot even be written, rather than merely being absent. Be candid about the cost: with four states and three transitions, an `enum` plus one method with a guard reads more easily. Reach for State when the transition table is big, and explain why.
- **Singleton.** A car park is one physical place, so the design has one `ParkingLot`; a private constructor plus one accessor enforce that. Interviewers nearly always probe two points, so raise them yourself: fetching the instance must be correct when several threads (independent paths of execution running at the same time inside one program) ask at once (use a lock — a guard that lets only one thread at a time run a piece of code — or create it eagerly), and a singleton is global state (data that any part of the program can read and change) that makes tests harder — which is why many teams create one instance and pass it around instead. Stating that trade-off earns more credit than naming the pattern.

- **Builder.** A `BookItem` needs a barcode, price, format, purchase date, publication date and rack — an eight-argument constructor is easy to call wrongly, because two neighbouring `String` parameters can be exchanged and the compiler says nothing. A builder names every field and produces the finished object at the end. Bring it up once a constructor in your design passes about six arguments, then move on: it improves readability, it does not change the structure.

Rules of thumb for the interview:

- **One or two patterns** in a 45-minute design is the right number, placed exactly where their signal appears. If you count six, the design has probably drifted away from the requirements.
- **Compare out loud.** For example: "Strategy for the rates, because a second tariff is clearly coming; no State for the ticket, because a single guard handles three statuses." Explaining the pattern you *rejected* shows as much judgement as the one you chose — and that comparison is the real reason to learn the names.

8.6 Anti-patterns: when patterns go wrong

What it is

An **anti-pattern** is a common "solution" that looks reasonable but reliably causes more problems than it solves. Knowing them is how you avoid misusing real patterns.

Anti-pattern	What it looks like	Why it hurts	The fix
God Object (God class, the Blob)	One class knows and does almost everything: `SystemManager` with 80 methods and 40 fields.	Violates Single Responsibility. Every change touches it, merge conflicts (clashing edits by different people to the same lines) everywhere, impossible to test in isolation.	Split by responsibility (Extract Class — Chapter 7), push behaviour to the objects that own the data.
Golden Hammer	"When all you have is a hammer, everything looks like a nail." The team's favourite tool or pattern is applied to every problem.	The solution is forced onto problems it does not fit, e.g. every object is a Singleton, every `if` becomes a Strategy.	Start from the problem's signal (8.5), not from the tool.
Pattern overuse ("patternitis")	Factories that create one class, interfaces with one implementation "just in case", five layers of indirection for a CRUD (create, read, update, delete) screen.	More classes, more hops to read, no benefit. Fowler calls the underlying smell **Speculative Generality** (building for imagined future needs).	Apply YAGNI ("You Aren't Gonna Need It", Chapter 5): add the pattern when the *second* variant actually arrives. Refactor towards a pattern, do not start from one.
Singletonitis	Singletons used as convenient global variables.	Hidden dependencies, order-dependent tests, hard to replace.	Dependency injection; pass the object in (8.8).
Cargo-cult patterns	Copying a pattern's structure without understanding its intent (e.g. a "Builder" that only copies fields with no validation, a "Factory Method" that is really a switch).	Ceremony with none of the guarantees.	Know the *intent*: what property does the pattern give you? If you cannot name it, drop it.

TL;DR

Patterns are a vocabulary, not a checklist. The God Object comes from ignoring design; the Golden Hammer and pattern overuse come from over-applying it.

Part B — Creational patterns

8.7 Creational patterns at a glance

What it is

Creational patterns control *how objects are created*, so that the rest of the system is not tied to the concrete classes or the construction details. They hide *which* class is instantiated, *how* it is assembled, and sometimes *whether* a new object is made at all.

Using them lets a system:

- be **independent** of how its objects are created, composed and represented;
- **hide the complexity** of creation (long setup, validation, choosing a subtype);
- keep **flexibility** about *what* gets created, *who* creates it, *how*, and *when*.

Pattern	Intent in one line
Singleton	Exactly one instance of a class, with one global way to reach it.
Factory Method	A base class defines a creation method; subclasses decide which concrete class it returns.
Abstract Factory	One factory object creates a whole *family* of related objects that must match.
Builder	Assemble one complex object step by step; the same construction process can produce different representations.
Prototype	Create new objects by cloning a prototypical instance.
Simple Factory (idiom)	One method with a switch/registry decides which class to create.
Object Pool (bonus)	Reuse a bounded set of expensive objects instead of creating and destroying them.

8.8 Singleton

What it is

The **Singleton** pattern restricts a class to **exactly one instance** (per process — one running program — or, strictly, per class loader — the part of the JVM, the Java Virtual Machine that runs compiled Java code, which loads classes into memory) and gives every caller **one well-known way to reach it**.

Everyday analogy. A country has one official government. Whoever needs "the government" — a citizen, a company, a foreign embassy — reaches the same one through a known channel. Nobody is allowed to set up a second one.

Terms used below:

- **Instance** — one object created from a class.
- **Eager initialization** — create the object as soon as the class is loaded.
- **Lazy initialization** — create it only the first time someone asks for it.
- **Thread-safe** — gives correct results even when many threads call it at the same moment.

GoF intent and participants. Intent: guarantee that a class never has more than one instance, and give every client a well-known way to reach it. There is a single participant, **Singleton**, which (a) defines a class-level operation (`getInstance()`) that lets clients reach the unique instance and (b) is usually responsible for creating that instance itself. Clients simply call that operation.

The problem it solves

A few things in a program really are "one per process": the settings loaded from a file when the app starts, the pool of database connections (a set of open connections kept ready for reuse), the registry that collects metrics (counters and timings the app records about itself). Let every module create its own and you end up with many copies of the settings, the same file parsed many times, and metrics split across registries whose totals no one can combine.

Picture a `ConfigStore` filled from a settings file: the cloud region to talk to, the retry count, the timeout (how long to wait before giving up). The billing service reads it, so does the nightly reporting job, so does the mailer. If each of them builds a private copy, the file is parsed three times, and a value one service changes never reaches the other two.

A plain global variable looks like the easy fix, but it has two problems. Tests all share it, so a test's result can depend on which test happened to run before it. And *some* code has to assign it before anyone reads it — get the start-up order wrong and a caller finds `null` (Java's "no object here" value). Singleton moves both jobs into the class: **the class decides how many instances may exist and creates the one it allows**, on demand. Callers never assign anything, so there is no ordering mistake to make.

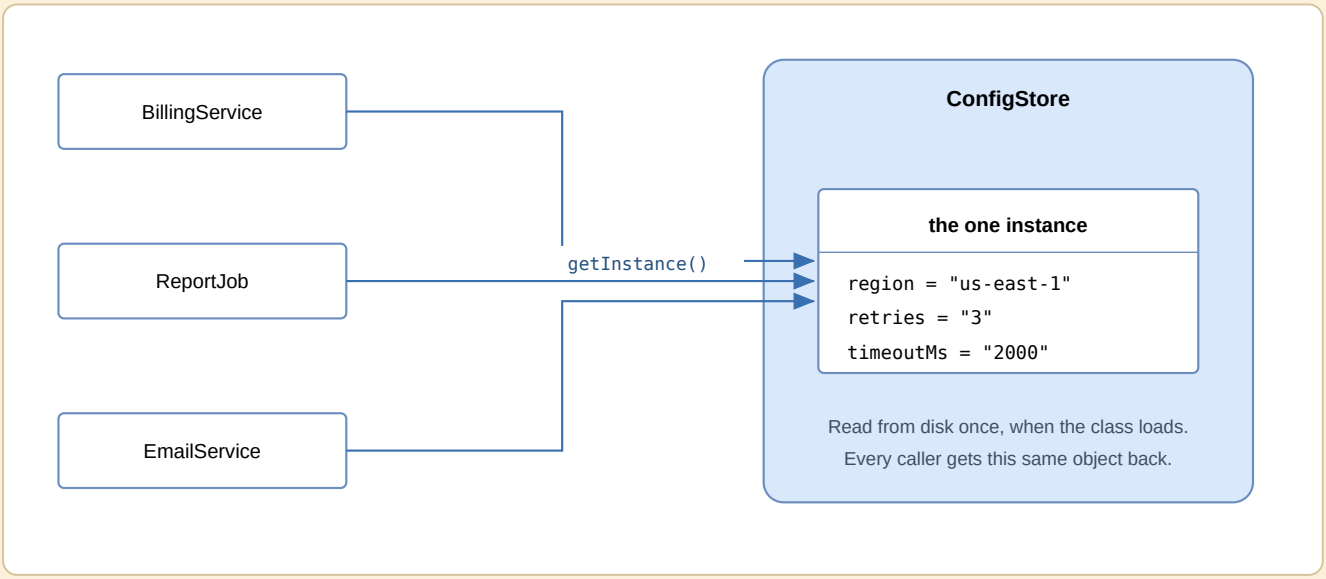


Figure — BillingService, ReportJob and EmailService all call getInstance() and receive the same ConfigStore, which was read from disk once (course diagram)

In the figure, none of the three services constructs a `ConfigStore`; each one asks the class for it and gets back the very same object, whose settings were loaded a single time.

How it works

1. Make the **constructor** `private`, so code outside the class cannot write `new ConfigStore()`. That access modifier (the `public` / `protected` / `private` keyword that says who may use a member) is the whole enforcement mechanism; make the constructor public and any caller can create a second one.
2. Keep the single instance in a `private static` **field** — static because it is owned by the class itself, not by whichever object asks for it.
3. Expose a `public static getInstance()` method. It has to be static: a caller has no instance yet (getting one is the whole point), so it can only call something on the class.
4. Inside `getInstance()`, return the stored instance, creating it first if needed (and doing that creation safely when threads are involved).
5. Add ordinary instance methods (`get`, `set`, …) for the actual work.

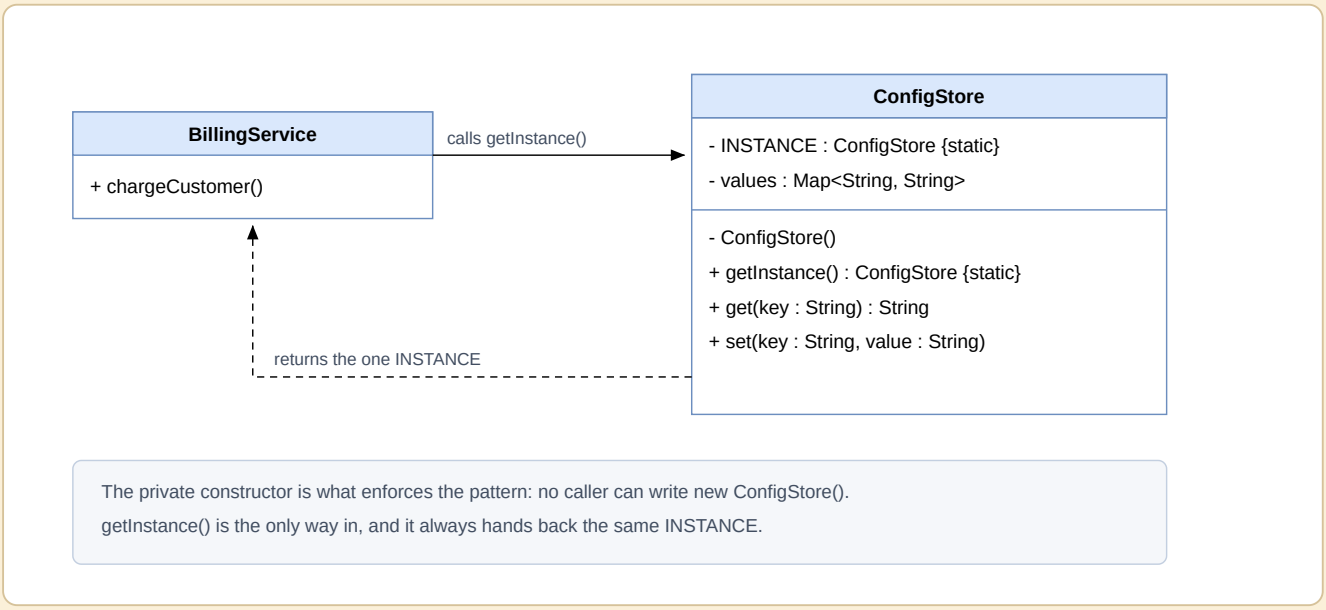
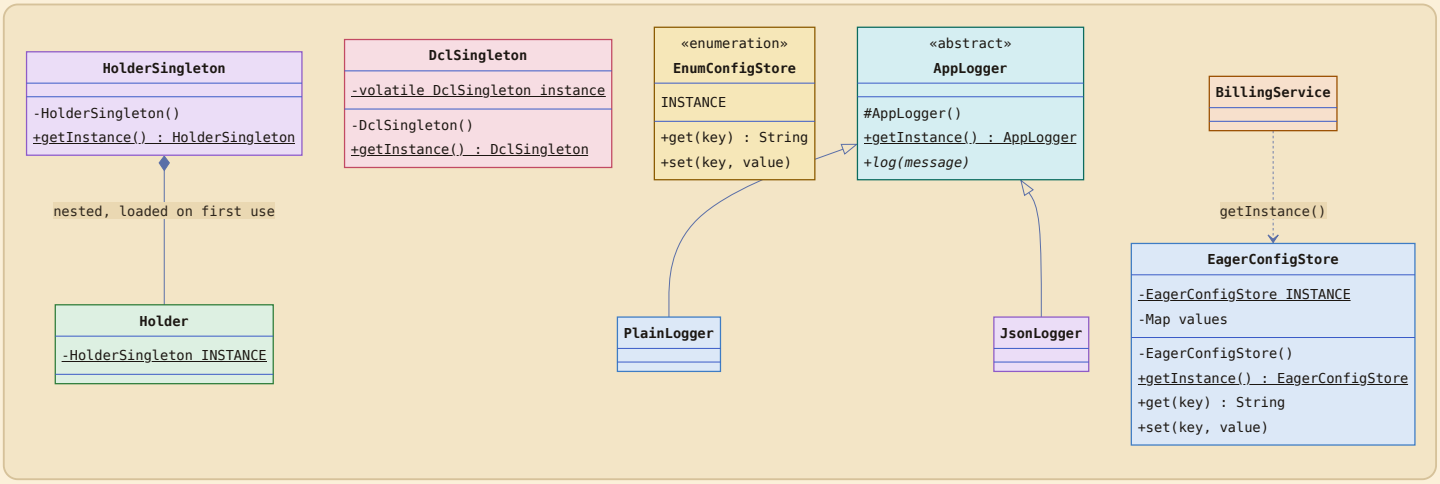


Figure — ConfigStore with a private static INSTANCE, a private constructor and a static getInstance(); BillingService only calls getInstance() (course diagram)

Our Java design (all the variants in the code package):



The six ways to write a Singleton in Java

Two terms in the table: **serialization** turns an object into bytes so it can be saved or sent (and **deserialization** rebuilds an object from those bytes); **reflection** is Java's ability to inspect and call a class's members — even private ones — at run time. "Contention" means many threads competing for the same lock.

#	Variant	Lazy?	Thread-safe?	Cost on each call	Survives serialization / reflection?	Verdict
1	**Eager** `static final` field	No (built when the class is initialized)	Yes — the JVM initializes a class exactly once	None	No / No (needs defences)	Default choice when construction is cheap
2	**Naive lazy** `if (instance == null)`	Yes	**No**	None	No / No	Never in multi-threaded code
3	**`synchronized` getInstance()**	Yes	Yes	A lock on *every* call	No / No	Correct but slow under contention
4	**Double-checked locking** + `volatile`	Yes	Yes (Java 5+)	One volatile read	No / No	When the instance depends on run-time input
5	**Bill Pugh / holder idiom**	Yes	Yes — class-loading guarantee	None	No / No	Best lazy option in plain Java
6	**`enum` with one constant**	No (created when the enum class initializes)	Yes	None	**Yes / Yes** (JVM guarantees)	Shortest and most robust; recommended by *Effective Java* (Joshua Bloch's best-practice book)

Let us walk through each one.

1 — Eager. The `static final` field is assigned while the class is being *initialized*. In Java that happens on the class's first active use — typically the first `getInstance()` call, but also the first access to any other static method or non-constant static field of the class — not at JVM start-up. The Java Language Specification (§12.4.2) guarantees that initialization runs exactly once, under an internal lock, even if many threads trigger it at the same moment, and that every thread afterwards sees the fully built object — so you get thread safety without writing any. The only cost: the object is built as soon as the class is touched for *any* reason, even if that code path never needs the instance. If the constructor opens a file or a socket (an endpoint of a network connection), that may matter.

2 — Naive lazy (broken). "Create on first use" is written as *check, then act*:

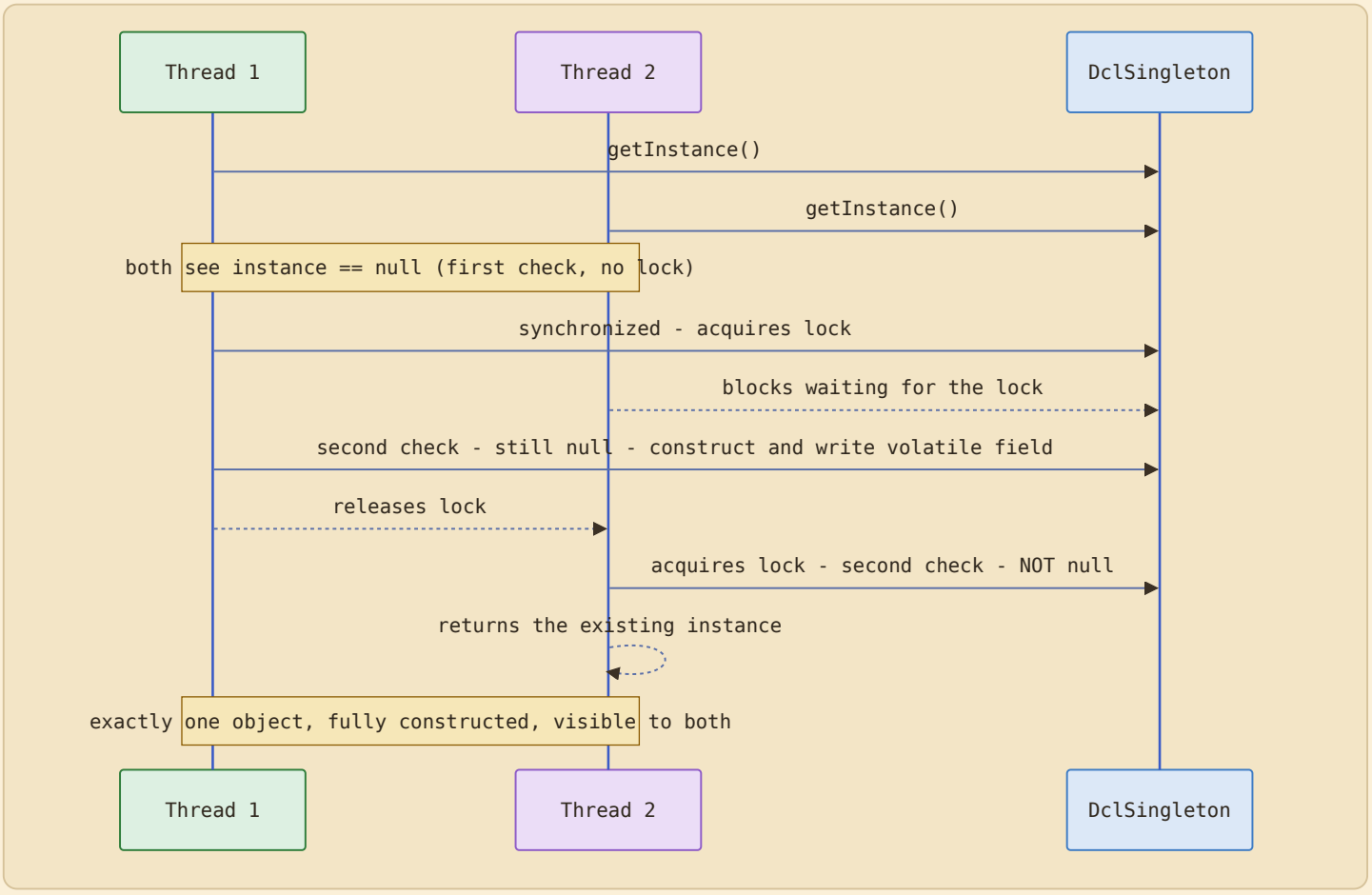
```java
// Do NOT use this in multi-threaded code.
public static ConfigStore getInstance() {
    if (instance == null) {           // two threads can both see null here...
        instance = new ConfigStore();  // ...and both construct
    }
    return instance;
}
```

Nothing stops thread A and thread B from both reading `null` before either has stored its object. Each then constructs its own store, and whatever one caller writes into its copy never shows up in the other's. Run it in a single-threaded test and it behaves perfectly, which is exactly why the bug slips through review. Our demo starts eight threads at the same instant: the naive version built **more than one** object (eight in the run shown below).

3 — `synchronized` method. Putting `synchronized` on `getInstance()` fixes the race (a **race condition**: a bug whose outcome depends on the unlucky timing of threads) — only one thread can be inside at a time. But the lock is taken on *every* call, forever, even though it is only needed for the first one.

4 — Double-checked locking (DCL). Check *without* the lock first (fast path), and only if the instance is missing, take the lock and **check again** (another thread may have created it while you waited):

- The second check is what prevents two constructions.
- The field **must be** `volatile` (a Java keyword that makes each write to the field immediately visible to every thread and forbids reordering around it). Creating an object is really three steps — allocate memory, run the constructor, store the reference. Without `volatile`, the Java Memory Model (the rules for when one thread's writes become visible to other threads) allows another thread to see the stored reference *before* the constructor's writes are visible, i.e. a half-built object. A `volatile` write/read creates a **happens-before** relationship: everything the constructor did is visible to any thread that reads the reference. (Before Java 5's memory model, DCL was broken even with `volatile`.)
- Copying the field into a local variable (`local`) means the common path reads the volatile field only once — a small optimization shown in *Effective Java*.



5 — Bill Pugh / initialization-on-demand holder. Put the instance in a private *nested* class (a class declared inside another class). The JVM does not initialize `Holder` until `getInstance()` first touches `Holder.INSTANCE`, so creation is lazy; and class initialization is guaranteed to run once and be safely published (made visible

to other threads only once fully built), so it is thread-safe with **no locks and no** `volatile`. Named after Bill Pugh, who led the work on the Java memory model. This is the usual choice for a lazy singleton.

6 — Enum. `enum EnumConfigStore { INSTANCE; ... }`. The JVM guarantees each enum constant exists once; `Constructor.newInstance` refuses to create enum objects; and enum serialization is special-cased to return the existing constant. It is the shortest correct version. Limits: an enum cannot extend another class (it can implement interfaces), and it is created when the enum class initializes rather than truly on demand.

Which one to pick. Default to **eager**: fewest lines and no race is possible; its only drawback matters when construction is costly. Switch to the **holder** idiom once you have *measured* a start-up cost worth avoiding. Choose **double-checked locking** when building the instance needs a value that only exists at run time (a holder's static initializer cannot take a parameter) — and always declare the field `volatile`. Choose **enum** when you also want serialization and reflection safety without writing any defensive code.

Breaking a Singleton — and the fixes

A `private` constructor is a *language* rule, not a JVM guarantee. There are three classic ways around it:

Attack	How it creates a second instance	Fix
Reflection	`getDeclaredConstructor().setAccessible(true).newInstance()` calls the private constructor.	Throw from the constructor if the instance already exists (reliable for eager/holder, racy for lazy). Or use an **enum** — the JVM refuses.
Serialization	`ObjectInputStream.readObject()` builds a new object without calling your constructor.	Add `private Object readResolve() { return INSTANCE; }` — the deserialized copy is replaced by the real one. Mark instance fields `transient` (skipped by serialization) if they should not travel. Or use an **enum**.
Cloning	If the class (or a parent) implements `Cloneable` and exposes `clone()`, `clone()` copies the object.	Override `clone()` to throw `CloneNotSupportedException` (or return `INSTANCE`). If no ancestor is `Cloneable`, `Object.clone()` is `protected` and this attack does not apply.
Multiple class loaders	Each class loader loads its own copy of the class, so each has its own "single" instance (common in app servers and plugin systems).	Load the class from a shared parent loader, or let a container own the instance.
Multiple processes / servers	A singleton is one per JVM. On four servers you have four.	Use an external coordinator (database, Redis — an in-memory data store, ZooKeeper — a coordination service) for anything that must be unique system-wide, e.g. ID generation or locks.

Our demo attacks an undefended singleton and a hardened one, then the enum (see output).

Subclassing a Singleton

The GoF book notes a singleton *can* be subclassed if its constructor is `protected` rather than `private`, and `getInstance()` decides **once, at run time**, which subclass becomes "the" instance. `AppLogger` below picks `PlainLogger` or `JsonLogger` from a system property (a key=value setting passed to the JVM at launch, e.g. `-Dlog.format=json`). With a `private` constructor (the usual case) the class cannot be subclassed at all — so the flexibility is only there if you design for it.

The dependency-injection alternative

In modern code the best singleton is often **not a Singleton class at all**. Write a normal class with a public constructor, create **one** instance at start-up (the *composition root*), and **pass it in** through constructors. "One instance" becomes a wiring decision, not a property baked into the class.

- Dependencies become visible in constructors (no hidden `getInstance()` calls).
- Tests pass a fake (a simple stand-in object written for tests) (`new BillingService(testConfig)`) — no reset hooks needed.

- Spring, Guice, Micronaut, Dagger, NestJS and friends (dependency-injection frameworks) do exactly this: in Spring every bean (an object that Spring creates and manages) is **singleton-scoped by default**, meaning *one shared instance per bean definition per container* (`ApplicationContext`) — not one per JVM, and without any private constructor. Two contexts in one JVM hold two instances, and nothing stops other code calling `new` on the class. Spring's opposite, `prototype` **scope**, creates a fresh instance on every lookup.

Java implementation

```java
// File: code/lld/ch08/singleton/EagerConfigStore.java
package lld.ch08.singleton;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

/** 1. Eager: the instance is created when the class is initialised. Thread-safe for free. */
public final class EagerConfigStore {

    private static final EagerConfigStore INSTANCE = new EagerConfigStore();

    private final Map<String, String> values = new ConcurrentHashMap<>();

    private EagerConfigStore() {                      // private: nobody else can call new
        System.out.println("  [EagerConfigStore] reading settings from disk (runs once)");
        values.put("region", "us-east-1");
        values.put("retries", "3");
    }

    public static EagerConfigStore getInstance() { return INSTANCE; }

    public String get(String key)             { return values.get(key); }
    public void set(String key, String value) { values.put(key, value); }
}
```

```java
// File: code/lld/ch08/singleton/LazyVariants.java
package lld.ch08.singleton;

import java.util.concurrent.atomic.AtomicInteger;

/** 2. Naive lazy: NOT thread-safe. Two threads can both see null and both construct. */
final class NaiveLazySingleton {
    static final AtomicInteger CONSTRUCTED = new AtomicInteger();
    private static NaiveLazySingleton instance;

    private NaiveLazySingleton() {
        CONSTRUCTED.incrementAndGet();
        try { Thread.sleep(20); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }

    static NaiveLazySingleton getInstance() {
        if (instance == null) {                 // check ...
            instance = new NaiveLazySingleton(); // ... then act: a race window
        }
        return instance;
    }
}

/** 3. Synchronized lazy: correct, but every call pays for the lock. */
final class SynchronizedLazySingleton {
    static final AtomicInteger CONSTRUCTED = new AtomicInteger();
    private static SynchronizedLazySingleton instance;

    private SynchronizedLazySingleton() {
        CONSTRUCTED.incrementAndGet();
        try { Thread.sleep(20); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }

    static synchronized SynchronizedLazySingleton getInstance() {
        if (instance == null) instance = new SynchronizedLazySingleton();
        return instance;
    }
}

/** 4. Double-checked locking: lock only on the first calls. volatile is mandatory. */
final class DclSingleton {
    static final AtomicInteger CONSTRUCTED = new AtomicInteger();
    private static volatile DclSingleton instance;

    private DclSingleton() {
        CONSTRUCTED.incrementAndGet();
        try { Thread.sleep(20); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }

    static DclSingleton getInstance() {
        DclSingleton local = instance;          // one volatile read on the fast path
        if (local == null) {                    // 1st check, no lock
            synchronized (DclSingleton.class) {
                local = instance;
                if (local == null) {            // 2nd check, under the lock
                    local = new DclSingleton();
                    instance = local;           // volatile write publishes a fully built object
                }
            }
        }
        return local;
    }
}

/** 5. Bill Pugh / initialization-on-demand holder: lazy, lock-free, thread-safe via class loading. */
final class HolderSingleton {
    static final AtomicInteger CONSTRUCTED = new AtomicInteger();

    private HolderSingleton() {
        CONSTRUCTED.incrementAndGet();
        System.out.println("  [HolderSingleton] constructed on first getInstance()");
    }

    private static final class Holder {         // not initialised until first touched
        private static final HolderSingleton INSTANCE = new HolderSingleton();
```

```
    }

    static HolderSingleton getInstance() { return Holder.INSTANCE; }
}
```

```
// File: code/lld/ch08/singleton/EnumConfigStore.java
package lld.ch08.singleton;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

/** 6. Enum singleton: shortest form, safe against reflection and serialization by the JVM itself. */
public enum EnumConfigStore {
    INSTANCE;

    private final Map<String, String> values = new ConcurrentHashMap<>(Map.of("region", "us-east-1"));

    public String get(String key)             { return values.get(key); }
    public void set(String key, String value) { values.put(key, value); }
}
```

```
// File: code/lld/ch08/singleton/BreakableSingleton.java
package lld.ch08.singleton;

import java.io.Serializable;

/** A parent that (unwisely) exposes a public clone(). Children inherit it. */
class CloneableBase implements Cloneable {
    @Override
    public Object clone() throws CloneNotSupportedException { return super.clone(); }
}

/** A classic eager singleton with NO defences. Reflection, serialization and cloning all break it. */
public final class BreakableSingleton extends CloneableBase implements Serializable {
    private static final long serialVersionUID = 1L;
    public static final BreakableSingleton INSTANCE = new BreakableSingleton();

    private BreakableSingleton() { }
}
```

```
// File: code/lld/ch08/singleton/HardenedSingleton.java
package lld.ch08.singleton;

import java.io.Serial;
import java.io.Serializable;

/** The same singleton with the three classic defences. */
public final class HardenedSingleton extends CloneableBase implements Serializable {
    @Serial private static final long serialVersionUID = 1L;
    public static final HardenedSingleton INSTANCE = new HardenedSingleton();

    private HardenedSingleton() {
        // Defence 1 - reflection: refuse a second construction.
        // (Works reliably with eager init, because INSTANCE is set during class init.)
        if (INSTANCE != null) {
            throw new IllegalStateException("Singleton already exists - use INSTANCE");
        }
    }

    // Defence 2 - serialization: replace whatever was deserialized with the real instance.
    @Serial
    private Object readResolve() { return INSTANCE; }

    // Defence 3 - cloning: override the inherited public clone() and refuse.
    @Override
    public Object clone() throws CloneNotSupportedException {
        throw new CloneNotSupportedException("Singleton cannot be cloned");
    }
}
```

```java
// File: code/lld/ch08/singleton/AppLogger.java
package lld.ch08.singleton;

/**
 * GoF note: a Singleton may be subclassed if the constructor is protected.
 * getInstance() decides ONCE, at run time, which subclass becomes "the" instance.
 */
public abstract class AppLogger {

    protected AppLogger() { }                              // protected, not private

    private static final class Holder {
        private static final AppLogger INSTANCE =
                "json".equals(System.getProperty("log.format")) ? new JsonLogger() : new PlainLogger();
    }

    public static AppLogger getInstance() { return Holder.INSTANCE; }

    public abstract void log(String message);
}

final class PlainLogger extends AppLogger {
    @Override public void log(String message) { System.out.println("  LOG " + message); }
}

final class JsonLogger extends AppLogger {
    @Override public void log(String message) { System.out.println("  {\"log\":\"" + message + "\"}"); }
}
```

```java
// File: code/lld/ch08/singleton/DiAlternative.java
package lld.ch08.singleton;

import java.util.Map;

/** The dependency-injection alternative: "one instance" is decided by the wiring code, not the class. */
interface Config {
    String get(String key);
}

record MapConfig(Map<String, String> values) implements Config {
    public String get(String key) { return values.get(key); }
}

final class BillingService {
    private final Config config;                           // dependency is visible in the constructor
    BillingService(Config config) { this.config = config; }
    String charge(int cents) { return "charged " + cents + " cents in " + config.get("region"); }
}

final class ReportJob {
    private final Config config;
    ReportJob(Config config) { this.config = config; }
    String run() { return "report for " + config.get("region"); }
}
```

```java
// File: code/lld/ch08/singleton/Main.java
package lld.ch08.singleton;

import java.io.*;
import java.lang.reflect.Constructor;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.Supplier;

public class Main {

    public static void main(String[] args) throws Exception {
        System.out.println("== 1. Eager singleton: every caller shares one store ==");
        EagerConfigStore billing = EagerConfigStore.getInstance();
        EagerConfigStore reports = EagerConfigStore.getInstance();
        System.out.println("  billing reads region = " + billing.get("region"));
        reports.set("region", "eu-west-1");
        System.out.println("  after reports.set, billing reads = " + billing.get("region"));
        System.out.println("  same object? " + (billing == reports));

        System.out.println("\n== 2-4. Lazy variants hit by 8 threads at the same instant ==");
        race("naive lazy (unsafe)", NaiveLazySingleton::getInstance, NaiveLazySingleton.CONSTRUCTED::get);
        race("synchronized lazy", SynchronizedLazySingleton::getInstance, SynchronizedLazySingleton.CONSTRUCTED::get);
        race("double-checked locking", DclSingleton::getInstance, DclSingleton.CONSTRUCTED::get);

        System.out.println("\n== 5. Bill Pugh holder: nothing built until first use ==");
        System.out.println("  constructed so far: " + HolderSingleton.CONSTRUCTED.get());
        HolderSingleton h1 = HolderSingleton.getInstance();
        HolderSingleton h2 = HolderSingleton.getInstance();
        System.out.println("  constructed now: " + HolderSingleton.CONSTRUCTED.get() + ", same? " + (h1 == h2));

        System.out.println("\n== 6. Enum singleton ==");
        EnumConfigStore.INSTANCE.set("retries", "5");
        System.out.println("  retries = " + EnumConfigStore.INSTANCE.get("retries"));

        System.out.println("\n== 7. Breaking a naive singleton ==");
        attack("BreakableSingleton", BreakableSingleton.INSTANCE, BreakableSingleton.class);
        System.out.println("\n== 8. The hardened version ==");
        attack("HardenedSingleton", HardenedSingleton.INSTANCE, HardenedSingleton.class);
        System.out.println("\n== 9. Enum resists reflection and serialization ==");
        attackEnum();

        System.out.println("\n== 10. Subclassable singleton (protected constructor) ==");
        AppLogger.getInstance().log("chosen subclass = " + AppLogger.getInstance().getClass().getSimpleName());

        System.out.println("\n== 11. DI alternative: one instance created by the composition root ==");
        Config shared = new MapConfig(Map.of("region", "ap-south-1"));
        BillingService billingService = new BillingService(shared);
        ReportJob reportJob = new ReportJob(shared);
        System.out.println("  " + billingService.charge(499));
        System.out.println("  " + reportJob.run());
        Config testConfig = new MapConfig(Map.of("region", "test-region"));
        System.out.println("  in a test: " + new BillingService(testConfig).charge(1));
    }

    /** Starts 8 threads that all call getInstance() at the same moment. */
    private static void race(String label, Supplier<Object> getInstance, Supplier<Integer> constructed)
            throws Exception {
        int threads = 8;
        CountDownLatch start = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newFixedThreadPool(threads)) {
            List<Future<Object>> results = new ArrayList<>();
            for (int i = 0; i < threads; i++) {
                results.add(pool.submit(() -> { start.await(); return getInstance.get(); }));
            }
            start.countDown();
            Set<Object> distinct = Collections.newSetFromMap(new IdentityHashMap<>());
            for (Future<Object> f : results) distinct.add(f.get());
            System.out.printf("  %-24s constructor ran %d time(s), callers saw %d distinct object(s)%n",
                    label, constructed.get(), distinct.size());
        }
    }

    private static void attack(String name, Object instance, Class<?> type) {
        // Reflection
```

```
        try {
            Constructor<?> c = type.getDeclaredConstructor();
            c.setAccessible(true);
            Object other = c.newInstance();
            System.out.println("  reflection    -> new object, same? " + (other == instance));
        } catch (ReflectiveOperationException | RuntimeException e) {
            Throwable cause = e.getCause() != null ? e.getCause() : e;
            System.out.println("  reflection    -> blocked: " + cause.getMessage());
        }
        // Serialization
        try {
            Object copy = roundTrip(instance);
            System.out.println("  serialization -> same? " + (copy == instance));
        } catch (Exception e) {
            System.out.println("  serialization -> failed: " + e);
        }
        // Cloning
        try {
            Object clone = ((CloneableBase) instance).clone();
            System.out.println("  clone         -> same? " + (clone == instance));
        } catch (CloneNotSupportedException e) {
            System.out.println("  clone         -> blocked: " + e.getMessage());
        }
    }

    private static void attackEnum() throws Exception {
        try {
            Constructor<?> c = EnumConfigStore.class.getDeclaredConstructors()[0];
            c.setAccessible(true);
            c.newInstance("SECOND", 1);
        } catch (IllegalArgumentException e) {
            System.out.println("  reflection    -> blocked: " + e.getMessage());
        }
        System.out.println("  serialization -> same? " + (roundTrip(EnumConfigStore.INSTANCE) ==
EnumConfigStore.INSTANCE));
    }

    private static Object roundTrip(Object o) throws IOException, ClassNotFoundException {
        ByteArrayOutputStream bytes = new ByteArrayOutputStream();
        try (ObjectOutputStream out = new ObjectOutputStream(bytes)) { out.writeObject(o); }
        try (ObjectInputStream in = new ObjectInputStream(new ByteArrayInputStream(bytes.toByteArray()))) {
            return in.readObject();
        }
    }
}
```

Explaining the parts:

- `EagerConfigStore` — the classic shape: private constructor, `static final INSTANCE`, `getInstance()`. Section 1 of `Main` shows why anyone wants this: `reports` updates the region and `billing` immediately reads the new value, even though neither holds a reference to the other — they share the one store.
- `LazyVariants.java` — each constructor sleeps 20 ms to widen the race window, and counts how many times it ran. The `race(...)` helper releases eight threads at once with a `CountDownLatch`.
- `HolderSingleton` — prints "constructed so far: 0" before first use, proving laziness.
- `BreakableSingleton` vs `HardenedSingleton` — identical except for the three defences (constructor guard, `readResolve`, throwing `clone`).
- `AppLogger` — a subclassable singleton using a holder; `DiAlternative.java` — the DI version with no singleton code at all.

Output:

```
== 1. Eager singleton: every caller shares one store ==
  [EagerConfigStore] reading settings from disk (runs once)
  billing reads region = us-east-1
  after reports.set, billing reads = eu-west-1
  same object? true

== 2-4. Lazy variants hit by 8 threads at the same instant ==
  naive lazy (unsafe)      constructor ran 8 time(s), callers saw 8 distinct object(s)
  synchronized lazy        constructor ran 1 time(s), callers saw 1 distinct object(s)
  double-checked locking   constructor ran 1 time(s), callers saw 1 distinct object(s)

== 5. Bill Pugh holder: nothing built until first use ==
  constructed so far: 0
  [HolderSingleton] constructed on first getInstance()
  constructed now: 1, same? true

== 6. Enum singleton ==
  retries = 5

== 7. Breaking a naive singleton ==
  reflection    -> new object, same? false
  serialization -> same? false
  clone         -> same? false

== 8. The hardened version ==
  reflection    -> blocked: Singleton already exists - use INSTANCE
  serialization -> same? true
  clone         -> blocked: Singleton cannot be cloned

== 9. Enum resists reflection and serialization ==
  reflection    -> blocked: Cannot reflectively create enum objects
  serialization -> same? true

== 10. Subclassable singleton (protected constructor) ==
  LOG chosen subclass = PlainLogger

== 11. DI alternative: one instance created by the composition root ==
  charged 499 cents in ap-south-1
  report for ap-south-1
  in a test: charged 1 cents in test-region
```

(The naive-lazy line is a race, so its count varies from run to run — anything above 1 is the bug.)

Trade-offs

Pros	Cons
Controlled access — the class guarantees a single instance and there is exactly one documented door to it.	**Global state** — in practice it tends to become shared mutable (changeable) state that anything can change, with all the classic problems that brings.
You choose when it is built — eagerly (simplest) or lazily on first request (cheaper start-up).	**Awkward to test** — replacing it with a fake needs a reset/injection hook, and that hook then ships in production code (the code running the live system real users depend on) where someone may misuse it.
Right fit for a shared resource — a pool, a cache or a device handle, where a second copy would be a bug.	**Invisible dependencies** — a class that calls `getInstance()` internally does not show that dependency in its constructor, so readers and tests cannot see it.
Setup cost paid once — the expensive initialization runs once per process instead of once per caller.	**Uniqueness ends at the process** (strictly, the class loader) — it cannot coordinate anything across machines, which surprises people who try.
Subclassing is possible if designed in — with a `protected` constructor, `getInstance()` can pick a subclass at run time (see `AppLogger`). With a `private` constructor, as in most examples, it cannot.	**Hard to undo** — by the time you want an ordinary class back, `getInstance()` calls are scattered everywhere. It also mixes two jobs (SRP — Single Responsibility Principle): managing its own lifecycle *and* doing its real work.

Often confused with:

Looks like	The actual difference
Static utility class (`Math`)	A class with only static members is not an object: it cannot implement an interface or be handed to a method as an argument. A Singleton *is* an object, so it can hide behind an interface and a test can substitute another implementation. That difference is the main argument for a Singleton over a static class.
Prototype	The intents point in opposite directions: Singleton caps the count at one, Prototype exists to churn out many copies.
DI container singleton scope	Achieves the same "one shared instance" with a better mechanism: the container manages the lifecycle, classes still declare what they need in their constructors, and tests swap in replacements freely.
Multiton / registry (`Logger.getLogger(name)`)	One instance *per key*, not one per class.

In other languages: in Python, a module's top-level code runs only once, so an object created at module level is already shared by every importer (the hand-written `__new__` version has the same check-then-act race as Java's naive lazy form and needs a `threading.Lock` with a double check). In JavaScript/TypeScript, exporting an object that is already constructed gives the same effect; because a single event loop (the one loop that runs JavaScript tasks one after another) never interleaves two synchronous blocks of code, no locking is needed — but each Web Worker (a background thread in the browser) has its own separate memory, so each gets its own copy. In Go, a package-level variable initialized through `sync.Once`. In C++, a function-local `static` (the "Meyers Singleton") is created on first call and the language standard guarantees that happens once even under concurrent calls. In Java, the single-element `enum` is the shortest thread-safe form and, unlike the field-based versions, survives serialization without extra code.

When not to use it

No creational pattern is misapplied more often than this one.

- **The real motive is convenience.** Wanting to reach an object from anywhere is not a design requirement — a constructor or method parameter delivers it just as well, and this motive explains most of the singletons that should never have existed.
- **It holds mutable state that tests read or write.** All tests then share that state, and you get the maddening case of a test that passes in isolation but fails when the whole suite (all the tests together) runs.
- **A framework already manages lifecycles.** In Spring, Guice, NestJS and similar containers, declare the class with singleton scope and write no singleton code at all.
- **More than one process is involved.** Each JVM has its own instance, so four servers means four "singletons". Relying on one for a distributed lock (a lock shared by several machines) or for unique IDs is a **correctness bug**, not a matter of style.

Common mistakes

- Using the **naive lazy** version in a web server or any multi-threaded code.
- Writing double-checked locking **without** `volatile`.
- Forgetting that `implements Serializable` silently breaks the guarantee unless you add `readResolve()`.
- Putting **mutable state** in a singleton without thread-safe structures (our `ConfigStore` uses `ConcurrentHashMap` because many threads may `set` concurrently).
- Adding a `public static void reset()` "for tests" — it will be called in production one day.
- Assuming "one instance" means one in the whole system (it is one per JVM / class loader).

Real-world examples

- **JDK** (Java Development Kit — Java's compiler, tools and standard library): `Runtime.getRuntime()` is the textbook case: `Runtime` has a private constructor and a single static instance for the whole JVM. `java.awt.Desktop.getDesktop()` also hands back one shared instance. `Collections.emptyList()` returns one shared immutable (unchangeable after creation) constant — singleton-*like* caching of an immutable value rather than the full pattern. `Logger.getLogger(name)` (java.util.logging) returns one shared logger per name — a registry (multiton), one instance per key.
- **Not singletons, despite the name `getInstance()`:** `Calendar.getInstance()`, `NumberFormat.getInstance()` and `Cipher.getInstance(...)` return a **new object on every call** — they are static factory methods (see 8.9). A method called `getInstance` proves nothing; what matters is whether only one instance can ever exist.

- **Spring:** beans are singleton-scoped by default — one per bean definition per `ApplicationContext`, managed by the container rather than by a private constructor. This is the DI alternative in production; it is "one per container", not "one per JVM".
- **Everyday infrastructure:** connection pools (HikariCP's pool is usually one per data source), metrics registries (Micrometer's global registry), feature-flag clients (switches that turn features on or off without redeploying) — each stands for something that truly exists once (one pool of sockets, one set of counters). Ask that question before choosing the pattern. The browser's `window` object is a singleton per page.
- **Head First's example:** a chocolate factory's boiler controller — there is one physical boiler, so two controller objects could both try to fill it and overflow it. Their fix walks through the same lazy → synchronized → double-checked / eager progression you saw above.

In the interview

> **30-second answer:** "Singleton guarantees one instance per JVM (strictly, per class loader) with a single access point: private constructor, static accessor. I'd use eager initialization — the class loader makes it thread-safe for free — or the holder idiom if construction is expensive. If it must survive serialization or reflection I'd use an enum. But it's global state with hidden dependencies, so in a Spring app I'd just make it a singleton-scoped bean and inject it."

Follow-ups:

- *Q: Why is the naive lazy version broken?* — A: It is check-then-act without synchronization; two threads can both see `null` and both construct.
- *Q: Why does DCL need `volatile`?* — A: Without it the reference can be published before the constructor's writes are visible, so another thread might use a half-initialized object. `volatile` gives a happens-before edge between the write and later reads.
- *Q: Why is the holder idiom thread-safe without locks?* — A: The JVM initializes a class once and publishes its static fields safely; the nested class is only initialized on first access.
- *Q: How can a singleton be broken?* — A: Reflection, serialization, cloning, multiple class loaders. Fixes: constructor guard, `readResolve`, throwing `clone`, or just use an enum.
- *Q: Singleton vs static class?* — A: A singleton is an object: it can implement interfaces, be passed around, be lazily created and be mocked (replaced in a test by a fake that records how it was called). A static class can do none of that.
- *Q: Is Singleton an anti-pattern?* — A: It is often *used* as one (glorified global). It is fine for genuinely single resources; prefer DI to enforce "one instance" from outside.
- *Q: How do you make a unique ID generator across 10 servers?* — A: Not with a Singleton — it is one per JVM. Use a database sequence (a database counter that hands out increasing numbers), a Snowflake-style ID (timestamp + machine id + counter), or UUIDs (128-bit randomly generated IDs that practically never repeat).

Self-check

- **Q:** Which single keyword enforces the pattern? **A:** `private` on the constructor.
- **Q:** Which Java variant is lazy, lock-free and thread-safe? **A:** The Bill Pugh holder idiom.
- **Q:** Which variant resists both reflection and serialization with no extra code? **A:** The single-element enum.
- **Q:** What does `readResolve` do? **A:** It replaces the object produced by deserialization with the one you return — the real instance.
- **Q:** In Spring, how many instances of a default bean exist? **A:** One per application context (container), not one per JVM.

TL;DR

Private constructor + static accessor = one instance per JVM. Prefer **eager**, then **holder**, then **enum** (bullet-proof); **DCL needs** `volatile`; never naive lazy. Know the four ways to break it. Mention the costs (global state, hidden deps, tests) and that DI singleton scope is usually the better tool.

8.9 Simple Factory (an idiom, not a GoF pattern)

What it is

A **Simple Factory** is one class (usually one static method) that takes a type value — a string or enum — and returns the matching object, typed as a common interface. It is extremely common and often the right answer, but it is **not** one of the 23 GoF patterns. *Head First* calls it a programming idiom rather than a real pattern.

Everyday analogy. A vending machine: you press "B3" and a snack comes out. You do not know or care how the machine picks and releases the item — you only know you asked for B3.

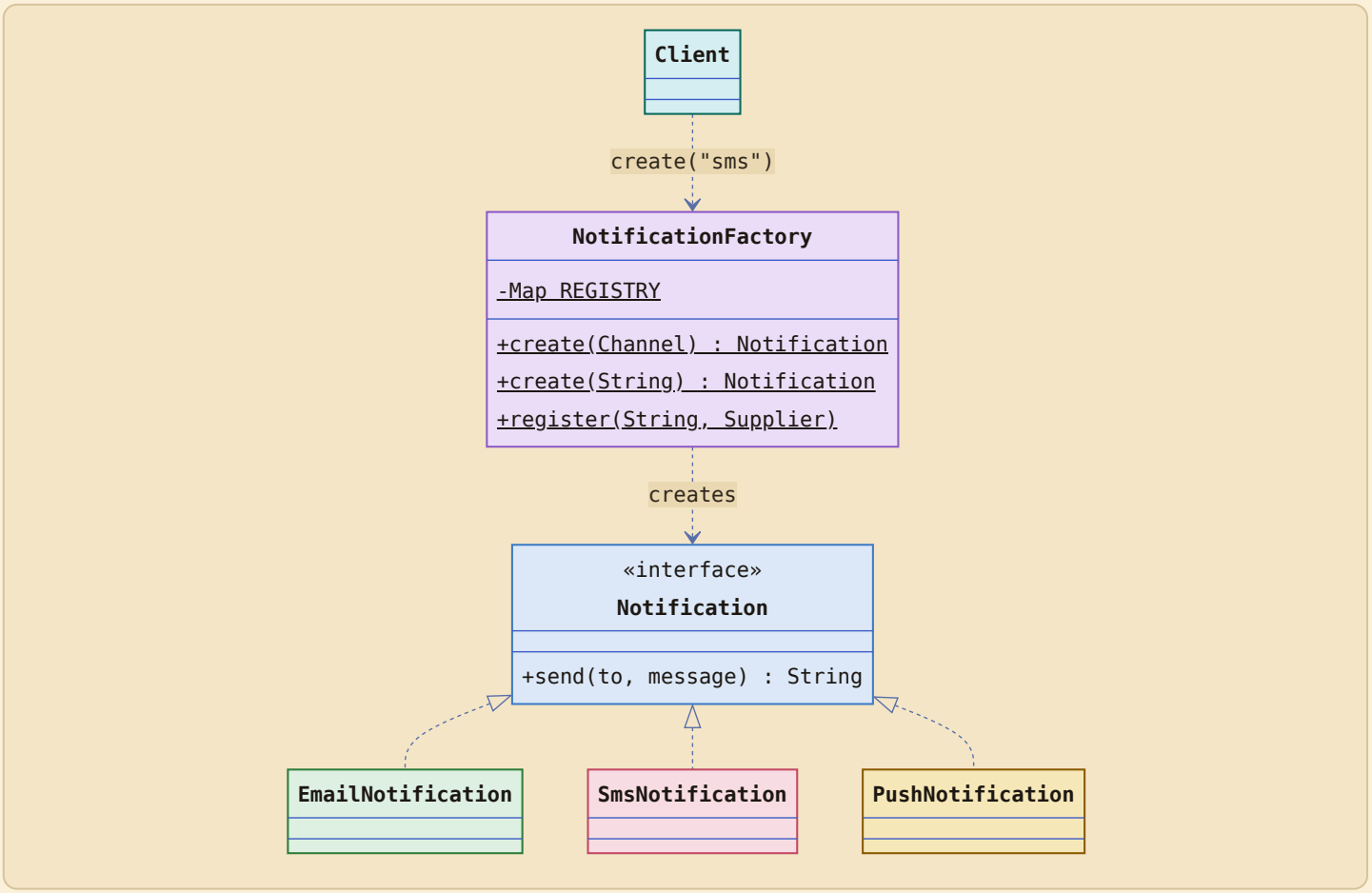
The problem it solves

Without it, the `if/else` or `switch` that turns `"sms"` into `new SmsNotification()` is copied into every place that needs a notification. A new channel means hunting down every copy. A simple factory **centralizes** that decision in one place, and callers depend only on the `Notification` interface.

Important: it **centralizes** the decision; it does not **remove** it. Adding a channel still means editing the factory, and the factory still knows every type. The registry version below softens this — new types are *registered* at start-up rather than coded into a switch.

How it works

1. Define a product **interface** (`Notification`).
2. Write the concrete classes (`EmailNotification`, `SmsNotification`, `PushNotification`).
3. Write one `create(type)` method that maps the type to a concrete class — with an exhaustive `switch` over an enum, or with a `Map<String, Supplier<Notification>>` registry (a lookup table from type names to `Supplier`s; a `Supplier` is Java's built-in "give me a new value" function type).
4. Callers call `create(...)` and hold the result as the interface. They never write `new ConcreteClass()`.
5. Reject unknown types loudly (`IllegalArgumentException`).



Java implementation

```java
// File: code/lld/ch08/simplefactory/Notification.java
package lld.ch08.simplefactory;

/** The product interface every caller programs against. */
public interface Notification {
    String send(String to, String message);
}

record EmailNotification() implements Notification {
    public String send(String to, String message) { return "EMAIL to " + to + ": " + message; }
}

record SmsNotification() implements Notification {
    public String send(String to, String message) { return "SMS to " + to + ": " + message; }
}

record PushNotification() implements Notification {
    public String send(String to, String message) { return "PUSH to " + to + ": " + message; }
}
```

```java
// File: code/lld/ch08/simplefactory/NotificationFactory.java
package lld.ch08.simplefactory;

import java.util.Map;
import java.util.Locale;
import java.util.concurrent.ConcurrentHashMap;
import java.util.function.Supplier;

/** Simple Factory (an idiom, not a GoF pattern): one static method owns the "which class?" decision. */
public final class NotificationFactory {

    public enum Channel { EMAIL, SMS, PUSH }

    private NotificationFactory() { }

    /** Version 1: a switch. Adding a channel means editing this method (and the enum). */
    public static Notification create(Channel channel) {
        return switch (channel) {                    // exhaustive switch: the compiler checks every enum value
            case EMAIL -> new EmailNotification();
            case SMS   -> new SmsNotification();
            case PUSH  -> new PushNotification();
        };
    }

    /** Version 2: a registry of suppliers. New types are registered, not coded into a switch. */
    private static final Map<String, Supplier<? extends Notification>> REGISTRY = new ConcurrentHashMap<>(Map.of(
            "email", EmailNotification::new,
            "sms", SmsNotification::new,
            "push", PushNotification::new));

    public static void register(String type, Supplier<? extends Notification> supplier) {
        REGISTRY.put(type.toLowerCase(Locale.ROOT), supplier);
    }

    public static Notification create(String type) {
        Supplier<? extends Notification> s = REGISTRY.get(type.toLowerCase(Locale.ROOT));
        if (s == null) throw new IllegalArgumentException("Unknown notification type: " + type);
        return s.get();
    }
}
```

```java
// File: code/lld/ch08/simplefactory/Main.java
package lld.ch08.simplefactory;

public class Main {
    public static void main(String[] args) {
        System.out.println("== Simple Factory with an enum switch ==");
        for (NotificationFactory.Channel c : NotificationFactory.Channel.values()) {
            Notification n = NotificationFactory.create(c);      // caller never writes "new XxxNotification"
            System.out.println("  " + n.send("ada", "Your order shipped"));
        }

        System.out.println("\n== Simple Factory with a registry (string keys from config) ==");
        System.out.println("  " + NotificationFactory.create("SMS").send("+91-98xxxx", "OTP 4821"));
        try {
            NotificationFactory.create("whatsapp");
        } catch (IllegalArgumentException e) {
            System.out.println("  rejected: " + e.getMessage());
        }
        // A new channel is registered at start-up - the factory's code is not edited.
        NotificationFactory.register("whatsapp", () -> (to, msg) -> "WHATSAPP to " + to + ": " + msg);
        System.out.println("  " + NotificationFactory.create("whatsapp").send("grace", "Table for 2 confirmed"));
    }
}
```

- The enum `switch` has **no** `default`: the Java compiler checks that every `Channel` value is covered, so adding `WHATSAPP` to the enum without handling it is a compile error. That is a nice safety net.
- The registry version maps lower-cased names to **constructor references** (`EmailNotification::new` — shorthand for "call this constructor"). `register(...)` lets start-up code add a channel without touching the factory. Because `Notification` has one abstract method, a lambda can implement it (an interface with exactly one abstract method is called a *functional interface*).
- `Locale.ROOT` avoids surprising lower-casing in some locales (language-and-region settings) (the Turkish dotless "i" problem).

Output:

```
== Simple Factory with an enum switch ==
  EMAIL to ada: Your order shipped
  SMS to ada: Your order shipped
  PUSH to ada: Your order shipped

== Simple Factory with a registry (string keys from config) ==
  SMS to +91-98xxxx: OTP 4821
  rejected: Unknown notification type: whatsapp
  WHATSAPP to grace: Table for 2 confirmed
```

Trade-offs

Simple Factory	Factory Method (next section)
One method, usually a switch	The decision is spread across subclasses, one each
Adding a type = edit the factory	Adding a type = add a class
No inheritance needed	Needs a creator subclass per product
Not a GoF pattern	A GoF pattern

Which to use: when the list of types is short, rarely changes and is selected by a configuration value, a Simple Factory is the shorter, clearer choice and is perfectly respectable. Move to Factory Method when new types will be contributed by *other* developers — perhaps in separate modules — who should be able to add them without modifying your class.

When not to use it

- There is only **one** concrete class — just call the constructor.
- The objects differ only in **data**, not behaviour — use one class with a field.

Common mistakes

- Calling a switch-based factory a "Factory Method" in an interview.

- A `default` branch that silently returns `null` or a "default" object instead of failing.
- Letting the factory grow into a God class that also configures, caches and logs.

Real-world examples

These are **static factory methods** (Joshua Bloch, *Effective Java* item 1): named static methods that return an instance, possibly of a hidden subclass. They are Simple-Factory style, **not** the GoF Factory Method — there is no Creator class whose subclasses override a creation hook — and **not** Singletons: most return a new object on each call.

- `Calendar.getInstance()` — returns a **new** `GregorianCalendar` on every call in most locales (a different calendar for a few, such as Japanese imperial or Buddhist). The name suggests Singleton; it is not one.
- `NumberFormat.getInstance()` / `getCurrencyInstance(locale)` — also a fresh (locale-specific, usually `DecimalFormat`) object per call.
- `EnumSet.noneOf(type)` — returns a `RegularEnumSet` (≤ 64 constants, one `long` bit-vector) or a `JumboEnumSet`; you never see either class.
- `List.of(...)`, `Map.of(...)`, `Optional.of(...)`, `Integer.valueOf(...)`, `Path.of(...)`.
- `Executors.newFixedThreadPool(n)`, `Executors.newVirtualThreadPerTaskExecutor()` — return configured `ExecutorService` implementations (Java's thread-pool interface for running tasks on reusable threads).
- **Spring:** `@Bean` methods (`@Bean` is an annotation — a `@` tag on code that a framework reads) in a `@Configuration` class are factory methods the container calls to create beans.

In the interview

> **30-second answer:** "A simple factory is one method that maps a type value to a concrete class and returns it as an interface. It centralizes the switch so callers don't depend on concrete classes. It's an idiom, not a GoF pattern — adding a type still edits the factory. If I want new types without editing it, I use a registry of suppliers or Factory Method."

- *Q: Is it Open–Closed?* — A: Not strictly: the factory is edited for each new type. A registry makes it open for extension via registration.
- *Q: Static factory method vs constructor?* — A: Static factories have names, can cache/return existing instances (`Integer.valueOf`), and can return subtypes. Constructors cannot.

Self-check

- **Q:** What does a Simple Factory centralize but not remove? **A:** The switch (knowledge of every concrete type).
- **Q:** Why does the enum `switch` have no `default`? **A:** So the compiler flags any unhandled enum constant.

TL;DR

One `create(type)` method, returns an interface. Great default for small, stable sets of types. Not GoF. Registry of `Supplier`s = extensible variant.

8.10 Factory Method

What it is

Factory Method defines a method for creating an object in a base class, but lets **subclasses decide which concrete class** that method returns. The base class then uses the created object without ever naming its concrete type.

Everyday analogy. A franchise restaurant chain writes one operations manual: take the order, *prepare the dish*, serve, bill. Every branch follows the manual exactly, but "prepare the dish" is left to each branch — the New York branch makes a thin-crust pizza, the Chicago branch a deep-dish one. (This is exactly *Head First*'s pizza-store example: `PizzaStore.orderPizza()` is fixed, and each regional store overrides `createPizza()`.)

Terms:

- **Product** — the interface of the object being created (`Document`).
- **Concrete product** — an implementation (`PdfDocument`).
- **Creator** — the base class that declares the factory method and contains the shared logic (`ReportGenerator`).
- **Concrete creator** — a subclass that overrides the factory method (`PdfReportGenerator`).

GoF intent and participants. Intent: declare an operation that creates an object, but leave the choice of concrete class to subclasses — the base class postpones instantiation to them. The four participants are **Product** (the interface of the objects created), **ConcreteProduct** (implements Product), **Creator** (declares the factory method returning a Product, may supply a default implementation, and — crucially — *calls* it from its own operations) and **ConcreteCreator** (overrides the factory method to return a ConcreteProduct).

> **Factory Method is not a "static factory method".** The word "factory" is overloaded. A *static factory method* (`List.of`, `Integer.valueOf`, `Calendar.getInstance`) and a *Simple Factory* (`NotificationFactory.create(type)`, 8.9) are **one static method that decides** what to return. They cannot be overridden (static methods are not polymorphic — a call cannot be redirected to a subclass's version at run time), and there is no Creator hierarchy. The GoF **Factory Method** is an **instance method, overridden in subclasses**, that the base class itself calls. If there is no inheritance and no overriding, it is not this pattern.

The problem it solves

Plenty of classes must create an object whose exact type they should not decide: a report generator needs a document, a parser needs tree nodes, a notification service needs messages. Once `ReportGenerator` contains `new PdfDocument()`, it is hard-wired to PDF, and supporting HTML means reopening a class that is finished and tested.

Our `ReportGenerator` does four things: gather the rows, create a document from them, render it, and announce the file name. **Only the "create a document" step cares about the output format**; the other three are identical for PDF, HTML or CSV. The first instinct is a `switch` on a format string. That holds up until a second method needs the same `switch`, then a third — and from then on each new format means hunting down every copy.

Factory Method instead leaves a deliberate **gap** in the base class: the complete procedure lives there, but at the moment a document is needed it calls an abstract method that the base class does not implement. Every subclass supplies just that one method.

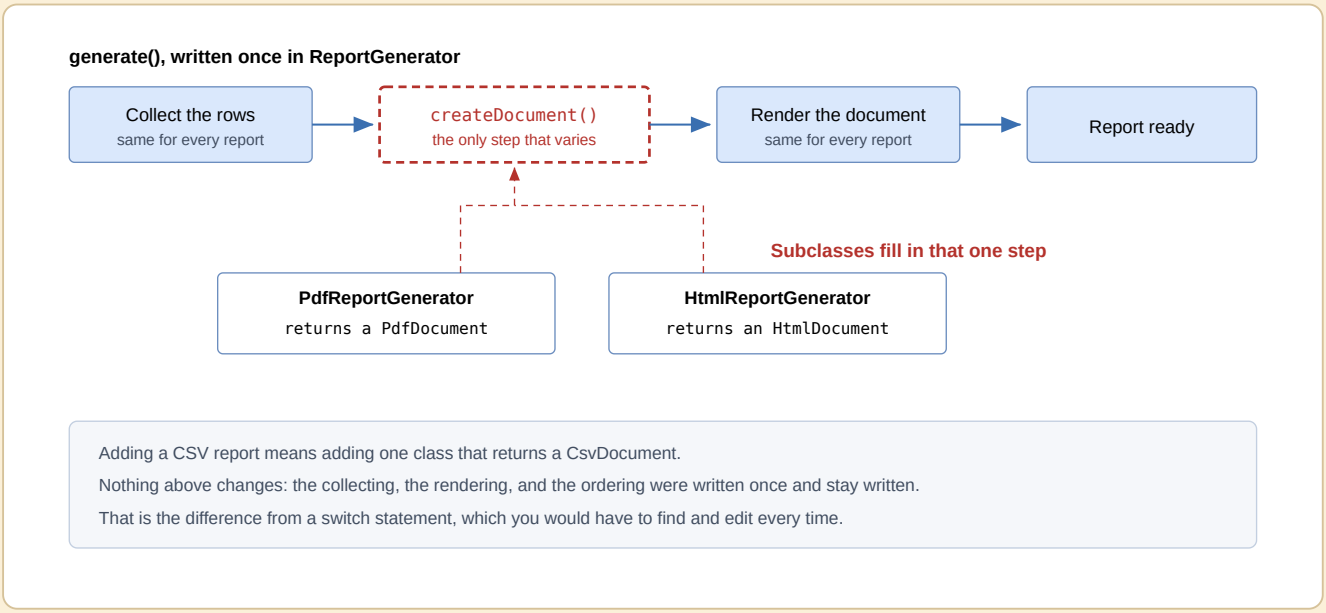


Figure — generate() is written once in ReportGenerator; only createDocument() varies, filled in by PdfReportGenerator or HtmlReportGenerator (course diagram)

In the figure, gathering, rendering and announcing are inherited unchanged; supporting CSV only requires one more subclass whose override returns a `CsvDocument`. Compare that with a `switch`, which has to be located and edited for every new format.

How it works

1. Define the **product interface** (`Document`: `render(rows)`, `fileName()`).
2. Write **concrete products** (`PdfDocument`, `HtmlDocument`, later `CsvDocument`).
3. Write the **creator** as an abstract class with an **abstract, `protected` factory method** (`createDocument()`). `protected` signals that it is an extension point meant for subclasses rather than something outside callers invoke — they call `generate()`.
4. Write the **shared logic** in the creator (`generate()`), calling `createDocument()` where it needs a product. It depends only on `Document`.

5. Write one **concrete creator** per product, overriding only the factory method. That override is the *only* place a concrete type is named.
6. The **client** picks a creator once and then works entirely through `ReportGenerator` and `Document`.

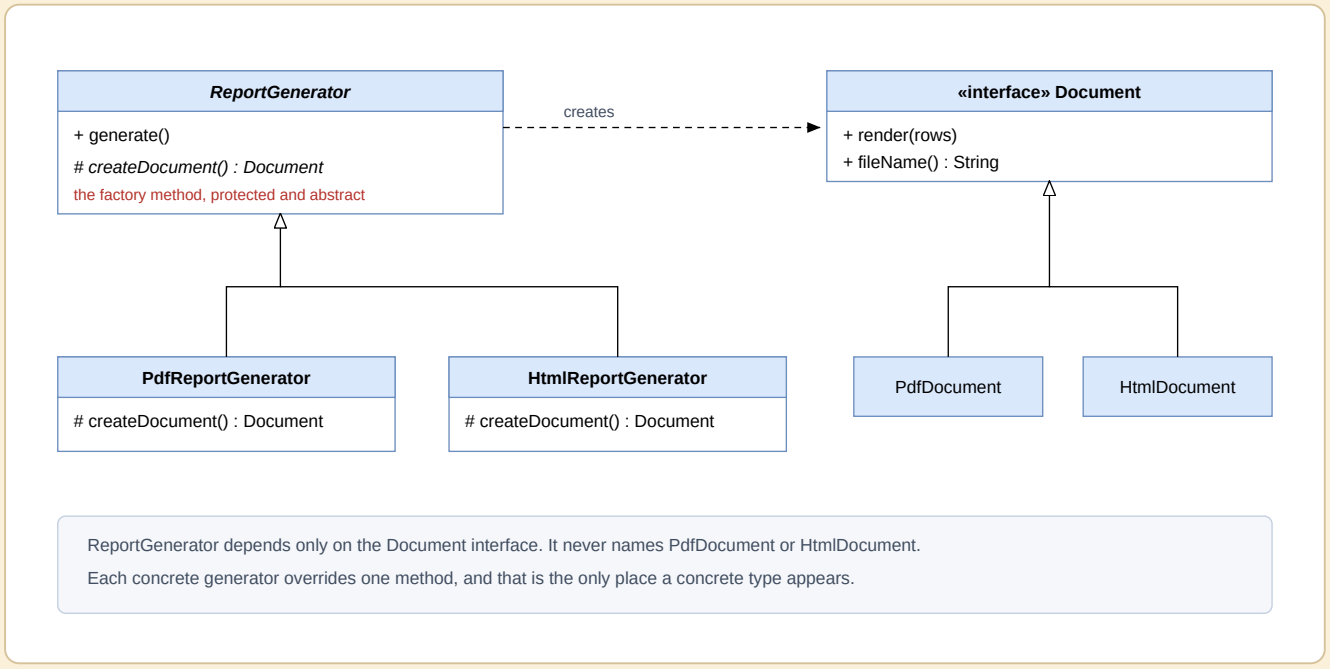
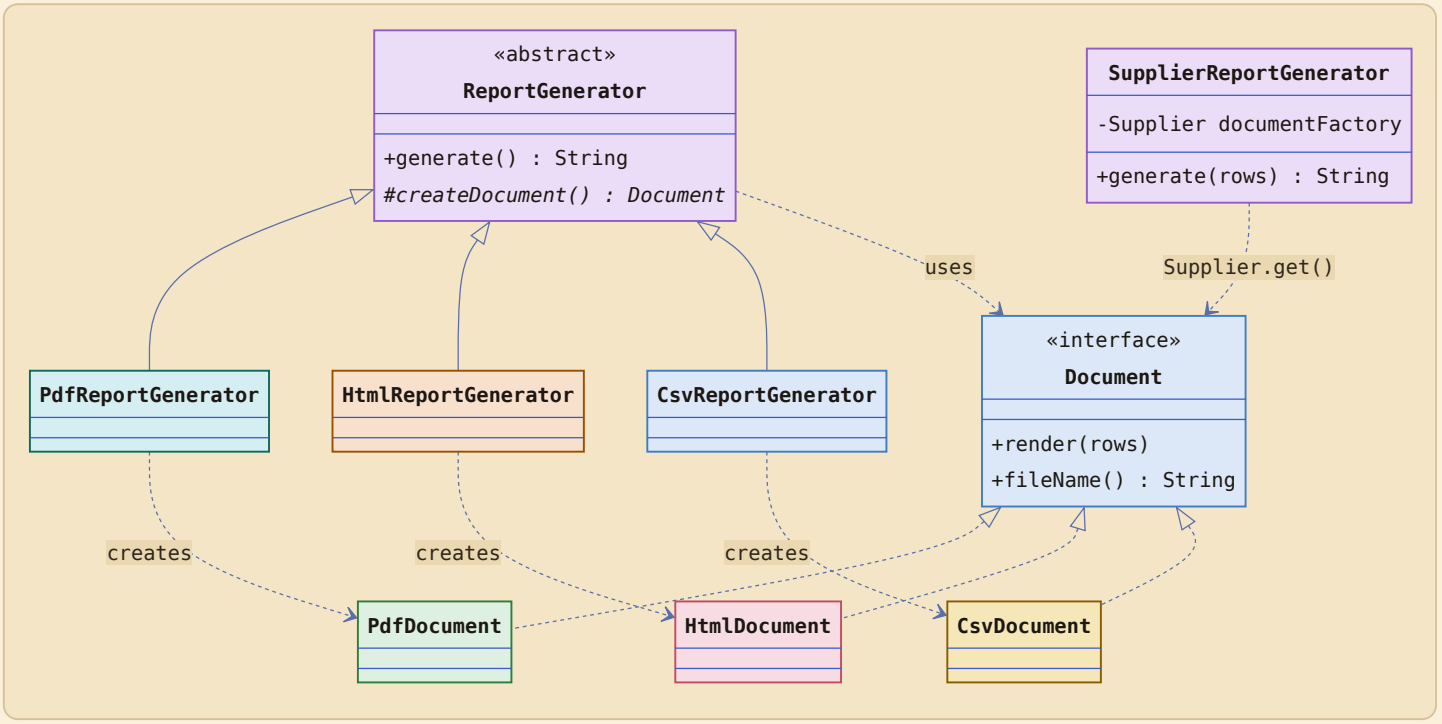


Figure — ReportGenerator declares the protected factory method createDocument(); each concrete generator overrides it to return its own Document (course diagram)



Java implementation

```java
// File: code/lld/ch08/factorymethod/Document.java
package lld.ch08.factorymethod;

import java.util.List;

/** Product: the only document type ReportGenerator knows about. */
public interface Document {
    void render(List<String> rows);
    String fileName();
}

final class PdfDocument implements Document {
    public void render(List<String> rows) { System.out.println("  Rendering as PDF: " + rows.size() + " rows"); }
    public String fileName() { return "sales.pdf"; }
}

final class HtmlDocument implements Document {
    public void render(List<String> rows) { System.out.println("  Rendering as HTML table: " + rows.size() + " rows"); }
    public String fileName() { return "sales.html"; }
}

/** Added later - no existing class had to change. */
final class CsvDocument implements Document {
    public void render(List<String> rows) { rows.forEach(r -> System.out.println("  csv> " + r)); }
    public String fileName() { return "sales.csv"; }
}
```

```java
// File: code/lld/ch08/factorymethod/ReportGenerator.java
package lld.ch08.factorymethod;

import java.util.List;

/** Creator: writes the whole procedure once and leaves one hole - the factory method. */
public abstract class ReportGenerator {

    /** The factory method. Protected: a hook for subclasses, not public API. */
    protected abstract Document createDocument();

    /** The template that USES the factory method. It never names a concrete document. */
    public final String generate() {
        System.out.println("  Collecting rows for the monthly sales report...");
        List<String> rows = List.of("north,120", "south,95", "east,140");
        Document document = createDocument();
        document.render(rows);
        System.out.println("  Report ready: " + document.fileName());
        return document.fileName();
    }
}

final class PdfReportGenerator extends ReportGenerator {
    @Override protected Document createDocument() { return new PdfDocument(); }
}

final class HtmlReportGenerator extends ReportGenerator {
    @Override protected Document createDocument() { return new HtmlDocument(); }
}

final class CsvReportGenerator extends ReportGenerator {
    @Override protected Document createDocument() { return new CsvDocument(); }
}
```

```java
// File: code/lld/ch08/factorymethod/SupplierReportGenerator.java
package lld.ch08.factorymethod;

import java.util.List;
import java.util.function.Supplier;

/** The modern, composition-based variant: pass the "factory method" in as a Supplier. */
public final class SupplierReportGenerator {
    private final Supplier<? extends Document> documentFactory;

    public SupplierReportGenerator(Supplier<? extends Document> documentFactory) {
        this.documentFactory = documentFactory;
    }

    public String generate(List<String> rows) {
        Document d = documentFactory.get();
        d.render(rows);
        return d.fileName();
    }
}
```

```java
// File: code/lld/ch08/factorymethod/Main.java
package lld.ch08.factorymethod;

import java.util.List;

public class Main {
    public static void main(String[] args) {
        // The client picks a creator once, then only talks to ReportGenerator and Document.
        for (ReportGenerator g : List.of(new PdfReportGenerator(), new HtmlReportGenerator(), new CsvReportGenerator())) {
            System.out.println("== " + g.getClass().getSimpleName() + " ==");
            g.generate();
        }

        System.out.println("\n== Supplier-based variant (composition instead of subclassing) ==");
        SupplierReportGenerator gen = new SupplierReportGenerator(HtmlDocument::new);
        System.out.println("  produced " + gen.generate(List.of("west,77")));

        // A test can plug in a fake product with a lambda - no subclass needed.
        SupplierReportGenerator testGen = new SupplierReportGenerator(() -> new Document() {
            public void render(List<String> rows) { System.out.println("  (fake) captured " + rows); }
            public String fileName() { return "fake.txt"; }
        });
        System.out.println("  produced " + testGen.generate(List.of("row-1")));
    }
}
```

- `generate()` is `final`: subclasses may change *what* is created, not the procedure. Notice it never mentions PDF, HTML or CSV.
- `CsvDocument` + `CsvReportGenerator` were "added later" — **no existing class was edited** (Open–Closed Principle).
- `SupplierReportGenerator` is the modern alternative: pass a `Supplier<Document>` (a function that creates a product) instead of subclassing. It composes better and a test can supply a lambda.

Output:

```
== PdfReportGenerator ==
  Collecting rows for the monthly sales report...
  Rendering as PDF: 3 rows
  Report ready: sales.pdf
== HtmlReportGenerator ==
  Collecting rows for the monthly sales report...
  Rendering as HTML table: 3 rows
  Report ready: sales.html
== CsvReportGenerator ==
  Collecting rows for the monthly sales report...
  csv> north,120
  csv> south,95
  csv> east,140
  Report ready: sales.csv

== Supplier-based variant (composition instead of subclassing) ==
  Rendering as HTML table: 1 rows
  produced sales.html
  (fake) captured [row-1]
  produced fake.txt
```

All three generators print an identical "Collecting rows…" line, since that code exists only once, in `ReportGenerator`; the rendering line is the only thing that changes.

Trade-offs

Pros	Cons
Extensible without edits — supporting another product means writing another class; no existing class is reopened.	**More classes** — every product type needs its own creator subclass, which is a lot of files for a small feature.
Common procedure in one place — everything around the creation step is written once in the base class.	**Indirection** — the concrete type is not visible where it is used; you have to find out which subclass you are holding.
Built-in test seam (a spot where a test can plug in its own object) — a test can subclass the creator and hand back a fake product.	**Relies on inheritance** — Java allows one superclass, and inheritance combines less flexibly than passing in a function.
Nobody holds the full list of types — no class has to know every format.	**Tempting too early** — with only one product type it is all cost and no benefit.

Often confused with:

Looks like	The actual difference
Simple Factory	One static `create` method with a `switch`. Not a GoF pattern, and frequently the better choice for small stable sets. It gathers the `switch` into one place instead of eliminating it.
Abstract Factory	Factory Method produces **one** product and varies it by **subclassing** the creator. Abstract Factory produces a **family** of products and is varied by **passing in** a different factory object (composition). Abstract Factories are often built out of factory methods.
Template Method	Same mechanism (a base-class method calls an overridable hook), different goal: Template Method lets subclasses vary a *step of an algorithm*; Factory Method lets them vary *which object gets created*. Factory Method can be seen as a special case of Template Method — `generate()` here *is* a template method.
Builder	Factory Method chooses a type in a single call; Builder assembles one complex object across many calls.
"Static factory method" (`List.of`)	A named static method used instead of a constructor, *Effective Java* style. It cannot be overridden, so it is not this pattern — only the word "factory" is shared.

In other languages (and modern Java): wherever functions are values, you can hand the creator a *creation function* instead of subclassing it. Python: `ReportGenerator(create_document=PdfDocument)` (a class is callable). JavaScript/TypeScript: pass `() => new PdfDocument()` into the constructor, and a test passes its own arrow function (JavaScript's short `() => ...` function syntax). Java: a `Supplier<Document>` field, as in `SupplierReportGenerator`. This is usually the better default because it needs no new class; keep the subclassing form for framework base classes that other teams extend — which is where the pattern was born.

When not to use it

- **There is one product type and no second one on the horizon.** Just call the constructor, and introduce the pattern on the day a second type actually appears. Adding it earlier is the most frequent over-design in the whole creational group.
- **The base class has no shared work of its own.** If `generate()` did nothing except call `createDocument()`, the abstract class would be an interface wearing a costume. The pattern pays only when real logic is written once in the creator.
- **Types differ by data, not by behaviour.** If a configuration string picks the type and the types do not behave differently, a `Map` from string to constructor reference (`Map<String, Supplier<Document>>`) is shorter than a subclass per type.
- **You need several objects that must match.** That is Abstract Factory's job.

Common mistakes

- Making the factory method **public** — it becomes part of the API (Application Programming Interface — the methods others are meant to call) and callers bypass the shared procedure.
- Calling the base class's factory method from its **constructor** — the subclass's fields are not initialized yet when the override runs.
- Calling a switch-in-one-method a Factory Method. (A learner asked exactly this about a code sample: if one method decides with `if/switch`, it is a *simple factory*; if subclasses override a creation hook used by shared base-class logic, it is a *factory method*.)

Real-world examples

- **JDK:** `Iterable.iterator()` / `Collection.iterator()` — the interface declares the creation method (Creator), and `ArrayList`, `LinkedList`, `HashSet`… (ConcreteCreators) each return their own `Iterator` implementation (ConcreteProduct); the enhanced `for` loop and every method written against `Iterator` (Product) never learn which one. `AbstractList` implements `iterator()` on top of its abstract `get(int)` — the template/factory mix in action.
- **JDK XML (as a building block):** `DocumentBuilderFactory.newDocumentBuilder()` and `TransformerFactory.newTransformer()` are abstract methods that each parser implementation's factory subclass overrides — factory methods living inside what, as a whole, is an Abstract Factory (8.11). Note that `DocumentBuilderFactory.newInstance()` itself is a *static* factory method, not a Factory Method.
- **Spring:** `FactoryBean<T>.getObject()` — a bean whose job is to create another object (e.g. proxies, `SqlSessionFactoryBean` in MyBatis-Spring). Spring's `AbstractApplicationContext` also exposes overridable creation hooks for subclasses.
- **Frameworks:** Android's `Fragment.onCreateView()` — you override a `create…` hook the framework calls. JUnit/TestNG (Java testing frameworks) fixture (shared test setup) base classes with overridable `create…` methods.

In the interview

> **30-second answer:** "Factory Method puts a creation hook in a base class. The base class writes the whole procedure once against a product interface; each subclass overrides the protected factory method to return its concrete product. New types are new classes, so it's Open–Closed. It only pays off when there's genuine shared logic in the base class — otherwise a constructor or a simple factory is simpler. In modern Java I'd often pass a `Supplier` instead of subclassing."

- *Q: Difference from Simple Factory?* — A: Simple Factory = one (static) method with a switch that knows every type. Factory Method = subclasses override an instance hook that the base class calls; nothing knows the full list. Raise this distinction yourself before you are asked — calling a `switch` a "Factory Method" is a classic slip interviewers listen for.
- *Q: Difference from Abstract Factory?* — A: One product via inheritance vs a family of matched products via a factory object passed in.
- *Q: Why protected?* — A: It is an extension point for subclasses, not something callers should invoke directly.
- *Q: How do you test `generate()`?* — A: Subclass the creator in the test (or pass a `Supplier`) and return a fake `Document` that records calls.

Self-check

- **Q:** Where is the only place a concrete product class is named? **A:** Inside each concrete creator's override of the factory method.
- **Q:** What must exist in the base class for this pattern to be worthwhile? **A:** Shared logic that uses the product (here, `generate()`).
- **Q:** Which principle does adding `CsvReportGenerator` demonstrate? **A:** Open–Closed.

TL;DR

Base class = procedure + abstract `protected createX()`; subclasses decide the concrete type. One product, varied by inheritance. Worth it only with shared logic and multiple (growing) product types; `Supplier` is the lightweight modern form.

8.11 Abstract Factory

What it is

Abstract Factory provides an interface for creating **families of related objects** that must be used together, without the client naming any concrete class. Choosing a family = choosing a factory; everything created afterwards automatically matches.

Everyday analogy. Furniture showrooms sell matching sets. If you pick the "Victorian" catalogue, the chair, sofa and table you order all come in the Victorian style. You cannot accidentally get a modern chair with a Victorian sofa, because you ordered from one catalogue. (The course summary uses the GUI (graphical user interface) version: one factory per OS (operating system) theme — Windows or macOS — producing buttons, checkboxes and scrollbars that belong together. *Head First* uses a pizza *ingredient* factory per region: New York dough, sauce and cheese vs Chicago dough, sauce and cheese.)

Terms:

- **Family** — a set of products designed to work together (Postgres connection + Postgres query builder; Postgres is short for PostgreSQL, a popular open-source database, and a query builder is an object that writes the SQL text for you).
- **Abstract product** — an interface per product kind (`Connection`, `QueryBuilder`).
- **Abstract factory** — an interface with one `create…` method per product kind (`StorageFactory`).
- **Concrete factory** — implements *every* create method for one family (`PostgresFactory`).

GoF intent and participants. Intent: offer one interface through which a client creates a whole set of related or interdependent objects, without naming their concrete classes. Participants: **AbstractFactory** (declares one creation operation per product kind — `StorageFactory`), **ConcreteFactory** (implements those operations for one family — `PostgresFactory`), **AbstractProduct** (the interface for one kind of product — `Connection`, `QueryBuilder`), **ConcreteProduct** (a product of one family, created by the matching concrete factory — `PostgresQueryBuilder`) and **Client** (uses *only* the AbstractFactory and AbstractProduct types — `ReportService`). GoF also notes that an application usually needs just one ConcreteFactory per family, which is why it is often made a Singleton (though passing it in is better).

The problem it solves

Certain objects are only correct alongside partners from the same variant. A Postgres connection needs a Postgres-flavoured query builder: Postgres wraps identifiers (table and column names) in double quotes, MySQL in backticks. A macOS-styled button should sit next to a macOS-styled checkbox. A test run wants a fake payment client, a fake mailer *and* a fake clock — never a mixture of fake and real.

When each of those objects is constructed independently, nothing prevents a bad combination. Pairing a `PostgresConnection` with a `MySqlQueryBuilder` compiles without a murmur; the mistake only surfaces later as an SQL (the standard language for querying relational databases) syntax error from the server, with a stack trace (the list of method calls that were active when the error happened) that points nowhere near the two lines that made the wrong choice.

This is also what separates it from Factory Method: Factory Method varies **one** product, Abstract Factory varies a **set** of products that must stay consistent.

Abstract Factory hands responsibility for the **entire set to one object**. Each concrete factory can only return members of its own family, so writing a mismatched pair is no longer possible through the factory. Remember

the pattern by what it guarantees: *it does not help you choose the right combination — it removes the ability to build a wrong one.*

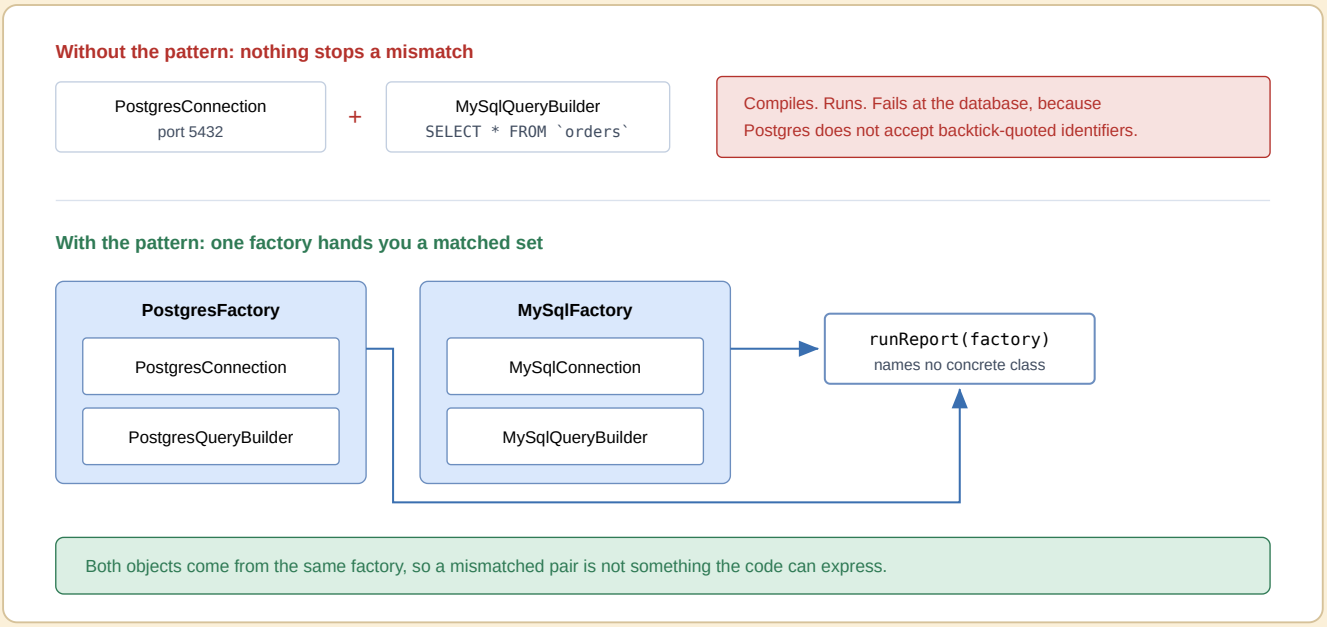


Figure — without the pattern a PostgresConnection can be paired with a MySqlQueryBuilder and fail at the database; with it, each factory hands out a matched set (course diagram)

The upper half of the figure shows the failure — the code compiles and starts, and then Postgres rejects the table name wrapped in backticks. The lower half shows the cure: `runReport(factory)` obtains both objects from one factory.

How it works

1. Identify the **product kinds** in the family and give each an interface (`Connection`, `QueryBuilder`).
2. Write the **abstract factory** interface with one creation method per product kind.
3. For each variant, write the **concrete products** and a **concrete factory** that implements *every* method and returns only that family's products (Java's covariant return types (an override may declare a more specific return type than the method it overrides) let `PostgresFactory.createConnection()` declare `PostgresConnection`, documenting the family).
4. The **client** receives a factory (composition, usually via its constructor) and only uses the abstract types.
5. Choose the concrete factory **once**, at the **composition root** — the spot close to `main` where the application wires its concrete parts together — and pass it down.

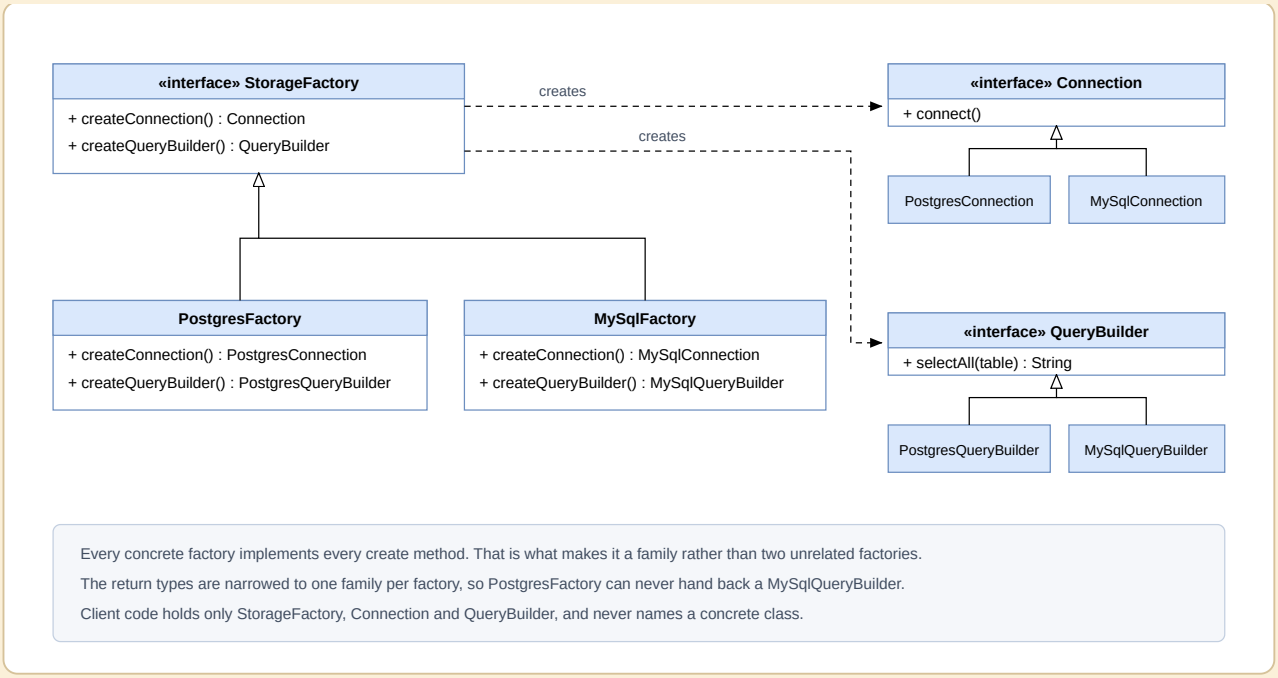
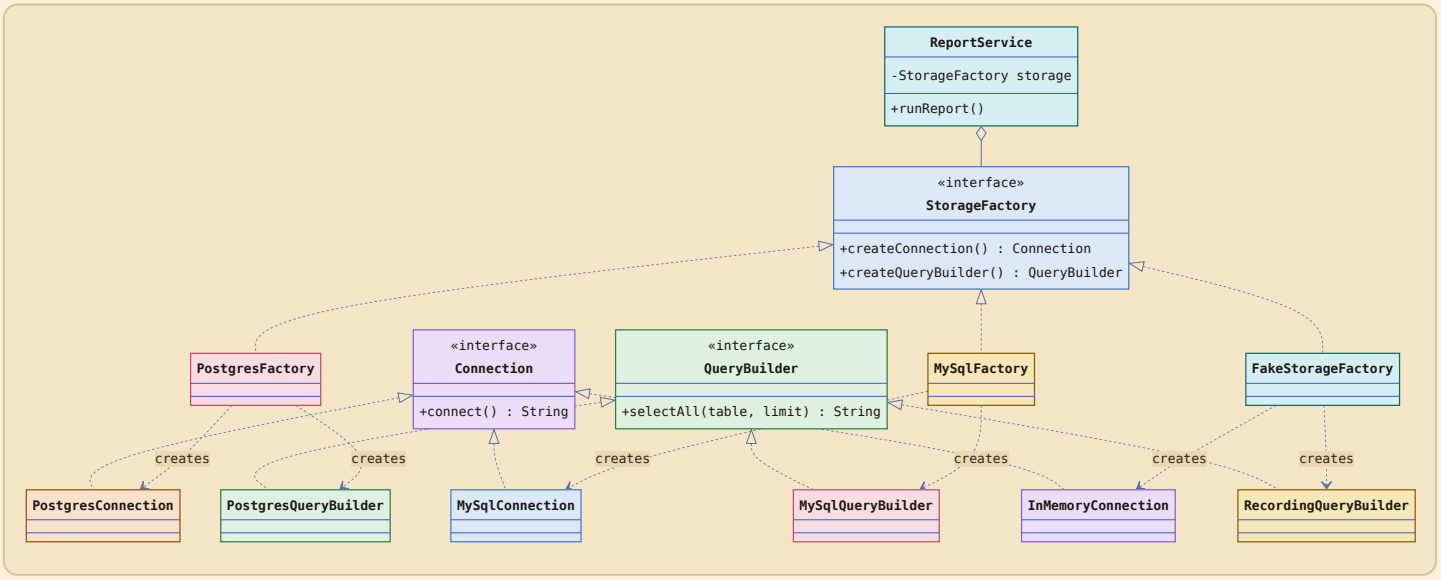
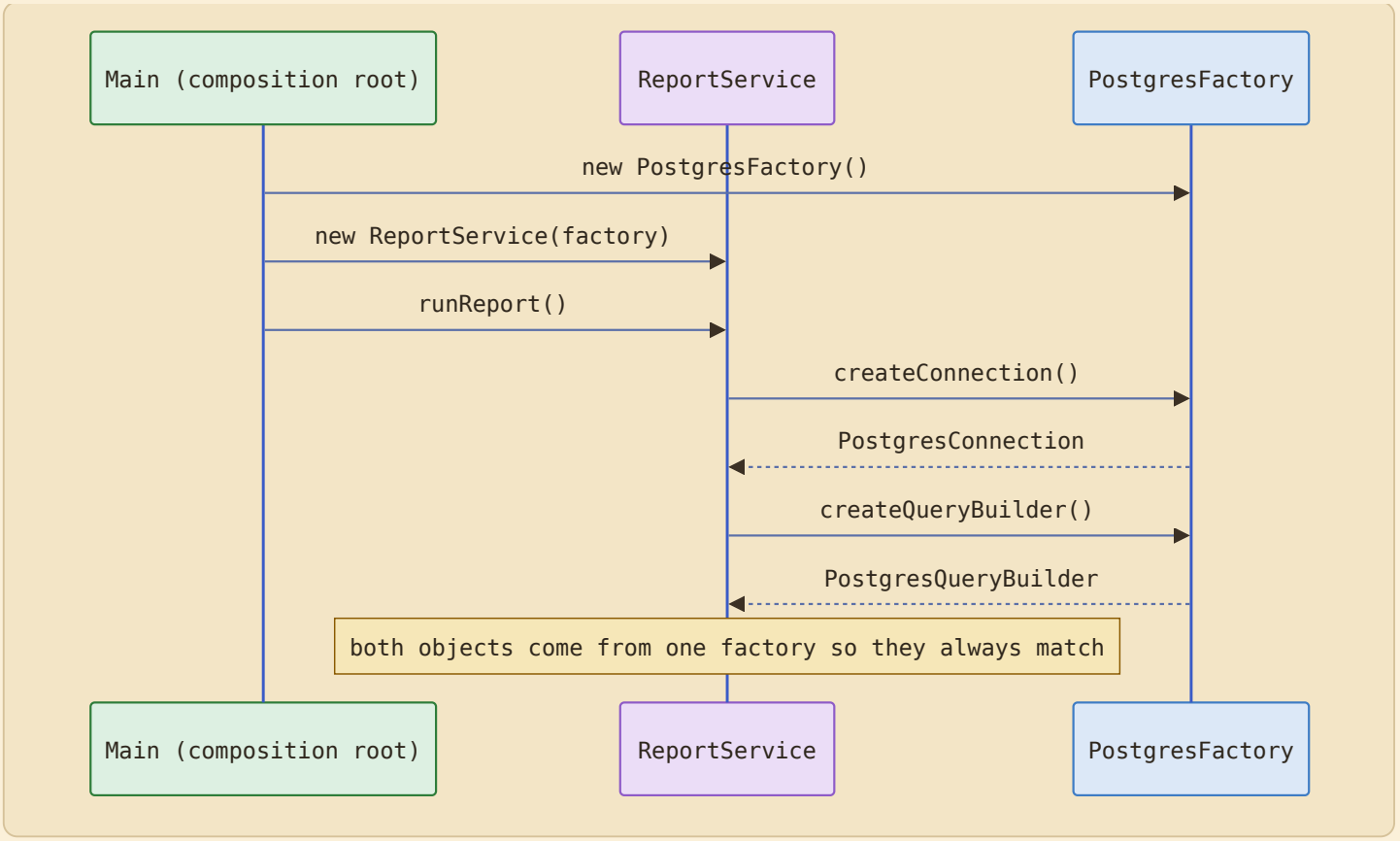


Figure — StorageFactory with createConnection() and createQueryBuilder(); PostgresFactory and MySqlFactory each create their own Connection and QueryBuilder (course diagram)

Note in the figure above that every concrete factory implements *every* create method — that is what makes it a family rather than two unrelated factories. (A learner pointed out that an older café example showed factories returning only one product each; a correct Abstract Factory must return **all** products of its family.)




Main (composition root)
ReportService
PostgresFactory
new PostgresFactory()
new ReportService(factory)
runReport()
createConnection()
PostgresConnection
createQueryBuilder()
PostgresQueryBuilder
both objects come from one factory so they always match
Main (composition root)
ReportService
PostgresFactory

Java implementation

```java
// File: code/lld/ch08/abstractfactory/Products.java
package lld.ch08.abstractfactory;

/** Abstract product 1. */
interface Connection {
    String connect();
}

/** Abstract product 2. Must agree with the Connection about the SQL dialect. */
interface QueryBuilder {
    String selectAll(String table, int limit);
}

// ---- Postgres family ----
final class PostgresConnection implements Connection {
    public String connect() { return "connected to postgres://localhost:5432/app"; }
}
final class PostgresQueryBuilder implements QueryBuilder {
    public String selectAll(String table, int limit) {
        return "SELECT * FROM \"" + table + "\" LIMIT " + limit;      // Postgres quotes with double quotes
    }
}

// ---- MySQL family ----
final class MySqlConnection implements Connection {
    public String connect() { return "connected to mysql://localhost:3306/app"; }
}
final class MySqlQueryBuilder implements QueryBuilder {
    public String selectAll(String table, int limit) {
        return "SELECT * FROM `" + table + "` LIMIT " + limit;        // MySQL quotes with backticks
    }
}

// ---- Fake family for tests ----
final class InMemoryConnection implements Connection {
    public String connect() { return "connected to in-memory store (no network)"; }
}
final class RecordingQueryBuilder implements QueryBuilder {
    public String selectAll(String table, int limit) { return "[recorded] selectAll(" + table + ", " + limit + ")"; }
}
```

```java
// File: code/lld/ch08/abstractfactory/StorageFactory.java
package lld.ch08.abstractfactory;

/** Abstract factory: one creation method per product in the family. */
public interface StorageFactory {
    Connection createConnection();
    QueryBuilder createQueryBuilder();
}

/** Concrete factories implement EVERY method and return only their own family (covariant return types). */
final class PostgresFactory implements StorageFactory {
    @Override public PostgresConnection createConnection()     { return new PostgresConnection(); }
    @Override public PostgresQueryBuilder createQueryBuilder() { return new PostgresQueryBuilder(); }
}

final class MySqlFactory implements StorageFactory {
    @Override public MySqlConnection createConnection()     { return new MySqlConnection(); }
    @Override public MySqlQueryBuilder createQueryBuilder() { return new MySqlQueryBuilder(); }
}

final class FakeStorageFactory implements StorageFactory {
    @Override public Connection createConnection()     { return new InMemoryConnection(); }
    @Override public QueryBuilder createQueryBuilder() { return new RecordingQueryBuilder(); }
}
```

```java
// File: code/lld/ch08/abstractfactory/ReportService.java
package lld.ch08.abstractfactory;

/** Client: holds only the abstract factory and abstract products. It cannot mix families. */
public final class ReportService {
    private final StorageFactory storage;

    public ReportService(StorageFactory storage) { this.storage = storage; }

    public void runReport() {
        Connection connection = storage.createConnection();
        QueryBuilder queries = storage.createQueryBuilder();
        System.out.println("  " + connection.connect());
        System.out.println("  " + queries.selectAll("orders", 10));
    }
}
```

```java
// File: code/lld/ch08/abstractfactory/Main.java
package lld.ch08.abstractfactory;

public class Main {

    /** Composition root: the ONE place that chooses the family. */
    static StorageFactory chooseFactory(String db) {
        return switch (db) {
            case "postgres" -> new PostgresFactory();
            case "mysql"    -> new MySqlFactory();
            case "fake"     -> new FakeStorageFactory();
            default -> throw new IllegalArgumentException("unknown db " + db);
        };
    }

    public static void main(String[] args) {
        String fromEnv = System.getenv().getOrDefault("DB", "postgres");
        for (String db : new String[] { fromEnv, "mysql", "fake" }) {
            System.out.println("== Using " + db + " ==");
            new ReportService(chooseFactory(db)).runReport();   // ReportService never changes
        }
    }
}
```

- `ReportService` contains **no branch** on the database — the SQL differs purely because the factory differs.
- `chooseFactory(...)` is the **composition root** and holds the program's only `switch` on the database. Had we put that `switch` inside `runReport`, it would merely have been relocated, and every other class that later needs a connection would sprout a duplicate of it.
- `FakeStorageFactory` shows the most valuable use: swap the **whole environment** for tests with one line.

Output (with no `DB` environment variable set — an environment variable is a named setting the operating system passes to a program):

```
== Using postgres ==
  connected to postgres://localhost:5432/app
  SELECT * FROM "orders" LIMIT 10
== Using mysql ==
  connected to mysql://localhost:3306/app
  SELECT * FROM `orders` LIMIT 10
== Using fake ==
  connected to in-memory store (no network)
  [recorded] selectAll(orders, 10)
```

Trade-offs

Pros	Cons
Wrong combinations cannot be written — the family is fixed once, when the factory is chosen.	**Many types** — one interface per product kind plus one class per product per family adds up fast.
One switch for a whole environment — real ↔ fake, Postgres ↔ MySQL, light ↔ dark theme.	**New product kinds are expensive** — the factory interface grows, and every concrete factory must be edited.
Client never names a concrete class — it depends only on abstractions (Dependency Inversion).	**Another layer to trace** — to know what was actually created you must find out which factory was picked.
Test seam — a single fake factory stands in for everything outside the process.	**Pointless with one family** — with a single variant it is pure ceremony.

The asymmetry to remember: adding a new **family** (e.g. `SqliteFactory`) is easy and Open–Closed; adding a new **product kind** (e.g. `createMigrationRunner()`) is hard, because every factory must change.

Abstract Factory vs Factory Method

	Factory Method	Abstract Factory
Creates	**One** product	A **family** of related products
Mechanism	**Inheritance** — subclass overrides a method	**Composition** — client holds a factory object
Who chooses the concrete type	The subclass of the creator	The concrete factory passed in
Shape	One abstract method in a class that has other logic	An interface whose *only* job is creation (several methods)
Guarantees	New types without editing the creator	Products from one factory are always compatible
Adding a new variant	New creator subclass + new product	New factory + one new product per kind
Adding a new product kind	n/a	Edit the interface and **every** factory
Relationship	—	Each `create…` method in a concrete factory is often itself a factory method
Typical example	`Collection.iterator()`	`DocumentBuilderFactory`, UI toolkits, DB dialects

Often confused with: *Builder* (Abstract Factory returns several finished products immediately; Builder assembles one product step by step and hands it over at the end); a *DI container* (same outcome through a more general mechanism — Spring profiles or Guice modules bind a whole set of implementations as configuration; if you already have a container, prefer it); a *Facade* over several factories (a Facade simplifies an existing interface, whereas Abstract Factory guarantees consistency — similar shapes, unrelated purposes).

In other languages: in Python or JavaScript, a plain dictionary/object of creation functions — `{"connection": PostgresConnection, "queries": PostgresQueryBuilder}` — already acts as a factory, no classes needed. In Java, a `record` (Java's short syntax for an immutable data class) with several `Supplier` fields (`record StorageKit(Supplier<Connection> connection, Supplier<QueryBuilder> queries)`) is often shorter than an interface plus one class per family. With a DI container, write the pattern by hand only when you are building the framework, not when using one.

When not to use it

- **The objects do not belong together.** A "factory" exposing `createLogger()` and `createInvoice()` is a grab-bag of unrelated dependencies, not a matched set — inject them separately.
- **There is only one variant** and no second is expected: an interface plus a single implementation is cost without benefit.
- **The set of product kinds keeps growing** (beyond roughly five or six) — each new kind edits every factory; a container or registry scales better.
- **Your DI container already does it** (Spring `@Profile`, Guice modules, NestJS providers) — that *is* this pattern, expressed as configuration instead of code.

Common mistakes

- A concrete factory that implements only some methods, or returns products from another family.
- Choosing the factory deep in the call stack (the chain of method calls that led to the current one) instead of once at the composition root.
- Making the factory a Singleton by reflex — pass it in instead.

Real-world examples

- **JDK:** `javax.xml.parsers.DocumentBuilderFactory`, `javax.xml.transform.TransformerFactory`, `XPathFactory` — the abstract factory class is what client code holds; the static `newInstance()` locates one implementation (e.g. the JDK's built-in Xerces/Xalan-based ones, or Saxon if configured), and that concrete factory hands out objects belonging to the same implementation (`TransformerFactory` creates both `Transformer` and `Templates`). Swapping the implementation swaps the whole set. JDBC's (Java Database Connectivity, Java's standard database API) `java.sql.Connection` acts as a factory for a driver-specific (a driver is the library that talks to one particular database) family: `createStatement()`, `prepareStatement()`, `prepareCall()`, `createBlob()` all return objects of the same driver.
- **UI toolkits:** the original GoF example — a factory per look-and-feel producing matching widgets. Swing's pluggable look-and-feel follows the same idea.
- **ORMs:** Hibernate's `Dialect` bundles database-specific SQL generation, type mapping and identifier quoting that must agree with each other. (An ORM — object-relational mapper — is a library that maps classes to database tables so you rarely write SQL by hand.)
- **Spring:** `@Profile("test")` vs `@Profile("prod")` configurations swap a whole set of beans (fake mailer, fake payments, in-memory DB) together.
- **Cloud SDKs:** switching between AWS and GCP clients in a multi-cloud abstraction (SDK = software development kit, a provider's client library; AWS and GCP are Amazon's and Google's cloud platforms) (storage client + queue client + secrets client per provider).

In the interview

> **30-second answer:** "Abstract Factory is an interface with one create method per product in a family; each concrete factory returns a matched set. The client holds only the factory and the abstract products, so it can't mix, say, a Postgres connection with a MySQL query builder. I pick the factory once at the composition root. The cost is that adding a new product kind means editing every factory."

- *Q: Factory Method vs Abstract Factory?* — A: One product via inheritance vs a family via composition; Abstract Factory often uses factory methods internally.
- *Q: How would you add SQLite?* — A: `SqliteConnection`, `SqliteQueryBuilder`, `SqliteFactory`, and one line in the composition root. Nothing else changes.
- *Q: How would you add a `MigrationRunner` product?* — A: Add `createMigrationRunner()` to the interface and implement it in every factory — this is the pattern's weak spot.
- *Q: How is this used in tests?* — A: A fake factory returns in-memory/recording products, so one line swaps the entire outside world.

Self-check

- **Q:** What property does Abstract Factory guarantee? **A:** Products from one factory are always from the same family — mismatches cannot be expressed.
- **Q:** What is the composition root? **A:** The one place near start-up where concrete choices (like which factory) are made and passed down.
- **Q:** Which change is easy and which is hard? **A:** New family = easy; new product kind = edit every factory.

TL;DR

One factory object → a matched family of products. Chosen once, passed down. Easy to add families, hard to add product kinds. Prefer a DI container's profiles if you already have one.

8.12 Builder

What it is

Builder separates the *construction* of a complex object from the object itself. A separate builder object collects the values **one named step at a time**, and a final `build()` call produces the finished — usually **immutable** — product. The GoF phrasing adds that the same construction process can create different representations.

Everyday analogy. Ordering a custom meal at a fast-food counter (the course's example) or a sandwich at Subway: bread → cheese → vegetables → sauce, each step named, many optional, and only at the end do you get the finished sandwich. Nobody hands the cashier a row of ten unlabelled values.

Terms:

- **Product** — the object being built (`HttpRequest`).
- **Builder** — collects the values and has `build()`.
- **Fluent interface** — each method returns `this` so calls can be chained: `.method("POST").timeoutSeconds(5)`.
- **Director** — optional class that knows a *recipe* (the order of steps) and runs it on a builder.
- **Immutable** — cannot change after creation (all fields `final`, no setters).

Two different "Builders" share the name — know both:

	GoF Builder (1994)	**Bloch's Builder (*Effective Java*, item 2)**
Problem it targets	One *construction process* should be able to produce *different representations* (the GoF example: one RTF reader converting a document to plain ASCII, TeX, or an editable text widget)	A constructor with many optional parameters (telescoping constructors) and the wish for an immutable, validated object
Participants	**Builder** (abstract interface with one method per part, e.g. `buildParagraph()`), **ConcreteBuilder** (implements the parts for one representation, keeps the result, exposes `getResult()`), **Director** (runs the construction steps through the Builder interface), **Product** (the complex object being built — its type can differ per ConcreteBuilder)	One product class plus one `static` nested `Builder` class: fluent setters that return `this`, and `build()` that validates and calls the product's private constructor. No Director, no abstract Builder interface
What varies	*Which* ConcreteBuilder is plugged in — the steps stay the same, the output type/format differs	Only the *values* the caller supplies
Examples	The GoF RTF converter; a report exporter whose one traversal drives a PDF builder, an HTML builder or a CSV builder	`HttpRequest.newBuilder()`, `Locale.Builder`, Lombok `@Builder`, this chapter's `HttpRequest.Builder`

Most code you will see (and write) is Bloch's fluent builder; interviews often expect you to *name* the GoF participants too. The two ideas overlap in the core insight — separate the step-by-step assembly from the finished object.

The problem it solves

A class keeps collecting fields, and most of them are optional. Its constructor grows until a call site looks like this:

```
new HttpRequest(url, "GET", headers, null, 30, 3, true, false, null);
```

A reader cannot tell what this does. Which of the `null`s is the body? What does the first `true` switch on? If someone exchanges the two booleans, the compiler accepts it, and the mistake only shows up in production — for example as requests that follow redirects although they were meant not to.

The two usual "fixes" are both bad:

Approach	What goes wrong
Telescoping constructors — one overload per useful combination: `Pizza(size)`, `Pizza(size, crust)`, `Pizza(size, crust, cheese)`, …	The number of overloads explodes with every field; call sites are still positional and unreadable; two parameters of the same type are easy to swap.
JavaBeans style — empty constructor + setters (methods that each change one field)	The object exists in a **half-built** state; nothing stops use before it is ready; it **cannot be immutable**; no single place to validate rules that span several fields.

Builder fixes both: every value is **named** at the call site, optional fields have **defaults**, the product can be **immutable**, and `build()` is **one gate** that can refuse to create an invalid object. Our API adopts the rule "a GET must not carry a body" (HTTP — the request/response protocol of the web, see System Design Fundamentals — gives a GET, its "read this resource" request, no defined meaning for a body — RFC 9110 §9.3.1, the official HTTP standard — and many servers and proxies (middlemen that forward requests) reject or drop it). That rule involves two fields at once (`method` and `body`), so no single setter can enforce it; `build()` is the natural place for it.

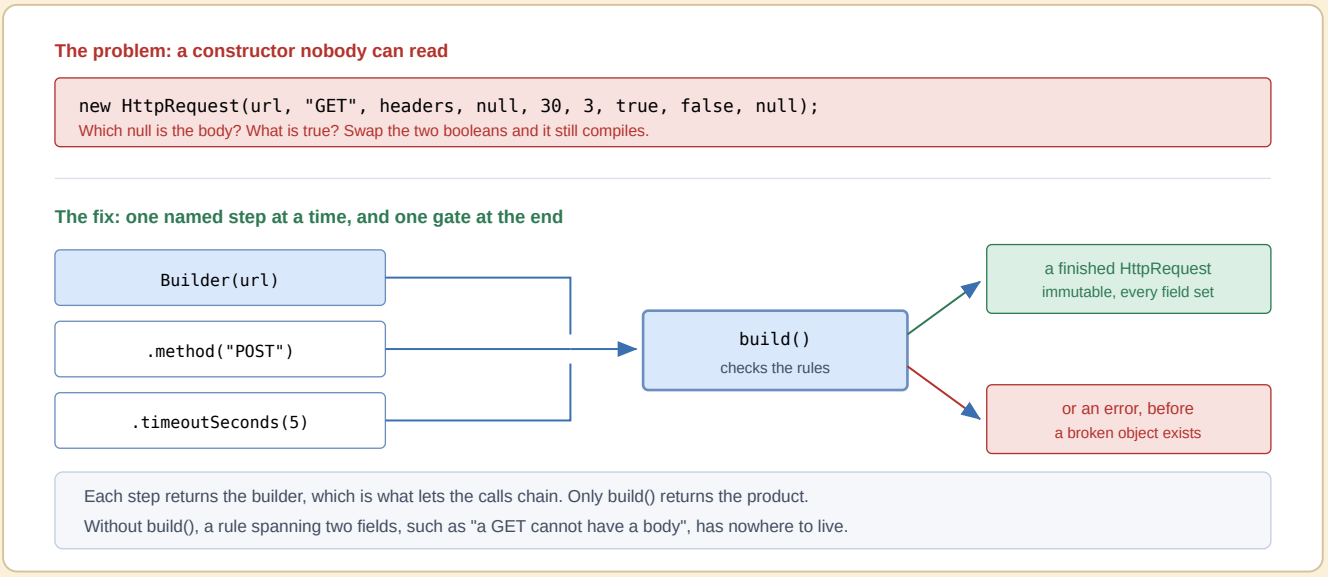


Figure — the unreadable positional constructor versus named builder steps that end in one build() gate which validates and returns an immutable HttpRequest or an error (course diagram)

In the figure, every step hands back the builder itself (which is what lets the calls chain), and only `build()` hands back a product — or throws, so an invalid request never comes into existence.

How it works

1. Make the product's fields `final` and its constructor `private`, taking the builder. Nobody can build a half-filled product directly.
2. Create a `static` **nested** `Builder` class with the same fields, mutable, each optional one with a **default**.
3. Take **required** values in the builder's constructor (or factory method `builder(url)`) so they cannot be forgotten.
4. Give every setter-like method a **name that says what it sets** and have it `return this`.
5. Write `build()`: validate cross-field rules, **copy mutable collections**, and call the private product constructor. Every other builder method returns the builder; `build()` alone returns the product.
6. Optionally add a **Director** for reusable recipes, a `toBuilder()` for copy-and-modify, or a **staged builder** so the compiler enforces required steps.

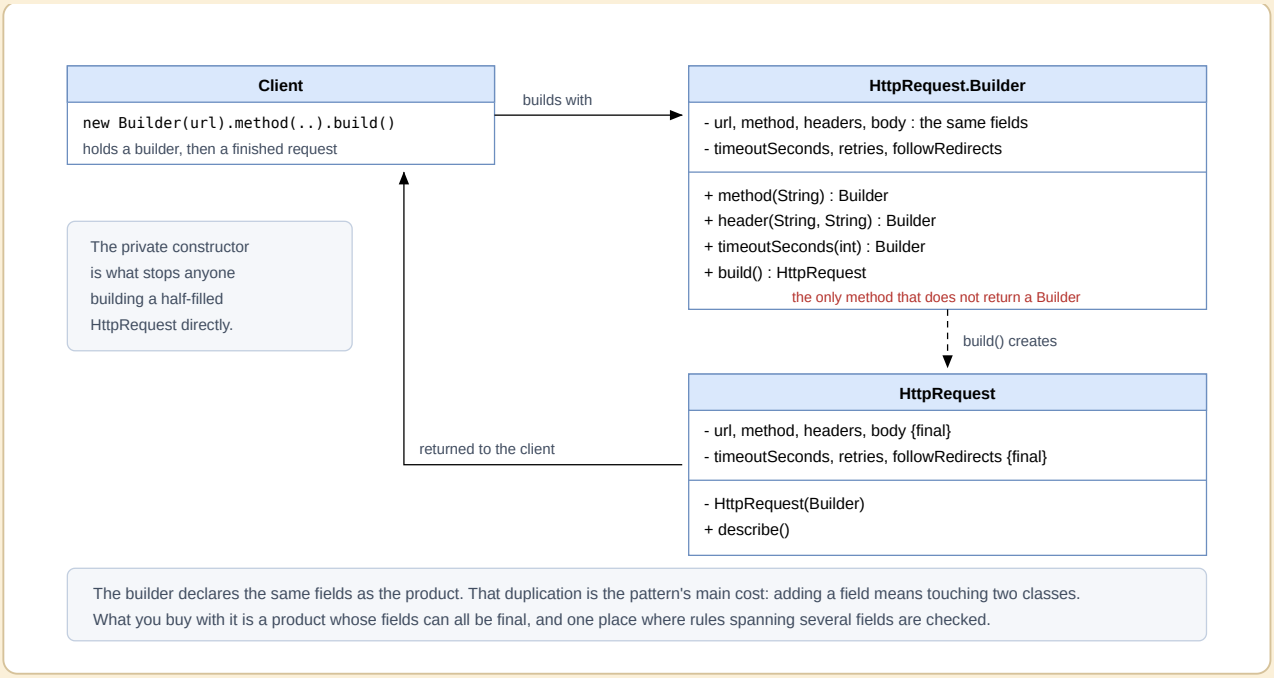
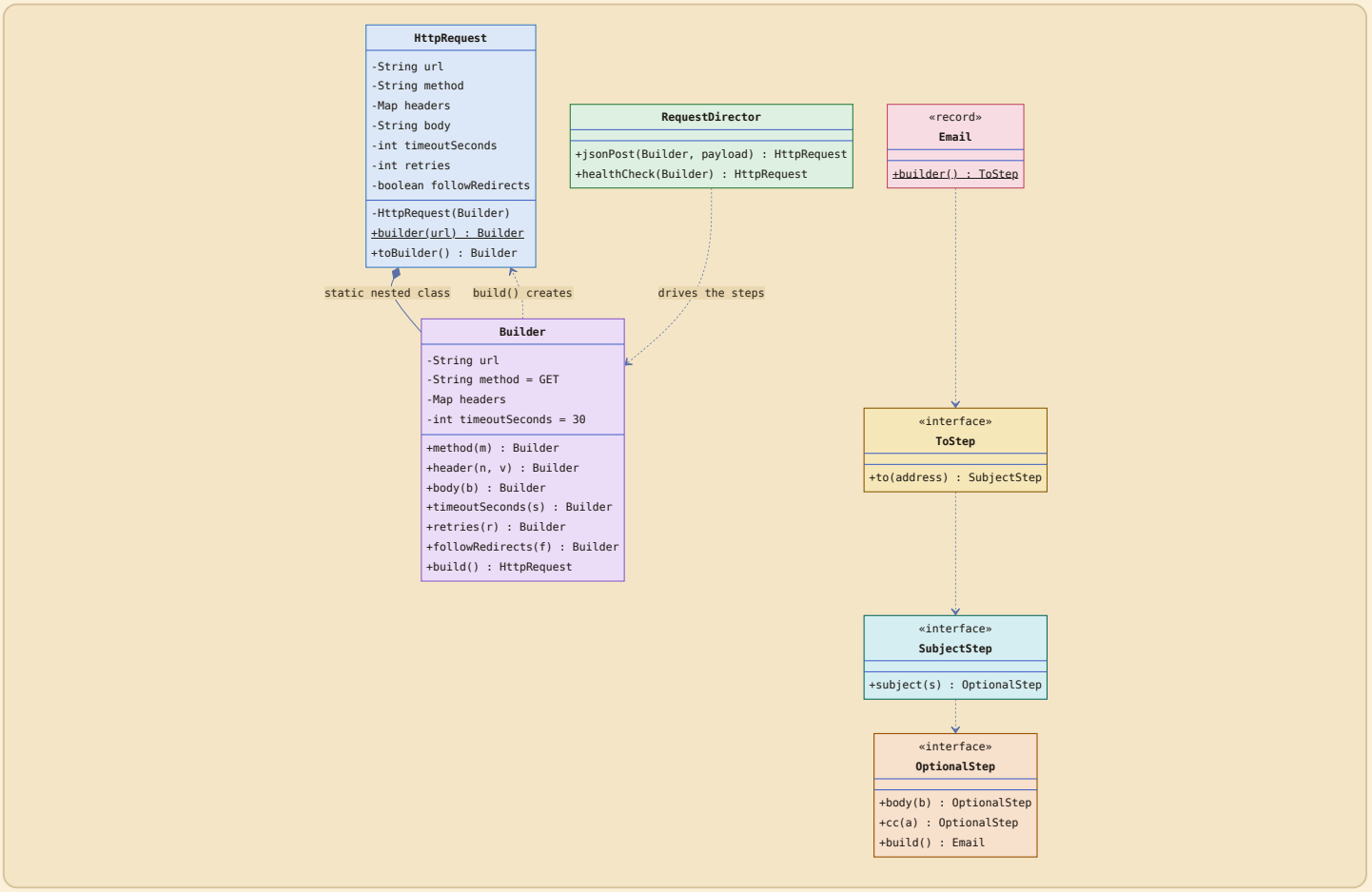


Figure — HttpRequest with final fields and a private constructor; HttpRequest.Builder with the same mutable fields, setters returning Builder, and build() returning HttpRequest (course diagram)

The diagram also exposes the price: the builder repeats every field of the product, so each new field must be added in two places. In exchange, the product has no setters, all of its fields are `final`, and every instance passes through one validation point.



The Director — and why you rarely see one

The GoF version has a fourth participant, the **Director**: a class that knows the **order of steps** and runs them on whichever builder it is given. It exists to make a *sequence* reusable — if many places build "a JSON POST with our usual headers and timeout", that recipe deserves a name, and the Director is its home. (In the GoF book's

example, one director — an RTF reader — drives different builders that output plain text or TeX: same process, different representations.)

Day to day you will mostly see the **fluent** builder, because what changes between call sites is normally the *values*, not the order of steps. If only values change, a Director adds nothing a caller could not write as one chained expression. **A quick test:** will this Director ever be given a *different* kind of builder? If so, it separates the recipe from the thing being built and is worth keeping. If not, remove it and let callers chain the calls themselves. (Our `RequestDirector` is deliberately simplified: it only accepts `HttpRequest.Builder`, so it shows the "named recipe" half of the idea. A full GoF Director would be typed against an abstract Builder interface with several implementations.)

The staged (step) builder

When some steps are required or must come in order, encode it in the **types**: each step returns an interface exposing only the legal next calls. `Email.builder()` returns a `ToStep` (only `to(...)`), which returns a `SubjectStep` (only `subject(...)`), which returns an `OptionalStep` that finally has `build()`. Now `Email.builder().build()` **does not compile**. This is the answer to "how would you stop a caller forgetting a required field?" — heavier than most code needs, but good to know.

Lombok-style builders and records

- **Lombok** `@Builder` (Lombok is a library that writes Java boilerplate for you at compile time) generates all of this from an annotation: a static `builder()` method, a nested builder class with one method per field, and `build()`. `@Builder(toBuilder = true)` adds `toBuilder()` for copy-and-modify; `@Singular` generates `header(name, value)`-style methods that add to collections. Lombok's builder does **not** validate by default — you add validation in the constructor it calls.
- **Java records** give you the immutable product for free. A record plus a static nested builder (like `Email`) is now a common shape. Put validation in the record's compact constructor (a constructor with no parameter list that runs on every creation) so it runs however the record is created.
- **Immutability checklist:** `final` class or private constructor, `final` fields, no setters, defensive copies (private copies, so outside code cannot change your internals) of mutable inputs (`Map.copyOf`, `List.copyOf`), and getters that do not leak mutable internals.

Java implementation

```java
// File: code/lld/ch08/builder/TelescopingPizza.java
package lld.ch08.builder;

/** The PROBLEM: telescoping constructors. Each new optional field adds another overload. */
public final class TelescopingPizza {
    private final String size;          // required
    private final String crust;         // optional
    private final boolean cheese;       // optional
    private final boolean olives;       // optional
    private final boolean jalapenos;    // optional

    public TelescopingPizza(String size) { this(size, "regular"); }
    public TelescopingPizza(String size, String crust) { this(size, crust, true); }
    public TelescopingPizza(String size, String crust, boolean cheese) { this(size, crust, cheese, false); }
    public TelescopingPizza(String size, String crust, boolean cheese, boolean olives) {
        this(size, crust, cheese, olives, false);
    }
    public TelescopingPizza(String size, String crust, boolean cheese, boolean olives, boolean jalapenos) {
        this.size = size; this.crust = crust; this.cheese = cheese; this.olives = olives; this.jalapenos = jalapenos;
    }

    @Override public String toString() {
        return "Pizza[size=" + size + ", crust=" + crust + ", cheese=" + cheese
                + ", olives=" + olives + ", jalapenos=" + jalapenos + "]";
    }
}
```

```java
// File: code/lld/ch08/builder/HttpRequest.java
package lld.ch08.builder;

import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Objects;

/** Product: immutable. Only the nested Builder can create one. */
public final class HttpRequest {
    private final String url;
    private final String method;
    private final Map<String, String> headers;
    private final String body;
    private final int timeoutSeconds;
    private final int retries;
    private final boolean followRedirects;

    private HttpRequest(Builder b) {
        this.url = b.url;
        this.method = b.method;
        this.headers = Map.copyOf(b.headers);      // defensive, unmodifiable copy
        this.body = b.body;
        this.timeoutSeconds = b.timeoutSeconds;
        this.retries = b.retries;
        this.followRedirects = b.followRedirects;
    }

    public static Builder builder(String url) { return new Builder(url); }

    /** Lombok-style toBuilder(): start a new builder pre-filled from this object. */
    public Builder toBuilder() {
        Builder b = new Builder(url).method(method).timeoutSeconds(timeoutSeconds)
                .retries(retries).followRedirects(followRedirects);
        headers.forEach(b::header);
        if (body != null) b.body(body);
        return b;
    }

    public String url() { return url; }
    public String method() { return method; }
    public Map<String, String> headers() { return headers; }

    @Override public String toString() {
        return method + " " + url + " headers=" + new java.util.TreeMap<>(headers)
                + (body == null ? "" : " body=" + body)
                + " timeout=" + timeoutSeconds + "s retries=" + retries + " followRedirects=" + followRedirects;
    }

    /** The builder: same fields, mutable, with defaults. Every setter returns this. */
    public static final class Builder {
        private final String url;                                  // required -> constructor
        private String method = "GET";                             // optional -> defaults
        private final Map<String, String> headers = new LinkedHashMap<>();
        private String body;
        private int timeoutSeconds = 30;
        private int retries = 3;
        private boolean followRedirects = true;

        private Builder(String url) { this.url = Objects.requireNonNull(url, "url"); }

        public Builder method(String m)             { this.method = m; return this; }
        public Builder header(String n, String v)   { headers.put(n, v); return this; }
        public Builder body(String b)               { this.body = b; return this; }
        public Builder timeoutSeconds(int s)        { this.timeoutSeconds = s; return this; }
        public Builder retries(int r)               { this.retries = r; return this; }
        public Builder followRedirects(boolean f)   { this.followRedirects = f; return this; }

        /** The single gate: cross-field validation happens here, before any product exists. */
        public HttpRequest build() {
            if (url.isBlank()) throw new IllegalStateException("url is required");
            if (method.equals("GET") && body != null) throw new IllegalStateException("a GET request cannot have a body");
            if (retries < 0) throw new IllegalStateException("retries cannot be negative");
            if (timeoutSeconds <= 0) throw new IllegalStateException("timeout must be positive");
            return new HttpRequest(this);
        }
```

```
    }
}
```

```java
// File: code/lld/ch08/builder/RequestDirector.java
package lld.ch08.builder;

/** Director: a named, reusable RECIPE (order of steps). It does not know the values the caller varies. */
public final class RequestDirector {

    public HttpRequest jsonPost(HttpRequest.Builder builder, String payload) {
        return builder.method("POST")
                .header("Content-Type", "application/json")
                .header("Accept", "application/json")
                .body(payload)
                .timeoutSeconds(5)
                .build();
    }

    public HttpRequest healthCheck(HttpRequest.Builder builder) {
        return builder.method("HEAD").timeoutSeconds(1).retries(0).followRedirects(false).build();
    }
}
```

```java
// File: code/lld/ch08/builder/Email.java
package lld.ch08.builder;

import java.util.ArrayList;
import java.util.List;

/**
 * Staged (step) builder: each step returns an interface exposing only the legal next calls,
 * so the compiler forces "to" and "subject" before build() is even visible.
 */
public record Email(String to, String subject, String body, List<String> cc) {

    public Email { cc = List.copyOf(cc); }                  // records + builder = immutable product

    public interface ToStep      { SubjectStep to(String address); }
    public interface SubjectStep { OptionalStep subject(String subject); }
    public interface OptionalStep {
        OptionalStep body(String body);
        OptionalStep cc(String address);
        Email build();
    }

    public static ToStep builder() { return new Steps(); }

    private static final class Steps implements ToStep, SubjectStep, OptionalStep {
        private String to, subject, body = "";
        private final List<String> cc = new ArrayList<>();
        public SubjectStep to(String a)       { this.to = a; return this; }
        public OptionalStep subject(String s) { this.subject = s; return this; }
        public OptionalStep body(String b)    { this.body = b; return this; }
        public OptionalStep cc(String a)      { cc.add(a); return this; }
        public Email build()                  { return new Email(to, subject, body, cc); }
    }
}
```

```java
// File: code/lld/ch08/builder/Main.java
package lld.ch08.builder;

public class Main {
    public static void main(String[] args) {
        System.out.println("== The problem: telescoping constructors ==");
        // Which boolean is which? Swap two of them and it still compiles.
        System.out.println("  " + new TelescopingPizza("large", "thin", true, false, true));

        System.out.println("\n== Fluent builder: every value is named ==");
        HttpRequest get = HttpRequest.builder("https://api.example.com/orders")
                .header("Accept", "application/json")
                .build();
        System.out.println("  " + get);

        HttpRequest post = HttpRequest.builder("https://api.example.com/orders")
                .method("POST")
                .header("Content-Type", "application/json")
                .body("{\"item\":\"widget\"}")
                .timeoutSeconds(5)
                .retries(0)
                .followRedirects(false)
                .build();
        System.out.println("  " + post);

        System.out.println("\n== build() refuses broken objects ==");
        try {
            HttpRequest.builder("https://api.example.com/orders").body("{}").build();
        } catch (IllegalStateException e) {
            System.out.println("  rejected: " + e.getMessage());
        }

        System.out.println("\n== Reusing a builder does not leak state into earlier products ==");
        HttpRequest.Builder reused = HttpRequest.builder("https://api.example.com/users").header("X-Trace", "1");
        HttpRequest first = reused.build();
        HttpRequest second = reused.header("X-Debug", "true").build();
        System.out.println("  first headers  = " + first.headers());
        System.out.println("  second headers = " + new java.util.TreeMap<>(second.headers()));

        System.out.println("\n== Product is immutable ==");
        try {
            post.headers().put("Hack", "yes");
        } catch (UnsupportedOperationException e) {
            System.out.println("  headers().put -> UnsupportedOperationException");
        }

        System.out.println("\n== toBuilder(): copy-and-change, like Lombok @Builder(toBuilder = true) ==");
        HttpRequest slowPost = post.toBuilder().timeoutSeconds(60).build();
        System.out.println("  " + slowPost);

        System.out.println("\n== Director: named recipes over any builder ==");
        RequestDirector director = new RequestDirector();
        System.out.println("  " + director.jsonPost(HttpRequest.builder("https://api.example.com/payments"), "
{\"amount\":499}"));
        System.out.println("  " + director.healthCheck(HttpRequest.builder("https://api.example.com/health")));

        System.out.println("\n== Staged builder: the compiler enforces required steps ==");
        Email email = Email.builder()
                .to("ada@example.com")          // must come first
                .subject("Welcome")             // then this
                .body("Glad to have you")       // optional, any order
                .cc("team@example.com")
                .build();
        System.out.println("  " + email);
        // Email.builder().build();   // does not compile: ToStep has no build()
    }
}
```

- `TelescopingPizza` is the anti-example: five constructors for five fields, and the call `("large", "thin", true, false, true)` hides which boolean means what.
- In `HttpRequest`, the constructor does `Map.copyOf(b.headers)`. Without that copy, building twice from one builder would hand both products **the same map**, and a header added later would appear in the earlier

product. The "reuse" demo proves `first` did not pick up `X-Debug`. `Map.copyOf` is also unmodifiable, which is why `headers().put(...)` throws.

- `build()` enforces four rules, including the cross-field one (GET + body).
- `toBuilder()` gives Lombok-style "same as this, but…" copies.

Output:

```
== The problem: telescoping constructors ==
  Pizza[size=large, crust=thin, cheese=true, olives=false, jalapenos=true]

== Fluent builder: every value is named ==
  GET https://api.example.com/orders headers={Accept=application/json} timeout=30s retries=3 followRedirects=true
  POST https://api.example.com/orders headers={Content-Type=application/json} body={"item":"widget"} timeout=5s retries=0
followRedirects=false

== build() refuses broken objects ==
  rejected: a GET request cannot have a body

== Reusing a builder does not leak state into earlier products ==
  first headers  = {X-Trace=1}
  second headers = {X-Debug=true, X-Trace=1}

== Product is immutable ==
  headers().put -> UnsupportedOperationException

== toBuilder(): copy-and-change, like Lombok @Builder(toBuilder = true) ==
  POST https://api.example.com/orders headers={Content-Type=application/json} body={"item":"widget"} timeout=60s retries=0
followRedirects=false

== Director: named recipes over any builder ==
  POST https://api.example.com/payments headers={Accept=application/json, Content-Type=application/json} body=
{"amount":499} timeout=5s retries=3 followRedirects=true
  HEAD https://api.example.com/health headers={} timeout=1s retries=0 followRedirects=false

== Staged builder: the compiler enforces required steps ==
  Email[to=ada@example.com, subject=Welcome, body=Glad to have you, cc=[team@example.com]]
```

Trade-offs

Pros	Cons
Self-describing call sites — each value is labelled where it is passed, so parameter order no longer matters.	**Duplicated fields** — the builder repeats every field of the product; a new field means two edits.
Immutable products — all fields can be `final` because they are fixed before the object exists; safe to share between threads.	**Forgettable last step** — omit `build()` and you are holding a builder, not the object you wanted.
A home for validation — rules that involve several fields are checked in `build()`.	**Easily overused** — for a value object (a small object defined only by its values, like `Money`) with three fields it only adds noise.
Free optional fields — defaults sit in the builder, callers mention only what they change.	**Reuse pitfall** — building twice from one builder can leave two products sharing one mutable collection unless it is copied.
Different representations (GoF) — a Director can drive different ConcreteBuilders through the same steps.	One extra short-lived object per construction — almost never measurable.

Often confused with:

Looks like	The actual difference
Factory Method	Chooses *which type* to create, in a single call. Builder assembles *one known type* over a series of calls.
Abstract Factory	Hands back several finished products straight away. Builder hands back one product after a sequence of steps.
Fluent interface	Chaining is merely a calling style. A chained API that modifies and returns the *same* object (a jQuery chain, a `StringBuilder`-style mutator used as the result itself) has no separate finished product, so on its own it is not the Builder pattern.
Prototype	Prototype begins with a ready-made object and duplicates it; Builder begins with nothing and fills values in. Both are ways around a huge constructor — one starts from the end result, the other from scratch.

In other languages: Python and Kotlin (and C#) have **named arguments with defaults** — `HttpRequest(url, method="POST", timeout=5)` — which already make call sites readable; Python's `@dataclass(frozen=True)` and Kotlin's `data class` (with `.copy(...)`) supply the immutable product. Use a builder there only for cross-field validation or staged construction. JavaScript/TypeScript usually pass an options object: `new HttpRequest({ url, method: "POST" })`.

When not to use it

- **A few fields, all required.** A plain constructor is shorter and just as clear — this is the misuse you will see most.
- **The language has named/default arguments** — see above.
- **The builder merely copies fields**, validates nothing, and the product is mutable anyway — that is an extra class with no payoff. Either add the validation or remove the builder.
- **The object is mutable and lives only inside one method** — ordinary setters are fine there.

Common mistakes

- Setters that **don't return the builder** (breaks chaining). Every step returns `Builder`; only `build()` returns the product.
- **No validation** in `build()`.
- **Sharing mutable collections** between the builder and the products.
- Making the product constructor public (then nobody has to use the builder or its validation).
- Using a Director when the order never varies.

Real-world examples

- **JDK:** `StringBuilder` (append repeatedly, `toString()` once); `java.net.http.HttpClient.newBuilder()` and `HttpRequest.newBuilder()` (Java 11+); `Locale.Builder`; `Calendar.Builder` (Java 8+); `Stream.builder()`; `Thread.ofVirtual().name("w").start(task)` — the `Thread.Builder` API in Java 21; `ProcessBuilder`.
- **Spring:** `UriComponentsBuilder`, `ResponseEntity.ok().header(...).body(...)`, `WebClient.builder()`, `MockMvcRequestBuilders.get("/x").param(...)`, Spring Security's `HttpSecurity` DSL (domain-specific language — a mini-language built for one job).
- **Libraries:** OkHttp `Request.Builder`; Protocol Buffers generated messages (immutable messages + generated builders); Guava `ImmutableList.builder()`; Lombok `@Builder`; test-data builders like `aUser().withEmail("x@y.com").build()`.

In the interview

30-second answer: "When a constructor has many optional parameters, I use a Builder: required values in the builder's constructor, one named method per optional field that returns the builder, and `build()` that validates cross-field rules and creates an immutable product through a private constructor. I'd mention the classic Director but usually skip it because only values vary. For tiny objects or languages with named arguments, I wouldn't bother."

- *Q: Why not setters?* — A: Half-built objects, no immutability, nowhere for cross-field validation.
- *Q: How do you enforce required fields?* — A: Take them in the builder's constructor, validate in `build()`, or use a staged builder so the compiler enforces them.
- *Q: Is `StringBuilder` a GoF Builder?* — A: Not in the full GoF structure (no Director, one representation), but it is the everyday example of the idea: accumulate parts step by step, produce the final `String` once.

- *Q: Is the built object thread-safe?* — A: The product is, if immutable. The builder itself is not — do not share a builder across threads.

Self-check

- **Q:** What is the only builder method that does not return the builder? **A:** `build()`.
- **Q:** What bug does copying the headers map in `build()`/the constructor prevent? **A:** Two products built from one builder sharing (and mutating) the same map.
- **Q:** When is a Director worth it? **A:** When the same recipe (order of steps) is reused, possibly with different builders.

TL;DR

Named steps + defaults + one validating `build()` → immutable product. Solves telescoping constructors and half-built JavaBeans. Director optional; staged builder for compile-time required steps; Lombok/records remove the boilerplate.

8.13 Prototype

What it is

Prototype creates new objects by **copying an existing, already-configured object** (the prototype) instead of building from scratch. The object itself offers a `copy()` method, so the caller does not need to know its concrete class.

Everyday analogy. A photocopier, or cell division: you do not rebuild a document or a cell from raw materials — you duplicate one that already exists and then change what differs. Another: a company's contract template. Legal drafted it once; each new contract is a copy with the name and date changed.

Terms:

- **Prototype** — the original object that gets copied.
- **Clone / copy** — the new object produced.
- **Shallow copy** — copies the fields; if a field is a reference, the copy points to the *same* object.
- **Deep copy** — copies the fields *and* the objects they point to, so nothing mutable is shared.
- **Copy constructor** — a constructor that takes another instance of the same class and copies it.
- **Prototype registry** — a map of named, pre-configured prototypes you ask for copies of.

GoF intent and participants. Intent: use a ready-configured instance as the model for the kind of object you want, and make new objects by copying that model. Participants: **Prototype** (declares the operation that copies itself — our `Shape` with `copy()`), **ConcretePrototype** (implements that copy operation — `Circle`, `Rectangle`, `Group`, or `NotificationTemplate`, which plays both roles) and **Client** (asks a prototype to copy itself instead of calling a constructor — `Main`, or `ShapeRegistry`, which GoF calls a *prototype manager*).

The problem it solves

There are two versions of the problem:

1. **Construction is expensive.** Initialization might load a file, hit a database, parse a template or fill in dozens of default values. Suppose a `NotificationTemplate` is parsed from disk at start-up; every welcome email differs from it only in the subject line and one added channel. Re-parsing the file for each recipient repeats the slowest work for nothing.
2. **You do not know the concrete class** — the more important case in practice. The code holds a reference typed as `Shape` or `Template` and needs *one more of the same*. Writing `new` is impossible, because you do not know which class name to write. Asking the object to copy itself solves it: the object always knows its own class. No factory or `switch` is needed to work the type out, and the copy starts from values that are already correct, so it skips the expensive setup.

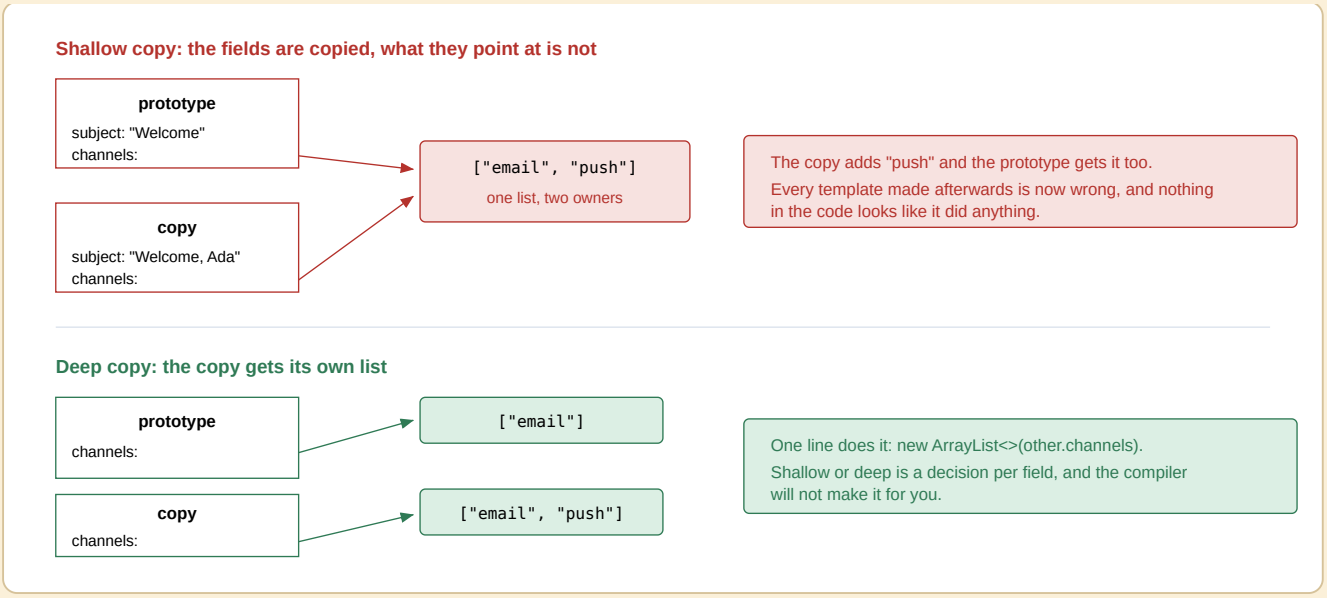


Figure — a shallow copy shares the channels list with the prototype, so adding "push" to the copy corrupts the prototype; a deep copy gets its own list (course diagram)

The figure highlights the one decision in this pattern that really needs thought: **how far down the object graph (every object reachable by following references from this one) the copy reaches.** The structure itself is simple: two entrances to the class — an ordinary constructor that does the expensive work, and a copy path that does none of it.

How it works

1. Give the class two ways in: the **normal constructor** (expensive setup) and a **copy path** that does none of it.
2. Implement the copy path, preferably as a **private copy constructor** `NotificationTemplate(NotificationTemplate other)`.
3. For each field, decide: immutable (share it — `String`, records of immutables) or mutable (copy it — `new ArrayList<>(other.channels)`), and recurse for nested mutable objects.
4. Expose a public `copy()` returning a new, independent instance.
5. If clients hold several unrelated types, add a **prototype interface** (`Shape { Shape copy(); }`) so they can copy polymorphically.
6. Optionally keep a **registry** of named prototypes that always hands out copies, never the original.

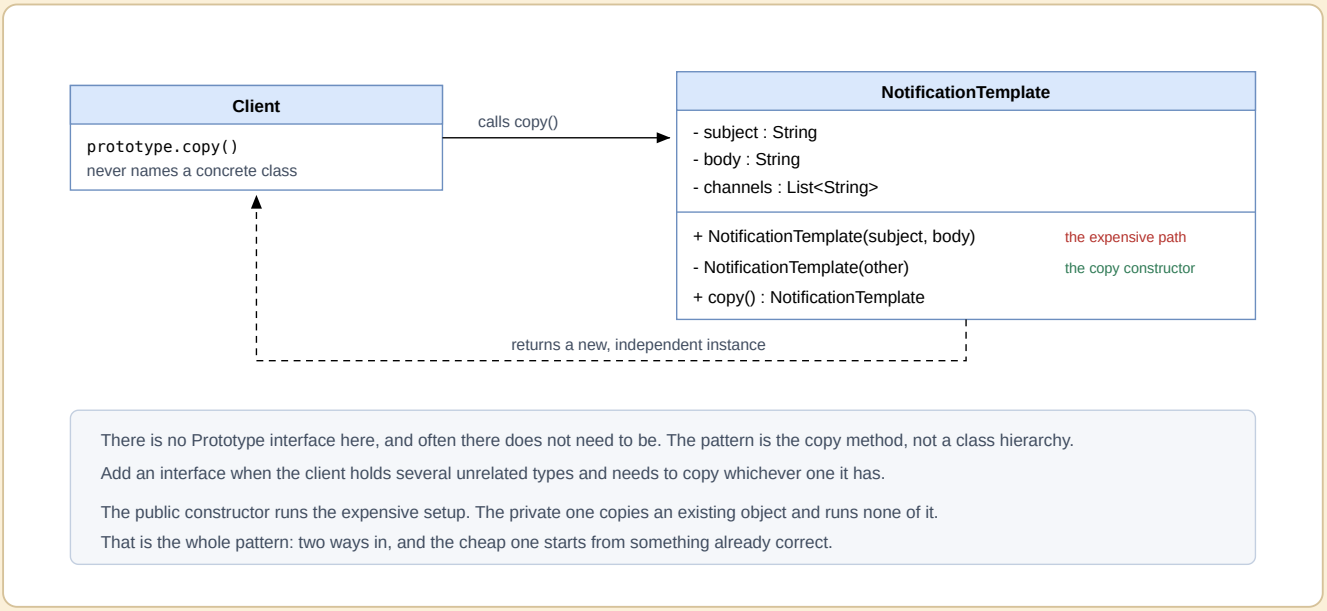
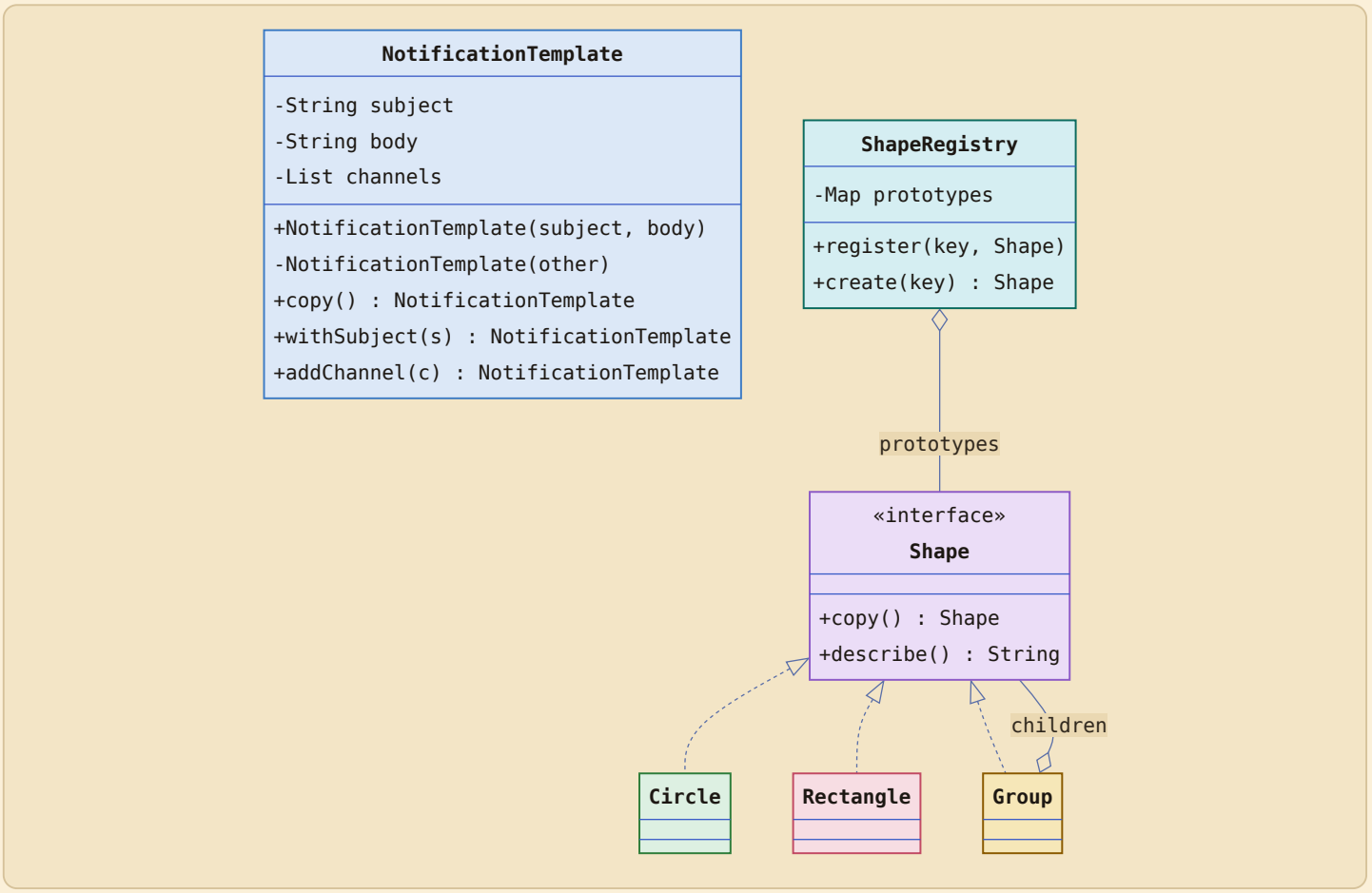


Figure — NotificationTemplate is both prototype and product: a public expensive constructor, a private copy constructor, and copy() that the Client calls (course diagram)

Notice in the class diagram that there is **no separate creator class and no Prototype interface** — unlike the other creational patterns, the product creates its own copies. What makes it Prototype is the copy method, not

a class hierarchy. Introduce a Prototype interface (like `Shape` below) only when the client holds several unrelated types and must copy whichever it happens to have.



Shallow vs deep copy

	Shallow copy	Deep copy
Primitives (`int`, `double`, `boolean`…), `String`, other immutables	Copied / safely shared	Copied / safely shared
Mutable fields (lists, maps, objects)	**Shared** — both objects point to the same one	**Duplicated** — each has its own
Cost	Cheap	Proportional to the size of the object graph
Risk	Changing the copy silently changes the original	Cycles, copying things you meant to share (caches), copying resources
Java default	`Object.clone()`	You write it (per field)

Suppose the copy constructor had written `this.channels = other.channels`. Both templates would then point at a single list, so adding "push" to the copy silently adds it to the prototype too — and **every later copy inherits the stray channel**, although no line of code visibly touched the prototype. That shared-mutable-collection bug is the one you will meet most often with this pattern. The cure is one line per mutable field — `new ArrayList<>(other.channels)` — i.e. a copy one level deeper. Had the list contained mutable objects instead of `String`s, those would need copying as well, and so on down the graph. **The depth of the copy is a design choice you make field by field**; neither Java nor the compiler will make it for you.

Three related traps:

- **A new field added without updating the copy** — the copy quietly drops it; no compiler error, and usually no test catches it, so the drift can go unnoticed for months.
- **Copying something that owns a resource** (connection, file handle) — two objects now think they own it and both try to close it.

- **Cycles** — a naive recursive deep copy of objects that point to each other never terminates. Real deep-copy utilities track visited objects.

Why Java's `Cloneable` is a trap

Java's built-in version of Prototype is `Object.clone()` + the `Cloneable` interface, and it is widely considered broken (*Effective Java*, item 13):

- `Cloneable` is a **marker interface with no methods** — it does not declare `clone()`. It only changes the behaviour of `Object.clone()` (without it, `clone()` throws `CloneNotSupportedException`).
- `Object.clone()` is `protected`, so you must override it as `public` to make it usable.
- It makes a **shallow** field-by-field copy.
- It **creates the object without running any constructor**, so constructor validation and invariants (rules that must always hold for the object) are skipped.
- To deep-copy you must reassign fields after `super.clone()` — which is **impossible for `final` fields**.
- It throws a **checked** exception (one the compiler forces you to catch or declare) you have to catch even when it cannot happen.

Prefer a copy constructor or a static copy factory (`static Foo copyOf(Foo other)`). It is ordinary code, runs your validation, works with `final` fields, and its behaviour is visible where you wrote it. The exception is arrays: `array.clone()` works well for primitive arrays.

Java implementation

```java
// File: code/lld/ch08/prototype/NotificationTemplate.java
package lld.ch08.prototype;

import java.util.ArrayList;
import java.util.List;

/** Prototype via a copy constructor: one expensive path, one cheap copy path. */
public final class NotificationTemplate {
    static int parses = 0;

    private String subject;
    private final String body;
    private final List<String> channels;

    /** The expensive path (imagine parsing a template file). */
    public NotificationTemplate(String subject, String body) {
        parses++;
        System.out.println("  Parsing template from disk... (expensive)");
        this.subject = subject;
        this.body = body;
        this.channels = new ArrayList<>(List.of("email"));
    }

    /** The copy path: no parsing. Deep-copies the mutable list. */
    private NotificationTemplate(NotificationTemplate other) {
        this.subject = other.subject;              // String is immutable -> sharing is safe
        this.body = other.body;
        this.channels = new ArrayList<>(other.channels);  // NEW list -> the copy owns its own
    }

    public NotificationTemplate copy() { return new NotificationTemplate(this); }

    public NotificationTemplate withSubject(String s) { this.subject = s; return this; }
    public NotificationTemplate addChannel(String c) { channels.add(c); return this; }

    @Override public String toString() { return "subject=\"" + subject + "\" channels=" + channels; }
}
```

```java
// File: code/lld/ch08/prototype/Shapes.java
package lld.ch08.prototype;

import java.util.ArrayList;
import java.util.List;

/** Prototype interface: lets code copy an object whose concrete class it does not know. */
interface Shape {
    Shape copy();
    String describe();
}

final class Circle implements Shape {
    private final int radius;
    private final String color;
    Circle(int radius, String color) { this.radius = radius; this.color = color; }
    private Circle(Circle other) { this(other.radius, other.color); }            // copy constructor
    @Override public Circle copy() { return new Circle(this); }                  // covariant return
    @Override public String describe() { return color + " circle r=" + radius; }
}

final class Rectangle implements Shape {
    private final int w, h;
    private final String color;
    Rectangle(int w, int h, String color) { this.w = w; this.h = h; this.color = color; }
    @Override public Rectangle copy() { return new Rectangle(w, h, color); }
    @Override public String describe() { return color + " rectangle " + w + "x" + h; }
}

/** A composite prototype: deep copy must copy the children too. */
final class Group implements Shape {
    private final List<Shape> children = new ArrayList<>();
    Group add(Shape s) { children.add(s); return this; }
    @Override public Group copy() {
        Group g = new Group();
        for (Shape s : children) g.add(s.copy());       // recursive deep copy
        return g;
    }
    @Override public String describe() { return "group" + children.stream().map(Shape::describe).toList(); }
    int size() { return children.size(); }
}
```

```java
// File: code/lld/ch08/prototype/ShapeRegistry.java
package lld.ch08.prototype;

import java.util.HashMap;
import java.util.Map;

/** Prototype registry (a.k.a. prototype manager): named, pre-configured objects handed out as copies. */
final class ShapeRegistry {
    private final Map<String, Shape> prototypes = new HashMap<>();

    void register(String key, Shape prototype) { prototypes.put(key, prototype); }

    Shape create(String key) {
        Shape p = prototypes.get(key);
        if (p == null) throw new IllegalArgumentException("no prototype named " + key);
        return p.copy();                                // never hand out the original
    }
}
```

```java
// File: code/lld/ch08/prototype/CloneablePitfall.java
package lld.ch08.prototype;

import java.util.ArrayList;
import java.util.List;

/** Why Cloneable is risky: Object.clone() is shallow, so mutable fields are shared. */
class ShallowPlaylist implements Cloneable {
    String name;
    List<String> songs = new ArrayList<>();

    @Override
    public ShallowPlaylist clone() {
        try {
            return (ShallowPlaylist) super.clone();       // field-by-field copy: 'songs' reference is shared
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e);                  // cannot happen: we implement Cloneable
        }
    }
}

/** The fixed clone(): deep-copy each mutable field by hand. (It cannot do this for a final field.) */
class DeepPlaylist implements Cloneable {
    String name;
    List<String> songs = new ArrayList<>();

    @Override
    public DeepPlaylist clone() {
        try {
            DeepPlaylist copy = (DeepPlaylist) super.clone();
            copy.songs = new ArrayList<>(songs);          // re-assigning is why 'songs' cannot be final
            return copy;
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e);
        }
    }
}
```

```java
// File: code/lld/ch08/prototype/Main.java
package lld.ch08.prototype;

public class Main {
    public static void main(String[] args) {
        System.out.println("== Copy instead of re-parsing ==");
        NotificationTemplate prototype = new NotificationTemplate("Welcome", "Hello there");
        NotificationTemplate ada = prototype.copy().withSubject("Welcome, Ada").addChannel("push");
        NotificationTemplate grace = prototype.copy().withSubject("Welcome, Grace").addChannel("sms");
        System.out.println("  ada       : " + ada);
        System.out.println("  grace     : " + grace);
        System.out.println("  prototype : " + prototype + "   <- unchanged (deep copy of the list)");
        System.out.println("  templates parsed: " + NotificationTemplate.parses);

        System.out.println("\n== Cloneable pitfall: shallow copy shares the list ==");
        ShallowPlaylist p1 = new ShallowPlaylist();
        p1.name = "Road trip"; p1.songs.add("Song A");
        ShallowPlaylist p2 = p1.clone();
        p2.songs.add("Song B");
        System.out.println("  original songs after editing the CLONE: " + p1.songs + "   <- bug");

        DeepPlaylist d1 = new DeepPlaylist();
        d1.name = "Road trip"; d1.songs.add("Song A");
        DeepPlaylist d2 = d1.clone();
        d2.songs.add("Song B");
        System.out.println("  deep clone -> original " + d1.songs + ", copy " + d2.songs);

        System.out.println("\n== Polymorphic copy: the caller only knows 'Shape' ==");
        Shape unknown = args.length > 0 ? new Rectangle(2, 3, "green") : new Circle(5, "red");
        Shape twin = unknown.copy();
        System.out.println("  copied a " + twin.getClass().getSimpleName() + ": " + twin.describe()
                + ", same object? " + (twin == unknown));

        System.out.println("\n== Prototype registry ==");
        ShapeRegistry registry = new ShapeRegistry();
        registry.register("button-bg", new Rectangle(120, 40, "blue"));
        registry.register("avatar", new Circle(24, "grey"));
        registry.register("badge", new Group().add(new Circle(8, "gold")).add(new Rectangle(30, 10, "gold")));
        Shape b1 = registry.create("badge");
        Shape b2 = registry.create("badge");
        System.out.println("  " + registry.create("button-bg").describe());
        System.out.println("  " + registry.create("avatar").describe());
        System.out.println("  " + b1.describe() + ", two badges distinct? " + (b1 != b2));
    }
}
```

- "Parsing template from disk" prints **once** although three templates exist — the copies skipped the expensive setup.
- The prototype still has only `[email]` — each copy owns its list.
- `ShallowPlaylist` shows the `Cloneable` bug: editing the clone changed the original. `DeepPlaylist` fixes it, but only because `songs` is not `final`.
- `Circle.copy()` returns `Circle` (a covariant return type) — handy when you know the type, while `Shape` callers still get a `Shape`.
- `Group.copy()` recursively copies its children — a deep copy of a composite.
- `ShapeRegistry.create()` always returns `p.copy()`, never the stored prototype, so callers cannot corrupt it.

Output:

```
== Copy instead of re-parsing ==
  Parsing template from disk... (expensive)
  ada       : subject="Welcome, Ada" channels=[email, push]
  grace     : subject="Welcome, Grace" channels=[email, sms]
  prototype : subject="Welcome" channels=[email]   <- unchanged (deep copy of the list)
  templates parsed: 1

== Cloneable pitfall: shallow copy shares the list ==
  original songs after editing the CLONE: [Song A, Song B]   <- bug
  deep clone -> original [Song A], copy [Song A, Song B]

== Polymorphic copy: the caller only knows 'Shape' ==
  copied a Circle: red circle r=5, same object? false

== Prototype registry ==
  blue rectangle 120x40
  grey circle r=24
  group[gold circle r=8, gold rectangle 30x10], two badges distinct? true
```

Trade-offs

Pros	Cons
Costly setup is paid once — the prototype does the parsing/loading; each copy is cheap.	**Every class carries copy code** — and it falls out of date as soon as someone adds a field and forgets the copy.
Create without naming a class — the only option when all you hold is an interface.	**Shallow vs deep must be decided per field** — a wrong call makes two objects secretly share state.
New kinds at run time — configured prototypes can be registered and copied without compiling a class for each (fewer subclasses).	**Cyclic graphs** need special care, or a naive deep copy recurses forever.
Saved starting points — users can keep a template, a filter or a layout and begin new work from it.	**Hazardous for objects with identity** — primary keys (unique row IDs in a database), handles and connections must not be duplicated.
	Uneven language support — some languages offer no built-in cloning, and Java's built-in one (`Cloneable`) is flawed, so you write the copy logic yourself.

Often confused with:

Looks like	The actual difference
Factory Method	Creates a fresh object from nothing, with the type chosen by a subclass. Prototype duplicates an existing instance, so the type comes along automatically.
Builder	Builder puts an object together from parts the caller supplies; Prototype takes a finished object and adjusts only what differs.
Copy constructor	The usual *implementation*. It becomes the pattern when callers use a polymorphic `copy()` without knowing the concrete type.
Object Pool	A pool lends out **existing** objects and takes them back, so identity is reused. Prototype produces **new** objects, so identity is never shared.
Spring `prototype` scope	Despite the name, Spring creates a **new instance** via the constructor on every `getBean()` — it does not clone anything.
JavaScript prototypes	JS prototypal inheritance (the prototype chain) is a language feature for sharing behaviour, not this pattern.

In other languages: Python has `copy.copy()` / `copy.deepcopy()` built in, and `dataclasses.replace(obj, subject="x")` returns a modified copy. JavaScript's spread `{...original, subject: "x"}` is a *shallow* copy (same trap as above), while `structuredClone()` deep-copies plain data. Kotlin/Scala `data class` gives `.copy(subject = "x")`. In C++ the copy constructor is part of the language, and the "rule of zero" means the compiler-generated one copies every member correctly as long as the class owns no raw resources. In Java, a `record` is a natural copy point (all components are available to a copy constructor or a `withX(...)` method), and a hand-written copy constructor is still preferred to `Cloneable`.

When not to use it

- **Creating from scratch is cheap** — a constructor is clearer and has no copy semantics to get wrong.
- **The object has identity** — duplicating something that holds a database primary key, a socket or an open file yields an object that *looks* fine but is not. Two rows sharing one ID is a far worse bug than a slow constructor.
- **The object graph is large** — a deep copy may cost more than rebuilding and may duplicate things you meant to share, such as a cache.
- **The object is immutable** — no one can change it, so just share the same instance.

Common mistakes

- Shallow-copying a mutable collection (the classic bug).
- Implementing `Cloneable` and trusting `super.clone()`.
- Forgetting to update the copy constructor when adding a field.
- Handing out the registry's original prototype instead of a copy.

Real-world examples

- **JDK:** `Object.clone()` and arrays' `clone()`; `ArrayList.clone()` and `HashMap.clone()` (documented as **shallow** — elements are not copied); copy constructors everywhere: `new ArrayList<>(collection)`, `new HashMap<>(map)`, `new TreeMap<>(sortedMap)`, `new EnumMap<>(enumMap)`; `List.copyOf(...)` for immutable snapshots.
- **Game engines:** Unity's `Instantiate(prefab)` clones a configured "prefab" object to spawn enemies, bullets and so on — load the model, stats and animations once, then copy it for each of the hundreds of enemies on a level.
- **Products:** "Duplicate" in Google Docs/Slides/Figma, "Save as template" / "Use template" in Notion or Jira, saved search filters and report layouts.
- **Tests:** a valid base fixture copied and nudged per test.

In the interview

> **30-second answer:** "Prototype creates objects by copying a configured instance through a `copy()` method on the object. It saves expensive setup and — the deeper reason — it lets me create an object when I only hold an interface. The main decision is shallow vs deep copy per field; sharing a mutable list is the classic bug. In Java I'd write a copy constructor rather than use `Cloneable`, and I'd refuse to copy anything with identity like a DB id or a connection."

- *Q: Why avoid `Cloneable`?* — A: No `clone()` in the interface, protected `Object.clone()`, shallow by default, skips constructors, cannot deep-copy `final` fields, checked exception.
- *Q: How do you deep-copy an object graph with cycles?* — A: Keep an identity map from original → copy; before copying an object, check the map and reuse the copy if it exists.
- *Q: Prototype vs Spring prototype scope?* — A: Spring's scope just means "new instance per request for the bean" — it constructs, it does not clone.
- *Q: Serialization-based deep copy?* — A: Possible (serialize then deserialize) but slow, needs everything `Serializable`, and bypasses constructors — a last resort.

Self-check

- **Q:** What is the difference between shallow and deep copy? **A:** Shallow copies references (mutable parts are shared); deep also copies what they point to.
- **Q:** Why can't `DeepPlaylist` make `songs` final? **A:** `clone()` must reassign it after `super.clone()`, which is impossible for a final field.
- **Q:** What should a prototype registry return? **A:** A copy of the stored prototype, never the original.

TL;DR

Copy a configured object instead of building one — saves setup and works when you only know the interface. Decide shallow vs deep per field. Copy constructors, not `Cloneable`. Never copy things with identity.

8.14 Bonus — Object Pool

What it is

An **Object Pool** keeps a **bounded set of reusable objects**. Instead of creating an object, using it once and throwing it away, a client **borrows** one from the pool (`acquire`), uses it, and **returns** it (`release`) so the next client can reuse it. It is not one of the GoF 23, but it is a well-known creational pattern.

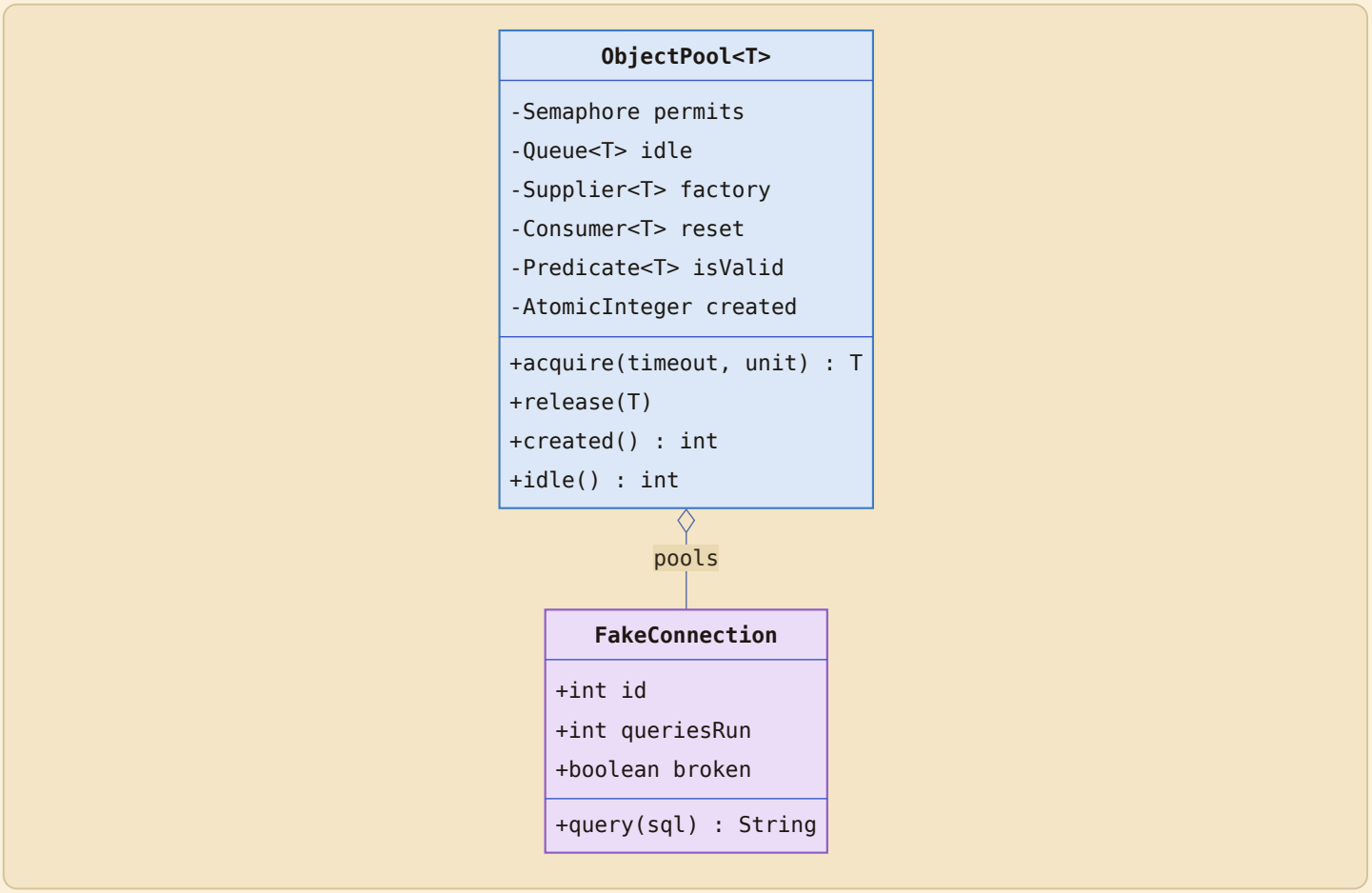
Everyday analogy. A library, or a bike-sharing dock. There are a fixed number of bikes. You take one, ride, and dock it again. If all are out, you wait (or give up after a while). Nobody builds a new bike for every ride.

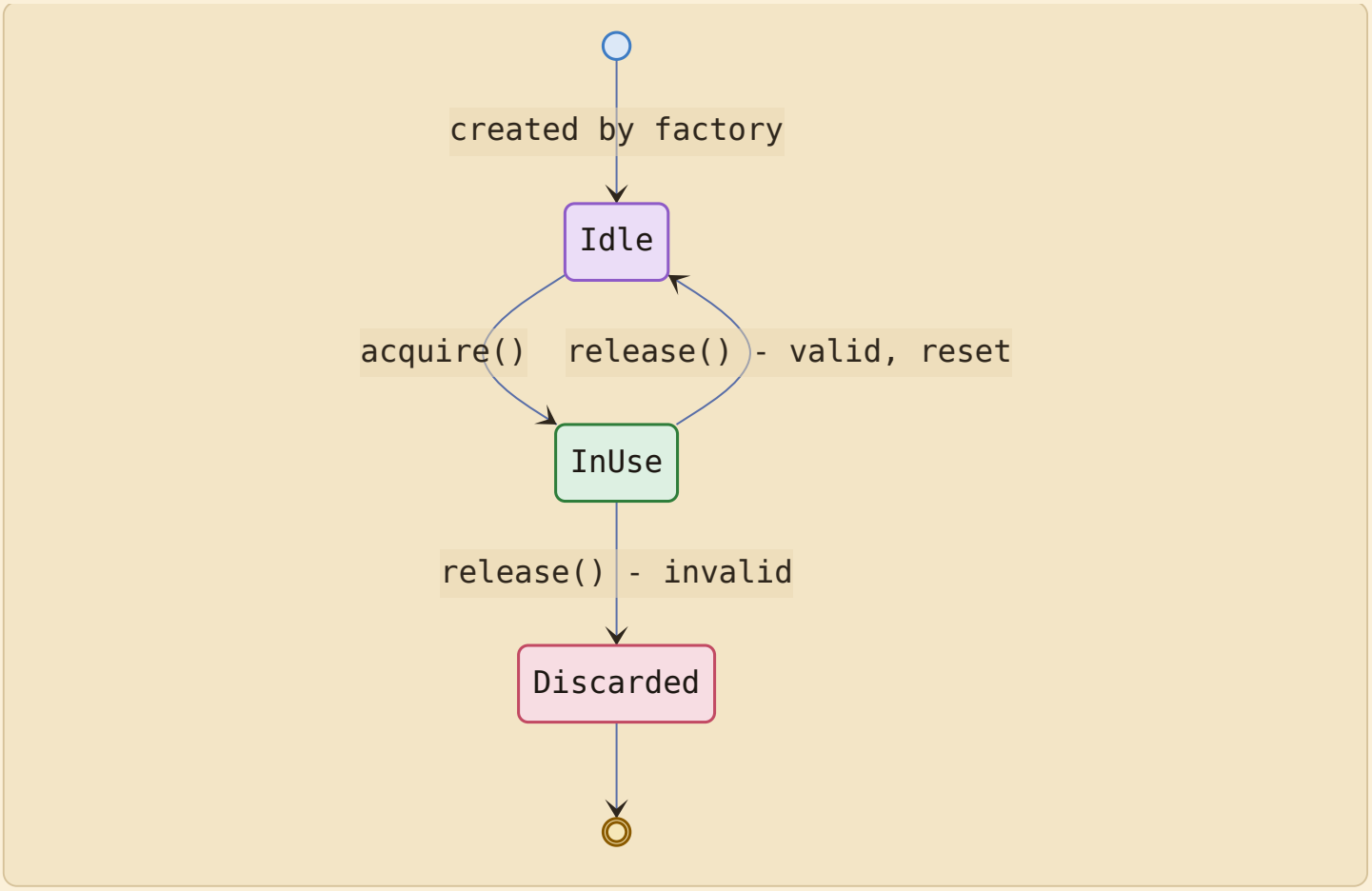
The problem it solves

Some objects are **expensive to create** (a database connection needs a TCP handshake — the back-and-forth that opens a network connection — then TLS, the encryption layer behind `https`, then authentication, proving who you are; see System Design Fundamentals) and/or **limited** (the database accepts only so many connections). Creating one per request is slow and can overwhelm the server. A pool pays the creation cost a few times and **caps** the total.

How it works

1. Keep an **idle queue** (a first-in, first-out waiting line) of ready objects, a count of how many are alive, and a cap on how many may be lent out at once (here a `Semaphore` — a counter of permits that a thread must take before going on and hand back afterwards — with one permit per object).
2. **acquire()**: wait (with a timeout) for a free slot; then take an idle object if there is one, otherwise create a new one. If no slot frees up before the timeout, fail.
3. The client uses the object and **always releases it in a `finally` block** (code that always runs, even if an exception was thrown) — a leaked (never returned) object is gone from the pool forever.
4. **release()**: **validate** the object (is the connection still alive?); if broken, discard it so a fresh one can be made later; otherwise **reset** its state (clear per-borrower data) and put it back. Either way, free the slot.
5. Make all of this **thread-safe** — pools exist *because* many threads compete.




created by factory
Idle
acquire()
release() - valid, reset
InUse
release() - invalid
Discarded

Java implementation

```java
// File: code/lld/ch08/objectpool/ObjectPool.java
package lld.ch08.objectpool;

import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.Semaphore;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.Consumer;
import java.util.function.Predicate;
import java.util.function.Supplier;

/**
 * A bounded, thread-safe object pool.
 * A Semaphore holds one permit per object that may be lent out, so at most maxSize objects
 * are ever borrowed at once. Objects are created lazily, handed out with acquire(), and
 * returned with release().
 */
public final class ObjectPool<T> {
    private final Semaphore permits;                                // one permit = one object may be borrowed
    private final ConcurrentLinkedQueue<T> idle = new ConcurrentLinkedQueue<>();
    private final Supplier<T> factory;        // how to make a new object
    private final Consumer<T> reset;          // how to clean an object before reuse
    private final Predicate<T> isValid;       // is a returned object still usable?
    private final AtomicInteger created = new AtomicInteger();    // objects currently alive (idle + borrowed)

    public ObjectPool(int maxSize, Supplier<T> factory, Consumer<T> reset, Predicate<T> isValid) {
        if (maxSize <= 0) throw new IllegalArgumentException("maxSize must be positive");
        this.permits = new Semaphore(maxSize, true);               // fair: waiters are served in order
        this.factory = factory;
        this.reset = reset;
        this.isValid = isValid;
    }

    /** Borrow an object, waiting up to the timeout if all are in use. */
    public T acquire(long timeout, TimeUnit unit) throws InterruptedException {
        if (!permits.tryAcquire(timeout, unit)) {
            throw new IllegalStateException("pool exhausted: timed out after " + timeout + " " + unit);
        }
        // Holding a permit guarantees we are within the cap: reuse an idle object or create one.
        T obj = idle.poll();
        if (obj != null) return obj;
        try {
            obj = factory.get();
            created.incrementAndGet();
            return obj;
        } catch (RuntimeException e) {
            permits.release();                                      // creation failed: give the permit back
            throw e;
        }
    }

    /** Give an object back. Broken objects are discarded so a fresh one can be created later. */
    public void release(T obj) {
        try {
            if (isValid.test(obj)) {
                reset.accept(obj);
                idle.offer(obj);
            } else {
                created.decrementAndGet();                          // discard it; its permit still comes back
            }
        } finally {
            permits.release();                                      // wakes one waiting borrower, if any
        }
    }

    public int created() { return created.get(); }
    public int idle() { return idle.size(); }
}
```

```java
// File: code/lld/ch08/objectpool/Main.java
package lld.ch08.objectpool;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {

    /** Stand-in for an expensive resource such as a DB connection. */
    static final class FakeConnection {
        private static final AtomicInteger IDS = new AtomicInteger();
        final int id = IDS.incrementAndGet();
        int queriesRun = 0;
        boolean broken = false;
        String query(String sql) { queriesRun++; return "conn#" + id + " ran: " + sql; }
    }

    public static void main(String[] args) throws Exception {
        ObjectPool<FakeConnection> pool = new ObjectPool<>(
                3,
                () -> { System.out.println("  (opening a new connection - expensive)"); return new FakeConnection(); },
                c -> c.queriesRun = 0,              // reset per-borrower state
                c -> !c.broken);                    // validate on return

        System.out.println("== 10 tasks on 6 threads share at most 3 connections ==");
        List<Integer> usedIds = new CopyOnWriteArrayList<>();
        try (ExecutorService workers = Executors.newFixedThreadPool(6)) {
            List<Future<?>> futures = new ArrayList<>();
            for (int i = 0; i < 10; i++) {
                final int task = i;
                futures.add(workers.submit(() -> {
                    FakeConnection c = pool.acquire(2, TimeUnit.SECONDS);
                    try {                                       // ALWAYS release in finally
                        c.query("SELECT " + task);
                        usedIds.add(c.id);
                        Thread.sleep(20);
                    } finally {
                        pool.release(c);
                    }
                    return null;
                }));
            }
            for (Future<?> f : futures) f.get();
        }
        System.out.println("  tasks finished: " + usedIds.size()
                + ", connections ever created: " + pool.created()
                + ", distinct connections used: " + usedIds.stream().distinct().count());

        System.out.println("\n== A broken connection is discarded, not reused ==");
        FakeConnection c = pool.acquire(1, TimeUnit.SECONDS);
        c.broken = true;
        pool.release(c);
        System.out.println("  created after discarding: " + pool.created() + ", idle: " + pool.idle());

        System.out.println("\n== Exhaustion: borrowers wait, then time out ==");
        List<FakeConnection> held = new ArrayList<>();
        for (int i = 0; i < 3; i++) held.add(pool.acquire(1, TimeUnit.SECONDS));
        try {
            pool.acquire(100, TimeUnit.MILLISECONDS);
        } catch (IllegalStateException e) {
            System.out.println("  " + e.getMessage());
        }
        held.forEach(pool::release);
        System.out.println("  after returning all: idle = " + pool.idle());
    }
}
```

- A `Semaphore` with `maxSize` permits enforces the cap: a borrower must obtain a permit first (`tryAcquire(timeout)` waits with a deadline), so no more than `maxSize` objects can ever be out at once — however many threads race. Holding a permit then means "reuse an idle object if there is one, otherwise it is safe to create one".

- `release()` returns the permit in a `finally` block, **even when the object is discarded** — so a borrower waiting for a slot wakes up and can create a replacement. (A design that only waits on the idle queue would leave that waiter stuck until its timeout although capacity is free.) If the factory throws, `acquire()` gives its permit back too, so a failed creation never shrinks the pool.
- `ConcurrentLinkedQueue` holds the idle objects; handing an object from one thread to another through a concurrent queue (and the semaphore) also makes the previous borrower's writes visible to the next one.
- Validation + reset on `release()` keep broken or dirty objects out.
- `Main` runs 10 tasks on 6 threads with a pool of 3: only 3 connections are ever opened.

Output:

```
== 10 tasks on 6 threads share at most 3 connections ==
  (opening a new connection - expensive)
  (opening a new connection - expensive)
  (opening a new connection - expensive)
  tasks finished: 10, connections ever created: 3, distinct connections used: 3

== A broken connection is discarded, not reused ==
  created after discarding: 2, idle: 2

== Exhaustion: borrowers wait, then time out ==
  (opening a new connection - expensive)
  pool exhausted: timed out after 100 MILLISECONDS
  after returning all: idle = 3
```

Trade-offs

Pros	Cons
Avoids repeated expensive creation	Complexity: sizing, validation, timeouts, leak detection
Caps resource usage (protects the DB)	Leaked (never released) objects shrink the pool until it deadlocks (everyone waits forever)
Predictable latency (time per request) after warm-up	Stale state if reset is incomplete — one user's data leaks to the next
	For cheap objects it is **slower** than `new` — modern JVM allocation and GC (garbage collection — the JVM's automatic memory clean-up) are very fast

When not to use it

- For **cheap** objects (small value objects, `StringBuilder`s) — the JVM allocates them faster than a pool can hand them out.
- For **virtual threads** (Java 21; lightweight threads managed by the JVM, cheap enough to create by the million) — they are cheap by design; create one per task instead of pooling them.
- When a **well-tested library** exists — use HikariCP, Apache Commons Pool 2 or your framework's pool rather than writing your own in production.

Common mistakes

- Not releasing in `finally` (or not using try-with-resources (Java's `try (...)` form that closes things automatically) wrappers).
- Not resetting borrower state.
- Unbounded waits — always use a timeout.
- Returning the same object twice (double release). In our teaching pool that would put one object in the idle queue twice *and* add an extra permit, silently raising the cap; production pools track which objects are currently borrowed and reject a second release.

Real-world examples

- **Database connection pools:** HikariCP (Spring Boot's default), Apache DBCP, c3p0.
- **Thread pools:** `ThreadPoolExecutor` / `Executors.newFixedThreadPool` reuse worker threads.
- **Apache Commons Pool 2** `GenericObjectPool` (used by Jedis for Redis connections).
- **Netty** `PooledByteBufAllocator` pools buffers to reduce GC pressure.

- Not pools, though often confused: the `String` intern pool and the `Integer` cache (-128..127) share immutable instances — that is **Flyweight** (Chapter 9), nothing is borrowed or returned.

In the interview

> **30-second answer:** "An object pool reuses a bounded set of expensive objects like DB connections. acquire takes an idle one, creates one if under the cap, or waits with a timeout; release validates, resets and returns it. It must be thread-safe, and clients must release in finally. For cheap objects it's counter-productive, and in production I'd use HikariCP."

- *Q: How do you size a pool?* — A: From measured concurrency and downstream limits (the capacity of the services you call, e.g. the database); HikariCP's guidance is that small pools (roughly core-count scale) often beat big ones, because the database can only do so much in parallel.
- *Q: How do you detect leaks?* — A: Record who borrowed each object and when; log if held longer than a threshold (HikariCP's `leakDetectionThreshold` does this).

Self-check

- **Q:** Object Pool vs Flyweight? **A:** Pool lends mutable objects exclusively and takes them back; Flyweight shares immutable objects among everyone at once.
- **Q:** Why validate on release? **A:** So broken objects (dead connections) are discarded instead of handed to the next borrower.

TL;DR

Borrow → use → return (in `finally`). Bounded, thread-safe, validated, reset. Worth it only for expensive or limited resources; use a mature library in production.

8.15 Creational patterns compared

At a glance

Pattern	Distinctive feature	Use when	Course example	JDK / Spring example	Pros	Cons
Singleton	Only one instance of the class exists	One shared resource: configuration, pool, registry	One `ConfigStore` (or one DB connection pool)	`Runtime.getRuntime()`, Spring singleton-scoped beans	Global access; single initialization	Hurts unit testing (testing one class on its own); hidden dependencies; global state
Simple Factory *(idiom)*	One method maps a type value to a class	Small, stable set of types chosen by config	Notification by channel	`Calendar.getInstance()`, `EnumSet.noneOf`, `@Bean` methods	Short, centralizes the switch	Edit it for every new type
Factory Method	Subclasses decide the created type via an overridable method	Shared procedure, varying product; others will add types	PDF / HTML report generator (documents of different types — PDF, Word, HTML…)	`Collection.iterator()`, Spring `FactoryBean`	Open–Closed; decouples creator from products	More classes; needs subclasses
Abstract Factory	Creates *families* of related objects without concrete classes	Products must be compatible; swap a whole set at once	Postgres vs MySQL storage; Windows vs macOS widgets	`DocumentBuilderFactory`, JDBC `Connection`, Spring profiles	Guarantees compatibility; Open–Closed for families	Complex; adding a product kind edits every factory
Builder	Separates construction of a complex object from its representation	Many optional fields, validation, immutability, step-by-step control	`HttpRequest`; building a custom meal	`StringBuilder`, `HttpClient.newBuilder()`, `UriComponentsBuilder`	Step-by-step, readable, immutable, validated	Verbose for simple objects; field duplication; Director optional
Prototype	Creates objects by copying an existing one	Setup is expensive, or only the interface is known	`NotificationTemplate`; cloning a customized car configuration	Copy constructors, `ArrayList.clone()`, Unity `Instantiate`	Skips setup; fewer subclasses; runtime-configured kinds	Deep copy is hard; `Cloneable` is broken; identity issues
Object Pool *(bonus)*	Reuses a bounded set of objects	Expensive or limited resources	DB connections	HikariCP, `ThreadPoolExecutor`	Fast reuse; caps load	Leaks, stale state, pointless for cheap objects

Pick by the question you are asking

You are asking…	Pattern
"How do I make sure there is only one?"	Singleton (or DI singleton scope)
"How do I avoid `new ConcreteClass()` scattered around?"	Simple Factory
"How do I let subclasses/plug-ins decide what to create?"	Factory Method
"How do I make sure these objects always match?"	Abstract Factory
"How do I make this 10-argument constructor readable and safe?"	Builder
"How do I make another one of *this*, cheaply or without knowing its class?"	Prototype
"How do I stop creating and destroying this expensive thing?"	Object Pool

How they combine

- An **Abstract Factory** is often implemented with **Factory Methods**, may use **Prototypes** (a factory configured with prototype objects it copies), and is frequently a **Singleton** (one factory per family) — though passing it in is better.
- A **Builder** can use a **Factory** to pick which parts to build, and a **Director** can be a Singleton-like stateless (keeps no data between calls) service.
- Designs often *start* with Factory Method (simple, subclass-based) and *evolve* towards Abstract Factory, Prototype or Builder as more flexibility is needed.

8.16 Chapter self-check

1. **Q:** Name the three GoF categories and how many patterns each has. **A:** Creational 5, Structural 7, Behavioral 11 = 23.
2. **Q:** Which two principles does the GoF book put at the centre? **A:** Program to an interface, not an implementation; favour composition over inheritance.
3. **Q:** What is the Golden Hammer? **A:** Applying a favourite tool/pattern to every problem regardless of fit.
4. **Q:** Why must the DCL field be `volatile`? **A:** To prevent another thread from seeing a reference to a not-yet-fully-constructed object (it creates a happens-before edge).
5. **Q:** Simple Factory vs Factory Method in one line? **A:** A switch in one method vs a creation hook overridden by subclasses.
6. **Q:** Factory Method vs Abstract Factory in one line? **A:** One product by inheritance vs a family of matching products by composition.
7. **Q:** Builder's single most important method, and why? **A:** `build()` — the one gate that validates cross-field rules and creates the immutable product.
8. **Q:** What is the classic Prototype bug? **A:** A shallow copy sharing a mutable collection with the original.
9. **Q:** Why is `enum` the most robust Java singleton? **A:** The JVM guarantees one instance per constant and blocks reflective creation and serialization duplicates.
10. **Q:** In an interview, how many patterns should a 45-minute design use? **A:** Usually one or two — and name the one you rejected.

8.17 Chapter TL;DR

- **Patterns** are named, proven solution templates with trade-offs — vocabulary first, code second. GoF (1994) catalogued 23 in three categories.
- **Pick from the signal, not the catalogue.** One or two patterns per interview design; always state the cost and the alternative.
- **Singleton** — one per JVM; eager / holder / enum; DCL needs `volatile`; prefer DI.
- **Simple Factory** — one `create(type)`; idiom, not GoF.
- **Factory Method** — base-class procedure + subclass-chosen product.
- **Abstract Factory** — one factory, one matching family; chosen at the composition root.
- **Builder** — named steps, one validating `build()`, immutable product.
- **Prototype** — copy a configured object; decide shallow vs deep; avoid `Cloneable`.

- **Object Pool** — borrow/return expensive objects; use a real library in production.

Chapter 9 — Structural Design Patterns

> **Where this fits.** Chapter 8 was about *creating* objects. This chapter is about *putting objects together*: how to connect, wrap, nest and share them so that larger structures stay flexible. Chapters 10–11 then cover how objects *talk to each other* (behavioral patterns).

What you will learn

#	Pattern	One-line intent	Everyday analogy
9.1	Adapter	Translate a class's interface into the one your code already uses	A travel plug adapter
9.2	Bridge	Separate two independently varying dimensions into two hierarchies joined by a reference	A TV remote (control) and the TV (device) are sold separately
9.3	Composite	Treat one object and a tree of objects the same way	A folder can hold files and other folders
9.4	Decorator	Add behaviour to one object at run time by wrapping it	Toppings on a coffee
9.5	Facade	One simple entry point in front of a complicated subsystem	A hotel concierge
9.6	Flyweight	Share the repeating part of many objects to save memory	One font glyph "e" reused for every "e" on a page
9.7	Proxy	A stand-in with the same interface that controls access	A receptionist who decides whether you may see the manager
9.8	Comparison	Adapter vs Decorator vs Proxy vs Facade vs Bridge	—

How to run every example in this chapter

```
cd chapters/09-patterns-structural/code
javac --release 21 -d /tmp/lld_09 $(find . -name '*.java')
java -cp /tmp/lld_09 lld.ch09.adapter.Main     # or bridge, composite, decorator, facade, flyweight, proxy
```

All seven `Main` classes compile with Java 21 (standard library only — the classes that ship with Java itself) and run. The output printed under each example is the real output of that program.

> **Code-style note.** Every top-level type has its own file, as in a real codebase, and the types are package-private (visible only to classes in the same package — a package being a named folder of related classes) because each example lives in one package. Where a section shows several short files in a row, read them top to bottom: interface first, then implementations, then `Main`. The whole folder compiles with `javac --release 21 -Xlint:all` with **zero warnings**.

9.0 What structural patterns are

What it is

A **design pattern** is a named, reusable solution to a design problem that keeps coming back. The "Gang of Four" (GoF) book — *Design Patterns: Elements of Reusable Object-Oriented Software* (Gamma, Helm, Johnson, Vlissides, 1994) — sorted its 23 patterns into three families:

Family	Question it answers	Examples
Creational (Ch. 8)	How do I create objects?	Singleton, Factory Method, Builder
Structural (this chapter)	**How do I combine classes and objects into bigger structures?**	**Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy**
Behavioral (Ch. 10–11)	How do objects share work and talk?	Strategy, Observer, Command

Structural patterns help you combine small objects and classes into larger, more complex structures while keeping those structures flexible and efficient. Used well, they:

- **simplify complex structures** (a Facade hides many steps; a Composite hides nesting),
- **promote reuse** (an Adapter lets you reuse a class whose interface does not fit),
- **improve flexibility by decoupling** classes from each other — i.e. reducing how much each one depends on the others' details (Bridge, Decorator, Proxy),
- **improve maintainability**, because the relationships between entities are clear and each change stays local.

There are many structural patterns, but the seven the GoF named — and that interviewers ask about — are **Adapter, Bridge, Composite, Decorator, Facade, Flyweight and Proxy**.

Three words you need first

- **Composition** ("has-a"): an object holds a reference to another object and uses it. Almost every structural pattern is built on composition.
- **Delegation**: forwarding a call to the object you hold. `wrapper.pay()` calls `inner.charge()`.
- **Interface** (in the design sense): the set of methods a caller can use on an object. In Java this is often, but not always, an `interface` type.

> **Key insight.** Four of these patterns (Adapter, Decorator, Proxy, Facade) have the *same physical shape*: one object holds another and forwards calls to it. What tells them apart is **intent** — what changes as the call passes through. Keep this in mind; section 9.8 puts them side by side.

9.1 Adapter Pattern

What it is

The **Adapter** pattern lets your code use a class whose methods have the "wrong" shape — one you are not able to change — by placing a small translator class between the two. GoF's intent: *convert the interface of a class into another interface clients expect; Adapter lets classes work together that couldn't otherwise because of incompatible interfaces.*

Analogy: your laptop charger has a US plug; the hotel wall in India has a different socket. You do not rewire the wall or cut the plug — you put a small travel adapter in between.

GoF also calls it **Wrapper**. Vocabulary:

Term	Meaning
Target	The interface your code wants to use (`PaymentGateway`).
Adaptee	The existing class with the "wrong" interface (`StripeClient`). You usually do not own it.
Adapter	The class that implements the Target and translates calls to the Adaptee (`StripeGateway`).
Client	Code that uses the Target (`checkout`).

The problem it solves

The shape your code calls and the shape the other component offers do not match. Typical cases:

- your checkout wants to call `pay(Money)`, but the payment SDK (software development kit — the vendor's ready-made client library) only offers `charge(amountInCents, currencyCode)`;
- an old service answers in XML, yet every consumer after it works with JSON (XML and JSON are two common text formats for structured data);
- the analytics library asks for `track(String, Map)`, while your code naturally produces an `Event` object.

The key constraint: **the other side is not yours to change.** It may be a third-party library (code written and owned by an outside company or project), a service owned by another team, or code with dozens of existing callers. Bending *your* code to its shape means touching every call site (each place in the code that calls the method) — and repeating the whole exercise the day that vendor is swapped out.

A concrete version: the checkout has to support Stripe now and perhaps PayPal in a few months (both are online payment providers). Stripe expects the amount as a whole number of **cents** plus a **lowercase** currency code;

PayPal expects a **decimal string** such as "42.50" plus an **uppercase** code. Your own code would rather reason about a `Money` value than either of those.

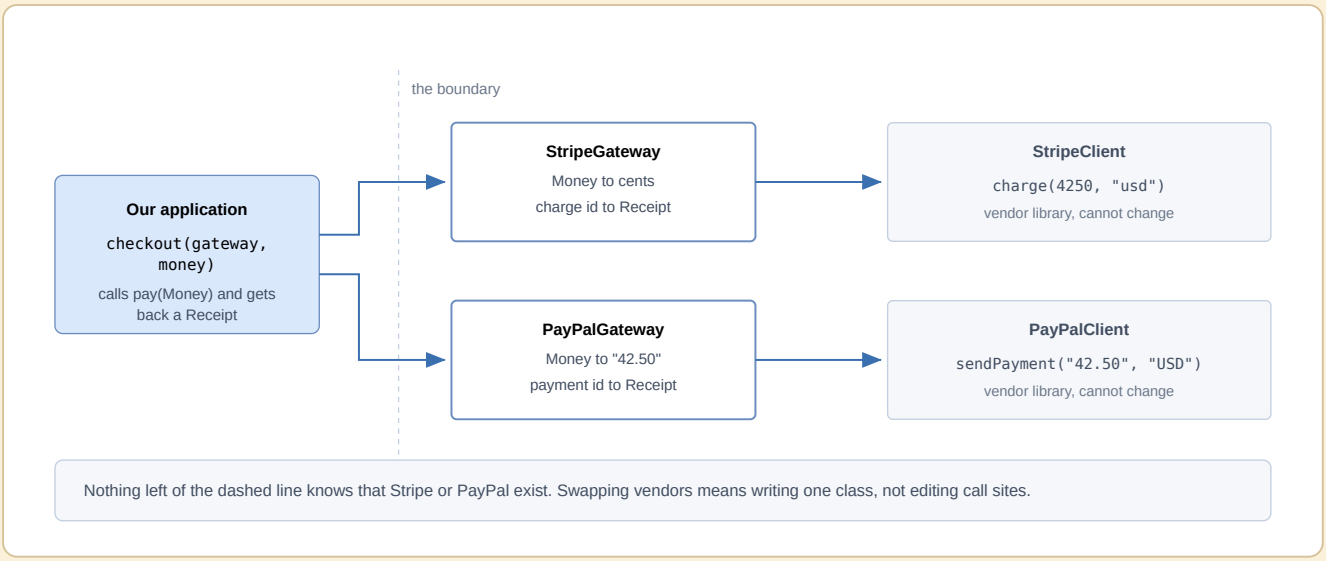


Figure — Two adapters (StripeGateway, PayPalGateway) translate our `pay(Money)` into each vendor's own call; nothing on the application side knows which vendor exists (course diagram)

In the figure above, everything to the left of the dashed boundary talks only in `Money` and `Receipt`. Swapping vendors means writing one new class, not touching call sites.

How it works

1. **Define the Target interface in your own vocabulary** (`PaymentGateway.pay(Money) -> Receipt`). It mentions no vendor.
2. **Write one adapter class per adaptee.** It `implements` the Target and **holds** the vendor object.
3. **Translate inputs**: your types into the vendor's types (`Money` → cents + lowercase code).
4. **Call the adaptee.**
5. **Translate outputs and errors back**: vendor result → `Receipt`; vendor exception → *your* exception.
6. **Client code depends only on the Target**, so it never learns which vendor answered.

> **The rule that makes it work:** the adapter's public methods mention **only your types** — parameters, return values *and* exceptions. The moment `pay` hands back a `StripeCharge`, or lets a `StripeCardException` propagate (travel up, uncaught, to the callers), callers depend on Stripe again and the adapter has decoupled nothing.

Structure

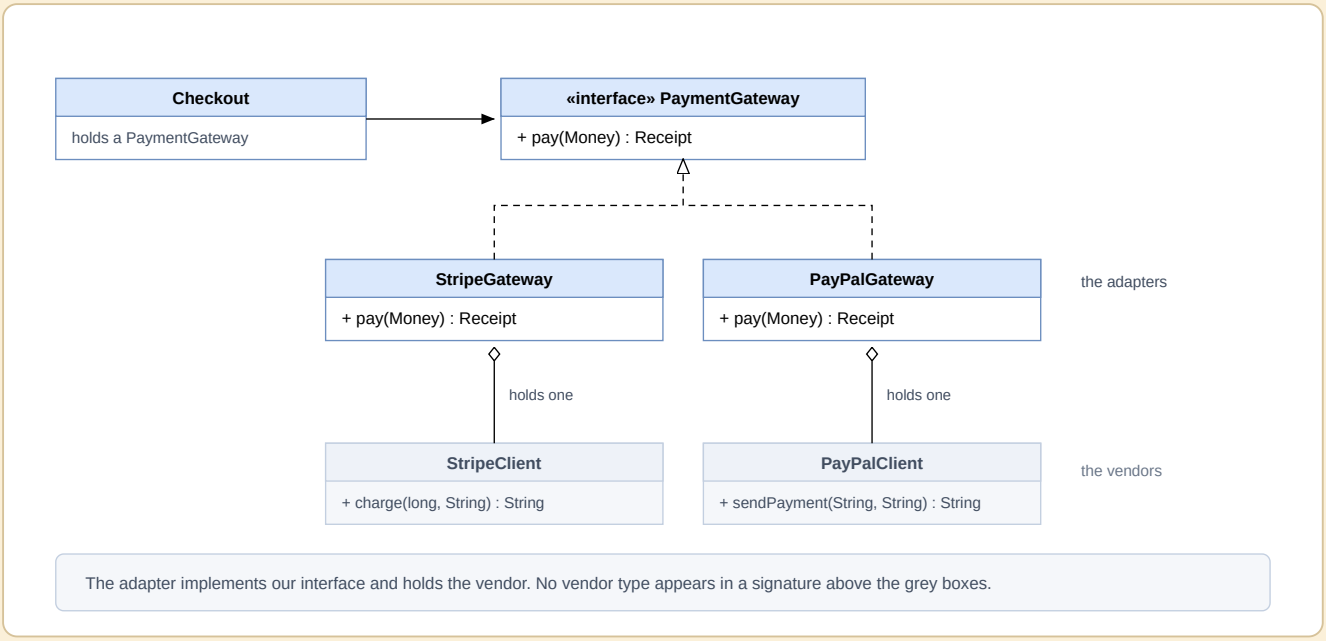
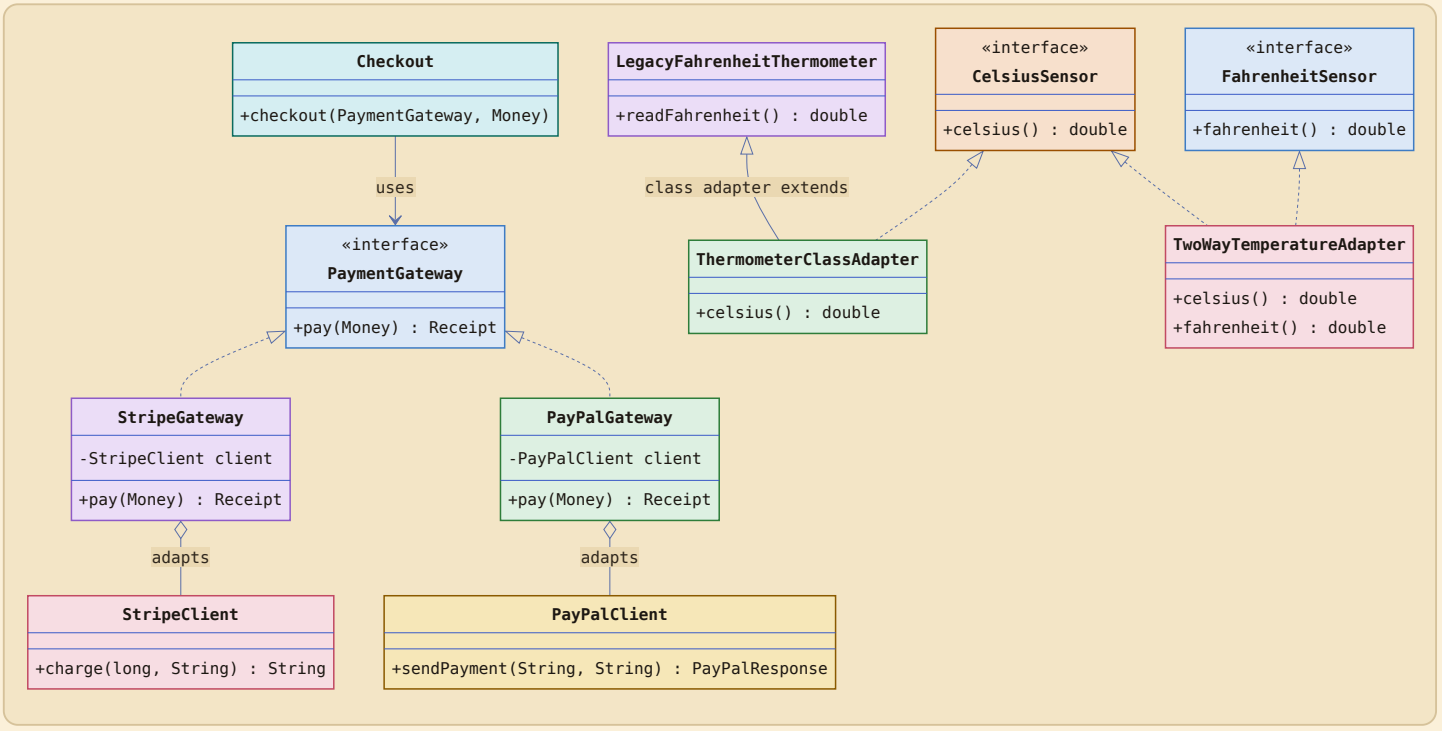


Figure — Adapter class diagram: `Checkout` holds a `PaymentGateway`; `StripeGateway` and `PayPalGateway` implement it and each wrap a vendor client, shown in grey because we do not own them (course diagram)

Reading the class diagram above: `PaymentGateway` is the Target, owned by your application; `StripeGateway` and `PayPalGateway` are the Adapters; `StripeClient` and `PayPalClient` are the Adaptees (third-party code); `Checkout` stands for any Client. Every mismatch — method names, parameter types, casing rules — is soaked up inside the two adapter classes, and nothing above them sees it.

Our Java design (including the class and two-way adapters):



Java example

```java
// File: code/lld/ch09/adapter/Money.java
package lld.ch09.adapter;

import java.util.Locale;

/** Our own vocabulary. No vendor type ever appears here. */
record Money(long cents, String currency) {
    Money {
        if (cents < 0) throw new IllegalArgumentException("negative amount");
        currency = currency.toUpperCase(Locale.ROOT);
    }

    /** 4250 cents -> "42.50" */
    String asDecimal() {
        return String.format(Locale.ROOT, "%d.%02d", cents / 100, cents % 100);
    }
}
```

```java
// File: code/lld/ch09/adapter/Receipt.java
package lld.ch09.adapter;

record Receipt(String id, String status) { }
```

```java
// File: code/lld/ch09/adapter/PaymentFailedException.java
package lld.ch09.adapter;

import java.io.Serial;

/** Our exception type: callers never see a vendor exception. */
class PaymentFailedException extends RuntimeException {
    @Serial private static final long serialVersionUID = 1L;

    PaymentFailedException(String message, Throwable cause) { super(message, cause); }
}
```

```java
// File: code/lld/ch09/adapter/PaymentGateway.java
package lld.ch09.adapter;

/** TARGET interface: the shape our application wants to talk to. */
interface PaymentGateway {
    Receipt pay(Money amount);
}
```

```java
// File: code/lld/ch09/adapter/StripeClient.java
package lld.ch09.adapter;

/*
 * ADAPTEES: pretend these come from third-party jars we cannot edit.
 * Note the different method names, argument types and conventions.
 */

/** Stripe-like SDK: integer cents + lowercase currency, throws its own exception. */
final class StripeClient {
    private int next = 1001;

    String charge(long amountInCents, String currencyCode) {
        System.out.printf("  [stripe] charge(%d, \"%s\")%n", amountInCents, currencyCode);
        if (amountInCents > 100_000) throw new StripeCardException("card_declined");
        return "ch_" + next++;
    }
}
```

```java
// File: code/lld/ch09/adapter/StripeCardException.java
package lld.ch09.adapter;

import java.io.Serial;

final class StripeCardException extends RuntimeException {
    @Serial private static final long serialVersionUID = 1L;

    StripeCardException(String code) { super(code); }
}
```

```java
// File: code/lld/ch09/adapter/PayPalClient.java
package lld.ch09.adapter;

/** PayPal-like SDK: decimal string + uppercase currency, returns a response object. */
final class PayPalClient {
    record PayPalResponse(String paymentId, String state) { }

    PayPalResponse sendPayment(String total, String currency) {
        System.out.printf("  [paypal] sendPayment(\"%s\", \"%s\")%n", total, currency);
        return new PayPalResponse("PAY-" + total.replace(".", ""), "approved");
    }
}
```

```java
// File: code/lld/ch09/adapter/StripeGateway.java
package lld.ch09.adapter;

import java.util.Locale;

/** OBJECT ADAPTER: implements our interface, HOLDS the vendor object (composition). */
final class StripeGateway implements PaymentGateway {
    private final StripeClient client;

    StripeGateway(StripeClient client) { this.client = client; }

    @Override
    public Receipt pay(Money amount) {
        try {
            String chargeId = client.charge(amount.cents(), amount.currency().toLowerCase(Locale.ROOT));
            return new Receipt(chargeId, "SUCCEEDED");
        } catch (StripeCardException e) {               // translate the vendor's error too
            throw new PaymentFailedException("Stripe declined: " + e.getMessage(), e);
        }
    }
}
```

```java
// File: code/lld/ch09/adapter/PayPalGateway.java
package lld.ch09.adapter;

final class PayPalGateway implements PaymentGateway {
    private final PayPalClient client;

    PayPalGateway(PayPalClient client) { this.client = client; }

    @Override
    public Receipt pay(Money amount) {
        PayPalClient.PayPalResponse r = client.sendPayment(amount.asDecimal(), amount.currency());
        String status = "approved".equals(r.state()) ? "SUCCEEDED" : "FAILED";
        return new Receipt(r.paymentId(), status);
    }
}
```

```java
// File: code/lld/ch09/adapter/CelsiusSensor.java
package lld.ch09.adapter;

/** Target used by our new weather dashboard. */
interface CelsiusSensor {
    double celsius();
}
```

```java
// File: code/lld/ch09/adapter/FahrenheitSensor.java
package lld.ch09.adapter;

/** Interface used by an old US reporting module. */
interface FahrenheitSensor {
    double fahrenheit();
}
```

```java
// File: code/lld/ch09/adapter/LegacyFahrenheitThermometer.java
package lld.ch09.adapter;

/** A legacy class we cannot change. It is NOT final, so a class adapter can extend it. */
class LegacyFahrenheitThermometer {
    private final double reading;

    LegacyFahrenheitThermometer(double reading) { this.reading = reading; }

    public double readFahrenheit() { return reading; }
}
```

```java
// File: code/lld/ch09/adapter/ThermometerClassAdapter.java
package lld.ch09.adapter;

/**
 * CLASS ADAPTER: inherits from the adaptee and implements the target.
 * Works in Java only because the target is an interface (no multiple class inheritance).
 */
final class ThermometerClassAdapter extends LegacyFahrenheitThermometer implements CelsiusSensor {
    ThermometerClassAdapter(double fahrenheit) { super(fahrenheit); }

    @Override
    public double celsius() { return (readFahrenheit() - 32) * 5.0 / 9.0; }
    // Downside: readFahrenheit() is still public on this object -> the adaptee's API leaks.
}
```

```java
// File: code/lld/ch09/adapter/TwoWayTemperatureAdapter.java
package lld.ch09.adapter;

/**
 * TWO-WAY ADAPTER: implements BOTH interfaces, so the same object can be handed to
 * new code (wants Celsius) and old code (wants Fahrenheit).
 */
final class TwoWayTemperatureAdapter implements CelsiusSensor, FahrenheitSensor {
    private final double celsius;

    private TwoWayTemperatureAdapter(double celsius) { this.celsius = celsius; }

    static TwoWayTemperatureAdapter from(CelsiusSensor s) {
        return new TwoWayTemperatureAdapter(s.celsius());
    }

    static TwoWayTemperatureAdapter from(FahrenheitSensor s) {
        return new TwoWayTemperatureAdapter((s.fahrenheit() - 32) * 5.0 / 9.0);
    }

    @Override public double celsius()    { return celsius; }
    @Override public double fahrenheit() { return celsius * 9.0 / 5.0 + 32; }
}
```

```java
// File: code/lld/ch09/adapter/Main.java
package lld.ch09.adapter;

import java.io.BufferedReader;
import java.io.ByteArrayInputStream;
import java.io.IOException;
import java.io.InputStreamReader;
import java.nio.charset.StandardCharsets;
import java.util.Arrays;
import java.util.List;

public class Main {

    /** Client code: talks only to OUR interface and never names a vendor. */
    static void checkout(PaymentGateway gateway, Money amount) {
        try {
            Receipt r = gateway.pay(amount);
            System.out.println("  receipt " + r.id() + " " + r.status());
        } catch (PaymentFailedException e) {
            System.out.println("  payment failed -> " + e.getMessage());
        }
    }

    public static void main(String[] args) throws IOException {
        System.out.println("== Object adapters (payment) ==");
        Money amount = new Money(4250, "usd");
        checkout(new StripeGateway(new StripeClient()), amount);
        checkout(new PayPalGateway(new PayPalClient()), amount);
        checkout(new StripeGateway(new StripeClient()), new Money(250_000, "USD"));

        System.out.println("== Class adapter ==");
        CelsiusSensor s = new ThermometerClassAdapter(98.6);
        System.out.printf("  98.6F -> %.1fC%n", s.celsius());

        System.out.println("== Two-way adapter ==");
        FahrenheitSensor oldSensor = () -> 212.0;
        TwoWayTemperatureAdapter both = TwoWayTemperatureAdapter.from(oldSensor);
        CelsiusSensor forNewCode = both;
        FahrenheitSensor forOldCode = both;
        System.out.printf("  new code sees %.1fC, old code sees %.1fF%n",
                forNewCode.celsius(), forOldCode.fahrenheit());

        System.out.println("== JDK adapters ==");
        String[] array = {"a", "b", "c"};
        List<String> view = Arrays.asList(array);   // adapts array -> List, no copy
        array[0] = "z";
        System.out.println("  Arrays.asList sees array change: " + view);
        try {
            view.add("d");
        } catch (UnsupportedOperationException e) {
            System.out.println("  cannot resize an array-backed list: UnsupportedOperationException");
        }
        var bytes = new ByteArrayInputStream("héllo".getBytes(StandardCharsets.UTF_8));
        // InputStreamReader adapts a byte stream (InputStream) to a char stream (Reader)
        try (var reader = new BufferedReader(new InputStreamReader(bytes, StandardCharsets.UTF_8))) {
            System.out.println("  InputStreamReader decoded: " + reader.readLine());
        }
    }
}
```

Output:

```
== Object adapters (payment) ==
  [stripe] charge(4250, "usd")
  receipt ch_1001 SUCCEEDED
  [paypal] sendPayment("42.50", "USD")
  receipt PAY-4250 SUCCEEDED
  [stripe] charge(250000, "usd")
  payment failed -> Stripe declined: card_declined
== Class adapter ==
  98.6F -> 37.0C
== Two-way adapter ==
  new code sees 100.0C, old code sees 212.0F
== JDK adapters ==
  Arrays.asList sees array change: [z, b, c]
  cannot resize an array-backed list: UnsupportedOperationException
  InputStreamReader decoded: héllo
```

What each part does

- `Money`, `Receipt`, `PaymentGateway`, `PaymentFailedException` — **our** vocabulary. The compact constructor (a constructor with no parameter list that runs on every creation) of the `Money` record (Java's short syntax for a data class whose fields cannot change) normalises the currency to upper case once, so every adapter can rely on it.
- `StripeClient`, `PayPalClient` — the **adaptees**. Different method names, argument types, casing and error styles. We treat them as untouchable.
- `StripeGateway` — an **object adapter**. It converts `Money` into `(cents, "usd")`, converts the returned charge id into a `Receipt`, and converts `StripeCardException` into `PaymentFailedException`. The third call in the output shows that translation: the client sees *our* error type.
- `PayPalGateway` — the same idea with a different translation (`Money.asDecimal()` → "42.50"; `"approved"` → `"SUCCEEDED"`).
- `checkout(...)` — the **client**. One function, two completely different vendors, and no `if (stripe)` branch anywhere.
- The JDK section shows two adapters you already use: `Arrays.asList` and `InputStreamReader`.

Object adapter vs class adapter

GoF describes two ways to build an adapter:

	Object adapter (composition)	**Class adapter (inheritance)**
How	Adapter *holds* an adaptee instance and implements the Target	Adapter *extends* the adaptee class and implements the Target
In our code	`StripeGateway`, `PayPalGateway`	`ThermometerClassAdapter extends LegacyFahrenheitThermometer implements CelsiusSensor`
Works when adaptee is `final`?	Yes	**No** — you cannot extend a final class (our `StripeClient` is `final`)
Can adapt a whole family (adaptee + subclasses)?	Yes — pass any subclass instance in	No — bound to one concrete class at compile time (fixed when the code is compiled)
Can override adaptee behaviour?	Harder — you would have to subclass the adaptee and adapt that subclass	Yes (it is a subclass)
Leaks the adaptee's API (its public methods)?	No — only the Target methods are visible	**Yes** — every public adaptee method (e.g. `readFahrenheit()`) is also on the adapter
Java constraint	none	Java has no multiple class inheritance (extending two classes at once), so the **Target must be an interface**

Rule of thumb: prefer the **object adapter**. It follows "favour composition over inheritance", works with final classes, can wrap several objects, and does not inherit methods you did not want. Use a class adapter only when you must override some adaptee behaviour and the adaptee is designed for extension.

Two-way adapter

A **two-way adapter** implements *both* interfaces, so the same object can be given to clients of either side. GoF mentions it for the case where two different clients need to view one object in two different ways.

In the code, `TwoWayTemperatureAdapter implements CelsiusSensor, FahrenheitSensor`. It can be built from either kind of sensor. New dashboard code sees `100.0C`; old US reporting code sees `212.0F` — the same object. This is handy during a **migration** (gradually moving code from an old system to a new one): old and new modules can share one object while you gradually move callers across.

Adapter vs Facade vs Decorator vs Proxy (quick view)

Pattern	What happens to the interface	What happens to the behaviour
Adapter	**Translated** — the client sees your interface, the wrapped object keeps its own	Same work, just re-expressed
Decorator	Kept identical on both sides	**Extended** — extra work runs around the call
Proxy	Kept identical	Same work, but **gated** — it may be postponed, served from a cache (a store of earlier results kept for quick reuse) or rejected
Facade	**Replaced** — a single simpler interface stands in for many	Same work, fewer calls for the client

In one line each: translating shapes → **Adapter**; adding work, same shape → **Decorator**; guarding access, same shape and same work → **Proxy**; one simple door in front of several objects → **Facade**.

A useful test: *adapters exist because someone else picked the interface.* Imagine the vendor rewrote its API tomorrow to match yours exactly — if your class would then have no reason to exist, it was an Adapter. The other three exist because you decided to add something. Bridge can be drawn with the same boxes, but its timing is reversed: a Bridge is planned *before* the classes exist so two hierarchies never fuse; an Adapter is bolted on *after* to reconcile two things that already exist.

Trade-offs

Pros	Cons
The foreign format is contained — only one class knows the vendor's conventions.	**An extra class and an extra call** — stack traces (the list of calls printed when an error happens) gain a frame (one entry in that list) whose only job is conversion.
Changing vendor is local — supporting PayPal beside Stripe means adding a class, not editing callers (Open/Closed Principle — open for extension, closed for modification).	**Conversions can drop information**, and callers cannot see that anything was lost.
Easy to test — faking your own one-method interface is far simpler than faking a vendor SDK.	**Two data models must stay aligned** — each vendor API change forces a mapping change.
Your code keeps its own language (`Money`, `Receipt` rather than cents and charge ids); conversion has a single home (Single Responsibility — one reason to change per class).	**Logic tends to creep in** — retries (automatically re-trying a failed call) and pricing rules drift into the adapter unless reviews stop them.

When not to use it

- **Both sides are yours.** Change one interface so they match. An adapter between two pieces of code you own is overhead you pay forever for a mismatch you could simply remove.
- **The two models really disagree.** When mapping one to the other drops data or forces guesses, the "adapter" grows into a mini-application with bugs of its own; redraw the boundary instead.
- **You want extra behaviour, not a translation.** That job belongs to a Decorator.
- **The conversion happens in exactly one place.** A few inline lines read better than a new class.

Common mistakes

- **Leaking vendor types** (`StripeCharge` return type, `StripeCardException` escaping). Then nothing is decoupled.
- **Putting business logic in the adapter** (discount rules, retries). Adapters translate; they do not decide.
- **Choosing a class adapter by default** — it exposes the adaptee's whole API and breaks on `final` classes.
- **One giant adapter for many vendors** with `if (vendor == STRIPE)` inside — that is the branch the pattern was meant to remove. One adapter per adaptee.
- **Silently dropping information** (e.g. vendor's "pending" state mapped to "SUCCEEDED"). Map every case deliberately.

Real-world examples

- **`java.util.Arrays.asList(T...)`** adapts an array to the `List` interface *without copying* — writes go through to the array (as the output shows) and the list cannot grow, so `add` throws `UnsupportedOperationException`.
- **`java.io.InputStreamReader` / `OutputStreamWriter`** adapt byte streams (raw bytes read one after another) to character streams (text) using a charset (the rule that maps bytes to characters, e.g. UTF-8).
- **`java.util.Collections.enumeration(Collection)`** adapts a modern `Collection` to the legacy (old but still supported) `Enumeration` interface. (*Head First Design Patterns* builds the reverse, an Enumeration-to-Iterator adapter.)
- **SLF4J providers/bindings** — SLF4J (Simple Logging Facade for Java) is one logging API, with an adapter module for each backend (the library that actually writes the log lines) (Logback, Log4j, `java.util.logging`).
- **JDBC (and ODBC) drivers** (JDBC is Java's standard database API; a driver is the library for one particular database) adapt each database's own wire protocol (the exact bytes it sends over the network) to the standard `java.sql` interfaces, which is why the same application code runs on several databases.
- **Spring MVC `HandlerAdapter`** lets the `DispatcherServlet` (the central object in Spring MVC, Spring's web framework, that receives every web request) invoke very different kinds of handlers (annotated controllers, `HttpRequestHandler`, etc.) through one interface.
- **Almost every "client" wrapper around a vendor SDK** in a real codebase — if it exists so the rest of the code does not import the SDK, it is an adapter.

In modern languages

- **First-class functions** (functions that can be stored in variables and passed around like values): if the Target has a single method, a **lambda** (a short unnamed function) is a complete adapter — `PaymentGateway g = m -> new Receipt(stripe.charge(m.cents(), "usd"), "SUCCEEDED");` (our `FahrenheitSensor oldSensor = () -> 212.0;` is the same trick).
- **Python**: duck typing ("if it has the right methods, it fits" — checked when the code runs, never declared) means any object with the expected method names fits, so a small wrapper class — or even a `types.SimpleNamespace` — is enough.
- **TypeScript**: structural typing accepts anything of the right shape, so adapters are frequently plain object literals (objects written inline as `{ ... }`).
- **Go**: interfaces are satisfied implicitly (a type that has the methods automatically counts — no `implements` needed), so the adapter is often just a small named type with one method.

In the interview

> **30-second answer:** "An Adapter converts one interface into another that the client expects. I use it at boundaries I do not control — a vendor SDK, a legacy service. The adapter implements *my* interface, holds the vendor object, and translates inputs, outputs and exceptions, so no vendor type leaks into my code. Swapping vendors becomes one new class. I prefer the object adapter over the class adapter because it uses composition and works with final classes."

Follow-ups

- *Q: Adapter vs Decorator?* — A: Adapter changes the interface and keeps the behaviour. Decorator keeps the interface and changes (adds to) the behaviour.
- *Q: Adapter vs Facade?* — A: An Adapter converts **one** interface to **one** other, usually because you only own one side. A Facade simplifies a **whole subsystem** of several objects into one entry point, usually over code you own.
- *Q: Why not just change the other side?* — A: Say this up front, unprompted: it is a vendor library, another team's API, or code with many callers. That constraint is what justifies the pattern — without it you are pattern-matching out of habit.
- *Q: What ruins an adapter?* — A: A vendor type or vendor exception in the adapter's public signature.
- *Q: How does it help testing?* — A: The checkout depends on a small interface I own, so in tests I pass a fake `PaymentGateway`; I never have to fake the vendor SDK.
- *Q: What is a two-way adapter?* — A: One class implementing both interfaces so it can serve clients of either side, useful during migrations.

Self-check

1. **Q:** Name the four roles in the Adapter pattern. **A:** Target, Adaptee, Adapter, Client.

2. **Q:** Why is `StripeClient` impossible to class-adapt in our code? **A:** It is `final`; a class adapter must extend the adaptee.
3. **Q:** What does `Arrays.asList(arr).add(x)` do, and why? **A:** Throws `UnsupportedOperationException`, because the list is a fixed-size view (adapter) over the array.
4. **Q:** You own both classes and their interfaces differ slightly. Adapter? **A:** No — change the interface; an adapter would be permanent overhead.
5. **Q:** Where should the vendor's exception be handled? **A:** Inside the adapter, translated into your own exception type.

TL;DR

- **Adapter = translator** between the interface you have and the interface you need.
- Implements **your** interface, **holds** the vendor object, translates **in, out and errors**.
- Prefer **object adapter** (composition) over **class adapter** (inheritance). Two-way adapters implement both sides.
- Use at boundaries you **don't control**; skip it when you own both sides.

9.2 Bridge Pattern

What it is

The **Bridge** pattern splits one class that varies in *two independent ways* into **two separate hierarchies** connected by a reference (the "bridge"), so each side can grow on its own.

Analogy: a universal TV remote. Remotes come in kinds (basic, advanced with mute); TVs come in brands (Sony, Samsung). You do not buy a "BasicSonyRemote" — any remote works any TV through a common protocol (an agreed set of signals).

GoF's intent: *decouple an abstraction from its implementation so that the two can vary independently.* GoF also lists **Handle/Body** as another name for it. Its names for the parts — the vocabulary interviewers expect:

Term	Meaning	In our code
Abstraction	The high-level side the client uses; holds a reference to an Implementor	`Notification`
Refined Abstraction	Subclasses of the abstraction	`AlertNotification`, `ReminderNotification`, `DigestNotification`
Implementor	The interface of the low-level side that does the physical work	`Channel`
Concrete Implementor	Real implementations of that low-level work	`EmailChannel`, `SmsChannel`, `PushChannel`

> **Naming trap.** In Bridge, "implementation" is *not* the everyday Java meaning ("a class that implements an interface"). It names the **low-level half of the split** — the side that does the platform-specific, physical work (sending bytes, drawing pixels). The GoF terms are awkward, but they are the ones you will be asked about.

The problem it solves — the class explosion

You have three kinds of notification (alert, reminder, digest) and three delivery channels (email, SMS, push). If both live in **one** inheritance hierarchy you need a class for every combination:

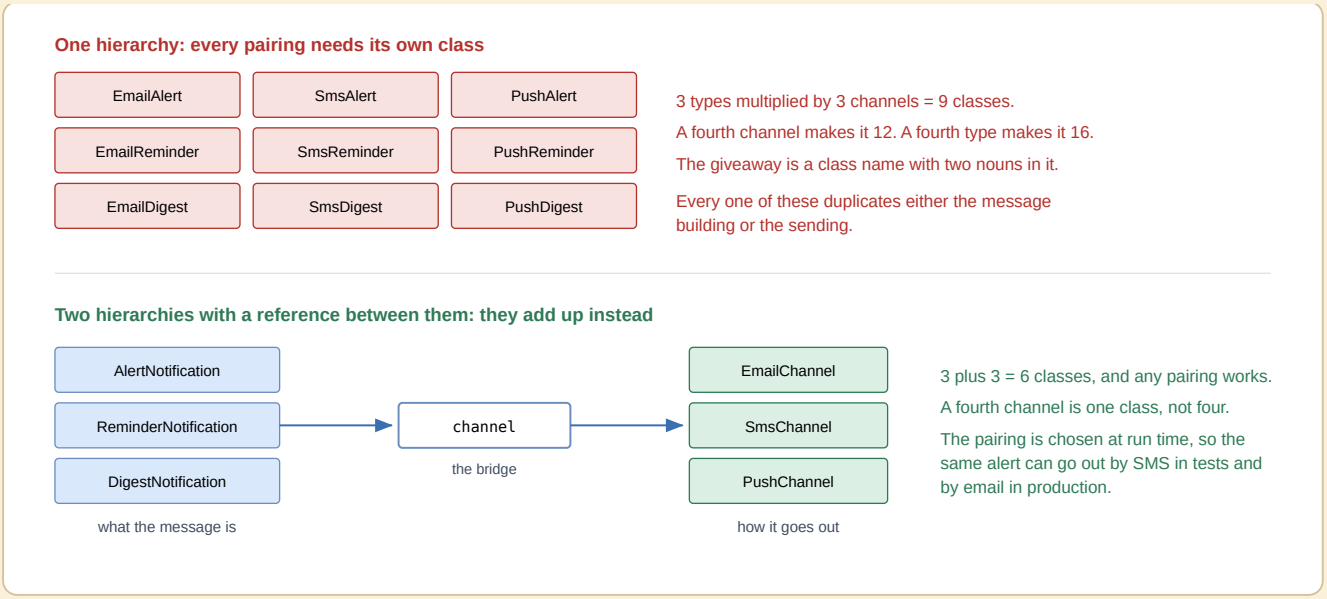


Figure — Top: one hierarchy needs 3 × 3 = 9 classes such as EmailAlert and SmsDigest. Bottom: two hierarchies joined by one `channel` reference need 3 + 3 = 6 (course diagram)

The figure shows the arithmetic. In a single hierarchy the concrete classes **multiply**: 3 × 3 = 9; one more channel gives 12; one more type on top gives 16. With a bridge they **add**: 3 + 3 = 6, and 4 + 4 = 8. (Counts are concrete classes; the `Notification` base and the `Channel` interface are the same in both designs.)

The **smell** (code smell: a surface symptom that hints at a deeper design problem) that points to a missing Bridge is **a class name made of two nouns** — `EmailAlert`, `SmsDigest`, `PushReminder`. Each such class repeats either the "build the message" code or the "send it" code, so changing how SMS delivery works means editing three classes that otherwise have nothing in common.

Splitting the two dimensions has one more pay-off: the pairing becomes a **run-time choice**. The same alert object can be built with an SMS channel in a test and an email channel in production (the live system real users depend on). The code is small; the genuinely hard part is deciding **which dimension is the abstraction and which is the implementation** (see "Getting the split wrong" below).

How it works

1. **Find the two dimensions** that vary independently (what the message *is* vs how it *goes out*).
2. **Make the lower-level dimension an interface** — the Implementor (`Channel.deliver(title, body)`). Keep it small and general.
3. **Make the higher-level dimension an abstract class** — the Abstraction — that **holds a reference** to the Implementor. That field is the bridge.
4. **Refined abstractions** build their own content and **delegate** the physical work across the bridge. They never check which implementor they have.
5. **Pair them at run time**: `new AlertNotification(new SmsChannel(...), ...)`. The same alert can go by SMS in a test and by email in production.

Structure

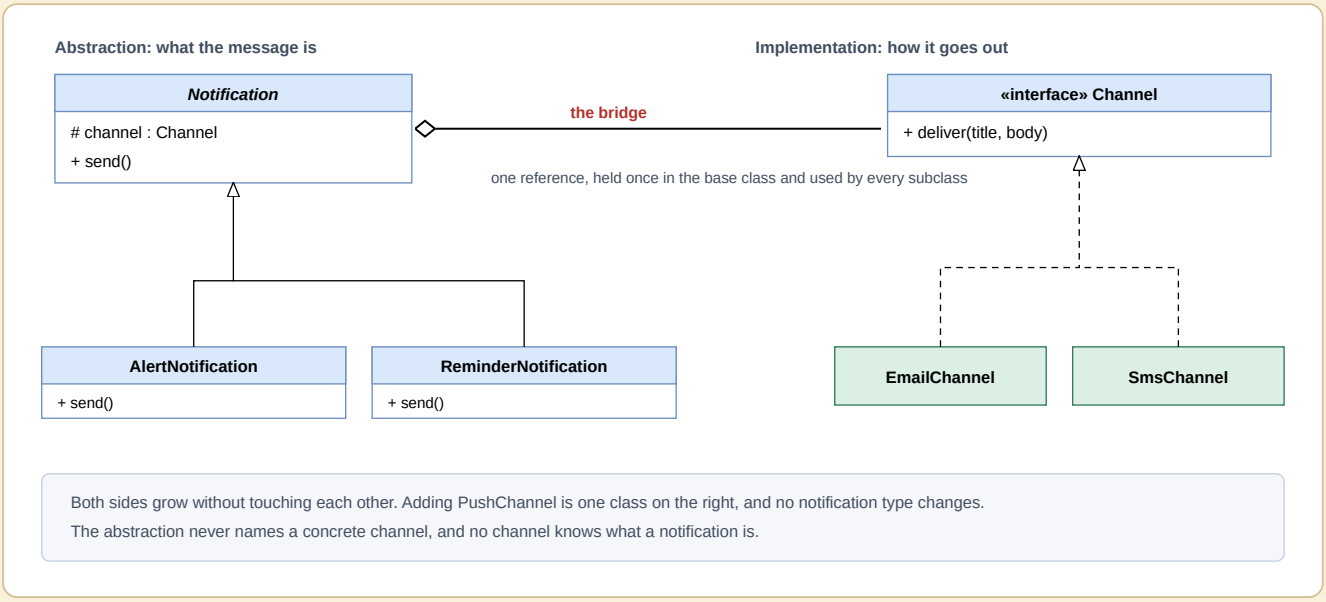
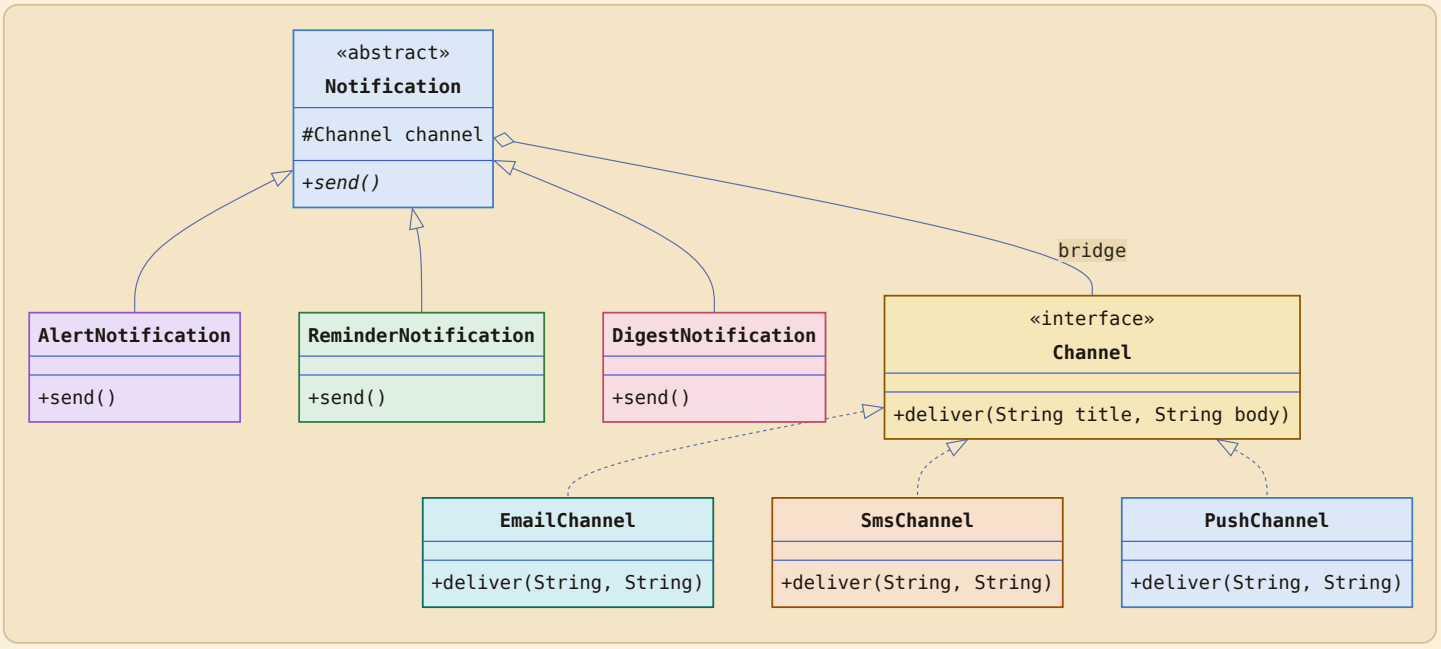


Figure — Bridge class diagram: `Notification` (abstraction) holds a `Channel` (implementor); alert/reminder notifications refine the abstraction; email/SMS channels implement the channel (course diagram)

In the class diagram above, the bridge is that one `channel` field: declared once in the base class, used by every subclass. The channel classes know nothing about notifications — neither what an alert is nor that alerts exist.



Java example

```java
// File: code/lld/ch09/bridge/Channel.java
package lld.ch09.bridge;

/**
 * IMPLEMENTOR: the low-level "how it goes out" side.
 * Kept small and general: two plain strings, nothing channel-specific.
 */
interface Channel {
    void deliver(String title, String body);
}
```

```java
// File: code/lld/ch09/bridge/EmailChannel.java
package lld.ch09.bridge;

/** CONCRETE IMPLEMENTORS: none of them knows what an alert or a digest is. */
final class EmailChannel implements Channel {
    private final String address;

    EmailChannel(String address) { this.address = address; }

    @Override
    public void deliver(String title, String body) {
        System.out.println("  [email to " + address + "] Subject: " + title + " | " + body);
    }
}
```

```java
// File: code/lld/ch09/bridge/SmsChannel.java
package lld.ch09.bridge;

final class SmsChannel implements Channel {
    private static final int LIMIT = 40;          // tiny limit so the demo shows truncation
    private final String phone;

    SmsChannel(String phone) { this.phone = phone; }

    @Override
    public void deliver(String title, String body) {
        String text = title + ": " + body;
        if (text.length() > LIMIT) text = text.substring(0, LIMIT - 3) + "...";
        System.out.println("  [sms to " + phone + "] " + text);
    }
}
```

```java
// File: code/lld/ch09/bridge/PushChannel.java
package lld.ch09.bridge;

final class PushChannel implements Channel {
    private final String deviceId;

    PushChannel(String deviceId) { this.deviceId = deviceId; }

    @Override
    public void deliver(String title, String body) {
        System.out.println("  [push to " + deviceId + "] (" + title + ") " + body);
    }
}
```

```java
// File: code/lld/ch09/bridge/Notification.java
package lld.ch09.bridge;

/**
 * ABSTRACTION: the high-level "what the message is" side.
 * The protected 'channel' field IS the bridge.
 */
abstract class Notification {
    protected final Channel channel;

    protected Notification(Channel channel) { this.channel = channel; }

    /** Builds the message, then hands it across the bridge. Never branches on channel type. */
    abstract void send();
}
```

```java
// File: code/lld/ch09/bridge/AlertNotification.java
package lld.ch09.bridge;

/** REFINED ABSTRACTIONS */
final class AlertNotification extends Notification {
    private final String system;
    private final String problem;

    AlertNotification(Channel channel, String system, String problem) {
        super(channel);
        this.system = system;
        this.problem = problem;
    }

    @Override
    void send() { channel.deliver("ALERT " + system, problem + " - act now"); }
}
```

```java
// File: code/lld/ch09/bridge/ReminderNotification.java
package lld.ch09.bridge;

final class ReminderNotification extends Notification {
    private final String what;
    private final String when;

    ReminderNotification(Channel channel, String what, String when) {
        super(channel);
        this.what = what;
        this.when = when;
    }

    @Override
    void send() { channel.deliver("Reminder", what + " at " + when); }
}
```

```java
// File: code/lld/ch09/bridge/DigestNotification.java
package lld.ch09.bridge;

import java.util.List;

final class DigestNotification extends Notification {
    private final List<String> items;

    DigestNotification(Channel channel, List<String> items) {
        super(channel);
        this.items = List.copyOf(items);
    }

    @Override
    void send() { channel.deliver("Daily digest (" + items.size() + ")", String.join("; ", items)); }
}
```

```java
// File: code/lld/ch09/bridge/Main.java
package lld.ch09.bridge;

import java.util.List;

public class Main {
    public static void main(String[] args) {
        Channel email = new EmailChannel("ops@shop.com");
        Channel sms = new SmsChannel("+1-555-0100");
        Channel push = new PushChannel("pixel-7");

        System.out.println("== Any type x any channel, paired at run time ==");
        List<Notification> outbox = List.of(
                new AlertNotification(email, "payments", "error rate 12%"),
                new AlertNotification(sms, "payments", "error rate 12%"),
                new ReminderNotification(push, "Stand-up", "09:30"),
                new DigestNotification(email, List.of("3 orders", "1 refund", "0 incidents")),
                new DigestNotification(sms, List.of("3 orders", "1 refund", "0 incidents")));
        outbox.forEach(Notification::send);

        System.out.println("== Class count ==");
        int types = 3, channels = 3;
        System.out.println("  one hierarchy (type x channel): " + types * channels + " classes");
        System.out.println("  bridge (type + channel):        " + (types + channels) + " classes");
        System.out.println("  with 4 types and 4 channels:    " + 4 * 4 + " vs " + (4 + 4));
    }
}
```

Output:

```
== Any type x any channel, paired at run time ==
  [email to ops@shop.com] Subject: ALERT payments | error rate 12% - act now
  [sms to +1-555-0100] ALERT payments: error rate 12% - act now
  [push to pixel-7] (Reminder) Stand-up at 09:30
  [email to ops@shop.com] Subject: Daily digest (3) | 3 orders; 1 refund; 0 incidents
  [sms to +1-555-0100] Daily digest (3): 3 orders; 1 refund;...
== Class count ==
  one hierarchy (type x channel): 9 classes
  bridge (type + channel):        6 classes
  with 4 types and 4 channels:    16 vs 8
```

What each part does

- `Channel` — the Implementor. Just `deliver(title, body)` with two plain strings, so every channel can support it.
- `EmailChannel`, `SmsChannel`, `PushChannel` — each knows *only* how to deliver. `SmsChannel` truncates long text; that rule lives inside SMS, not in any notification.
- `Notification` — the Abstraction, holding `protected final Channel channel` (the bridge).
- `AlertNotification` etc. — each builds a title and body and calls `channel.deliver(...)`. No `instanceof` (Java's check of an object's concrete type), no `switch` on channel type.
- Adding `SlackChannel` = one new class; adding `PromoNotification` = one new class. Nothing else changes.

Bridge vs Strategy

Written down, the two are **indistinguishable** — an object keeps a reference to an interface and delegates through it. Interviewers and code reviewers both ask about this, so have an answer ready: the difference is the **reason the reference is there**, not how it looks.

	Bridge	**Strategy**
What varies	A whole **dimension** of the design	**One algorithm** or policy
When chosen	Usually once, at construction, then it stays	Often swapped repeatedly, even per call
Why it exists	To stop two hierarchies multiplying	To make one behaviour replaceable
When decided	Up front, as a structural decision	Later, when a second algorithm appears
Example	`Notification` → `Channel`	`list.sort(comparator)`

Practical test: picture adding one more implementation. If it is *a new kind of thing the system supports* (another delivery channel), you have a Bridge. If it is *one more way to perform a single step* (another compression algorithm), you have a Strategy. When you truly can't decide, relax — the code is identical and either name will be understood. What does matter is not building either one while an axis has only a single member.

Getting the split wrong

Bridge is quick to write but costly to re-cut once both sides have several classes. Two ways it goes wrong:

- **Low-level details leaking up.** An `if (channel instanceof SmsChannel)` inside `AlertNotification` means the bridge is no longer doing its job — that SMS-specific rule should move into `SmsChannel`.
- **An implementor interface shaped around one implementation.** Add `setEmailReplyTo(...)` to `Channel` and every non-email channel must now implement a method it can only throw from. Keep the low-level interface minimal and generic; that is why `deliver` accepts two plain strings instead of a rich message type.

Trade-offs

Pros	Cons
Class counts add rather than multiply — 3 types and 3 channels need 6 classes, not 9.	**Two hierarchies** to understand instead of one.
Each side extends independently (Open/Closed) — a new channel touches no notification class.	**More indirection** — following one call means reading two files.
Pairing picked at run time — real channel in production, fake in tests, per-environment config.	**Hard to undo** — once both sides have implementations, moving the split line is a large refactor (restructuring code without changing what it does).
Platform code kept apart — only one class knows about SMTP (the protocol for sending email) (Single Responsibility).	**Pointless with one member** — a bridge with a single implementor is just an interface plus ceremony.

When not to use it

- **An axis has only one member.** With a single channel the bridge abstracts nothing; introduce it when the second channel actually shows up, not in advance.
- **The axes are not truly independent.** If some combinations are forbidden (e.g. digests must never be sent by SMS), Bridge happily lets you build them, and you end up adding run-time checks that separate subclasses would have made impossible.
- **You can't state in one sentence which side is the abstraction.** Then the design isn't ready — picking the wrong split is more expensive than living with a few extra classes.
- **The second axis is really a behaviour, not a kind of thing.** Use Strategy.

Common mistakes

- Building the bridge "just in case" with a single implementor.
- Letting the abstraction branch on the concrete implementor type.
- Making the implementor interface mirror one specific implementation.
- Confusing Bridge with Adapter: Adapter is **retrofitted** to make existing things fit; Bridge is **designed in** so the two sides never get fused into one hierarchy.

Real-world examples

- **JDBC**: your code uses `java.sql.Connection` / `Statement`; each database driver supplies the implementation. Application code and drivers evolve separately, which is why the same query code can run on PostgreSQL or MySQL. (JDBC is often cited as both Bridge — two independent hierarchies — and Adapter — each driver adapts a native protocol. Both views are fair.)
- **SLF4J and similar logging APIs**: the logging API you call is one side and the backend that actually writes log lines is the other. (SLF4J calls itself a *facade*; the API/backend split underneath is Bridge-shaped.)
- **Window systems and device/graphics drivers**: GoF's own motivating example is a `Window` abstraction whose `WindowImp` implementation is written once per platform (X Window System, IBM Presentation Manager). OS drawing APIs over per-card graphics drivers follow the same idea.
- **`java.awt` peers** (the textbook Java example): each AWT component held a platform-specific peer object that did the actual native drawing.

- **Notification / payment services in most apps**: a `NotificationService` interface with pluggable providers (SES, Twilio, FCM — Amazon's email service, an SMS provider, Google's push-notification service) is this pattern, whether or not anyone called it Bridge.
- **Dependency-injection containers** (frameworks that create objects and hand each one the dependencies it needs) that bind an interface to a different implementation per environment give you Bridge-style pairing that is configured rather than coded.

In modern languages

- **First-class functions**: if the implementor has one method, it can simply be a function — `Notification(channel = send_email)` is the same design without a `Channel` class.
- **Python / JavaScript**: duck typing means the implementor side needs no declared interface — any object with a `deliver` method will do.
- **Go**: a small interface declared by the *consumer*, with implementations in other packages, is ordinary idiomatic (natural, conventional-style) Go — Bridge without the name.
- **DI containers** (Spring, Guice, .NET): the pairing is wired by configuration per environment.

Other patterns it is confused with

Looks like	Actual difference
Strategy	Same code shape; Bridge separates a permanent structural dimension, Strategy makes one algorithm swappable.
Adapter	The class diagrams can look alike, but the timing is reversed: an Adapter is retrofitted to reconcile two things that already exist; a Bridge is planned before the classes are written.
Abstract Factory	Abstract Factory decides *which family of objects gets created*; Bridge describes *how an object hands off work once it exists*. They work well together — a factory commonly supplies the implementor a bridge receives.
Decorator	A Decorator shares the interface of the object inside it and adds behaviour; in a Bridge the two sides have unrelated interfaces and neither one wraps the other.

In the interview

> **30-second answer:** "Bridge separates an abstraction from its implementation so both can vary independently. If notifications vary by type and by channel, one hierarchy needs type × channel classes — 3 × 3 = 9, and 16 with a fourth of each. I split it: a `Notification` hierarchy holding a reference to a `Channel` interface. Now it is type + channel classes — 6, then 8 — and the pairing is chosen at run time. I wouldn't build it while there is only one channel."

Follow-ups

- *Q: How is it different from Strategy?* — A: The code is the same; the intent differs. Bridge splits a permanent dimension decided up front; Strategy makes one algorithm replaceable, often per call.
- *Q: What is the smell that says you need it?* — A: Class names with two nouns (`SmsDigest`) and duplicated code across them.
- *Q: When would you not use it?* — A: With only one channel today — I would add the bridge when the second one lands. Also when some combinations are invalid.
- *Q: Who decides which implementor a notification gets?* — A: Whoever builds it — often a factory or the DI container, configured per environment.

Self-check

1. **Q:** With 5 notification types and 4 channels, how many classes without and with Bridge? **A:** 20 without, 9 with.
2. **Q:** Which field is "the bridge"? **A:** The `Channel channel` reference held by the `Notification` base class.
3. **Q:** An `AlertNotification` checks `channel instanceof SmsChannel`. What is wrong? **A:** Implementation detail has leaked upward; move that behaviour into `SmsChannel`.
4. **Q:** Is Bridge added before or after the classes exist? **A:** Before — it is an up-front structural decision (Adapter is the after-the-fact one).

TL;DR

- **Two dimensions varying → two hierarchies + one reference.** Multiplication becomes addition.
- Abstraction (what) holds Implementor (how). Keep the implementor small and general.
- Same code as Strategy; different intent. Don't build it for one implementor.

9.3 Composite Pattern

What it is

The **Composite** pattern gives an individual item and a whole nested group of items **one common interface**, so code that uses them never needs to check whether it has a **single object** or an **entire tree**.

Analogy: a company org chart. You can ask any manager "what is your team's total salary?" — a manager answers by asking each report and adding up; an individual contributor just answers with their own salary. The question is the same for both.

Term	Meaning	In our code
Component	The shared interface for leaves and containers	`FileSystemNode`, `MenuComponent`
Leaf	An object with no children; does the work itself	`FileNode`, `MenuItem`
Composite	A container that holds children (typed as Component) and delegates to them	`FolderNode`, `Menu`
Client	Code that works with Components only	`report(...)`

GoF's intent in plain words: compose objects into tree structures that represent **part-whole hierarchies**, and let clients treat individual objects and compositions **uniformly**.

The problem it solves

Plenty of data is nested: folders contain files *and* folders; an invoice contains line items *and* bundles that have their own items; a UI (user interface) panel contains widgets (buttons, text boxes and other screen controls) *and* sub-panels. Ask for a total size, a total price or a rendering of the whole thing, and — without the pattern — each operation (`size`, `print`, `search`, `copy`) opens with the same type test:

```
if (node instanceof Folder f) { /* loop over children, recurse */ }
else if (node instanceof File f) { /* read the size */ }
```

That test gets pasted into every operation. Introduce a third kind of node — say a symbolic link (a file-system entry that points to another file or folder) — and you have to hunt down each copy; overlook one and the new node type quietly gets the wrong answer.

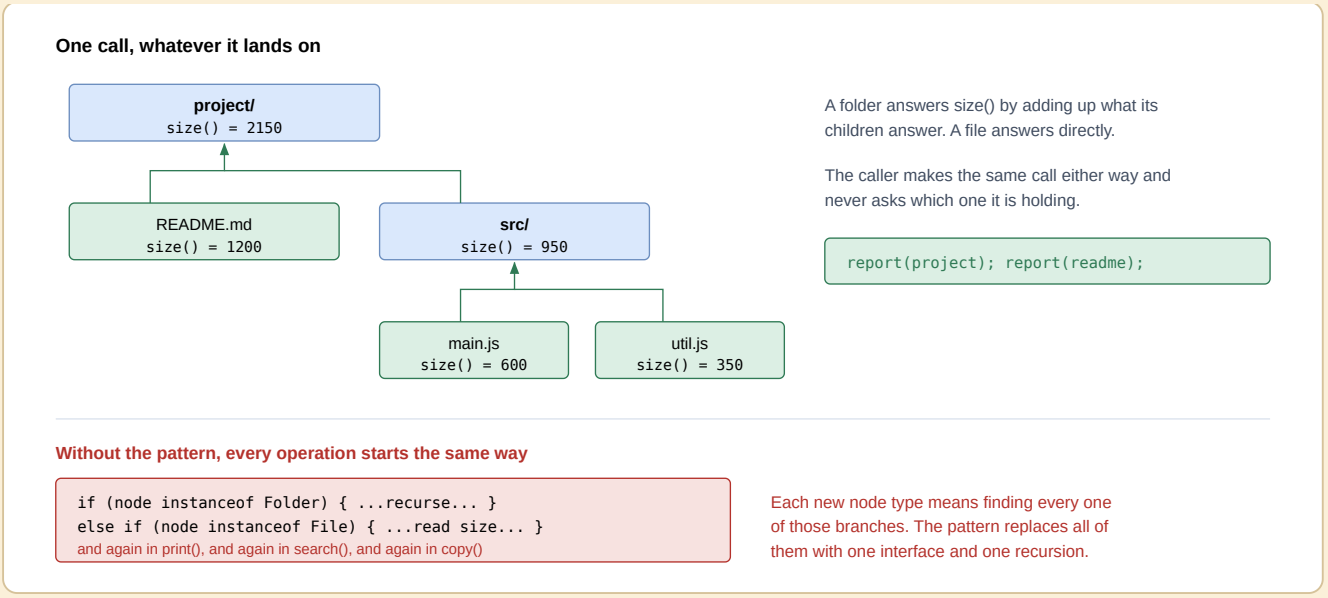


Figure — `size()` on the project folder (2150) is the sum of README.md (1200) and src/ (950), which itself sums main.js and util.js. The caller makes the same call on a tree or a single file (course diagram)

The figure shows the fix: a file answers `size()` with its own byte count, and a folder answers by asking each child and summing. Because a child may be a file or another folder and both respond to the same call, a single piece of recursive code (code that calls itself on the smaller parts) handles any depth. The caller writes `report(project)` or `report(readme)` and never asks which one it holds.

How it works

1. **Define a Component interface** with operations that make sense for *both* leaves and containers (`name()`, `size()`, `print()`).
2. **Leaf** implements them directly (a file returns its own byte count).
3. **Composite** stores a `List<Component>` — the **interface type**, not the leaf type — and implements each operation by **asking each child and combining the results**.
4. Because a child can itself be a composite, the recursion is written **once** and works to any depth.
5. **Clients** hold a `Component` and never type-check.

Structure

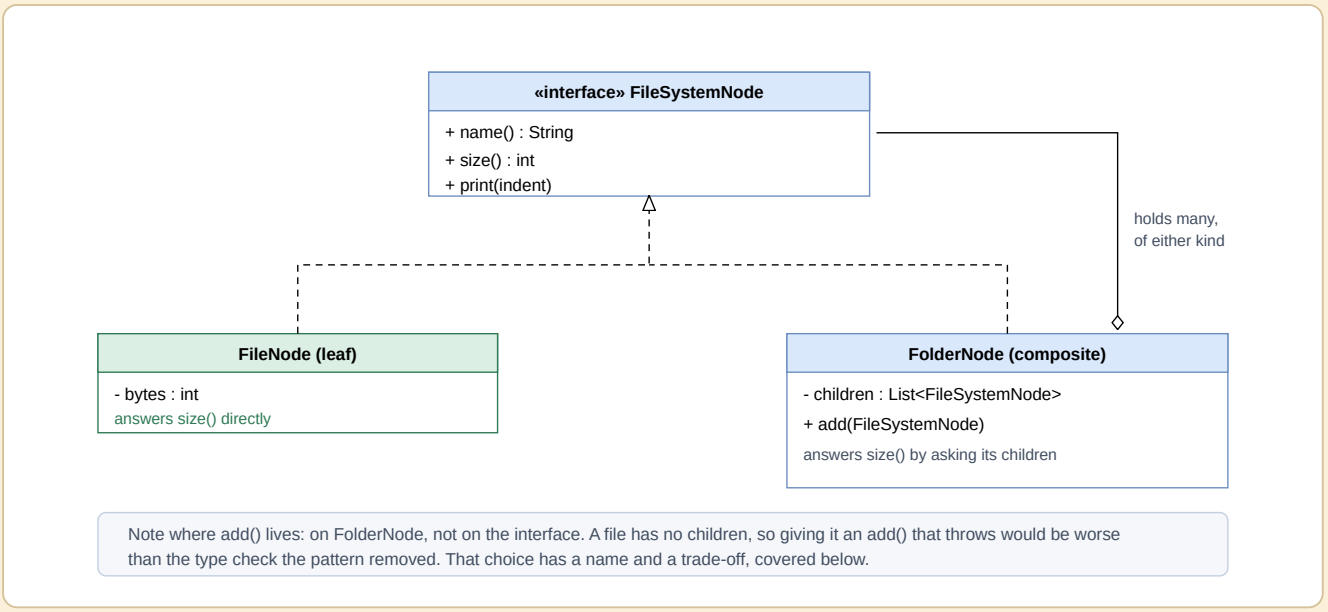
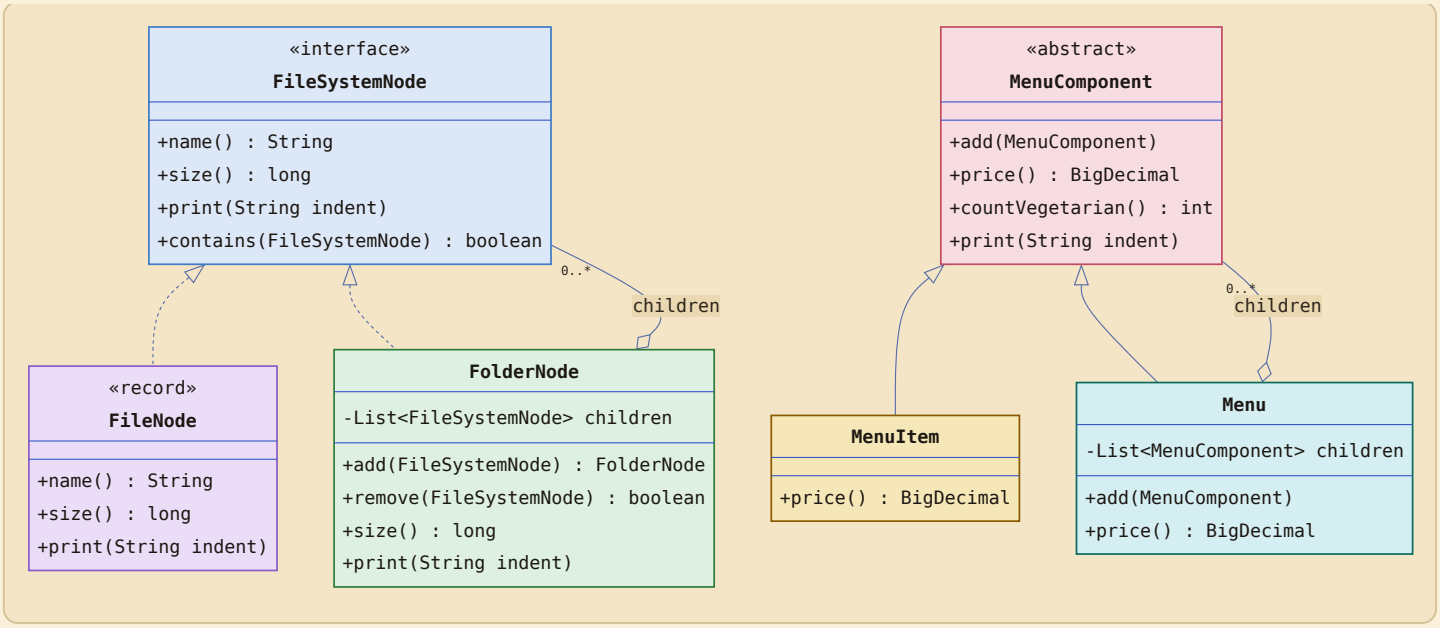


Figure — Composite class diagram: `FileNode` (leaf) and `FolderNode` (composite) implement `FileSystemNode`; the folder holds a list of `FileSystemNode` and has `add()` (course diagram)

Two details in the class diagram above matter: the folder's child list is typed as the **interface** — that is what lets folders hold folders — and `add()` is declared on `FolderNode` rather than on the shared interface. The second is a deliberate trade-off (the "safe" design), covered right after the code.



Java example

```java
// File: code/lld/ch09/composite/FileSystemNode.java
package lld.ch09.composite;

/**
 * COMPONENT (safe version): only operations that make sense for BOTH leaf and composite.
 * add/remove are NOT here - they live on FolderNode only.
 */
interface FileSystemNode {
    String name();
    long size();
    void print(String indent);

    /** True if 'target' is this node or somewhere below it. Used for the cycle check. */
    default boolean contains(FileSystemNode target) { return this == target; }
}
```

```java
// File: code/lld/ch09/composite/FileNode.java
package lld.ch09.composite;

/** LEAF: no children, answers directly. */
record FileNode(String name, long size) implements FileSystemNode {
    @Override
    public void print(String indent) {
        System.out.println(indent + "- " + name + " (" + size + " B)");
    }
}
```

```java
// File: code/lld/ch09/composite/FolderNode.java
package lld.ch09.composite;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/** COMPOSITE: holds children typed as the INTERFACE, so folders can hold folders. */
final class FolderNode implements FileSystemNode {
    private final String name;
    private final List<FileSystemNode> children = new ArrayList<>();

    FolderNode(String name) { this.name = name; }

    /** Returns this so trees can be built fluently. Rejects cycles at insert time. */
    FolderNode add(FileSystemNode child) {
        if (child.contains(this)) {
            throw new IllegalArgumentException("cycle: " + child.name() + " already contains " + name);
        }
        children.add(child);
        return this;
    }

    boolean remove(FileSystemNode child) { return children.remove(child); }

    List<FileSystemNode> children() { return Collections.unmodifiableList(children); }

    @Override public String name() { return name; }

    /** The recursion lives in ONE place. */
    @Override
    public long size() {
        long total = 0;
        for (FileSystemNode child : children) total += child.size();
        return total;
    }

    @Override
    public void print(String indent) {
        System.out.println(indent + "+ " + name + "/ (" + size() + " B)");
        for (FileSystemNode child : children) child.print(indent + "   ");
    }

    @Override
    public boolean contains(FileSystemNode target) {
        if (this == target) return true;
        for (FileSystemNode child : children) {
            if (child.contains(target)) return true;
        }
        return false;
    }
}
```

```java
// File: code/lld/ch09/composite/MenuComponent.java
package lld.ch09.composite;

import java.math.BigDecimal;

/**
 * COMPONENT (transparent version): child management is on the shared type,
 * so callers never cast - but a leaf must reject it at run time.
 */
abstract class MenuComponent {
    void add(MenuComponent c) { throw new UnsupportedOperationException(name() + " is a leaf"); }

    abstract String name();
    abstract BigDecimal price();            // total price of everything under this node (money: BigDecimal, never
double)
    abstract int countVegetarian();
    abstract void print(String indent);
}
```

```java
// File: code/lld/ch09/composite/MenuItem.java
package lld.ch09.composite;

import java.math.BigDecimal;

final class MenuItem extends MenuComponent {
    private final String name;
    private final BigDecimal price;
    private final boolean vegetarian;

    MenuItem(String name, String price, boolean vegetarian) {
        this.name = name;
        this.price = new BigDecimal(price);     // from a String, so "3.49" is exactly 3.49
        this.vegetarian = vegetarian;
    }

    @Override String name() { return name; }
    @Override BigDecimal price() { return price; }
    @Override int countVegetarian() { return vegetarian ? 1 : 0; }
    @Override void print(String indent) {
        System.out.println(indent + name + (vegetarian ? " (v)" : "") + " $" + price);
    }
}
```

```java
// File: code/lld/ch09/composite/Menu.java
package lld.ch09.composite;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;

final class Menu extends MenuComponent {
    private final String name;
    private final List<MenuComponent> children = new ArrayList<>();

    Menu(String name) { this.name = name; }

    @Override void add(MenuComponent c) { children.add(c); }
    @Override String name() { return name; }
    @Override BigDecimal price() {
        return children.stream().map(MenuComponent::price).reduce(BigDecimal.ZERO, BigDecimal::add);
    }
    @Override int countVegetarian() { return children.stream().mapToInt(MenuComponent::countVegetarian).sum(); }
    @Override void print(String indent) {
        System.out.println(indent + "[" + name + "]");
        children.forEach(c -> c.print(indent + "  "));
    }
}
```

```java
// File: code/lld/ch09/composite/Main.java
package lld.ch09.composite;

public class Main {

    /** Client: one parameter type, no instanceof anywhere. */
    static void report(FileSystemNode node) {
        System.out.println("  " + node.name() + " -> " + node.size() + " bytes");
    }

    public static void main(String[] args) {
        System.out.println("== Safe composite: file system ==");
        FileNode readme = new FileNode("README.md", 1200);
        FolderNode src = new FolderNode("src")
                .add(new FileNode("main.js", 600))
                .add(new FileNode("util.js", 350));
        FolderNode project = new FolderNode("project").add(readme).add(src);

        project.print("  ");
        report(project);   // a whole tree
        report(readme);    // a single file - same call

        try {
            src.add(project);   // project already contains src -> would loop forever
        } catch (IllegalArgumentException e) {
            System.out.println("  rejected: " + e.getMessage());
        }

        System.out.println("== Transparent composite: restaurant menu ==");
        MenuComponent all = new Menu("All menus");
        MenuComponent breakfast = new Menu("Breakfast");
        MenuComponent desserts = new Menu("Desserts");
        breakfast.add(new MenuItem("Pancakes", "3.49", true));
        breakfast.add(new MenuItem("Bacon & eggs", "4.99", false));
        desserts.add(new MenuItem("Apple pie", "2.59", true));
        breakfast.add(desserts);          // a menu inside a menu
        all.add(breakfast);
        all.add(new MenuItem("Coffee", "1.50", true));

        all.print("  ");
        System.out.println("  total $" + all.price() + ", vegetarian items: " + all.countVegetarian());
        try {
            new MenuItem("Tea", "1.00", true).add(new MenuItem("Milk", "0.20", true));
        } catch (UnsupportedOperationException e) {
            System.out.println("  transparent cost: " + e.getMessage());
        }
    }
}
```

Output:

```
== Safe composite: file system ==
  + project/ (2150 B)
     - README.md (1200 B)
     + src/ (950 B)
        - main.js (600 B)
        - util.js (350 B)
  project -> 2150 bytes
  README.md -> 1200 bytes
  rejected: cycle: project already contains src
== Transparent composite: restaurant menu ==
  [All menus]
    [Breakfast]
      Pancakes (v) $3.49
      Bacon & eggs $4.99
      [Desserts]
        Apple pie (v) $2.59
    Coffee (v) $1.50
  total $12.57, vegetarian items: 3
  transparent cost: Tea is a leaf
```

What each part does

- `FileSystemNode` — the Component. Only operations valid for both files and folders.

- `FileNode` (a `record`) — the Leaf: immutable, answers `size()` directly.
- `FolderNode` — the Composite. `size()` is the four-line recursion and the **only** place in the program that knows a tree can nest. `add()` returns `this` so trees build fluently.
- `contains(...)` + the check in `add()` — **cycle protection** at insert time. Trying to put `project` inside its own child `src` is rejected (see output).
- `MenuComponent`/`Menu`/`MenuItem` — the *transparent* variant (the restaurant-menu example popularised by *Head First Design Patterns*): `add` is on the shared abstract class and a leaf throws.
- `report(...)` — the client, called with a whole tree and with a single file.

Transparent vs safe composite

Where do `add` and `remove` belong? GoF named both answers:

	Transparent	Safe
Where `add/remove` live	On the shared Component	Only on the Composite
Uniformity	Full — every node looks the same; no casts (forcing a value to a more specific type)	Partial — builders must hold a `FolderNode`
Type safety	Weaker — `leaf.add(...)` compiles, then **throws at run time**	Strong — the compiler rejects `file.add(...)`
Our code	`MenuComponent` (`Tea is a leaf` in the output)	`FileSystemNode` / `FolderNode`
Choose when	Callers add children to arbitrary nodes without knowing their type (e.g. a UI editor where anything might be a container)	The tree is built in one place and read in many (most cases)

The transparent version swaps a compile-time guarantee for a possible run-time exception. The safe version keeps the read-side interface small, but any code that builds or edits the tree must hold a `FolderNode` — and code that walks a tree and then wants to insert needs a cast or type check after all. Neither is wrong; in review, check that the choice was **made on purpose**. A throwing `add` that nobody intended is the real defect.

Head First Design Patterns makes an interesting point here: the transparent composite knowingly bends the Single Responsibility Principle (the component both manages children and does work) to gain transparency. That is a conscious design trade-off, not an accident.

Two traps worth knowing

- **Cycles.** If a folder is ever placed inside one of its own descendants, `size()` recurses forever (in Java: `StackOverflowError`, thrown when nested calls use up the call stack — the memory that tracks calls in progress). **Validate when inserting**, not during traversal (walking through every node) — once you are traversing, you are already stuck. Our `add()` does exactly this.
- **Repeated work.** Our `print()` calls `size()` on every level, so the same subtotals are recomputed again and again — about quadratic (work grows with the square of the depth) for a deep chain. For big real trees, **store each subtree's total** and invalidate upward when a child changes (which needs a parent pointer).

Trade-offs

Pros	Cons
Callers never type-check — the same call works on a leaf or a whole tree.	**The shared interface tends to bloat** with operations that only half-fit one of the two kinds.
The recursion lives in one place, reused by each new operation.	**Weaker type safety** (transparent variant) — the compiler allows adding a child to a leaf.
Adding a node type doesn't touch callers (Open/Closed) — e.g. a `SymlinkNode`.	**Deep trees mean deep call stacks**, and a cycle shows up as endless recursion instead of a clear error.
Nesting comes for free — a composite that stores the interface can store other composites.	**Totals get recomputed** on every call unless you cache them.

When not to use it

- **Nothing nests.** A flat list gains nothing from a shared leaf/container interface.

- **Leaves and containers share little.** A forced common interface fills up with methods that throw, which is worse than the type test it replaced.
- **Deep tree on a hot path.** (A hot path is code that runs so often that its speed matters.) Each level costs a stack frame and likely CPU cache misses (fetching data from slow main memory instead of the processor's fast cache); precomputed totals, or a loop that manages its own work-list instead of recursing, can be faster.
- **The tree is modified while being traversed.** Concurrent changes (made at the same time by another thread) during recursion are dangerous, and Composite offers no protection.

Common mistakes

- Typing the child list as the **leaf** type (`List<FileNode>`) — then folders cannot hold folders.
- Returning the internal mutable child list — callers can bypass `add()` and its cycle check. We return `Collections.unmodifiableList(children)`.
- Adding `add()` to the interface by accident (transparent without meaning to).
- No cycle check.
- Using Composite when a single recursive function with one type check would do — the pattern pays off when **operations multiply**.

Real-world examples

- **The HTML DOM** (Document Object Model — the browser's in-memory tree of page elements) — an element holds child nodes, each possibly an element with children; `textContent` walks the whole subtree.
- **UI toolkits** — in Swing/AWT, `java.awt.Container` *is a* `Component` and holds `Component`s, which is why panels nest without limit. React (a JavaScript UI library) component trees and Android's `ViewGroup` (a `View` that holds `View`s) follow the same idea.
- **File systems** — the example used here.
- **Abstract syntax trees** (the tree a compiler builds from source code) in compilers — an expression node holds sub-expressions; evaluating any node evaluates its subtree.
- `java.io.File` represents both files and directories with one class — a fair illustration of the transparent cost: `listFiles()` returns `null` when called on a regular file.
- **JUnit 3 test suites** (JUnit is the standard Java testing framework; a suite is a named group of tests) — `TestSuite` implements `Test` and holds `Test`s (cases or other suites); running a suite runs everything beneath it. (JUnit 5 keeps the idea as a tree of test descriptors.)
- **Spring's** `CompositeCacheManager` delegates to a list of `CacheManager`s while itself being a `CacheManager`.
- **Invoices / bundles** — an e-commerce "bundle" priced as the sum of its items, which may themselves be bundles.

In modern languages

- **Python / JavaScript**: duck typing — leaf and container just need the same method names; a composite is often a class with a `children` list.
- **Sealed types + pattern matching** (Java 21, Kotlin, Scala, Rust) — a sealed type lists exactly which subtypes may exist, and pattern matching lets a `switch` branch on which subtype it has: a `sealed interface Node permits File, Folder` plus an exhaustive `switch` is a real alternative. The compiler then reports any node type a `switch` forgets — a guarantee Composite does not give. Prefer it when the set of node types is closed.
- **Functional languages**: a recursive algebraic data type (a type defined as "one of these shapes", where a shape may contain the type itself) is Composite with language support; a `fold` (a function that combines all parts of a structure into one result) over it is the recursion written once.
- **Java streams** (a pipeline API for processing collections): `flatMap` (turns each element into several and flattens the result) makes walking a composite read nicely once the structure exists.

Other patterns it is confused with

Looks like	Actual difference
Decorator	Both hold objects of their own interface. A Decorator wraps exactly **one** and adds behaviour; a Composite holds **many** and combines their results.
Iterator	Composite gives the tree a uniform interface; Iterator gives a way to walk it without exposing its structure. Often used together.
Visitor	Composite makes the structure uniform; Visitor adds new operations over it without editing node classes — the usual next step when operations keep multiplying.
A plain recursive function	If one function with one type check does the job, you don't need the pattern. Composite pays off when **many** operations would each repeat that branch.

In the interview

30-second answer: "Composite lets a client treat a single object and a tree of objects the same way. Leaf and container share one interface; the container stores children *typed as that interface* — which is what allows arbitrary nesting — and implements each operation by delegating to its children and combining the results. Its value is what it deletes: the `instanceof` branch in every operation. I choose between the transparent and safe variants depending on whether callers need `add` on arbitrary nodes, I reject cycles on insert, and I cache subtree totals if the tree is large."

Follow-ups

- *Q: Transparent or safe?* — A: Safe if the tree is built in one place and read in many; transparent if callers add children to arbitrary nodes and casts would spread everywhere.
- *Q: What about cycles?* — A: Check on insert: reject `add(child)` if the child already contains the parent.
- *Q: Performance on a big tree?* — A: Cache subtree totals, invalidate upward on change; or use an iterative walk to avoid deep recursion.
- *Q: Composite vs Decorator?* — A: Both hold objects of their own interface. Decorator holds **one** and adds behaviour; Composite holds **many** and combines their results.
- *Q: How do you add a new operation without touching every node?* — A: Use Visitor over the composite structure.

Self-check

1. **Q:** Why must the child list be `List<FileSystemNode>`, not `List<FileNode>`? **A:** So a folder can contain other folders — arbitrary nesting.
2. **Q:** In the transparent variant, what happens when you call `add` on a leaf? **A:** It compiles but throws at run time (`UnsupportedOperationException`).
3. **Q:** Where should a cycle be detected? **A:** At insertion time.
4. **Q:** Name a JDK class that is both a component and a container of components. **A:** `java.awt.Container`.

TL;DR

- **Part-whole trees, one interface.** Leaf answers directly; composite asks children and combines.
- Child list holds the **interface**. Recursion lives in one place.
- **Safe** (add only on composite, compile-time safe) vs **transparent** (add everywhere, run-time throw). Guard against **cycles**; cache big totals.

9.4 Decorator Pattern

What it is

The **Decorator** pattern extends what **a single object** does, **while the program runs**, by enclosing it in a wrapper that implements **the same interface**. The original class is never edited, and you don't need a separate subclass for each mix of extras.

Analogy: ordering coffee. Start with an espresso, add milk, add mocha, add whipped cream. Each topping wraps the drink and adds to its price and description; the result is still "a drink".

Term	Meaning	In our code
Component	The shared interface	`UserRepository`, `Beverage`, `InputStream`
Concrete Component	The real object that does the core work	`DatabaseUserRepository`, `Espresso`
Decorator (base)	Implements the Component, holds a Component, forwards everything	`RepositoryDecorator`, `java.io.FilterInputStream`
Concrete Decorator	Adds one behaviour around the forwarded call	`LoggingRepository`, `CachingRepository`, `Mocha`, `UpperCaseInputStream`

GoF's intent: *attach additional responsibilities to an object dynamically — a flexible alternative to subclassing for extending functionality.* GoF also calls it **Wrapper** (the same nickname as Adapter).

The key property: **a decorator IS-A Component and HAS-A Component.** Since it has the very type it wraps, a decorator can sit around the real object, around another decorator, or around a test double (any stand-in object used in tests — fake, stub or mock), and code that only sees the interface cannot tell which.

The problem it solves

First a repository (an object that loads and saves data, hiding the database behind methods) needs logging, then caching, later retries and metrics (counters and timings the app records about itself). Each caller wants its own mix of these, and sometimes in its own order.

- **Subclassing can't keep up.** A `LoggingCachingRetryingUserRepository` serves one mix only. Four optional concerns give 2^4 = **16** possible mixes, and the class count doubles with every new concern — a class explosion.
- **One class with on/off flags** holds up until a caller wants "everything except the cache". One boolean turns into four, and you end with `new Repo(true, false, true, true)`, which nobody can read.

Decorator gives **each concern its own class** with the component's interface. Four concerns cost **four classes**, and each caller assembles exactly the stack it needs — by wrapping at run time rather than inheriting at compile time. That is the Open/Closed Principle at work: new behaviour arrives as new classes, and existing classes stay untouched.

How it works

1. Start from a **Component interface** (`UserRepository`).
2. Write the **concrete component** that does the real work (`DatabaseUserRepository`).
3. Write an **abstract base decorator** that implements the interface, holds an `inner` of the **interface type**, and **forwards every method** by default.
4. Each **concrete decorator** extends the base and overrides only the methods it changes, doing something **before and/or after** calling `inner`.
5. **Compose by wrapping**: `new Logging(new Caching(new Database()))`. The outermost object is what the caller holds.

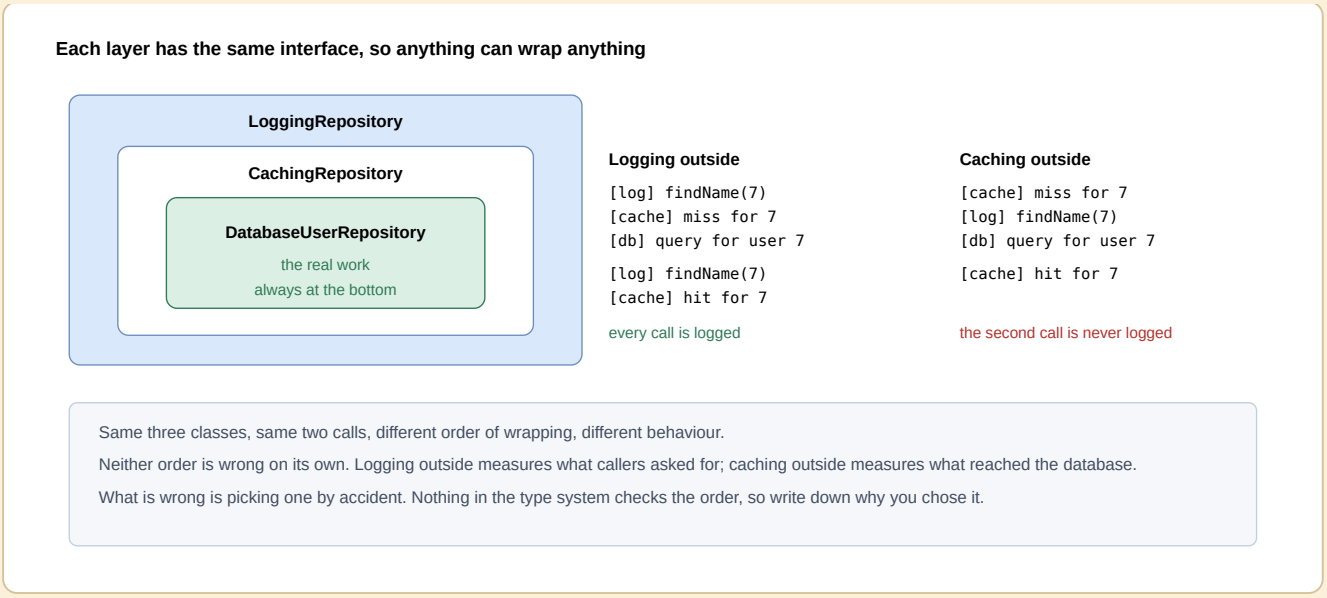


Figure — The same three classes stacked in two orders. Logging outside caching logs every call; caching outside logging means the second call (a cache hit) is never logged (course diagram)

Structure

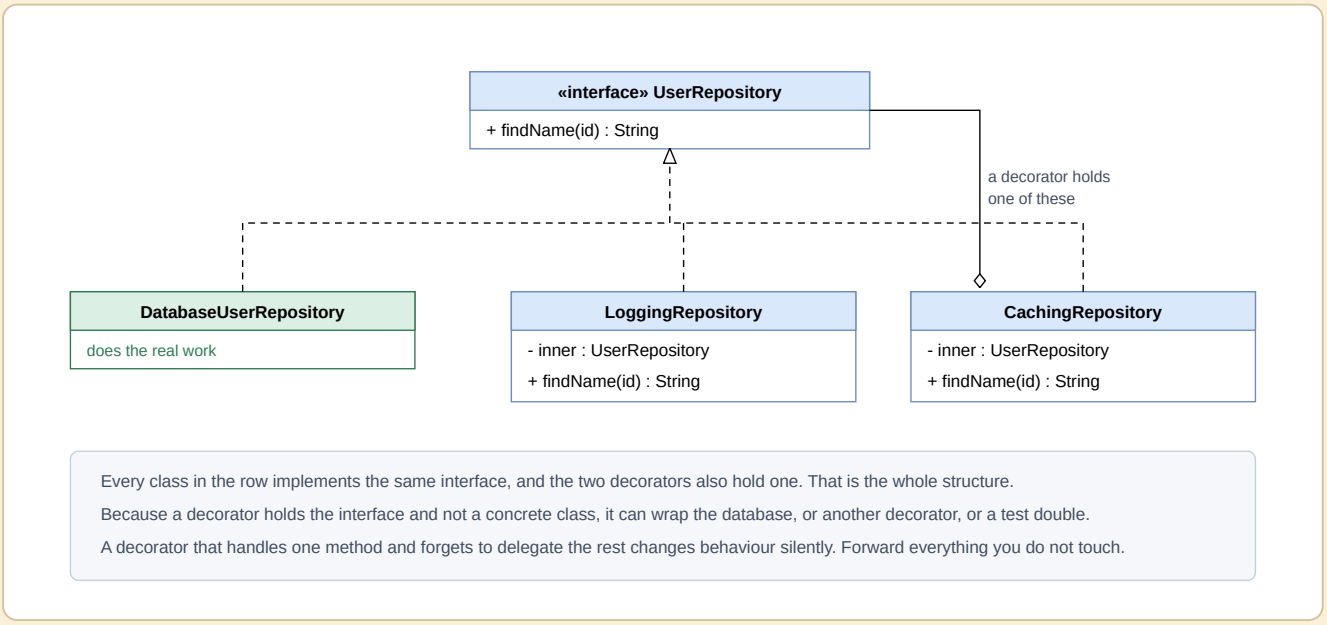
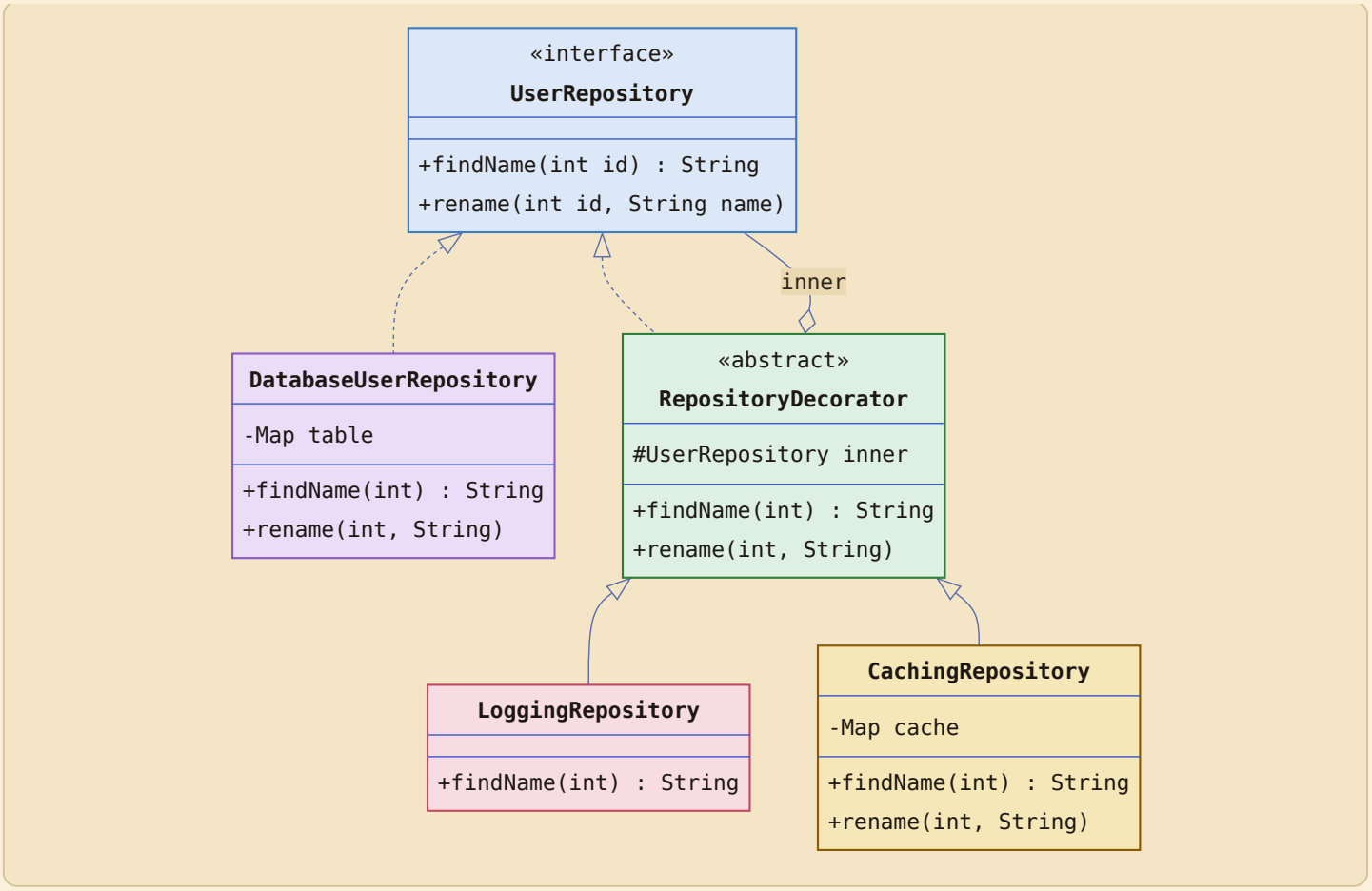


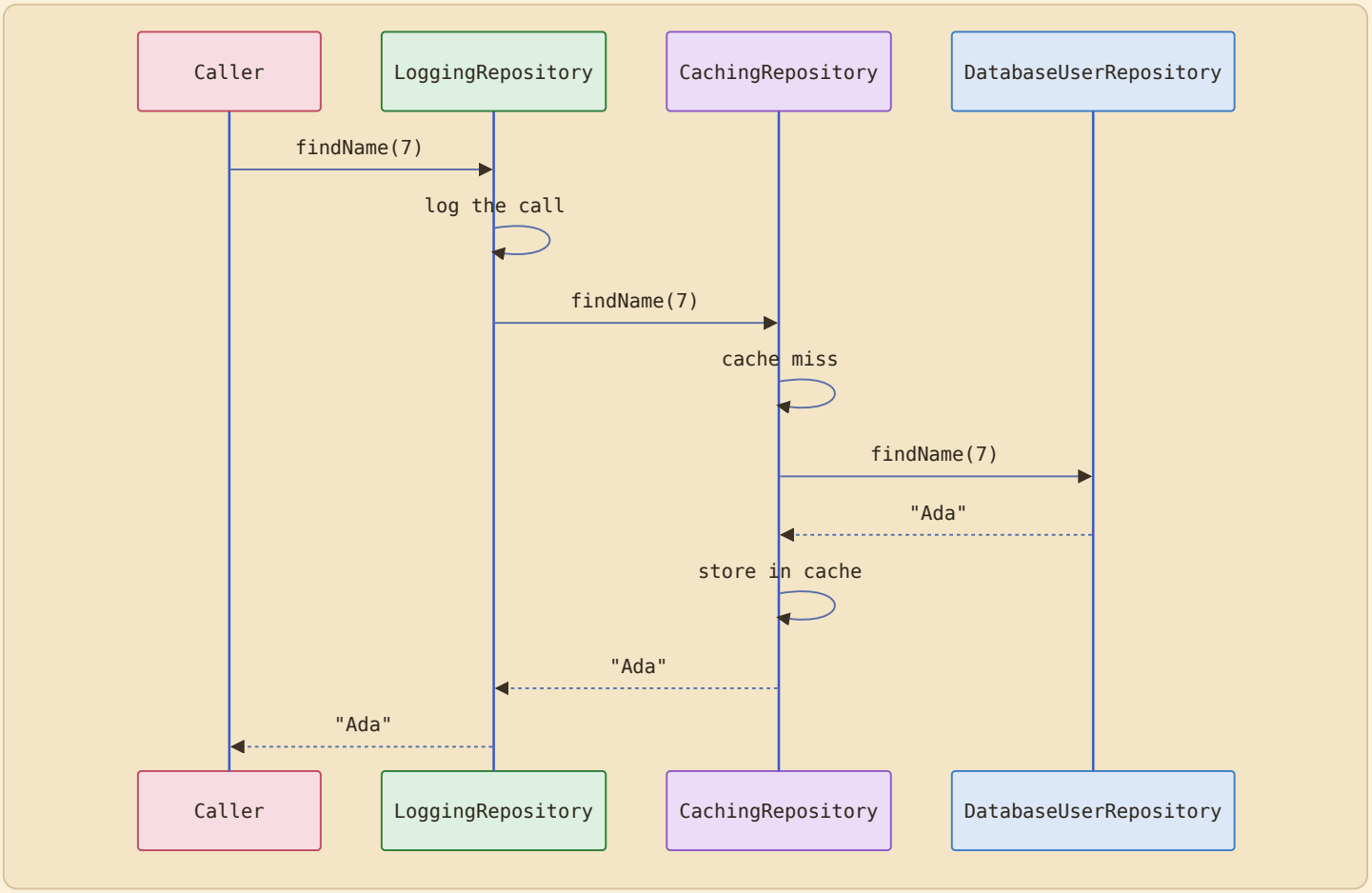
Figure — Decorator class diagram: `DatabaseUserRepository`, `LoggingRepository` and `CachingRepository` all implement `UserRepository`; each decorator holds an `inner` UserRepository (course diagram)

In the diagram above, every class implements `UserRepository` — the one property everything else depends on. `DatabaseUserRepository` always sits at the bottom and wraps nothing. Because each decorator's `inner` field is typed as the interface, decorators can wrap each other in any combination and any order.

The implementation detail that bites in practice is **delegation**: a decorator must forward every method it is not deliberately changing. One that overrides `findName` but silently swallows `rename` changes behaviour in a way that is very hard to spot. In Java the usual guard is an abstract base decorator that forwards everything by default (our `RepositoryDecorator`).



A decorated call, logging outside caching:



Java example

```java
// File: code/lld/ch09/decorator/UserRepository.java
package lld.ch09.decorator;

/** COMPONENT interface: every box in the stack implements it. */
interface UserRepository {
    String findName(int id);
    void rename(int id, String name);
}
```

```java
// File: code/lld/ch09/decorator/DatabaseUserRepository.java
package lld.ch09.decorator;

import java.util.HashMap;
import java.util.Map;

/** CONCRETE COMPONENT: the real work, always at the bottom of the stack. */
final class DatabaseUserRepository implements UserRepository {
    private final Map<Integer, String> table = new HashMap<>(Map.of(7, "Ada", 8, "Linus"));

    @Override
    public String findName(int id) {
        System.out.println("    [db]    query for user " + id);
        return table.get(id);
    }

    @Override
    public void rename(int id, String name) {
        System.out.println("    [db]    update user " + id);
        table.put(id, name);
    }
}
```

```java
// File: code/lld/ch09/decorator/RepositoryDecorator.java
package lld.ch09.decorator;

/**
 * BASE DECORATOR: holds the INTERFACE (not a concrete class) and forwards everything.
 * Subclasses override only what they change, so no method is ever forgotten.
 */
abstract class RepositoryDecorator implements UserRepository {
    protected final UserRepository inner;

    protected RepositoryDecorator(UserRepository inner) { this.inner = inner; }

    @Override public String findName(int id)          { return inner.findName(id); }
    @Override public void rename(int id, String name) { inner.rename(id, name); }
}
```

```java
// File: code/lld/ch09/decorator/LoggingRepository.java
package lld.ch09.decorator;

/** CONCRETE DECORATOR 1 */
final class LoggingRepository extends RepositoryDecorator {
    LoggingRepository(UserRepository inner) { super(inner); }

    @Override
    public String findName(int id) {
        System.out.println("    [log]   findName(" + id + ")");
        return super.findName(id);
    }
}
```

```java
// File: code/lld/ch09/decorator/CachingRepository.java
package lld.ch09.decorator;

import java.util.HashMap;
import java.util.Map;

/** CONCRETE DECORATOR 2 - note it must also invalidate on writes. */
final class CachingRepository extends RepositoryDecorator {
    private final Map<Integer, String> cache = new HashMap<>();

    CachingRepository(UserRepository inner) { super(inner); }

    @Override
    public String findName(int id) {
        if (cache.containsKey(id)) {
            System.out.println("    [cache] hit for " + id);
            return cache.get(id);
        }
        System.out.println("    [cache] miss for " + id);
        String name = super.findName(id);
        cache.put(id, name);
        return name;
    }

    @Override
    public void rename(int id, String name) {
        cache.remove(id);
        super.rename(id, name);
    }
}
```

```java
// File: code/lld/ch09/decorator/Beverage.java
package lld.ch09.decorator;

/** A second, classic decorator example: a coffee order priced by stacking add-ons.
 *  Prices are whole cents (int), so no floating-point money anywhere. */
interface Beverage {
    String description();
    int costCents();
}
```

```java
// File: code/lld/ch09/decorator/Espresso.java
package lld.ch09.decorator;

record Espresso() implements Beverage {
    public String description() { return "Espresso"; }
    public int costCents() { return 199; }
}
```

```java
// File: code/lld/ch09/decorator/HouseBlend.java
package lld.ch09.decorator;

record HouseBlend() implements Beverage {
    public String description() { return "House Blend"; }
    public int costCents() { return 89; }
}
```

```java
// File: code/lld/ch09/decorator/Milk.java
package lld.ch09.decorator;

/** Add-ons are decorators: they ARE a Beverage and HAVE a Beverage. */
record Milk(Beverage inner) implements Beverage {
    public String description() { return inner.description() + ", Milk"; }
    public int costCents() { return inner.costCents() + 10; }
}
```

```java
// File: code/lld/ch09/decorator/Mocha.java
package lld.ch09.decorator;

record Mocha(Beverage inner) implements Beverage {
    public String description() { return inner.description() + ", Mocha"; }
    public int costCents() { return inner.costCents() + 20; }
}
```

```java
// File: code/lld/ch09/decorator/Whip.java
package lld.ch09.decorator;

record Whip(Beverage inner) implements Beverage {
    public String description() { return inner.description() + ", Whip"; }
    public int costCents() { return inner.costCents() + 10; }
}
```

```java
// File: code/lld/ch09/decorator/UpperCaseInputStream.java
package lld.ch09.decorator;

import java.io.FilterInputStream;
import java.io.IOException;
import java.io.InputStream;

/**
 * Our own java.io decorator. FilterInputStream is the JDK's "base decorator":
 * it holds an InputStream and forwards every call to it.
 * Works byte-by-byte, so it is only correct for ASCII text.
 */
final class UpperCaseInputStream extends FilterInputStream {
    UpperCaseInputStream(InputStream in) { super(in); }

    @Override
    public int read() throws IOException {
        int b = super.read();
        return b == -1 ? -1 : Character.toUpperCase((char) b);
    }

    @Override
    public int read(byte[] buf, int off, int len) throws IOException {
        int n = super.read(buf, off, len);
        for (int i = off; i < off + Math.max(n, 0); i++) {
            buf[i] = (byte) Character.toUpperCase((char) buf[i]);
        }
        return n;
    }
}
```

```java
// File: code/lld/ch09/decorator/Main.java
package lld.ch09.decorator;

import java.io.BufferedInputStream;
import java.io.ByteArrayInputStream;
import java.io.IOException;
import java.io.InputStream;
import java.math.BigDecimal;
import java.nio.charset.StandardCharsets;

public class Main {

    static void lookUpTwice(String label, UserRepository repo) {
        System.out.println("  " + label);
        repo.findName(7);
        repo.findName(7);
    }

    public static void main(String[] args) throws IOException {
        System.out.println("== Same layers, different order ==");
        lookUpTwice("Logging outside caching (logs every caller request):",
                new LoggingRepository(new CachingRepository(new DatabaseUserRepository())));
        lookUpTwice("Caching outside logging (logs only what reaches the db):",
                new CachingRepository(new LoggingRepository(new DatabaseUserRepository())));

        System.out.println("== Write goes through every layer and invalidates the cache ==");
        UserRepository repo = new LoggingRepository(new CachingRepository(new DatabaseUserRepository()));
        repo.findName(8);
        repo.rename(8, "Linus T.");
        System.out.println("  now: " + repo.findName(8));

        System.out.println("== Coffee: stack add-ons at run time ==");
        Beverage order = new Whip(new Mocha(new Mocha(new Espresso())));
        System.out.println("  " + order.description() + " = $" + BigDecimal.valueOf(order.costCents(), 2));
        Beverage latte = new Milk(new HouseBlend());
        System.out.println("  " + latte.description() + " = $" + BigDecimal.valueOf(latte.costCents(), 2));

        System.out.println("== java.io is built from decorators ==");
        InputStream in = new UpperCaseInputStream(
                new BufferedInputStream(
                        new ByteArrayInputStream("decorators wrap streams".getBytes(StandardCharsets.US_ASCII))));
        try (in) {
            System.out.println("  " + new String(in.readAllBytes(), StandardCharsets.US_ASCII));
        }
    }
}
```

Output:

```
== Same layers, different order ==
  Logging outside caching (logs every caller request):
    [log]   findName(7)
    [cache] miss for 7
    [db]    query for user 7
    [log]   findName(7)
    [cache] hit for 7
  Caching outside logging (logs only what reaches the db):
    [cache] miss for 7
    [log]   findName(7)
    [db]    query for user 7
    [cache] hit for 7
== Write goes through every layer and invalidates the cache ==
    [log]   findName(8)
    [cache] miss for 8
    [db]    query for user 8
    [db]    update user 8
    [log]   findName(8)
    [cache] miss for 8
    [db]    query for user 8
  now: Linus T.
== Coffee: stack add-ons at run time ==
  Espresso, Mocha, Mocha, Whip = $2.49
  House Blend, Milk = $0.99
== java.io is built from decorators ==
  DECORATORS WRAP STREAMS
```

What each part does

- `RepositoryDecorator` — the **abstract base decorator**. It forwards *every* method, so a concrete decorator that only cares about `findName` cannot accidentally drop `rename`. This is the standard Java defence against forgotten delegation.
- `LoggingRepository` — prints, then delegates.
- `CachingRepository` — answers from its map on a hit (the value is already cached; a *miss* means it is not), otherwise delegates and stores. (More on caches in System Design Fundamentals.) It also overrides `rename` to **invalidate** (throw away the cached copy that is now out of date) — the third block of output shows the next read going to the database again.
- `Beverage` records — the classic coffee example (from *Head First Design Patterns*, rewritten): `new Whip(new Mocha(new Mocha(new Espresso())))` costs 1.99 + 0.20 + 0.20 + 0.10 = **$2.49**. Double mocha is just wrapping twice — impossible with one subclass per combination.
- `UpperCaseInputStream extends FilterInputStream` — our own `java.io` decorator. `FilterInputStream` is the JDK's base decorator: it holds an `InputStream` and forwards to it.

Layer order changes behaviour

Compare the two stacks in the output — they behave differently:

- **Logger on the outside, cache inside**: every request is logged, even those the cache serves → the log records *what callers requested*.
- **Cache on the outside, logger inside**: only cache misses are logged → the log records *what hit the database*, and the repeat request is absent.

Both are valid; they answer different questions. The mistake is picking one **without meaning to** and then reading the log as if it answered the other question. The same choice comes up wherever layers are stacked:

Pair	Outside	Inside
Retry vs cache	Repeats the entire lookup	Repeats only the database call, and a retry that succeeds populates the cache. Rule of thumb: put retry **next to the thing that fails**.
Metrics vs cache	Times what the caller experiences	Times the database alone — the number an on-call engineer (whoever is currently responsible for answering alerts) is paged on. Many teams record both.
Authorization (checking the caller is allowed) vs cache	**Correct** — every request is checked	**Security bug** — a cached answer is returned without the check ever running

The type checker accepts every ordering. **Record the reason for the order in a comment where the stack is built**, or the next person adding a layer will guess.

Two related traps

- **Stateful decorators that assume one wrap.** (Stateful means it keeps data between calls.) Apply a counting decorator twice to the same object and every call is counted twice.
- **Lost identity.** The wrapper is a different object from what it wraps, so `==` (same object?), `equals` (equal value?) and `instanceof` give answers callers don't expect. Code that digs through layers to "reach the real object" is working against the pattern.

Decorator vs inheritance

	Subclassing	Decorator
When behaviour is chosen	Compile time	Run time
How many at once	One subclass	Any number of layers, any order
Classes for N optional features	Up to 2^N	N
Works on `final` classes / classes you don't own	No	Yes (only needs the interface)
Can reach private state	Protected members, yes	No — only the public interface

Java I/O streams: the decorator you already use

`java.io` is built almost entirely from decorators:

```
new BufferedReader(new InputStreamReader(new FileInputStream("data.txt"), UTF_8))
```

- `FileInputStream` — concrete component (reads bytes from a file).
- `InputStreamReader` — strictly an **adapter** (bytes → characters), not a decorator.
- `BufferedReader` — a **decorator** on `Reader` (adds buffering — reading big chunks into memory to serve many small reads — and `readLine()`).

On the byte side, `FilterInputStream` is the abstract decorator, and `BufferedInputStream`, `DataInputStream`, `PushbackInputStream`, `java.util.zip.GZIPInputStream` (via `InflaterInputStream`) and `java.security.DigestInputStream` are concrete decorators. *Head First Design Patterns* notes the downside: the design produces many small classes, which can overwhelm newcomers until they see the pattern.

Trade-offs

Pros	Cons
Any mix without a class per mix — 4 concerns need 4 classes, not 16.	**Deeper stack traces** — the real work is buried under every layer.
Concerns tested in isolation — logging without a database, caching without a logger (Single Responsibility).	**Identity surprises** — `equals` / `instanceof` on the wrapper don't match the wrapped object.
Assembled at run time — different per caller, environment or feature flag (an on/off switch for a feature).	**Ordering matters for correctness**, and no compiler checks it.
The wrapped class is never edited (Open/Closed) — works even on classes you don't own.	**Over-layering** — eight wrappers deep, nobody knows which one changed a value.

When not to use it

- **Exactly one concern, forever.** Just put it in the class; a decorator for one behaviour is a whole class wrapped around a single `if`.
- **The concern needs private state.** A decorator can only use the public interface, so it is the wrong tool.
- **The framework already offers it** — middleware and interceptors (code a framework runs around every request or call), Spring AOP aspects (aspect-oriented programming: extra code applied automatically around chosen methods) are decorators with tooling. Don't hand-write ten wrappers beside them.
- **The stack would get deep.** Every extra layer is another frame in each stack trace, and with many layers "which one changed this value?" becomes a long debugging session.

Common mistakes

- Holding the **concrete class** instead of the interface (`private DatabaseUserRepository inner`) — then decorators cannot stack.
- **Forgetting to forward** some methods (use an abstract base decorator).
- A caching decorator that **doesn't invalidate on writes** (ours does in `rename`).
- Putting authorization **inside** a cache.
- Relying on `instanceof` of the outer object.

Real-world examples

- `java.io` **streams** — see above.
- `java.util.Collections.unmodifiableList/synchronizedList/checkedList` — each returns a wrapper with the same interface and one changed behaviour (read-only, synchronized, type-checked). Some authors call `unmodifiableList` a protection **proxy**; both labels are defensible.
- `javax.servlet.http.HttpServletRequestWrapper` (Jakarta: `jakarta.servlet...`) — a base decorator you subclass to change selected request methods in a filter (code that runs before and after each web request in a Java servlet — i.e. web — application).
- **HTTP middleware** (HTTP is the web's request/response protocol) — the web frameworks Express, Django, ASP.NET and Go's `http.Handler` wrap a handler in layers (logging, auth, compression, rate limiting — capping how many requests a client may make per time window). Middleware is this pattern with framework support.
- **Resilience4j (Java) and Polly (.NET)** — compose retry, circuit breaker (stops calling a failing service for a while), timeout and bulkhead (separate, limited resources so one slow dependency cannot use them all) as stacked wrappers, e.g. `Decorators.ofSupplier(...).withCircuitBreaker(...).withRetry(...).withBulkhead(...).decorate()`; both document the ordering carefully because it changes behaviour.
- **Python's** `@functools.lru_cache` (and `@retry` from libraries such as tenacity) — function decorators (LRU = least recently used: a full cache drops the entry that has gone unused longest): the same idea applied to functions, and the reason Python uses the name.

In modern languages

- **Java**: abstract forwarding base class (as here), or a dynamic proxy (a class generated at run time; see 9.7) that generates the forwarding.
- **Python**: function decorators (`@lru_cache` and friends) are this pattern with syntax; for objects, `__getattr__` forwarding (and `functools.wraps` for functions) avoids hand-writing every delegating method.
- **JavaScript / TypeScript**: a higher-order function (one that takes or returns functions) that returns a wrapped function; a `Proxy` object can forward every method automatically.
- **Go**: a struct (Go's plain record-like type) that embeds the interface value gets delegation for free — why Go middleware reads so cleanly.
- **C#**: `DelegatingHandler` in `HttpClient` is a framework-provided decorator chain.

In the interview

> **30-second answer:** "Decorator adds responsibilities to an object at run time by wrapping it in another object with the same interface. Without it, four optional concerns mean sixteen subclasses; with it, each concern — logging, caching, retries — is one class and I compose the stack I need. The decorator holds the interface, not the concrete class, which is what lets layers stack. And order matters: retry goes closest to what fails, and authorization must sit outside a cache or a cache hit skips the check."

Follow-ups

- *Q: Decorator vs Proxy?* — A: Same structure; different intent. Decorator **adds behaviour** and normally forwards the call; the client hands it the object to wrap and often stacks several. Proxy **controls access** — it may defer, cache or refuse — often creates or locates the real object itself, and is usually handed to a client that doesn't know it isn't the real thing. The boundary is genuinely fuzzy (a caching wrapper is fairly called either), and saying so is a better answer than pretending it is sharp.
- *Q: Decorator vs Adapter?* — A: Decorator keeps the interface and changes behaviour; Adapter changes the interface and keeps behaviour.
- *Q: Decorator vs Chain of Responsibility?* — A: Both are chains of objects. In Decorator every layer normally does its extra work and passes the call on to the core (a caching layer answering a hit is the usual

exception). In Chain of Responsibility, typically one handler handles the request and the rest never see it.

- *Q: Decorator vs Composite?* — A: Decorator wraps exactly one child and adds behaviour; Composite holds many children and aggregates.
- *Q: Give a JDK example.* — A: `new BufferedInputStream(new FileInputStream(f))`; `FilterInputStream` is the base decorator.

Self-check

1. **Q:** How many classes for 5 optional features with decorators vs one subclass per combination? **A:** 5 vs up to 32.
2. **Q:** Why does `RepositoryDecorator` forward every method? **A:** So concrete decorators cannot silently drop methods they don't override.
3. **Q:** Why is "authorization inside cache" a bug? **A:** A cache hit returns data without ever running the authorization check.
4. **Q:** Is `InputStreamReader` a decorator? **A:** No — it changes the interface (bytes to chars), so it is an adapter. `BufferedReader` is the decorator.

TL;DR

> - **Wrap to add behaviour, keep the interface.** IS-A and HAS-A the same type.
> - N features = N classes, stacked at run time, in any order — and **order matters**.
> - Java I/O, `Collections.synchronizedList`, middleware, Resilience4j are all decorators. A decorator normally forwards every call; one that short-circuits (a cache hit) is drifting towards Proxy.

9.5 Facade Pattern

What it is

The **Facade** pattern gives a complicated subsystem **one simple front door**, so that most callers never deal with the machinery behind it. GoF's intent: *provide a unified interface to a set of interfaces in a subsystem; the facade defines a higher-level interface that makes the subsystem easier to use.*

Analogy: a hotel concierge. You say "book me dinner and a taxi"; the concierge calls the restaurant, the taxi company and the front desk in the right order. You can still phone the restaurant yourself if you want something unusual.

Term	Meaning	In our code
Facade	Knows which subsystem classes to call, in what order, and how to clean up	`OrderFacade`
Subsystem classes	Do the real work; unaware of the facade	`InventoryService`, `PaymentService`, `ShippingService`, `EmailService`
Client	Uses the facade instead of the subsystem	`Main` / a checkout page

The problem it solves

One useful action needs several objects, called in the right order. To place an order you must **reserve stock → charge the card → create a shipment → send a confirmation**. Every place in the code that places an order repeats those four calls — and each copy must also know what to do if the card is declined **after** the stock has been reserved.

That recovery step is where bugs live. Copying the success path is easy; copying the undo correctly is not. A caller that forgets to release the reservation leaves units locked for an order that was never placed.

Nothing is wrong with the subsystem itself — it is **flexible**, and flexibility means many knobs. The trouble is that nearly every caller wants the same standard sequence and writes it out yet again.

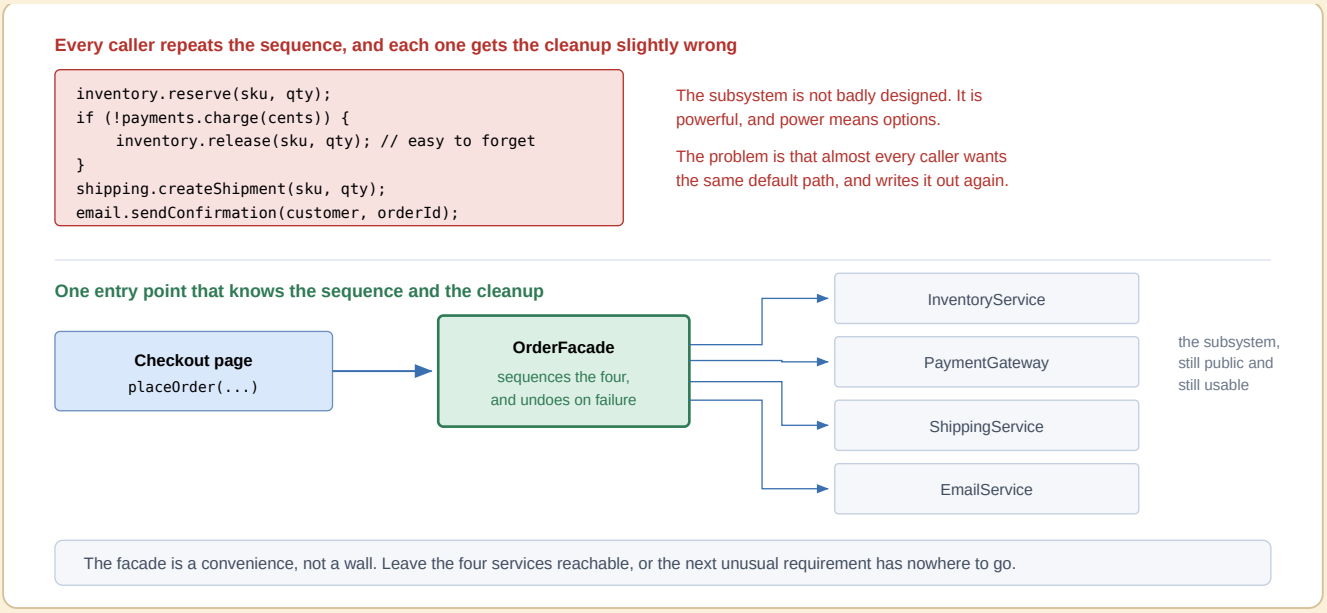


Figure — Top: each caller repeats the reserve/charge/release/ship/email sequence and may forget the release. Bottom: the checkout page calls `OrderFacade.placeOrder(...)`, which sequences the four services and undoes on failure; the services stay public (course diagram)

How it works

1. **Identify the common path** that many callers repeat.
2. **Create a facade class** that holds the subsystem objects.
3. **Expose one intention-revealing method** (`placeOrder`) that calls them in the correct order.
4. **Own the failure handling** — if payment fails, the facade releases the reservation (a *compensating action*).
5. **Keep the subsystem public**, so unusual callers can still reach past the facade.
6. **Add no business rules** — the facade *sequences*, it does not *decide*.

Structure

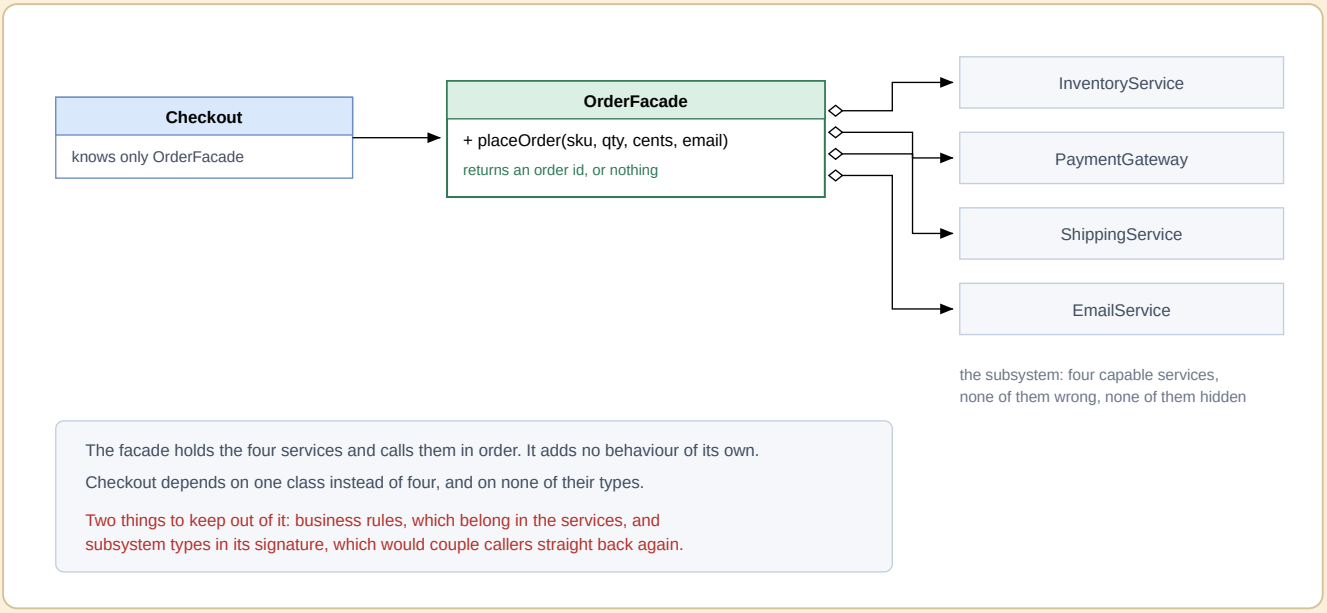
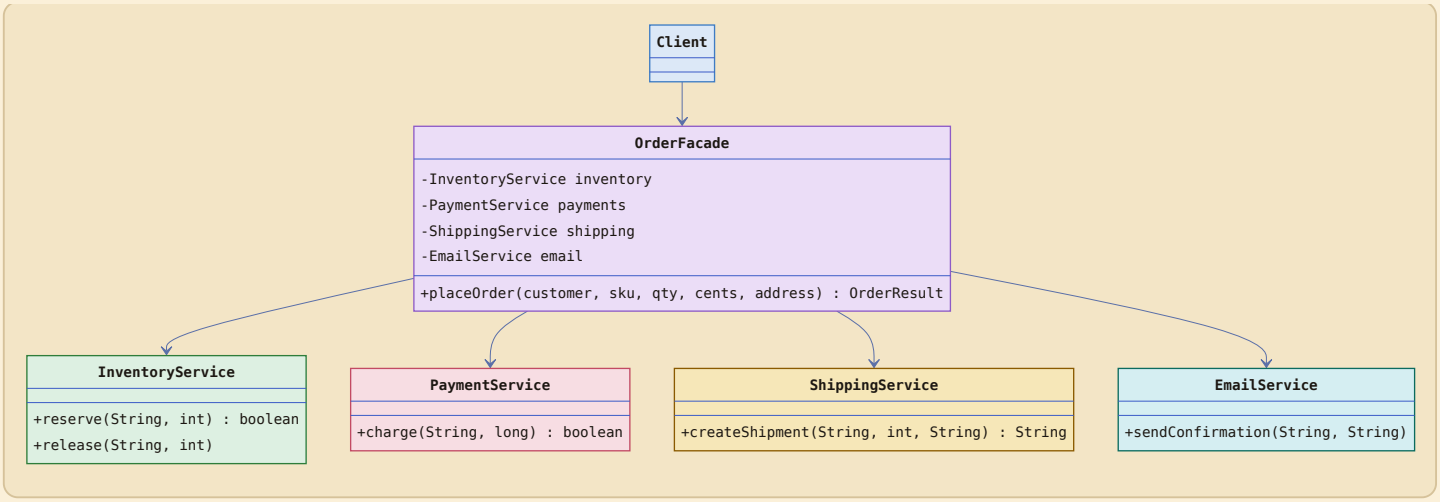
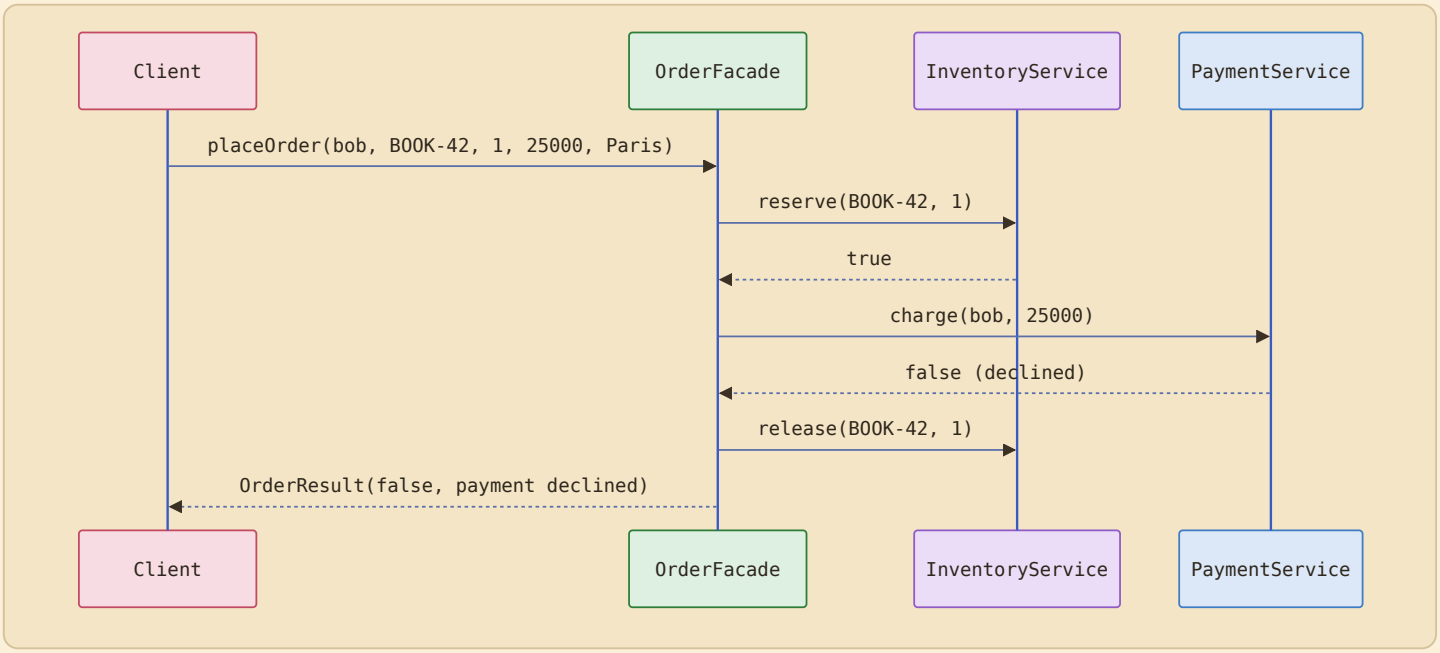


Figure — Facade class diagram: `Checkout` depends only on `OrderFacade`, which holds the four services (shown in grey) and exposes `placeOrder`. The figure labels the payment service `PaymentGateway` and shows a shorter `placeOrder` signature; our code calls it `PaymentService` and also passes customer and address (course diagram)

In the class diagram above, dependencies run in **one direction** only: none of the services knows the facade exists, so they remain usable on their own. Note also that there is no interface: a facade is normally a concrete convenience class, not an abstraction. Introduce an interface only if you plan to substitute the entire subsystem — which is a different job.





Java example

```java
// File: code/lld/ch09/facade/InventoryService.java
package lld.ch09.facade;

import java.util.HashMap;
import java.util.Map;

/*
 * SUBSYSTEM: four independent, still-public services.
 * None of them knows the facade exists.
 */

final class InventoryService {
    private final Map<String, Integer> stock = new HashMap<>(Map.of("BOOK-42", 5));

    boolean reserve(String sku, int qty) {
        int have = stock.getOrDefault(sku, 0);
        if (have < qty) return false;
        stock.put(sku, have - qty);
        System.out.println("  [inventory] reserved " + qty + " x " + sku + " (left " + (have - qty) + ")");
        return true;
    }

    void release(String sku, int qty) {
        stock.merge(sku, qty, Integer::sum);
        System.out.println("  [inventory] released " + qty + " x " + sku + " (left " + stock.get(sku) + ")");
    }
}
```

```java
// File: code/lld/ch09/facade/PaymentService.java
package lld.ch09.facade;

final class PaymentService {
    /** Declines anything over $100 so the demo can show the failure path. */
    boolean charge(String customer, long cents) {
        boolean ok = cents <= 10_000;
        System.out.println("  [payment]   charge " + customer + " " + cents + "c -> " + (ok ? "OK" : "DECLINED"));
        return ok;
    }
}
```

```java
// File: code/lld/ch09/facade/ShippingService.java
package lld.ch09.facade;

final class ShippingService {
    private int next = 1;

    String createShipment(String sku, int qty, String address) {
        String id = "SHP-" + next++;
        System.out.println("  [shipping]  " + id + ": " + qty + " x " + sku + " to " + address);
        return id;
    }
}
```

```java
// File: code/lld/ch09/facade/EmailService.java
package lld.ch09.facade;

final class EmailService {
    void sendConfirmation(String customer, String orderId) {
        System.out.println("  [email]     confirmation for " + orderId + " sent to " + customer);
    }
}
```

```java
// File: code/lld/ch09/facade/OrderResult.java
package lld.ch09.facade;

/** What callers get back: our own small result type. */
record OrderResult(boolean success, String orderId, String message) { }
```

```java
// File: code/lld/ch09/facade/OrderFacade.java
package lld.ch09.facade;

/**
 * FACADE: one method that knows the correct SEQUENCE and the CLEANUP.
 * It sequences; it does not decide business rules (no pricing, no fraud logic here).
 */
final class OrderFacade {
    private final InventoryService inventory;
    private final PaymentService payments;
    private final ShippingService shipping;
    private final EmailService email;
    private int orderSeq = 100;

    OrderFacade(InventoryService inventory, PaymentService payments,
                ShippingService shipping, EmailService email) {
        this.inventory = inventory;
        this.payments = payments;
        this.shipping = shipping;
        this.email = email;
    }

    OrderResult placeOrder(String customer, String sku, int qty, long cents, String address) {
        String orderId = "ORD-" + orderSeq++;
        if (!inventory.reserve(sku, qty)) {
            return new OrderResult(false, orderId, "out of stock");
        }
        if (!payments.charge(customer, cents)) {
            inventory.release(sku, qty);            // compensating step callers used to forget
            return new OrderResult(false, orderId, "payment declined");
        }
        String shipmentId = shipping.createShipment(sku, qty, address);
        email.sendConfirmation(customer, orderId);
        return new OrderResult(true, orderId, "shipped as " + shipmentId);
    }
}
```

```java
// File: code/lld/ch09/facade/Main.java
package lld.ch09.facade;

public class Main {
    public static void main(String[] args) {
        InventoryService inventory = new InventoryService();
        OrderFacade orders = new OrderFacade(inventory, new PaymentService(),
                new ShippingService(), new EmailService());

        System.out.println("== Happy path: one call instead of four ==");
        System.out.println("  -> " + orders.placeOrder("ada@x.com", "BOOK-42", 2, 5_000, "London"));

        System.out.println("== Declined card: the facade undoes the reservation ==");
        System.out.println("  -> " + orders.placeOrder("bob@x.com", "BOOK-42", 1, 25_000, "Paris"));

        System.out.println("== Out of stock ==");
        System.out.println("  -> " + orders.placeOrder("cy@x.com", "BOOK-42", 99, 1_000, "Rome"));

        System.out.println("== The subsystem is still reachable for unusual needs ==");
        inventory.reserve("BOOK-42", 1);   // e.g. a warehouse tool holding stock manually
    }
}
```

Output:

```
== Happy path: one call instead of four ==
  [inventory] reserved 2 x BOOK-42 (left 3)
  [payment]   charge ada@x.com 5000c -> OK
  [shipping]  SHP-1: 2 x BOOK-42 to London
  [email]     confirmation for ORD-100 sent to ada@x.com
  -> OrderResult[success=true, orderId=ORD-100, message=shipped as SHP-1]
== Declined card: the facade undoes the reservation ==
  [inventory] reserved 1 x BOOK-42 (left 2)
  [payment]   charge bob@x.com 25000c -> DECLINED
  [inventory] released 1 x BOOK-42 (left 3)
  -> OrderResult[success=false, orderId=ORD-101, message=payment declined]
== Out of stock ==
  -> OrderResult[success=false, orderId=ORD-102, message=out of stock]
== The subsystem is still reachable for unusual needs ==
  [inventory] reserved 1 x BOOK-42 (left 2)
```

What each part does

- The four services are ordinary classes with their own responsibilities; none mention `OrderFacade`.
- `OrderFacade.placeOrder` is the whole pattern: the correct **sequence** plus the **compensating step** (`inventory.release`) on a declined payment. Look at the second block of output — reserved, declined, released — without the caller writing any of it.
- `OrderResult` — a small result type so callers don't need subsystem types.
- The last line of `main` calls `inventory.reserve` directly: the facade is **a convenience, not a wall**.

Facade vs Adapter vs Mediator

	Facade	**Adapter**	**Proxy**	**Mediator (Ch. 10–11)**
Intent	Simplify a subsystem	Convert one interface to another	Control access to one object	Coordinate peers that would otherwise talk directly
Objects wrapped	**Several**	**One**	**One**	Several colleagues
Interface offered	New, smaller one	The client's (Target)	**Same** as the real object	Mediator's own
Who chose the interface	**You** — a simpler one over things you usually control	**Someone else** — you must fit it	Already fixed by the real object	You
Do the wrapped objects know about it?	**No** — one-way	No	No	**Yes** — colleagues talk *through* the mediator
Direction of communication	Client → facade → subsystem	Client → adapter → adaptee	Client → proxy → real subject	Colleague ↔ mediator ↔ colleague (both ways)
Example	`OrderFacade.placeOrder`	`StripeGateway.pay`	`LazyReport`	An air-traffic control tower; a chat room

The quickest way to spot a facade is the **"objects wrapped"** row: Adapter, Decorator and Proxy each stand in front of exactly one object; a Facade stands in front of **several**.

Another test: suppose the subsystem changed its API tomorrow to exactly what callers want. An **Adapter** would then be pointless; a **Facade** would still be useful, because its value was the *simplification*, not a translation.

Facade vs "service layer". (A service layer is the set of classes that expose the app's use cases — such as placing an order — to its UI or API.) Most of the time these are the same thing under two names. A service class that only *sequences* calls is a facade; once it holds business rules it has become something else (a domain/application service with logic of its own).

The failure mode to watch for

Facades **pull in features**. Day one it is four calls in order; then a discount rule lands in it ("orders are placed here, after all"), then a fraud check, and a year later it is the **god object** (one class that knows and does almost everything) everyone is afraid to edit. Keep one rule: **a facade orchestrates; it does not decide.**

If it is making a business decision instead of calling the thing that makes it, that logic belongs elsewhere — and moving it out later is far harder than keeping it out now.

This connects to the **Principle of Least Knowledge** (Law of Demeter) that *Head First Design Patterns* pairs with Facade: "talk only to your immediate friends". Clients talk to the facade, not to a chain of subsystem objects, which reduces coupling.

Trade-offs

Pros	Cons
One call replaces a multi-step sequence, so the common path is no longer duplicated.	**One more class to maintain** as the subsystem evolves.
The order and the undo logic live in one place — exactly the part callers got wrong.	**Feature magnet** — grows into a god object unless someone pushes back.
One seam for swapping the subsystem later — only the facade changes.	**Opinionated** — some caller will need an option the facade left out.
Smaller public surface for the module; lower coupling.	**Can mask problems** — a neat front over a subsystem that still fails; everything funnels through it, so it can become a single point of failure (one part whose failure stops everything).

When not to use it

- **The subsystem is already simple** — wrapping two calls just adds a file to maintain.
- **Callers need the full range** — if every caller bypasses the facade for some option, its shape is wrong: widen it on purpose or remove it.
- **You are covering a mess instead of fixing it** — you get a neat front and the same failures behind it; sometimes the honest move is to repair the subsystem.
- **You want it to be the only entrance** — hiding the subsystem forces every unusual need into the facade, which is how god objects are born.

Common mistakes

- Putting business rules (pricing, fraud) in the facade.
- Making the subsystem inaccessible.
- Returning subsystem types from the facade so callers are still coupled to them.
- Forgetting the failure path — a facade that only does the happy path (the path where nothing goes wrong) misses its biggest value.
- Confusing it with Mediator (see table).

Real-world examples

- **Application service layers** — almost every `OrderService` or `RegistrationService` is a facade over repositories, gateways and notifiers (as long as it only sequences).
- **`java.net.URL`** — `url.openStream()` hides protocol handlers, connection setup and content handlers behind one call. (Since Java 20 the `URL` constructors are deprecated (officially discouraged and due for removal); create one with `URI.create(s).toURL()`. The facade idea is unchanged.)
- **SLF4J** `LoggerFactory.getLogger(...)` — hides configuration, appenders (log destinations such as files or the console) and formatters behind one call. (SLF4J literally calls itself a "Simple Logging Facade".)
- **`javax.faces.context.FacesContext`**, **jQuery** (historically the clearest example: one `$` function in front of DOM, events, animation and XHR (the browser's API for sending HTTP requests from JavaScript), each still usable directly), and Python's **`requests`**, which covers the common "just fetch this URL" cases on top of `urllib3`.
- **E-commerce checkout**: when you press "Place order" on Amazon or Flipkart, one backend (server-side) call orchestrates (coordinates) inventory, payment, shipping and notifications — a facade (often called an orchestrator or backend-for-frontend — a server layer built for one particular UI).

In modern languages

- **Python / JavaScript**: a module exporting one function and keeping the rest private is a facade with no class at all.
- **Go**: a package's exported functions are the facade; lowercase identifiers stay internal.

- **Java**: a package with one public class (package-private subsystem), or a JPMS (Java Platform Module System) module that exports only the facade's package.
- **Any language**: a single well-named function is often the whole pattern — if `placeOrder` needs no state of its own, it does not have to be a class.

In the interview

> **30-second answer:** "A Facade gives one simple entry point to a complex subsystem. For order placement, `OrderFacade.placeOrder` reserves stock, charges, ships and emails in the right order, and if payment fails it releases the reservation — so no caller has to remember that cleanup. It wraps several objects, unlike Adapter, Decorator and Proxy which wrap one. I keep the subsystem public, because a facade is a convenience, not a wall, and I keep business rules out of it — the main risk is that it grows into a god object."

Follow-ups

- *Q: How is it different from Adapter?* — A: Adapter converts one interface because someone else chose it; Facade simplifies many objects behind a new interface you chose.
- *Q: And from Mediator?* — A: Subsystem classes don't know the facade exists; colleagues in a Mediator know it and communicate through it, in both directions.
- *Q: Risk?* — A: God object. A facade should sequence, not decide.
- *Q: Should the subsystem be hidden?* — A: Usually no; leave it reachable for unusual needs.
- *Q: Can a system have several facades?* — A: Yes — e.g. one per use case, or layered facades over sub-subsystems.

Self-check

1. **Q:** What is the fastest structural way to tell a Facade from Adapter/Decorator/Proxy? **A:** It wraps several objects; the others wrap one.
2. **Q:** In our code, what does the facade do when payment is declined? **A:** Releases the stock reservation and returns a failed `OrderResult`.
3. **Q:** A teammate adds a 10% discount rule inside `placeOrder`. Good idea? **A:** No — business rules belong in a service the facade calls; the facade only sequences.
4. **Q:** Which OO principle is closely related to Facade? **A:** Principle of Least Knowledge (Law of Demeter).

TL;DR

> - **One door into a complex subsystem.** Names the common path and owns the cleanup.
> - Wraps **several** objects; subsystem stays **public**; facade **sequences, never decides**.
> - Different from Adapter (translation of one) and Mediator (two-way coordination of peers).

9.6 Flyweight Pattern

What it is

The **Flyweight** pattern **shares** the portion of each object's state that is the same across many objects, so that huge numbers of them fit in memory. GoF's intent: *use sharing to support large numbers of fine-grained objects efficiently.*

Analogy: a printed book. Every letter "e" on every page uses the same metal type design (shared), placed at a different position (unique). You don't carve a new "e" for every occurrence.

Two terms carry the whole pattern:

Term	Meaning	In the map example
Intrinsic state	Part of the thing's own identity and **identical for every holder**. Lives inside the shared flyweight.	icon bitmap, colour, size
Extrinsic state	**Varies with the context** the object is used in; different for each holder. Stays outside and is **handed in** as method arguments.	latitude, longitude

The words are GoF's and interviewers use them, even though on their own they explain little — remember "intrinsic = inside the shared object", "extrinsic = supplied from outside".

Role	In our code
Flyweight (shared, immutable, intrinsic only)	`MarkerStyle`
Flyweight factory (owns the pool, returns existing or creates once)	`MarkerStyleFactory`
Context (one per logical object: extrinsic state + reference to a flyweight)	`Marker`
Client (asks the factory, supplies extrinsic state)	`Main`

GoF's own participant list is *Flyweight* (the interface that accepts extrinsic state), *ConcreteFlyweight* (the shared implementation — our `MarkerStyle` plays both roles since we need no interface), *UnsharedConcreteFlyweight* (objects of the same interface that are **not** shared, e.g. a row object holding its own children), *FlyweightFactory* and *Client*. "Context" is the name later sources (such as refactoring.guru) give to the object that stores the extrinsic state.

The problem it solves

Picture a map app showing **100,000 markers**. Every marker stores its latitude and longitude — plus an icon bitmap, a colour and a size. Positions are all different, but the icon is the same for thousands of markers, and the icon is by far the largest field. Keeping 100,000 copies of a 2 MB bitmap is not "a bit slow": it needs ~195 GB, so **the program simply runs out of memory.**

The same situation shows up in a game with a million trees that all use one mesh and texture (a 3D shape and the image painted onto it), in a text editor that would otherwise store font data for every character, and anywhere many objects **differ slightly but share a large common part**.

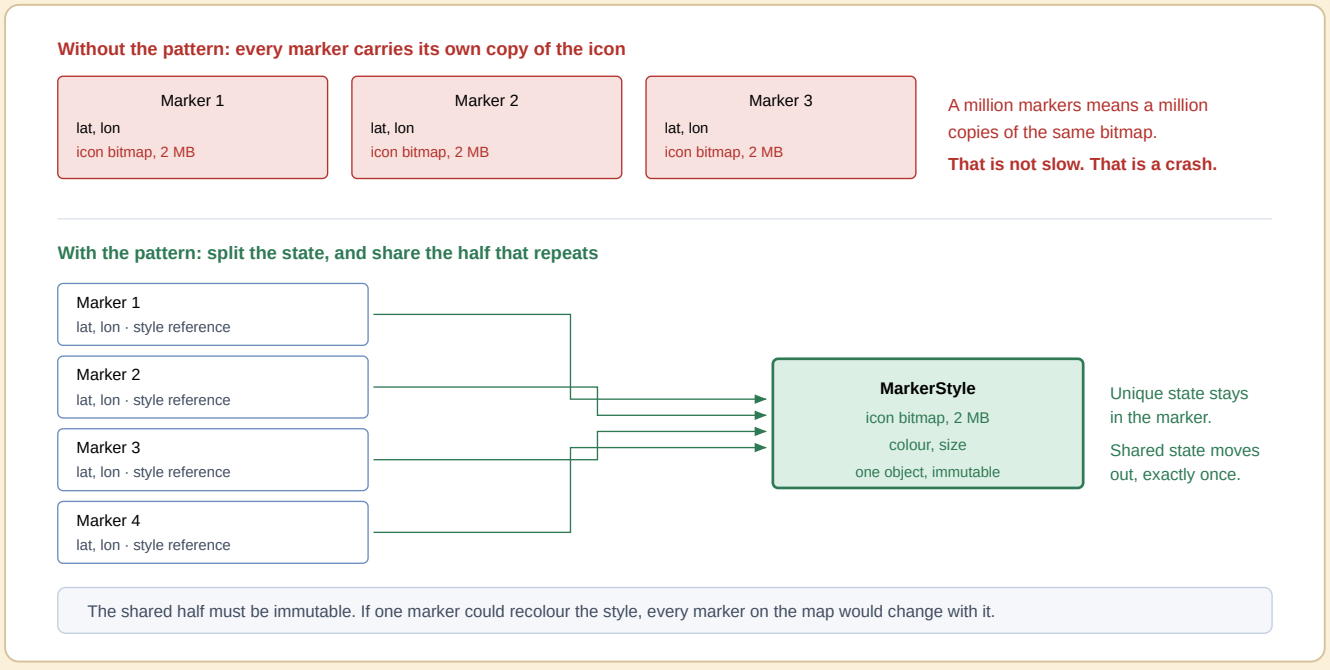


Figure — Top: every marker carries its own 2 MB icon. Bottom: each marker keeps only lat/lon and a style reference; one immutable `MarkerStyle` holds the icon, colour and size (course diagram)

How it works

1. **Split the state**: intrinsic (shared, invariant) vs extrinsic (per holder).
2. **Move intrinsic state into an immutable flyweight class** — all fields `final`, no setters.
3. **Pass extrinsic state as method arguments** (`draw(lat, lon)`) instead of storing it in the flyweight.
4. **Create a factory that owns the pool**, keyed by the intrinsic values. `get(...)` returns the existing object or creates it the first time.
5. **Clients must go through the factory**, never the constructor (enforce with package-private/private constructors).
6. **Context objects** keep only extrinsic state plus a **reference** (not a copy) to the flyweight.

> **Two rules — break either and the pattern silently stops working.**

1. **Flyweights never change.** Many holders point at the same object, so a modification made for one is seen by every other.
2. **Only the factory creates them.** If callers can use the constructor directly, identical copies appear and the memory saving is gone.

Structure

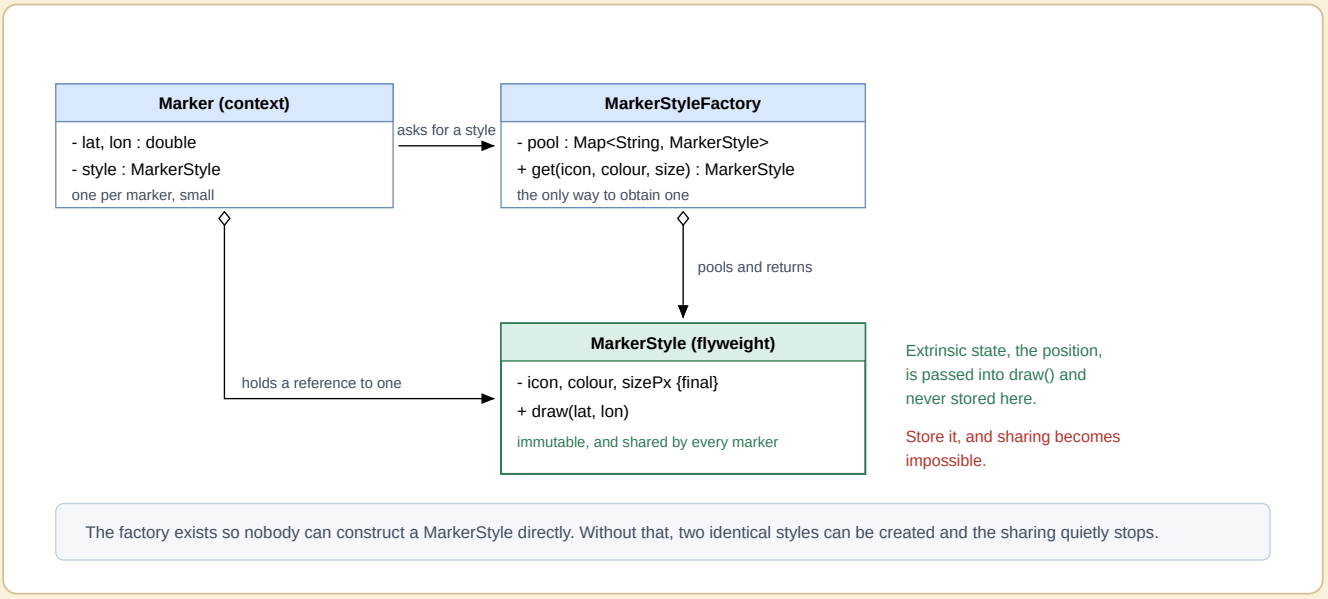
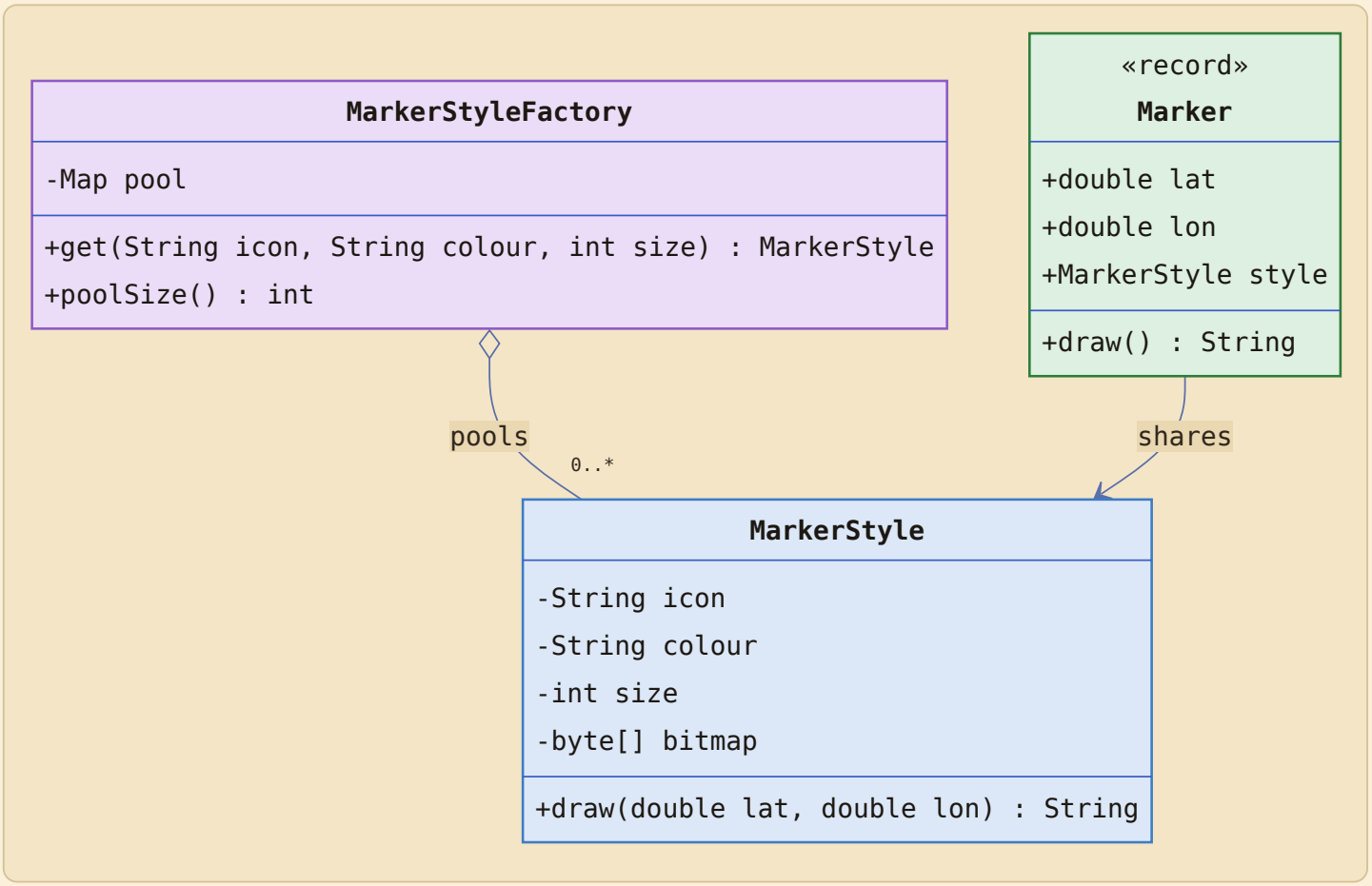


Figure — Flyweight class diagram: `MarkerStyle` (final fields, `draw(lat, lon)`), `MarkerStyleFactory` with a pool and `get()`, and `Marker` holding position plus a style reference (course diagram)

In the diagram above, `draw` receives the position as arguments rather than keeping it in a field, `Marker` holds a *reference* to a style rather than its own copy, and clients obtain styles from the factory, never from the constructor. That last rule has to be enforced — by a package-private (or private) constructor, or at least by convention and code review.



Java example

```java
// File: code/lld/ch09/flyweight/MarkerStyle.java
package lld.ch09.flyweight;

/**
 * FLYWEIGHT: holds only INTRINSIC state (same for every marker using this style).
 * Immutable: all fields final, no setters, and the bitmap array never leaves the object.
 * The constructor is package-private, so code outside this package cannot bypass the
 * factory. (Inside the package it is a convention: only MarkerStyleFactory calls it.)
 */
final class MarkerStyle {
    static final int ICON_BYTES = 2 * 1024 * 1024;   // pretend every icon bitmap is 2 MB

    private final String icon;
    private final String colour;
    private final int size;
    private final byte[] bitmap;   // the heavy part we want exactly one copy of

    MarkerStyle(String icon, String colour, int size) {
        System.out.println("  loading icon '" + icon + "' (" + colour + ", " + size + "px)");
        this.icon = icon;
        this.colour = colour;
        this.size = size;
        this.bitmap = new byte[ICON_BYTES];
    }

    /** EXTRINSIC state (the position) is passed in, never stored. */
    String draw(double lat, double lon) {
        return String.format("%s %s marker at (%.4f, %.4f)", colour, icon, lat, lon);
    }

    int bitmapBytes() { return bitmap.length; }
}
```

```java
// File: code/lld/ch09/flyweight/MarkerStyleFactory.java
package lld.ch09.flyweight;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

/** FLYWEIGHT FACTORY: owns the pool. Thread-safe via ConcurrentHashMap.computeIfAbsent. */
final class MarkerStyleFactory {
    /** The pool key is the combination of intrinsic values. */
    private record Key(String icon, String colour, int size) { }

    private final Map<Key, MarkerStyle> pool = new ConcurrentHashMap<>();

    MarkerStyle get(String icon, String colour, int size) {
        return pool.computeIfAbsent(new Key(icon, colour, size),
                k -> new MarkerStyle(k.icon(), k.colour(), k.size()));
    }

    int poolSize() { return pool.size(); }
}
```

```java
// File: code/lld/ch09/flyweight/Marker.java
package lld.ch09.flyweight;

/** CONTEXT: one per marker. Holds extrinsic state + a REFERENCE to a shared style. */
record Marker(double lat, double lon, MarkerStyle style) {
    String draw() { return style.draw(lat, lon); }
}
```

```java
// File: code/lld/ch09/flyweight/Main.java
package lld.ch09.flyweight;

import java.util.ArrayList;
import java.util.List;
import java.util.Random;

public class Main {
    public static void main(String[] args) {
        MarkerStyleFactory styles = new MarkerStyleFactory();

        System.out.println("== Five markers, three styles ==");
        List<Marker> few = List.of(
                new Marker(51.5074, -0.1278, styles.get("cafe", "brown", 24)),
                new Marker(48.8566, 2.3522, styles.get("cafe", "brown", 24)),
                new Marker(40.7128, -74.0060, styles.get("fuel", "green", 24)),
                new Marker(35.6762, 139.6503, styles.get("hospital", "red", 32)),
                new Marker(52.5200, 13.4050, styles.get("fuel", "green", 24)));
        few.forEach(m -> System.out.println("  " + m.draw()));
        System.out.println("  same shared object? " + (few.get(0).style() == few.get(1).style()));

        System.out.println("== Scale to 100,000 markers ==");
        String[] icons = {"cafe", "fuel", "hospital"};
        String[] colours = {"brown", "green", "red"};
        int[] sizes = {24, 24, 32};
        Random rnd = new Random(42);
        List<Marker> many = new ArrayList<>();
        for (int i = 0; i < 100_000; i++) {
            int t = rnd.nextInt(3);
            many.add(new Marker(rnd.nextDouble() * 180 - 90, rnd.nextDouble() * 360 - 180,
                    styles.get(icons[t], colours[t], sizes[t])));
        }
        long withoutMb = (long) many.size() * MarkerStyle.ICON_BYTES / (1024 * 1024);
        long withMb = (long) styles.poolSize() * MarkerStyle.ICON_BYTES / (1024 * 1024);
        System.out.println("  markers: " + many.size() + ", styles in pool: " + styles.poolSize());
        System.out.println("  icon memory without flyweight: ~" + withoutMb + " MB");
        System.out.println("  icon memory with flyweight:    ~" + withMb + " MB");

        System.out.println("== Flyweights already in the JDK ==");
        Integer a = 127, b = 127;          // autoboxing calls Integer.valueOf
        Integer c = 128, d = 128;
        System.out.println("  Integer 127 == 127 -> " + (a == b) + "  (cached -128..127)");
        System.out.println("  Integer 128 == 128 -> " + (c == d) + " (outside default cache)");
        System.out.println("  equals() is always right -> " + c.equals(d));
        System.out.println("  Boolean.valueOf(true) == Boolean.TRUE -> " + (Boolean.valueOf(true) == Boolean.TRUE));

        String lit = "hello";
        String built = new String("hello");
        System.out.println("  \"hello\" == \"hel\" + \"lo\" -> " + (lit == "hel" + "lo") + " (compile-time constant,
pooled)");
        System.out.println("  \"hello\" == new String(\"hello\") -> " + (lit == built));
        System.out.println("  \"hello\" == built.intern() -> " + (lit == built.intern()));
    }
}
```

Output:

```
== Five markers, three styles ==
  loading icon 'cafe' (brown, 24px)
  loading icon 'fuel' (green, 24px)
  loading icon 'hospital' (red, 32px)
  brown cafe marker at (51.5074, -0.1278)
  brown cafe marker at (48.8566, 2.3522)
  green fuel marker at (40.7128, -74.0060)
  red hospital marker at (35.6762, 139.6503)
  green fuel marker at (52.5200, 13.4050)
  same shared object? true
== Scale to 100,000 markers ==
  markers: 100000, styles in pool: 3
  icon memory without flyweight: ~200000 MB
  icon memory with flyweight:    ~6 MB
== Flyweights already in the JDK ==
  Integer 127 == 127 -> true  (cached -128..127)
  Integer 128 == 128 -> false (outside default cache)
  equals() is always right -> true
  Boolean.valueOf(true) == Boolean.TRUE -> true
  "hello" == "hel" + "lo" -> true (compile-time constant, pooled)
  "hello" == new String("hello") -> false
  "hello" == built.intern() -> true
```

What each part does

- `MarkerStyle` — the flyweight. Final fields, no setters, package-private constructor, and `draw(lat, lon)` receives extrinsic state.
- `MarkerStyleFactory` — the pool. `ConcurrentHashMap.computeIfAbsent` makes "get or create once" **thread-safe and atomic** (correct when many threads — independent paths of execution running at once — use it, and done as one indivisible step), so two threads can't create two copies of the same style.
- `Key` record — the intrinsic values as a value object (an object defined only by its values); records give correct `equals`/`hashCode` (the methods a hash map uses to find a key) for free.
- `Marker` record — the context: two doubles plus one reference.
- Output: "loading icon" is printed **3 times for 5 markers**, and still only 3 styles for 100,000 markers — about 6 MB of icon data instead of ~200,000 MB (≈195 GB).
- JDK section: `Integer.valueOf` caching and the `String` pool (below).

Flyweights already in Java

- **`Integer.valueOf(int)`** (used by autoboxing — Java's automatic wrapping of an `int` into an `Integer` object) returns cached objects for **−128 to 127** — the Java Language Specification requires that range to be cached. That is why `Integer a = 127, b = 127; a == b` is `true` but the same with 128 is `false` on a default JVM (Java Virtual Machine, the program that runs Java code; the upper bound can be raised with `-XX:AutoBoxCacheMax`, so never rely on it). **Always compare boxed numbers with `equals`.** `Short`, `Byte`, `Long` (same range) and `Character` (0–127) have similar caches; `Boolean.valueOf` returns the two shared constants `Boolean.TRUE` / `Boolean.FALSE`.
- **The String pool**: string literals and compile-time constant expressions (`"hel" + "lo"`) are interned (stored once in a shared pool and reused), so equal literals share one object. `new String("hello")` always creates a new object; `intern()` returns the pooled one.
- **Enums**: exactly one instance per constant, shared everywhere.

Flyweight is not a cache

The two are mixed up all the time — even in code reviews — and the difference decides what you may do with the pool:

	Flyweight	Cache
Purpose	Make the data *fit in memory* at all	Make repeated work *quicker*
Contents	One copy of immutable state that many holders point to	Stored results of costly computations or lookups
May entries be removed?	Not while any holder still points to one	Yes, at any moment
Effect of removing one	Holders lose their shared object, or duplicates get created	None, apart from a slower next request

So an LRU cache in front of a database is not a flyweight, and a pool of shared marker styles is not a cache.

Two mistakes that undo the pattern

- **A flyweight that can change.** One caller updates a colour and every marker on the map changes with it — the classic failure, and hard to debug because the effect shows up far from the cause. Fix: `final` fields, no setters, no leaking of internal arrays.
- **An ever-growing pool keyed by open-ended input.** If users decide the keys, the pool never stops growing — a memory leak (memory that is never released) disguised as an optimisation. Flyweight is safe when the set of intrinsic values is **small and bounded** (3 styles, a few hundred glyphs). An open key space needs weak references (references the garbage collector is allowed to clear) and eviction (removing entries), and at that point you have built a cache — so call it a cache.

Measure first

Flyweight trades readability for memory, so using it without profiling (measuring where a program really spends time or memory) is textbook premature optimisation (optimising before you know it is needed). A thousand objects never need it; a million might. Answer two questions **with numbers** first: *how many of these objects will exist at once*, and *how much of each one is identical across all of them?* If you can't put numbers on both, it is too early; if most of each object is unique, there is nothing worth sharing.

Trade-offs

Pros	Cons
Memory use can fall by orders of magnitude — sometimes the difference between running and not running.	**State is split across two places**, which is harder to follow and change than one class.
Costly construction happens once per distinct value, not once per object.	**The pool keeps strong references (ordinary references, which stop the garbage collector from freeing an object)**, so it can leak memory itself.
Friendlier to CPU caches — the shared data stays hot (recently used, so fast to reach) instead of being scattered.	**Thread safety is on you** — the pool must be thread-safe and the flyweight immutable.
"Load once" logic gets one obvious home instead of being spread across callers.	**Tempting to apply too early**, and callers must pass extrinsic state on every call (some CPU cost).

When not to use it

- You **haven't measured**.
- The shared part **cannot be immutable**.
- The **counts are small**.
- The **unique state is most of the object**.
- The **key space is unbounded** (that's a cache).

Common mistakes

- Storing extrinsic state inside the flyweight (then it can't be shared).
- Public constructor so callers bypass the factory.
- Non-thread-safe pool (`HashMap` + check-then-put from many threads → duplicates or worse).
- Comparing boxed `Integer`s or `String`s with `==` because "it worked in the test" with small values or literals.

Other patterns it is confused with

Looks like	Actual difference
Cache	See table above.
Singleton	Singleton = exactly one instance of a class. Flyweight = one instance **per distinct intrinsic value** (3, or 300). A flyweight pool that happens to hold one entry only *looks* like a Singleton.
Object pool	An object pool **lends out mutable** objects and takes them back (e.g. DB connections). Flyweights are **immutable**, shared, never returned.
Prototype	Prototype makes many objects by copying; Flyweight avoids making them at all — opposite answers to the same "many similar objects" observation.

Real-world examples

- `Integer.valueOf`, `Boolean.valueOf`, `String.intern()`, **enums** — see above. (Python also interns small integers and many short strings automatically.)
- **Text rendering / word processors** — one glyph object per character in a font, shared by every occurrence on the page (the classic GoF example).
- **Game engines and map renderers** — meshes, textures and sprite atlases are loaded once and referenced by every entity (e.g. a forest of identical tree models, rendered with GPU *instancing* — the graphics card drawing one model many times in a single pass).
- **Map apps** — thousands of pins typically share a handful of icon styles (the example in this section).
- `java.time.ZoneOffset` caches offsets that are whole multiples of 15 minutes, so `ZoneOffset.ofHours(5)` returns a shared instance.

In modern languages

- **Java**: `Integer.valueOf`, `String.intern` and enums are built-in flyweights; a `record` is an immutable flyweight in one line, and a `Map` plus `computeIfAbsent` is the whole factory (as in our code).
- **Python**: small ints and many short strings are interned automatically; `functools.lru_cache` on a factory function gives a pool with a size bound built in.
- **JavaScript**: `Object.freeze` on the shared object plus a `Map` for the pool — freezing makes immutability enforced rather than a convention.
- **C++**: `std::shared_ptr<const Style>` expresses both the sharing and the immutability in the type.
- **Any language with value types**: an immutable value object plus a memoising (remembers results it already produced) factory is this pattern, usually unnamed.

In the interview

> **30-second answer:** "Flyweight saves memory when there are huge numbers of similar objects. I split state into intrinsic — shared and invariant, like an icon — and extrinsic — per object, like a position — which is passed in to methods. The intrinsic part lives in an immutable flyweight handed out by a factory that pools one instance per distinct value. `Integer.valueOf` and the String pool are JDK examples. I'd only do it after measuring."

Follow-ups

- *Q: Why must the flyweight be immutable?* — A: It is shared; mutating it changes every holder.
- *Q: How do you make the factory thread-safe?* — A: `ConcurrentHashMap.computeIfAbsent`, which runs the creation function at most once per key.
- *Q: Flyweight vs cache?* — A: A cache may evict anything and exists for speed; a flyweight exists because otherwise the program doesn't fit, and entries can't be dropped while referenced.
- *Q: Why is `Integer 128 == 128` false?* — A: Autoboxing uses `Integer.valueOf`, which only guarantees caching for −128..127; outside that range you get distinct objects on a default JVM. Use `equals`.
- *Q: Flyweight vs Singleton?* — A: One instance per distinct value vs one instance total.

Self-check

1. **Q:** In a chess game with a million saved positions, what is intrinsic and extrinsic for a piece? **A:** Intrinsic: type and colour (and image). Extrinsic: the square it is on.

2. **Q:** What goes wrong if `MarkerStyle` had `setColour`? **A:** Changing one marker's colour would change every marker sharing that style.
3. **Q:** Is `"a" + "b" == "ab"` true in Java? **A:** Yes — it is a compile-time constant expression, so it is interned. With a non-final variable in the expression, it would be a new object.
4. **Q:** When does a flyweight pool become a leak? **A:** When its key space is unbounded (e.g. keyed by user input) and it never evicts.

TL;DR

> - **Share what repeats, pass in what varies.** Intrinsic inside, extrinsic outside.
> - **Immutable** flyweight + **factory-owned** pool (thread-safe).
> - JDK: `Integer.valueOf` (−128..127), String pool, enums. **Measure first**; not a cache.

9.7 Proxy Pattern

What it is

The **Proxy** pattern places a **stand-in with the identical interface** in front of a real object in order to control **who** may use it, **when** it gets created, or **how much** each use costs. GoF's intent: *provide a surrogate or placeholder for another object to control access to it.* (GoF also lists the name **Surrogate**.)

Analogy: a credit card is a proxy for your bank account. Shops accept it wherever cash is accepted (same "interface"), but it adds control — limits, fraud checks — and the money only moves when needed.

Term	Meaning	In our code
Subject	The interface shared by the proxy and the real object	`Report`, `RateService`, `Device`
Real Subject	The object that does the real work	`RealReport`, `LiveRateService`, `RealDevice`
Proxy	Holds (or creates) the real subject, same interface, decides what happens before/instead of/around the call	`LazyReport`, `AuditedReport`, `CachingRateProxy`, `RemoteRateProxy`, the `SharedDevice` handle

The problem it solves

There are times when callers should **not touch the real object directly — or not yet**:

- **Building it is costly and usually unnecessary.** Loading a 100 MB report during start-up is wasted work on the many runs that never display it — most screens only show its title.
- **Only some callers are permitted** to use it, and the permission logic has no business living inside the object.
- **It runs on a different machine**, and callers should not have to write networking code.
- **Each call must be logged or counted**, but the class belongs to someone else.

Since the proxy offers **the very same interface**, callers can't tell it apart from the real thing, so you can insert one **without editing any caller**. That is also its danger: a slow or remote call now **looks just like a cheap local one**.

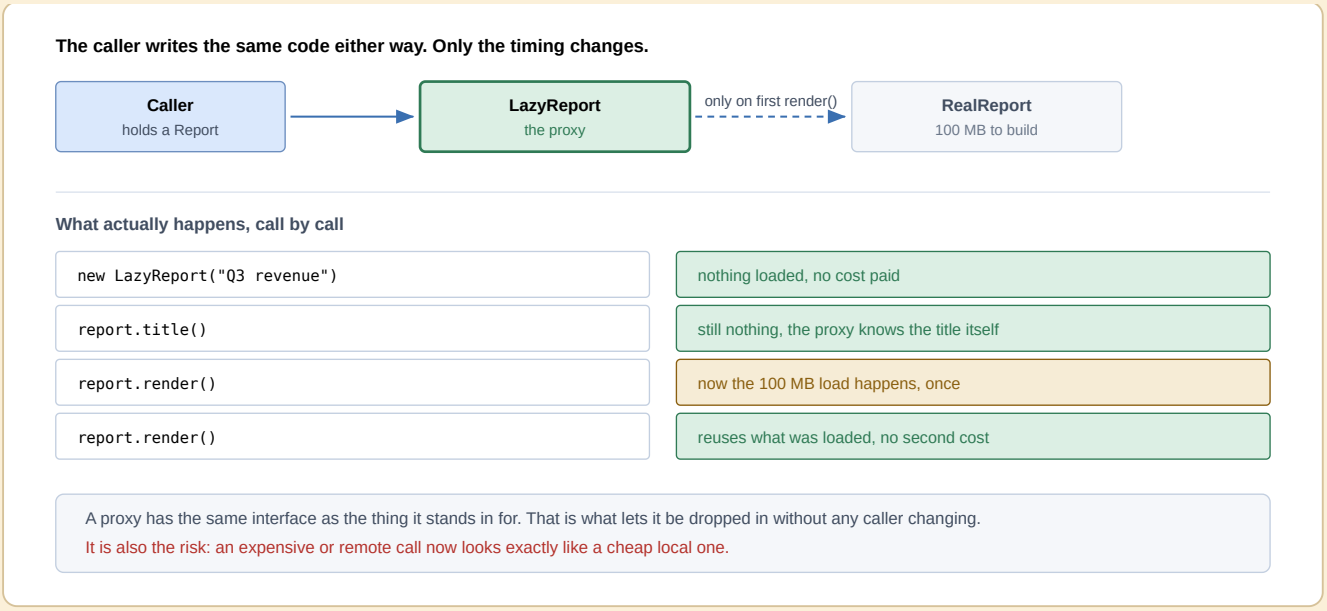


Figure — The caller holds a `Report`. Creating the `LazyReport` and calling `title()` cost nothing; the 100 MB load happens once on the first `render()`, and later renders reuse it (course diagram)

How it works

1. **Define (or reuse) the Subject interface** that callers already use.
2. **Write a proxy class implementing that interface** and holding a reference to the real subject — or the information needed to create it later.
3. In each method, **do the proxy's job first** — create lazily, check permission, look in a cache, marshal (convert into a message that can be sent) over a network, count references — then **delegate** (or refuse).
4. **Give callers the proxy** in place of the real object (often via a factory or DI container so they never see the difference).

Kinds of proxy

Kind	What it controls	Our code / real example
Virtual proxy	*When* the expensive object is created (lazy initialisation)	`LazyReport`; Hibernate (a Java ORM) lazy-loaded entities
Protection proxy	*Who* may call (access control)	`AuditedReport`; `Collections.unmodifiableList`
Remote proxy	*Where* the object lives — hides the network	`RemoteRateProxy` (stub) + `RateServer` (skeleton); RMI/gRPC (remote-call frameworks) stubs
Caching proxy	*Whether* the real call is needed — reuses results	`CachingRateProxy` (bounded LRU + TTL — time to live, how long an entry stays valid)
Logging / monitoring proxy	Records each call before/after forwarding	`LoggingHandler` (dynamic proxy)
Smart reference	Bookkeeping on access — reference counting, locking, loading on first use	`SharedDevice` handles close the device when the last one closes

GoF names four kinds — remote, virtual, protection and smart reference; they differ only in what the proxy does before delegating. Caching and logging proxies are widely used variants described by later sources such as refactoring.guru (GoF would file logging/counting under smart reference).

Structure

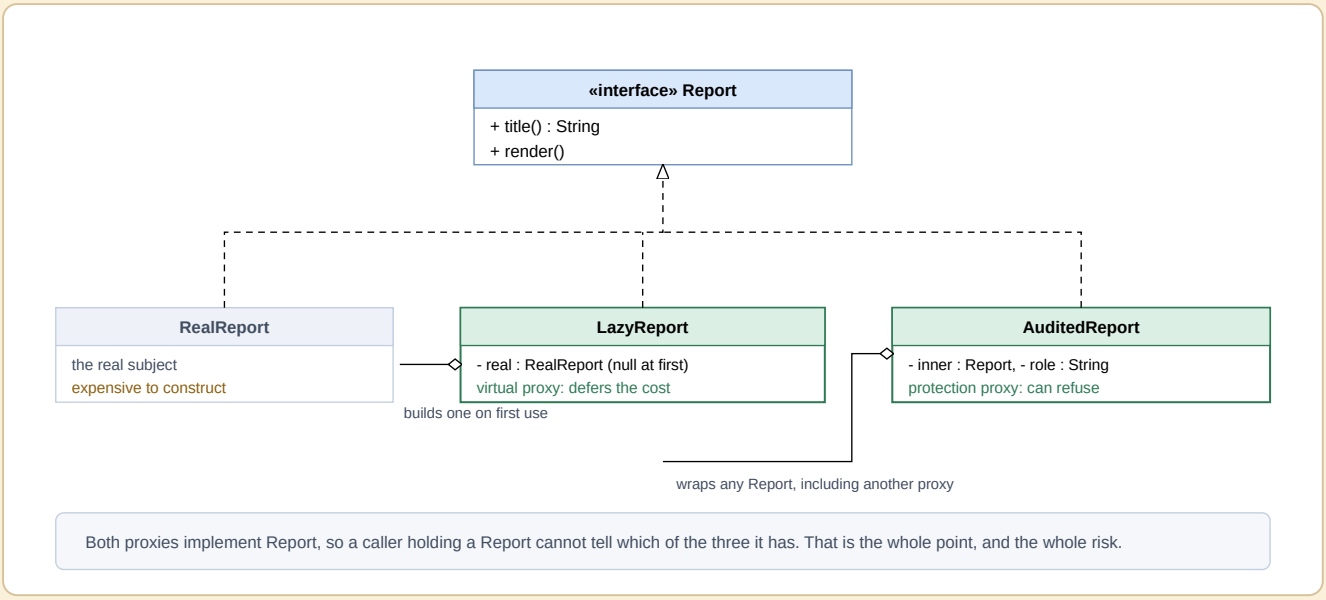
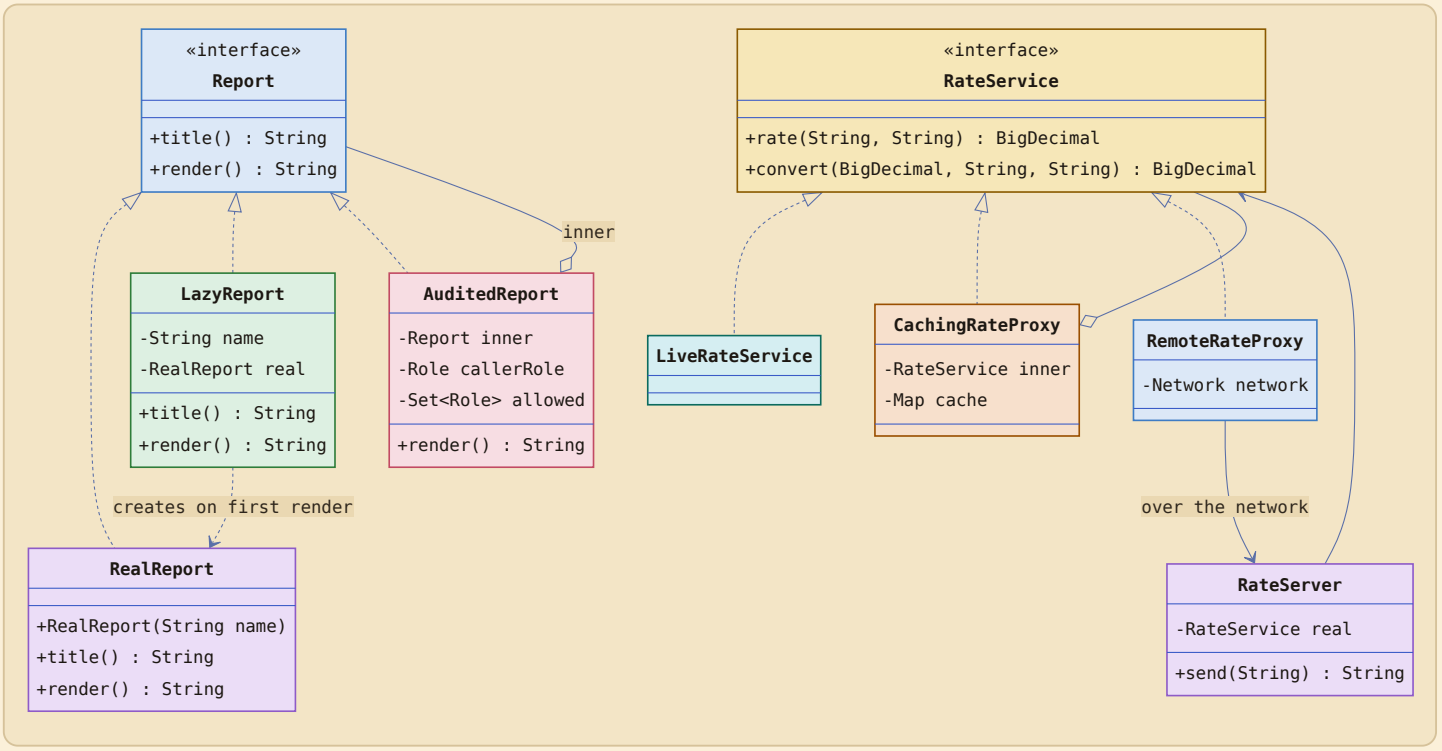
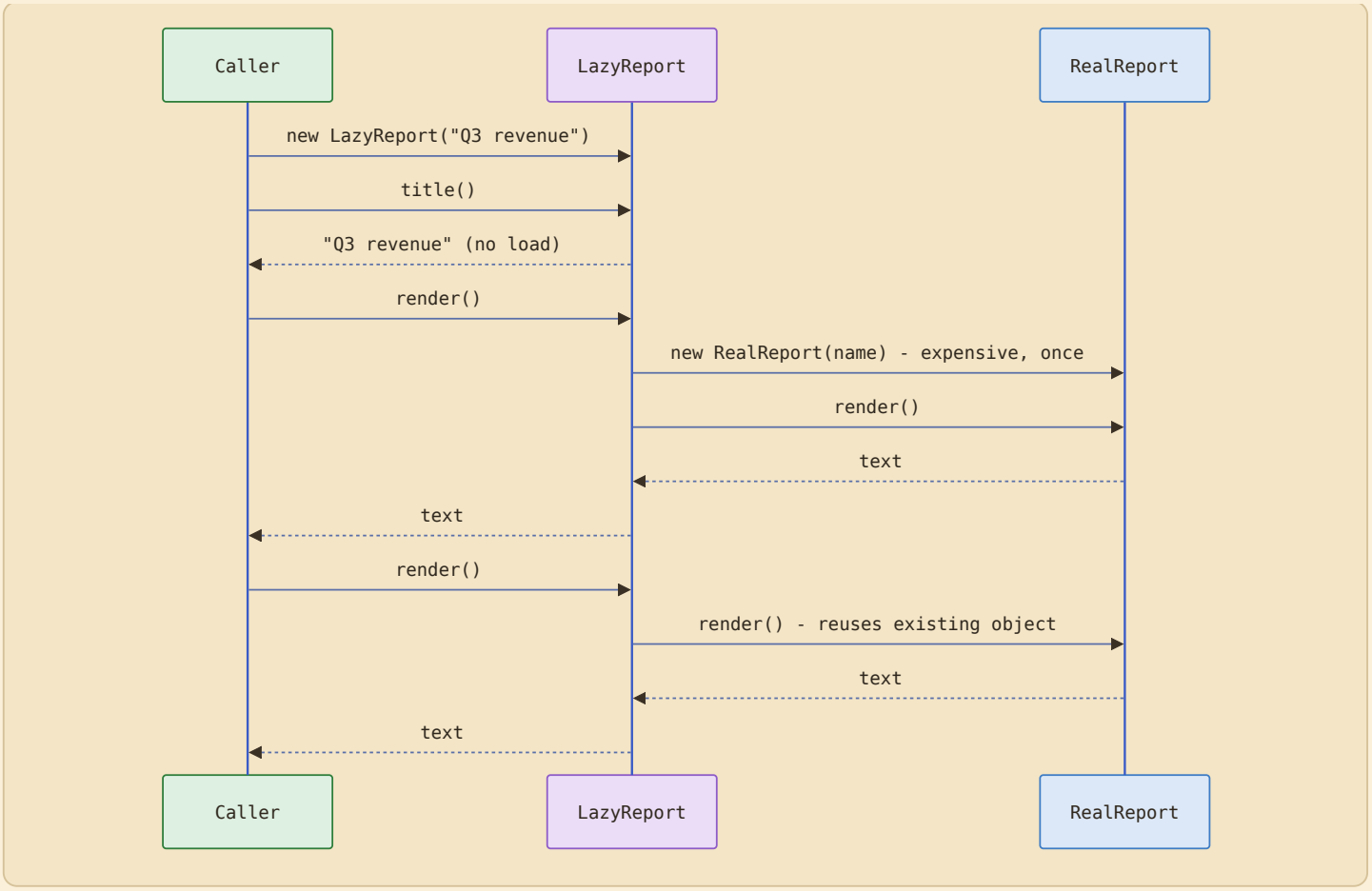


Figure — Proxy class diagram: `RealReport`, `LazyReport` (virtual proxy) and `AuditedReport` (protection proxy) all implement `Report`; `LazyReport` creates the `RealReport` itself, `AuditedReport` wraps any `Report` (course diagram)

In the class diagram above, `RealReport` does the expensive work in its constructor; `LazyReport` keeps only the name, answers `title()` itself and constructs the `RealReport` the first time `render()` is called; `AuditedReport` checks a role before letting a call through. Because `AuditedReport` holds the **`interface`**, it can guard the real report, the lazy proxy or another protection proxy — our demo wraps the lazy one. Strip `LazyReport` down and only one line is special — the null check that triggers construction; the rest simply forwards calls.





Java example

```java
// File: code/lld/ch09/proxy/Report.java
package lld.ch09.proxy;

/** SUBJECT interface: callers only ever hold this type. */
interface Report {
    String title();
    String render();
}
```

```java
// File: code/lld/ch09/proxy/RealReport.java
package lld.ch09.proxy;

/** REAL SUBJECT: expensive constructor. */
final class RealReport implements Report {
    private final String name;
    private final int rows;

    RealReport(String name) {
        System.out.println("    [real] loading 100 MB of data for '" + name + "'...");
        this.name = name;
        this.rows = 1_000_000;
    }

    @Override public String title()  { return name; }
    @Override public String render() { return name + " (" + rows + " rows)"; }
}
```

```java
// File: code/lld/ch09/proxy/LazyReport.java
package lld.ch09.proxy;

/**
 * VIRTUAL PROXY: answers title() itself, builds the RealReport on first render().
 * Thread-safe lazy init with double-checked locking (field must be volatile).
 */
final class LazyReport implements Report {
    private final String name;
    private volatile RealReport real;      // created by the proxy itself, not handed in

    LazyReport(String name) { this.name = name; }

    @Override public String title() { return name; }   // no load needed

    @Override
    public String render() { return realReport().render(); }

    private RealReport realReport() {
        RealReport r = real;
        if (r == null) {
            synchronized (this) {
                r = real;
                if (r == null) real = r = new RealReport(name);
            }
        }
        return r;
    }
}
```

```java
// File: code/lld/ch09/proxy/Role.java
package lld.ch09.proxy;

enum Role { VIEWER, FINANCE, ADMIN }
```

```java
// File: code/lld/ch09/proxy/AuditedReport.java
package lld.ch09.proxy;

import java.util.Set;

/** PROTECTION PROXY: checks permission, may REFUSE the call. */
final class AuditedReport implements Report {
    private final Report inner;
    private final Role callerRole;
    private final Set<Role> allowed;

    AuditedReport(Report inner, Role callerRole, Set<Role> allowed) {
        this.inner = inner;
        this.callerRole = callerRole;
        this.allowed = Set.copyOf(allowed);
    }

    @Override public String title() { return inner.title(); }  // titles are public

    @Override
    public String render() {
        if (!allowed.contains(callerRole)) {
            throw new SecurityException(callerRole + " may not render " + inner.title());
        }
        return inner.render();
    }
}
```

```java
// File: code/lld/ch09/proxy/RateService.java
package lld.ch09.proxy;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** Subject for the caching, remote and dynamic-proxy demos. Money and rates are BigDecimal, never double. */
interface RateService {
    BigDecimal rate(String from, String to);
    BigDecimal convert(BigDecimal amount, String from, String to);

    /** Shared rounding rule: amounts are returned to 2 decimal places. */
    static BigDecimal toMoney(BigDecimal value) { return value.setScale(2, RoundingMode.HALF_EVEN); }
}
```

```java
// File: code/lld/ch09/proxy/LiveRateService.java
package lld.ch09.proxy;

import java.math.BigDecimal;
import java.math.MathContext;
import java.util.Map;

/** Real subject: pretend each call is a slow lookup. */
class LiveRateService implements RateService {
    private static final Map<String, BigDecimal> PER_USD = Map.of(
            "USD", new BigDecimal("1"), "EUR", new BigDecimal("0.92"), "INR", new BigDecimal("83"));
    int lookups = 0;

    @Override
    public BigDecimal rate(String from, String to) {
        lookups++;
        return PER_USD.get(to).divide(PER_USD.get(from), MathContext.DECIMAL64);
    }

    @Override
    public BigDecimal convert(BigDecimal amount, String from, String to) {
        return RateService.toMoney(amount.multiply(rate(from, to)));   // self-call: goes to THIS object, not through any
proxy
    }
}
```

```java
// File: code/lld/ch09/proxy/CachingRateProxy.java
package lld.ch09.proxy;

import java.math.BigDecimal;
import java.util.LinkedHashMap;
import java.util.Map;

/** CACHING PROXY: bounded (LRU) and time-limited, because an unbounded cache is a leak. */
final class CachingRateProxy implements RateService {
    private record CachedRate(BigDecimal value, long expiresAt) { }

    private final RateService inner;
    private final long ttlMillis;
    private final Map<String, CachedRate> cache;

    CachingRateProxy(RateService inner, int maxEntries, long ttlMillis) {
        this.inner = inner;
        this.ttlMillis = ttlMillis;
        this.cache = new LinkedHashMap<>(16, 0.75f, true) {   // access order = LRU
            @Override
            protected boolean removeEldestEntry(Map.Entry<String, CachedRate> eldest) {
                return size() > maxEntries;
            }
        };
    }

    @Override
    public synchronized BigDecimal rate(String from, String to) {
        String key = from + "->" + to;
        CachedRate e = cache.get(key);
        long now = System.currentTimeMillis();
        if (e != null && e.expiresAt() > now) return e.value();
        BigDecimal v = inner.rate(from, to);
        cache.put(key, new CachedRate(v, now + ttlMillis));
        return v;
    }

    @Override
    public BigDecimal convert(BigDecimal amount, String from, String to) {
        return RateService.toMoney(amount.multiply(rate(from, to)));
    }
}
```

```java
// File: code/lld/ch09/proxy/Network.java
package lld.ch09.proxy;

/*
 * REMOTE PROXY: looks like a local RateService, but marshals each call into a
 * message and sends it across a "network". Here the network is simulated in-process.
 */
interface Network {
    String send(String request);
}
```

```java
// File: code/lld/ch09/proxy/RateServer.java
package lld.ch09.proxy;

import java.math.BigDecimal;

/** Server side "skeleton": unmarshals the request and calls the real object. */
final class RateServer implements Network {
    private final RateService real;

    RateServer(RateService real) { this.real = real; }

    @Override
    public String send(String request) {                  // format: "rate|FROM|TO"
        String[] p = request.split("\\|");
        return real.rate(p[1], p[2]).toPlainString();       // BigDecimal -> exact text on the wire
    }
}
```

```java
// File: code/lld/ch09/proxy/RemoteRateProxy.java
package lld.ch09.proxy;

import java.math.BigDecimal;

/** Client side "stub" = the remote proxy. */
final class RemoteRateProxy implements RateService {
    private final Network network;

    RemoteRateProxy(Network network) { this.network = network; }

    @Override
    public BigDecimal rate(String from, String to) {
        String request = "rate|" + from + "|" + to;
        System.out.println("    [stub] sending '" + request + "' over the wire");
        return new BigDecimal(network.send(request));
    }

    @Override
    public BigDecimal convert(BigDecimal amount, String from, String to) {
        return RateService.toMoney(amount.multiply(rate(from, to)));
    }
}
```

```java
// File: code/lld/ch09/proxy/Device.java
package lld.ch09.proxy;

/** A heavyweight resource, e.g. a connection to a device. */
interface Device extends AutoCloseable {
    String query(String command);
    @Override void close();
}
```

```java
// File: code/lld/ch09/proxy/RealDevice.java
package lld.ch09.proxy;

final class RealDevice implements Device {
    RealDevice() { System.out.println("    [device] opened (expensive)"); }
    @Override public String query(String command) { return "ok:" + command; }
    @Override public void close() { System.out.println("    [device] closed"); }
}
```

```java
// File: code/lld/ch09/proxy/SharedDevice.java
package lld.ch09.proxy;

/**
 * SMART REFERENCE PROXY: reference counting. Each client gets its own handle (a proxy).
 * The real device opens on the first handle and closes when the LAST handle closes.
 * Thread-safe: every read or write of 'refs', 'real' and a handle's 'closed' flag
 * happens while holding the SharedDevice's lock, so the count can never go wrong.
 */
final class SharedDevice {
    private int refs;              // guarded by 'this'
    private RealDevice real;       // guarded by 'this'

    synchronized Device acquire() {
        if (refs++ == 0) real = new RealDevice();
        System.out.println("    [ref] handle acquired, count=" + refs);
        return new Handle();
    }

    private final class Handle implements Device {
        private boolean closed;    // guarded by SharedDevice.this

        @Override
        public String query(String command) {
            synchronized (SharedDevice.this) {
                if (closed) throw new IllegalStateException("handle already closed");
                return real.query(command);
            }
        }

        @Override
        public void close() {
            synchronized (SharedDevice.this) {
                if (closed) return;          // closing a handle twice must not double-decrement
                closed = true;
                refs--;
                System.out.println("    [ref] handle released, count=" + refs);
                if (refs == 0) {
                    real.close();
                    real = null;
                }
            }
        }
    }
}
```

```java
// File: code/lld/ch09/proxy/LoggingHandler.java
package lld.ch09.proxy;

import java.lang.reflect.InvocationHandler;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.lang.reflect.Proxy;
import java.util.Arrays;

/**
 * DYNAMIC PROXY: one InvocationHandler that logs calls on ANY interface.
 * java.lang.reflect.Proxy generates the implementing class at run time.
 */
final class LoggingHandler implements InvocationHandler {
    private final Object target;

    private LoggingHandler(Object target) { this.target = target; }

    @SuppressWarnings("unchecked")
    static <T> T wrap(T target, Class<T> iface) {
        return (T) Proxy.newProxyInstance(
                iface.getClassLoader(), new Class<?>[] {iface}, new LoggingHandler(target));
    }

    @Override
    public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
        String call = method.getName() + (args == null ? "()" : Arrays.toString(args));
        System.out.println("    [proxy] -> " + call);
        try {
            Object result = method.invoke(target, args);   // forward to the real object
            System.out.println("    [proxy] <- " + call + " returned " + result);
            return result;
        } catch (InvocationTargetException e) {
            throw e.getCause();          // rethrow the real exception, not the reflection wrapper
        }
    }
}
```

```java
// File: code/lld/ch09/proxy/Main.java
package lld.ch09.proxy;

import java.lang.reflect.Proxy;
import java.math.BigDecimal;
import java.util.Set;

public class Main {
    public static void main(String[] args) {
        System.out.println("== Virtual proxy: pay only when needed ==");
        Report report = new LazyReport("Q3 revenue");
        System.out.println("  created proxy; title = " + report.title());
        System.out.println("  render #1 -> " + report.render());
        System.out.println("  render #2 -> " + report.render());

        System.out.println("== Protection proxy wrapping the virtual proxy ==");
        Set<Role> allowed = Set.of(Role.FINANCE, Role.ADMIN);
        Report forFinance = new AuditedReport(new LazyReport("Payroll"), Role.FINANCE, allowed);
        Report forViewer = new AuditedReport(new LazyReport("Payroll"), Role.VIEWER, allowed);
        System.out.println("  finance -> " + forFinance.render());
        try {
            forViewer.render();
        } catch (SecurityException e) {
            System.out.println("  viewer  -> refused: " + e.getMessage() + " (nothing was loaded)");
        }

        System.out.println("== Caching proxy (LRU, max 2 entries, 60 s TTL) ==");
        LiveRateService live = new LiveRateService();
        RateService cached = new CachingRateProxy(live, 2, 60_000);
        cached.rate("USD", "EUR");
        cached.rate("USD", "EUR");
        cached.rate("USD", "INR");
        cached.rate("EUR", "INR");   // evicts USD->EUR (least recently used)
        cached.rate("USD", "EUR");
        System.out.println("  5 calls, real lookups = " + live.lookups);

        System.out.println("== Remote proxy (stub) ==");
        RateService remote = new RemoteRateProxy(new RateServer(new LiveRateService()));
        System.out.println("  100 USD = " + remote.convert(new BigDecimal("100"), "USD", "INR") + " INR");

        System.out.println("== Smart reference (reference counting) ==");
        SharedDevice printer = new SharedDevice();
        try (Device a = printer.acquire(); Device b = printer.acquire()) {
            System.out.println("  a -> " + a.query("status") + ", b -> " + b.query("print"));
        }

        System.out.println("== Dynamic proxy: one handler, any interface ==");
        Report loggedReport = LoggingHandler.wrap(new LazyReport("Q4 plan"), Report.class);
        loggedReport.title();
        loggedReport.render();
        RateService loggedRates = LoggingHandler.wrap(new LiveRateService(), RateService.class);
        loggedRates.convert(new BigDecimal("100"), "USD", "EUR");
        System.out.println("  ^ convert() called rate() on 'this', so the inner rate() call was NOT logged");
        System.out.println("  generated at run time: isProxyClass = "
                + Proxy.isProxyClass(loggedRates.getClass()));
    }
}
```

Output:

```
== Virtual proxy: pay only when needed ==
  created proxy; title = Q3 revenue
    [real] loading 100 MB of data for 'Q3 revenue'...
  render #1 -> Q3 revenue (1000000 rows)
  render #2 -> Q3 revenue (1000000 rows)
== Protection proxy wrapping the virtual proxy ==
    [real] loading 100 MB of data for 'Payroll'...
  finance -> Payroll (1000000 rows)
  viewer  -> refused: VIEWER may not render Payroll (nothing was loaded)
== Caching proxy (LRU, max 2 entries, 60 s TTL) ==
  5 calls, real lookups = 4
== Remote proxy (stub) ==
    [stub] sending 'rate|USD|INR' over the wire
  100 USD = 8300.00 INR
== Smart reference (reference counting) ==
    [device] opened (expensive)
    [ref] handle acquired, count=1
    [ref] handle acquired, count=2
  a -> ok:status, b -> ok:print
    [ref] handle released, count=1
    [ref] handle released, count=0
    [device] closed
== Dynamic proxy: one handler, any interface ==
    [proxy] -> title()
    [proxy] <- title() returned Q4 plan
    [proxy] -> render()
    [real] loading 100 MB of data for 'Q4 plan'...
    [proxy] <- render() returned Q4 plan (1000000 rows)
    [proxy] -> convert[100, USD, EUR]
    [proxy] <- convert[100, USD, EUR] returned 92.00
  ^ convert() called rate() on 'this', so the inner rate() call was NOT logged
  generated at run time: isProxyClass = true
```

What each part does

- `LazyReport` — **virtual proxy**. `title()` is answered locally; `render()` creates the `RealReport` on first use. The "loading" line appears **once**, only at the first render.
- **Thread-safe lazy init.** Without care, two threads calling `render()` at the same time both build the expensive object (harmless for a report, but a real bug if construction opens a connection or charges a card). We use **double-checked locking** (check without a lock, then lock and check again — see Chapter 8); the field **must be** `volatile` (a Java keyword that makes a write immediately visible to every thread) so a second thread never sees a half-constructed object.
- `AuditedReport` — **protection proxy**, wrapping a virtual proxy. The viewer is refused **before** anything is loaded — notice no second "loading 'Payroll'" line for the viewer.
- `RateService` works in `BigDecimal` (Java's exact decimal number type) — rates and money are never `double` — and `convert` rounds to 2 decimal places (`HALF_EVEN`, "banker's rounding": an exact half goes to the even digit).
- `CachingRateProxy` — **caching proxy**, **bounded** (LRU via `LinkedHashMap` in access order with `removeEldestEntry`) and **time-limited** (TTL). 5 calls → 4 real lookups: the second USD→EUR is a hit, the last one is a miss because it was evicted.
- `RemoteRateProxy` + `RateServer` — **remote proxy** (client "stub") and server "skeleton". The caller just calls `convert`; the stub turns it into a message and sends it over a (simulated) network.
- `SharedDevice` — **smart reference**. Each `acquire()` returns a lightweight handle proxy; the real device opens with the first handle and closes when the last handle closes. All bookkeeping (`refs`, `real`, each handle's `closed` flag) happens under the `SharedDevice` lock, and `close()` is idempotent (calling it twice has the same effect as calling it once), so concurrent or double closes can't corrupt the count.
- `LoggingHandler` — a **dynamic proxy** (next section).

Static vs dynamic proxies

	Static proxy	Dynamic proxy
How it's made	You hand-write a class per interface (`LazyReport`, `CachingRateProxy`)	Generated at **run time** from an interface list + a handler
Code per interface	One proxy class each; every method written out	One `InvocationHandler` for **any** interface
Type safety	Full; compiler checks each method	Calls arrive as `Method` + `Object[]` — reflective, less checked
Good for	Logic specific to one type (lazy load, access rules for a domain object)	**Cross-cutting concerns** (needs shared by many classes) on many types: logging, timing, transactions, security
JDK API	—	`java.lang.reflect.Proxy.newProxyInstance(loader, interfaces, handler)`
Limitation	Boilerplate	JDK proxies work **only for interfaces**; class-based proxies need bytecode (compiled JVM instructions) libraries (CGLIB, ByteBuddy)

How the JDK dynamic proxy works:

1. `Proxy.newProxyInstance(classLoader, new Class<?>[]{Report.class}, handler)` generates a class (named like `$Proxy0`) that implements `Report`.
2. **Every** method call on that object is routed to `handler.invoke(proxy, method, args)`.
3. The handler does its extra work and calls `method.invoke(target, args)` to forward.
4. If the target throws, reflection wraps it in `InvocationTargetException` — **unwrap it** (`e.getCause()`) so callers see the real exception.

The self-invocation trap (and Spring AOP)

Look at the last lines of the proxy output: we called `convert` through the logging proxy, and `convert` internally called `rate(...)` on `this` — the real object — so that inner call **bypassed the proxy** and was never logged.

This is exactly how **Spring AOP** behaves:

- Spring implements annotations (`@` tags on code that it reads) such as `@Transactional` (run this method inside a database transaction — all its changes succeed or fail together), `@Cacheable`, `@Async`, `@PreAuthorize` and custom `@Aspect` advice (the extra code AOP runs around a method) by wrapping your bean (an object the Spring container creates and manages) in a **proxy**. Callers get the proxy injected, not your object.
- Spring uses a **JDK dynamic proxy** when the target implements at least one interface, and a **CGLIB** subclass proxy otherwise (or when `proxyTargetClass = true`; Spring Boot 2.0+ defaults to CGLIB class-based proxies).
- **Self-invocation is not intercepted**: if `placeOrder()` calls `this.saveAudit()` which is `@Transactional(propagation = REQUIRES_NEW)`, the annotation is silently ignored because the call never leaves the object.
- With CGLIB proxies, **`final` classes cannot be proxied and `final`/`private` methods cannot be advised**.
- Fixes, in order of preference: move the method into another bean; inject the bean's own proxy (e.g. a lazily injected self-reference); or (discouraged) `AopContext.currentProxy()`, which also requires `exposeProxy = true`. AspectJ weaving (inserting the extra code directly into the compiled classes, at compile or load time) avoids the problem because it doesn't use proxies.

Proxy vs Decorator

This is the hardest pair in the catalogue to tell apart: both implement an interface, hold another object of that interface and forward calls to it. GoF itself says the implementations can be similar and that the difference is **purpose**: a decorator *adds responsibilities*, a proxy *controls access*.

	Proxy	Decorator
Purpose	**Controls access** to the object	**Adds behaviour** around the object
Does the call get through?	Not necessarily — may defer, cache or refuse	Normally yes — the added work runs around a forwarded call
Who creates / finds the real object?	**Often the proxy itself** (`LazyReport` builds `RealReport`; a remote proxy only knows an address)	**The caller**, who hands it in
Does the client know it has a wrapper?	Usually not — a factory/DI container hands it the proxy as if it were the real object	Usually yes — the client (or its configuration) builds the stack
How many layers?	Usually one	Often several, stacked recursively on purpose

The **creation test** is the most practical, because you can check it in the constructor: `LazyReport(String name)` builds its own subject; `LoggingRepository(UserRepository inner)` is given an existing one. A protection proxy such as `AuditedReport` *is* handed its subject, and is structurally identical to a decorator — there, only intent separates them. A caching wrapper genuinely sits on the line — both names are defensible — so the more useful question in review is whether its cache is bounded and invalidated.

Where proxies go wrong

- **Unsynchronised lazy creation** — two threads call `render()` together and both construct the expensive object: sometimes harmless, sometimes a duplicate connection or a double charge.
- **A caching proxy with no size limit or expiry** — a caching proxy *is* a cache, and every cache needs a bound and an invalidation story; without them it leaks memory and serves stale data. (Ours has both.)
- **Concealing a real cost** — a remote proxy that looks local tempts callers to use it inside loops. Network calls disguised as field access cause some of the nastiest distributed-system (many machines cooperating over a network) performance bugs; the cure is usually to make the cost visible again (batch APIs that fetch many items in one call, explicit `fetch...` names), not to tune the proxy.
- **ORM N+1 queries** (an ORM, or object-relational mapper, turns database rows into objects; N+1 means one query for the list plus one more per item) — iterating 100 orders and touching `order.getCustomer().getName()` fires one query per lazy proxy.

Trade-offs

Pros	Cons
Pay later — costly construction happens only if something needs it.	**Extra indirection**; stack traces show the proxy, not the real class.
No caller changes — the interface is identical (Open/Closed).	**Latency and failures are disguised** (latency = how long a call takes) as an ordinary method call.
Somewhere to put cross-cutting rules for a class you cannot edit.	**Lazy creation brings a thread-safety question** that is easy to get wrong.
Permission checks stay outside the object, which keeps to its own job.	**A cache with no eviction** leaks memory and returns stale data.

When not to use it

- **The real object is cheap** — making something small lazy only adds a null check to every call.
- **Callers need to see the cost** — sometimes the honest interface is a method literally named `fetchOverNetwork()`.
- **The framework already generates one** — ORMs, DI containers and service meshes (a layer of proxies placed next to every service to manage the traffic between them) create proxies; stacking a hand-written one on top of those just doubles the hidden behaviour.
- **You mean to add behaviour, not guard access** — that is a Decorator.

Common mistakes

- Non-`volatile` field in double-checked locking.
- Forgetting to unwrap `InvocationTargetException` in a dynamic proxy.
- Expecting Spring annotations to work on self-calls, `private` methods or `final` methods.

- Unbounded caching proxies.
- Using a remote proxy in a tight loop.

Real-world examples

- **ORM lazy loading** (Hibernate/JPA — JPA is Java's standard ORM API — also Entity Framework and Django) — `entityManager.getReference(...)` and lazy associations return proxy objects; the SQL (database query) runs the first time you touch a non-id field. The most widely used virtual proxy — and the source of the N+1 problem and, in Hibernate, `LazyInitializationException` when the session (Hibernate's open unit of work with the database) is already closed.
- **Spring AOP** — `@Transactional`, `@Cacheable`, `@Async`, method security: all proxies. Spring's `@Configuration` classes are also CGLIB-enhanced subclasses so that calling one `@Bean` method from another returns the container's singleton (unless `proxyBeanMethods = false`).
- **Remote stubs** — Java RMI stubs, gRPC and Thrift generated clients, SOAP clients: a network call behind an ordinary method.
- `java.lang.reflect.Proxy` — JDK support for dynamic proxies (used by many frameworks, e.g. Spring Data repositories and MyBatis mappers are interfaces whose implementations are generated proxies).
- `Collections.unmodifiableList` — a protection proxy that refuses mutation (also seen as a decorator).
- **Service meshes / sidecars** (Envoy in Istio) — intercept traffic for retries, mTLS (mutual TLS: both sides prove their identity and the traffic is encrypted) and telemetry (metrics and traces sent for monitoring): the proxy idea at infrastructure level. **Reverse proxies / CDNs** (Nginx, Cloudflare) cache and protect origin servers (your own servers holding the original content). A reverse proxy is a server that sits in front of yours and receives requests on their behalf; a CDN (content delivery network) caches content on servers close to users — see System Design Fundamentals.
- **C++** `shared_ptr` — a smart-reference proxy doing reference counting.

In modern languages

- **Java**: `java.lang.reflect.Proxy`, plus bytecode generation (CGLIB, ByteBuddy) for classes.
- **JavaScript/TypeScript**: built-in `Proxy` intercepts property access and calls.
- **Python**: `__getattr__` forwards anything not defined locally, so a proxy only writes the methods it intercepts; `functools.cached_property` is a virtual proxy for one field.
- **C#**: `DispatchProxy`, and `Lazy<T>` — a virtual proxy for one value.
- **C++**: `std::shared_ptr` and other smart pointers are smart-reference proxies that count references behind `operator->`.

Other patterns it is confused with

Looks like	Actual difference
Decorator	Same structure; see the table above. Decorator adds behaviour to an object it is handed; Proxy controls access and often creates or locates the real object itself.
Adapter	An Adapter has a **different** interface on purpose — it exists to translate. A Proxy has the **same** interface — it exists to substitute.
Facade	A Facade offers a new, simpler interface over **several** objects. A Proxy offers the same interface over **one**.
A cache	A caching proxy is a cache with a proxy's shape. Calling it a proxy does not remove the need to design its size bound and eviction.

In the interview

> **30-second answer:** "A Proxy is a stand-in with the same interface as the real object that controls access to it. I'd name the kind: a virtual proxy defers expensive creation, a protection proxy checks permissions, a remote proxy hides the network, a caching proxy reuses results, and a smart reference does bookkeeping like reference counting. Because the interface is identical, callers don't change — which is also the danger, since an expensive call looks cheap. Lazy creation needs thread safety, and ORM lazy-loading proxies are the classic source of N+1 queries. In Java I'd use `java.lang.reflect.Proxy` for cross-cutting concerns; that's how Spring AOP implements `@Transactional`."

Follow-ups

- *Q: Proxy vs Decorator?* — A: Proxy controls access and often creates the real object; Decorator adds behaviour to an object handed to it and normally forwards every call. Check the constructor.
- *Q: Make the virtual proxy thread-safe.* — A: Double-checked locking with a `volatile` field (as in `LazyReport`), or simply make the getter `synchronized` if contention (threads waiting on each other for the lock) is low. (The static holder-class idiom only works for a *static* lazy field, so it does not fit a per-instance proxy.)
- *Q: Why doesn't `@Transactional` work when I call the method from the same class?* — A: Spring's proxy only intercepts calls that come through the proxy; `this.method()` goes straight to the target.
- *Q: JDK proxy vs CGLIB?* — A: JDK proxies implement interfaces only; CGLIB creates a subclass, so it works without interfaces but can't handle `final` classes/methods.
- *Q: How does lazy loading cause N+1?* — A: Loading N parents and then touching each parent's lazy child proxy triggers N extra queries; fix with fetch joins / entity graphs / batch fetching (ways to load the children together with their parents).

Self-check

1. **Q:** Which proxy kind is `LazyReport`? `AuditedReport`? **A:** Virtual; protection.
2. **Q:** Why is the field in `LazyReport` `volatile`? **A:** So double-checked locking is safe — other threads see a fully constructed object.
3. **Q:** In the dynamic proxy demo, why wasn't the inner `rate()` call logged? **A:** `convert()` called `rate()` on `this` (the real object), bypassing the proxy — the self-invocation trap.
4. **Q:** Can `java.lang.reflect.Proxy` proxy a class with no interface? **A:** No; it needs interfaces. Use CGLIB/ByteBuddy for classes.
5. **Q:** What two properties must a caching proxy have? **A:** A size bound (eviction) and an invalidation/expiry policy.

TL;DR

- **Same interface, controlled access.** Virtual, protection, remote, caching, logging, smart reference.
- **Static** = hand-written per interface; **dynamic** = `java.lang.reflect.Proxy` + `InvocationHandler` for any interface.
- Spring AOP = proxies → beware self-invocation, `final`, `private`. Watch thread safety and hidden costs.

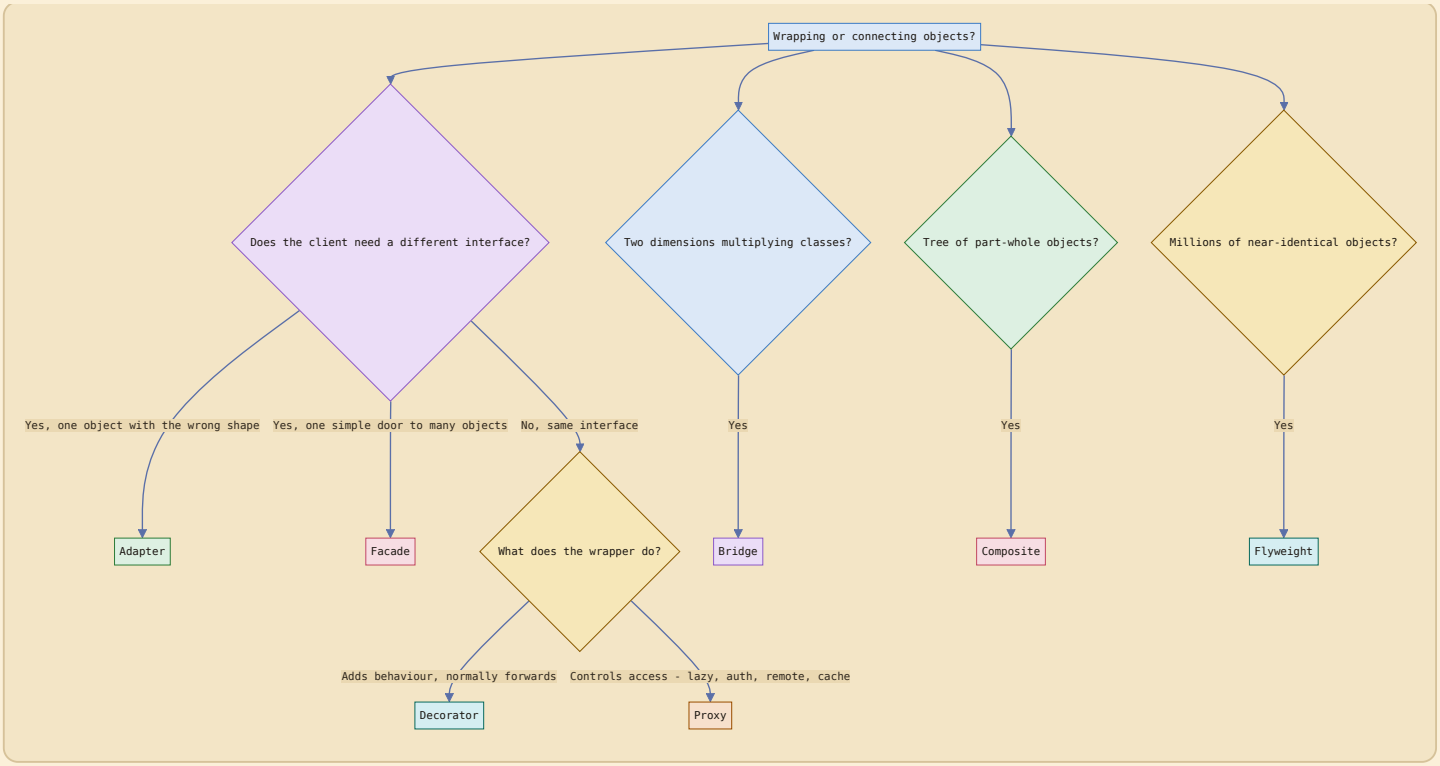
9.8 Putting it all together — comparison

Summary of all seven structural patterns

Pattern	Distinctive feature	Use it when	Example	Pros	Cons
Adapter	Bridges incompatible interfaces	You must use an existing class whose interface doesn't fit your code	Travel plug adapter; `StripeGateway`	Interoperability; reuse of existing code	Extra class and complexity
Bridge	Separates abstraction from implementation	Two dimensions vary independently; avoid permanent binding	UI shapes × platform renderers; notifications × channels	Both sides vary independently; scales by addition	More concepts up front; indirection
Composite	Part-whole tree with one interface	Treat single objects and groups uniformly	Drawing editor with shapes and groups; file system	Simpler client code; easy to add node types	Interface can become over-general
Decorator	Adds responsibilities dynamically	Extend an object's behaviour at run time, in combinations	Adding scrolling/border to a text view; Java I/O	More flexible than subclassing; no feature-bloated classes	Many small objects; hard to debug; order matters
Facade	Simple interface to a complex subsystem	Most callers need one common path through many classes	Home-theatre "watch movie" button; `OrderFacade`	Simplifies use; fewer dependencies	Can become a god object or single point of failure
Flyweight	Shares common state among many objects	Huge numbers of similar objects with little variation	Characters in a word processor; map markers	Big memory savings	Complexity; managing shared immutable state
Proxy	Placeholder that controls access	Lazy loading, access control, remote, caching, logging, smart reference	Internet proxy server; Hibernate lazy entity	Controls access; defers costly work	Indirection; hidden latency

Adapter vs Decorator vs Proxy vs Facade vs Bridge

	Adapter	Decorator	Proxy	Facade	Bridge
Intent	Convert an interface to the one the client expects	Add behaviour without changing the interface	Control access to an object	Simplify a complex subsystem	Let abstraction and implementation vary independently
Interface seen by client	**Different** from the wrapped object's (it's yours)	**Same** as wrapped	**Same** as real subject	**New, simpler** interface	Abstraction's own interface
Behaviour	Unchanged (translated)	**Extended**	Unchanged but **controlled** (may defer/cache/refuse)	Unchanged (fewer calls)	Delegated to implementor
Objects wrapped	One	One (stackable)	One	**Several**	One implementor (by reference)
Who creates the wrapped object	Usually passed in	Passed in by client	**Often the proxy itself**	Passed in / owned	Passed in at construction
Structure	Implements Target, holds Adaptee	IS-A and HAS-A Component	IS-A Subject, HAS-A Real Subject	HAS-A many subsystem objects	Abstraction HAS-A Implementor; two hierarchies
When introduced	**After the fact**, to make existing things fit	Any time a concern is added	Any time access needs controlling	When callers repeat a multi-step sequence	**Up front**, before the class explosion
Typical trigger	"The vendor API doesn't match ours"	"Sometimes we need logging, sometimes caching, sometimes both"	"It's expensive / remote / restricted"	"Every caller copies these 5 steps"	"Class names with two nouns"
JDK / framework example	`Arrays.asList`, `InputStreamReader`	`BufferedInputStream`, `Collections.synchronizedList`	`java.lang.reflect.Proxy`, Spring `@Transactional`, Hibernate lazy load	SLF4J `LoggerFactory`, service layers	JDBC `Connection` + drivers, AWT peers



Memory hooks

- **Adapter** — *"make it fit"* (changes the interface).
- **Decorator** — *"add to it"* (same interface, more behaviour, stackable).
- **Proxy** — *"guard it"* (same interface, controlled access).
- **Facade** — *"simplify it"* (one door, many rooms).
- **Bridge** — *"split it"* (two hierarchies, add instead of multiply).
- **Composite** — *"nest it"* (one interface for leaf and tree).
- **Flyweight** — *"share it"* (intrinsic shared, extrinsic passed in).

Chapter self-check

1. **Q:** Which of these patterns wrap exactly one object? **A:** Adapter, Decorator, Proxy (Bridge also holds one implementor, but by design rather than as a wrapper). Facade wraps several; Composite holds many children.
2. **Q:** Which two have identical code shape but differ in intent and timing? **A:** Adapter and Bridge (after-the-fact vs up-front); also Bridge and Strategy; also Decorator and Proxy.
3. **Q:** A wrapper returns cached results with the same interface. Decorator or Proxy? **A:** Either is defensible; the important question is whether the cache is bounded and invalidated.
4. **Q:** You have `WindowsButton`, `MacButton`, `WindowsCheckbox`, `MacCheckbox`… and a Linux port is coming. Which pattern? **A:** Bridge (widget abstraction × platform implementor) — often combined with Abstract Factory to pick the implementor.
5. **Q:** Which pattern explains why `Integer.valueOf(100) == Integer.valueOf(100)` is true? **A:** Flyweight (Integer cache).

Chapter TL;DR

- Structural patterns **compose objects into bigger structures** while keeping them flexible.
- **Adapter / Decorator / Proxy / Facade** share a shape; tell them apart by **what changes as the call passes through** and **how many objects are wrapped**.
- **Bridge** prevents class explosion across two dimensions; **Composite** makes trees uniform; **Flyweight** shares immutable state to save memory.
- In interviews: name the pattern, state the **constraint** that justifies it, name its **trap** (vendor leaks, wrong split, cycles, decorator order, god facade, mutable flyweight, self-invocation), and give a **real JDK/Spring example**.

Chapter 10 — Behavioral Design Patterns (Part 1)

> **What you will learn.** What behavioral patterns are, and six of them in depth: **Chain of Responsibility, Command, Iterator, Mediator, Memento** and **Interpreter**. For each one you get a plain definition, the problem it fixes, the class diagram, Java 21 code that compiles and runs, real products and JDK (Java Development Kit — Java's compiler, tools and standard library) classes that use it, and interview questions with answers. Chapter 11 covers the other five GoF behavioral patterns (Observer, State, Strategy, Template Method, Visitor) and some important patterns that are not in the GoF book.

How to run the code in this chapter

```
cd chapters/10-patterns-behavioral-1/code
javac --release 21 -d /tmp/lld_10 $(find . -name '*.java')
java -cp /tmp/lld_10 lld.ch10.chain.Main        # also: command, iterator, mediator, memento, interpreter
```

10.0 Behavioral patterns: the big picture

What it is

Creational patterns (Chapter 8) are about **how objects are made**. Structural patterns (Chapter 9) are about **how objects are put together**. **Behavioral patterns** are about **how objects share work and talk to each other**: who is responsible for what, and who calls whom.

Everyday analogy: a restaurant kitchen. The structural question is "which stations exist?" The behavioral question is "when an order comes in, who hears about it first, who passes it on, who decides the order of the dishes, and who can cancel it?"

Behavioral patterns help you to:

- let objects work together in a flexible, efficient way;
- cut tight coupling (classes depending heavily on each other's internal details) between classes, so each class is easier to change and test;
- turn tangled communication between many objects into something simple to follow;
- change behaviour **while the program runs** (at run time), not only when you compile it.

The eleven GoF behavioral patterns

The "Gang of Four" (GoF) book, *Design Patterns* (Gamma, Helm, Johnson, Vlissides, 1994), lists eleven behavioral patterns. This chapter covers the first six; Chapter 11 covers the rest.

Pattern	One-line intent	Chapter
Chain of Responsibility	Hand a request down a row of handler objects; each may deal with it, stop it, or forward it.	10
Command	Wrap a request in an object, which can then be stored, queued, logged or reversed.	10
Interpreter	Represent the grammar of a small language as classes, and evaluate sentences in it.	10
Iterator	Walk through a collection one element at a time without seeing how it is stored.	10
Mediator	Route communication between many objects through one coordinator.	10
Memento	Let an object save a snapshot of its own state and later return to it, while keeping that state private.	10
Observer	When one object changes, notify all the objects that depend on it.	11
State	Let an object change its behaviour when its internal state changes.	11
Strategy	Choose one of several interchangeable algorithms at run time.	11
Template Method	Fix the skeleton of an algorithm in a base class; let subclasses fill in some steps.	11
Visitor	Add a new operation to a set of classes without changing those classes.	11

Class scope vs object scope. GoF splits behavioral patterns into two kinds. *Class* behavioral patterns use **inheritance** to share behaviour: only **Template Method** and **Interpreter**. All the others are *object* behavioral patterns: they use **composition** (objects holding references to other objects) and can therefore be rewired at run time.

One idea runs through all of them: take a piece of behaviour that is hidden inside a big method (a chain of `if`s, a hard-coded call, a loop) and turn it into an **object** you can name, pass around, swap, store and test on its own.

10.1 Chain of Responsibility

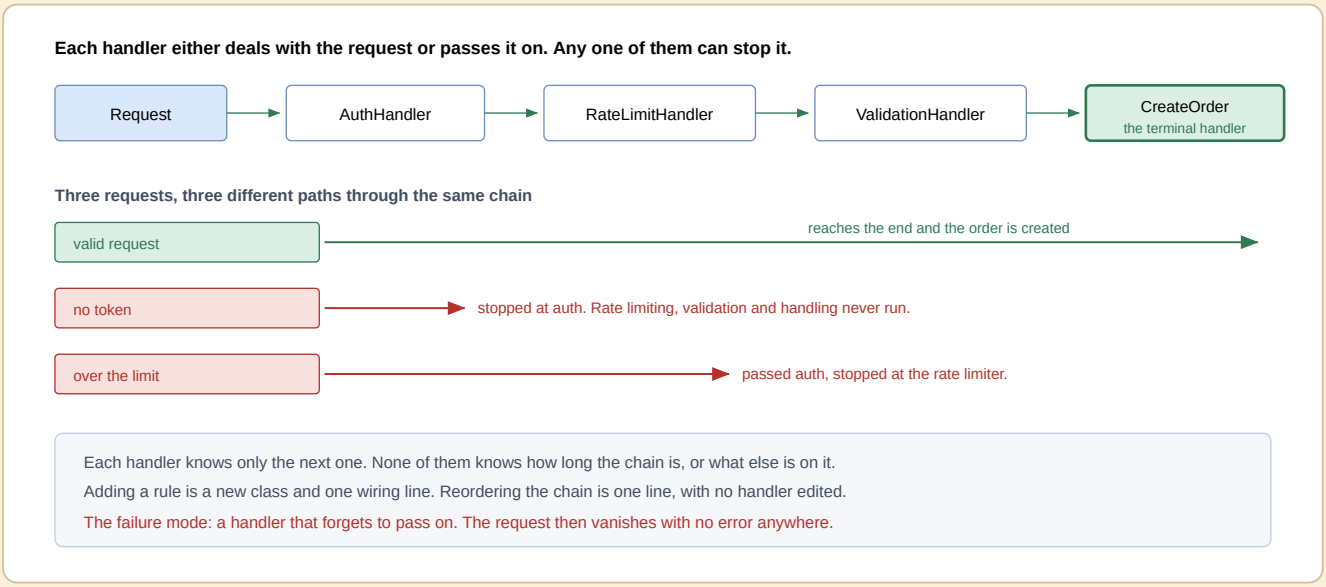


Figure — Three requests go into the same chain of handlers and each stops at a different place (course diagram)

What it is

Chain of Responsibility (CoR) sends a request through a row of **handler** objects. Every handler makes one of three choices: deal with the request, reject it (so nobody later sees it), or forward it to its successor (the next handler in the row). The sender talks only to the first handler and never learns which one finally took care of the request.

GoF intent, in plain words: *avoid tying the sender of a request to one particular receiver by giving several objects a chance to handle it; link those objects and forward the request until one handles it.* GoF participants: **Handler** (declares the handling method, optionally holds the successor link), **ConcreteHandler** (handles what it is responsible for, otherwise forwards), and **Client** (starts the request at a handler in the chain).

Everyday analogy: a customer-support phone line. Level-1 support tries first. If they cannot help, they pass you to level 2, then to an engineer. You only ever dialled one number.

In the figure above, one chain (auth → rate limit → validation → create order) receives three requests. A valid request reaches the end. A request with no token stops at auth. A request over the limit passes auth but stops at the rate limiter. Each handler knows only the next one.

The problem it solves

Before an order is created, a request must survive several gates: who is calling (authentication), are they calling too often (rate limit), and is the body (the data sent with the request) well-formed (validation). Written directly, one method does all of them:

- The method grows every time a rule is added.
- The **order of the checks is hidden inside the method body**, yet it matters: authenticating before rate limiting is a different system from rate limiting before authenticating.
- Some routes (the URL paths a server handles, such as `/orders`) need only two of the checks. Soon there are boolean flags, then a second copy of the method that someone forgets to update.

CoR makes **each check its own small object**, and links them into a line. No handler knows how long the line is or what else is on it, so **the line itself is configuration, not logic**: a new rule costs one new class plus one wiring line; changing the order touches only the wiring; each route can be given its own line, assembled when the application starts.

How it works

1. Define a **Handler** type with a method such as `handle(request)` and a reference to the **next** handler.
2. Write one **concrete handler** per check. Each one does its own check, then either **stops** (returns a result) or **passes on** (`next.handle(request)`).
3. Put a **terminal handler** at the end that does the real work (or reports "nobody handled this").
4. **Wire** the chain once, at start-up: `auth.setNext(rateLimit).setNext(validate).setNext(createOrder)`.
5. The client calls only the **first** handler. It never knows how long the chain is.

Structure (UML)

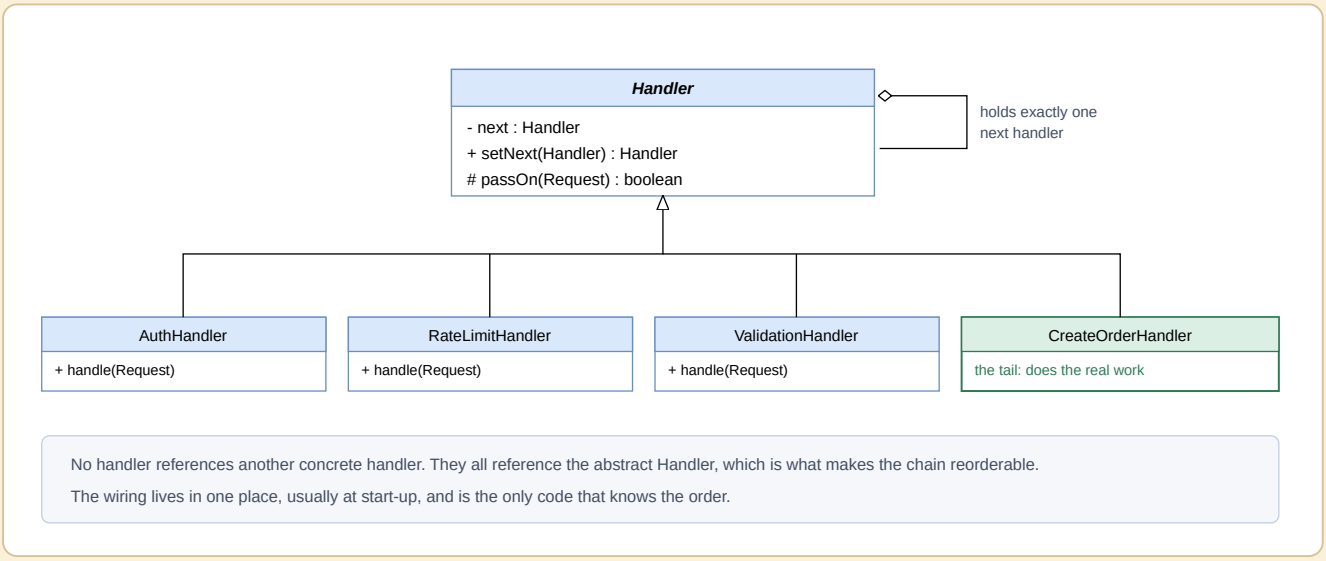
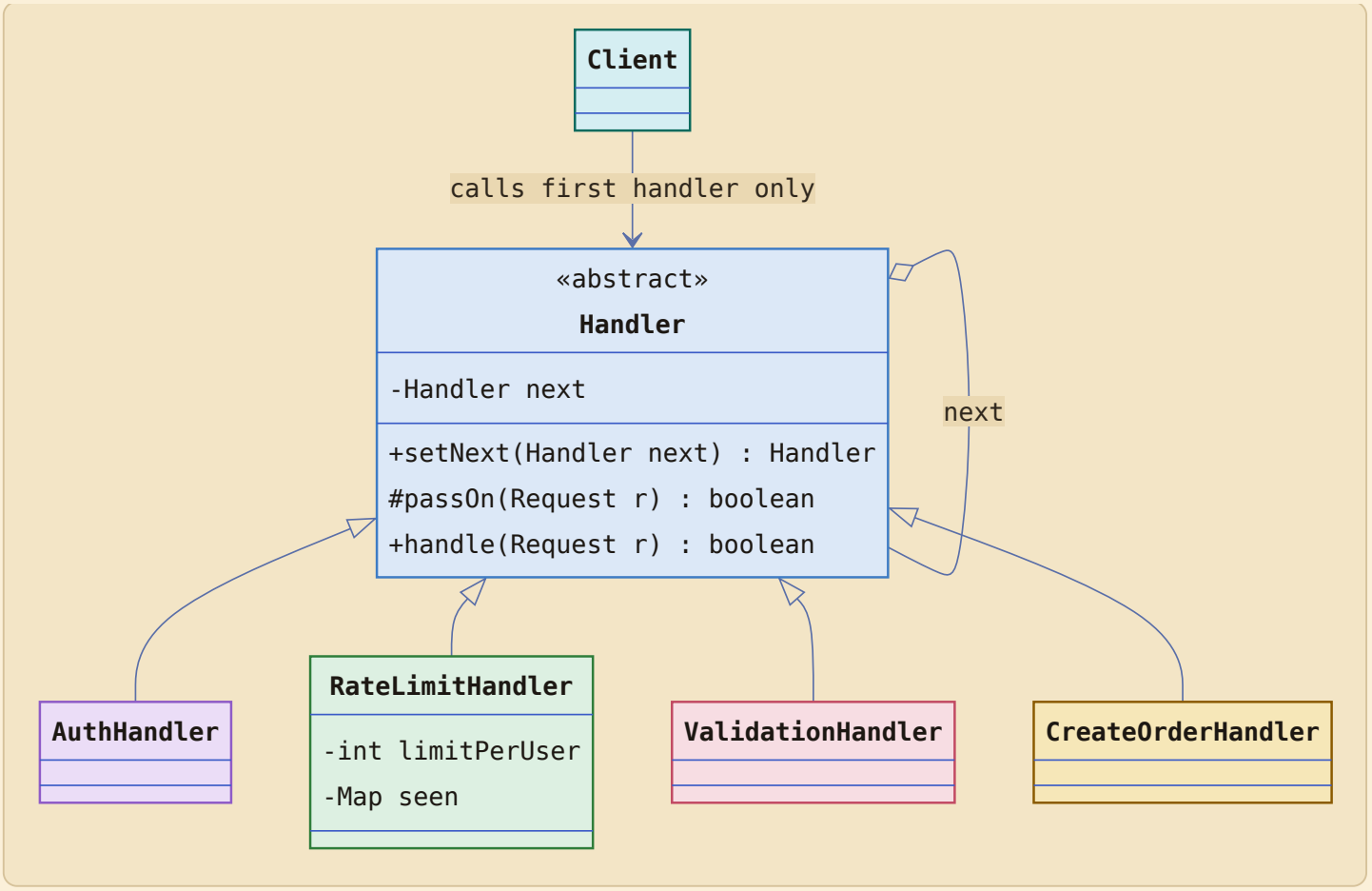


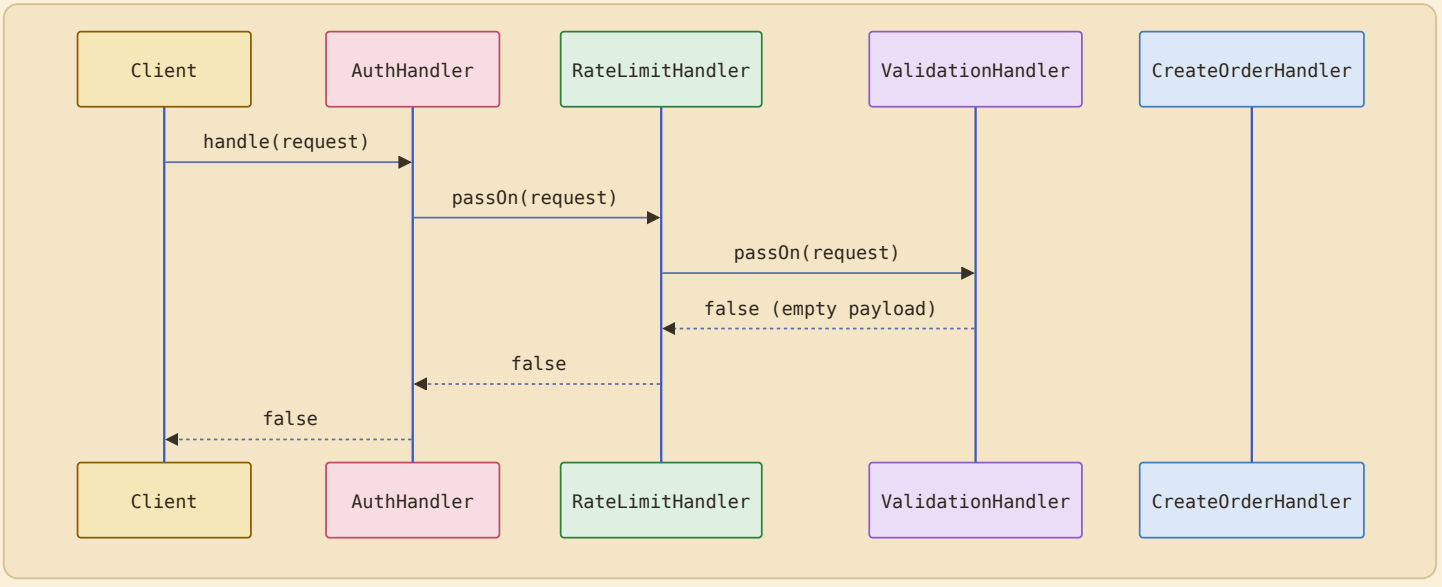
Figure — Class diagram: an abstract Handler holds `next`; concrete handlers extend it; CreateOrderHandler is the tail (course diagram)

In the class diagram above (a UML — Unified Modeling Language — diagram, the standard notation for classes and their links), notice what is **missing**: no handler refers to another *concrete* handler. Every link is typed as the abstract `Handler`, and that is why the stages can be shuffled freely. `setNext` hands back its argument, which lets the wiring be written as one fluent expression (calls chained with dots, each returning the object for the next call).

> **Our code vs the course pseudocode.** In the course version, `passOn` returns `true` ("success") when there is no next handler. Our `passOn` **throws** instead, because silently reporting success when the chain simply ran out is exactly the kind of invisible failure this pattern is prone to. Either choice is legal; just make it deliberate.

Our Java design:





Java example

The package `lld.ch10.chain` shows **four flavours** of the pattern:

1. **Classic linked handlers** (`Handler`, `AuthHandler`, `RateLimitHandler`, `ValidationHandler`, `CreateOrderHandler`). Each handler returns an *explicit* outcome: `false` = rejected, `passOn(...)` = continue. `passOn` **throws** if there is no next handler, so a chain without a terminal handler fails loudly instead of silently "succeeding".
2. **Approval chain** (`Approver`, `Expense`). A pure CoR: the *first* approver whose limit is high enough approves, and nobody after it sees the request. Note that there is only **one** class — the chain is just data (role + limit). Amounts and limits are `BigDecimal` (Java's exact decimal number type) compared with `compareTo` — never `double` for money.
3. **Servlet-style filters** (`Filter`, `FilterChain`, `Endpoint`, `HttpRequest`). This is how `jakarta.servlet.Filter` (part of the servlet API, Java's standard way to handle web requests) works: the "next" step is passed **in as an argument** (`chain.doFilter(...)`). A filter can act before *and after* the rest of the chain (see the timing filter), or stop the chain by not calling it (the auth filter).

4. **Logging levels** (`LogHandler`, `ConsoleLog`, `FileLog`, `PagerLog`, `Level`). A common *variant*: each handler acts if the message is severe enough and then **always** passes it on. Several handlers may act. Strictly, when nothing can stop the flow it is closer to a *pipeline* (a fixed series of steps that every item passes through), but textbooks often show logging this way, and it is worth knowing the difference.

```
// File: code/lld/ch10/chain/Handler.java
package lld.ch10.chain;

/** Classic Chain of Responsibility base: each handler knows only "the next one". */
public abstract class Handler {
    private Handler next;

    /** Links the next handler and returns it, so wiring reads as one chained expression. */
    public Handler setNext(Handler next) {
        this.next = next;
        return next;
    }

    /** Forwards the request. Falling off the end means the chain has no terminal handler: a wiring bug. */
    protected boolean passOn(Request request) {
        if (next == null) {
            throw new IllegalStateException(
                getClass().getSimpleName() + " tried to pass on, but the chain has no terminal handler");
        }
        return next.handle(request);
    }

    /** @return true if the request was fully handled, false if some handler rejected it. */
    public abstract boolean handle(Request request);
}
```

```
// File: code/lld/ch10/chain/Request.java
package lld.ch10.chain;

/** The request that travels down the API pipeline. */
public record Request(String user, String token, String payload) {}
```

```
// File: code/lld/ch10/chain/AuthHandler.java
package lld.ch10.chain;

/** Stage 1: reject requests with no token. */
final class AuthHandler extends Handler {
    @Override
    public boolean handle(Request r) {
        if (r.token() == null || r.token().isBlank()) {
            System.out.println("  [auth] rejected: missing token");
            return false;                    // explicit outcome: the chain stops here
        }
        System.out.println("  [auth] ok for " + r.user());
        return passOn(r);                    // explicit outcome: continue
    }
}
```

```java
// File: code/lld/ch10/chain/RateLimitHandler.java
package lld.ch10.chain;

import java.util.HashMap;
import java.util.Map;

/** Stage 2: at most N requests per user (keyed by user because auth ran first). */
final class RateLimitHandler extends Handler {
    private final int limitPerUser;
    private final Map<String, Integer> seen = new HashMap<>();

    RateLimitHandler(int limitPerUser) { this.limitPerUser = limitPerUser; }

    @Override
    public boolean handle(Request r) {
        int count = seen.merge(r.user(), 1, Integer::sum);
        if (count > limitPerUser) {
            System.out.println("  [ratelimit] rejected: " + r.user() + " made " + count + " requests, limit " +
limitPerUser);
            return false;
        }
        System.out.println("  [ratelimit] ok (" + count + "/" + limitPerUser + ")");
        return passOn(r);
    }
}
```

```java
// File: code/lld/ch10/chain/ValidationHandler.java
package lld.ch10.chain;

/** Stage 3: the payload must not be empty. */
final class ValidationHandler extends Handler {
    @Override
    public boolean handle(Request r) {
        if (r.payload() == null || r.payload().isBlank()) {
            System.out.println("  [validate] rejected: empty payload");
            return false;
        }
        System.out.println("  [validate] ok");
        return passOn(r);
    }
}
```

```java
// File: code/lld/ch10/chain/CreateOrderHandler.java
package lld.ch10.chain;

/** Terminal handler: does the real work and never passes on. */
final class CreateOrderHandler extends Handler {
    @Override
    public boolean handle(Request r) {
        System.out.println("  [order] created order for " + r.user() + ": " + r.payload());
        return true;
    }
}
```

```java
// File: code/lld/ch10/chain/Expense.java
package lld.ch10.chain;

import java.math.BigDecimal;

/** An expense claim that needs someone with enough authority to approve it. Money is BigDecimal, never double. */
record Expense(String id, BigDecimal amountUsd) {}
```

```java
// File: code/lld/ch10/chain/Approver.java
package lld.ch10.chain;

import java.math.BigDecimal;

/**
 * Approval chain. Pure Chain of Responsibility: the FIRST handler able to deal
 * with the request handles it, and nobody after it sees the request.
 * Note there is only one class: the chain is data (name + limit), not code.
 */
final class Approver {
    private final String role;
    private final BigDecimal limitUsd;
    private Approver next;

    Approver(String role, BigDecimal limitUsd) {
        this.role = role;
        this.limitUsd = limitUsd;
    }

    Approver then(Approver next) {
        this.next = next;
        return next;
    }

    String approve(Expense e) {
        if (e.amountUsd().compareTo(limitUsd) <= 0) {   // compareTo, not equals: 300 and 300.00 are the same amount
            return role + " approved " + e.id() + " ($" + e.amountUsd() + ")";
        }
        if (next == null) {   // explicit terminal outcome instead of a silent "nothing happened"
            return "REJECTED " + e.id() + " ($" + e.amountUsd() + "): above every approver's limit";
        }
        System.out.println("    " + role + " (limit $" + limitUsd + ") escalates " + e.id());
        return next.approve(e);
    }
}
```

```java
// File: code/lld/ch10/chain/HttpRequest.java
package lld.ch10.chain;

import java.util.Map;

/** Minimal HTTP request for the servlet-style demo. */
record HttpRequest(String path, Map<String, String> headers) {}
```

```java
// File: code/lld/ch10/chain/Filter.java
package lld.ch10.chain;

/** Same shape as jakarta.servlet.Filter: "next" is an explicit argument, so forgetting it is visible. */
@FunctionalInterface
interface Filter {
    void doFilter(HttpRequest request, StringBuilder response, FilterChain chain);
}
```

```java
// File: code/lld/ch10/chain/Endpoint.java
package lld.ch10.chain;

/** The endpoint at the end of the chain (like a Servlet). */
@FunctionalInterface
interface Endpoint {
    void service(HttpRequest request, StringBuilder response);
}
```

```java
// File: code/lld/ch10/chain/FilterChain.java
package lld.ch10.chain;

import java.util.List;

/** Walks the filter list. One FilterChain object per request, because it holds a position. */
final class FilterChain {
    private final List<Filter> filters;
    private final Endpoint endpoint;
    private int position = 0;

    FilterChain(List<Filter> filters, Endpoint endpoint) {
        this.filters = filters;
        this.endpoint = endpoint;
    }

    void doFilter(HttpRequest request, StringBuilder response) {
        if (position < filters.size()) {
            filters.get(position++).doFilter(request, response, this);
        } else {
            endpoint.service(request, response);   // the terminal handler
        }
    }
}
```

```java
// File: code/lld/ch10/chain/Level.java
package lld.ch10.chain;

/** Severity levels, ordered from least to most severe. */
enum Level { DEBUG, INFO, ERROR }
```

```java
// File: code/lld/ch10/chain/LogHandler.java
package lld.ch10.chain;

/**
 * Logging-level chain. A VARIANT of the pattern: each handler acts if the record
 * is severe enough, and then ALWAYS passes it on, so several handlers may act.
 * (If nobody can stop the flow, it is really a pipeline; the classic form stops.)
 */
abstract class LogHandler {
    private final Level threshold;
    private LogHandler next;

    LogHandler(Level threshold) { this.threshold = threshold; }

    LogHandler setNext(LogHandler next) {
        this.next = next;
        return next;
    }

    final void log(Level level, String message) {
        if (level.compareTo(threshold) >= 0) {
            write(level, message);
        }
        if (next != null) {
            next.log(level, message);
        }
    }

    protected abstract void write(Level level, String message);
}
```

```java
// File: code/lld/ch10/chain/ConsoleLog.java
package lld.ch10.chain;

final class ConsoleLog extends LogHandler {
    ConsoleLog() { super(Level.DEBUG); }
    @Override protected void write(Level l, String m) { System.out.println("    console <- " + l + " " + m); }
}
```

```java
// File: code/lld/ch10/chain/FileLog.java
package lld.ch10.chain;

final class FileLog extends LogHandler {
    FileLog() { super(Level.INFO); }
    @Override protected void write(Level l, String m) { System.out.println("    app.log <- " + l + " " + m); }
}
```

```java
// File: code/lld/ch10/chain/PagerLog.java
package lld.ch10.chain;

final class PagerLog extends LogHandler {
    PagerLog() { super(Level.ERROR); }
    @Override protected void write(Level l, String m) { System.out.println("    PAGER   <- " + l + " " + m); }
}
```

```java
// File: code/lld/ch10/chain/Main.java
package lld.ch10.chain;

import java.math.BigDecimal;
import java.util.List;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== 1. API request pipeline (classic linked handlers) ===");
        // Order is a design decision: auth BEFORE rate limiting so the limiter can key on the user,
        // and so anonymous traffic cannot burn a real user's quota.
        Handler pipeline = new AuthHandler();
        pipeline.setNext(new RateLimitHandler(2))
                .setNext(new ValidationHandler())
                .setNext(new CreateOrderHandler());

        List<Request> requests = List.of(
            new Request("alice", "t-123", "2 x widget"),
            new Request("bob", "", "1 x gadget"),
            new Request("alice", "t-123", " "),
            new Request("alice", "t-123", "1 x gadget"));
        for (Request r : requests) {
            System.out.println("request from " + r.user() + ":");
            System.out.println("  => handled = " + pipeline.handle(r));
        }

        System.out.println("\n=== 2. Expense approval chain (first capable handler wins) ===");
        Approver chain = new Approver("TeamLead", new BigDecimal("1000"));
        chain.then(new Approver("Manager", new BigDecimal("10000")))
             .then(new Approver("Director", new BigDecimal("50000")));
        for (Expense e : List.of(new Expense("EXP-1", new BigDecimal("300")),
                                 new Expense("EXP-2", new BigDecimal("7500")),
                                 new Expense("EXP-3", new BigDecimal("120000")))) {
            System.out.println("  " + chain.approve(e));
        }

        System.out.println("\n=== 3. Servlet-style filters (next is an explicit argument) ===");
        Filter timing = (req, res, next) -> {
            System.out.println("  [timing] start " + req.path());
            next.doFilter(req, res);                        // filters can act before AND after
            System.out.println("  [timing] end   " + req.path() + " -> " + res);
        };
        Filter auth = (req, res, next) -> {
            if (!req.headers().containsKey("Authorization")) {
                res.append("401 Unauthorized");             // does not call next: chain stops
                return;
            }
            next.doFilter(req, res);
        };
        Endpoint ordersEndpoint = (req, res) -> res.append("200 OK orders list");

        for (HttpRequest req : List.of(
                new HttpRequest("/orders", Map.of("Authorization", "Bearer abc")),
                new HttpRequest("/orders", Map.of()))) {
            new FilterChain(List.of(timing, auth), ordersEndpoint).doFilter(req, new StringBuilder());
        }

        System.out.println("\n=== 4. Logging levels (variant: every handler may act) ===");
        LogHandler logger = new ConsoleLog();
        logger.setNext(new FileLog()).setNext(new PagerLog());
        System.out.println("  log DEBUG:");
        logger.log(Level.DEBUG, "cache miss for key=42");
        System.out.println("  log ERROR:");
        logger.log(Level.ERROR, "payment service down");
    }
}
```

Output:

```
=== 1. API request pipeline (classic linked handlers) ===
request from alice:
  [auth] ok for alice
  [ratelimit] ok (1/2)
  [validate] ok
  [order] created order for alice: 2 x widget
  => handled = true
request from bob:
  [auth] rejected: missing token
  => handled = false
request from alice:
  [auth] ok for alice
  [ratelimit] ok (2/2)
  [validate] rejected: empty payload
  => handled = false
request from alice:
  [auth] ok for alice
  [ratelimit] rejected: alice made 3 requests, limit 2
  => handled = false

=== 2. Expense approval chain (first capable handler wins) ===
  TeamLead approved EXP-1 ($300)
    TeamLead (limit $1000) escalates EXP-2
  Manager approved EXP-2 ($7500)
    TeamLead (limit $1000) escalates EXP-3
    Manager (limit $10000) escalates EXP-3
  REJECTED EXP-3 ($120000): above every approver's limit

=== 3. Servlet-style filters (next is an explicit argument) ===
  [timing] start /orders
  [timing] end   /orders -> 200 OK orders list
  [timing] start /orders
  [timing] end   /orders -> 401 Unauthorized

=== 4. Logging levels (variant: every handler may act) ===
  log DEBUG:
    console <- DEBUG cache miss for key=42
  log ERROR:
    console <- ERROR payment service down
    app.log <- ERROR payment service down
    PAGER   <- ERROR payment service down
```

Walk through the output:

- Request 1 travels the full chain. Request 2 stops at auth — the rate limiter, validator and order code never run. Request 3 passes auth and the limiter (2 of 2) and stops at validation. Request 4 is the third request from alice, so the limiter stops it.
- `EXP-2` is escalated once and approved by the Manager. `EXP-3` is above every limit, so the chain gives an explicit "REJECTED" outcome rather than disappearing.
- The timing filter prints both before and after the rest of the chain. For the unauthenticated request, the auth filter wrote `401` (the HTTP status code for "not authenticated") and did not call `next`, so the endpoint never ran.
- A `DEBUG` message only reaches the console; an `ERROR` goes to all three.

Trade-offs

Pros	Cons
A new stage costs one class and one wiring line; no working class is edited (Open/Closed — open for extension, closed for modification).	To see the route a request takes you must read the wiring plus several files.
Every stage can be unit-tested (tested in isolation) on its own against a stub (a minimal stand-in) successor.	When a request is refused, you may have to step through several handlers to learn which one refused it.
Each route or environment can get its own chain, assembled at start-up.	No compiler or framework verifies that the chain is complete or correctly ordered.
The sender does not know which object will end up handling its request.	A handler that neither handles nor forwards makes the request vanish without any error.
	Each extra stage adds a call; rarely matters, but can on very hot paths (code that runs extremely often) with long chains.

The order of the stages is a design decision that no tool verifies. Auth before rate limiting means the limiter can count per *user*, and requests without valid credentials are turned away before they can use up a real user's quota (allowed number of requests). Rate limiting before auth means the limiter can only key on something like the IP address (the network address of the calling machine) — but it shields the (possibly expensive) auth step from floods (huge bursts of requests). Both are reasonable; they behave differently. **Put a comment with the reason beside the wiring** (see `Main`): the order is written down nowhere else.

When not to use it

- **Every request always goes through every stage in a fixed order.** Four calls in one method are clearer and have no wiring to get wrong.
- **Handlers need each other's results.** Then the stages are not independent: what you really have is a *pipeline* in which each step reshapes a value for the next. Call the steps one after another and pass an accumulating context object between them instead.
- **No terminal handler exists.** If the end of the chain can be reached with nothing done, failures become silent. Add an explicit terminal handler first.
- **The chain is long and on a very hot path**, where every extra call is measurable.

Common mistakes

1. **The swallowing bug.** A handler does its check and then just `return`s instead of calling `passOn`. No exception, no log; orders "sometimes don't appear". Fixes: give every path an explicit outcome (handled / rejected with reason / passed on); prefer the style where `next` is an explicit parameter (servlet filters, Express, Go `http.Handler`) so forgetting it is visible; unit-test each handler against a stub successor that remembers whether it got the request.
2. **No terminal handler.** Reaching the end should be an error or a clear "not handled" result — never a shrug.
3. **Wrong order, undocumented.** See the trade-offs above.
4. **Handlers that know about each other** (`new RateLimitHandler()` inside `AuthHandler`). The chain is then fixed in code again.
5. **Sharing a stateful chain object between requests.** The servlet-style `FilterChain` keeps a position, so it must be **one per request** (our `Main` creates a new one for each).

Real-world examples

- **Servlet filters**: `jakarta.servlet.Filter.doFilter(request, response, chain)` — calling `chain.doFilter(...)` *is* "pass on". **Spring Security** (Spring's authentication and authorisation library) is a chain of such filters (`SecurityFilterChain`).
- **Spring MVC** `HandlerInterceptor`: `preHandle` returns `false` to stop the request.
- **HTTP middleware** (code a web framework runs around every request; HTTP is the web's request/response protocol) in Express (`next()`), Django, Ruby's Rack, ASP.NET Core, Go (`func(http.Handler) http.Handler`, the usual way to structure a Go web service). **OkHttp** interceptors call `chain.proceed(request)`. **Netty**'s `ChannelPipeline` passes events from handler to handler.

- **In languages with first-class functions** (functions that can be stored and passed around like values) the chain is usually a list of `(request, next) => ...` functions rather than classes; JavaScript frameworks often build it by folding (combining a list into one value step by step) the list with `Array.reduce`, and in Python several decorators (`@` wrappers around a function) stacked on one view function (the function that handles a web request) form a chain written as syntax. Passing `next` explicitly removes the "forgot to pass on" bug by design.
- **`java.util.logging`**: a `LogRecord` is published to the logger's handlers and then, by default, to its parent logger's handlers; `setUseParentHandlers(false)` stops that. This is why setting a level or handler on a parent logger affects its children.
- **Exception handling**: each stack frame (one method call in progress) either catches the exception or lets it travel up — the language's own chain.
- **DOM event bubbling** (the DOM — Document Object Model — is the browser's tree of page elements): a click event climbs from the clicked element through its parent elements; any listener can call `stopPropagation()` to end the climb.
- **Approval workflows** (expenses, leave requests, loan approvals) where authority is based on amount or role, and **customer-support escalation** tiers.

In the interview

> **30-second answer:** "Chain of Responsibility splits a series of checks into separate handler objects, each linked only to its successor. The request moves down the line until some handler completes it or refuses it. Because only the wiring knows the order, adding, removing or reordering a rule is cheap. Servlet filters and HTTP middleware are the everyday examples. The classic bug is a handler that forgets to pass on and swallows the request, so I give every path an explicit outcome and put a terminal handler at the end."

Follow-ups:

- *How is it different from Decorator?* Both link objects that share one interface. With a **Decorator**, all wrappers run and the call is always delivered to the inner object; the point is to *add* behaviour. With **CoR**, a handler may **end** the request, and handlers further down never see it.
- *How is it different from Observer?* Observer tells **every** subscriber, and no subscriber can block the rest. CoR ends at the **first** handler that accepts the job.
- *How is it different from a pipeline?* A pipeline pushes a value through **all** of its steps and produces a transformed result. A chain *routes* a request to whichever handler takes it. Rule of thumb: if no stage can stop the flow, you have a pipeline.
- *Combine it with Command?* Yes — the request moving through the handlers is frequently itself a Command object (e.g. a job checked by validation and authorisation handlers before a worker runs it).
- *What if no handler can handle the request?* Put an explicit terminal handler (e.g. `NoHandlerFound`) that returns an error or logs, so failures are never silent.
- *How do you test it?* Test each handler with a fake next handler that records whether it was called and with what; then one integration test (a test of several parts working together) for the wiring order.
- *Where should rate limiting go, before or after auth?* Explain the trade-off above and say you would document the reason next to the wiring.

Self-check

1. **Q:** What does each handler hold a reference to? **A:** Only the next handler (through the abstract `Handler` type).
2. **Q:** Name the most common CoR bug. **A:** A handler that neither handles nor passes on — the request silently disappears.
3. **Q:** Why do modern frameworks pass `next` as a parameter? **A:** Because a missing `next(...)` call is then easy to spot in the handler's own body, and because the handler can do work both before and after the rest of the chain runs.
4. **Q:** Is a logging chain where every handler always passes on a "pure" CoR? **A:** Not strictly; since no handler can stop the flow it behaves like a pipeline. It is a common variant.
5. **Q:** When should you *not* use CoR? **A:** When every stage always runs in a fixed order, when stages depend on each other's results, or when there is no terminal handler.

TL;DR

A request walks a line of handlers; each handles, rejects, or passes on. The chain is data, so it is easy to change — but **order matters**, **always have a terminal handler**, and **never swallow a request**.

10.2 Command

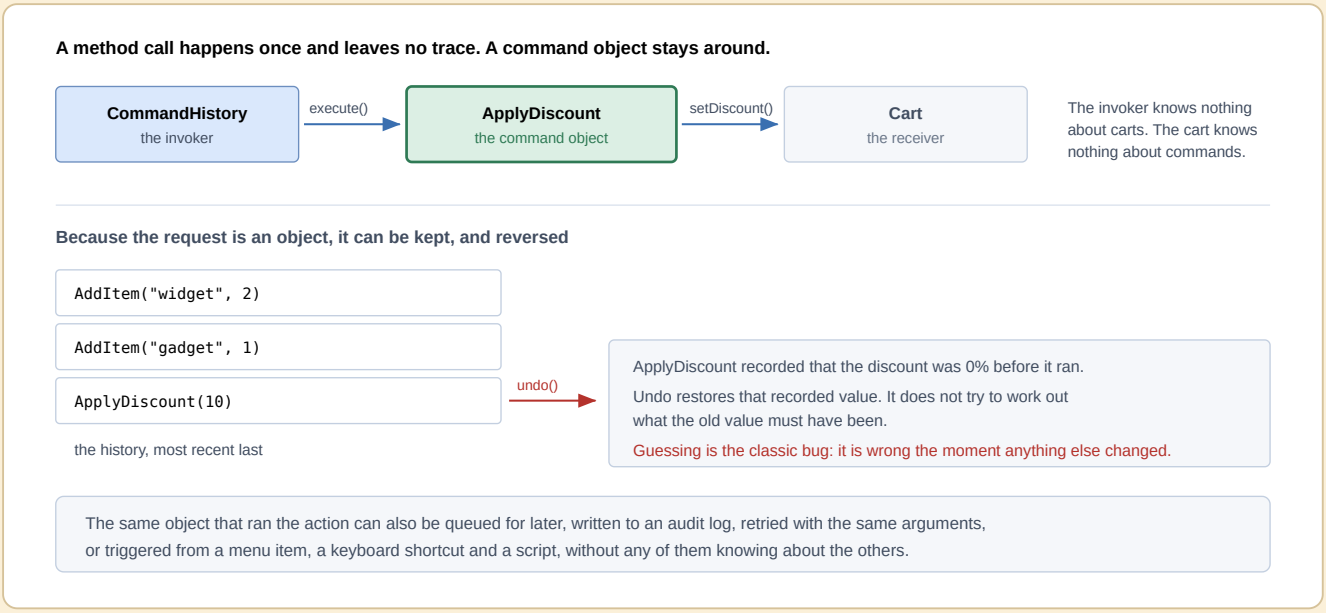


Figure — The invoker (CommandHistory) runs command objects against the receiver (Cart) and keeps them so they can be undone (course diagram). The figure assumes the discount was 0% before; our code deliberately starts at 5% to prove undo restores the recorded value.

What it is

Command packages a request ("add 2 widgets to the cart") as an **object** containing everything required to perform it later. Once the request is a value you can hold, you can **keep it in a list, put it on a queue (a first-in, first-out waiting line), write it to an audit log (a permanent record of who did what), run it again, hand it to a worker thread (a background thread — an independent path of execution — that takes jobs and runs them)** — and, when it also knows its own inverse, **reverse it**.

GoF intent, in plain words: *wrap a request in an object, so clients can be given different requests, requests can be queued or logged, and operations can be undone.*

Everyday analogy: a restaurant order slip. The waiter (invoker) does not cook; they write the order on a slip (command) and put it on the queue. The cook (receiver) acts on it later. The slip can be queued, prioritised, or cancelled.

The problem it solves

A plain method call such as `cart.setDiscount(10)` runs immediately and is forgotten the moment it returns. That is not enough once you need any of these:

- **Undo** — "reverse my last two changes".
- **A queue** — run the work later, on a worker thread, not in the request thread (the thread serving the user's request).
- **A log of intent** — an auditor asks which operations were *requested*, not just what state the data ended up in.
- **Retry** — run a failed action again with the same arguments.
- **One action, many triggers** — a menu item, a hot-key (keyboard shortcut) and an automation script should all fire the same "apply discount".

Command gives the request a body that outlives the call.

How it works

Interviewers usually want **three roles** named — **invoker**, **command**, **receiver** — plus the **client** that creates commands. GoF lists five participants in total: **Command** (the interface), **ConcreteCommand** (binds a receiver to

an action), **Client**, **Invoker** and **Receiver**:

Role	Job	In our code
Command	Interface with `execute()` (and optionally `undo()`).	`Command`, `UndoableCommand`
Concrete command	Stores its receiver and arguments; can perform (and reverse) exactly one operation.	`AddItemCommand`, `ApplyDiscountCommand`, `MacroCommand`
Receiver	The object whose state is really modified. Has no idea commands exist.	`Cart`
Invoker	Triggers commands and keeps a record of them. Has no idea carts exist.	`CommandHistory`, `ExecutorService`
Client	Builds a concrete command and gives it its receiver.	`Main`

Steps:

1. The client creates a concrete command, passing the **receiver** and the **arguments**.
2. It hands the command to the **invoker**.
3. The invoker calls `execute()` and, if the command is undoable, pushes it on an **undo stack** (a stack is a last-in, first-out pile: you add and remove only at the top).
4. **Undo** = pop the last command and call `undo()`; push it on a **redo stack**.
5. **Redo** = pop from redo, `execute()` again, push back on undo. **Running a fresh command empties the redo stack**: the branch of history you might have re-applied has been replaced.

Structure (UML)

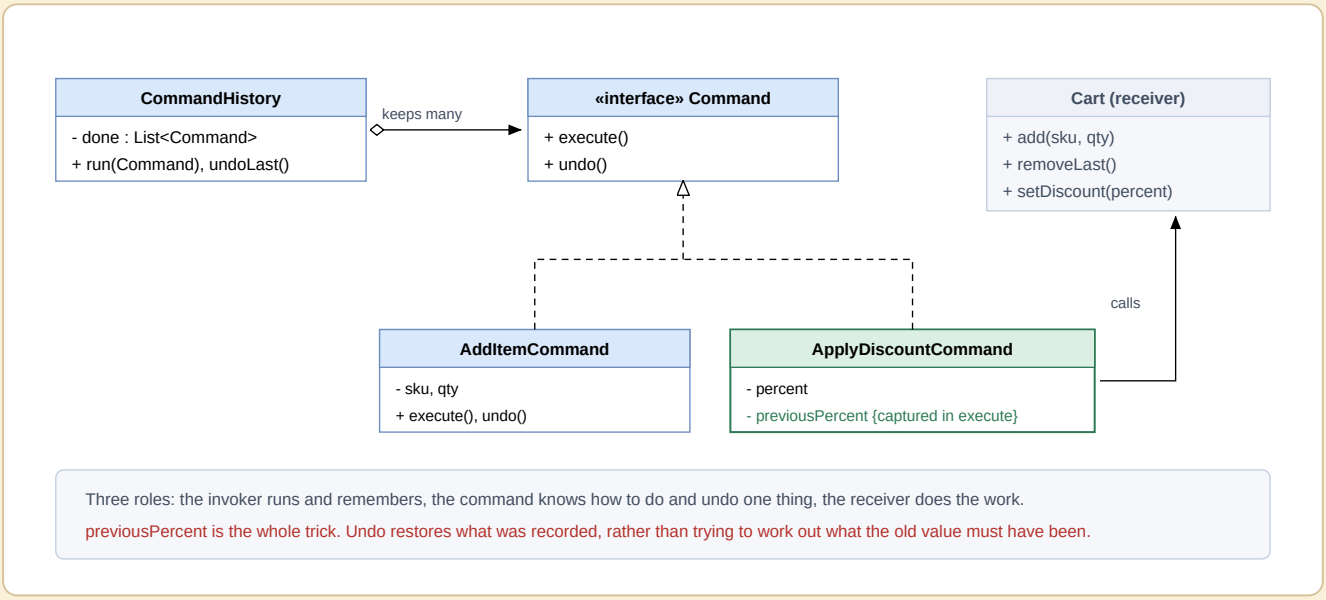
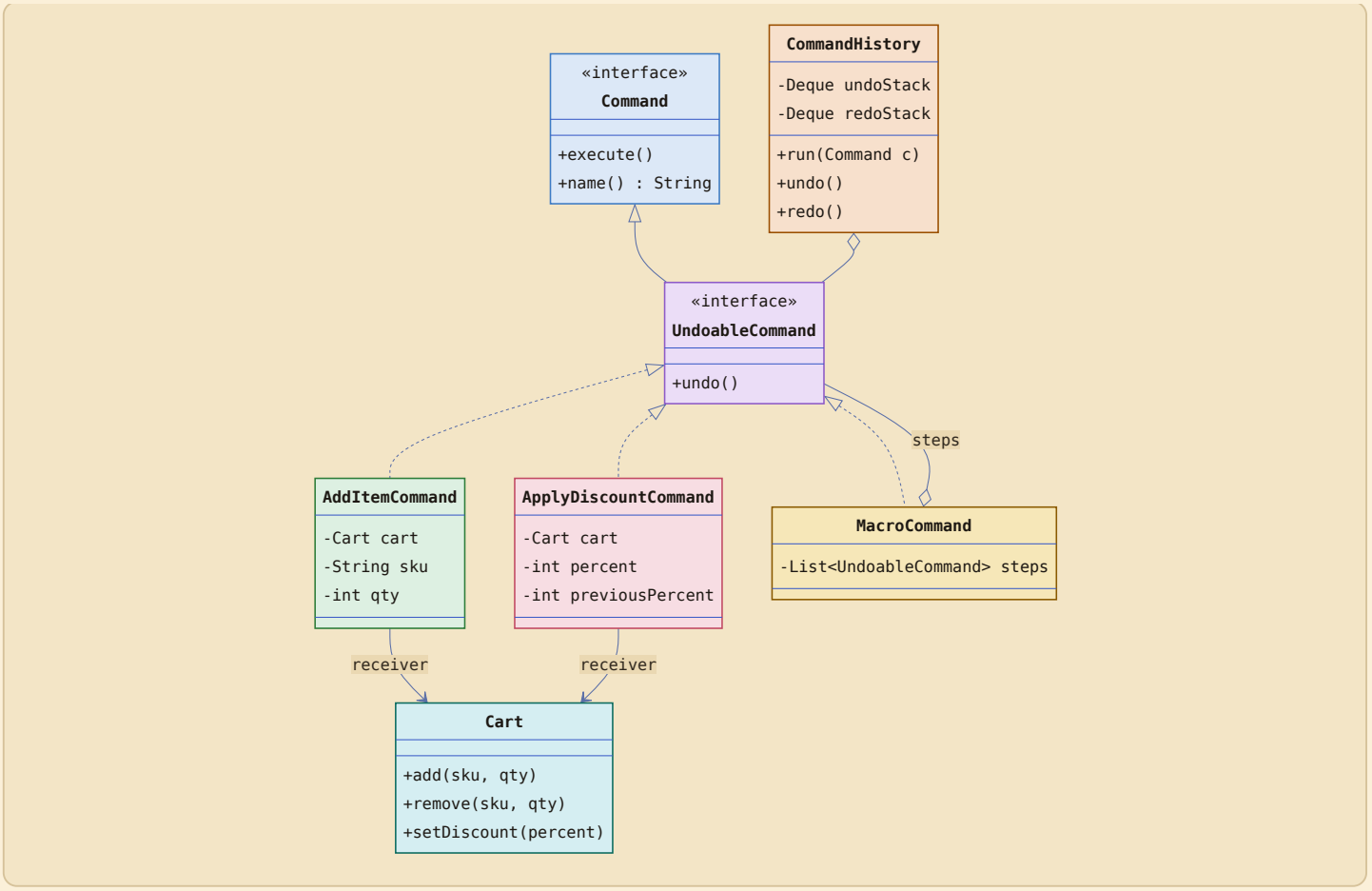
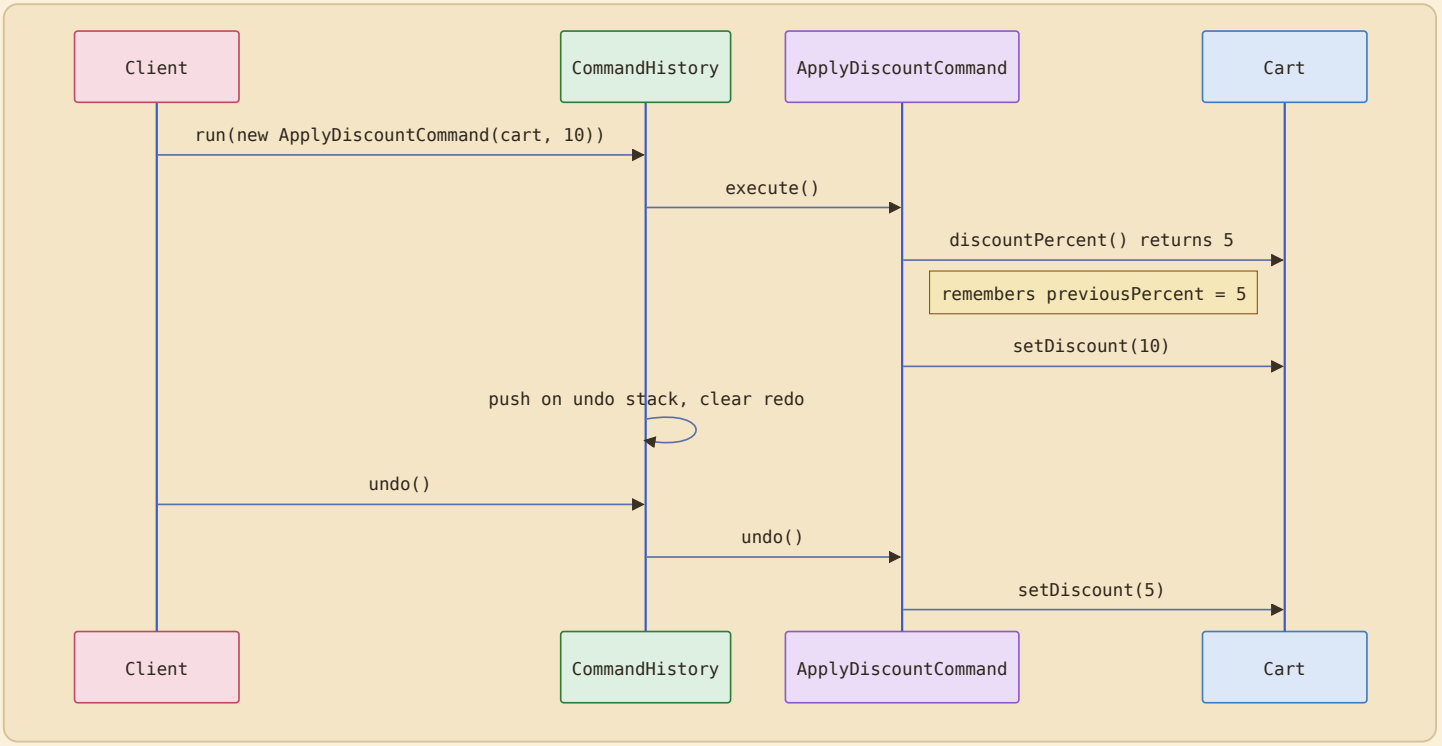


Figure — Class diagram: Command interface with execute/undo, two concrete commands, Cart as receiver, CommandHistory as invoker (course diagram; its `undoLast`/`removeLast` are named `undo`/`remove` in our code)

In the diagram above, each command keeps its own reference to the `Cart`. It has to: when it finally runs, the code that created it may have finished long ago, so the target object must travel inside the command along with the arguments. Also note `ApplyDiscountCommand.previousPercent` — the single field that makes its undo correct. And note that `CommandHistory` never mentions `Cart`; hand it any other commands and it still works, which is why one history can serve a whole application.

Our Java design:





Java example

Key design points in `lld.ch10.command`:

- `Command` has **one abstract method**, so any lambda (a short unnamed function) or `Runnable`-like block (`Runnable` is Java's built-in "code to run" interface) can be a command. Only commands that can *honestly* be reversed implement `UndoableCommand`. `CommandHistory` keeps irreversible commands out of the undo stack.
- `ApplyDiscountCommand` **captures the old value in** `execute()` and restores exactly that in `undo()`. The cart deliberately starts at 5%, to show that "undo = set to 0" would be wrong.

- `MacroCommand` is a command made of commands (a **Composite** of commands): it executes steps in order and undoes them in **reverse** order.
- Section 4 of `Main` shows the JDK's own version: `Runnable` is the command interface and an `ExecutorService` (Java's thread-pool service for running tasks) is the invoker that runs commands on a worker thread.

```
// File: code/lld/ch10/command/Command.java
package lld.ch10.command;

/** A request turned into an object. One abstract method, so a lambda is a Command. */
@FunctionalInterface
public interface Command {
    void execute();

    default String name() { return getClass().getSimpleName(); }
}
```

```
// File: code/lld/ch10/command/UndoableCommand.java
package lld.ch10.command;

/** Only commands that can honestly be reversed implement this. */
public interface UndoableCommand extends Command {
    void undo();
}
```

```
// File: code/lld/ch10/command/Cart.java
package lld.ch10.command;

import java.util.ArrayList;
import java.util.List;

/** The RECEIVER: knows how to change itself and nothing about commands. */
public final class Cart {
    public record Line(String sku, int qty) {}

    private final List<Line> lines = new ArrayList<>();
    private int discountPercent = 0;

    public void add(String sku, int qty) { lines.add(new Line(sku, qty)); }

    /** Removes the most recent line matching sku and qty. */
    public void remove(String sku, int qty) {
        for (int i = lines.size() - 1; i >= 0; i--) {
            if (lines.get(i).equals(new Line(sku, qty))) {
                lines.remove(i);
                return;
            }
        }
    }

    public int discountPercent() { return discountPercent; }
    public void setDiscount(int percent) { this.discountPercent = percent; }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder("Cart[");
        for (int i = 0; i < lines.size(); i++) {
            if (i > 0) sb.append(", ");
            sb.append(lines.get(i).sku()).append(" x").append(lines.get(i).qty());
        }
        return sb.append("] discount=").append(discountPercent).append('%').toString();
    }
}
```

```java
// File: code/lld/ch10/command/AddItemCommand.java
package lld.ch10.command;

/** Adds a line; undo removes exactly that line. */
final class AddItemCommand implements UndoableCommand {
    private final Cart cart;      // the command carries its receiver...
    private final String sku;     // ...and all the arguments it needs to run later
    private final int qty;

    AddItemCommand(Cart cart, String sku, int qty) {
        this.cart = cart;
        this.sku = sku;
        this.qty = qty;
    }

    @Override public void execute() { cart.add(sku, qty); }
    @Override public void undo()    { cart.remove(sku, qty); }
    @Override public String name()  { return "AddItem(" + sku + ", " + qty + ")"; }
}
```

```java
// File: code/lld/ch10/command/ApplyDiscountCommand.java
package lld.ch10.command;

/** Sets a discount; captures the previous value during execute so undo RESTORES, never guesses. */
final class ApplyDiscountCommand implements UndoableCommand {
    private final Cart cart;
    private final int percent;
    private int previousPercent;          // filled in by execute()

    ApplyDiscountCommand(Cart cart, int percent) {
        this.cart = cart;
        this.percent = percent;
    }

    @Override
    public void execute() {
        previousPercent = cart.discountPercent();   // capture BEFORE changing
        cart.setDiscount(percent);
    }

    @Override public void undo()   { cart.setDiscount(previousPercent); }
    @Override public String name() { return "ApplyDiscount(" + percent + "%)"; }
}
```

```java
// File: code/lld/ch10/command/MacroCommand.java
package lld.ch10.command;

import java.util.List;

/** A command made of commands. Runs in order; undoes in REVERSE order. */
final class MacroCommand implements UndoableCommand {
    private final String label;
    private final List<UndoableCommand> steps;

    MacroCommand(String label, List<UndoableCommand> steps) {
        this.label = label;
        this.steps = List.copyOf(steps);
    }

    @Override public void execute() { steps.forEach(Command::execute); }

    @Override
    public void undo() {
        for (int i = steps.size() - 1; i >= 0; i--) steps.get(i).undo();
    }

    @Override public String name() { return "Macro:" + label; }
}
```

```java
// File: code/lld/ch10/command/CommandHistory.java
package lld.ch10.command;

import java.util.ArrayDeque;
import java.util.Deque;

/** The INVOKER: runs commands and remembers them. Knows nothing about carts. */
public final class CommandHistory {
    private final Deque<UndoableCommand> undoStack = new ArrayDeque<>();
    private final Deque<UndoableCommand> redoStack = new ArrayDeque<>();

    public void run(Command command) {
        command.execute();
        if (command instanceof UndoableCommand u) {
            undoStack.push(u);
            redoStack.clear();     // a new action destroys the "future" you could have redone
            System.out.println("  ran  " + command.name());
        } else {
            System.out.println("  ran  " + command.name() + " (irreversible: kept out of history)");
        }
    }

    public void undo() {
        if (undoStack.isEmpty()) { System.out.println("  nothing to undo"); return; }
        UndoableCommand c = undoStack.pop();
        c.undo();
        redoStack.push(c);
        System.out.println("  undo " + c.name());
    }

    public void redo() {
        if (redoStack.isEmpty()) { System.out.println("  nothing to redo"); return; }
        UndoableCommand c = redoStack.pop();
        c.execute();
        undoStack.push(c);
        System.out.println("  redo " + c.name());
    }
}
```

```java
// File: code/lld/ch10/command/Main.java
package lld.ch10.command;

import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class Main {
    public static void main(String[] args) throws InterruptedException {
        Cart cart = new Cart();
        CommandHistory history = new CommandHistory();

        System.out.println("=== 1. Undo / redo ===");
        cart.setDiscount(5);   // the cart does NOT start at 0%: a guessing undo would be wrong
        history.run(new AddItemCommand(cart, "widget", 2));
        history.run(new AddItemCommand(cart, "gadget", 1));
        history.run(new ApplyDiscountCommand(cart, 10));
        System.out.println("  " + cart);
        history.undo();
        history.undo();
        System.out.println("  " + cart + "   <- discount restored to the recorded 5%, not 0%");
        history.redo();
        System.out.println("  " + cart);
        history.run(new AddItemCommand(cart, "cable", 3));   // clears the redo stack
        history.redo();

        System.out.println("\n=== 2. Macro command ===");
        UndoableCommand starterKit = new MacroCommand("starter-kit", List.of(
            new AddItemCommand(cart, "keyboard", 1),
            new AddItemCommand(cart, "mouse", 1),
            new ApplyDiscountCommand(cart, 15)));
        history.run(starterKit);
        System.out.println("  " + cart);
        history.undo();                                      // one undo reverses all three steps
        System.out.println("  " + cart);

        System.out.println("\n=== 3. Irreversible command (no undo offered) ===");
        Command sendReceipt = new Command() {
            @Override public void execute() { System.out.println("  [email] receipt sent for " + cart); }
            @Override public String name()  { return "SendReceipt"; }
        };
        history.run(sendReceipt);

        System.out.println("\n=== 4. Commands on a queue: Runnable + ExecutorService (the invoker) ===");
        List<Runnable> jobs = List.of(
            () -> System.out.println("  [worker] resize image 1"),
            () -> System.out.println("  [worker] resize image 2"),
            () -> System.out.println("  [worker] send newsletter"));
        System.out.println("  queued " + jobs.size() + " jobs");
        ExecutorService worker = Executors.newSingleThreadExecutor();   // one thread => FIFO order
        jobs.forEach(worker::submit);
        worker.shutdown();
        worker.awaitTermination(5, TimeUnit.SECONDS);
        System.out.println("  all jobs done");
    }
}
```

Output:

```
=== 1. Undo / redo ===
  ran  AddItem(widget, 2)
  ran  AddItem(gadget, 1)
  ran  ApplyDiscount(10%)
  Cart[widget x2, gadget x1] discount=10%
  undo ApplyDiscount(10%)
  undo AddItem(gadget, 1)
  Cart[widget x2] discount=5%   <- discount restored to the recorded 5%, not 0%
  redo AddItem(gadget, 1)
  Cart[widget x2, gadget x1] discount=5%
  ran  AddItem(cable, 3)
  nothing to redo

=== 2. Macro command ===
  ran  Macro:starter-kit
  Cart[widget x2, gadget x1, cable x3, keyboard x1, mouse x1] discount=15%
  undo Macro:starter-kit
  Cart[widget x2, gadget x1, cable x3] discount=5%

=== 3. Irreversible command (no undo offered) ===
  [email] receipt sent for Cart[widget x2, gadget x1, cable x3] discount=5%
  ran  SendReceipt (irreversible: kept out of history)

=== 4. Commands on a queue: Runnable + ExecutorService (the invoker) ===
  queued 3 jobs
  [worker] resize image 1
  [worker] resize image 2
  [worker] send newsletter
  all jobs done
```

Undo must restore, not recompute

It is tempting to write a discount undo as "reset to 0" or "calculate what the old value probably was". Both are **guesses**. If the cart began at 5%, or some other command touched the discount meanwhile, the user ends up with a value that never existed. The rule: `execute` **must record whatever** `undo` **will need** — normally just the fields it is about to overwrite, not a copy of the whole object.

Undo history is strictly last-in, first-out. Commands are reversed newest first: the sequence `AddItem, AddItem, ApplyDiscount` is undone as `ApplyDiscount`, then the second `AddItem`, then the first. Reversing them in any other order can produce states that never existed, hence a stack, never a list you can index into at random.

When undo is a lie. Some effects escape your system: an email that was sent, a card that was charged, a call made to a partner's API (the interface a service exposes for other programs to call). An `undo()` on those promises something impossible, which is worse than having no undo at all. Two honest options: keep them out of the history (our `SendReceipt`), or use **compensation** — a *new* action that produces a similar end state, like a **refund** for a charge. A refund is not an undo: both events stay in the record.

Trade-offs

Pros	Cons
Since the request lives on after the call, undo/redo, queues, audit logs, retries and scheduling all become possible.	Every action needs its own class, so the number of classes tracks the number of actions.
The invoker has no knowledge of what it runs, so a single history works for all commands.	Each command has to record enough to run later and to reverse itself; subtle mistakes are easy.
One action object can back a menu entry, a hot-key, a script and an API endpoint.	Extra hop when reading code: to learn what a button does you first find the command it builds.
Commands can be grouped into macro commands.	A single faulty `undo` corrupts the whole history.

When not to use it

- **The call is simple and immediate.** A `SaveCommand` class that nobody ever undoes, queues or logs is extra code with no benefit.
- **Undo is not really possible.** Use compensation and call it that.
- **You only need deferral.** A lambda (`Runnable`) already is a one-method command. Promote it to a class once you need `undo`, a readable name in logs, or serialization (turning it into bytes or text so it can be stored or sent).

- **Commands must outlive the process.** They then have to be serialized, which drags in format versioning; a durable queue (one that survives restarts — see System Design Fundamentals) or an event log (Kafka — a durable, distributed message log — or a jobs table) handles that better than hand-rolled command classes.

Common mistakes

1. **Recomputing in `undo`** instead of restoring captured state.
2. **Forgetting to clear the redo stack** when a new command runs.
3. **Putting business logic in the invoker** (`if (cmd instanceof AddItem) ...` — `instanceof` checks an object's concrete type). The invoker should only run and remember.
4. **Commands that capture a mutable object by reference** (mutable = changeable after creation) and then read it later, when it has changed. Capture values.
5. **Unbounded history** — cap the depth, as editors do.

Real-world examples

- **Undo in editors**: word processors, IDEs (integrated development environments — code editors such as IntelliJ), Photoshop. The JDK has `javax.swing.undo.UndoManager` with `UndoableEdit` objects that have `undo()`/`redo()`.
- **`Runnable` / `Callable`** are the JDK's command interfaces; `ExecutorService`, `ScheduledExecutorService` and `Thread` are invokers.
- **GUI actions** (GUI = graphical user interface): Swing's (Java's desktop UI toolkit) `javax.swing.Action` lets one action back a menu item, a toolbar button and a shortcut; WPF's `ICommand` in .NET plays the same role.
- **Event sourcing** (Chapter 11; storing every change as an event and rebuilding state by replaying them): recording the *requested changes* instead of only the resulting state is the command idea applied to a whole system.
- **Job queues**: a Sidekiq/Celery (background-job libraries for Ruby and Python) job, a Spring `@Async` task, a message on a Kafka/RabbitMQ (message brokers) queue that asks a worker to "send invoice #42" — a serialized command.
- **Database migrations** (versioned scripts that change a database's structure) with `up`/`down` (e.g. Liquibase changesets with rollback) mirror `execute`/`undo`.
- **Redux** (a JavaScript state-management library) actions are command objects: the store dispatches them (invoker role) and reducers apply them to state (receiver role); Redux DevTools' time travel replays them.
- **In other languages**: in Python, a callable object (one defining `__call__`) with an extra `undo` method works well, and `functools.partial` can bind the arguments; in C#, UI frameworks ship an `ICommand` interface, and a `record` gives you an immutable (unchangeable after creation) command in a single line.
- **Transactional outbox / task tables** (the job is written to a database table in the same transaction — the same all-or-nothing database update — as the data change, so it cannot be lost) in food-delivery or e-commerce back ends store "commands" (send SMS, notify restaurant) to be run reliably by workers.

In the interview

> **30-second answer:** "Command wraps a request, together with its receiver and arguments, in an object — so the request can be kept, queued, logged, retried or reversed. Three roles matter: the invoker triggers and records commands without knowing what they touch, each command can perform and reverse a single operation, and the receiver holds the state that actually changes. For undo, `execute` must capture the old value and `undo` restores it; history is a stack, and a new command clears redo. Irreversible actions like charging a card get compensation, not undo. In Java, `Runnable` plus an `ExecutorService` is the everyday form."

Follow-ups:

- *Command vs Strategy?* Each puts behaviour inside an object. A **Strategy** answers *how* a job is done and is picked and replaced by the caller. A **Command** captures *which* job to do, arguments included, and is held on to so it can run later or be reversed.
- *Command vs a plain lambda?* A one-method closure (a function that remembers variables from where it was created) already is a command. Write a class only when you also need `undo`, a readable log name, or to serialize it.
- *Command vs Memento for undo?* Command stores the **action** plus a small piece of state; undo runs the inverse. Memento stores the **whole state**; undo puts a snapshot back. Command is cheaper per step; Memento is always correct. Real systems combine them (snapshot occasionally, replay commands in between).

- *How do you implement redo?* A second stack; undo moves a command to it; redo moves it back after re-executing; a new command clears it.
- *How do you make a command queue thread-safe (correct when many threads use it at once)?* Use a `BlockingQueue` (a queue whose `take()` waits until an item arrives) or an `ExecutorService`; make commands immutable apart from captured undo state.
- *How would you persist commands?* Serialize them (JSON — a common text format for structured data — with a type field and a version), store in a durable queue or table, make them **idempotent** (running one twice has the same effect as running it once — see System Design Fundamentals) so a retry is safe.

Self-check

1. **Q:** What must a command carry? **A:** Its receiver and all arguments needed to run later (and any state needed for undo).
2. **Q:** Why does `ApplyDiscountCommand` store `previousPercent`? **A:** So undo restores the real previous value instead of guessing.
3. **Q:** What happens to the redo stack when a new command runs? **A:** It is cleared.
4. **Q:** How do you "undo" a card charge? **A:** You don't; you issue a compensating action (a refund) and keep both in the record.
5. **Q:** Which JDK interface is a command? **A:** `Runnable` (and `Callable`).

TL;DR

Make the request an object: **invoker** runs it, **command** knows do/undo, **receiver** changes. Undo **restores captured state**, history is a **stack**, and irreversible actions get **compensation**. A lambda is a command when you only need deferral.

10.3 Iterator

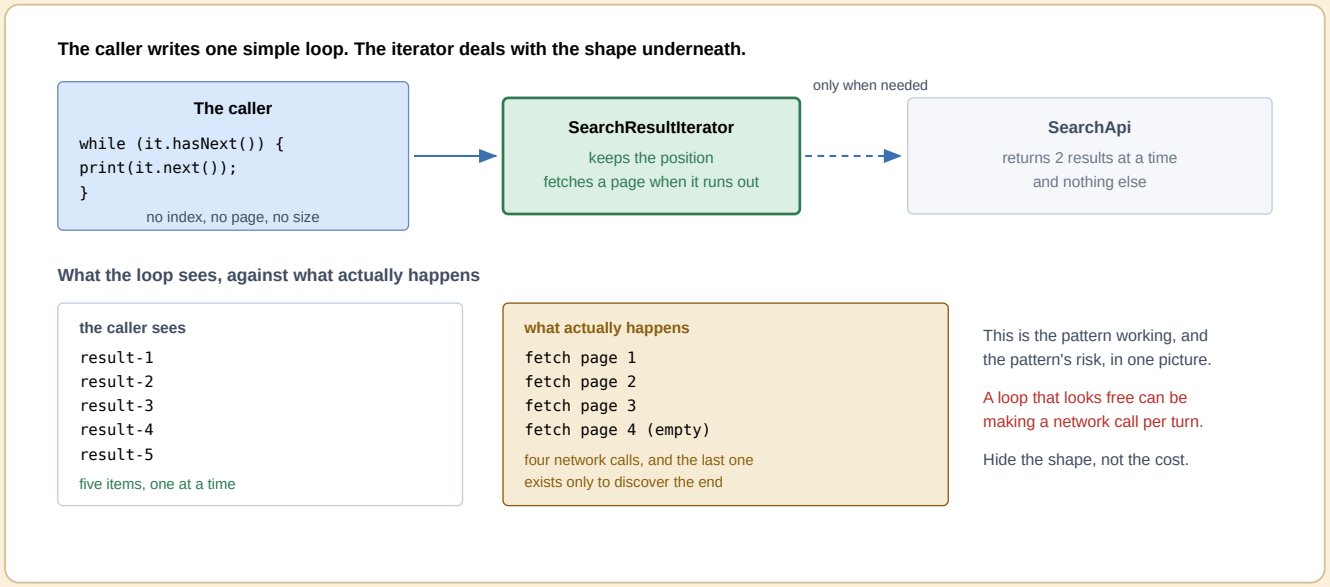


Figure — The caller writes one simple loop; the iterator fetches pages from the API only when needed (course diagram)

What it is

Iterator lets you visit the elements of a collection **one at a time, in order, without knowing how the collection stores them** — array, linked list (elements chained together by references), tree, or a remote API that returns two results per page.

GoF intent, in plain words: *give sequential access to the elements of an aggregate (GoF's word for a collection) without exposing its internal representation.* GoF participants: **Iterator** (interface for accessing and traversing), **ConcreteIterator** (implements it and tracks the current position), **Aggregate** (interface for creating an iterator) and **ConcreteAggregate** (returns the right ConcreteIterator).

Everyday analogy: a TV remote's "next channel" button. You do not need to know how the channels are stored in the TV; you just press *next* until there are no more.

The problem it solves

- Code that loops with `get(i)` quietly assumes the data sits in an array. Move the data into a linked list, a tree or a paginated (returns results one page at a time) remote service, and each of those loops must be rewritten.
- Many sources **cannot be indexed at all**: a tree has several valid visiting orders (depth-first — follow one branch all the way down before the next; breadth-first — level by level); a graph (nodes joined by edges, possibly with loops) must remember which nodes were already seen; a search API hands out pages. When each caller writes its own traversal, each caller also re-implements paging — and some implement it wrongly.

Iterator moves the walking logic **into a separate object** that answers two questions: *are any elements left?* (`hasNext`) and *hand me the following element* (`next`).

Key detail: the current position is stored inside the iterator object, never in the collection. So two traversals of one collection can be in progress together without disturbing each other, and every call to `iterator()` must build a **new** iterator instead of rewinding a shared cursor (a marker of the current position).

How it works

1. The **aggregate** (the collection) has a method `iterator()` that creates a new iterator. In Java it implements `Iterable<T>`.
2. The **iterator** implements `Iterator<T>`: `hasNext()` and `next()` (and optionally `remove()`).
3. The iterator keeps its own **cursor** (index, stack of tree nodes, current page...).
4. The client loops: `while (it.hasNext()) use(it.next());` — or just `for (T x : collection)`, which the Java compiler turns into exactly that.

Structure (UML)

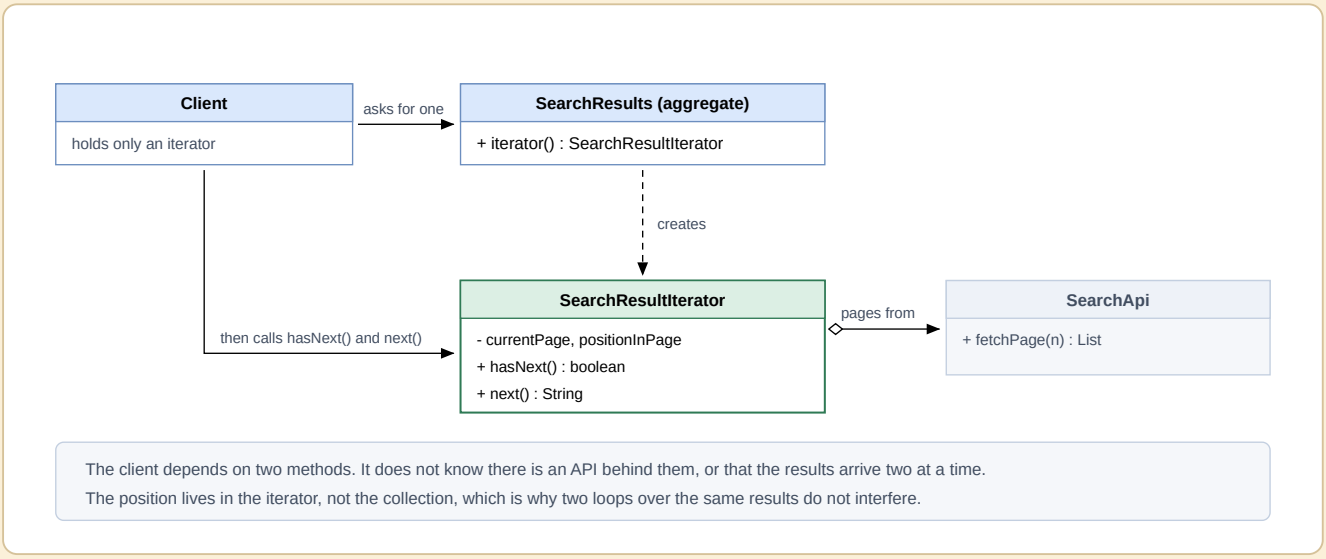
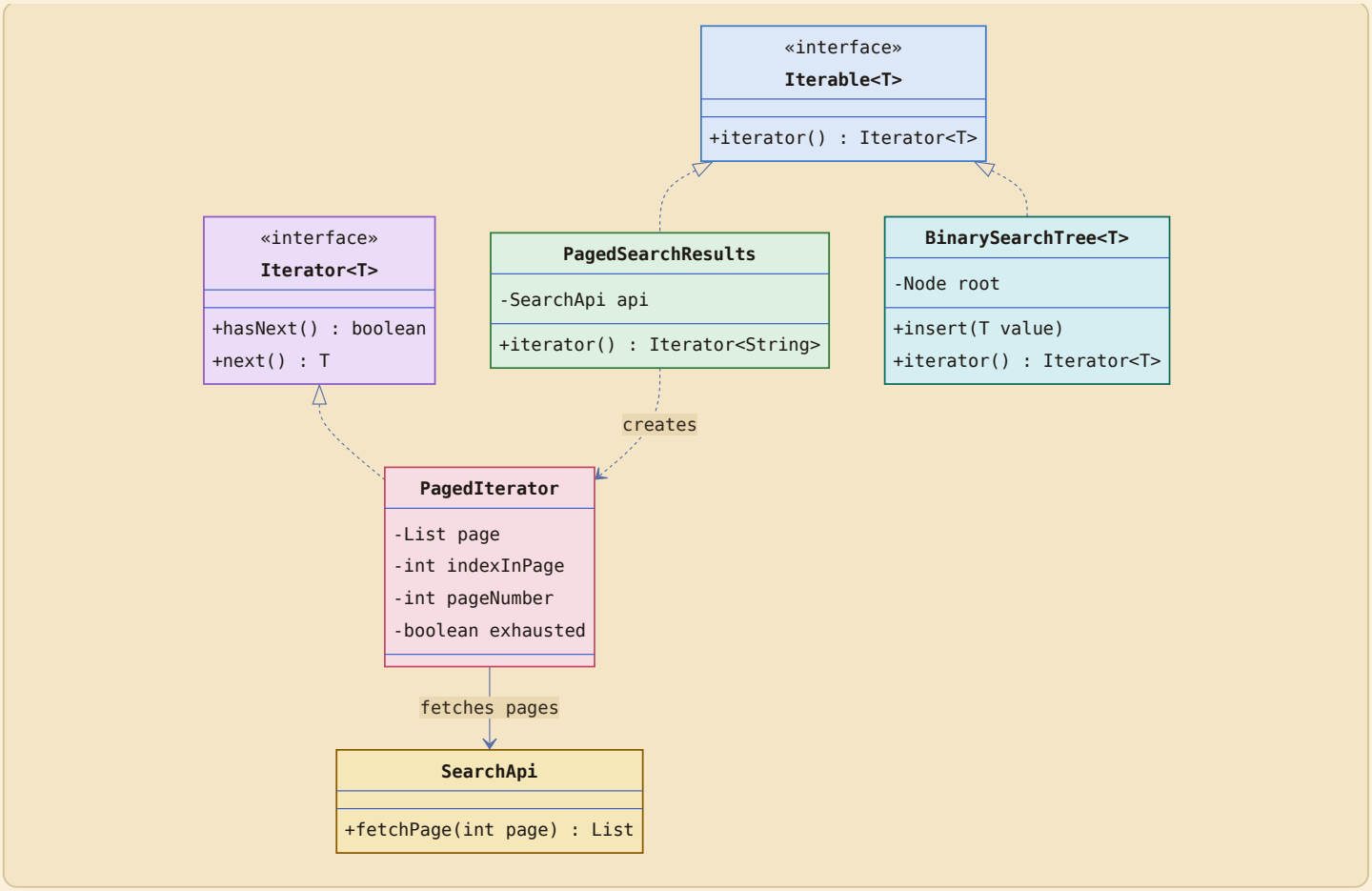


Figure — Class diagram: SearchResults (aggregate) hands out a SearchResultIterator, which calls SearchApi page by page; the client only calls hasNext/next (course diagram)

In the diagram above, `hasNext` does real work: once the buffered page runs out, it asks the API for another page. A *lazy* iterator (one that produces elements only on demand) commonly behaves like this — so never assume `hasNext` is cheap or has no side effects (effects beyond returning a value, such as a network call).

Our Java design:



Java example

`lld.ch10.iterator` shows:

1. **A paged remote source behind a plain for-each.** `PagedSearchResults implements Iterable<String>`, so `for (String r : results)` just works. The inner `PagedIterator` fetches a page only when the current one is used up, and follows the `Iterator` contract (the documented rules every implementation must obey) (`next()` throws `NoSuchElementException` when empty).
2. **A tree iterator.** `BinarySearchTree` hides an in-order walk (left subtree, then the node, then the right subtree — which yields sorted order) behind `hasNext/next` using an explicit stack; two iterators over the same tree run independently.
3. **External vs internal iteration.** *External*: the **client** drives (`hasNext/next`) and can stop early with `break`. *Internal*: the **collection** drives and calls your code for each element (`forEach(lambda)`, streams — Java's pipeline API for processing collections).
4. **Fail-fast vs fail-safe.** Structurally changing an `ArrayList` (adding or removing) while a for-each loop is walking it *usually* makes the next `next()` throw `ConcurrentModificationException`. It is only **best-effort**: the demo also removes the second-to-last element, after which `hasNext()` simply returns `false` — no exception, and the last element is silently never visited. The safe way is `Iterator.remove()` (or `removeIf`). `CopyOnWriteArrayList` iterates over a **snapshot**, so it never throws — but the loop does not see the new elements. ("Fail-safe" is an informal name; the Javadoc (Java's official API documentation) talks about *snapshot* and *weakly consistent* iterators.)
5. `Spliterator` (Java 8): an iterator that can **split** itself (`trySplit`) so parallel streams can process parts on different threads. It reports **characteristics** such as `SIZED` and `ORDERED`.

```java
// File: code/lld/ch10/iterator/SearchApi.java
package lld.ch10.iterator;

import java.util.List;

/** A remote source that returns two results per page and knows nothing about iteration. */
final class SearchApi {
    private static final List<String> ALL = List.of("result-1", "result-2", "result-3", "result-4", "result-5");
    private static final int PAGE_SIZE = 2;
    private int calls = 0;

    List<String> fetchPage(int page) {          // pages are 1-based
        calls++;
        System.out.println("    [api] fetching page " + page);
        int from = (page - 1) * PAGE_SIZE;
        if (from >= ALL.size()) return List.of();
        return ALL.subList(from, Math.min(from + PAGE_SIZE, ALL.size()));
    }

    int calls() { return calls; }
}
```

```java
// File: code/lld/ch10/iterator/PagedSearchResults.java
package lld.ch10.iterator;

import java.util.Iterator;
import java.util.List;
import java.util.NoSuchElementException;

/**
 * The AGGREGATE. Implements Iterable, so the enhanced for-loop works on it.
 * Its only job is to hand out a FRESH iterator each time.
 */
final class PagedSearchResults implements Iterable<String> {
    private final SearchApi api;

    PagedSearchResults(SearchApi api) { this.api = api; }

    @Override
    public Iterator<String> iterator() { return new PagedIterator(); }

    /** The ITERATOR. The position lives here, not in the collection. */
    private final class PagedIterator implements Iterator<String> {
        private List<String> page = List.of();
        private int indexInPage = 0;
        private int pageNumber = 0;
        private boolean exhausted = false;

        @Override
        public boolean hasNext() {                // NOT a passive question: may hit the network
            if (indexInPage < page.size()) return true;
            if (exhausted) return false;
            page = api.fetchPage(++pageNumber);
            indexInPage = 0;
            if (page.isEmpty()) {
                exhausted = true;
                return false;
            }
            return true;
        }

        @Override
        public String next() {
            if (!hasNext()) throw new NoSuchElementException();   // the Iterator contract
            return page.get(indexInPage++);
        }
    }
}
```

```java
// File: code/lld/ch10/iterator/BinarySearchTree.java
package lld.ch10.iterator;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Iterator;
import java.util.NoSuchElementException;

/** A tree has no index. The iterator hides an in-order walk behind hasNext/next. */
final class BinarySearchTree<T extends Comparable<T>> implements Iterable<T> {
    private static final class Node<T> {
        final T value;
        Node<T> left, right;
        Node(T value) { this.value = value; }
    }

    private Node<T> root;

    void insert(T value) {
        Node<T> fresh = new Node<>(value);
        if (root == null) { root = fresh; return; }
        Node<T> cur = root;
        while (true) {
            if (value.compareTo(cur.value) < 0) {
                if (cur.left == null) { cur.left = fresh; return; }
                cur = cur.left;
            } else {
                if (cur.right == null) { cur.right = fresh; return; }
                cur = cur.right;
            }
        }
    }

    @Override
    public Iterator<T> iterator() {
        return new Iterator<>() {
            // The stack holds the path still to visit: this IS the iterator's position.
            private final Deque<Node<T>> stack = new ArrayDeque<>();
            { pushLeft(root); }

            private void pushLeft(Node<T> n) {
                for (; n != null; n = n.left) stack.push(n);
            }

            @Override public boolean hasNext() { return !stack.isEmpty(); }

            @Override
            public T next() {
                if (stack.isEmpty()) throw new NoSuchElementException();
                Node<T> n = stack.pop();
                pushLeft(n.right);
                return n.value;
            }
        };
    }
}
```

```java
// File: code/lld/ch10/iterator/Main.java
package lld.ch10.iterator;

import java.util.ArrayList;
import java.util.ConcurrentModificationException;
import java.util.Iterator;
import java.util.List;
import java.util.Spliterator;
import java.util.concurrent.CopyOnWriteArrayList;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== 1. Paged remote results behind a plain for-each ===");
        SearchApi api = new SearchApi();
        for (String r : new PagedSearchResults(api)) {
            System.out.println("  loop sees " + r);
        }
        System.out.println("  5 items, " + api.calls() + " network calls (the last one only discovers the end)");

        System.out.println("\n=== 2. Two independent iterators over one tree ===");
        BinarySearchTree<Integer> tree = new BinarySearchTree<>();
        for (int v : new int[] {50, 30, 70, 20, 40, 60, 80}) tree.insert(v);
        Iterator<Integer> a = tree.iterator();
        Iterator<Integer> b = tree.iterator();
        System.out.println("  a: " + a.next() + ", " + a.next() + ", " + a.next());
        System.out.println("  b: " + b.next() + "   <- b starts from the beginning: position lives in the iterator");
        StringBuilder all = new StringBuilder();
        for (int v : tree) all.append(v).append(' ');
        System.out.println("  in-order: " + all.toString().trim());

        System.out.println("\n=== 3. External vs internal iteration ===");
        List<String> names = List.of("ann", "bob", "cy");
        Iterator<String> it = names.iterator();               // external: the CLIENT drives
        while (it.hasNext()) {
            String n = it.next();
            if (n.equals("bob")) { System.out.println("  external: found bob, stopping early"); break; }
        }
        names.forEach(n -> System.out.println("  internal: " + n)); // internal: the COLLECTION drives

        System.out.println("\n=== 4. Fail-fast vs fail-safe ===");
        List<String> tags = new ArrayList<>(List.of("java", "tmp", "lld", "tmp"));
        try {
            for (String t : tags) {
                if (t.equals("tmp")) tags.remove(t);          // structural change behind the iterator's back
            }
        } catch (ConcurrentModificationException e) {
            System.out.println("  fail-fast: ConcurrentModificationException");
        }
        List<String> quiet = new ArrayList<>(List.of("java", "tmp", "lld"));
        List<String> visited = new ArrayList<>();
        for (String t : quiet) {
            visited.add(t);
            if (t.equals("tmp")) quiet.remove(t);             // removes the SECOND-TO-LAST element
        }
        System.out.println("  best-effort only: no exception, visited " + visited + ", 'lld' silently skipped");
        tags = new ArrayList<>(List.of("java", "tmp", "lld", "tmp"));
        for (Iterator<String> i = tags.iterator(); i.hasNext(); ) {
            if (i.next().equals("tmp")) i.remove();           // the safe way: remove through the iterator
        }
        System.out.println("  after Iterator.remove: " + tags);
        List<String> cow = new CopyOnWriteArrayList<>(List.of("x", "y"));
        for (String s : cow) cow.add(s + "!");                  // iterates a snapshot: no exception
        System.out.println("  fail-safe CopyOnWriteArrayList: " + cow);

        System.out.println("\n=== 5. Spliterator: an iterator that can split for parallel work ===");
        List<Integer> nums = List.of(1, 2, 3, 4, 5, 6, 7, 8);
        Spliterator<Integer> right = nums.spliterator();
        Spliterator<Integer> left = right.trySplit();          // hands off (roughly) the first half
        System.out.println("  SIZED=" + right.hasCharacteristics(Spliterator.SIZED)
                + " ORDERED=" + right.hasCharacteristics(Spliterator.ORDERED));
        StringBuilder l = new StringBuilder(), r = new StringBuilder();
        left.forEachRemaining(x -> l.append(x).append(' '));
        right.forEachRemaining(x -> r.append(x).append(' '));
        System.out.println("  left half:  " + l.toString().trim());
        System.out.println("  right half: " + r.toString().trim());
```

```
        System.out.println("  parallel stream sum = " + nums.parallelStream().mapToInt(Integer::intValue).sum());
    }
}
```

Output:

```
=== 1. Paged remote results behind a plain for-each ===
    [api] fetching page 1
  loop sees result-1
  loop sees result-2
    [api] fetching page 2
  loop sees result-3
  loop sees result-4
    [api] fetching page 3
  loop sees result-5
    [api] fetching page 4
  5 items, 4 network calls (the last one only discovers the end)

=== 2. Two independent iterators over one tree ===
  a: 20, 30, 40
  b: 20   <- b starts from the beginning: position lives in the iterator
  in-order: 20 30 40 50 60 70 80

=== 3. External vs internal iteration ===
  external: found bob, stopping early
  internal: ann
  internal: bob
  internal: cy

=== 4. Fail-fast vs fail-safe ===
  fail-fast: ConcurrentModificationException
  best-effort only: no exception, visited [java, tmp], 'lld' silently skipped
  after Iterator.remove: [java, lld]
  fail-safe CopyOnWriteArrayList: [x, y, x!, y!]

=== 5. Spliterator: an iterator that can split for parallel work ===
  SIZED=true ORDERED=true
  left half:  1 2 3 4
  right half: 5 6 7 8
  parallel stream sum = 36
```

Hide the shape, not the cost

Look at section 1 of the output: five results, **four** network calls. The last call comes back empty; its only purpose is to find out that the data has ended, because this API reports neither a total nor a "next page" token. That is genuine behaviour, not a bug in the demo. The takeaway: **a tidy loop conceals how much each step costs.** The **N+1 query problem** in ORMs (object-relational mappers: libraries that turn database rows into objects) has the same shape (one query for the list, then one more query *per element* as you iterate). Ways to keep the cost visible:

- **Name the type honestly** (`PagedResults`, `LazySearchResults`, not `List`).
- **Offer a batch API** for callers who want everything, with bigger pages.
- **Return the total or a next-page cursor** when the source knows it, so the extra call is not needed.
- **Log or count the fetches**, so a growing request count gets noticed.

Java's iterator family (quick reference)

Type	What it adds
`Iterable<T>`	`iterator()`; lets a type be used in the enhanced for-loop. Also `forEach` (internal iteration) and `spliterator()`.
`Iterator<T>`	`hasNext()`, `next()`, optional `remove()`, `forEachRemaining()`.
`ListIterator<T>`	Bidirectional: `hasPrevious()`, `previous()`, indexes, `set()`, `add()`.
`Spliterator<T>`	`tryAdvance`, `forEachRemaining`, `trySplit` for parallelism, `characteristics()` like `ORDERED`, `SIZED`, `DISTINCT`, `IMMUTABLE`.
`Enumeration<E>`	The old (Java 1.0) version: `hasMoreElements()`, `nextElement()`. Prefer `Iterator`.

Fail-fast, weakly consistent, snapshot. Iterators of `ArrayList`, `HashMap` and friends are **fail-fast**: they detect a structural change made behind their back and throw `ConcurrentModificationException`. The Javadoc says this is done on a **best-effort** basis — never write code that relies on the exception (section 4 of the demo shows a removal that goes unnoticed). Despite its name, the exception is most often caused by **one thread** changing a collection inside its own loop, not by concurrency (several threads running at the same time). `ConcurrentHashMap` iterators are **weakly consistent** (never throw, may or may not show concurrent changes). `CopyOnWriteArrayList` iterators work on a **snapshot** (and their `remove()` throws `UnsupportedOperationException`).

Trade-offs

Pros	Cons
Client code no longer cares whether data lives in an array, in a tree or behind an API.	One more object is created per traversal (only noticeable in extremely hot loops).
One structure can offer several named, reusable visiting orders.	Changing the collection mid-loop is a bug unless you use the iterator's own `remove` or a concurrent collection.
Elements can be produced lazily, so even huge or endless sequences work.	If a lazy iterator owns a file or DB cursor and the loop breaks early, the resource leaks unless it is closed.
Paging, buffering and prefetching (loading the next data before it is asked for) disappear behind an ordinary loop.	Per-step cost becomes invisible — the root of N+1 problems.

When not to use it

- **The data is already an in-memory list.** Hand out the list (or its own iterator); writing a custom iterator around an array adds code and no value.
- **Callers jump around** ("I need element 4,000"). Iterators only move forward step by step; offer indexed access instead.
- **All elements are in memory and always consumed in full.** An ordinary loop over the list is simpler and faster.
- **A step has side effects the caller must decide on.** Silently loading another page is acceptable; silently charging a customer is not.

Common mistakes

1. **Modifying the collection while iterating** (other than through `Iterator.remove`). Collect changes and apply them after the loop, or use `removeIf`.
2. **Leaking a resource** from a lazy iterator on early `break`. Use try-with-resources (Java's `try (...)` form that closes resources automatically) (`Files.lines`, `DirectoryStream` and JDBC (Java's standard database API) `ResultSet` are all closeable).
3. **Reusing a one-shot iterator** or stream a second time: it is already exhausted. A Java `Stream` at least throws `IllegalStateException`; a Python generator (a function that produces values one at a time with `yield`) just yields nothing the second time, so the bug shows up as missing data. That is why `iterator()` must return a *new* object.
4. **`next()` without `hasNext()`**, or a `next()` that returns `null` at the end instead of throwing `NoSuchElementException`.
5. **Storing the cursor in the collection** — then two loops interfere.
6. **Assuming every language fails loudly.** Java's fail-fast iterators usually throw `ConcurrentModificationException` when you modify a list mid-loop; Python may simply skip elements and finish with a wrong result. Record the intended changes while looping and make them once the loop has finished.

Real-world examples

- **Every for-each loop** in Java, Python, C#, JavaScript (`for...of`).
- **JDBC** `ResultSet` moves through rows that need not all be loaded at once; `Scanner` implements `Iterator<String>`; `Files.newDirectoryStream` returns an `Iterable` over directory entries.
- **Paginated SDKs** (software development kits — a provider's ready-made client libraries): the AWS (Amazon Web Services) SDK for Java v2 "paginators" and Stripe's auto-pagination give you an iterable that fetches pages behind the scenes — exactly section 1.
- **Java** `Stream`**s** are built on `Spliterator`s; that is how parallel streams split work. C#'s LINQ (built-in query syntax for collections) is likewise a lazy layer over iteration, which is why both can work on endless sequences.

- **Generators**: Python's `yield`, JavaScript's `function*` and C#'s `yield return` let you write an iterator as a *function*; the language remembers the position for you. A generator is the same pattern with less code.
- **Kafka consumer `poll()` loops**, and **infinite feeds** (Instagram/Twitter timelines) that load the next page as you scroll.

In the interview

> **30-second answer:** "Iterator pulls traversal out into a separate object with `hasNext` and `next`, so a caller can walk data without caring whether the data is an array, a tree or a paginated API. Each iterator keeps its own position, so two loops over the same data are independent. Rather than a home-made interface I'd implement the language's protocol — `Iterable<T>` in Java — so the native for-each works. My example would be the lazy paged case, not an array, and I'd flag the risk: when every step can hit the network, the loop hides its cost — that is how N+1 problems start."

Follow-ups:

- *Internal vs external iterator?* External: the client controls stepping (`hasNext/next`) and can stop, pause or interleave two iterators. Internal: the collection controls the loop and calls your function (`forEach`, streams); simpler, easier to parallelise, but harder to stop early or to walk two collections in lock-step (one element from each at a time).
- *What is fail-fast?* The iterator checks a modification counter and throws `ConcurrentModificationException` if the collection was structurally changed outside the iterator. Best-effort only.
- *How do you remove while iterating?* `Iterator.remove()` or `Collection.removeIf(predicate)` (a predicate is a function that returns true or false).
- *What is a Spliterator?* A splittable iterator used by streams; `trySplit` hands off part of the elements so another thread can process them; characteristics tell the stream framework what it can optimise.
- *Iterator vs Visitor?* Iterator decides **how you move** through a structure; Visitor decides **what you do** at each element. They are often used together.
- *Iterator vs Composite?* Composite makes leaves and branches of a tree look alike; an iterator supplies a way to visit them. The two are often used together.
- *Iterator vs generator?* A generator (Python `yield`, JS `function*`) is the same pattern with the class replaced by a function that `yield`s.
- *Iterator vs stream?* Streams sit on top of iteration and add operations such as `map` and `filter`. When you need to transform data, not merely visit it, reach for streams.
- *How is Iterator expressed in other languages?* Python: `__iter__`/`__next__` or a generator. JavaScript/TypeScript: `[Symbol.iterator]` or `function*`, which makes `for...of` and spread work. C#: `IEnumerable<T>` with `yield return`. C++: `begin()`/`end()` for range-based for and the standard algorithms. Go: traditionally a `Next() bool` plus a value accessor (`sql.Rows`, `bufio.Scanner`), and since Go 1.23 range-over-function iterators.

Self-check

1. **Q:** Where does the cursor live? **A:** In the iterator object.
2. **Q:** What does the enhanced for-loop need from a class? **A:** That it implements `Iterable<T>`.
3. **Q:** Why did five results take four API calls? **A:** Without a total or cursor, the iterator can only discover the end by asking for one more page.
4. **Q:** Does `CopyOnWriteArrayList`'s iterator throw on concurrent modification? **A:** No; it iterates over a snapshot taken when the iterator was created.
5. **Q:** What should `next()` do when there are no more elements? **A:** Throw `NoSuchElementException`.

TL;DR

A separate object walks the collection with `hasNext/next`; the **position lives in the iterator**. Implement `Iterable` in Java. Know **fail-fast** vs snapshot iterators, **internal vs external** iteration, and don't let a lazy loop **hide its cost**.

10.4 Mediator

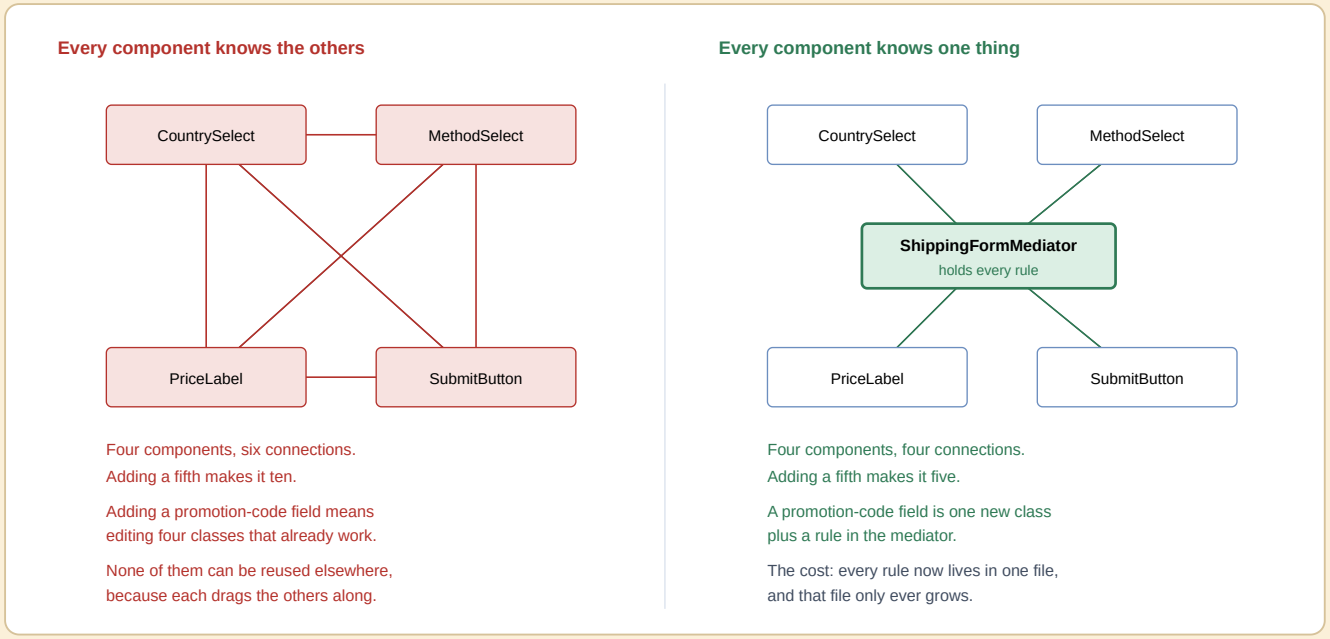


Figure — Left: four form components all know each other (six links). Right: each knows only the mediator (four links) (course diagram)

What it is

Mediator makes a group of objects talk through **a single coordinator** instead of holding references to one another. Each object only reports "this just happened" to the mediator, and the mediator works out how the rest should respond.

GoF intent, in plain words: *define an object that encapsulates how a set of objects interact, so they stop referring to each other explicitly and their interaction can be varied independently.* GoF participants: **Mediator** (interface for talking to colleagues), **ConcreteMediator** (knows the colleagues and implements the coordination), and **Colleague** classes (each knows its mediator and talks to it instead of to other colleagues). Some literature simply calls the mediator a *controller*.

Everyday analogy: **air traffic control**. Planes do not radio each other to agree who lands first. Each plane talks only to the tower, and the tower holds the rules about spacing and order.

The problem it solves

Picture a checkout form with four widgets: a **country** picker, a **delivery-method** picker, a **price** label and a **submit** button. Picking a country refreshes the list of delivery methods; the new method changes the price; the price situation decides whether submit is clickable. For some destinations no delivery is offered at all.

If each widget talks to the others directly:

- every widget holds references to three others — four widgets need **six** connections, five need **ten** (n(n−1)/2, growing roughly with the square);
- a new "promo code" field forces changes to four classes that were already tested and working;
- no widget can be reused on another screen: if `CountrySelect` calls `methodSelect.reload()`, it cannot be used anywhere without bringing `MethodSelect` too.

With a mediator, each widget has **one** reference (to the mediator): four widgets, four links; a fifth makes five. `CountrySelect` is now unaware that any price label exists, so another screen can reuse it with its own mediator and its own rules. Each widget also has no idea how many other widgets exist.

The honest trade: the dependencies are not removed — **they are relocated** into the mediator, a class that tends to keep getting bigger.

How it works

1. Define a **Mediator** interface, e.g. `notify(sender, event)`.
2. Each **component** (colleague) holds a reference to the mediator — and nothing else.
3. When something happens, a component calls `mediator.notify(this, event)`.

4. The **concrete mediator** holds all components and all the rules: "when the country changes, reload methods, then show the price, then enable or disable submit".
5. The mediator calls back **down** into specific components. Traffic is **two-way** through one object.

Structure (UML)

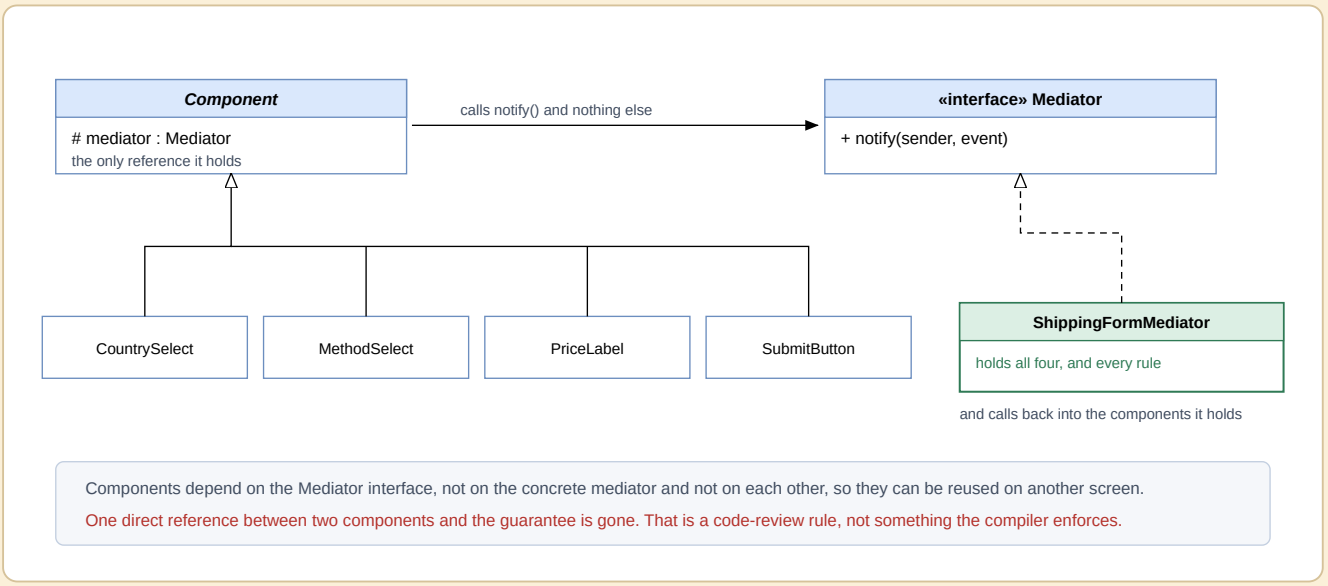
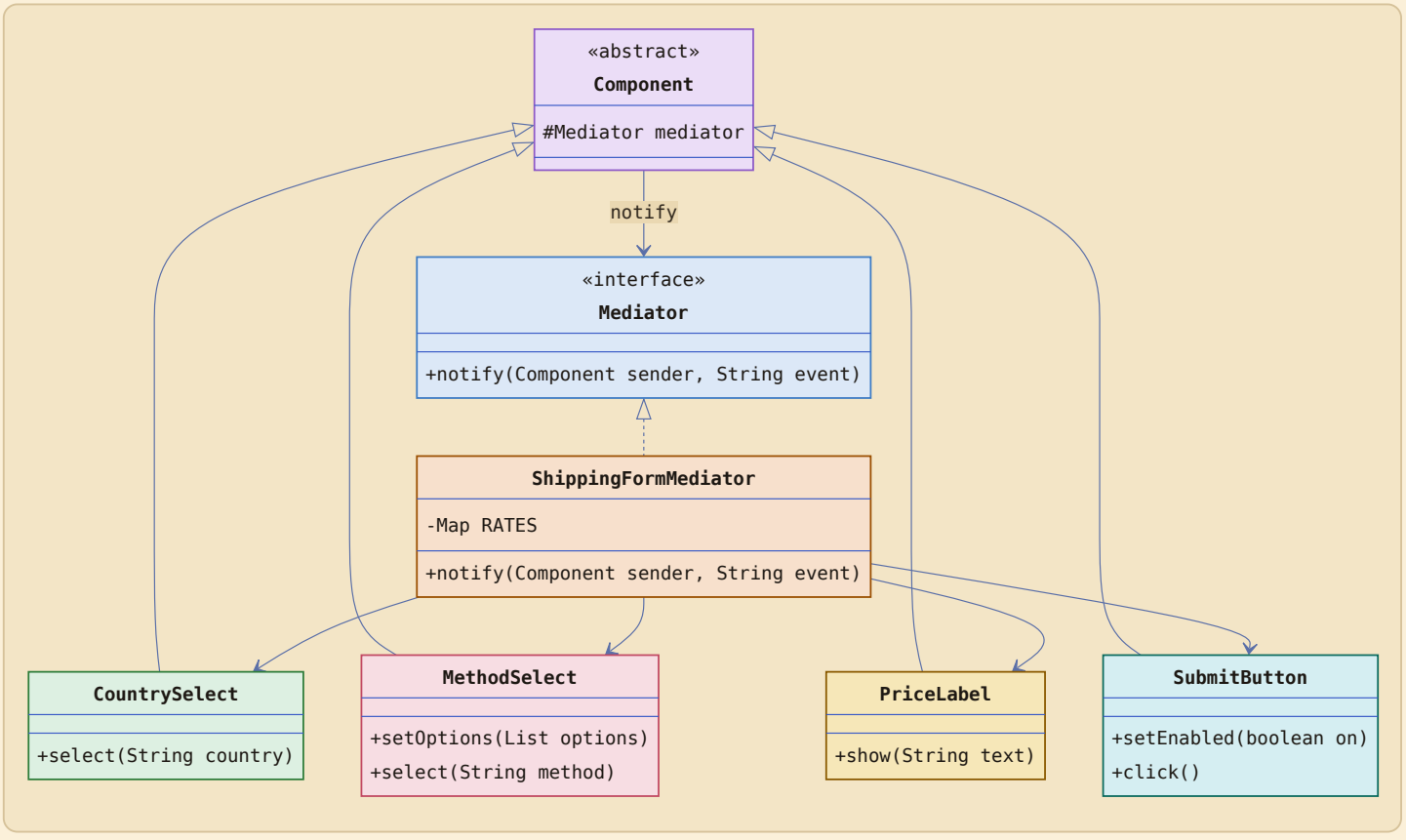
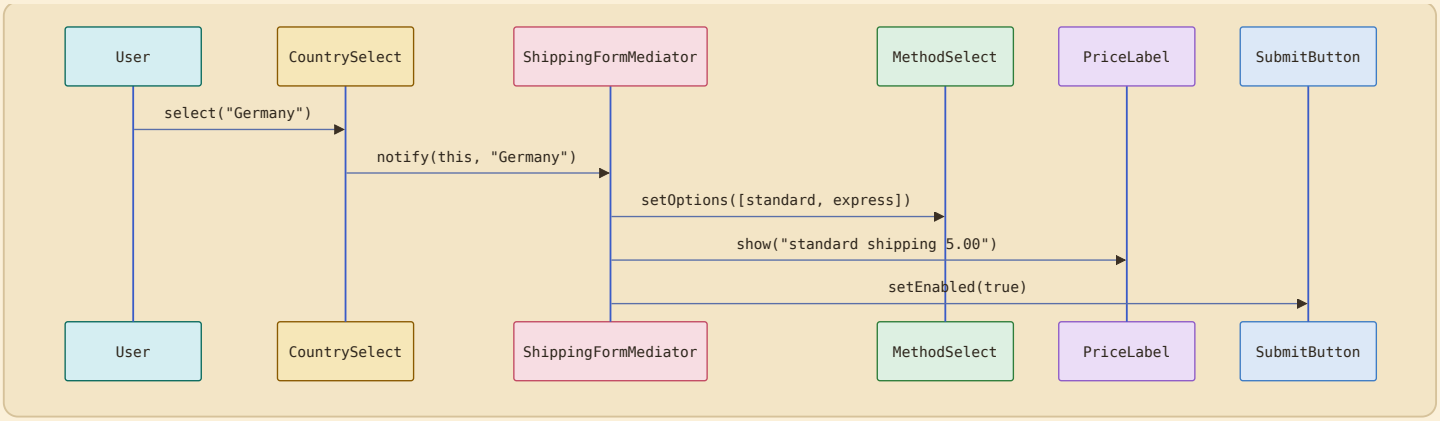


Figure — Class diagram: components extend Component (which holds a Mediator); ShippingFormMediator implements Mediator and holds all four components (course diagram)

In the diagram above, follow the arrows: widgets report **upward** by calling `notify`, and the mediator issues instructions **downward** to the widgets it owns. Traffic in both directions passing through one hub is what separates a mediator from an observer.

Our Java design:





Java example

`lld.ch10.mediator` has three mediators:

1. **Shipping form** (`ShippingFormMediator` + four components). No code except `notify` knows that choosing a country influences the price. Components identify themselves by reference (`sender == country`), not by strings. Shipping rates are `BigDecimal` — money is never stored as `double`.
2. **Chat room** (`ChatRoom`, `ChatUser`). Users never hold references to each other. The room routes broadcasts and private messages and applies **moderation** (filtering or blocking unwanted messages) rules — rules live in the mediator, not in the users.
3. **Air traffic control** (`ControlTower`, `Flight`). Flights only request landing and report "vacated"; the tower owns the runway and the holding queue.

```java
// File: code/lld/ch10/mediator/Mediator.java
package lld.ch10.mediator;

/** Components report "something happened" here. They depend on this interface only. */
public interface Mediator {
    void notify(Component sender, String event);
}
```

```java
// File: code/lld/ch10/mediator/Component.java
package lld.ch10.mediator;

/** Base for every form widget: its ONLY reference is the mediator. */
public abstract class Component {
    protected Mediator mediator;

    void setMediator(Mediator mediator) { this.mediator = mediator; }
}
```

```java
// File: code/lld/ch10/mediator/CountrySelect.java
package lld.ch10.mediator;

final class CountrySelect extends Component {
    private String selected;

    void select(String country) {
        selected = country;
        System.out.println("  [country] user picked " + country);
        mediator.notify(this, country);          // report up; never call a sibling
    }

    String selected() { return selected; }
}
```

```java
// File: code/lld/ch10/mediator/MethodSelect.java
package lld.ch10.mediator;

import java.util.List;

final class MethodSelect extends Component {
    private List<String> options = List.of();
    private String selected;

    void setOptions(List<String> options) {
        this.options = options;
        this.selected = options.isEmpty() ? null : options.get(0);
        System.out.println("  [methods] reloaded: " + options + (selected == null ? "" : ", default " + selected));
    }

    void select(String method) {
        if (!options.contains(method)) throw new IllegalArgumentException(method + " not offered");
        selected = method;
        System.out.println("  [methods] user picked " + method);
        mediator.notify(this, method);
    }

    String selected() { return selected; }
}
```

```java
// File: code/lld/ch10/mediator/PriceLabel.java
package lld.ch10.mediator;

final class PriceLabel extends Component {
    void show(String text) { System.out.println("  [price] " + text); }
}
```

```java
// File: code/lld/ch10/mediator/SubmitButton.java
package lld.ch10.mediator;

final class SubmitButton extends Component {
    private boolean enabled;

    void setEnabled(boolean enabled) {
        this.enabled = enabled;
        System.out.println("  [submit] " + (enabled ? "enabled" : "disabled"));
    }

    void click() {
        if (enabled) mediator.notify(this, "click");
        else System.out.println("  [submit] click ignored: button is disabled");
    }
}
```

```java
// File: code/lld/ch10/mediator/ShippingFormMediator.java
package lld.ch10.mediator;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.Map;

/** The ONLY place that knows how a country change affects methods, price and submit. */
final class ShippingFormMediator implements Mediator {
    private static final Map<String, Map<String, BigDecimal>> RATES = Map.of(
        "Germany", ordered("standard", "5.00", "express", "15.00"),
        "USA", ordered("standard", "8.00", "express", "20.00"),
        "Antarctica", Map.of());

    private final CountrySelect country;
    private final MethodSelect methods;
    private final PriceLabel price;
    private final SubmitButton submit;

    ShippingFormMediator(CountrySelect c, MethodSelect m, PriceLabel p, SubmitButton s) {
        this.country = c; this.methods = m; this.price = p; this.submit = s;
        for (Component comp : new Component[] {c, m, p, s}) comp.setMediator(this);
        submit.setEnabled(false);
    }

    @Override
    public void notify(Component sender, String event) {
        if (sender == country) {
            Map<String, BigDecimal> available = RATES.getOrDefault(event, Map.of());
            methods.setOptions(new ArrayList<>(available.keySet()));
            if (available.isEmpty()) {
                price.show("shipping not available");
                submit.setEnabled(false);
            } else {
                showPrice();                          // ORDER matters: reload methods first, then price
                submit.setEnabled(true);
            }
        } else if (sender == methods) {
            showPrice();
        } else if (sender == submit) {
            System.out.println("  [mediator] order placed: " + country.selected() + " / " + methods.selected());
        }
    }

    private void showPrice() {
        BigDecimal p = RATES.get(country.selected()).get(methods.selected());   // money: BigDecimal, never double
        price.show(methods.selected() + " shipping " + p.toPlainString());
    }

    private static Map<String, BigDecimal> ordered(String k1, String v1, String k2, String v2) {
        Map<String, BigDecimal> m = new LinkedHashMap<>();   // keeps "standard" before "express"
        m.put(k1, new BigDecimal(v1));
        m.put(k2, new BigDecimal(v2));
        return m;
    }
}
```

```java
// File: code/lld/ch10/mediator/ChatRoom.java
package lld.ch10.mediator;

import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Set;

/** Chat mediator: users never hold references to each other, only to the room. */
final class ChatRoom {
    private final Map<String, ChatUser> members = new LinkedHashMap<>();
    private final Set<String> bannedWords = Set.of("spam");

    void join(ChatUser user) {
        members.put(user.name(), user);
        user.joined(this);
        broadcast("room", user.name() + " joined");
    }

    /** Room rules (moderation, routing) live here, not in the users. */
    void send(ChatUser from, String to, String text) {
        if (bannedWords.stream().anyMatch(text.toLowerCase()::contains)) {
            from.receive("room", "message blocked by moderation");
            return;
        }
        if (to == null) {
            members.values().stream().filter(u -> u != from).forEach(u -> u.receive(from.name(), text));
        } else if (members.containsKey(to)) {
            members.get(to).receive(from.name() + " (private)", text);
        } else {
            from.receive("room", to + " is not in the room");
        }
    }

    private void broadcast(String from, String text) {
        members.values().forEach(u -> u.receive(from, text));
    }
}
```

```java
// File: code/lld/ch10/mediator/ChatUser.java
package lld.ch10.mediator;

final class ChatUser {
    private final String name;
    private ChatRoom room;

    ChatUser(String name) { this.name = name; }

    String name() { return name; }
    void joined(ChatRoom room) { this.room = room; }
    void say(String text) { room.send(this, null, text); }
    void whisper(String to, String text) { room.send(this, to, text); }
    void receive(String from, String text) { System.out.println("  " + name + " sees <" + from + "> " + text); }
}
```

```java
// File: code/lld/ch10/mediator/ControlTower.java
package lld.ch10.mediator;

import java.util.ArrayDeque;
import java.util.Queue;

/** Air traffic control: aircraft talk to the tower, never to each other. */
final class ControlTower {
    private Flight onRunway;                          // null = runway free
    private final Queue<Flight> holding = new ArrayDeque<>();

    void requestLanding(Flight f) {
        if (onRunway == null) {
            clear(f);
        } else {
            holding.add(f);
            System.out.println("  [tower] " + f.callsign() + ": hold, runway busy with " + onRunway.callsign());
        }
    }

    void runwayVacated(Flight f) {
        System.out.println("  [tower] " + f.callsign() + " vacated the runway");
        onRunway = null;
        Flight next = holding.poll();
        if (next != null) clear(next);
    }

    private void clear(Flight f) {
        onRunway = f;
        System.out.println("  [tower] " + f.callsign() + ": cleared to land");
    }
}
```

```java
// File: code/lld/ch10/mediator/Flight.java
package lld.ch10.mediator;

record Flight(String callsign, ControlTower tower) {
    void requestLanding() { tower.requestLanding(this); }
    void landed() { tower.runwayVacated(this); }
}
```

```java
// File: code/lld/ch10/mediator/Main.java
package lld.ch10.mediator;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== 1. Shipping form: widgets coordinated by a mediator ===");
        CountrySelect country = new CountrySelect();
        MethodSelect methods = new MethodSelect();
        PriceLabel price = new PriceLabel();
        SubmitButton submit = new SubmitButton();
        new ShippingFormMediator(country, methods, price, submit);

        country.select("Germany");        // reloads methods, shows price, enables submit
        methods.select("express");        // updates price only
        submit.click();
        country.select("Antarctica");     // no methods: price hidden, submit disabled
        submit.click();

        System.out.println("\n=== 2. Chat room ===");
        ChatRoom room = new ChatRoom();
        ChatUser asha = new ChatUser("asha"), ben = new ChatUser("ben"), chen = new ChatUser("chen");
        room.join(asha);
        room.join(ben);
        room.join(chen);
        asha.say("standup in 5");
        ben.whisper("chen", "coffee first?");
        chen.say("buy cheap SPAM now");

        System.out.println("\n=== 3. Air traffic control ===");
        ControlTower tower = new ControlTower();
        Flight ai101 = new Flight("AI101", tower), ba22 = new Flight("BA22", tower), lh7 = new Flight("LH7", tower);
        ai101.requestLanding();
        ba22.requestLanding();
        lh7.requestLanding();
        ai101.landed();
        ba22.landed();
    }
}
```

Output:

```
=== 1. Shipping form: widgets coordinated by a mediator ===
  [submit] disabled
  [country] user picked Germany
  [methods] reloaded: [standard, express], default standard
  [price] standard shipping 5.00
  [submit] enabled
  [methods] user picked express
  [price] express shipping 15.00
  [mediator] order placed: Germany / express
  [country] user picked Antarctica
  [methods] reloaded: []
  [price] shipping not available
  [submit] disabled
  [submit] click ignored: button is disabled

=== 2. Chat room ===
  asha sees <room> asha joined
  asha sees <room> ben joined
  ben sees <room> ben joined
  asha sees <room> chen joined
  ben sees <room> chen joined
  chen sees <room> chen joined
  ben sees <asha> standup in 5
  chen sees <asha> standup in 5
  chen sees <ben (private)> coffee first?
  chen sees <room> message blocked by moderation

=== 3. Air traffic control ===
  [tower] AI101: cleared to land
  [tower] BA22: hold, runway busy with AI101
  [tower] LH7: hold, runway busy with AI101
  [tower] AI101 vacated the runway
  [tower] BA22: cleared to land
  [tower] BA22 vacated the runway
  [tower] LH7: cleared to land
```

One user action ("picked Germany") updated three components — and **no component called another**.

Mediator or Observer?

Both decouple objects. The difference is **what happens to the news**.

	Mediator	**Observer**
What travels	An event **and** the decision on what should follow	Only the event
Where reaction logic sits	In one place: the mediator	Spread across the observers, each deciding alone
Do the parties know each other?	No — not even how many peers exist	No; the subject only knows it has a list of subscribers
Flow	Both ways: colleagues report in, the mediator calls out to them	One way: the subject publishes, observers respond
Order of reactions	Written explicitly in the mediator — a reason to choose it	Not part of the contract; observers should not depend on each other's order

Practical test: does something need to be coordinated? "The country changed; anyone interested, update yourself" is Observer, and it is the lighter choice. "The country changed: refresh the methods, *after that* price the new default method, *after that* decide about submit" is sequenced coordination — Mediator territory. Short version: **with Observer the listeners decide; with Mediator the hub decides and gives orders.** (A mediator is often *implemented* using observer-style notifications from the components.)

Trade-offs

Pros	Cons
Every widget depends on exactly one thing, so links grow linearly instead of quadratically.	Every new rule lands in the mediator, so it keeps growing.
All interaction rules sit in one class you can read and unit-test.	Anyone changing the screen changes that same file, so merge conflicts (clashing edits to the same lines) pile up there.
Widgets are reusable because they no longer pull their neighbours with them.	To learn why a widget changed you read the mediator, not the widget.
A new widget costs one class plus one rule.	It can quietly become a **god object** (a class that knows and does too much).

Keeping the mediator from becoming a god object:

- **Split by area** — one mediator per screen section or workflow, not one per application. Two 80-line mediators are much easier to live with than one 160-line one.
- **Keep it to coordination.** Tax and pricing calculations belong in their own classes; the mediator decides *what to call*, not *how to compute* (a Facade has to respect the same boundary).
- **Reject shortcuts in code review.** A single widget-to-widget reference breaks the whole guarantee, and the compiler will not catch it.
- **Dispatch by table, not by `if`-chain.** In languages with first-class functions the mediator can keep a map from event (or sender) to handler function — e.g. a Python `dict` of `{event: method}` — so each rule is easy to find.

When not to use it

- **Only two objects interact.** Let them call each other; a coordinator for a single relationship adds a layer that does nothing.
- **The interactions are few and stable.** The pattern swaps many small, scattered links for one bigger central class; that only pays off once the tangle is large.
- **Nothing needs coordinating** — components react independently. Use Observer.
- **The mediator is already a 400-line god object.** More rules will not help; break it into smaller mediators first.

Common mistakes

1. **Components calling each other "just this once".**
2. **Computation inside the mediator** (tax, pricing), turning it into a god object.
3. **Confusing it with Facade.** A Facade is placed *in front of* a subsystem, and the subsystem's classes are unaware of it; calls flow one way. A Mediator is placed *among* peers that all know it and report to it; calls flow both ways.
4. **String-typed events everywhere** (`"country"`, `"method"`) that silently break on a typo. Use the sender reference, an enum, or typed events.

Real-world examples

- **UI controllers / view-models** coordinating several widgets (MVC — Model–View–Controller — controllers, MVVM — Model–View–ViewModel — view-models, Android `ViewModel` wiring). GoF's own example is a dialog box coordinating its widgets.
- **Chat servers** (Slack, WhatsApp): clients send messages to the server, which routes them to rooms and people and applies moderation.
- **Air traffic control** — the classic analogy.
- **Workflow / saga orchestrators** (a saga is a multi-step process spread across services, each step with its own undo; e.g. Temporal, Camunda, AWS Step Functions): a checkout orchestrator invokes payment, stock reservation and shipping in sequence and holds the recovery plan for each failure. Ride-hailing dispatch (matching riders and drivers through a central service rather than drivers talking to riders directly) has the same shape.
- **Redux store** at application scale (components dispatch actions instead of calling each other; reducers hold the rules); **MediatR** in .NET popularised the name on the server side, using it to dispatch each request to its handler.

- **Message brokers / event buses** (servers that receive messages and deliver them to the right consumers) at system scale: a service hands messages to the broker rather than calling other services; the broker's routing configuration is the mediator logic — and it can become a god object too.
- `java.util.Timer` and `Executor`/`ExecutorService` are often listed (including by the course) as JDK mediators: they stand between submitted tasks and the threads that run them, so neither side knows the other. Treat this as a *loose* fit — they do not coordinate peer colleagues the way the GoF dialog example does, so in an interview lead with UI controllers or orchestrators.

In the interview

> **30-second answer:** "Mediator swaps peer-to-peer references for a single hub. Widgets just report events to it; the hub owns the rules and instructs particular widgets. Four widgets go from six links to four, and a fifth gives five instead of ten. The dependencies aren't gone — they're concentrated in one class that I can read and test. The risk is a god object, so I split mediators per screen area or workflow and push computation into its own classes."

Follow-ups:

- *Mediator vs Observer?* With Observer each listener decides for itself; with Mediator the hub decides and instructs. When the sequence of reactions matters, pick Mediator.
- *Mediator vs Facade?* Facade simplifies a subsystem for outside callers (one-way, subsystem unaware). Mediator coordinates peers that know it (two-way).
- *How does it relate to Command?* A mediator often creates and dispatches commands.
- *How do you test a mediator?* Give it fake components and assert which calls it made for each event.
- *What about in microservices (a system split into many small, separately deployed services)?* Orchestration (a central saga orchestrator) is mediator-style; choreography (services react to each other's events) is observer-style.

Self-check

1. **Q:** How many links do 5 fully connected components need, and how many with a mediator? **A:** 10 vs 5.
2. **Q:** Does a mediator remove coupling? **A:** No, it moves it into one class.
3. **Q:** What is the main failure mode? **A:** The mediator grows into a god object.
4. **Q:** Which way does traffic flow in a Mediator vs an Observer? **A:** Mediator two-way; Observer one-way.
5. **Q:** What should *not* go into a mediator? **A:** Business calculations — only coordination.

TL;DR

Peers talk through **one coordinator** that holds the interaction rules. Links become linear and components reusable, but the coupling **moves** into the mediator — keep it small and split it before it becomes a god object.

10.5 Memento

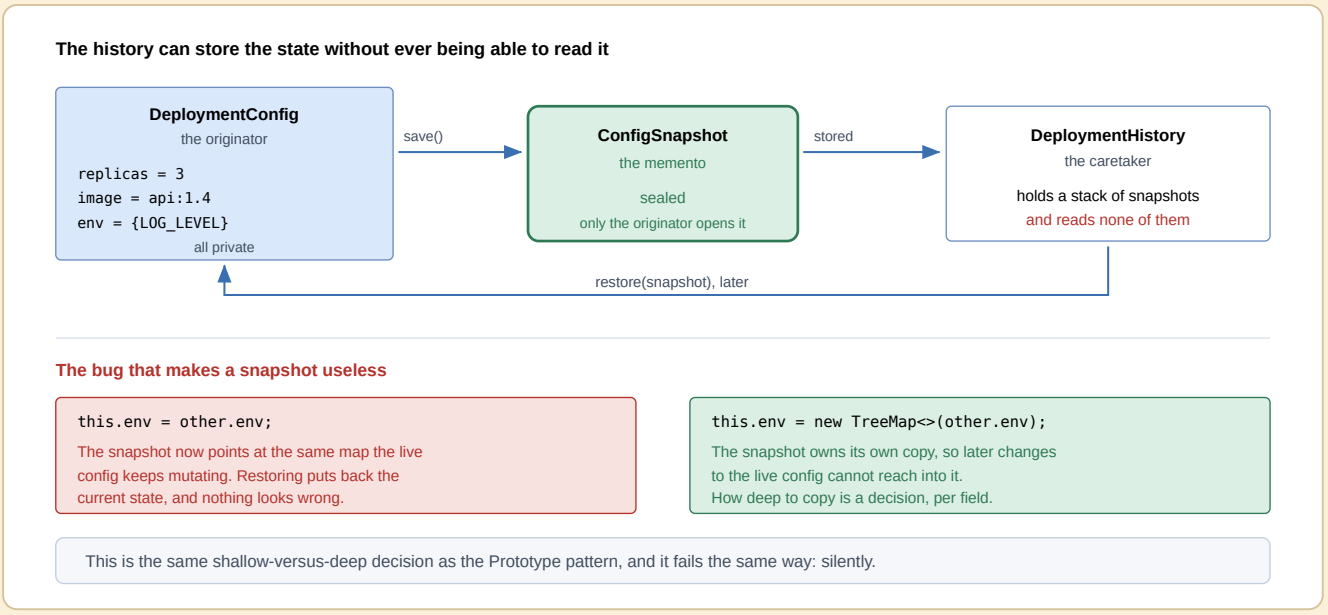


Figure — The originator (DeploymentConfig) saves a sealed snapshot; the caretaker (DeploymentHistory) stores it without being able to read it; below, the shared-map bug that makes a snapshot useless (course diagram)

What it is

Memento lets an object record its internal state in a **snapshot** and later **return** to that state — while nobody else gets to **see** what the snapshot contains.

GoF intent, in plain words: *without breaking encapsulation, capture and externalize an object's internal state so the object can be put back into that state later.* GoF participants: **Memento** (stores the state; offers a narrow interface to the caretaker and a wide one to the originator), **Originator** (creates a memento of its current state and uses one to restore itself) and **Caretaker** (keeps mementos safe but never operates on or examines their contents).

Everyday analogy: a **save game**. The game writes a save file; the menu lists the saves by name and date and lets you load one. The menu never needs to understand what is inside the file.

The problem it solves

You need a **rollback** (a return to the previous working version): record the deployment configuration just before a risky change, and if the rollout (the release of a change to the running servers) misbehaves, bring the recorded version back.

The obvious way — something *outside* reads every field and stores them — breaks two things:

1. **Encapsulation.** The history class must now understand replicas (copies of the service running side by side), image names (which packaged build to run) and environment variables (named settings passed to a program). Every new config field forces a change in the history class, and details that were private are now exposed.
2. **Atomicity.** Atomicity means all-or-nothing; a restore done one field at a time can stop half-way. If three fields are written and an exception hits before the fourth, the config holds a combination that was never real.

With Memento the object **takes its own snapshot** and hands it out as a sealed token that **only it can open**. Restoring becomes **one call into the originator**, which is in full control of the restore and can make it all-or-nothing (our `restore` validates the token first, then assigns every field; nothing in between can fail half-way).

How it works

Three roles, with standard names interviewers use:

Role	Job	In our code
Originator	Owns the state; produces snapshots (`save`) and returns to them (`restore`). No other class can look inside a snapshot.	`DeploymentConfig`
Memento	Carries the state; opaque to everyone else.	`DeploymentConfig.Snapshot` (narrow) / private `State` record (wide)
Caretaker	Decides **when** to save and restore; never looks **inside**.	`DeploymentHistory`

Steps:

1. Caretaker asks the originator: `snapshot = config.save(label)`.
2. Caretaker stores the snapshot (usually on a stack).
3. Later, caretaker calls `config.restore(snapshot)`.
4. The originator copies the state back in one step.

How opacity is enforced in Java. GoF describes a **narrow interface** (what the caretaker sees) and a **wide interface** (what the originator sees). Our design does this with types: the public interface `Snapshot` exposes only `label()`; the real data is in a **private nested record** `State` that only `DeploymentConfig` can see. `restore` checks `instanceof State`, so even a forged `Snapshot` (see section 4 of the output) is refused. (Strictly, reflection — Java's ability to inspect and call even private members at run time — or a debugger can still peek; Java access control protects against honest code, not a hostile JVM, the Java Virtual Machine that runs the program.) Other options by language: package-private accessors in Java; the originator declared a `friend` in C++; a private nested class in C#; Python and JavaScript offer no true enforcement, so it is a naming convention (leading underscore) plus code review — although a JavaScript closure *can* hide the contents in a way a class field cannot.

Structure (UML)

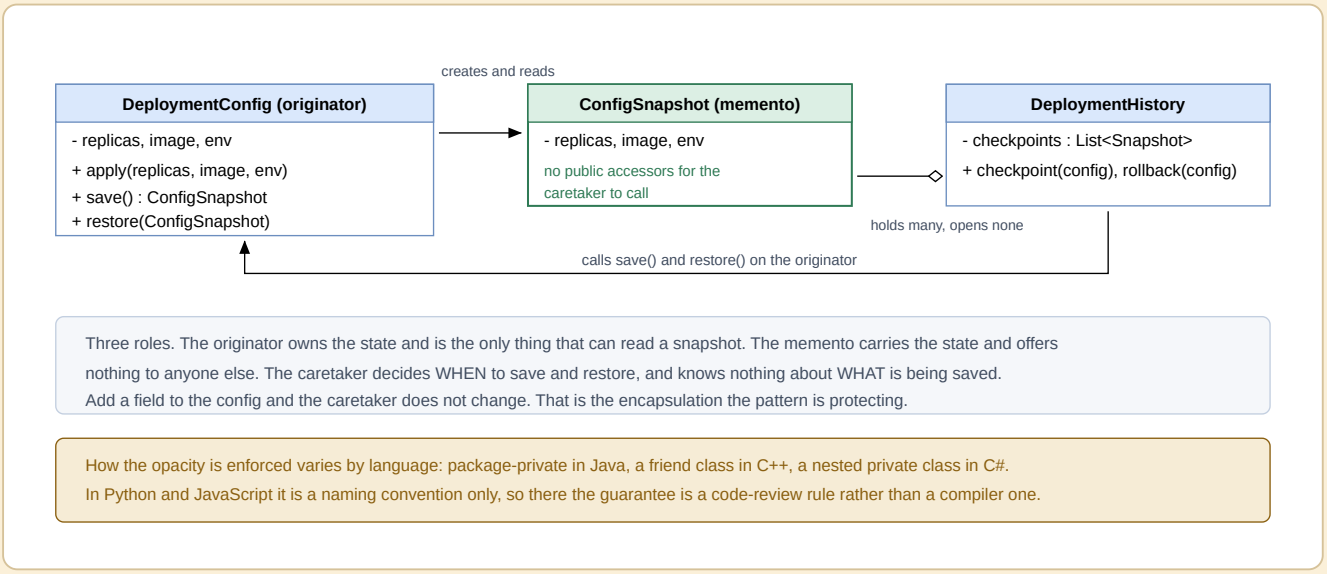
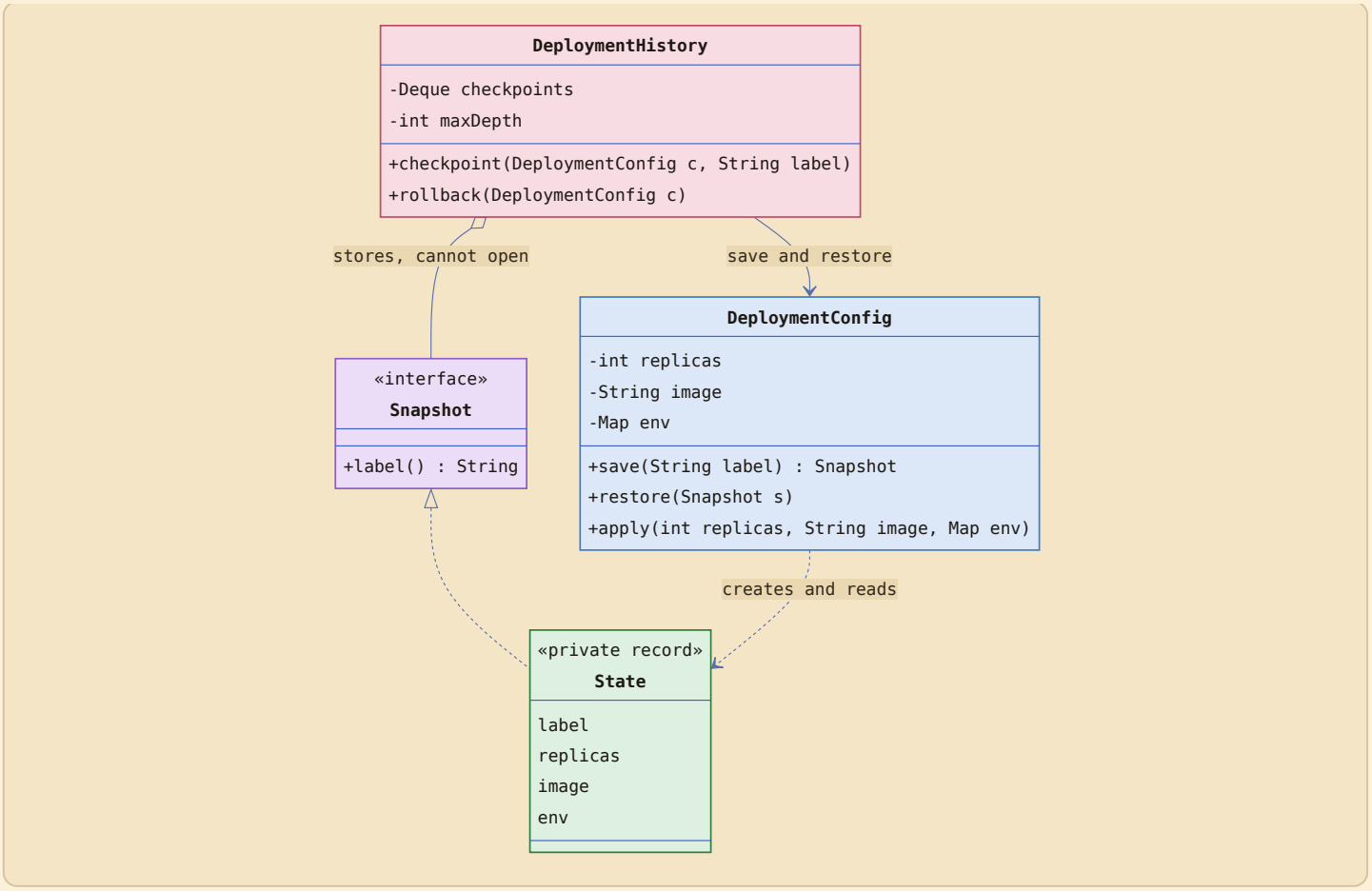
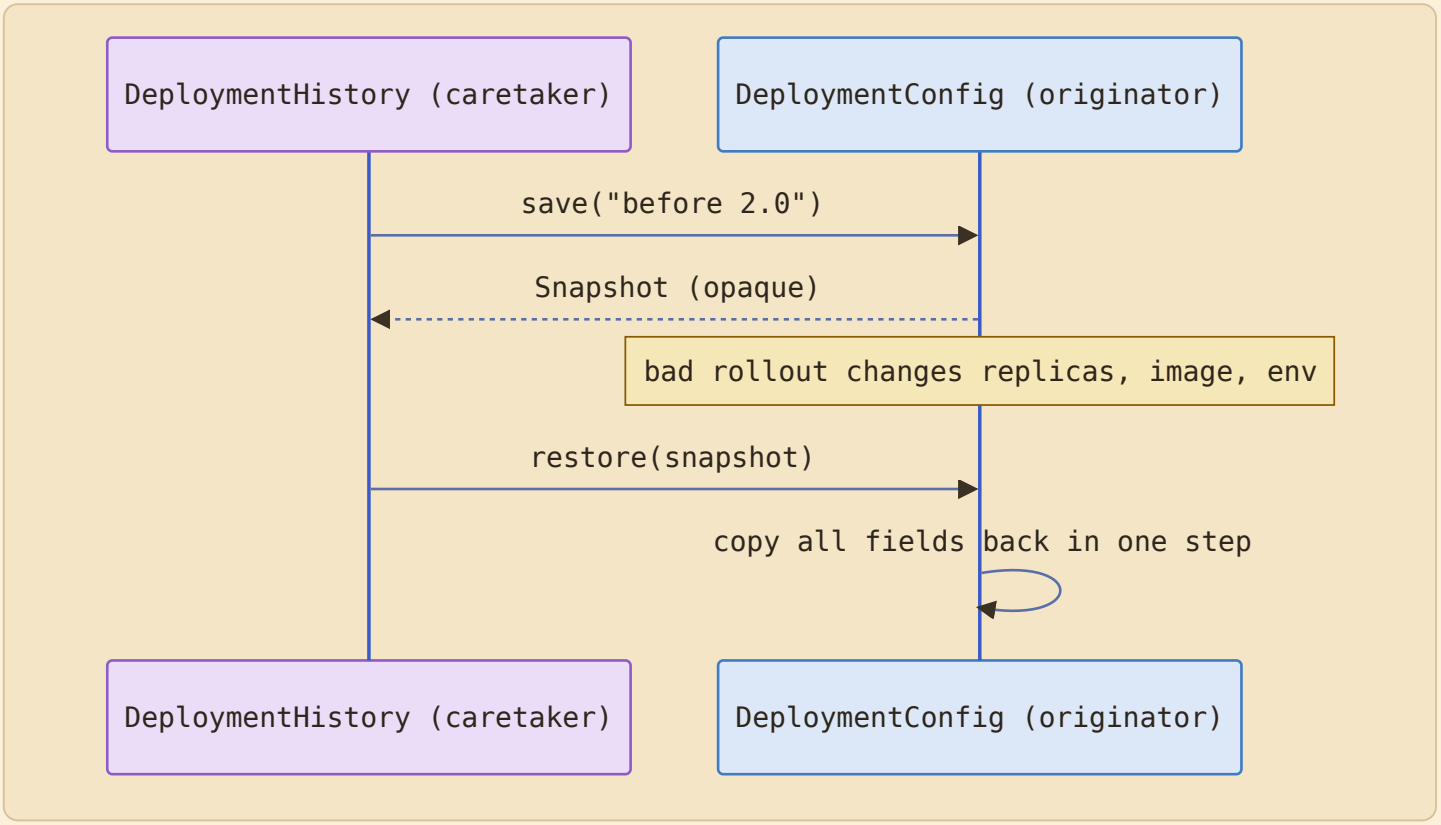


Figure — Class diagram: DeploymentConfig creates and restores ConfigSnapshot; DeploymentHistory holds a list of snapshots (course diagram)

In the diagram above, `DeploymentHistory` never refers to a single configuration field. That is the quick check for a real Memento: if the caretaker names the fields, you have only relocated them.

Our Java design:





Java example

Points to notice in `lld.ch10.memento`:

- The `State` record's (a record is Java's short syntax for an immutable data class) compact constructor (a constructor with no parameter list that runs on every creation) does `env = Map.copyOf(env)`, so the snapshot **owns its own copy**. In section 1 the live map is changed *after* saving (`FEATURE_X`), and the rollback still restores the original.

- `DeploymentHistory` is **bounded** (`maxDepth`) and drops the oldest checkpoint when full.
- Section 3 shows the **shallow-copy bug** in isolation.

```java
// File: code/lld/ch10/memento/DeploymentConfig.java
package lld.ch10.memento;

import java.util.Map;
import java.util.TreeMap;

/** The ORIGINATOR: owns the state, and is the only class that can read a snapshot. */
public final class DeploymentConfig {

    /**
     * The NARROW interface the caretaker sees: a label, nothing else.
     * (GoF: caretaker gets a narrow interface, originator a wide one.)
     */
    public interface Snapshot {
        String label();
    }

    /** The WIDE interface: private, so only DeploymentConfig can open it. Immutable record. */
    private record State(String label, int replicas, String image, Map<String, String> env)
            implements Snapshot {
        State {
            env = Map.copyOf(env);   // DEEP enough: the snapshot owns its own copy, not the live map
        }
    }

    private int replicas;
    private String image;
    private final Map<String, String> env = new TreeMap<>();

    public DeploymentConfig(int replicas, String image, Map<String, String> env) {
        apply(replicas, image, env);
    }

    public void apply(int replicas, String image, Map<String, String> env) {
        this.replicas = replicas;
        this.image = image;
        this.env.clear();
        this.env.putAll(env);
    }

    public void setEnv(String key, String value) { env.put(key, value); }

    public Snapshot save(String label) {
        return new State(label, replicas, image, env);
    }

    /** One call restores everything: all-or-nothing. */
    public void restore(Snapshot snapshot) {
        if (!(snapshot instanceof State s)) {
            throw new IllegalArgumentException("not a snapshot produced by DeploymentConfig");
        }
        apply(s.replicas(), s.image(), s.env());
    }

    @Override
    public String toString() {
        return "replicas=" + replicas + " image=" + image + " env=" + env;
    }
}
```

```java
// File: code/lld/ch10/memento/DeploymentHistory.java
package lld.ch10.memento;

import java.util.ArrayDeque;
import java.util.Deque;

/** The CARETAKER: decides WHEN to save/restore, never WHAT is inside. Bounded to avoid leaks. */
public final class DeploymentHistory {
    private final Deque<DeploymentConfig.Snapshot> checkpoints = new ArrayDeque<>();
    private final int maxDepth;

    public DeploymentHistory(int maxDepth) { this.maxDepth = maxDepth; }

    public void checkpoint(DeploymentConfig config, String label) {
        if (checkpoints.size() == maxDepth) {
            DeploymentConfig.Snapshot dropped = checkpoints.removeLast();   // oldest
            System.out.println("  [history] full, dropped oldest checkpoint '" + dropped.label() + "'");
        }
        checkpoints.push(config.save(label));
        System.out.println("  [history] checkpoint '" + label + "' taken");
    }

    public void rollback(DeploymentConfig config) {
        if (checkpoints.isEmpty()) { System.out.println("  [history] nothing to roll back to"); return; }
        DeploymentConfig.Snapshot s = checkpoints.pop();
        config.restore(s);
        System.out.println("  [history] rolled back to '" + s.label() + "'");
    }
}
```

```java
// File: code/lld/ch10/memento/Main.java
package lld.ch10.memento;

import java.util.HashMap;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== 1. Checkpoint, bad rollout, rollback ===");
        DeploymentConfig config = new DeploymentConfig(3, "api:1.4", Map.of("LOG_LEVEL", "info"));
        DeploymentHistory history = new DeploymentHistory(2);
        System.out.println("  live: " + config);

        history.checkpoint(config, "before 2.0");
        config.apply(10, "api:2.0", Map.of("LOG_LEVEL", "debug"));   // the risky rollout
        config.setEnv("FEATURE_X", "on");                             // mutates the live map after saving
        System.out.println("  live: " + config);
        history.rollback(config);
        System.out.println("  live: " + config + "   <- snapshot was a real copy");

        System.out.println("\n=== 2. Bounded history ===");
        history.checkpoint(config, "v1");
        history.checkpoint(config, "v2");
        history.checkpoint(config, "v3");      // drops v1
        history.rollback(config);
        history.rollback(config);
        history.rollback(config);

        System.out.println("\n=== 3. The shallow-copy bug, shown in isolation ===");
        Map<String, String> liveEnv = new HashMap<>(Map.of("LOG_LEVEL", "info"));
        Map<String, String> badSnapshot = liveEnv;                  // shares the live map
        Map<String, String> goodSnapshot = new HashMap<>(liveEnv);  // owns a copy
        liveEnv.put("LOG_LEVEL", "debug");
        System.out.println("  bad snapshot  = " + badSnapshot + "  <- changed with the live object");
        System.out.println("  good snapshot = " + goodSnapshot);

        System.out.println("\n=== 4. The caretaker cannot open a snapshot ===");
        DeploymentConfig.Snapshot s = config.save("peek");
        System.out.println("  caretaker can read only: label=" + s.label());
        try {
            config.restore(() -> "forged");                         // a Snapshot not made by the originator
        } catch (IllegalArgumentException e) {
            System.out.println("  forged snapshot refused: " + e.getMessage());
        }
    }
}
```

Output:

```
=== 1. Checkpoint, bad rollout, rollback ===
  live: replicas=3 image=api:1.4 env={LOG_LEVEL=info}
  [history] checkpoint 'before 2.0' taken
  live: replicas=10 image=api:2.0 env={FEATURE_X=on, LOG_LEVEL=debug}
  [history] rolled back to 'before 2.0'
  live: replicas=3 image=api:1.4 env={LOG_LEVEL=info}   <- snapshot was a real copy

=== 2. Bounded history ===
  [history] checkpoint 'v1' taken
  [history] checkpoint 'v2' taken
  [history] full, dropped oldest checkpoint 'v1'
  [history] checkpoint 'v3' taken
  [history] rolled back to 'v3'
  [history] rolled back to 'v2'
  [history] nothing to roll back to

=== 3. The shallow-copy bug, shown in isolation ===
  bad snapshot  = {LOG_LEVEL=debug}  <- changed with the live object
  good snapshot = {LOG_LEVEL=info}

=== 4. The caretaker cannot open a snapshot ===
  caretaker can read only: label=peek
  forged snapshot refused: not a snapshot produced by DeploymentConfig
```

The bug that makes a snapshot useless

If the memento keeps a **pointer to the very map (or list) the originator goes on mutating**, you have not taken a snapshot: the "saved" copy drifts along with the live object, and a restore just re-applies today's state. Nothing throws and nothing looks wrong — until somebody notices the rollback had no effect. Always copy mutable parts (`Map.copyOf`, `new TreeMap<>(...)`, deep copies — copies that also duplicate the objects inside — of nested objects). It is the same shallow-vs-deep copy decision as the Prototype pattern (Chapter 8).

Two more things to bound: history depth — keep a cap and evict the oldest entry, because a short undo list is useful while an endless one is just a memory leak (memory that keeps growing and is never released) — and **state that cannot be captured** (live sockets — open network connections — open files, data owned by another system). A snapshot that claims to cover those will fail, usually long after the mistake was made.

Memento or Command undo?

	Memento	Command undo
Saved per step	A full copy of the state at that moment	The operation plus the little bit of state its reversal needs
How undo happens	Re-install an earlier copy	Execute the opposite operation
Memory cost per step	Proportional to the state size	Proportional to one operation
Correctness risk	Low — the copy was a genuine state	Every reverse operation must be written correctly
Prefer it when	Reversal is tricky or the state is small	Reversal is simple or the state is big

A drawing app that snapshots the whole canvas on every brush stroke runs out of memory — use Command ("stroke added, here are its points"). A wizard (a multi-step form) where step 3 changes six interacting fields is the opposite — six correct inverses are hard, one snapshot is easy. **Real systems combine them:** take a snapshot occasionally, log commands in between; to go back, restore the nearest snapshot and replay commands forward. Databases and event-sourced systems (Chapter 11) do exactly this.

Trade-offs

Pros	Cons
Rollback is possible while the internals stay private.	Memory use rises with every stored snapshot, quickly for large state.
New fields in the originator need no change in the caretaker.	Mutable parts must be deep-copied, or snapshots share live data.
The originator controls the restore, so it can be all-or-nothing, and it returns a state that really existed.	Some languages cannot truly hide the snapshot, only by convention.
Checkpoints, wizards, save games, transactions: one mechanism.	Some state (sockets, external systems) cannot be captured at all.

When not to use it

- **Large state that changes often** — use Command undo or periodic snapshots + replay.
- **State you cannot capture** (sockets, file handles, remote systems).
- **The object is immutable.** An old instance already *is* a perfect snapshot; holding a reference to it is enough (which is why functional code rarely names this pattern).
- **Each operation has an easy exact inverse** — per-step Command undo costs less memory.

Common mistakes

1. Storing references to mutable internals (the shallow-copy bug).
2. Letting the caretaker read or edit the snapshot's fields (encapsulation lost).
3. Restoring field by field from outside (not atomic).
4. Unbounded history.
5. Snapshotting things that cannot really be restored (connections, locks).

Real-world examples

- **Database transactions** (groups of changes that succeed or fail together): `BEGIN` ... `ROLLBACK`. The DB keeps enough information (undo logs) to put things back; your code never sees it.
- **Version control** (tools that record the history of files): a Git commit is a snapshot of the tree; `git checkout` restores it, and the history (the caretaker) does not need to understand file contents to do its job.
- **Kubernetes Deployments** (Kubernetes runs and manages containers across many machines) keep old ReplicaSets (the group of identical running copies for each version) as revision history (bounded by `revisionHistoryLimit`, default 10), and `kubectl rollout undo` restores a previous pod template — a bounded caretaker, just like ours.
- **VM / container snapshots** (VM = virtual machine, a whole computer simulated in software; a container is a lighter, isolated, packaged process) and infrastructure state files that make `rollback` possible.
- **Game saves and checkpoints**; **form wizards** with a "back" button; **editor undo** for small documents.
- In Java, `Serializable` or a JSON mapper is often *how* a memento is stored when it must outlive the process; a `record` gives an immutable memento in one line.
- **Other languages**: in Python, `copy.deepcopy(self.__dict__)` is a quick memento and `dataclasses.replace` covers immutable objects; in JavaScript, `structuredClone(state)` deep-copies plain data.

In the interview

> **30-second answer:** "With Memento an object packs its state into a token that only the object itself can unpack. The originator produces and consumes snapshots, the memento holds the data, and the caretaker chooses when to save or roll back without ever inspecting the contents. Restoring is a single call the originator controls, so it can be all-or-nothing, and a new field in the object never touches the history code. I'd make sure the snapshot deep-copies mutable parts, bound the history, and compare it with Command undo: Memento stores state, Command stores actions; real systems combine periodic snapshots with a command log."

Follow-ups:

- *How do you enforce that the caretaker can't read the memento in Java?* A narrow public interface (e.g. only a label) implemented by a private nested record; or package-private accessors.
- *Memento vs Prototype?* Each copies state. A prototype clone becomes a live object you keep working with. A memento does nothing on its own; its sole purpose is to be given back to the originator.
- *Memento vs a backup?* A backup targets disaster recovery (getting systems back after a major failure) and lives outside the process (disk, another region — a separate data-centre location). A memento is usually an in-memory checkpoint for undo. Same idea, very different lifetime and cost.
- *How would you implement undo for a large document?* Command undo for each edit, plus periodic snapshots so undo far back doesn't replay thousands of commands.
- *How does it relate to serialization?* Serialization is one common *storage format* for a memento; the pattern itself is about *access* — only the originator may look inside.

Self-check

1. **Q:** Name the three roles. **A:** Originator, memento, caretaker.
2. **Q:** Who may read the memento's contents? **A:** Only the originator.
3. **Q:** Why copy the `env` map in the snapshot? **A:** Otherwise the snapshot shares the live map and changes with it.
4. **Q:** When is Command undo better than Memento? **A:** When state is large and each action has an easy inverse.
5. **Q:** What does Kubernetes' `revisionHistoryLimit` correspond to? **A:** A bounded caretaker history.

TL;DR

The object **snapshots itself**; the caretaker **stores but cannot read**; restore is **one call the originator controls**. **Deep-copy** mutable parts, **bound** the history, and choose Memento (store state) vs Command (store actions) by which is smaller.

10.6 Interpreter

(Not covered by a lesson in the course beyond its one-line entry in the behavioral-patterns table; this section is from the GoF book and standard practice.)

What it is

Interpreter represents the **grammar** of a small language as a set of classes — **one class per grammar rule** — and evaluates a sentence of that language by building a **tree** of those objects and asking the root to `interpret` itself.

Everyday analogy: a calculator. `2 + 3 * 4` is a sentence in a tiny language. The calculator understands that `*` binds tighter than `+`, builds the structure `2 + (3 * 4)` in its head, and works it out from the inside.

Key terms:

- **Grammar** — the rules saying which sentences are valid, e.g. "an expression is terms joined by `+` or `-`".
- **DSL (domain-specific language)** — a small language for one job: search filters, pricing rules, feature-flag conditions, cron schedules (a compact text format for repeating times, such as "every weekday at 9").
- **AST (abstract syntax tree)** — the tree form of a sentence, with operators as inner nodes and numbers/variables as leaves.
- **Terminal expression** — a leaf rule (a number, a variable).
- **Nonterminal expression** — a rule made of other expressions (addition, multiplication).
- **Context** — outside information the evaluation needs, e.g. the current values of variables.

The problem it solves

Some problems keep coming back as *sentences*: "price * qty − discount", "age > 18 AND country = IN", "every weekday at 9:00". Hard-coding each rule in Java means a redeploy (building and releasing the application again) for every new rule. Letting business users write rules in a tiny language, and interpreting them, turns rules into **data**. Interpreter is the simplest object-oriented way to evaluate such a language when the grammar is **small**.

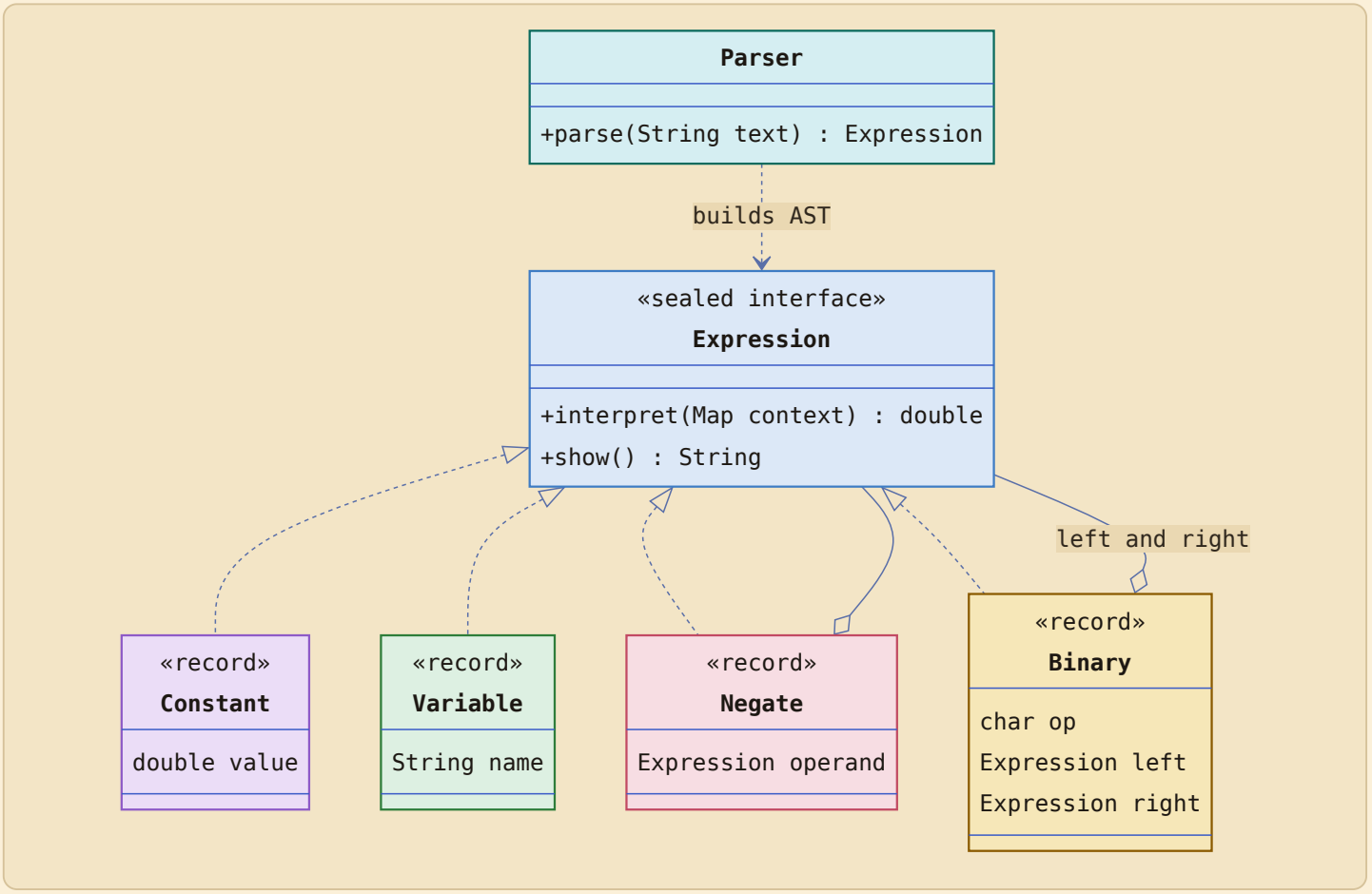
How it works

1. **Write the grammar** (here in EBNF, a standard notation):

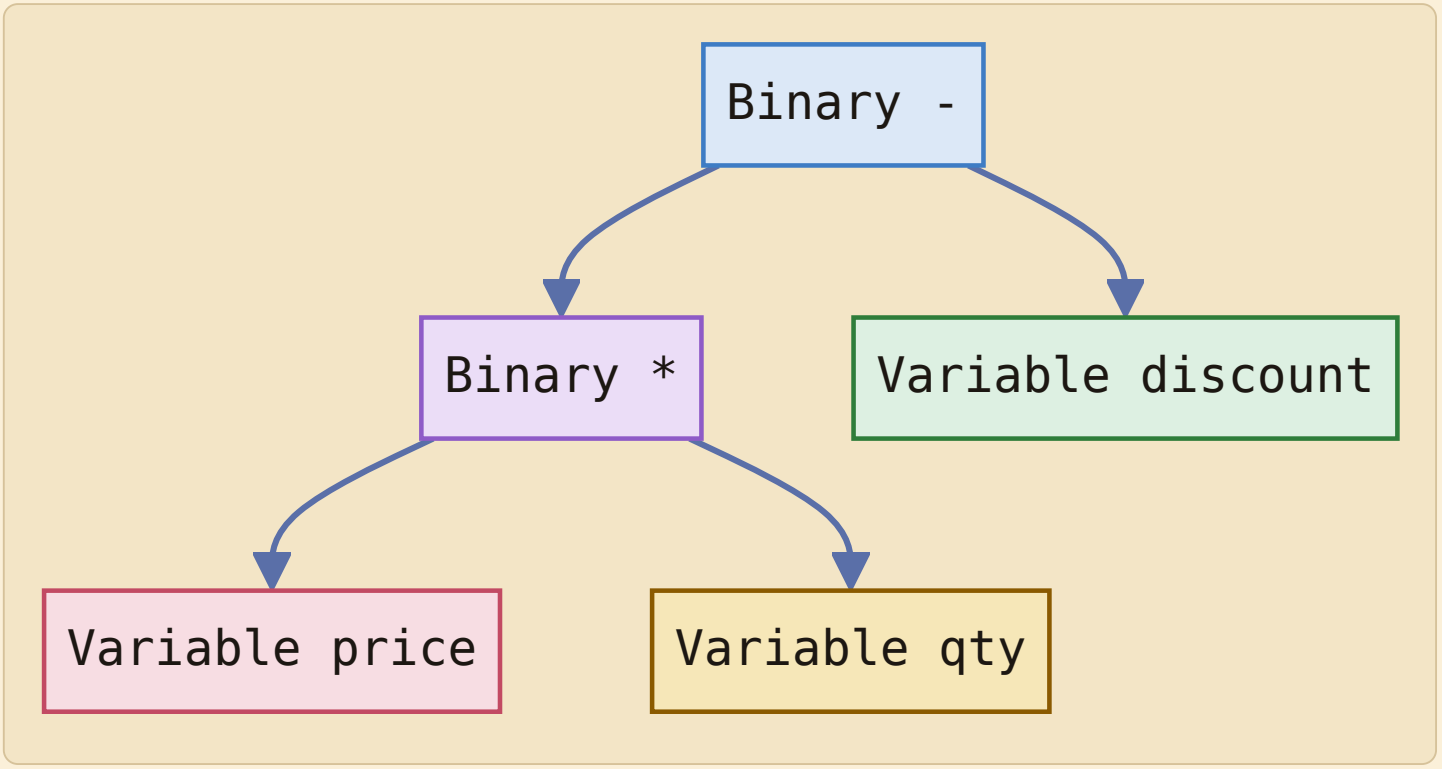
   ```
   expr   := term   (('+' | '-') term)*
   term   := factor (('*' | '/') factor)*
   factor := NUMBER | NAME | '(' expr ')' | '-' factor
   ```

2. **One class per rule**: `Constant` and `Variable` (terminals), `Negate` and `Binary` (nonterminals), all implementing `Expression.interpret(context)`.
3. **Build the AST.** GoF deliberately leaves **parsing** (reading text and building a structure such as the AST from it) out of the pattern; any parser will do. We use a small *recursive-descent* parser: one method per grammar rule, which calls the others.
4. **Interpret**: the root calls `interpret` on its children and combines their results (a *recursive* evaluation).
5. **Reuse the tree**: parse once, interpret many times with different contexts.

Structure (UML)



The AST for `price * qty - discount`:



Notice the structure is a **Composite** (Chapter 9): leaves and inner nodes share one interface. Interpreter is essentially "Composite + an `interpret` operation".

Java example

`lld.ch10.interpreter`: `Expression` is a **sealed interface** (only the four listed records may implement it), each record is one grammar rule, and `Parser` is a recursive-descent parser whose methods `expr`, `term` and `factor` mirror the grammar lines. Errors (unknown variable, division by zero, bad syntax) are reported with clear messages.

> **Why `double` here?** This is a general-purpose arithmetic calculator used to teach the pattern, and variable names like `price` are only sample data. A real pricing or promotion DSL must evaluate money with `BigDecimal` (and an explicit `RoundingMode` for division), exactly as the domain code elsewhere in this chapter does — `double` cannot represent values like 0.10 exactly.

```java
// File: code/lld/ch10/interpreter/Expression.java
package lld.ch10.interpreter;

import java.util.Map;

/**
 * AbstractExpression. One class per grammar rule; each knows how to interpret itself.
 * Grammar (EBNF):
 *   expr   := term   (('+' | '-') term)*
 *   term   := factor (('*' | '/') factor)*
 *   factor := NUMBER | NAME | '(' expr ')' | '-' factor
 */
public sealed interface Expression
        permits Expression.Constant, Expression.Variable, Expression.Negate, Expression.Binary {

    /** Evaluate against a context (variable name -> value). */
    double interpret(Map<String, Double> context);

    /** Print the tree back as fully-parenthesised text. */
    String show();

    /** TerminalExpression: a literal number. */
    record Constant(double value) implements Expression {
        public double interpret(Map<String, Double> ctx) { return value; }
        public String show() { return value == Math.rint(value) ? String.valueOf((long) value) : String.valueOf(value); }
    }

    /** TerminalExpression: a variable looked up in the context. */
    record Variable(String name) implements Expression {
        public double interpret(Map<String, Double> ctx) {
            Double v = ctx.get(name);
            if (v == null) throw new IllegalArgumentException("unknown variable '" + name + "'");
            return v;
        }
        public String show() { return name; }
    }

    /** NonterminalExpression: unary minus. */
    record Negate(Expression operand) implements Expression {
        public double interpret(Map<String, Double> ctx) { return -operand.interpret(ctx); }
        public String show() { return "(-" + operand.show() + ")"; }
    }

    /** NonterminalExpression: + - * / . Interprets its children, then combines. */
    record Binary(char op, Expression left, Expression right) implements Expression {
        public double interpret(Map<String, Double> ctx) {
            double l = left.interpret(ctx), r = right.interpret(ctx);
            return switch (op) {
                case '+' -> l + r;
                case '-' -> l - r;
                case '*' -> l * r;
                case '/' -> {
                    if (r == 0) throw new ArithmeticException("division by zero in " + show());
                    yield l / r;
                }
                default -> throw new IllegalStateException("unknown operator " + op);
            };
        }
        public String show() { return "(" + left.show() + " " + op + " " + right.show() + ")"; }
    }
}
```

```java
// File: code/lld/ch10/interpreter/Parser.java
package lld.ch10.interpreter;

/**
 * Recursive-descent parser: turns text into an Abstract Syntax Tree (AST).
 * Parsing is NOT part of the GoF Interpreter pattern; the pattern starts once the tree exists.
 * One method per grammar rule, mirroring the grammar in Expression.
 */
public final class Parser {
    private final String src;
    private int pos = 0;

    private Parser(String src) { this.src = src; }

    public static Expression parse(String text) {
        Parser p = new Parser(text);
        Expression e = p.expr();
        p.skipSpaces();
        if (p.pos != p.src.length()) throw p.error("unexpected '" + p.src.charAt(p.pos) + "'");
        return e;
    }

    private Expression expr() {
        Expression left = term();
        while (peek('+') || peek('-')) {
            char op = src.charAt(pos++);
            left = new Expression.Binary(op, left, term());   // left-associative
        }
        return left;
    }

    private Expression term() {
        Expression left = factor();
        while (peek('*') || peek('/')) {
            char op = src.charAt(pos++);
            left = new Expression.Binary(op, left, factor());
        }
        return left;
    }

    private Expression factor() {
        skipSpaces();
        if (pos >= src.length()) throw error("unexpected end of input");
        char c = src.charAt(pos);
        if (c == '-') { pos++; return new Expression.Negate(factor()); }
        if (c == '(') {
            pos++;
            Expression inner = expr();
            if (!peek(')')) throw error("missing ')'");
            pos++;
            return inner;
        }
        int start = pos;
        if (Character.isDigit(c) || c == '.') {
            while (pos < src.length() && (Character.isDigit(src.charAt(pos)) || src.charAt(pos) == '.')) pos++;
            return new Expression.Constant(Double.parseDouble(src.substring(start, pos)));
        }
        if (Character.isLetter(c)) {
            while (pos < src.length() && Character.isLetterOrDigit(src.charAt(pos))) pos++;
            return new Expression.Variable(src.substring(start, pos));
        }
        throw error("unexpected '" + c + "'");
    }

    private boolean peek(char expected) {
        skipSpaces();
        return pos < src.length() && src.charAt(pos) == expected;
    }

    private void skipSpaces() {
        while (pos < src.length() && Character.isWhitespace(src.charAt(pos))) pos++;
    }

    private IllegalArgumentException error(String msg) {
        return new IllegalArgumentException(msg + " at position " + pos + " in \"" + src + "\"");
```

```
    }
}
```

```
// File: code/lld/ch10/interpreter/Main.java
package lld.ch10.interpreter;

import java.util.List;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        Map<String, Double> context = Map.of("price", 250.0, "qty", 3.0, "discount", 50.0, "a", 7.0, "b", 3.0);
        System.out.println("context: " + new java.util.TreeMap<>(context));

        List<String> inputs = List.of(
            "price * qty - discount",
            "2 + 3 * 4",
            "(2 + 3) * 4",
            "(a + b) * (a - b)",
            "-a + 10 / 4",
            "price / (qty - 3)",
            "price * tax",
            "2 + * 3");

        for (String text : inputs) {
            try {
                Expression ast = Parser.parse(text);          // build the tree once...
                double value = ast.interpret(context);        // ...interpret it (could be many times)
                System.out.printf("  %-24s AST %-32s = %s%n", text, ast.show(), value);
            } catch (IllegalArgumentException | ArithmeticException e) {
                System.out.printf("  %-24s ERROR: %s%n", text, e.getMessage());
            }
        }

        System.out.println("\nSame tree, different contexts (parse once, evaluate many):");
        Expression total = Parser.parse("price * qty - discount");
        for (double q : new double[] {1, 2, 4}) {
            System.out.println("  qty=" + (int) q + " -> " + total.interpret(Map.of("price", 250.0, "qty", q, "discount",
50.0)));
        }
    }
}
```

Output:

```
context: {a=7.0, b=3.0, discount=50.0, price=250.0, qty=3.0}
  price * qty - discount   AST ((price * qty) - discount)       = 700.0
  2 + 3 * 4                AST (2 + (3 * 4))                    = 14.0
  (2 + 3) * 4              AST ((2 + 3) * 4)                    = 20.0
  (a + b) * (a - b)        AST ((a + b) * (a - b))              = 40.0
  -a + 10 / 4              AST ((-a) + (10 / 4))                = -4.5
  price / (qty - 3)        ERROR: division by zero in (price / (qty - 3))
  price * tax              ERROR: unknown variable 'tax'
  2 + * 3                  ERROR: unexpected '*' at position 4 in "2 + * 3"

Same tree, different contexts (parse once, evaluate many):
  qty=1 -> 200.0
  qty=2 -> 450.0
  qty=4 -> 950.0
```

Notice operator precedence (which operator binds first) (`2 + 3 * 4` = 14, but `(2 + 3) * 4` = 20) comes from the **grammar**: `term` handles `*` and `/` below `expr`, so they bind tighter.

Trade-offs

Pros	Cons
Easy to change and extend the grammar: a new rule is a new class.	One class per rule: large grammars become a jungle of classes.
Each rule's evaluation is small and testable.	Tree-walking is slow compared with compiling to bytecode (compiled JVM instructions).
Rules become data (stored in a DB, edited by users).	Adding a new *operation* (pretty-print, type-check, optimise) means touching every class — that is where **Visitor** (Chapter 11) comes in.
Parse once, evaluate many times.	You still need a parser, which the pattern does not give you.

When not to use it

- **The grammar is large or complex** (a real programming language, SQL). Use a parser generator (a tool that writes a parser from a grammar file) such as **ANTLR** and a compiler/visitor design.
- **Performance matters** and expressions run millions of times — compile to bytecode or use an existing engine.
- **An existing language fits.** Regular expressions (patterns for matching text), SpEL (Spring's expression language), JSONPath (queries into JSON data), CEL (Google's Common Expression Language), or a scripting engine (an embedded interpreter for a full language such as JavaScript) may already do the job.
- **Security matters.** Never let users run a *general-purpose* script engine on your server; a small, restricted DSL is safer precisely because it can do only what its grammar allows.

Common mistakes

1. **Mixing parsing and evaluation** in one pass, so each evaluation re-parses the text.
2. **Ignoring precedence and associativity** (associativity decides how operators of equal precedence group: `8 - 3 - 2` must be `(8 - 3) - 2`; our `expr` loop builds left-associative trees).
3. **Poor error messages** — always report the position.
4. **Adding many operations to every node** instead of switching to Visitor or pattern matching (a `switch` that branches on an object's type and unpacks its fields).
5. **Growing a "tiny" DSL** until it is an accidental programming language.

Real-world examples

- `java.util.regex.Pattern`: compiling a regex builds an internal tree of node objects, and a `Matcher` walks it — a classic interpreter.
- **Spring Expression Language (SpEL)**: `SpelExpressionParser` parses text like `#user.age > 18` into an AST of `SpelNode`s, and `getValue()` evaluates it against a context. Jakarta Expression Language (EL) is similar.
- **SQL engines** evaluate `WHERE` clauses as expression trees; **spreadsheet formulas** (`=SUM(A1:A3)*2`) are a DSL.
- **Rule and pricing engines**: e-commerce promotions ("10% off if cart > 999 and category = electronics"), feature-flag targeting rules, and Google's **CEL** (Common Expression Language) used in Kubernetes validation rules and IAM (identity and access management — who may do what) conditions.
- **Cron expressions** (`0 9 * * MON-FRI`), e.g. Quartz's (a Java job-scheduling library) `CronExpression`.

In the interview

> **30-second answer:** "Interpreter models a small language's grammar as classes, one per rule — terminals like numbers and variables, nonterminals like addition. A sentence is parsed into an AST made of those objects and evaluated by calling `interpret` on the root with a context. It's really Composite plus an evaluate operation. It's great for small DSLs like pricing rules or filters; for big grammars I'd use a parser generator like ANTLR, and when I need many operations over the tree I'd switch to Visitor or sealed types with pattern matching."

Follow-ups:

- *Is parsing part of the pattern?* No. GoF assumes the tree already exists; parsing is a separate concern (hand-written recursive descent, or ANTLR).
- *Interpreter vs Visitor?* Interpreter puts evaluation **inside** the node classes. Visitor takes operations **out** into separate classes. A real interpreter usually starts as the first and becomes the second when a second pass (type-check, optimise, pretty-print) arrives.

- *Interpreter vs Composite?* Composite is the structure; Interpreter uses that structure to evaluate a language.
- *How would you make it faster?* Cache parsed trees; constant-fold (pre-compute parts with no variables); or compile to bytecode/lambdas once.
- *How do you design a rule engine for promotions?* A small grammar (comparisons, AND/OR/NOT, fields from the cart), rules stored as text, parsed once into trees and cached, evaluated per cart.

Self-check

1. **Q:** What is the difference between terminal and nonterminal expressions? **A:** Terminals are leaves (numbers, variables); nonterminals combine other expressions (operators).
2. **Q:** What is the "context"? **A:** External information needed for evaluation, such as variable values.
3. **Q:** Why does `2 + 3 * 4` give 14? **A:** The grammar puts `*` in `term`, below `expr`, so it binds tighter.
4. **Q:** Name a JDK class that is an interpreter. **A:** `java.util.regex.Pattern`.
5. **Q:** When should you not use it? **A:** Large grammars, performance-critical evaluation, or when an existing language fits.

TL;DR

One class per grammar rule; parse a sentence into an **AST**; evaluate by calling `interpret` recursively with a **context**. Perfect for **tiny DSLs**; for big grammars use a parser generator and Visitor.

10.7 Chapter recap: the six patterns side by side

Pattern	The object you create	The question it answers	Classic JDK / real example	Often confused with
Chain of Responsibility	A handler per check	*Who* should handle this request?	Servlet filters, logging handlers	Decorator (all layers run)
Command	An object per request	How do I store, queue or undo *this request*?	`Runnable`, `UndoManager`	Strategy (how vs what)
Iterator	A cursor object	How do I walk this without knowing its shape?	`Iterator`, `Spliterator`, `ResultSet`	Visitor (what to do at each node)
Mediator	One coordinator	How do many objects cooperate without knowing each other?	UI controllers, chat servers, saga orchestrators	Observer (broadcast only), Facade
Memento	A sealed snapshot	How do I go back to an earlier state safely?	DB rollback, Git, `kubectl rollout undo`	Command undo, Prototype
Interpreter	A node per grammar rule	How do I evaluate sentences of a small language?	`Pattern`, SpEL	Composite, Visitor

Interview combo facts to remember:

- The request flowing through a **Chain of Responsibility** is frequently a **Command** object.
- A **Mediator** often creates and dispatches **Commands**.
- **Memento + Command** together give scalable undo (snapshots + replay).
- **Iterator + Composite** walk trees; **Interpreter = Composite + evaluate**; when operations multiply, move to **Visitor** (Chapter 11).

Self-check (whole chapter)

1. **Q:** Which two GoF behavioral patterns are class-scoped (inheritance-based)? **A:** Template Method and Interpreter.
2. **Q:** A request must pass auth, then a rate limit, then validation, and any step can reject it. Which pattern? **A:** Chain of Responsibility.
3. **Q:** You need undo/redo for a text editor. Which patterns? **A:** Command (per edit), optionally Memento for periodic snapshots.
4. **Q:** Five form widgets depend on each other's values. Which pattern? **A:** Mediator.

5. **Q:** Your team needs business users to write discount rules. Which pattern? **A:** Interpreter (a small DSL), possibly with Specification (Chapter 11).

> **TL;DR**

Behavioral patterns turn **hidden control flow** into **named objects**: handlers (CoR), requests (Command), cursors (Iterator), coordinators (Mediator), snapshots (Memento) and grammar nodes (Interpreter). Each one buys flexibility with indirection — use it when the flexibility is a real requirement, not "just in case".

Chapter 11 — Behavioral Design Patterns (Part 2) & Beyond GoF

> **What you will learn.** The remaining five **GoF** ("Gang of Four" — the four authors of the 1994 book *Design Patterns*, which catalogued 23 classic patterns) behavioral patterns — **Observer, State, Strategy, Template Method, Visitor** — including the three that share one class diagram (Strategy, State, Bridge) and how to tell them apart. Then six widely used patterns that are *not* in the GoF book but come up constantly in interviews and real code: **Null Object, Dependency Injection, Repository, Specification, MVC** (the Head First "compound pattern") and **Event Sourcing (lite)**. The chapter ends with a "which behavioral pattern do I need?" decision table.

How to run the code in this chapter

```
cd chapters/11-patterns-behavioral-2/code
javac --release 21 -Xlint:all -d /tmp/lld_11 $(find . -name '*.java')   # compiles with 0 warnings
java -cp /tmp/lld_11 lld.ch11.observer.Main
# also: state, strategy, templatemethod, visitor, nullobject, di, repository, specification, mvc, eventsourcing
```

Chapter 10 introduced behavioral patterns: patterns about **how objects share work and talk to each other**. Keep one idea in mind for this chapter: most of these patterns take a conditional (`if` / `switch`) or a hard-coded call and turn it into an **object** that can be swapped, subscribed, or visited.

11.1 Observer

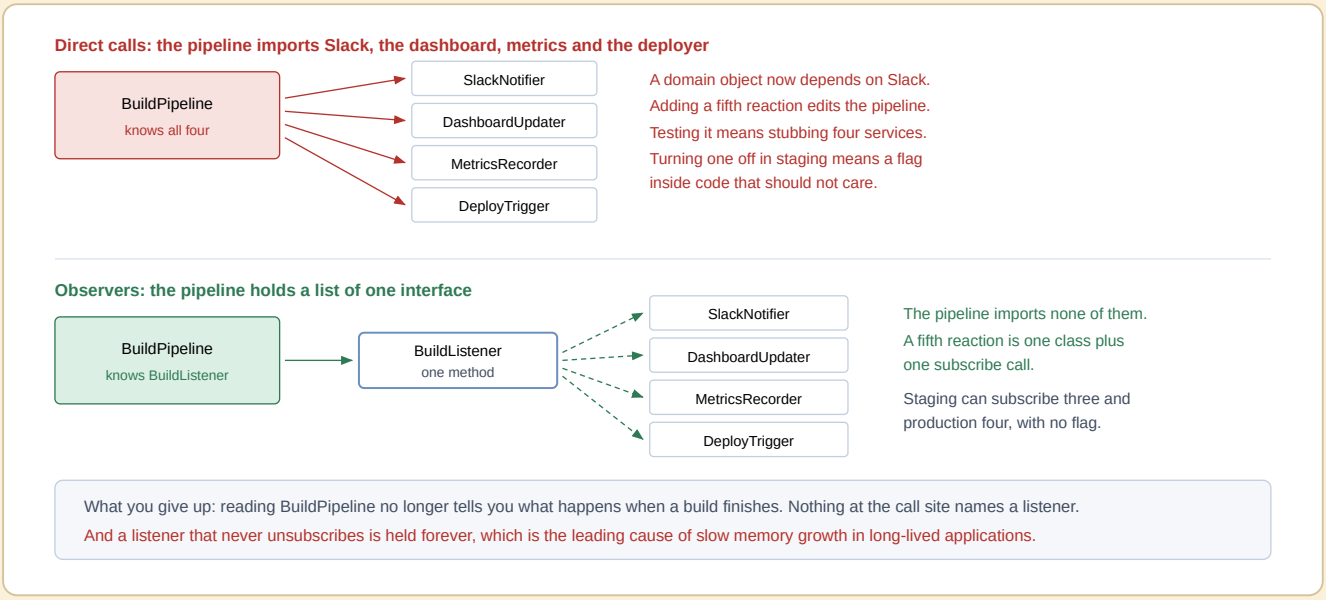


Figure — Top: the build pipeline calls Slack, dashboard, metrics and deployer directly. Bottom: it holds one list of BuildListener and imports none of them (course diagram)

What it is

Observer lets one object (the **subject**, also called *publisher* or *observable*) notify many dependent objects (the **observers**, *subscribers* or *listeners*) when something happens — **without knowing who they are or what they do**.

Everyday analogy: a YouTube channel. You subscribe; when a new video is published, every subscriber is notified. The channel does not know or care what each subscriber does with the notification, and you can unsubscribe at any time.

GoF intent (paraphrased): set up a one-to-many dependency, so that when one object's state changes, every object that depends on it is told and can update itself automatically.

GoF participants (and who plays them in our code):

Participant	Role	In `lld.ch11.observer`
Subject	Knows its observers (any number) and offers attach/detach operations	`BuildPipeline` (`addListener` + the returned unsubscribe handle)
Observer	The update interface that subjects call	`BuildListener`
ConcreteSubject	Holds the state observers care about; notifies them when it changes	`BuildPipeline` (GoF split Subject into an abstract class and a concrete one; in modern Java one class usually plays both)
ConcreteObserver	Implements the update interface; may keep its own state in step with the subject	`SlackNotifier`, `DashboardUpdater`, `MetricsRecorder`, `DeployTrigger`

The problem it solves

Suppose four reactions must follow every finished build: a Slack message, a dashboard refresh, a metrics record and a deployment. The naive version calls all four itself:

```
void finish(int buildNumber, String result) {
    slack.post(buildNumber, result);
    dashboard.update(result);
    metrics.record(buildNumber);
    deployer.start();
}
```

This costs more than it looks: the build pipeline now **depends on a chat service**; a fifth reaction means editing the pipeline; a **unit test** (an automated test that checks one class on its own) must fake four services; and disabling deployment in **staging** (the pre-production copy of the system used for final checks) needs an environment flag inside a class that ought to know nothing about environments.

Observer replaces the four references with **one list of one interface**. Anything that cares subscribes. **The source only announces; every listener chooses its own reaction.** No one orchestrates them, and listeners are unaware of one another.

How it works

1. Define an **observer interface** with one method, e.g. `onBuildFinished(buildNumber, result)`.
2. The **subject** keeps a list of observers and offers subscribe and unsubscribe operations (classically `addListener` / `removeListener`; our `addListener` instead returns a handle that unsubscribes).
3. When its state changes, the subject **first finishes changing its state**, then loops over the list and calls each observer.
4. Observers can be added and removed at run time (e.g. staging subscribes three listeners, production four).

Push vs pull — two ways to deliver the data:

	Push model	Pull model
What the observer receives	All the data about the change (e.g. a `Reading` record)	Just "I changed" plus a reference to the subject
How the observer gets data	It is in the call	It calls getters on the subject for what it needs
Pros	Simple; observers don't depend on the subject's getters	Each observer takes only what it needs; the interface doesn't change when new data is added
Cons	Sends data some observers don't need; adding data changes the interface	Observers depend on the subject; they must work out *what* changed

`java.util.Observer.update(Observable o, Object arg)` supported both: `o` for pull, `arg` for push. *Head First Design Patterns* shows the same trade-off with its weather-station example.

Structure (UML)

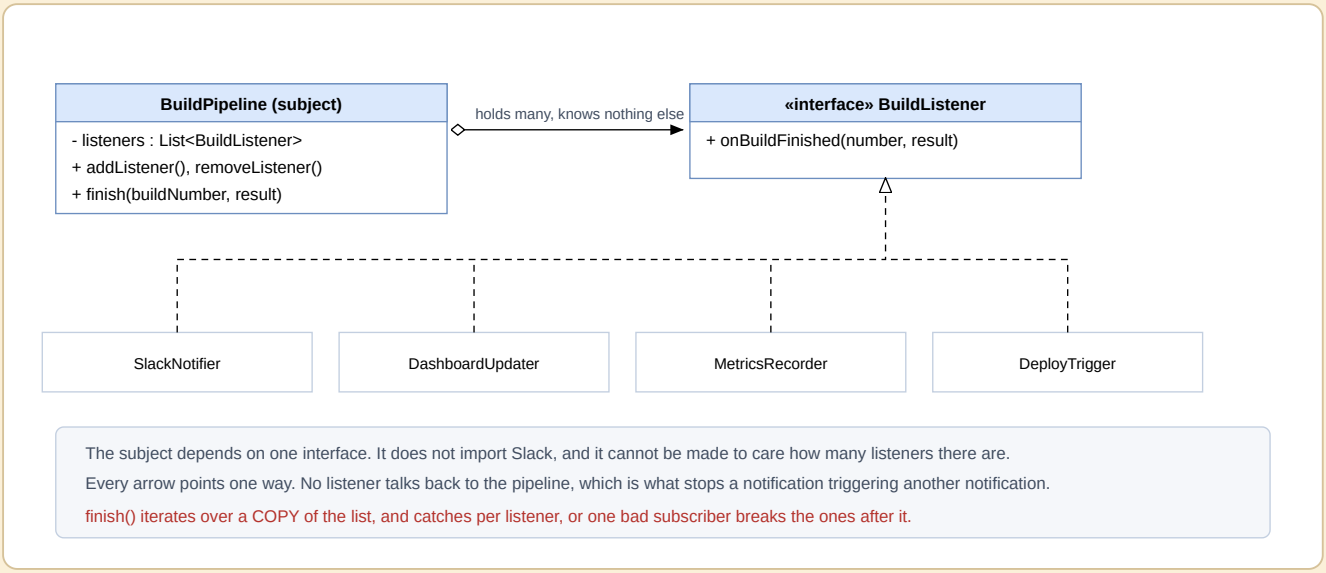
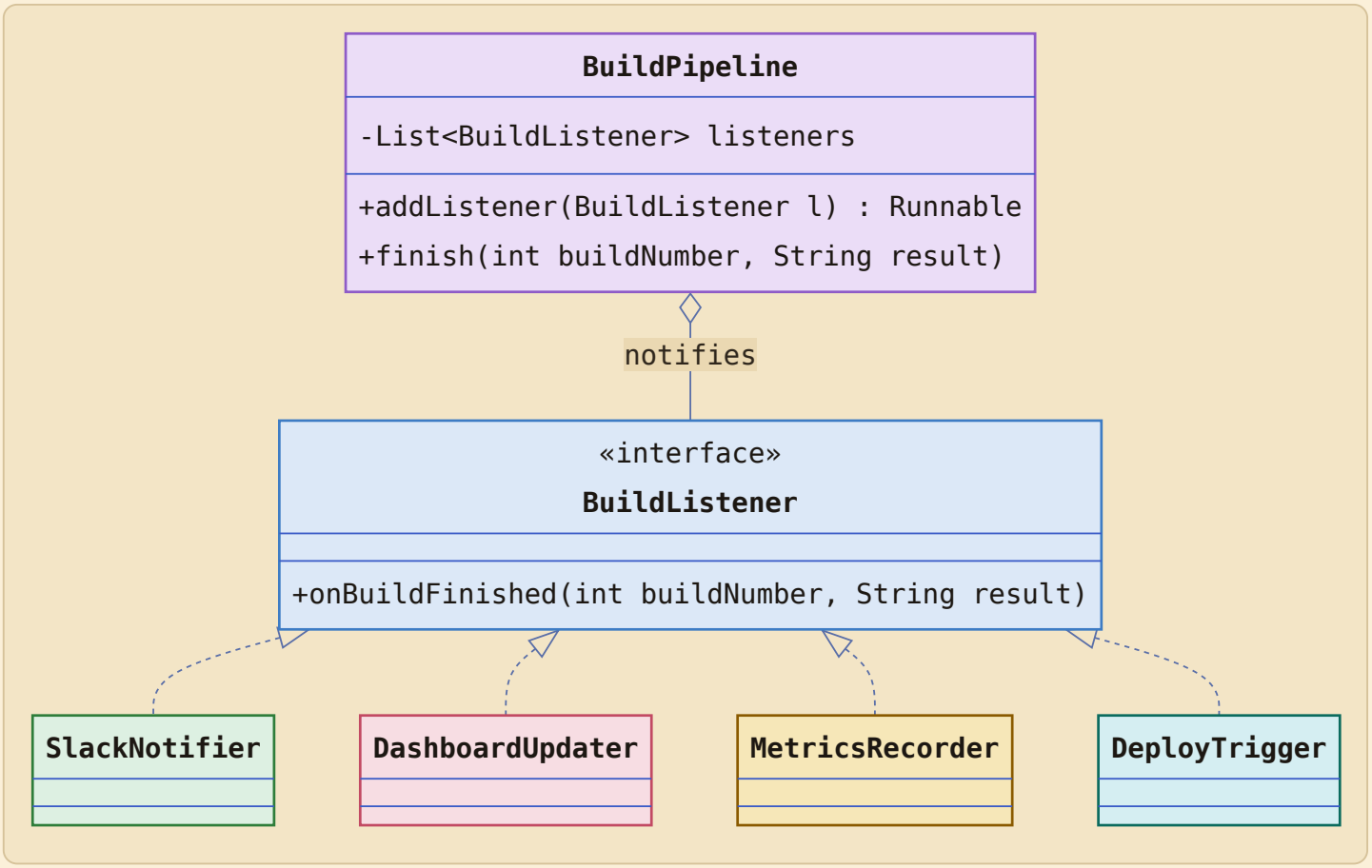
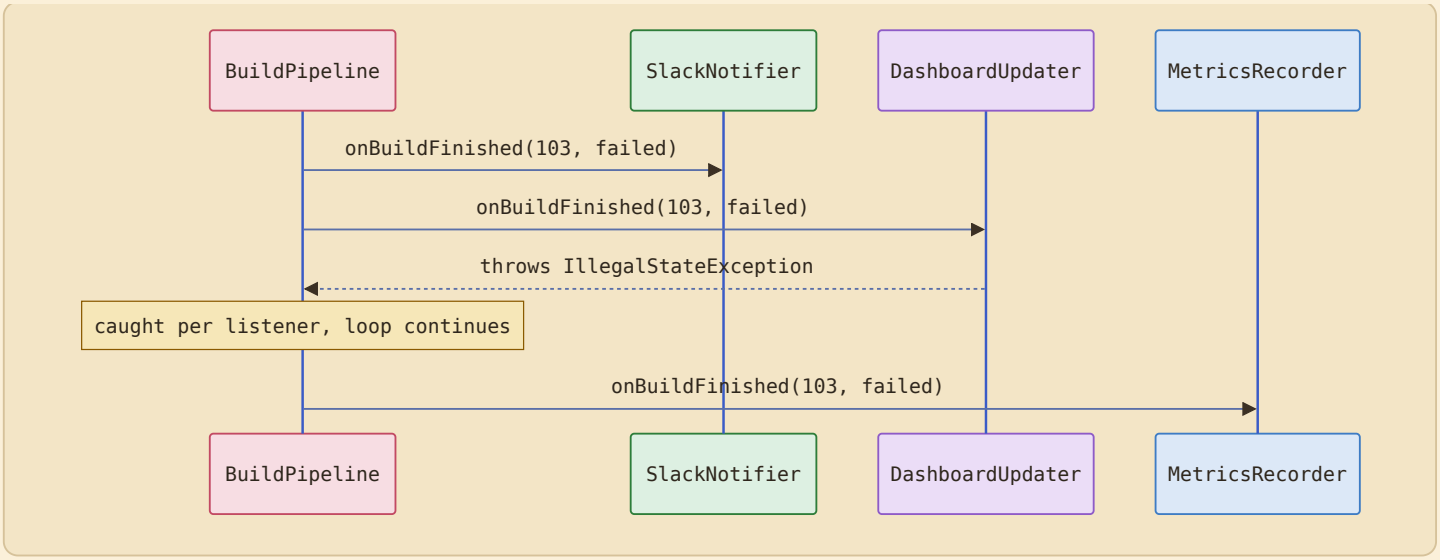


Figure — Class diagram: BuildPipeline holds a list of BuildListener; SlackNotifier, DashboardUpdater, MetricsRecorder and DeployTrigger implement it (course diagram)

In the diagram above (a **UML** class diagram — UML, the Unified Modeling Language, is the standard box-and-arrow notation for classes and how they relate) all dependencies run in a single direction. Listeners never call the pipeline back, so one notification cannot set off another.

Our Java design:





Java example

`lld.ch11.observer` shows four things:

1. **Build pipeline.** Listeners live in a `CopyOnWriteArrayList` (a JDK list that copies its internal array on every write, so readers never see it change under them), whose iterator works on a **snapshot** — so a listener that unsubscribes during notification cannot break the loop, and the list is **thread-safe** (safe to use from several *threads* — independent paths of execution running at the same time — at once). `finish` **catches exceptions per listener**, so a broken dashboard does not stop metrics. `addListener` returns a `Runnable` (Java's built-in interface for "a piece of code to run later", with one method `run()`) that unsubscribes: whoever subscribes owns the matching unsubscribe.
2. **Push vs pull** (`WeatherStation`): one display receives the whole `Reading`; the other receives the station and pulls only the temperature.
3. **A tiny event bus** (`EventBus`): subscribers register for an event *type* (a class); publishers just publish objects. The returned `Subscription` is the unsubscribe handle. `Main` closes it in a `finally` block, so the refunds handler is removed even if publishing throws. (`Subscription` is also `AutoCloseable` — an interface whose `close()` method is called automatically when a *try-with-resources* block ends — so `try (var s = bus.subscribe(...)) { ... }` works too. But if the body never uses `s`, `javac -Xlint` warns that the resource is never referenced. Java 22+ lets you write `try (var _ = ...)` to say "unused on purpose". On Java 21, `try`/`finally` is the warning-free form.) The bus's handler list is a `CopyOnWriteArrayList`, so subscribing and publishing from different threads is safe. Delivery is still **synchronous on the publisher's thread** (the publisher waits while each handler runs, one after another) and nothing is stored, so it is not a **queue** (a waiting line that stores items and hands them out later, in order).
4. `java.beans.PropertyChangeSupport` (the price is whole cents in an `int` — exact, never `double`), which the `Observable` **Javadoc** (the official API documentation generated from comments in Java source code) itself points to (the `java.beans` package) as the richer event model. Note it does **not** fire when the old and new values are both non-null and equal. (Since Java 9, `java.beans` lives in the `java.desktop` module; on the class path (the list of folders and JAR files where Java looks for classes) this needs nothing extra, but a modular app (one that declares its dependencies in a `module-info.java` file) must `requires java.desktop`.)

```java
// File: code/lld/ch11/observer/BuildListener.java
package lld.ch11.observer;

/** The OBSERVER interface: one method. A lambda can implement it. */
@FunctionalInterface
public interface BuildListener {
    void onBuildFinished(int buildNumber, String result);
}
```

```java
// File: code/lld/ch11/observer/BuildPipeline.java
package lld.ch11.observer;

import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

/** The SUBJECT: holds a list of one interface and imports none of the concrete reactions. */
public final class BuildPipeline {
    // CopyOnWriteArrayList iterates over a snapshot: a listener that unsubscribes
    // itself mid-notification cannot break the loop, and it is safe across threads.
    private final List<BuildListener> listeners = new CopyOnWriteArrayList<>();

    /** Returns a handle: whoever subscribes owns the matching unsubscribe (prevents leaks). */
    public Runnable addListener(BuildListener listener) {
        listeners.add(listener);
        return () -> listeners.remove(listener);
    }

    public void finish(int buildNumber, String result) {
        System.out.println("Build " + buildNumber + " finished: " + result);
        for (BuildListener l : listeners) {
            try {
                l.onBuildFinished(buildNumber, result);
            } catch (RuntimeException e) {          // one bad listener must not starve the rest
                System.out.println("  [pipeline] a listener failed: " + e.getMessage());
            }
        }
    }
}
```

```java
// File: code/lld/ch11/observer/SlackNotifier.java
package lld.ch11.observer;

final class SlackNotifier implements BuildListener {
    @Override
    public void onBuildFinished(int n, String result) {
        System.out.println("  [slack] #builds: build " + n + " " + result);
    }
}
```

```java
// File: code/lld/ch11/observer/DashboardUpdater.java
package lld.ch11.observer;

final class DashboardUpdater implements BuildListener {
    boolean offline = false;

    @Override
    public void onBuildFinished(int n, String result) {
        if (offline) throw new IllegalStateException("dashboard is offline");
        System.out.println("  [dashboard] status tile -> " + result);
    }
}
```

```java
// File: code/lld/ch11/observer/MetricsRecorder.java
package lld.ch11.observer;

final class MetricsRecorder implements BuildListener {
    private int recorded = 0;

    @Override
    public void onBuildFinished(int n, String result) {
        recorded++;
        System.out.println("  [metrics] recorded build " + n + " (total " + recorded + ")");
    }
}
```

```java
// File: code/lld/ch11/observer/DeployTrigger.java
package lld.ch11.observer;

final class DeployTrigger implements BuildListener {
    @Override
    public void onBuildFinished(int n, String result) {
        if (result.equals("success")) System.out.println("  [deploy] deploying build " + n);
    }
}
```

```java
// File: code/lld/ch11/observer/WeatherStation.java
package lld.ch11.observer;

import java.util.ArrayList;
import java.util.List;
import java.util.function.Consumer;

/** A subject that supports both notification styles, to compare PUSH and PULL. */
public final class WeatherStation {
    /** PUSH: the subject sends the data with the notification. */
    public record Reading(double tempC, double humidity, double pressureHpa) {}

    private final List<Consumer<Reading>> pushObservers = new ArrayList<>();
    private final List<Consumer<WeatherStation>> pullObservers = new ArrayList<>();
    private Reading current = new Reading(0, 0, 0);

    public void subscribePush(Consumer<Reading> o) { pushObservers.add(o); }
    public void subscribePull(Consumer<WeatherStation> o) { pullObservers.add(o); }

    // PULL: observers get only "I changed" plus a reference, then ask for what they need.
    public double temperature() { return current.tempC(); }
    public double humidity() { return current.humidity(); }

    public void measure(Reading r) {
        current = r;                                          // change state FIRST...
        pushObservers.forEach(o -> o.accept(r));              // ...then notify
        pullObservers.forEach(o -> o.accept(this));
    }
}
```

```java
// File: code/lld/ch11/observer/EventBus.java
package lld.ch11.observer;

import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.function.Consumer;

/**
 * A tiny in-process event bus: publishers and subscribers share only event TYPES,
 * never references to each other. Synchronous, not durable (unlike Kafka).
 */
public final class EventBus {
    private record Handler<E>(Class<E> type, Consumer<? super E> action) {
        void deliver(Object event) {
            if (type.isInstance(event)) action.accept(type.cast(event));
        }
    }

    /** AutoCloseable so try-with-resources can unsubscribe for you. */
    public interface Subscription extends AutoCloseable {
        @Override void close();
    }

    private final List<Handler<?>> handlers = new CopyOnWriteArrayList<>();

    public <E> Subscription subscribe(Class<E> type, Consumer<? super E> action) {
        Handler<E> h = new Handler<>(type, action);
        handlers.add(h);
        return () -> handlers.remove(h);
    }

    public void publish(Object event) {
        handlers.forEach(h -> h.deliver(event));
    }
}
```

```java
// File: code/lld/ch11/observer/PricedItem.java
package lld.ch11.observer;

import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;

/** The JDK's recommended replacement for java.util.Observable: java.beans.PropertyChangeSupport. */
public final class PricedItem {
    private final PropertyChangeSupport support = new PropertyChangeSupport(this);
    private final String name;
    private int priceCents;

    public PricedItem(String name, int priceCents) {
        this.name = name;
        this.priceCents = priceCents;
    }

    public void addPropertyChangeListener(PropertyChangeListener l) { support.addPropertyChangeListener(l); }
    public void removePropertyChangeListener(PropertyChangeListener l) { support.removePropertyChangeListener(l); }

    public void setPriceCents(int newPrice) {
        int old = this.priceCents;
        this.priceCents = newPrice;
        support.firePropertyChange("priceCents", old, newPrice);   // no event if old == new
    }

    public String name() { return name; }
}
```

```java
// File: code/lld/ch11/observer/Main.java
package lld.ch11.observer;

import java.math.BigDecimal;

public class Main {
    record OrderPlaced(String orderId, BigDecimal amount) {}
    record OrderCancelled(String orderId) {}

    public static void main(String[] args) {
        System.out.println("=== 1. Build pipeline: subscribe, unsubscribe, failing listener ===");
        BuildPipeline pipeline = new BuildPipeline();
        DashboardUpdater dashboard = new DashboardUpdater();
        pipeline.addListener(new SlackNotifier());
        pipeline.addListener(dashboard);
        pipeline.addListener(new MetricsRecorder());
        Runnable unsubscribeDeploy = pipeline.addListener(new DeployTrigger());

        pipeline.finish(101, "success");        // every listener runs
        unsubscribeDeploy.run();                // e.g. staging has no auto-deploy
        pipeline.finish(102, "success");        // three react
        dashboard.offline = true;
        pipeline.finish(103, "failed");         // dashboard throws, the others still run

        System.out.println("\n=== 2. Push vs pull ===");
        WeatherStation station = new WeatherStation();
        station.subscribePush(r -> System.out.println("  [push display] got everything: " + r));
        station.subscribePull(s -> System.out.println("  [pull display] asked only for temp: " + s.temperature() + " C"));
        station.measure(new WeatherStation.Reading(31.5, 0.62, 1008));

        System.out.println("\n=== 3. Event bus (typed topics, scoped unsubscribe) ===");
        EventBus bus = new EventBus();
        bus.subscribe(OrderPlaced.class, e -> System.out.println("  [email] confirmation for " + e.orderId()));
        bus.subscribe(Object.class, e -> System.out.println("  [audit] " + e));
        EventBus.Subscription refunds = bus.subscribe(OrderCancelled.class,
                e -> System.out.println("  [refunds] start refund for " + e.orderId()));
        try {
            bus.publish(new OrderPlaced("A-1", new BigDecimal("499.00")));
            bus.publish(new OrderCancelled("A-1"));
        } finally {
            refunds.close();                                // refunds handler removed here, even on error
        }
        bus.publish(new OrderCancelled("A-2"));             // only audit sees this one

        System.out.println("\n=== 4. JDK: PropertyChangeSupport (replacement for deprecated Observable) ===");
        PricedItem phone = new PricedItem("phone", 49_900);
        phone.addPropertyChangeListener(evt -> System.out.println(
            "  [price watcher] " + phone.name() + " " + evt.getPropertyName() + ": "
            + evt.getOldValue() + " -> " + evt.getNewValue()));
        phone.setPriceCents(44_900);
        phone.setPriceCents(44_900);    // same value: no event fired
        phone.setPriceCents(39_900);
    }
}
```

Output:

```
=== 1. Build pipeline: subscribe, unsubscribe, failing listener ===
Build 101 finished: success
  [slack] #builds: build 101 success
  [dashboard] status tile -> success
  [metrics] recorded build 101 (total 1)
  [deploy] deploying build 101
Build 102 finished: success
  [slack] #builds: build 102 success
  [dashboard] status tile -> success
  [metrics] recorded build 102 (total 2)
Build 103 finished: failed
  [slack] #builds: build 103 failed
  [pipeline] a listener failed: dashboard is offline
  [metrics] recorded build 103 (total 3)

=== 2. Push vs pull ===
  [push display] got everything: Reading[tempC=31.5, humidity=0.62, pressureHpa=1008.0]
  [pull display] asked only for temp: 31.5 C

=== 3. Event bus (typed topics, scoped unsubscribe) ===
  [email] confirmation for A-1
  [audit] OrderPlaced[orderId=A-1, amount=499.00]
  [audit] OrderCancelled[orderId=A-1]
  [refunds] start refund for A-1
  [audit] OrderCancelled[orderId=A-2]

=== 4. JDK: PropertyChangeSupport (replacement for deprecated Observable) ===
  [price watcher] phone priceCents: 49900 -> 44900
  [price watcher] phone priceCents: 44900 -> 39900
```

Why `java.util.Observer` / `Observable` were deprecated

Since **Java 9**, `java.util.Observer` and `java.util.Observable` are **deprecated** (officially marked "don't use in new code; may be removed one day") but not removed. The official reasons (Javadoc): the event model is **quite limited**, the **order of notifications is unspecified**, and **state changes are not one-for-one with notifications**. Other well-known problems:

- `Observable` is a **class**, not an interface — your subject must *extend* it, using up Java's **single inheritance** (a Java class can extend only one parent class).
- `setChanged()` is **protected**, so you cannot use `Observable` by composition (holding one as a field) without subclassing.
- The `update(Observable, Object)` signature is untyped (`Object arg`), so observers must cast.
- Its thread safety is only partial: its methods are `synchronized` (only one thread at a time may run them, because each call first takes the object's **lock** — a token only one thread can hold), but `notifyObservers` calls observers outside the lock on a snapshot, so (as a comment in the **JDK** — Java Development Kit: the compiler, runtime and standard library — source admits) a newly added observer may miss a notification and a just-removed one may still get it.

What to use instead (the Javadoc names the first, third and fourth): `java.beans.PropertyChangeSupport`/`PropertyChangeListener` for bean properties; your own typed listener interface (as in `BuildPipeline`); `java.util.concurrent` data structures (queues) for reliable, ordered hand-off between threads; the `java.util.concurrent.Flow` API (Java 9) for **reactive streams** (asynchronous streams of data items flowing from a producer to consumers) with **back-pressure** (the consumer tells the producer to slow down when it cannot keep up).

Trade-offs

Pros	Cons
The source no longer depends on its reactions (a build pipeline no longer imports Slack).	Harder to trace: the call site gives no hint of which code will execute.
Reactions are added/removed at run time; environments differ without flags.	**Memory leaks** (memory that is never freed) if an observer is never removed (the subject holds them forever).
Each reaction is testable alone.	Stack traces start inside a loop over a list; debugging is harder.
A new reaction = one class + one subscribe call (Open/Closed: open to extension, closed to modification).	**No ordering or delivery guarantee**; a crash loses the reactions not yet called.
	With synchronous observers, the subject takes as long as all its listeners combined.

Observer vs Mediator, Chain of Responsibility, callbacks

Looks like	The actual difference
Mediator	Observer carries only "this changed"; each observer decides for itself, one way, and no order between reactions can be expressed. A mediator receives the change **and decides what happens next**, calling specific components back (two-way), and explicit ordering is its reason to exist.
Publish-subscribe with a broker (a server that stores and forwards messages)	Same idea, but the broker contributes persistence, redelivery, ordering guarantees and a network hop — none of which in-process observers offer.
Chain of Responsibility	Observer informs **all** listeners, and no listener can prevent the rest from running. A chain ends as soon as one handler accepts the request.
A callback	One callback is one observer. It becomes the Observer pattern when the number of listeners varies and they subscribe and unsubscribe at run time.

Observer outside Java. With **first-class functions** (functions that can be stored in variables and passed around like any other value) a listener is just a **lambda** (a short anonymous function, written `x -> ...` in Java) — keep the handle, or you cannot unsubscribe. JavaScript has `EventTarget` / `EventEmitter`, and `AbortController` can unsubscribe by signal. Python often uses a plain list of callables, and `weakref.WeakSet` avoids the leak. C# has `event`, `IObservable<T>`, and `IDisposable` subscriptions. In every language, **weak references** (references that do not keep their target alive, so the garbage collector may still free it) fix the leak, at the cost of a listener disappearing when nothing else holds it.

When not to use it

- **One listener, forever.** Call it directly.
- **You need ordering or guaranteed delivery.** Use a queue or message broker (Kafka, RabbitMQ, SQS — popular broker products): same idea plus persistence, retries and a network — and very different failure modes.
- **Reactions are slow.** Either notify **asynchronously** (hand the work to another thread and return at once — e.g. via an *executor*, a pool of worker threads, or a queue) or don't use in-process observers.
- **The whole flow must be auditable/traceable.** Explicit calls are kinder to readers.

Common mistakes (the four classic ways observers go wrong, plus one)

1. **No unsubscribe → memory leak.** The subject keeps a **strong reference** (an ordinary reference, which keeps the object alive in memory) to the listener and everything it points to. In long-running apps this is a top cause of slow memory growth. Every `add` needs an owner for the matching `remove`. (Weak references are one fix, at the cost of listeners vanishing when nothing else holds them.)
2. **A throwing listener kills the loop.** Without a per-listener try/catch, listener 3 fails and 4 and 5 never run.
3. **Changing the list during notification** → `ConcurrentModificationException` (the error Java throws when a collection is changed while it is being looped over) (or silently skipped listeners in some languages). Iterate over a copy, or use `CopyOnWriteArrayList`.

4. **Listeners calling back into the subject** → cycles or endless recursion. Finish updating state before notifying, and never let a notification synchronously cause another one.
5. **Notifying before the state is updated**, so pull-style observers read stale data.
6. **Relying on notification order.** Listeners are called in the order they happened to subscribe, which is a side effect of start-up code rather than a decision anyone made. If "update the dashboard, then decide whether to deploy" matters, that is coordination, and it belongs in a Mediator.

Real-world examples

- **Every UI event handler**: Swing `ActionListener`, Android `OnClickListener`, DOM `addEventListener`.
- **JDK**: `PropertyChangeListener`, `java.util.concurrent.Flow` (`Publisher`/`Subscriber`), servlet listeners such as `HttpSessionListener`.
- **Spring** (the most widely used Java application framework): `ApplicationEventPublisher` + `@EventListener` (in-process events, optionally `@Async` or `@TransactionalEventListener`).
- **Guava** `EventBus`, **RxJava / Project Reactor / RxJS**, Kotlin `Flow`, and signals in front-end frameworks — observers with composable operators (map, filter, debounce — wait until events pause, then pass on only the latest) layered on top.
- **Model–view separation**: a view registers with its model, which then updates freely with no idea that any screen is attached (see MVC in §11.7.5).
- **Distributed publish–subscribe**: Kafka topics, Google Pub/Sub, AWS SNS. When an order is placed on an e-commerce site, the order service publishes one event and email, inventory, analytics and recommendation services each consume it independently — Observer at system scale, with durability added.
- **Database triggers and change data capture** (CDC: streaming every row change out of a database's internal log; Debezium is a popular tool) — observers at the storage layer.

In the interview

> **30-second answer:** "Observer swaps a handful of hard-coded calls for a list of objects that share one listener interface. The subject just says "this happened" and every observer picks its own response, so each reaction can be plugged in, unplugged and unit-tested on its own, and the subject no longer depends on any of them. In production I iterate over a copy of the listener list, catch exceptions per listener, and make sure every subscribe has a matching unsubscribe to avoid leaks. When I need durability, retries or ordering I use a message broker instead."

Follow-ups:

- *Push or pull?* Push sends the data (simple, but the interface grows with the data); pull sends a reference and observers ask for what they need (flexible, but couples observers to the subject's getters).
- *Why was `java.util.Observable` deprecated?* Limited event model, unspecified notification order, notifications not one-for-one with changes; plus it's a class (uses up inheritance) with protected `setChanged`. Use `PropertyChangeSupport`, custom listeners, or `Flow`.
- *Observer vs Pub/Sub?* In Observer the subject holds its observers directly. In publish–subscribe a **broker/event bus** sits in between, so publishers and subscribers don't know each other at all; brokers add durability, retries and ordering.
- *Observer vs Mediator?* With Observer, the news is broadcast and every listener makes its own call; with Mediator, one central object makes the decision and tells particular components what to do. When the sequence of reactions matters, pick a mediator.
- *How do you make it thread-safe?* `CopyOnWriteArrayList` for listeners (reads far outnumber writes), no locks held while calling listeners, and dispatch to an executor if listeners are slow.
- *How do you avoid leaks?* Return a subscription handle, tie it to a lifecycle (try-with-resources, `onDestroy`, `@PreDestroy`), or use weak references.

Self-check

1. **Q:** What does the subject know about its observers? **A:** Only that they implement the observer interface.
2. **Q:** Why iterate over a copy of the listener list? **A:** So a listener that (un)subscribes during notification can't break the loop.
3. **Q:** What is the classic Observer memory leak? **A:** Listeners that are never removed stay strongly referenced by the subject.
4. **Q:** What should replace `java.util.Observable`? **A:** `PropertyChangeSupport`, a custom typed listener interface, or the `Flow` API.

5. **Q:** When is a message broker better than in-process observers? **A:** When you need durability, retries, ordering, or slow/remote consumers.

TL;DR

One subject, a list of listeners, one-way notification. **Iterate over a copy, catch per listener, always unsubscribe.** Push vs pull is a trade-off; `Observable` is deprecated; use a broker when you need guarantees.

11.2 State

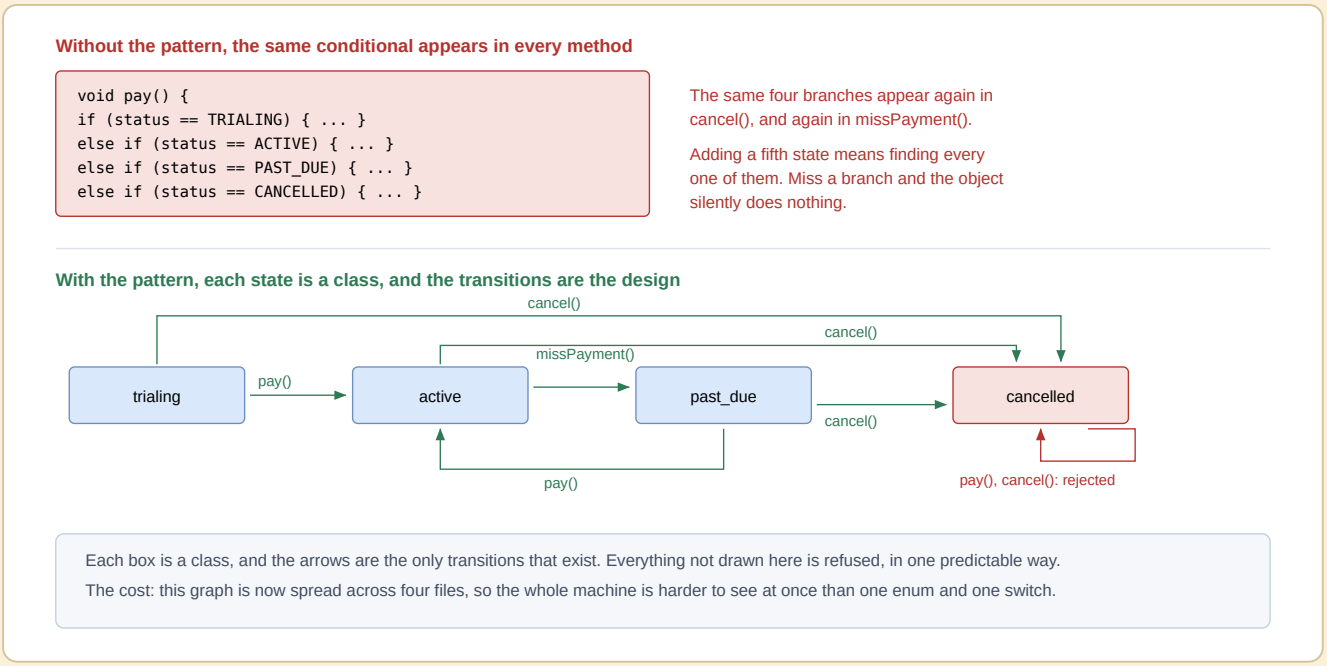


Figure — Top: the same four-branch status check repeated in every method. Bottom: each state is a class and the transitions form a graph (course diagram)

What it is

State (GoF intent, paraphrased): when an object's internal condition changes, **its behaviour changes with it**, so from the outside it looks as if the object had switched to a different class. Rather than every method testing a status field, the object hands each action to a **state object**, and replaces that object whenever the condition changes.

GoF participants (and who plays them in our code):

Participant	Role	In `lld.ch11.state`
Context	The object clients use; holds the current state object and forwards requests to it	`Subscription`, `VendingMachine`
State	Interface declaring one method per request the context forwards	`SubscriptionState`, `VendingState`
ConcreteState	One class per state, implementing the behaviour for that state	`Trialing`, `Active`, `PastDue`, `Cancelled`; `Idle`, `HasMoney`, `SoldOut`

GoF leaves open *who* performs the transitions: the context (with a fixed table) or the states themselves. Letting the states choose their successor, as here, is more flexible but makes the states know about each other.

Everyday analogy: a phone's ringer switch. Pressing "call received" does something different in *ring*, *vibrate* and *silent* mode — the phone is the same, its mode decides the behaviour.

The problem it solves

Take a subscription whose life runs **trialing → active → past due → cancelled**. Written naively, each public method opens with an identical four-way branch:

```
public void pay() {
    if (status == TRIALING)       { /* charge the card, switch to ACTIVE */ }
    else if (status == ACTIVE)    { /* renew */ }
    else if (status == PAST_DUE)  { /* settle arrears, back to ACTIVE */ }
    else if (status == CANCELLED) { /* reject */ }
}
```

`cancel()` and `missPayment()` repeat those four branches. A fifth state forces you to hunt down every copy. Everything that defines one state is spread over all the methods. Worst of all, a **forgotten branch** quietly does nothing, and the bug surfaces weeks later as a vague "subscriptions randomly stop working".

With the State pattern, **each state is a class**. The conditional vanishes (each context method shrinks to a single line), and everything about one state is gathered in one file — exactly what a reviewer or auditor wants to read.

How it works

1. Define a **State interface** with one method per action (`pay`, `missPayment`, `cancel`). Each state has to respond to each action, even when the response is "not allowed".
2. Write one **concrete state** class per state.
3. The **context** (`Subscription`) holds the current state and delegates each action: `state.pay(this)`.
4. A state rules on whether the action is **legal** and which state comes **after** it, and advances the machine by calling `context.setState(new Active())`.
5. Terminal states refuse everything, loudly, in one place.

The objects choose their own successors — the machine advances itself. That is the key difference from Strategy.

Structure (UML)

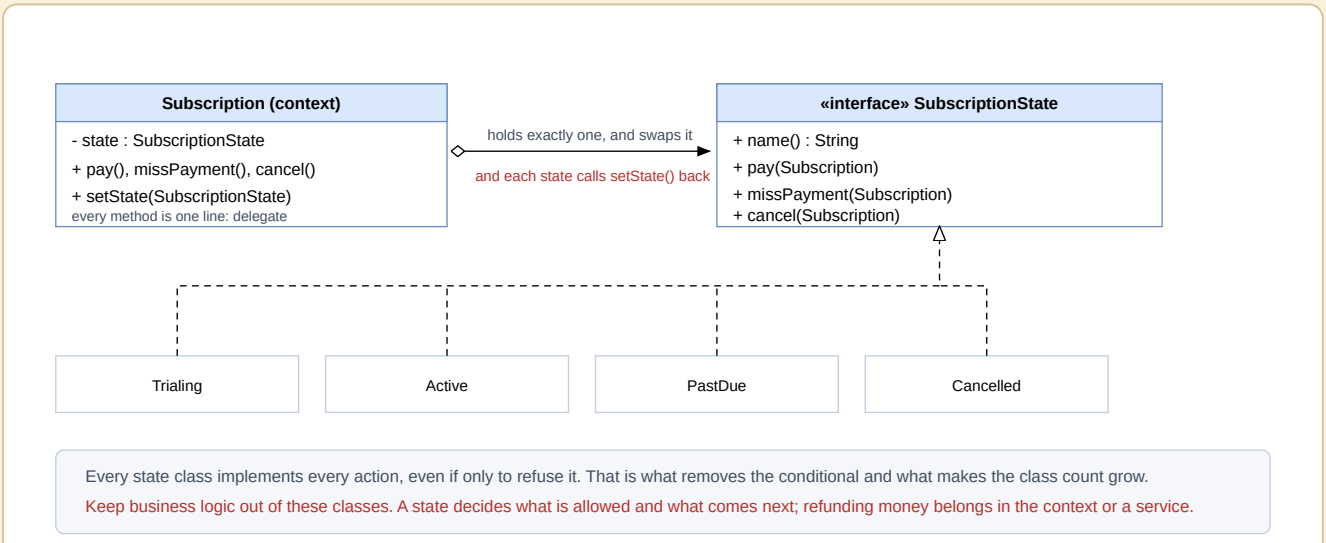
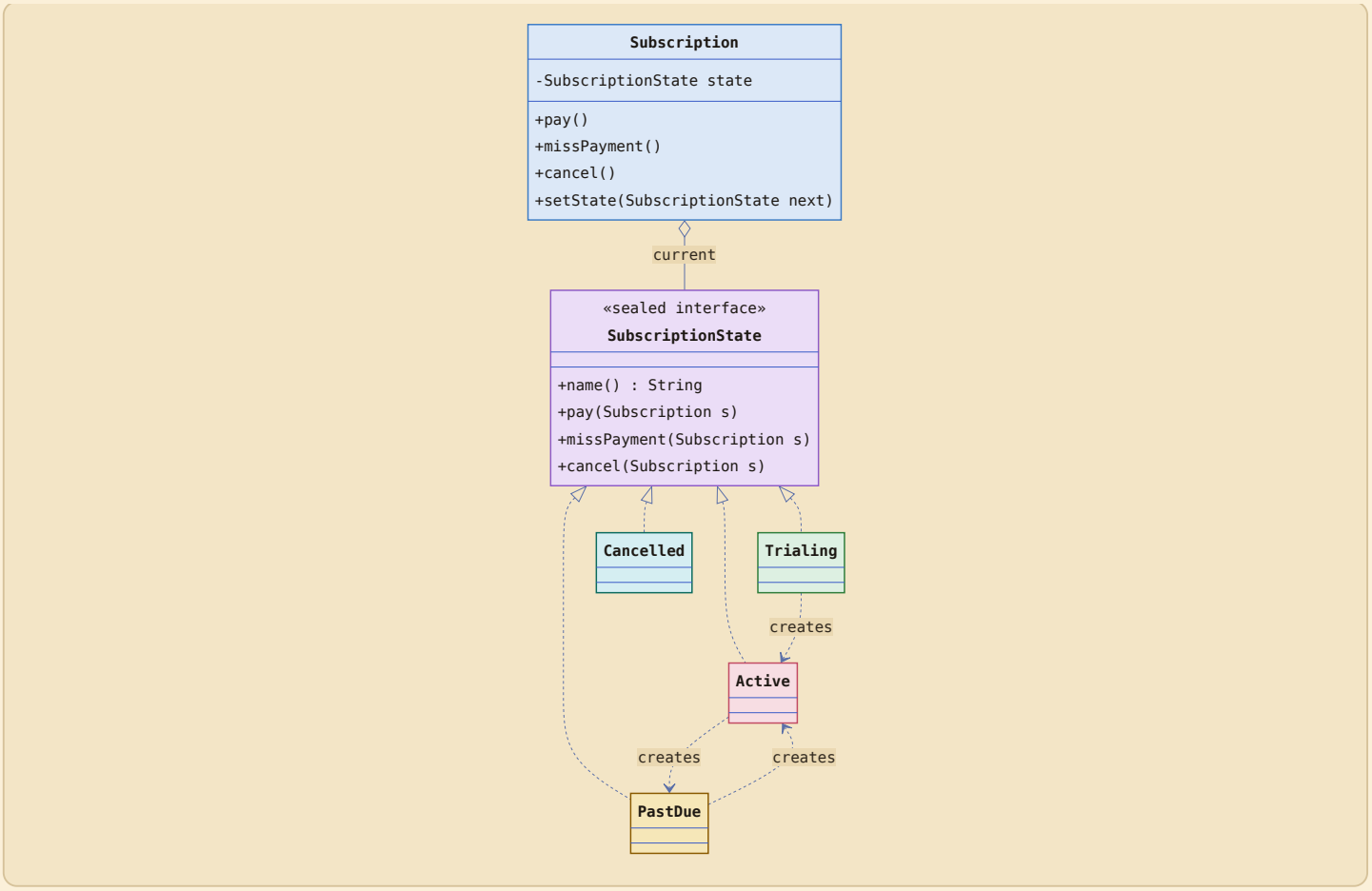


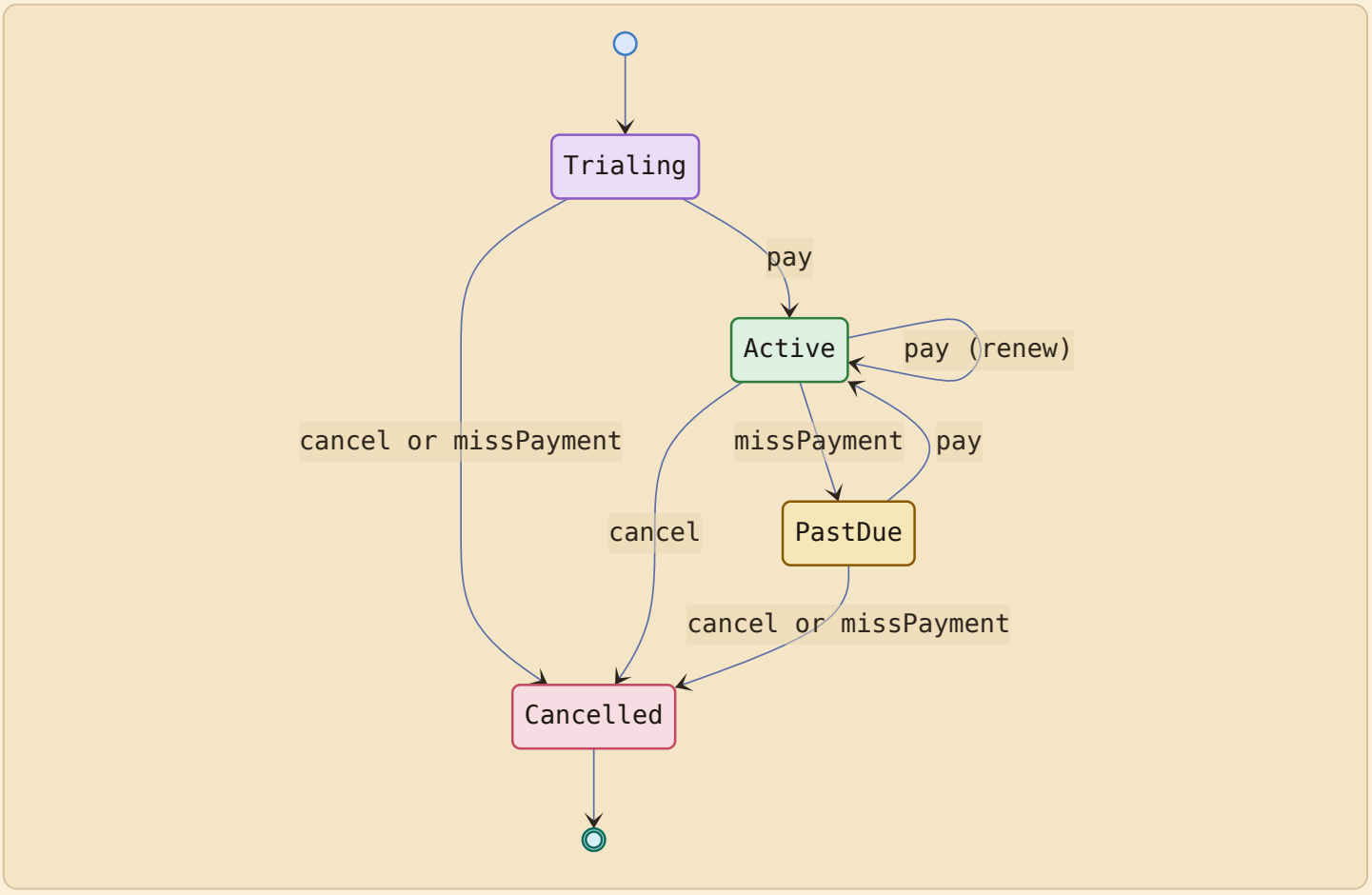
Figure — Class diagram: Subscription (context) holds a SubscriptionState; Trialing, Active, PastDue and Cancelled implement it and call setState on the context (course diagram)

In the diagram above the states reach **back** into the context to replace themselves, and the state classes know each other (Trialing creates an Active).

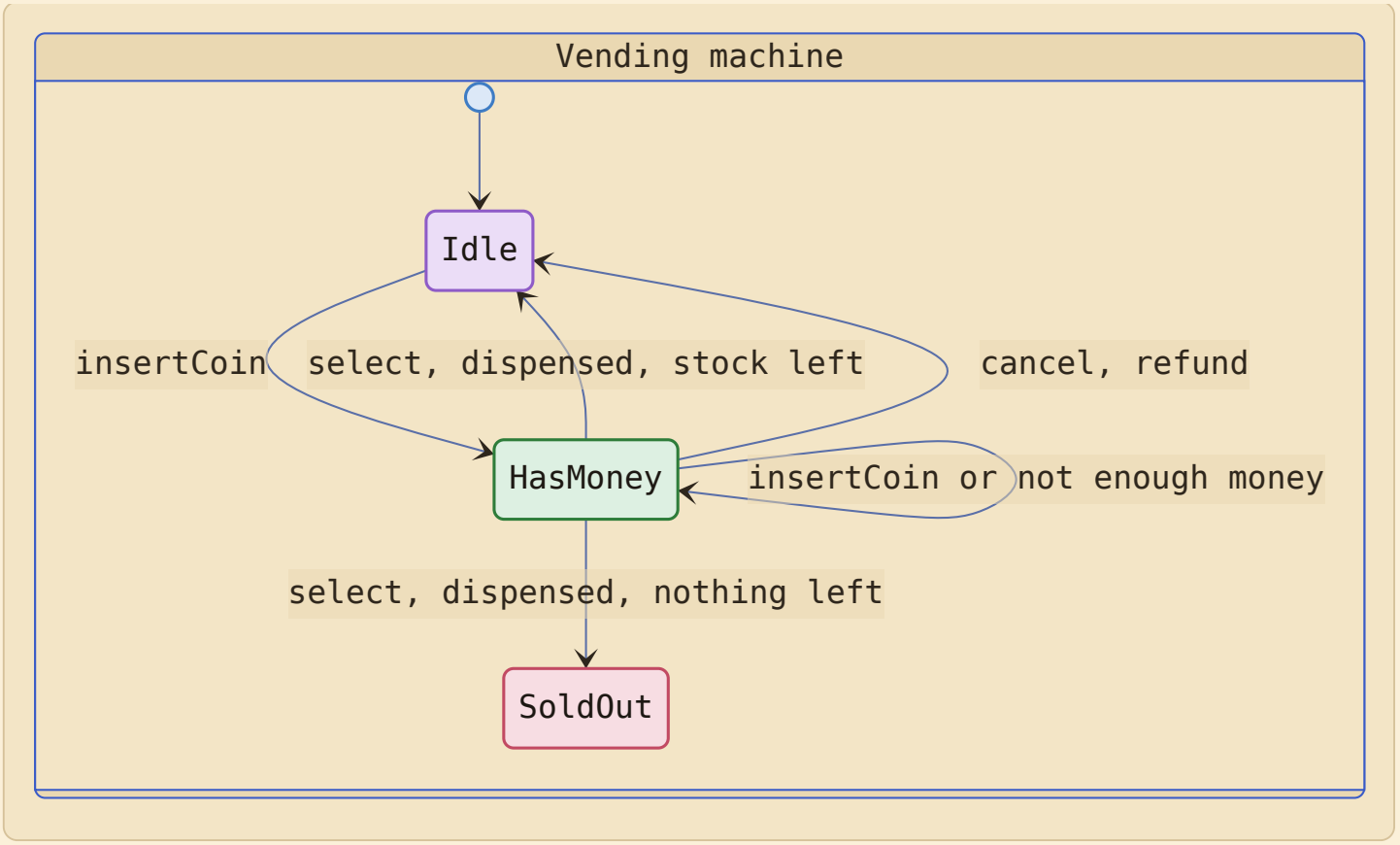
Our Java design (class-per-state version):

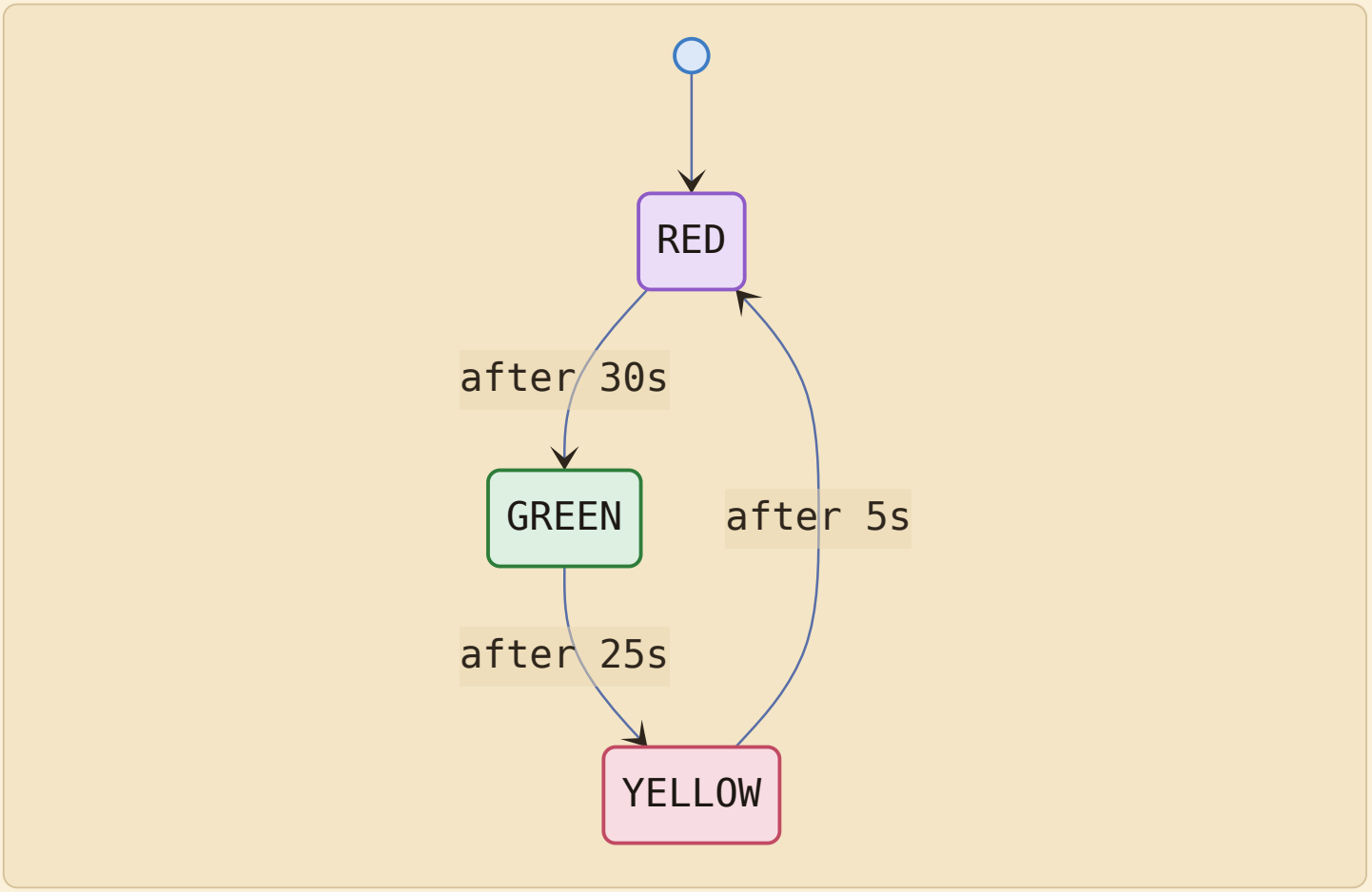


The state machine itself:



Vending machine and traffic light:





Java example

`lld.ch11.state` shows three styles:

1. **Class per state** (`Subscription` + four `record` states). `SubscriptionState` is a **sealed** interface (one that lists exactly which types may implement it): the compiler knows the complete set of states. The states have

no fields, so **records** (Java's compact data classes whose fields are fixed at creation) with no components are a neat, **immutable** (cannot change after creation) choice.

2. **Enum-based state machine** (`TrafficLight`). Each enum constant overrides `next()`; shared behaviour is a **default method** (an interface method with a body, inherited by implementers unless they override it). The **whole machine is readable in one file** — the idiomatic Java answer for simple machines.
3. **Vending machine** (`VendingMachine` + `Idle`, `HasMoney`, `SoldOut`). The state interface uses **default methods that refuse**, so each state overrides only the actions it allows (`SoldOut` overrides nothing). Business actions (dispensing, returning change) live in the **context**; states only rule on whether an action is legal and which state follows. Money is kept as whole **cents in an `int`**, which is exact (unlike `double`); a real machine would use a `Money`/`BigDecimal` type (`BigDecimal` is Java's exact decimal-number class), as the Strategy and Visitor examples do.

```java
// File: code/lld/ch11/state/SubscriptionState.java
package lld.ch11.state;

/**
 * STATE interface. Each state implements all actions, including ones it only turns down.
 * Sealed: the compiler knows the full set of states.
 */
public sealed interface SubscriptionState permits Trialing, Active, PastDue, Cancelled {
    String name();
    void pay(Subscription s);
    void missPayment(Subscription s);
    void cancel(Subscription s);
}
```

```java
// File: code/lld/ch11/state/Trialing.java
package lld.ch11.state;

public record Trialing() implements SubscriptionState {
    public String name() { return "trialing"; }
    public void pay(Subscription s) {
        System.out.println("  [trialing] first payment taken");
        s.setState(new Active());                    // the state chooses its successor
    }
    public void missPayment(Subscription s) {
        System.out.println("  [trialing] trial ended unpaid");
        s.setState(new Cancelled());
    }
    public void cancel(Subscription s) { s.setState(new Cancelled()); }
}
```

```java
// File: code/lld/ch11/state/Active.java
package lld.ch11.state;

public record Active() implements SubscriptionState {
    public String name() { return "active"; }
    public void pay(Subscription s) { System.out.println("  [active] renewed for another period"); }
    public void missPayment(Subscription s) {
        System.out.println("  [active] payment failed, grace period started");
        s.setState(new PastDue());
    }
    public void cancel(Subscription s) { s.setState(new Cancelled()); }
}
```

```java
// File: code/lld/ch11/state/PastDue.java
package lld.ch11.state;

public record PastDue() implements SubscriptionState {
    public String name() { return "past_due"; }
    public void pay(Subscription s) {
        System.out.println("  [past_due] arrears paid");
        s.setState(new Active());
    }
    public void missPayment(Subscription s) {
        System.out.println("  [past_due] grace period over");
        s.setState(new Cancelled());
    }
    public void cancel(Subscription s) { s.setState(new Cancelled()); }
}
```

```java
// File: code/lld/ch11/state/Cancelled.java
package lld.ch11.state;

/** Terminal state: refuses everything, loudly, in one place. */
public record Cancelled() implements SubscriptionState {
    public String name() { return "cancelled"; }
    public void pay(Subscription s) { System.out.println("  [cancelled] refused: no payments after cancellation"); }
    public void missPayment(Subscription s) { System.out.println("  [cancelled] ignored: nothing is billed"); }
    public void cancel(Subscription s) { System.out.println("  [cancelled] rejected: already cancelled"); }
}
```

```java
// File: code/lld/ch11/state/Subscription.java
package lld.ch11.state;

/** The CONTEXT: every public action is one line of delegation. No status switch anywhere. */
public final class Subscription {
    private SubscriptionState state = new Trialing();

    void setState(SubscriptionState next) {
        System.out.println("    " + state.name() + " -> " + next.name());
        state = next;
    }

    public String status() { return state.name(); }

    public void pay()         { state.pay(this); }
    public void missPayment() { state.missPayment(this); }
    public void cancel()      { state.cancel(this); }
}
```

```java
// File: code/lld/ch11/state/TrafficLight.java
package lld.ch11.state;

/**
 * State pattern the idiomatic-Java way: an enum whose constants override behaviour.
 * The whole machine is readable in one file.
 */
public enum TrafficLight {
    RED(30) {
        TrafficLight next() { return GREEN; }
        boolean pedestriansMayCross() { return true; }
    },
    GREEN(25) {
        TrafficLight next() { return YELLOW; }
    },
    YELLOW(5) {
        TrafficLight next() { return RED; }
    };

    private final int seconds;

    TrafficLight(int seconds) { this.seconds = seconds; }

    int seconds() { return seconds; }
    abstract TrafficLight next();
    boolean pedestriansMayCross() { return false; }    // default for most states
}
```

```java
// File: code/lld/ch11/state/VendingState.java
package lld.ch11.state;

/** Vending machine states. Default methods = "refuse", so each state overrides only what it allows. */
interface VendingState {
    String name();

    default void insertCoin(VendingMachine m, int cents) {
        System.out.println("  [" + name() + "] coin rejected, returned " + cents);
    }

    default void select(VendingMachine m, String code) {
        System.out.println("  [" + name() + "] selection ignored");
    }

    default void cancel(VendingMachine m) {
        System.out.println("  [" + name() + "] nothing to cancel");
    }
}
```

```java
// File: code/lld/ch11/state/Idle.java
package lld.ch11.state;

final class Idle implements VendingState {
    public String name() { return "idle"; }

    @Override
    public void insertCoin(VendingMachine m, int cents) {
        m.addBalance(cents);
        m.setState(new HasMoney());
    }
}
```

```java
// File: code/lld/ch11/state/HasMoney.java
package lld.ch11.state;

final class HasMoney implements VendingState {
    public String name() { return "has_money"; }

    @Override
    public void insertCoin(VendingMachine m, int cents) { m.addBalance(cents); }

    @Override
    public void select(VendingMachine m, String code) {
        if (!m.inStock(code)) { System.out.println("  [has_money] " + code + " is empty, choose another"); return; }
        int price = m.priceOf(code);
        if (m.balance() < price) {
            System.out.println("  [has_money] " + code + " costs " + price + ", balance " + m.balance());
            return;
        }
        m.dispense(code);                                  // business action lives in the context
        m.returnChange();
        m.setState(m.anythingInStock() ? new Idle() : new SoldOut());
    }

    @Override
    public void cancel(VendingMachine m) {
        m.returnChange();
        m.setState(new Idle());
    }
}
```

```java
// File: code/lld/ch11/state/SoldOut.java
package lld.ch11.state;

final class SoldOut implements VendingState {
    public String name() { return "sold_out"; }        // every action uses the refusing defaults
}
```

```java
// File: code/lld/ch11/state/VendingMachine.java
package lld.ch11.state;

import java.util.LinkedHashMap;
import java.util.Map;

/** CONTEXT: owns inventory and money; states rule on legality and pick the next state. */
final class VendingMachine {
    private record Slot(String product, int priceCents, int count) {}

    private final Map<String, Slot> slots = new LinkedHashMap<>();
    private VendingState state = new Idle();
    private int balance = 0;

    void load(String code, String product, int price, int count) { slots.put(code, new Slot(product, price, count)); }

    // actions: delegate to the current state
    void insertCoin(int cents) { System.out.println("insert " + cents); state.insertCoin(this, cents); }
    void select(String code)   { System.out.println("select " + code); state.select(this, code); }
    void cancel()              { System.out.println("cancel"); state.cancel(this); }

    // helpers the states call
    void setState(VendingState next) {
        System.out.println("    " + state.name() + " -> " + next.name());
        state = next;
    }
    void addBalance(int cents) { balance += cents; System.out.println("  balance " + balance); }
    int balance() { return balance; }
    boolean inStock(String code) { return slots.containsKey(code) && slots.get(code).count() > 0; }
    boolean anythingInStock() { return slots.values().stream().anyMatch(s -> s.count() > 0); }
    int priceOf(String code) { return slots.get(code).priceCents(); }

    void dispense(String code) {
        Slot s = slots.get(code);
        slots.put(code, new Slot(s.product(), s.priceCents(), s.count() - 1));
        balance -= s.priceCents();
        System.out.println("  >> dispensed " + s.product());
    }

    void returnChange() {
        if (balance > 0) System.out.println("  >> change returned " + balance);
        balance = 0;
    }
}
```

```java
// File: code/lld/ch11/state/Main.java
package lld.ch11.state;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== 1. Subscription lifecycle (class per state) ===");
        Subscription sub = new Subscription();
        sub.pay();           // trialing -> active
        sub.missPayment();   // active -> past_due
        sub.pay();           // past_due -> active
        sub.cancel();        // active -> cancelled
        sub.pay();           // rejected, stays cancelled
        System.out.println("  final status: " + sub.status());

        System.out.println("\n=== 2. Traffic light (enum-based state machine) ===");
        TrafficLight light = TrafficLight.RED;
        for (int i = 0; i < 4; i++) {
            System.out.printf("  %-6s for %2ds, pedestrians may cross: %s%n",
                light, light.seconds(), light.pedestriansMayCross());
            light = light.next();
        }

        System.out.println("\n=== 3. Vending machine ===");
        VendingMachine vm = new VendingMachine();
        vm.load("A1", "cola", 150, 1);
        vm.load("B2", "chips", 100, 0);
        vm.select("A1");        // idle: ignored
        vm.insertCoin(100);     // idle -> has_money
        vm.select("A1");        // not enough
        vm.select("B2");        // empty slot
        vm.insertCoin(100);
        vm.select("A1");        // dispense, change 50, -> sold_out (nothing left)
        vm.insertCoin(25);      // sold_out: rejected
    }
}
```

Output:

```
=== 1. Subscription lifecycle (class per state) ===
  [trialing] first payment taken
    trialing -> active
  [active] payment failed, grace period started
    active -> past_due
  [past_due] arrears paid
    past_due -> active
    active -> cancelled
  [cancelled] refused: no payments after cancellation
  final status: cancelled

=== 2. Traffic light (enum-based state machine) ===
  RED    for 30s, pedestrians may cross: true
  GREEN  for 25s, pedestrians may cross: false
  YELLOW for  5s, pedestrians may cross: false
  RED    for 30s, pedestrians may cross: true

=== 3. Vending machine ===
select A1
  [idle] selection ignored
insert 100
  balance 100
    idle -> has_money
select A1
  [has_money] A1 costs 150, balance 100
select B2
  [has_money] B2 is empty, choose another
insert 100
  balance 200
select A1
  >> dispensed cola
  >> change returned 50
    has_money -> sold_out
insert 25
  [sold_out] coin rejected, returned 25
```

The final subscription line shows the payoff: an **illegal action** is refused the same way every time, by code in a single file.

State vs Strategy (and Bridge) — same diagram, different reasons

These patterns draw **the same class diagram** (an object keeps a reference to an interface and forwards work to it). It is one of the most common interview questions, and the diagram cannot answer it:

	State	Strategy	Bridge
Who chooses the object	The **states**, which replace themselves via the context	The **caller**, from outside	Chosen once at construction, usually by configuration
How often it changes	Repeatedly — that is the point	Set once, sometimes per call	Effectively never after wiring
Do the alternatives reference one another?	**Yes** (`Trialing` builds an `Active`)	**No** — never	No
What varies	The object's current *condition*, and therefore its allowed actions	*How* one job gets done	A whole dimension of the design
Why it exists	To model a lifecycle	To make one algorithm replaceable	To stop two class hierarchies multiplying

The reliable test: do the alternatives name each other? If yes, it is a State machine. If no, it is Strategy (or Bridge — then ask how often the choice changes).

Two more things State gets confused with:

Looks like	The actual difference
An enum plus one `switch`	Frequently the better answer, and easy to defend in an interview. Class-per-state starts to win only once the same status switch is repeated in three or more methods.
A state-machine library (Spring Statemachine, XState)	The same idea expressed as data (states, events, guards, actions) rather than classes. Once a machine grows past roughly a dozen states, the declarative form is easier to read and to draw.

Trade-offs

Pros	Cons
Everything about one state sits in a single file.	One class per state; small classes multiply.
The repeated conditional disappears; no branch can be missed.	The transitions live in several files, so no single place shows the whole machine.
Illegal actions are rejected consistently.	Each state class must implement all actions, even if only to say no.
A new state = a new class; the compiler lists what it must implement.	Persisting needs a name↔class mapping — keep it in exactly one place.

When not to use it

- **Only two states and a handful of methods** — a boolean flag with an `if` reads better.
- **Behaviour doesn't change, only data does** — store the differing value in a field; a class per state adds nothing.
- **A big, formal machine** (more than about a dozen states with guards) — use a **transition table** (`Map<(state, event), handler>`) or a state-machine library (e.g. **Spring Statemachine**), where the diagram can be the source of truth.
- **Rule of thumb from the course:** the pattern pays off once **three or more methods repeat the same status switch**. Below that, an enum with a switch is a perfectly defensible answer.
- **A signal you have outgrown it:** you keep sketching the machine on paper just to follow your own code. At that point make the sketch (a transition table or state chart) the real source of truth, instead of classes that only approximate it.

Common mistakes

1. **Successors hard-coded everywhere.** Fine at 4 states, painful at 12. Past that, let the context or a transition table own the transitions.
2. **Business logic inside state classes.** A state's job is to rule on *legality* and pick the *successor*; charging cards and sending emails belong in the context or a service it calls. Otherwise the state classes become a dumping ground.
3. **Forgetting to make illegal actions explicit** (silently doing nothing).
4. **Storing state objects in the DB** — store a name/enum and map it back to a state in exactly one place.
5. **Sharing stateful state objects** between contexts. Stateless states (like ours) can be shared safely; stateful ones cannot.

State in other languages

- **Java:** an `enum` whose constants override behaviour (like `TrafficLight`) keeps the whole graph in one file. A `sealed` interface plus exhaustive pattern-matching `switch` gives compiler-checked coverage of every state.
- **Kotlin / Scala:** `sealed` hierarchies with exhaustive `when` / `match`.
- **Rust:** enums with data, matched exhaustively. With the **typestate** pattern each state is a separate *type*, so calling an action that is illegal in that state is a compile error.
- **Python:** a dict mapping `(state, action)` to a handler is a compact transition table that shows every transition in one place.
- **TypeScript:** a **discriminated union** (a set of object types that each carry a fixed tag field saying which kind they are) plus a `switch` over its tag field, which the compiler can check for missing cases.

Real-world examples

- **Order lifecycles** in food-delivery and e-commerce apps: placed → accepted → preparing → out for delivery → delivered (or cancelled); which actions are legal depends on the status (you can cancel while "placed" but usually not once "out for delivery").
- **Payment and subscription lifecycles** (Stripe subscription statuses such as `trialing`, `active`, `past_due`, `canceled`).
- **TCP connections** (TCP is the protocol under most internet traffic, giving ordered, reliable delivery; see System Design Fundamentals) — GoF's own example: `LISTEN`, `ESTABLISHED`, `CLOSED`, each responding differently to the same call.
- **Workflow/approval engines** (draft → in review → approved/rejected), **document editors' modes**, **game characters** (idle, running, jumping).
- **Lexers/parsers** (programs that split text into tokens, then build a structure from them), **vending machines, ATMs, elevators, traffic lights** — favourite **machine-coding** questions (an interview round where you write working code for a small system in an hour or two) (see the case-study chapters).
- **Libraries**: Spring Statemachine; in the JDK, `Thread.State` is an *enum of states* (the data), not the pattern itself.

In the interview

> **30-second answer:** "State swaps a status field, and the switch statements repeated around it, for one class per state. The context forwards each action to whichever state object it currently holds, and that object decides whether the action is legal and which state follows. Each state's rules sit together in one file, and an illegal action is refused the same way every time. It draws exactly like Strategy, but states name one another and move the machine forward on their own, while strategies are picked from outside and never reference each other. In Java, for small machines, I'd use an enum with per-constant behaviour or a sealed interface."

Follow-ups:

- *State vs Strategy?* Use the naming test and the "who chooses" test (table above).
- *Where should transitions live?* In the states for small machines; in the context or a transition table when the machine grows, so the whole graph is visible in one place.
- *How do you persist it?* Store the state's name (enum) in the DB (database); rebuild the state object through one factory/mapping.
- *How do you handle concurrency?* Two requests (e.g. "cancel" and "deliver") can **race** (run at the same moment and interfere, so the result depends on timing): guard transitions with a lock or, in a DB, an **optimistic**

(no lock held; the write simply fails if someone changed the row first) `UPDATE ... WHERE status = 'PLACED'` and check the affected row count.

- *Design a vending machine.* States Idle, HasMoney, SoldOut (maybe Dispensing, Maintenance); context owns inventory and balance; states allow or refuse actions (as in our code).

Self-check

1. **Q:** Who changes the current state in the State pattern? **A:** The state objects (via the context's `setState`).
2. **Q:** What is the "naming test"? **A:** If the alternatives reference each other, it's State; if never, it's Strategy/Bridge.
3. **Q:** When is an `enum` better than class-per-state? **A:** When the machine is small; the whole machine is visible in one file.
4. **Q:** What belongs in the context, not the state? **A:** Business actions and data (dispensing, charging, inventory).
5. **Q:** What does a sealed state interface give you? **A:** The compiler knows every state, so `switch`es over states can be checked for exhaustiveness.

TL;DR

A class for each state; the context forwards every action; each state picks what follows it. Same diagram as Strategy, but states **know each other**. Use enums for small machines, tables/libraries for big ones.

11.3 Strategy

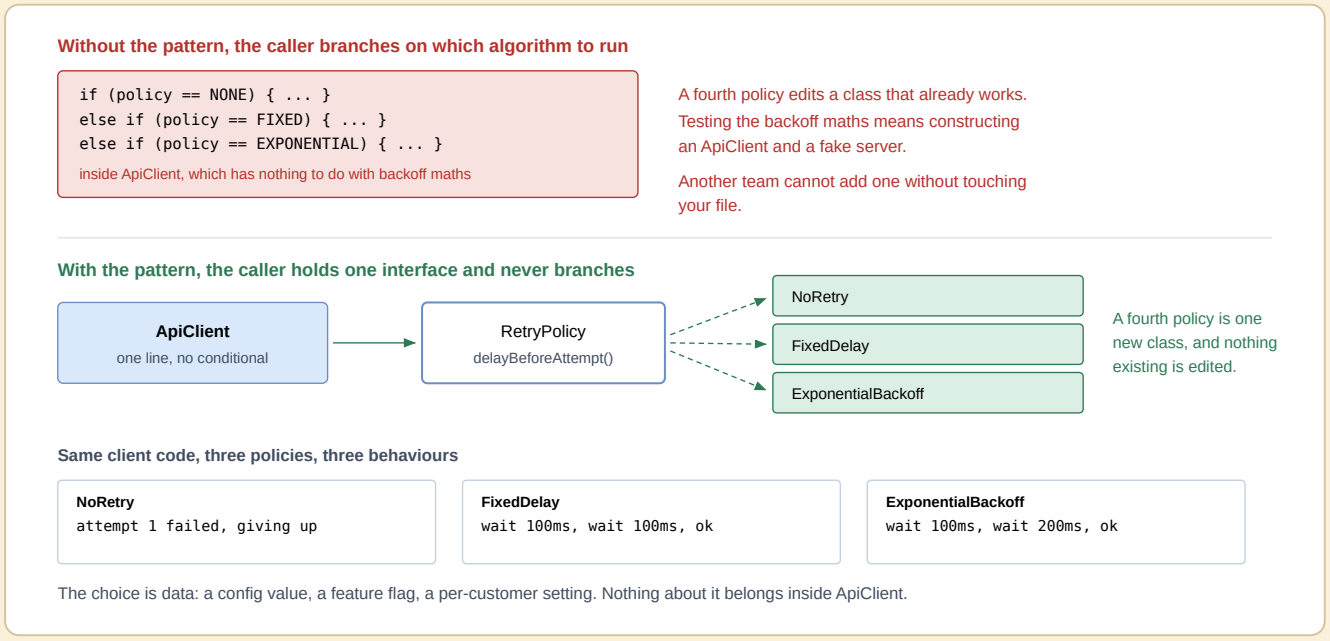


Figure — Top: ApiClient branches on which retry policy to use. Bottom: ApiClient holds one RetryPolicy interface and never branches (course diagram)

What it is

Strategy (GoF intent, paraphrased): group related algorithms into a **family**, wrap each one in its own class behind a **common interface**, and make them interchangeable, so the algorithm can change without touching the code that calls it; the choice can be made at run time. The object using it (the **context**) does not branch on which algorithm it has.

GoF participants (and who plays them in our code):

Participant	Role	In `lld.ch11.strategy`
Strategy	Interface shared by all the algorithms; the context calls only this	`RetryPolicy` (also `Comparator` in part 2)
ConcreteStrategy	One implementation per algorithm	`FixedDelay`, `ExponentialBackoff`, the `RetryPolicy.none()` lambda (the course diagram names it `NoRetry`)
Context	Is configured with a strategy object and delegates to it; may pass it the data it needs	`ApiClient`

Everyday analogy: getting to the airport. Car, train or taxi — same goal, different algorithm. You pick based on time, money and weather; the trip itself ("get to the airport") doesn't change.

The problem it solves

Retrying a failed request can mean: no retry, a fixed delay, or **exponential backoff** (doubling the wait after each failure — e.g. 100 ms, 200 ms, 400 ms). Written directly, the branch sits inside `ApiClient`:

```
if (policy == NONE)             { /* give up */ }
else if (policy == FIXED)       { /* wait 100ms, try again */ }
else if (policy == EXPONENTIAL) { /* double the wait after each failure */ }
```

`ApiClient` now contains backoff arithmetic that has nothing to do with calling an API. A fourth policy means editing a working class; you cannot test the backoff maths without building a client and faking a server; and a different team has to edit your class just to contribute a policy.

Strategy gives every algorithm a class of its own. `ApiClient` asks its `RetryPolicy` "how long before the next attempt?" and **never checks what kind of policy it holds**. The choice is made **outside** the client — from configuration, a **feature toggle** (a setting that switches behaviour on or off without redeploying), or a customer-specific setting.

How it works

1. Define a **strategy interface** — small, ideally one question (`delayBeforeAttempt(attempt)`).
2. Write one **concrete strategy** per algorithm; each holds only its own parameters.
3. The **context** receives a strategy (constructor, setter or method parameter) and calls it.
4. A **factory, configuration or DI container** (a library that creates objects and hands them their dependencies — see §11.7.2) decides which strategy to use — not every call site.

Structure (UML)

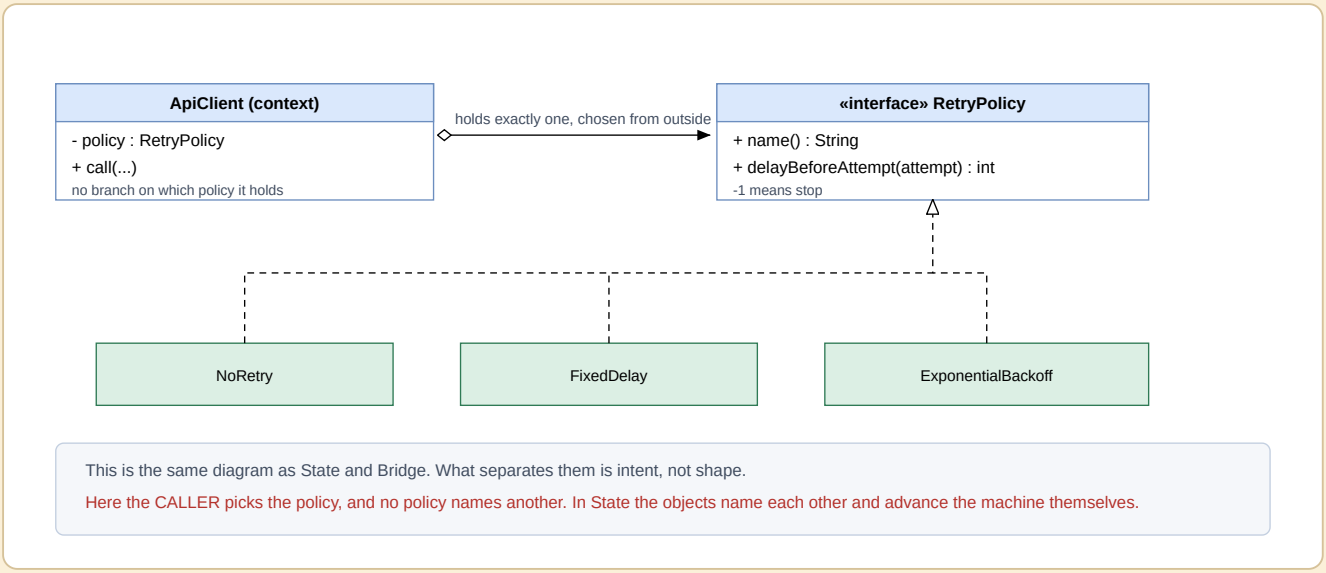
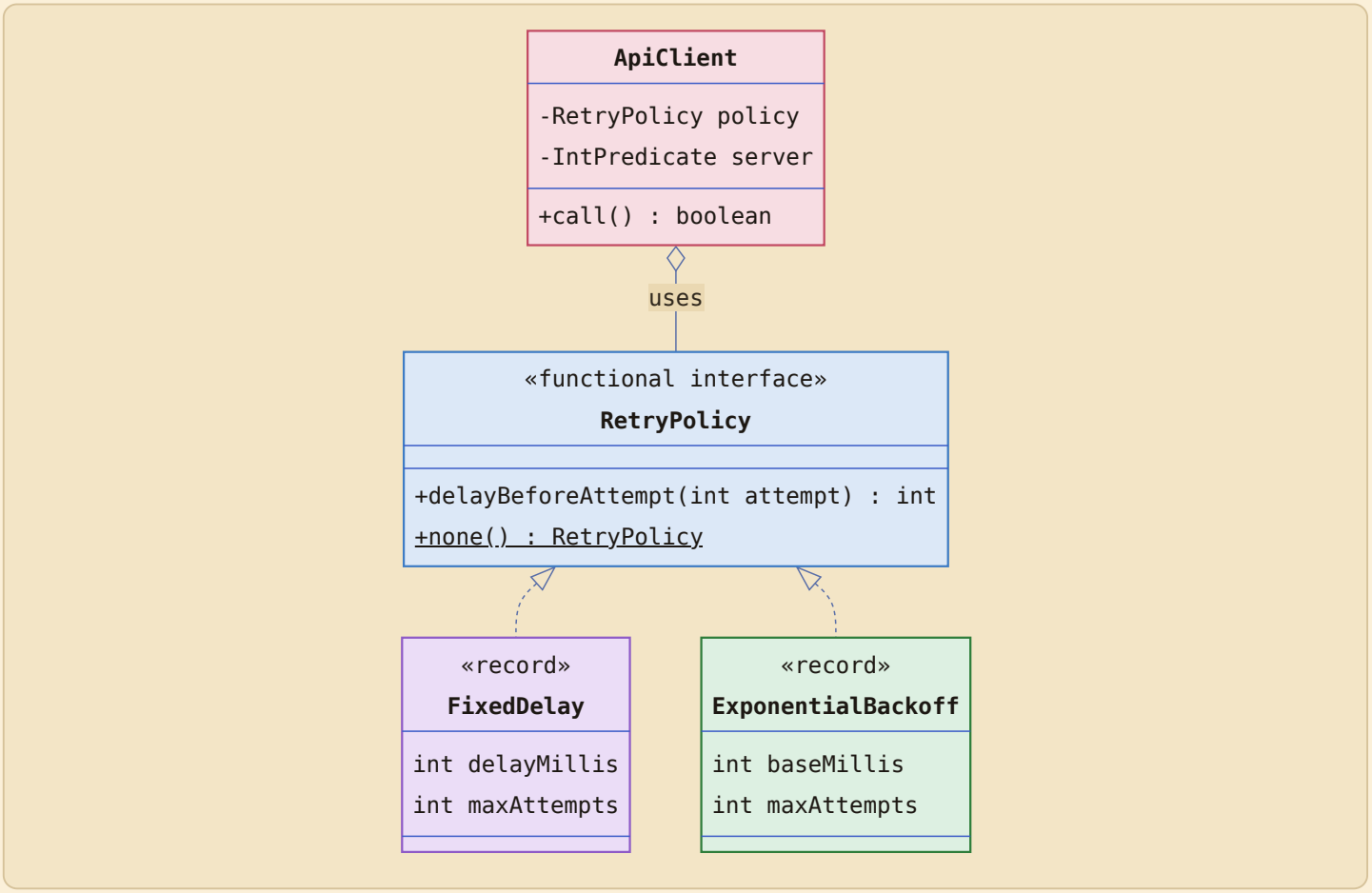


Figure — Class diagram: ApiClient holds a RetryPolicy; NoRetry, FixedDelay and ExponentialBackoff implement it (course diagram)

In the diagram above, **no strategy references another** — `ExponentialBackoff` has never heard of `FixedDelay`. That absence is how you tell Strategy from State.

Our Java design:



Java example

`lld.ch11.strategy`:

1. **Retry policies.** `RetryPolicy` has one abstract method, so `RetryPolicy.none()` is just a **lambda**. Strategies with configuration (`FixedDelay`, `ExponentialBackoff`) are **records**. `ApiClient.call()` is identical for all runs; only the injected policy differs. (Delays are printed, not slept, to keep the demo fast.)
2. `Comparator` — the JDK's everyday strategy. `List.sort(comparator)` keeps the sorting algorithm fixed and lets you plug in the comparison. `Comparator.comparing(...).reversed().thenComparing(...)` composes strategies.
3. **Strategy registry.** A `Map<String, LongUnaryOperator>` of lambdas picked by a configuration key — short and readable when each strategy is one line.

```java
// File: code/lld/ch11/strategy/RetryPolicy.java
package lld.ch11.strategy;

/**
 * STRATEGY interface: one small question. Returns the delay in ms before the given
 * attempt (attempt >= 2), or STOP. One abstract method, so a lambda can be a policy.
 */
@FunctionalInterface
public interface RetryPolicy {
    int STOP = -1;

    int delayBeforeAttempt(int attempt);

    static RetryPolicy none() { return attempt -> STOP; }
}
```

```java
// File: code/lld/ch11/strategy/FixedDelay.java
package lld.ch11.strategy;

/** Strategies with configuration earn a real type (a record here). */
public record FixedDelay(int delayMillis, int maxAttempts) implements RetryPolicy {
    public int delayBeforeAttempt(int attempt) {
        return attempt > maxAttempts ? STOP : delayMillis;
    }
}
```

```java
// File: code/lld/ch11/strategy/ExponentialBackoff.java
package lld.ch11.strategy;

/** 100, 200, 400, ... ms. Knows nothing about FixedDelay: strategies never name each other. */
public record ExponentialBackoff(int baseMillis, int maxAttempts) implements RetryPolicy {
    public int delayBeforeAttempt(int attempt) {
        return attempt > maxAttempts ? STOP : baseMillis << (attempt - 2);
    }
}
```

```java
// File: code/lld/ch11/strategy/ApiClient.java
package lld.ch11.strategy;

import java.util.function.IntPredicate;

/** CONTEXT: holds a policy and never asks WHICH policy it has. */
public final class ApiClient {
    private final RetryPolicy policy;
    private final IntPredicate server;     // fake server: attempt number -> success?

    public ApiClient(RetryPolicy policy, IntPredicate server) {
        this.policy = policy;
        this.server = server;
    }

    public boolean call() {
        for (int attempt = 1; ; attempt++) {
            if (server.test(attempt)) {
                System.out.println("  attempt " + attempt + ": ok");
                return true;
            }
            int delay = policy.delayBeforeAttempt(attempt + 1);
            if (delay == RetryPolicy.STOP) {
                System.out.println("  attempt " + attempt + ": failed, giving up");
                return false;
            }
            System.out.println("  attempt " + attempt + ": failed, waiting " + delay + "ms");  // simulated, no sleep
        }
    }
}
```

```java
// File: code/lld/ch11/strategy/Main.java
package lld.ch11.strategy;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.function.IntPredicate;
import java.util.function.UnaryOperator;

public class Main {
    record Product(String name, long priceCents, double rating) {}

    public static void main(String[] args) {
        System.out.println("=== 1. Retry policies: same ApiClient code, three strategies ===");
        IntPredicate succeedsOnThirdTry = attempt -> attempt >= 3;
        List<Map.Entry<String, RetryPolicy>> policies = List.of(
            Map.entry("none (a lambda)", RetryPolicy.none()),
            Map.entry("FixedDelay(100, 4)", new FixedDelay(100, 4)),
            Map.entry("ExponentialBackoff(100, 4)", new ExponentialBackoff(100, 4)));
        for (Map.Entry<String, RetryPolicy> e : policies) {
            System.out.println(e.getKey() + ":");
            new ApiClient(e.getValue(), succeedsOnThirdTry).call();
        }
        System.out.println("ExponentialBackoff(100, 4) against a server that is always down:");
        new ApiClient(new ExponentialBackoff(100, 4), attempt -> false).call();

        System.out.println("\n=== 2. Comparator: the JDK's everyday strategy ===");
        List<Product> products = new ArrayList<>(List.of(
            new Product("mouse", 1_999, 4.5), new Product("keyboard", 4_999, 4.7),
            new Product("cable", 499, 4.5), new Product("monitor", 19_999, 4.2)));
        Comparator<Product> byPrice = Comparator.comparingLong(Product::priceCents);
        Comparator<Product> byRatingThenName =
            Comparator.comparingDouble(Product::rating).reversed().thenComparing(Product::name);
        products.sort(byPrice);
        System.out.println("  by price:          " + products.stream().map(Product::name).toList());
        products.sort(byRatingThenName);
        System.out.println("  by rating, name:   " + products.stream().map(Product::name).toList());

        System.out.println("\n=== 3. Strategy registry: lambdas chosen by configuration ===");
        Map<String, UnaryOperator<BigDecimal>> pricing = Map.of(
            "regular", p -> p,
            "member", p -> p.multiply(new BigDecimal("0.90")).setScale(2, RoundingMode.HALF_EVEN),  // 10% off
            "flash-sale", p -> p.subtract(new BigDecimal("20.00")).max(BigDecimal.ZERO));          // flat 20.00 off
        BigDecimal keyboardPrice = new BigDecimal("49.99");
        for (String tier : List.of("regular", "member", "flash-sale")) {   // e.g. read from config
            UnaryOperator<BigDecimal> strategy = pricing.get(tier);
            System.out.printf("  %-10s keyboard costs %s%n", tier, strategy.apply(keyboardPrice));
        }
    }
}
```

Extending it is cheap: a policy that adds random **jitter** to the delay, or one that honours a server's `Retry-After` header (an **HTTP** response header in which the server says how long to wait; HTTP is the request/response protocol of the web), is one more class (or lambda) plus one line where the policy is chosen. `ApiClient` does not change.

Output:

```
=== 1. Retry policies: same ApiClient code, three strategies ===
none (a lambda):
  attempt 1: failed, giving up
FixedDelay(100, 4):
  attempt 1: failed, waiting 100ms
  attempt 2: failed, waiting 100ms
  attempt 3: ok
ExponentialBackoff(100, 4):
  attempt 1: failed, waiting 100ms
  attempt 2: failed, waiting 200ms
  attempt 3: ok
ExponentialBackoff(100, 4) against a server that is always down:
  attempt 1: failed, waiting 100ms
  attempt 2: failed, waiting 200ms
  attempt 3: failed, waiting 400ms
  attempt 4: failed, giving up

=== 2. Comparator: the JDK's everyday strategy ===
  by price:          [cable, mouse, keyboard, monitor]
  by rating, name:   [keyboard, cable, mouse, monitor]

=== 3. Strategy registry: lambdas chosen by configuration ===
  regular    keyboard costs 49.99
  member     keyboard costs 44.99
  flash-sale keyboard costs 29.99
```

Lambdas as strategies

Of all the patterns, Strategy is the one a plain function replaces most often, and recognising that is part of understanding it. Since Java 8, any interface with a **single abstract method** (a *functional interface*) can be implemented by a lambda or method reference. So many strategies need **no class at all**: `Comparator<T>`, `Function`, `Predicate`, `Supplier`, or your own `RetryPolicy`. Keep a named class (or record) when the strategy has **configuration or state** (`FixedDelay(100, 4)`), needs a **name for logging**, or must be **discovered/registered** (e.g. Spring beans — objects the Spring container creates and manages — or `ServiceLoader`, the JDK's built-in plug-in discovery mechanism).

Other languages: Python passes a function (or `functools.partial` to bind its parameters), and a dict from name to function is a one-line registry. JavaScript/TypeScript pass a function argument, or an object whose properties are functions. Kotlin/Scala take a function-typed constructor parameter, often with a default. Go uses a one-method interface (satisfied implicitly) or a `func` field.

The registry in part 3 of the demo uses `BigDecimal` for prices. Money in domain code should never be `double`, because binary floating point cannot represent most cent values exactly.

Trade-offs

Pros	Cons
New variants without editing working code — even by other teams (Open/Closed).	One class per variant (unless lambdas), one more file to open.
Each algorithm is testable alone.	Someone must choose — ideally a factory or container, not every caller.
Behaviour is configurable at run time (flag, setting, header).	The interface must fit every variant, including future ones.
The context keeps to its real responsibility (no backoff maths in an API client).	Tempting to overuse: three strategies of one line each means three classes for three lines.

When not to use it

- **Just two branches, with no third in sight** — an `if` reads better. Introduce the pattern when a third variant actually shows up.
- **One-line branches** — a `Map` from key to lambda is shorter than a class for each.
- **Variants need different inputs** (one needs the customer, another the warehouse) — the shared interface is forced; rethink the split.
- **The choice is fixed for the life of the program** — decide it when wiring the program (or use Bridge) instead.

Common mistakes

1. **Making every call site choose** — the branch moved instead of disappearing. Let a factory/config/DI container choose.
2. **A bloated interface** with parameters half the implementations ignore — a sign the variants do different jobs and should not share one abstraction.
3. **Strategies that know about each other** — then you have State.
4. **Stateful strategies shared between threads** — keep strategies immutable (records help).

Real-world examples

- `Comparator` with `Collections.sort` / `List.sort` / `TreeMap` — the textbook example.
- `ThreadPoolExecutor`**'s** (the JDK's configurable thread pool) `RejectedExecutionHandler`: `AbortPolicy`, `CallerRunsPolicy`, `DiscardPolicy`, `DiscardOldestPolicy` — pick what happens when the pool is full.
- **Spring Security** `PasswordEncoder` (`BCryptPasswordEncoder`, `Argon2PasswordEncoder`, ...) chosen by configuration, so upgrading the **hash** (a one-way scrambling of the password, so the stored value can't be turned back into it) does not touch login code.
- **Resilience4j** (a Java fault-tolerance library) retry `IntervalFunction` (fixed, exponential, randomized backoff) and **Polly** in .NET — exactly this lesson's retry policies.
- **AWT/Swing** `LayoutManager` (`FlowLayout`, `BorderLayout`, `GridLayout`) — a container delegates layout to a strategy.
- **Payment method at checkout** (card, UPI [India's instant bank-to-bank payment system], wallet, cash on delivery), **pricing/discount rules by customer tier**, **route planning modes** (drive, walk, transit) in map apps, **compression/serialization formats** (serialization = turning objects into bytes or text for storage or sending) chosen by a content-type header.

In the interview

> **30-second answer:** "Strategy gives every variant of one job its own class behind a common interface, and something outside — usually a factory or configuration — picks the variant at run time. The context stops branching: my API client asks its retry policy how long to wait and never asks which policy it is. Each algorithm is testable alone and new ones don't touch working code. In Java a lambda is often enough, like a `Comparator`; I keep a class when the strategy has configuration. It shares its diagram with State, but strategies never reference each other."

Follow-ups:

- *Strategy vs State?* Strategies are picked from outside and don't know each other; states pick their successors.
- *Strategy vs Template Method?* Template Method swaps **one step** of an algorithm whose order is fixed, through **inheritance** (fixed once the object is created, by the subclass you instantiate). Strategy swaps **the complete algorithm** through **composition** (holding another object in a field and delegating to it) (chosen at run time, can vary several axes at once). Prefer Strategy when both fit.
- *Strategy vs Bridge?* Same diagram again. Bridge splits off a dimension of the design that is fixed when the program is assembled; Strategy is a decision that is meant to change, even from one call to the next. If a reviewer cannot tell which you meant, little is lost — the code is identical. What matters more is not using any of the three for two branches that will never become three.
- *Strategy vs Command?* Strategy is *how* to do something; Command is *what* to do, with its arguments, kept to run later or undo.
- *How is the strategy selected?* Config + factory (`Map<String, Strategy>`), DI container (Spring can inject a `Map<String, PaymentStrategy>` of all beans), or per-request (header, customer tier).
- *Relation to Open/Closed and DIP?* The context depends on an abstraction (DIP) and new strategies extend behaviour without modifying it (OCP).

Self-check

1. **Q:** Who chooses the strategy? **A:** Someone outside the context — ideally a factory/config/DI container.
2. **Q:** Why can `RetryPolicy.none()` be a lambda? **A:** `RetryPolicy` is a functional interface (one abstract method).
3. **Q:** Name two JDK strategies. **A:** `Comparator`; `RejectedExecutionHandler` (also `LayoutManager`).
4. **Q:** When is a map of lambdas better than classes? **A:** When each strategy is one line and needs no configuration.

5. **Q:** How do you tell Strategy from State in code? **A:** Strategies never reference each other.

TL;DR

Interchangeable algorithms behind one interface, chosen from outside. The context stops branching. Use lambdas for one-liners, records/classes when configured. Don't confuse with State (states know each other) or Template Method (inheritance, one step).

11.4 Template Method

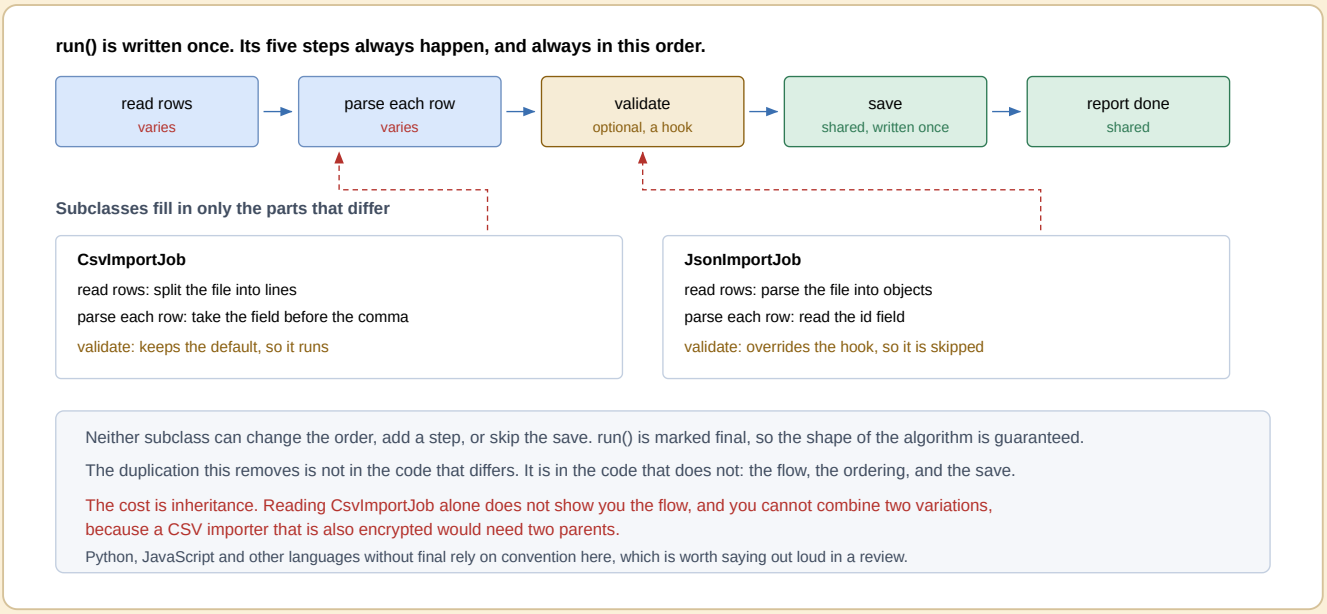


Figure — run() always performs five steps in the same order; subclasses fill in only reading and parsing, and one optional hook (course diagram)

What it is

Template Method (GoF intent, paraphrased): put the **outline of an algorithm** into one base-class operation (the *template method*) and leave certain steps for **subclasses** to fill in; subclasses may redefine those steps but cannot alter the overall sequence.

GoF participants (and who plays them in our code):

Participant	Role	In `lld.ch11.templatemethod`
AbstractClass	Declares the primitive steps (abstract or with a default) and implements the template method that calls them in order	`ImportJob`, `CaffeineBeverage`
ConcreteClass	Implements the primitive steps that differ for its variant	`CsvImportJob`, `JsonImportJob`, `Tea`, `Coffee`

Everyday analogy: a recipe card with blanks. "Boil water → brew ___ → pour into cup → add ___ (if wanted)." Tea and coffee fill the blanks differently; the order never changes.

The problem it solves

A data import job always: reads rows, parses each one, validates, saves, reports "done". Only reading and parsing differ between CSV and JSON. Written as separate classes, the **shared flow** (order, validation, saving, reporting) is copied into each. When saving must also write an audit record, three files change — and the missed one is the bug.

Template Method writes the flow once, in a method subclasses **cannot override** (`final`). The varying steps become methods the subclasses must supply.

How it works

1. Write the **template method** in an abstract class and mark it `final` so the order is guaranteed.
2. Inside it, call the steps in order. Steps come in three kinds:

- **Required (abstract) steps** — every subclass must implement them; the compiler enforces it.
- **Hooks** — steps with a **default** (often empty or `true`) that a subclass *may* override, e.g. `shouldValidate()`.
- **Shared steps** — `private`, written once, not changeable.

3. Each subclass implements only what differs.

The Hollywood Principle — "Don't call us, we'll call you." (*Head First Design Patterns*.) The base class (high-level component) decides **when** the subclass's methods run; the subclass never calls the base class's flow itself. That **inversion of control** is the backbone of most **frameworks** (libraries that own the program's flow and call your code at set points): **the framework runs the sequence, and your class supplies only your own pieces.**

Structure (UML)

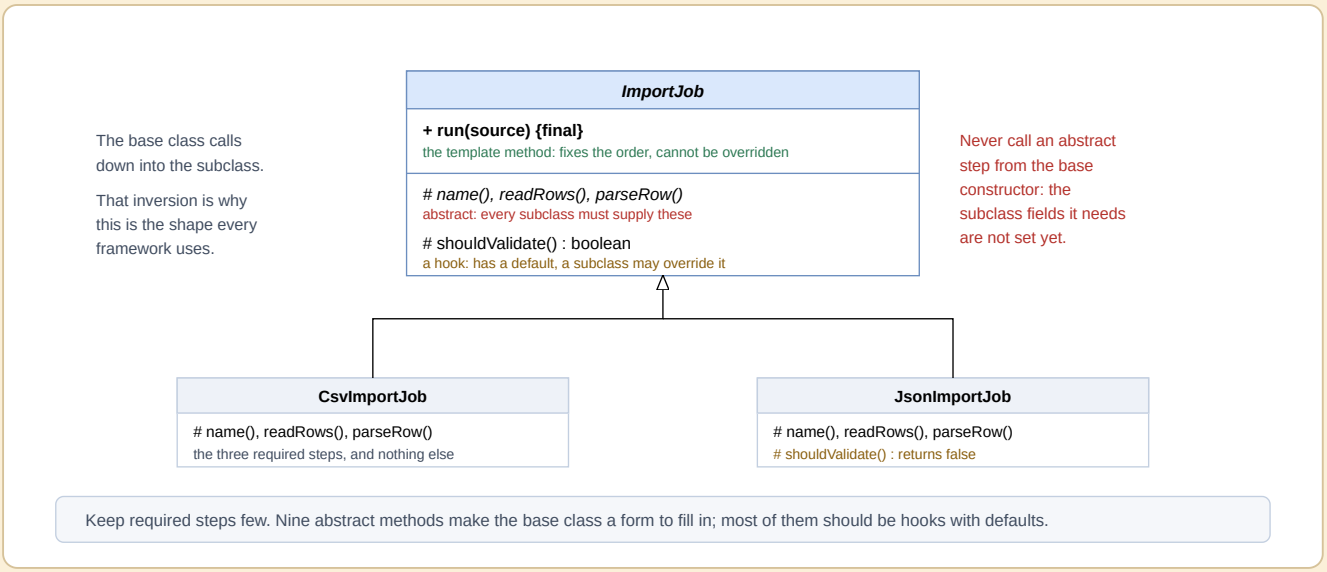
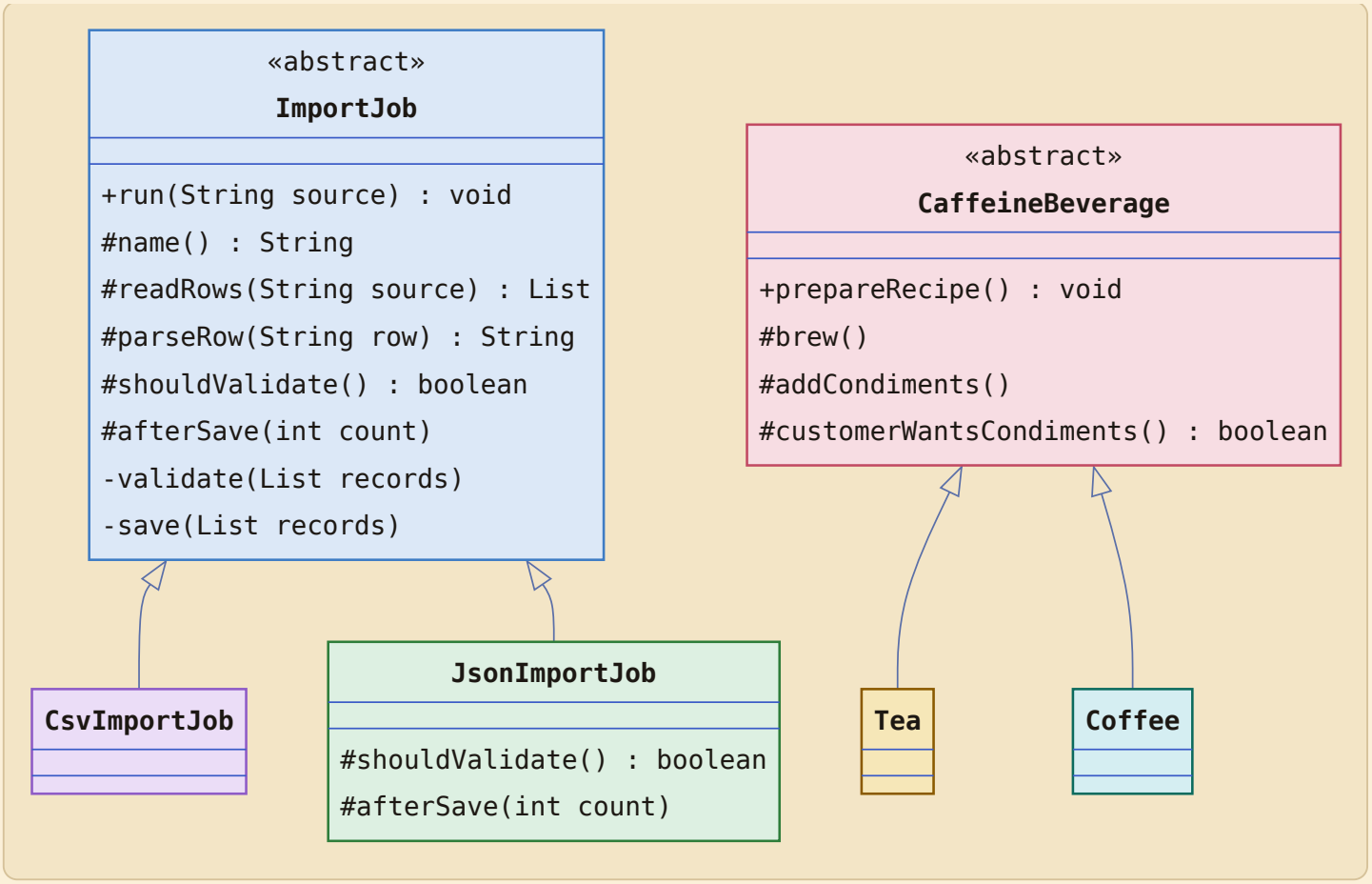


Figure — Class diagram: ImportJob with final run(), abstract steps name/readRows/parseRow, hook shouldValidate and private save; CsvImportJob and JsonImportJob override only what differs (course diagram)

In the diagram above, the subclasses are tiny — but `CsvImportJob` on its own does not reveal the sequence it will follow at run time; only the parent shows that. That is the trade-off.

Our Java design:



`run()` and `prepareRecipe()` are `final` in the code (Mermaid has no marker for `final`).

Java example

`lld.ch11.templatemethod`:

1. **Import jobs.** `ImportJob.run` is `final`. `name`, `readRows`, `parseRow` are **abstract** (required). `shouldValidate` and `afterSave` are **hooks** (defaults `true` and "do nothing"). `validate` and `save` are **private** shared steps. `JsonImportJob` overrides both hooks.
2. **Beverages** (the *Head First* example). `customerWantsCondiments()` is a hook: `Coffee(black = true)` turns off the optional step.

```java
// File: code/lld/ch11/templatemethod/ImportJob.java
package lld.ch11.templatemethod;

import java.util.ArrayList;
import java.util.List;

/** ABSTRACT CLASS holding the TEMPLATE METHOD. */
public abstract class ImportJob {

    /** Template method. Marked final: subclasses cannot change the order, drop a step or insert one. */
    public final void run(String source) {
        System.out.println("Running the " + name() + " import:");
        List<String> rows = readRows(source);                // required step
        List<String> records = new ArrayList<>();
        for (String row : rows) records.add(parseRow(row));   // required step
        System.out.println("  parsed " + records.size() + " records: " + records);
        if (shouldValidate()) {                              // hook
            validate(records);
        } else {
            System.out.println("  skipping validation, source is validated upstream");
        }
        save(records);                                       // shared, fixed
        afterSave(records.size());                           // hook with an empty default
        System.out.println("  done");
    }

    // Required steps: the compiler forces every subclass to supply them.
    protected abstract String name();
    protected abstract List<String> readRows(String source);
    protected abstract String parseRow(String row);

    // Hooks: sensible defaults, overriding is optional.
    protected boolean shouldValidate() { return true; }
    protected void afterSave(int count) { }

    // Shared steps: written once, private, cannot be changed by subclasses.
    private void validate(List<String> records) {
        long blank = records.stream().filter(String::isBlank).count();
        System.out.println("  validated " + records.size() + " records, " + blank + " blank");
    }

    private void save(List<String> records) {
        System.out.println("  saved " + records.size() + " records (+ audit entry)");
    }
}
```

```java
// File: code/lld/ch11/templatemethod/CsvImportJob.java
package lld.ch11.templatemethod;

import java.util.Arrays;
import java.util.List;

/** Supplies only what differs for CSV. */
public final class CsvImportJob extends ImportJob {
    @Override protected String name() { return "CSV"; }

    @Override
    protected List<String> readRows(String source) {
        return Arrays.stream(source.split("\n")).skip(1).toList();   // skip the header line
    }

    @Override
    protected String parseRow(String row) { return row.split(",")[0].trim(); }
}
```

```java
// File: code/lld/ch11/templatemethod/JsonImportJob.java
package lld.ch11.templatemethod;

import java.util.List;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

/** Supplies what differs for JSON, and overrides two hooks. (Toy parsing: no JSON library.) */
public final class JsonImportJob extends ImportJob {
    private static final Pattern OBJECT = Pattern.compile("\\{[^}]*}");
    private static final Pattern ID = Pattern.compile("\"id\"\\s*:\\s*\"([^\"]*)\"");

    @Override protected String name() { return "JSON"; }

    @Override
    protected List<String> readRows(String source) {
        return OBJECT.matcher(source).results().map(m -> m.group()).toList();
    }

    @Override
    protected String parseRow(String row) {
        Matcher m = ID.matcher(row);
        return m.find() ? m.group(1) : "";
    }

    @Override protected boolean shouldValidate() { return false; }

    @Override
    protected void afterSave(int count) { System.out.println("  [hook] notified upstream: " + count + " ids consumed"); }
}
```

```java
// File: code/lld/ch11/templatemethod/CaffeineBeverage.java
package lld.ch11.templatemethod;

/** The classic beverage example: same recipe skeleton, two steps and one hook vary. */
public abstract class CaffeineBeverage {
    public final void prepareRecipe() {          // template method
        boilWater();
        brew();
        pourInCup();
        if (customerWantsCondiments()) addCondiments();
    }

    protected abstract void brew();
    protected abstract void addCondiments();

    /** Hook: "Don't call us, we'll call you" - the base class decides WHEN this runs. */
    protected boolean customerWantsCondiments() { return true; }

    private void boilWater() { System.out.println("  boiling water"); }
    private void pourInCup() { System.out.println("  pouring into cup"); }
}
```

```java
// File: code/lld/ch11/templatemethod/Tea.java
package lld.ch11.templatemethod;

public final class Tea extends CaffeineBeverage {
    @Override protected void brew() { System.out.println("  steeping the tea"); }
    @Override protected void addCondiments() { System.out.println("  adding lemon"); }
}
```

```java
// File: code/lld/ch11/templatemethod/Coffee.java
package lld.ch11.templatemethod;

public final class Coffee extends CaffeineBeverage {
    private final boolean black;

    public Coffee(boolean black) { this.black = black; }

    @Override protected void brew() { System.out.println("  dripping coffee through filter"); }
    @Override protected void addCondiments() { System.out.println("  adding sugar and milk"); }
    @Override protected boolean customerWantsCondiments() { return !black; }
}
```

```java
// File: code/lld/ch11/templatemethod/Main.java
package lld.ch11.templatemethod;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== 1. Import jobs ===");
        new CsvImportJob().run("id,name\nu1, Asha\nu2, Ben\nu3, Chen");
        new JsonImportJob().run("[{\"id\":\"p9\",\"qty\":1},{\"id\":\"p7\",\"qty\":4}]");

        System.out.println("\n=== 2. Beverages (hook controls an optional step) ===");
        System.out.println("tea:");
        new Tea().prepareRecipe();
        System.out.println("black coffee:");
        new Coffee(true).prepareRecipe();
    }
}
```

Output:

```
=== 1. Import jobs ===
Running the CSV import:
  parsed 3 records: [u1, u2, u3]
  validated 3 records, 0 blank
  saved 3 records (+ audit entry)
  done
Running the JSON import:
  parsed 2 records: [p9, p7]
  skipping validation, source is validated upstream
  saved 2 records (+ audit entry)
  [hook] notified upstream: 2 ids consumed
  done

=== 2. Beverages (hook controls an optional step) ===
tea:
  boiling water
  steeping the tea
  pouring into cup
  adding lemon
black coffee:
  boiling water
  dripping coffee through filter
  pouring into cup
```

Template Method vs Strategy

	Template Method	Strategy
What varies	**One step**; the surrounding algorithm is fixed	**The entire** algorithm
Mechanism	**Inheritance** — subclass supplies the step	**Composition** — caller passes an object in
When chosen	When the object is created (by which subclass you instantiate); fixed after that	Run time, can change per call
How many axes can vary	One (a class has one parent)	Several (hold several strategies)
What one file shows	Just the varying pieces	The complete algorithm

Strategy is usually the more flexible choice. Template Method pays off when **most of the flow is common**, only small pieces differ, and you want the compiler to **enforce the sequence**. They combine well: the base class fixes the sequence, but each step is a **strategy object** (or lambda) handed in rather than inherited — so the order is still enforced, yet you are not limited to one parent. Spring's `JdbcTemplate` (a helper around **JDBC**, Java's standard API for talking to SQL databases) works like this: it owns the fixed flow (get connection, create statement, execute, handle exceptions, close resources) and takes your varying part as a **callback** (a function or object you hand over so the other code can call you back — here `RowMapper`, `PreparedStatementSetter`) rather than a subclass.

Trade-offs

Pros	Cons
The shared flow is written once (DRY: Don't Repeat Yourself).	No single file tells the whole story: the parent holds the order, the child holds the pieces.
The order is guaranteed (`final`).	One parent only: two independent variations can't combine.
New variants are small.	Parent and children are bound tightly; altering one step's signature forces every subclass to change.
Abstract steps make the compiler report anything a subclass has not supplied.	Base classes tend to swell until they demand many abstract methods and nobody wants to extend them.

When not to use it

- **The flow is two lines** — nothing to protect.
- **Variants must combine** (an encrypted CSV import would need both an "encrypted" and a "CSV" parent) — use composition.
- **Behaviour must change after creation** — use Strategy.
- **Subclasses override every step** — the flow was never really shared.

Common mistakes

1. **Too many required steps.** A subclass should only have to write two or three methods; give the rest a default and make them hooks.
2. **Calling an overridable step from the base constructor.** The base constructor runs **before** the subclass's fields are initialised, so in Java the step sees `null`/`0`/`false`. (In C++ it is different but still wrong: during the base constructor a virtual call is dispatched to the base version, never the subclass override.) Call steps from the template method, never the constructor.
3. **Leaving out `final` on the template method** — then a subclass can reorder or skip steps; nothing is enforced any more and the pattern becomes a mere agreement. (C++: make it non-virtual. Python and JavaScript cannot enforce it at all, so it must be documented and checked in review.)
4. **Making steps `public`** — callers could run step 3 without steps 1 and 2. Keep them `protected` (or move reusable steps into a separate object).
5. **Violating Liskov** (the Liskov Substitution Principle: a subclass must work anywhere its parent is expected) in overrides (a hook that throws, or a step that changes the contract).

Template Method in other languages

- **Java:** `final` template method, `protected abstract` required steps, `protected` hooks with a body. An interface `default` method can also act as a template method, but an interface has no instance state and a default method cannot be `final`, so any implementer may override it.
- **Python:** `abc.ABC` with `@abstractmethod`. There is no `final` enforced at run time, so a comment (or `typing.final` for type checkers) is the only guard; a leading underscore marks internal steps.
- **TypeScript:** `abstract` classes give the compile-time check; plain JavaScript uses base methods that throw.
- **C++:** a non-virtual public template method calling `protected virtual` steps — the **non-virtual interface (NVI)** idiom.
- **Go and Rust** have no inheritance: a function taking an interface/trait with the varying steps — i.e. the Strategy shape.

Real-world examples

- **`java.io.InputStream`**: `read(byte[], int, int)` is implemented in terms of the abstract `read()`; subclasses supply the single-byte read (and override the bulk method for speed).
- **`java.util.AbstractList`**: implement `get(int)` and `size()`, and you inherit `iterator()`, `indexOf`, `contains`, `equals`, `hashCode`...
- **`HttpServlet.service()`** (Jakarta/Java Servlet) dispatches to `doGet`/`doPost`/`doPut`... by HTTP method; you override the ones you support, and the ones you do not override reject the request (405 Method Not Allowed on HTTP/1.1).
- **`AbstractQueuedSynchronizer`** (AQS — the JDK's internal building block for locks; behind `ReentrantLock`, `Semaphore`, `CountDownLatch`): `acquire()` is the template; subclasses implement `tryAcquire`/`tryRelease`.
- **Test frameworks**: JUnit (`@BeforeEach`/`@AfterEach`) and pytest (fixtures, `setup_method`/`teardown_method`) own the set-up → test → tear-down flow; your hooks never call each other. **Mobile lifecycles**: Android `Activity` (`onCreate`, `onStart`, `onResume`...) and iOS `UIViewController` (`viewDidLoad`, `viewWillAppear`...). **Maven/Gradle** (Java build tools) build lifecycles (fixed phases, you plug in goals/tasks).
- **Sorting with a comparison**: `Arrays.sort` owns the algorithm and needs just one step from you, the comparison. Old designs had the elements supply it (`Comparable.compareTo`); modern APIs pass it in as a `Comparator` — the Strategy form of the same idea.
- **Spring** `JdbcTemplate`, `RestTemplate`, `TransactionTemplate` — "template + callback" variants of the same idea.

In the interview

> **30-second answer:** "Template Method puts an algorithm's outline in a single final method of a base class, and subclasses supply only the steps that differ. Steps a subclass must write are abstract; steps it may change are hooks with a default body; steps nobody should change are private. It's the Hollywood principle — the base class calls the subclass, not the other way round — which is how frameworks like servlets, JUnit and Android lifecycles work. It varies one step via inheritance; if I need run-time choice or several axes I prefer Strategy, or pass the steps in as callbacks like JdbcTemplate does."

Follow-ups:

- *What is a hook?* A step with a default implementation that subclasses may override (e.g. `shouldValidate()`), giving optional control points.
- *What is the Hollywood Principle?* "Don't call us, we'll call you": high-level components call low-level ones at the right time; low-level components plug in but don't call up. It prevents "dependency rot" (dependencies tangled in every direction, so nothing can change safely).
- *Why `final`?* It is the guarantee: without it a subclass could reorder or drop steps, and the pattern would be only a gentleman's agreement.
- *Template Method vs Factory Method?* A factory method is often one step of a template method — the step whose job is to create an object (GoF notes that template methods frequently call factory methods). The course frames it more strongly — a factory method is a special template-method step whose subclass-supplied job is creation, while most template steps create nothing. Take that as "a factory method usually lives inside a template method", not as a strict rule: a factory method can also be called directly by clients.
- *Template Method vs Decorator?* A decorator wraps an existing object at run time, and several can be layered; a template-method subclass is decided when the object is built and cannot be layered.
- *Template Method vs Builder?* Both impose an order, but a Builder director invokes the steps on a **different** object (the builder it holds), while a template method invokes them on **its own** instance via inheritance.
- *Constructor trap?* Never call overridable methods from a constructor; the subclass isn't initialised yet.

Self-check

1. **Q:** Which method is `final`? **A:** The template method (`run`, `prepareRecipe`).
2. **Q:** Difference between an abstract step and a hook? **A:** Abstract steps must be implemented; hooks have defaults and are optional.
3. **Q:** Name two JDK template methods. **A:** `InputStream.read(byte[],int,int)`, `AbstractList`'s methods built on `get`/`size`, AQS `acquire`.
4. **Q:** Why is `JdbcTemplate` more flexible than a classic template method? **A:** It takes the varying step as a callback object (composition), not a subclass.
5. **Q:** What goes wrong if the base constructor calls an abstract step? **A:** The subclass fields aren't initialised yet.

TL;DR

Fixed order in a `final` base method; subclasses fill steps; hooks are optional steps. Hollywood principle. Inheritance-based and compile-time — prefer Strategy/callbacks when you need flexibility.

11.5 Visitor

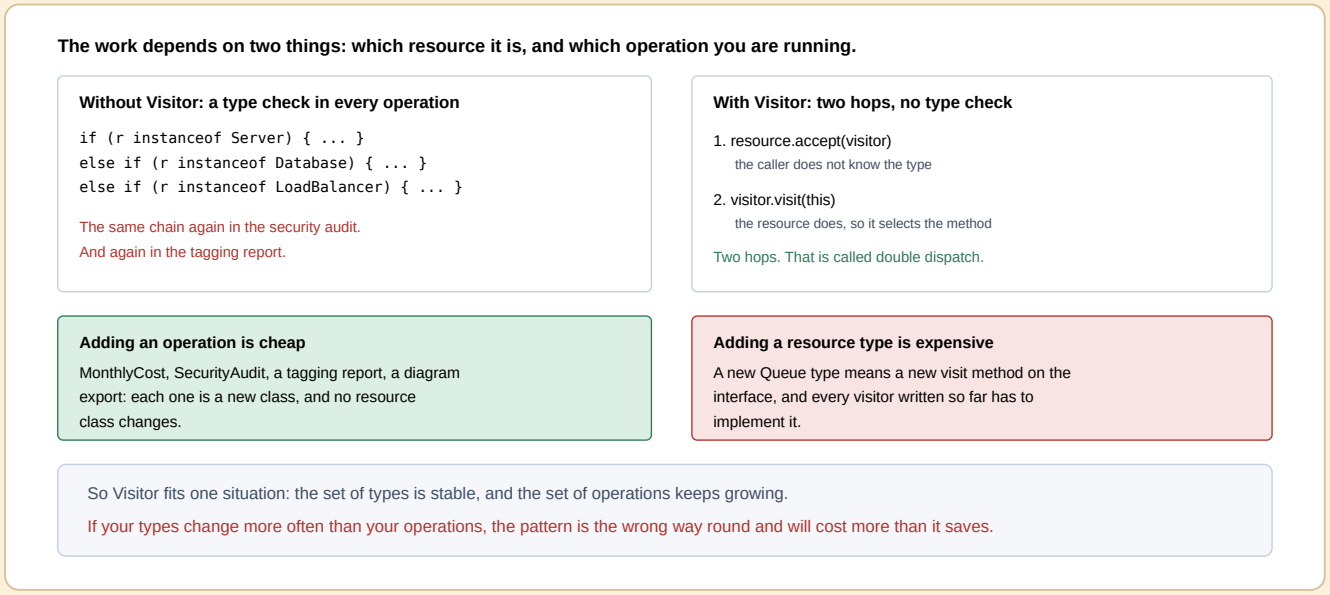


Figure — Without Visitor, every operation repeats an instanceof chain; with Visitor, two calls (accept, then visit) select the right code with no type check (course diagram)

What it is

Visitor lets you **introduce new operations** over a group of classes **while leaving those classes untouched**. Every operation lives in its own **visitor** class, which has a separate method for each kind of element; each element keeps one tiny method, `accept(visitor)`, whose body is `visitor.visit(this)`.

GoF intent (paraphrased): represent an operation to be performed on the elements of an object structure as a separate object, so you can define a new operation without changing the classes of the elements it works on.

GoF participants (and who plays them in our code):

Participant	Role	In `lld.ch11.visitor`
Visitor	Declares one `visit` operation per ConcreteElement class	`PlanVisitor`
ConcreteVisitor	Implements every `visit`; one class per operation, often accumulating a result	`MonthlyCost`, `SecurityAudit`
Element	Declares `accept(visitor)`	`Resource`
ConcreteElement	Implements `accept` as `visitor.visit(this)`	`Server`, `Database`, `LoadBalancer`, `ResourceGroup`
ObjectStructure	Holds the elements and lets a visitor reach each of them (a list or a Composite)	the `ResourceGroup` tree built in `Main`

Everyday analogy: a tax inspector visiting different businesses. The businesses don't each contain tax rules; the inspector brings them and applies the right rule for "restaurant", "shop" or "factory". Next year, a fire-safety inspector visits the same businesses with different rules — the businesses don't change.

The problem it solves

Picture an infrastructure plan made of just three resource kinds — server, database, load balancer. That **list of kinds is stable**. The **operations never stop arriving**: a monthly bill, a security review, a tag report, a diagram export, and so on.

- Put each operation **inside** the resource classes → `Server` holds pricing, security, tagging and drawing code; unrelated teams edit the same file.
- Put it **outside** with type checks → an `instanceof` chain in every operation. Add a resource kind, and you must find every chain; the compiler won't tell you which one you missed.

Visitor gives each operation its own class, and uses **double dispatch** to pick the right method without any type check.

How it works — double dispatch

Java chooses **which override** (a subclass's replacement of an inherited method) of a method runs based on the **run-time type of the receiver** — the object before the dot (single dispatch) — but chooses **which overload** (one of several same-named methods with different parameter types: `visit(Server)` vs `visit(Database)`) at **compile time**, from the **static type** (the type the compiler sees in the declaration) of the argument. If you hold a `Resource r`, `visitor.visit(r)` cannot compile to the right overload — the compiler only knows "Resource".

Visitor makes **two calls**:

1. `resource.accept(visitor)` — dispatch #1 on the **resource's** run-time type: `Server.accept` runs.
2. Inside `Server.accept`, `visitor.visit(this)` — here `this` has static type `Server`, so the compiler picks `visit(Server)`; dispatch #2 then happens on the **visitor's** run-time type (`MonthlyCost` or `SecurityAudit`).

The method that finally runs depends on **two** types — element and visitor. That is **double dispatch**.

Structure (UML)

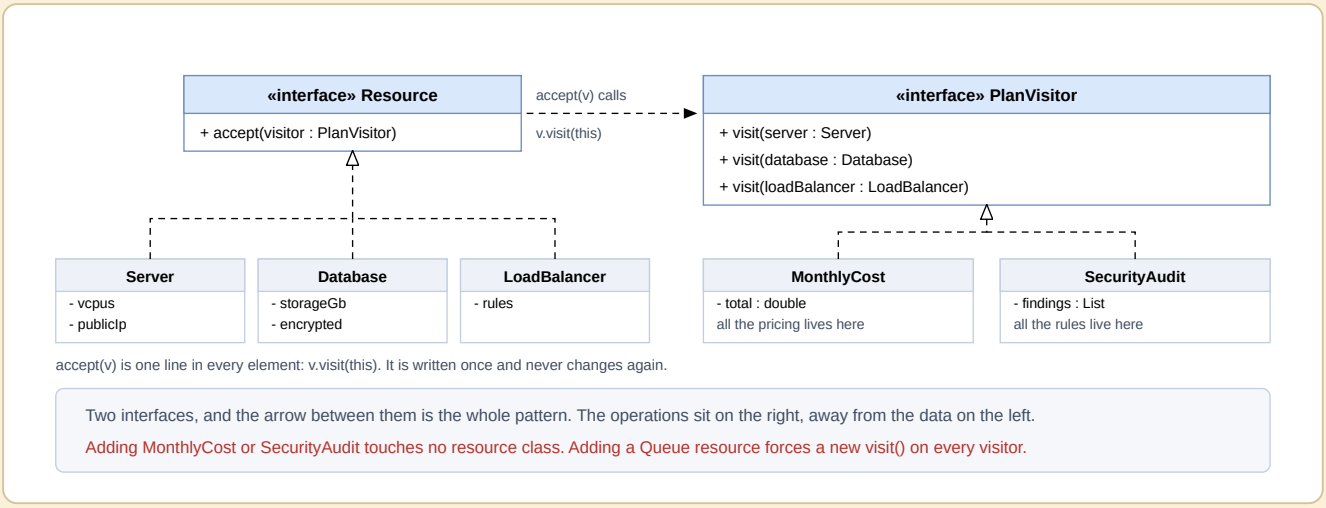
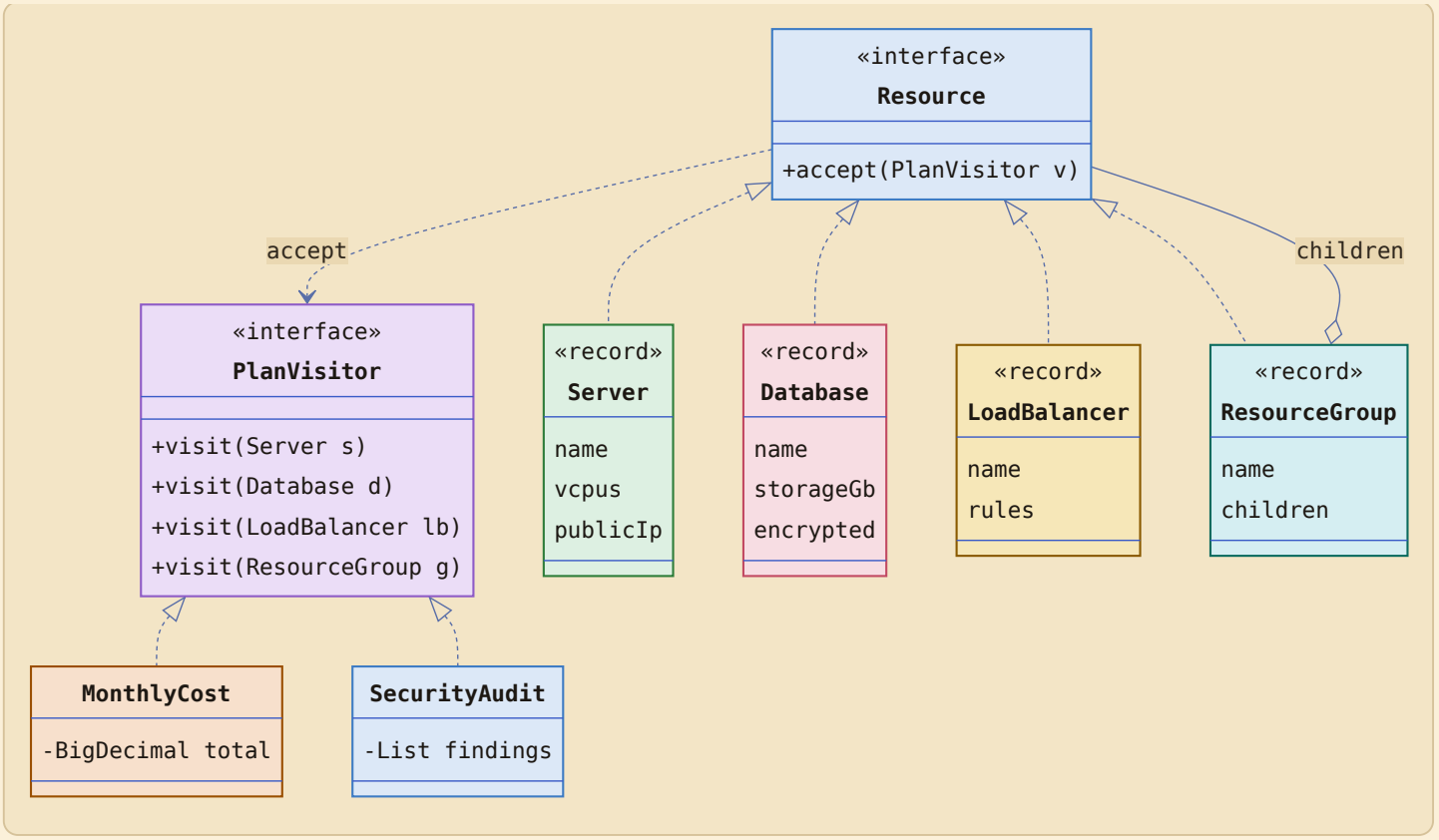
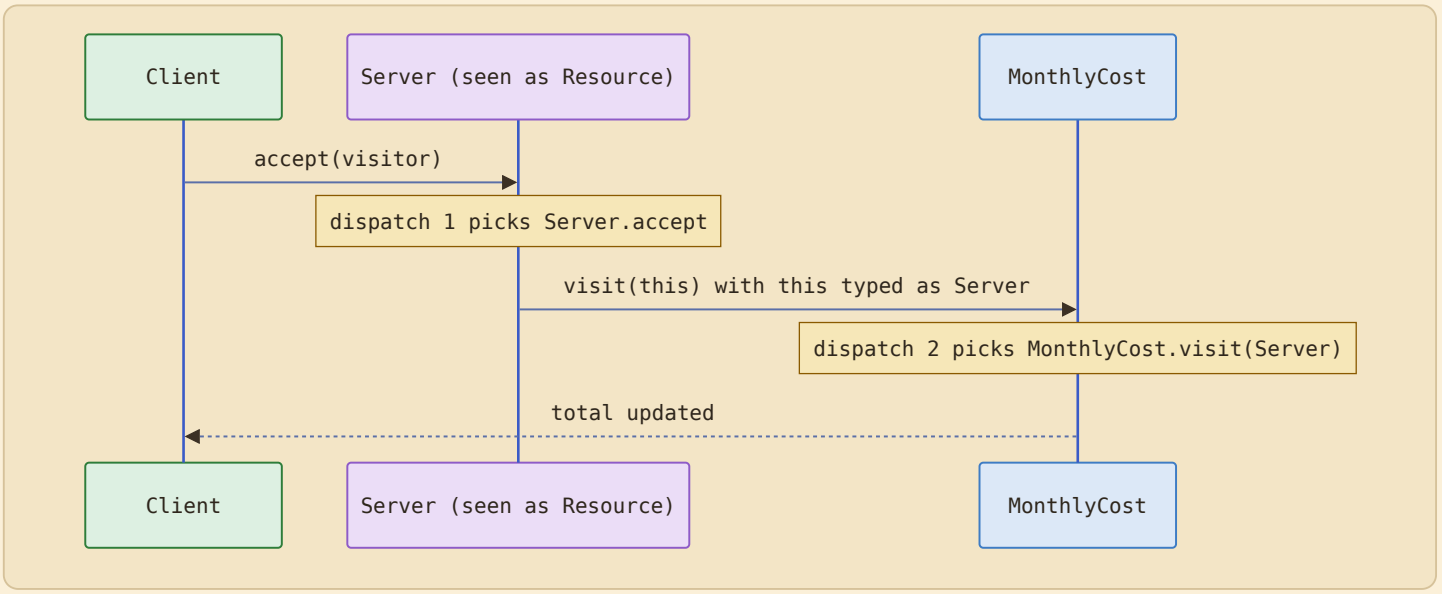


Figure — Class diagram: Resource with accept; Server, Database, LoadBalancer implement it; PlanVisitor has one visit per resource type; MonthlyCost and SecurityAudit implement it (course diagram; it shows `total : double`, our Java code uses `BigDecimal` for money)

In the diagram above the two sets of arrows point in opposite directions: each element depends on the visitor interface (its `accept` has a visitor parameter), while each visitor depends on every element type (it has a `visit` for each). That second arrow is why **adding an element type is expensive**.

Our Java design (with a Composite group element):





Java example

`lld.ch11.visitor` has **both** styles over the same domain:

1. **Classic Visitor.** Elements are **records** (so visitors read data through accessors, not public fields). `ResourceGroup` is a **Composite** (the pattern in which a group and a single item share one interface, so a tree can be handled uniformly): its `accept` visits itself and then its children. (A tree is optional: Visitor only requires a stable set of element types, and it works just as well over a flat list. Composite is simply the most common route into Visitor.) Prices are `BigDecimal`, never `double`. `MonthlyCost` and `SecurityAudit` each keep all their rules in one file; neither touches a resource class. A new visitor instance is used per run because they accumulate results.
2. **Java 21 alternative — sealed interface + pattern matching** (`ModernPlan`). `Infra` is `sealed ... permits Vm, Sql, Lb`. `monthlyCost` is a `switch` with **no** `default` — the compiler checks it is **exhaustive** (covers every possible case), so adding a new permitted record breaks every switch that forgot it (the same safety net Visitor gave you). `audit` uses **record patterns** (deconstruction, `case Sql(String n, int gb, boolean enc)`) and **guards** (`when !enc`). A guarded case never counts toward exhaustiveness (the compiler cannot know the guard is sometimes true), which is why `audit` also needs the three unguarded `case Vm vm` / `Sql db` / `Lb lb` lines.

```java
// File: code/lld/ch11/visitor/Resource.java
package lld.ch11.visitor;

/** ELEMENT interface: the only thing every resource must provide is accept. */
public interface Resource {
    void accept(PlanVisitor visitor);
}
```

```java
// File: code/lld/ch11/visitor/PlanVisitor.java
package lld.ch11.visitor;

/** VISITOR interface: one visit per element type. This list is what makes new types expensive. */
public interface PlanVisitor {
    void visit(Server server);
    void visit(Database database);
    void visit(LoadBalancer loadBalancer);
    void visit(ResourceGroup group);
}
```

```java
// File: code/lld/ch11/visitor/Server.java
package lld.ch11.visitor;

/** Records expose data through accessors, so visitors need no public fields. */
public record Server(String name, int vcpus, boolean publicIp) implements Resource {
    public void accept(PlanVisitor v) { v.visit(this); }   // 2nd dispatch: overload picked by static type Server
}
```

```java
// File: code/lld/ch11/visitor/Database.java
package lld.ch11.visitor;

public record Database(String name, int storageGb, boolean encrypted) implements Resource {
    public void accept(PlanVisitor v) { v.visit(this); }
}
```

```java
// File: code/lld/ch11/visitor/LoadBalancer.java
package lld.ch11.visitor;

public record LoadBalancer(String name, int rules) implements Resource {
    public void accept(PlanVisitor v) { v.visit(this); }
}
```

```java
// File: code/lld/ch11/visitor/ResourceGroup.java
package lld.ch11.visitor;

import java.util.List;

/** COMPOSITE element: visits itself, then passes the visitor to every child. */
public record ResourceGroup(String name, List<Resource> children) implements Resource {
    public void accept(PlanVisitor v) {
        v.visit(this);
        for (Resource child : children) child.accept(v);
    }
}
```

```java
// File: code/lld/ch11/visitor/MonthlyCost.java
package lld.ch11.visitor;

import java.math.BigDecimal;

/**
 * Operation 1: ALL pricing rules in one file. New instance per run (it accumulates).
 * Money is BigDecimal: exact decimal arithmetic, no binary rounding surprises.
 */
public final class MonthlyCost implements PlanVisitor {
    static final BigDecimal PER_VCPU = new BigDecimal("24.00");
    static final BigDecimal DB_BASE = new BigDecimal("40.00");
    static final BigDecimal PER_GB = new BigDecimal("0.25");
    static final BigDecimal LB_BASE = new BigDecimal("18.00");
    static final BigDecimal PER_RULE = new BigDecimal("1.50");

    private BigDecimal total = BigDecimal.ZERO;

    public BigDecimal total() { return total; }

    public void visit(Server s)        { total = total.add(PER_VCPU.multiply(BigDecimal.valueOf(s.vcpus()))); }
    public void visit(Database d)      { total =
total.add(DB_BASE.add(PER_GB.multiply(BigDecimal.valueOf(d.storageGb())))); }
    public void visit(LoadBalancer lb) { total =
total.add(LB_BASE.add(PER_RULE.multiply(BigDecimal.valueOf(lb.rules())))); }
    public void visit(ResourceGroup g) { /* groups are free */ }
}
```

```java
// File: code/lld/ch11/visitor/SecurityAudit.java
package lld.ch11.visitor;

import java.util.ArrayList;
import java.util.List;

/** Operation 2: added later without touching a single resource class. */
public final class SecurityAudit implements PlanVisitor {
    private final List<String> findings = new ArrayList<>();

    public List<String> findings() { return findings; }

    public void visit(Server s) {
        if (s.publicIp()) findings.add(s.name() + ": server has a public IP");
    }
    public void visit(Database d) {
        if (!d.encrypted()) findings.add(d.name() + ": storage not encrypted");
    }
    public void visit(LoadBalancer lb) {
        if (lb.rules() == 0) findings.add(lb.name() + ": load balancer has no rules");
    }
    public void visit(ResourceGroup g) {
        if (g.children().isEmpty()) findings.add(g.name() + ": empty group");
    }
}
```

```java
// File: code/lld/ch11/visitor/ModernPlan.java
package lld.ch11.visitor;

import java.math.BigDecimal;
import java.util.Optional;

/**
 * The Java 21 alternative: a SEALED hierarchy + pattern-matching switch.
 * No accept(), no visitor interface. The compiler still checks exhaustiveness:
 * add a new record to 'permits' and every switch without it stops compiling.
 */
public final class ModernPlan {
    private ModernPlan() {}

    public sealed interface Infra permits Vm, Sql, Lb {}
    public record Vm(String name, int vcpus, boolean publicIp) implements Infra {}
    public record Sql(String name, int storageGb, boolean encrypted) implements Infra {}
    public record Lb(String name, int rules) implements Infra {}

    /** Operation 1 as a plain function. No 'default' branch, on purpose. Same prices as MonthlyCost. */
    public static BigDecimal monthlyCost(Infra r) {
        return switch (r) {
            case Vm vm -> MonthlyCost.PER_VCPU.multiply(BigDecimal.valueOf(vm.vcpus()));
            case Sql db -> MonthlyCost.DB_BASE.add(MonthlyCost.PER_GB.multiply(BigDecimal.valueOf(db.storageGb())));
            case Lb lb -> MonthlyCost.LB_BASE.add(MonthlyCost.PER_RULE.multiply(BigDecimal.valueOf(lb.rules())));
        };
    }

    /** Operation 2 with record deconstruction patterns and guards. */
    public static Optional<String> audit(Infra r) {
        return switch (r) {
            case Vm(String n, int cpus, boolean pub) when pub -> Optional.of(n + ": server has a public IP");
            case Sql(String n, int gb, boolean enc) when !enc -> Optional.of(n + ": storage not encrypted");
            case Lb(String n, int rules) when rules == 0 -> Optional.of(n + ": load balancer has no rules");
            case Vm vm -> Optional.empty();     // no finding
            case Sql db -> Optional.empty();
            case Lb lb -> Optional.empty();
        };
    }
}
```

```java
// File: code/lld/ch11/visitor/Main.java
package lld.ch11.visitor;

import java.math.BigDecimal;
import java.util.List;
import java.util.Optional;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== 1. Classic Visitor (double dispatch) over a composite plan ===");
        Resource plan = new ResourceGroup("prod", List.of(
            new Server("web-1", 4, true),
            new Server("worker-1", 2, false),
            new Database("orders-db", 200, false),
            new LoadBalancer("edge-lb", 4),
            new ResourceGroup("staging", List.of())));

        MonthlyCost cost = new MonthlyCost();
        plan.accept(cost);
        System.out.println("  monthly cost: " + cost.total());

        SecurityAudit audit = new SecurityAudit();
        plan.accept(audit);
        audit.findings().forEach(f -> System.out.println("  finding: " + f));

        System.out.println("\n=== 2. Java 21: sealed interface + pattern matching switch ===");
        List<ModernPlan.Infra> infra = List.of(
            new ModernPlan.Vm("web-1", 4, true), new ModernPlan.Vm("worker-1", 2, false),
            new ModernPlan.Sql("orders-db", 200, false), new ModernPlan.Lb("edge-lb", 4));
        BigDecimal total = infra.stream().map(ModernPlan::monthlyCost).reduce(BigDecimal.ZERO, BigDecimal::add);
        System.out.println("  monthly cost: " + total);
        infra.stream().map(ModernPlan::audit).flatMap(Optional::stream)
             .forEach(f -> System.out.println("  finding: " + f));
    }
}
```

Output:

```
=== 1. Classic Visitor (double dispatch) over a composite plan ===
  monthly cost: 258.00
  finding: web-1: server has a public IP
  finding: orders-db: storage not encrypted
  finding: staging: empty group

=== 2. Java 21: sealed interface + pattern matching switch ===
  monthly cost: 258.00
  finding: web-1: server has a public IP
  finding: orders-db: storage not encrypted
```

Both styles give the same answer (cost 96.00 + 48.00 + 90.00 + 24.00 = 258.00; the empty `staging` group is free). The modern one needs no `accept`, no visitor interface, and each operation is just a function.

The one question that decides it: the expression problem

	Add a new operation	**Add a new type**
Methods on the classes (plain OO)	Edit every class	One new class
Visitor (or sealed + switch)	One new visitor/function	Edit the visitor interface **and every visitor** (compiler finds them)

So ask: **which changes more often here — the types or the operations?** Settled types + growing operations → Visitor (or sealed types with pattern matching). Growing types → put the operations on the classes. No ordinary OO design makes both cheap; this trade-off is called the **expression problem**.

Open/Closed trade-off: Visitor makes the element hierarchy **closed for modification but open for new operations** — at the price of being **closed to new element types** (each one forces edits to all visitors).

Sealed interfaces + pattern matching (Java 17/21) as the modern alternative: sealed classes arrived in Java 17 (JEP 409 — a JEP, JDK Enhancement Proposal, is the numbered design document for a Java feature); pattern

matching for `switch` and record patterns were finalised in Java 21 (JEPs 441 and 440). Together they give the exhaustiveness check that was Visitor's main advantage, with far less code. The exact rules:

- A `switch` **expression** must always be exhaustive. A `switch` **statement** must be exhaustive too as soon as it uses patterns or `case null` (old-style statements over constants need not be).
- Over a `sealed` type, covering every permitted subtype with an **unguarded** pattern makes the switch exhaustive with no `default`. Guarded cases (`when ...`) do not count.
- Over a type that is **not** sealed, the compiler cannot know all subtypes, so you need a `default` (or a total pattern such as `case Infra i`) — and then the check is gone.
- If a new subtype appears at run time without recompiling the switch (separate compilation), the compiler-generated fallback throws `MatchException`.
- **Dominance:** cases are tried top to bottom, and a case that can never match because an earlier one already covers it is a compile error. That is why the guarded `case Vm(...) when pub` must come **before** the unguarded `case Vm vm`, never after.
- **`null`:** a pattern `switch` without a `case null` throws `NullPointerException` when the selector is `null`; `default` alone does not match `null`.

Write a classic Visitor when the element hierarchy **is not and cannot be sealed** — typically because the element classes **belong to another team or library** and you cannot edit them — but already exposes `accept` (typical of libraries such as ASM — a library for reading and rewriting Java **bytecode**, the compiled instructions the JVM (Java Virtual Machine) runs — or javac's tree API, which ship their own visitor interfaces), when you need a **walk with per-node callbacks** over a big tree, or when each operation should be a separately testable, injectable class.

Visitor outside Java. Kotlin: a sealed class with an exhaustive `when`. Python: `match` with class patterns, or `functools.singledispatch` (dispatch on the argument type with no `accept`). C++: `std::variant` + `std::visit` — the pattern as a library feature. Rust and Swift: enums with associated values plus `match`/`switch`, where a missing case is a build error. Languages without overloading (Python, JavaScript) name the methods `visitServer`, `visitDatabase`, ... instead of overloading `visit`.

Trade-offs

Pros	Cons
Adding an operation = writing one class; elements stay as they are.	Adding an element type forces a change in every existing visitor.
Related logic stays together (all pricing in one file).	Double dispatch is indirect; `accept` tells you nothing about what runs.
The compiler reports any element type a visitor has not handled.	Elements often must expose their data (weaker encapsulation).
Elements stay small: data + one-line `accept`.	Ceremony for small sets of types.

When not to use it

- **New types arrive often** — every one edits every visitor.
- **A tiny model** (say three element types and a single operation) — a type check or a plain method is simpler.
- **The operation concerns one type only** — it's a method.
- **Your language checks switch exhaustiveness** (Java 21 sealed types, Kotlin `when`, Rust `match`) — use that.

Common mistakes

1. Reaching for Visitor with two element types.
2. Making element fields **public** for visitors (use accessors/records).
3. **Reusing a stateful visitor** (like `MonthlyCost`) across runs without resetting — create a new one per run.
4. Adding a `default` branch to a sealed-type switch — it **turns off** the exhaustiveness check you wanted.
5. Forgetting that the composite's `accept` must forward to children (or deciding where traversal lives: element, visitor, or an Iterator).
6. **Writing a visitor where a sealed-type `switch` would do.** If your language checks exhaustiveness (Java 21, Kotlin, Rust, Swift), that gives the same safety with a fraction of the code.

Real-world examples

- **Compilers and linters** (tools that flag suspicious code): an **AST** (abstract syntax tree — the tree a compiler builds from source code) has a fixed set of node kinds and endless passes (type check, optimise, format, lint rules). The JDK's `com.sun.source.tree.TreeVisitor` (javac tree API) and `javax.lang.model.element.ElementVisitor` (annotation processing) are visitors; ANTLR (a parser generator) generates visitor classes for your grammar.
- **Bytecode libraries**: ASM's `ClassVisitor` / `MethodVisitor`.
- **`java.nio.file.FileVisitor`** with `Files.walkFileTree` — named after the pattern and often cited, but strictly it is a **tree-walk callback** interface (`preVisitDirectory`, `visitFile`, `visitFileFailed`, `postVisitDirectory`): there is no `accept` and no double dispatch over element classes. The *spirit* (operation kept outside the file-system objects) is Visitor's; the mechanism is closer to a Template Method walk calling your hooks.
- **Document models**: a **DOM** (Document Object Model — the tree of elements that makes up a web page or XML document) has a fixed set of node kinds and ever more operations — serialise to HTML/PDF, pull out the plain text, collect all links; **infrastructure tools** computing cost, policy and diagrams over resource plans (this lesson's example); **query planners** (the database component that decides how to run a SQL query: cost estimation, rewriting, EXPLAIN).

In the interview

> **30-second answer:** "Visitor pulls an operation out of the element classes and into a separate class that has a method for each element type. Each element has a one-line `accept` that calls `visitor.visit(this)`, which gives double dispatch: the method chosen depends on both the element type and the visitor type, so no `instanceof` chains. It makes adding operations cheap and adding types expensive — the expression problem — so it fits stable type sets like ASTs. In Java 21 I'd usually use a sealed interface with an exhaustive pattern-matching switch instead, which gives the same compiler check with much less code."

Follow-ups:

- *Explain double dispatch.* Java picks overloads at compile time from static types; `accept` runs on the element's run-time type, and inside it `this` has the concrete static type, so the right `visit` overload is chosen; then the visitor's run-time type picks the implementation.
- *Why not just overload `visit` and call `visitor.visit(resource)`?* With a `Resource` static type, the compiler can't choose `visit(Server)`; it needs `visit(Resource)`, which doesn't exist.
- *Visitor vs Iterator?* Iterator decides the route through the structure; Visitor decides the work done at each stop. Real code often combines them.
- *Visitor vs Strategy?* A strategy is a single algorithm that ignores element types; a visitor is a set of methods, one for each element type, and exists precisely to dispatch on type.
- *Visitor vs Composite?* Composite lets a client treat a whole tree and a single node alike; Visitor adds new operations over a stable set of types, whether or not they form a tree. The two frequently appear together.
- *Visitor vs Interpreter?* Interpreter keeps evaluation **in** each node class; Visitor moves it **out**. Many interpreters begin in the first style and switch to the second once another pass (type checking, pretty-printing) is needed.
- *How do you add a new element type?* Add it to the interface's `visit` list (or the `permits` clause) and fix every visitor/switch the compiler flags.

Self-check

1. **Q:** What is the body of every `accept` method? **A:** `visitor.visit(this)`.
2. **Q:** What change is cheap with Visitor, and what is expensive? **A:** New operations are cheap; new element types are expensive.
3. **Q:** What Java 21 features replace most visitors? **A:** Sealed interfaces + pattern-matching `switch` (with record patterns), checked for exhaustiveness.
4. **Q:** Why no `default` in `monthlyCost`'s switch? **A:** A `default` would hide missing cases; without it the compiler forces every permitted type to be handled.
5. **Q:** Where does the traversal live for a composite? **A:** Here, in the composite's `accept` (it visits itself, then its children).

TL;DR

Operations become visitor classes; elements just `accept`; double dispatch picks the method. Cheap new operations, expensive new types (expression problem). In Java 21, prefer **sealed interfaces + exhaustive**

pattern matching.

11.6 GoF behavioral patterns: quick-reference summary

A single table with the key features, when to apply each pattern, an example, and pros and cons (compiled from the course summary plus this book's material).

Pattern	Distinctive feature	Apply when	Example	Pros	Cons
Chain of Responsibility	Passes a request along a chain of handlers	Several objects may handle a request and the handler isn't known in advance	Help systems, HTTP middleware (code run before/after each web request)	Decouples sender and receiver	A request may go unhandled; long chains can be inefficient and slow to debug
Command	Wraps a request as an object	Actions must be queued, logged, retried or undone	GUI actions, job queues	Decouples invoker from receiver; undo/redo	Many small command classes
Interpreter	One class per grammar rule	Evaluating sentences of a small language	Regex, rule DSLs (small domain-specific languages)	Easy to extend the grammar	Unmanageable for large grammars
Iterator	Sequential access without exposing representation	You need a standard way to traverse any collection	Browsing a playlist	Simple client code; hides the structure	Hidden costs; state handling in lazy iterators
Mediator	Centralises communication between peers	Many objects interact in tangled ways	Chat application, air traffic control	Fewer dependencies; reusable components	The mediator can become a god object (one class that knows and does too much)
Memento	Captures and externalises state without breaking encapsulation	Undo, checkpoints, rollback	Editor undo, save games	Preserves encapsulation; atomic restore	Memory-heavy for big/frequent snapshots
Observer	Notifies dependents of state changes	A change in one object must reach others	Live stock/data feeds	Subject and observers are loosely coupled	Leaks, unexpected performance costs, no ordering
State	Behaviour changes with internal state	An object's behaviour depends on a state that changes at run time	Game modes, order lifecycles	Localises state-specific behaviour	More classes; graph spread across files
Strategy	Algorithm chosen at run time	Several ways to do one task, chosen dynamically	Compression algorithms, retry policies	Swap algorithms; isolate each	More classes; someone must choose
Template Method	Algorithm skeleton in a base class, steps in subclasses	The structure is fixed but some steps vary	Frameworks' lifecycles	Code reuse; enforced order	Inheritance rigidity; harder to follow
Visitor	Separates operations from the objects they work on	Many operations over a stable set of types	Processing document elements, compiler passes	Easy to add operations; related code together	Adding element types is hard; bloated visitor interface

---

11.7 Beyond GoF: patterns every LLD interview expects

The GoF book (1994) is not the whole story. The patterns below come from later sources — *Patterns of Enterprise Application Architecture* (Fowler, 2002), *Domain-Driven Design* (Evans, 2003 — DDD, an approach that shapes code around the business domain and its vocabulary), *Head First Design Patterns*, and decades of framework practice. You will use them in almost every case study in this book.

11.7.1 Null Object

What it is

A **Null Object** is a real object that implements an interface with **neutral, do-nothing behaviour**, used instead of `null` to mean "no object here". Callers can call it safely without `if (x != null)` checks.

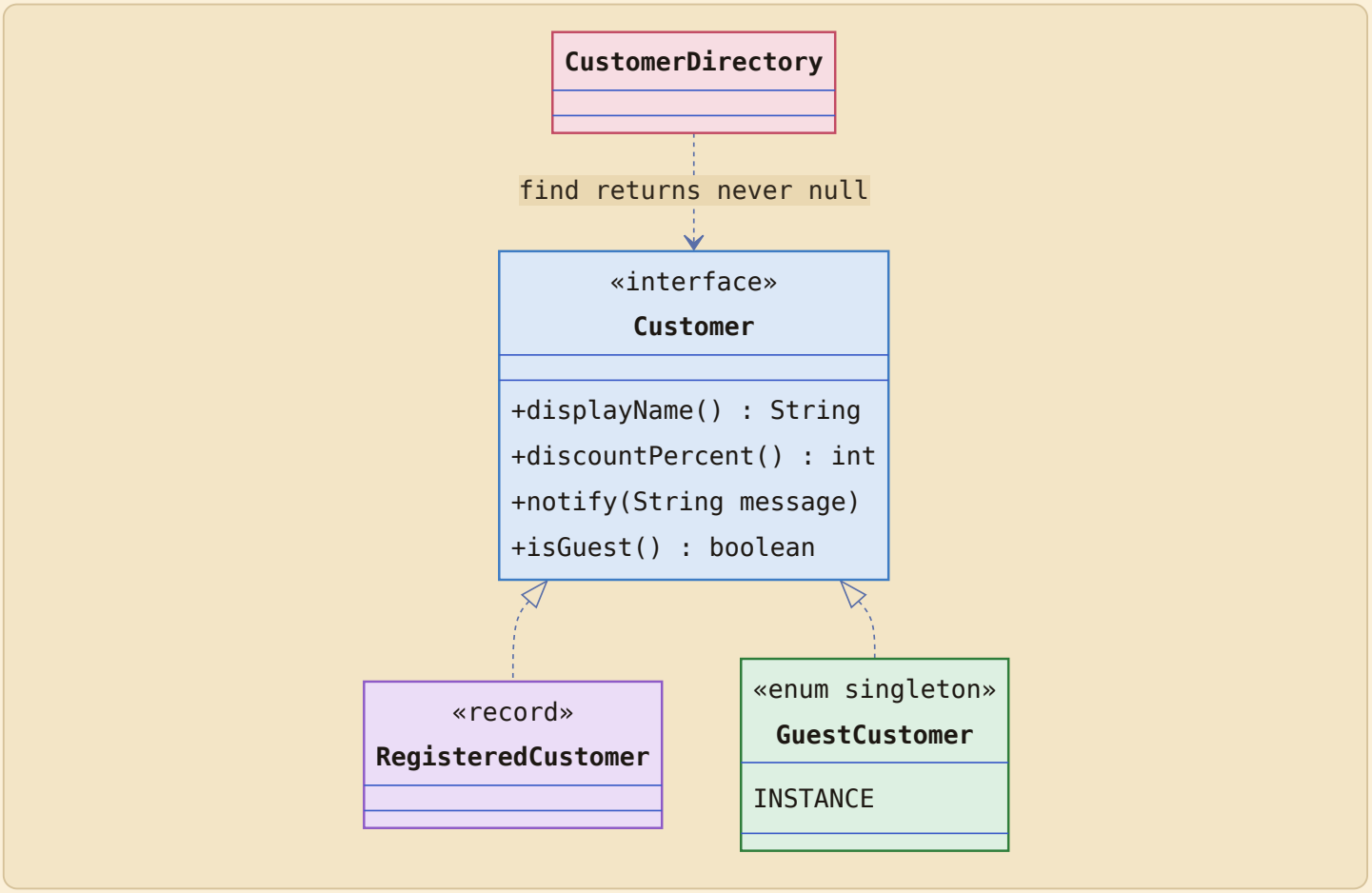
Analogy: a "guest" badge at an office. Guests can walk through the lobby like anyone else; the badge simply opens no special doors.

The problem it solves

Returning `null` for "not found" forces **every caller** to remember a null check. One forgotten check → `NullPointerException` (NPE — the error Java throws when you call a method on `null`) in production. The checks also clutter the business logic.

How it works

1. Define an interface (`Customer`).
2. Implement the real class (`RegisteredCustomer`) and a **null class** (`GuestCustomer`) whose methods return neutral values (0% discount) or do nothing (`notify` is empty).
3. Where you would return `null`, return the null object instead. Stateless null objects can be a **singleton** (a class with exactly one shared instance — here an `enum` constant).
4. If some caller really must know, offer an `isGuest()`/`isNull()` query — sparingly.



Java example

```java
// File: code/lld/ch11/nullobject/Customer.java
package lld.ch11.nullobject;

/** The abstraction both the real object and the Null Object implement. */
public interface Customer {
    String displayName();
    int discountPercent();
    void notify(String message);
    default boolean isGuest() { return false; }
}
```

```java
// File: code/lld/ch11/nullobject/RegisteredCustomer.java
package lld.ch11.nullobject;

public record RegisteredCustomer(String name, int discountPercent, String email) implements Customer {
    public String displayName() { return name; }
    public void notify(String message) { System.out.println("    email to " + email + ": " + message); }
}
```

```java
// File: code/lld/ch11/nullobject/GuestCustomer.java
package lld.ch11.nullobject;

/**
 * NULL OBJECT: a real, do-nothing implementation with neutral values.
 * Stateless, so one shared instance is enough (an enum singleton).
 */
public enum GuestCustomer implements Customer {
    INSTANCE;

    public String displayName() { return "guest"; }
    public int discountPercent() { return 0; }            // neutral value
    public void notify(String message) { }                // does nothing, on purpose
    @Override public boolean isGuest() { return true; }   // for the rare caller that must know
}
```

```java
// File: code/lld/ch11/nullobject/CustomerDirectory.java
package lld.ch11.nullobject;

import java.util.Map;
import java.util.Optional;

public final class CustomerDirectory {
    private final Map<String, Customer> byId = Map.of(
        "c1", new RegisteredCustomer("Asha", 10, "asha@example.com"));

    /** Style 1: never return null; return the Null Object instead. */
    public Customer find(String id) { return byId.getOrDefault(id, GuestCustomer.INSTANCE); }

    /** Style 2: make absence explicit in the type; the caller chooses the fallback. */
    public Optional<Customer> lookup(String id) { return Optional.ofNullable(byId.get(id)); }
}
```

```java
// File: code/lld/ch11/nullobject/Main.java
package lld.ch11.nullobject;

import java.math.BigDecimal;
import java.math.RoundingMode;

public class Main {
    private static final BigDecimal HUNDRED = BigDecimal.valueOf(100);

    static void checkout(Customer c, BigDecimal price) {
        // No "if (customer != null)" anywhere: the Null Object answers every call safely.
        BigDecimal payable = price.multiply(BigDecimal.valueOf(100 - c.discountPercent()))
                                  .divide(HUNDRED, 2, RoundingMode.HALF_EVEN);
        System.out.printf("  %-6s pays %s%n", c.displayName(), payable);
        c.notify("thanks for your order");
    }

    public static void main(String[] args) {
        CustomerDirectory dir = new CustomerDirectory();
        System.out.println("=== Null Object ===");
        checkout(dir.find("c1"), new BigDecimal("100.00"));
        checkout(dir.find("unknown-id"), new BigDecimal("100.00"));

        System.out.println("=== Optional (absence visible in the signature) ===");
        Customer c = dir.lookup("unknown-id").orElse(GuestCustomer.INSTANCE);
        System.out.println("  lookup(unknown-id) present? " + dir.lookup("unknown-id").isPresent() + " -> using " +
c.displayName());
    }
}
```

Output:

```
=== Null Object ===
  Asha   pays 90.00
    email to asha@example.com: thanks for your order
  guest  pays 100.00
=== Optional (absence visible in the signature) ===
  lookup(unknown-id) present? false -> using guest
```

`checkout` has no null checks, and the guest's `notify` silently does nothing. (Prices are `BigDecimal` with an explicit rounding mode, never `double`.)

Trade-offs, when not to use it, mistakes

Pros	Cons
Removes repetitive null checks; fewer NPEs.	Can **hide real errors** — a missing object that *should* exist fails silently.
Polymorphism instead of conditionals.	One more class per interface.
Easy to test; a natural default.	Callers may not realise they got a "nothing" object.

- **Don't use it** when absence is an **error** (throw an exception) or when the caller must decide what absence means (return `Optional<T>` — a JDK container that either holds a value or is empty).
- **Null Object vs `Optional`**: Null Object hides absence behind neutral behaviour; `Optional` makes absence **visible in the type** and lets each caller choose (`orElse`, `orElseThrow`). Use `Optional` for return values of lookups; use Null Object for **collaborators** that are legitimately optional (a logger, a listener, a discount policy).
- **Common mistakes:** a null object with **side effects** or non-neutral values; a mutable null-object singleton; overusing `isNull()` checks (which brings the conditionals back).

Real-world examples

`Collections.emptyList()` / `emptyMap()` (empty, not null); `InputStream.nullInputStream()`, `OutputStream.nullOutputStream()`, `Reader.nullReader()`, `Writer.nullWriter()` (Java 11); SLF4J's (a common Java logging API) `NOPLogger`; a "no-op" (does nothing) `MeterRegistry`/tracer in **observability** libraries (metrics, logs and traces that show what a running system is doing).

In the interview

30-second answer: "Null Object replaces `null` with an implementation that does nothing or returns neutral values, so callers never need null checks — like returning an empty list instead of null. I use it for optional collaborators like loggers or discount policies, and `Optional` for lookups where the caller should decide. I avoid it when absence is actually an error, because it would hide the bug."

Follow-up — is it a GoF pattern? No; it was described by Bobby Woolf (in *Pattern Languages of Program Design 3*) and appears in Fowler's *Refactoring* as "Introduce Special Case" (earlier "Introduce Null Object").

Self-check. Q: What does `Collections.emptyList()` have to do with this? A: It's a null object for lists — safe to iterate, never null. Q: When should you throw instead? A: When the object must exist and its absence is a bug.

TL;DR: Return a harmless "nothing" object instead of `null`; use `Optional` when the caller must choose; throw when absence is a bug.

11.7.2 Dependency Injection (DI)

What it is

Dependency Injection means a class **receives** the objects it depends on (its *dependencies*) from outside, instead of **creating** them itself with `new` or looking them up. It is the most common way to implement **Inversion of Control (IoC)** — the framework or a "composition root" builds and wires objects, not the objects themselves. The name was settled by Martin Fowler in his 2004 article on IoC containers.

Analogy: a restaurant kitchen doesn't grow its own vegetables; suppliers deliver them. Change the supplier, and the chef's recipes don't change.

The problem it solves

```
class OrderService {
    private final EmailSender sender = new EmailSender();   // hard-wired
}
```

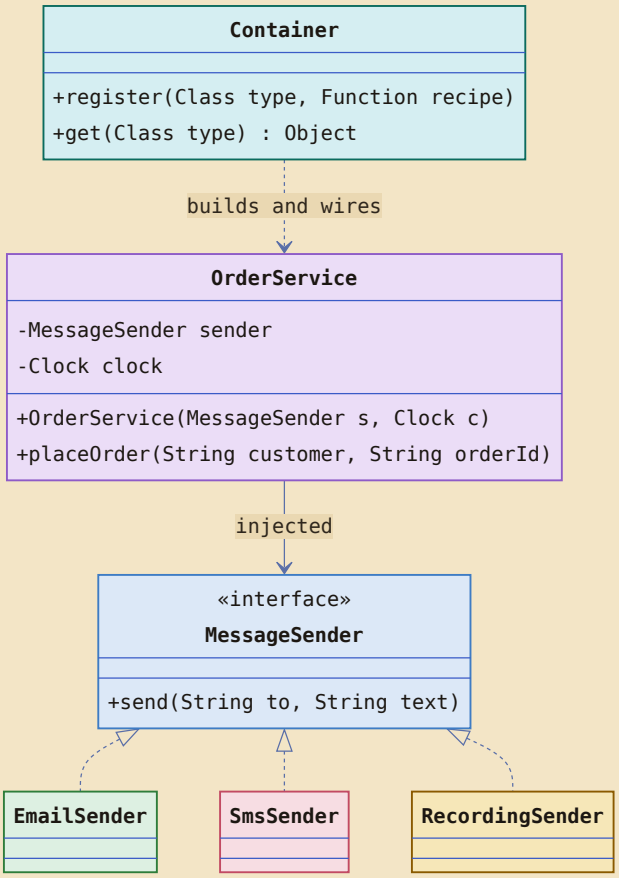
This class can never send SMS, can't be unit-tested without really sending email, and hides its dependencies. DI fixes all three: depend on an **interface** (`MessageSender`), receive it through the **constructor**, and let someone else decide which implementation.

How it works

1. The class declares dependencies as **constructor parameters** typed by **interfaces** (Dependency Inversion Principle, Chapter 4).
2. A **composition root** (usually `main`, or a DI container at start-up) creates the concrete objects and passes them in.
3. Tests pass **fakes** (simple stand-in implementations used only in tests) instead.

Three injection styles:

Style	How	Use for
Constructor injection	Dependencies are constructor parameters, stored in `final` fields	Required dependencies — the default choice
Setter injection	`setX(...)` after construction	Optional dependencies or ones that can change
Field injection	Framework writes directly into fields (`@Autowired` annotation on a field)	Avoid: hides dependencies and makes testing harder



Java example

`OrderService` receives both a `MessageSender` and a `java.time.Clock` — injecting the clock makes time **testable** (`Clock.fixed`). `Container` is a toy IoC container: you register a *recipe* per type; `get` builds objects on demand (recipes can ask the container for their own dependencies) and caches them as singletons.

```java
// File: code/lld/ch11/di/MessageSender.java
package lld.ch11.di;

/** The abstraction the service depends on (Dependency Inversion). */
public interface MessageSender {
    void send(String to, String text);
}
```

```java
// File: code/lld/ch11/di/EmailSender.java
package lld.ch11.di;

public final class EmailSender implements MessageSender {
    public void send(String to, String text) { System.out.println("  [smtp] to " + to + ": " + text); }
}
```

```java
// File: code/lld/ch11/di/SmsSender.java
package lld.ch11.di;

public final class SmsSender implements MessageSender {
    public void send(String to, String text) { System.out.println("  [sms] to " + to + ": " + text); }
}
```

```java
// File: code/lld/ch11/di/RecordingSender.java
package lld.ch11.di;

import java.util.ArrayList;
import java.util.List;

/** Test double: records instead of sending. Only possible because the dependency is injected. */
public final class RecordingSender implements MessageSender {
    public final List<String> sent = new ArrayList<>();
    public void send(String to, String text) { sent.add(to + "|" + text); }
}
```

```java
// File: code/lld/ch11/di/OrderService.java
package lld.ch11.di;

import java.time.Clock;
import java.time.LocalTime;

/**
 * CONSTRUCTOR INJECTION: the service declares what it needs and never calls 'new'
 * on its collaborators. Even the clock is injected, so time is testable.
 */
public final class OrderService {
    private final MessageSender sender;
    private final Clock clock;

    public OrderService(MessageSender sender, Clock clock) {
        this.sender = sender;
        this.clock = clock;
    }

    public void placeOrder(String customer, String orderId) {
        LocalTime now = LocalTime.now(clock);
        sender.send(customer, "order " + orderId + " placed at " + now);
    }
}
```

```java
// File: code/lld/ch11/di/Container.java
package lld.ch11.di;

import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;

/**
 * A toy IoC container: register how to build each type once; ask for the type later.
 * Real containers (Spring, Guice, Dagger) add scanning, scopes, proxies and lifecycles.
 */
public final class Container {
    private final Map<Class<?>, Function<Container, ?>> recipes = new HashMap<>();
    private final Map<Class<?>, Object> singletons = new HashMap<>();

    public <T> void register(Class<T> type, Function<Container, ? extends T> recipe) {
        recipes.put(type, recipe);
    }

    public <T> T get(Class<T> type) {
        Object existing = singletons.get(type);
        if (existing != null) return type.cast(existing);
        Function<Container, ?> recipe = recipes.get(type);
        if (recipe == null) throw new IllegalStateException("no binding for " + type.getSimpleName());
        T created = type.cast(recipe.apply(this));   // the recipe may ask the container for its own deps
        singletons.put(type, created);
        return created;
    }
}
```

```java
// File: code/lld/ch11/di/Main.java
package lld.ch11.di;

import java.time.Clock;
import java.time.Instant;
import java.time.ZoneOffset;

public class Main {
    public static void main(String[] args) {
        Clock fixed = Clock.fixed(Instant.parse("2026-01-15T09:30:00Z"), ZoneOffset.UTC);

        System.out.println("=== 1. Manual DI: the composition root wires everything ===");
        new OrderService(new EmailSender(), fixed).placeOrder("asha@example.com", "A-1");
        new OrderService(new SmsSender(), fixed).placeOrder("+91-90000-00000", "A-2");

        System.out.println("\n=== 2. In a unit test: inject a fake ===");
        RecordingSender fake = new RecordingSender();
        new OrderService(fake, fixed).placeOrder("ben", "T-9");
        System.out.println("  fake recorded: " + fake.sent);

        System.out.println("\n=== 3. A toy IoC container ===");
        Container c = new Container();
        c.register(Clock.class, k -> fixed);
        c.register(MessageSender.class, k -> new EmailSender());
        c.register(OrderService.class, k -> new OrderService(k.get(MessageSender.class), k.get(Clock.class)));
        c.get(OrderService.class).placeOrder("chen@example.com", "A-3");
        System.out.println("  same instance each time: " + (c.get(OrderService.class) == c.get(OrderService.class)));
    }
}
```

Output:

```
=== 1. Manual DI: the composition root wires everything ===
  [smtp] to asha@example.com: order A-1 placed at 09:30
  [sms] to +91-90000-00000: order A-2 placed at 09:30

=== 2. In a unit test: inject a fake ===
  fake recorded: [ben|order T-9 placed at 09:30]

=== 3. A toy IoC container ===
  [smtp] to chen@example.com: order A-3 placed at 09:30
  same instance each time: true
```

Trade-offs, when not to use it, mistakes

Pros	Cons
Swap implementations without editing the class (OCP, DIP).	More indirection: "where does this come from?"
Easy unit tests with fakes.	Containers add "magic" (reflection — code inspecting classes at run time; proxies — generated wrapper objects) and start-up cost.
Dependencies are explicit in the constructor.	Misconfiguration may only show at start-up (or run time).

- **Don't** inject value objects or simple data (`new Money(...)` is fine), or add interfaces with only one implementation "just for DI" when no test or variation needs them.
- **Common mistakes:** the **Service Locator** anti-pattern (classes call `container.get(...)` themselves — dependencies hidden again); field injection; **constructor with 8+ dependencies** (a sign the class does too much — split it); **circular dependencies** (A needs B while B needs A).

Real-world examples

Spring (`@Component` + constructor injection, `ApplicationContext` as the container), **Google Guice**, **Dagger/Hilt** (compile-time DI on Android), **Jakarta CDI**, **Micronaut/Quarkus** (compile-time DI for fast start-up). Spring can even inject a `Map<String, PaymentStrategy>` of all strategy beans — DI + Strategy together.

In the interview

> **30-second answer:** "Dependency injection means a class receives its collaborators through its constructor, typed by interfaces, instead of creating them. It's how we apply Dependency Inversion and Inversion of Control: a composition root or a container like Spring wires the graph. The benefits are swappable implementations and easy testing with fakes — I even inject the Clock. I prefer constructor injection, avoid field injection and service locators, and treat a huge constructor as a **design smell** (a surface sign that something deeper is wrong)."

Follow-ups: **DI vs IoC?** IoC is the general idea (the framework calls you / controls object creation); DI is one way to achieve it. **DI vs Service Locator?** With a locator the class asks for dependencies (hidden); with DI they're handed in (visible). **DI vs Factory?** A factory creates objects on request; DI supplies ready-made dependencies — containers use factories internally.

Self-check. Q: Why inject `Clock`? A: To make time-dependent code deterministic in tests. Q: Which injection style is preferred? A: Constructor injection.

> **TL;DR:** Don't `new` your collaborators — **receive them** (constructor, interfaces). Wire at the composition root or with a container.

11.7.3 Repository

What it is

A **Repository** is a **collection-like interface for loading and saving domain objects**, written in the language of the domain (`findByEmail`, `findActive`), that **hides how and where data is stored** (SQL — relational databases queried with the SQL language; NoSQL — non-relational stores such as key-value or document databases; files; memory). Described by Fowler (*PoEAA*) and central to Evans's *Domain-Driven Design*.

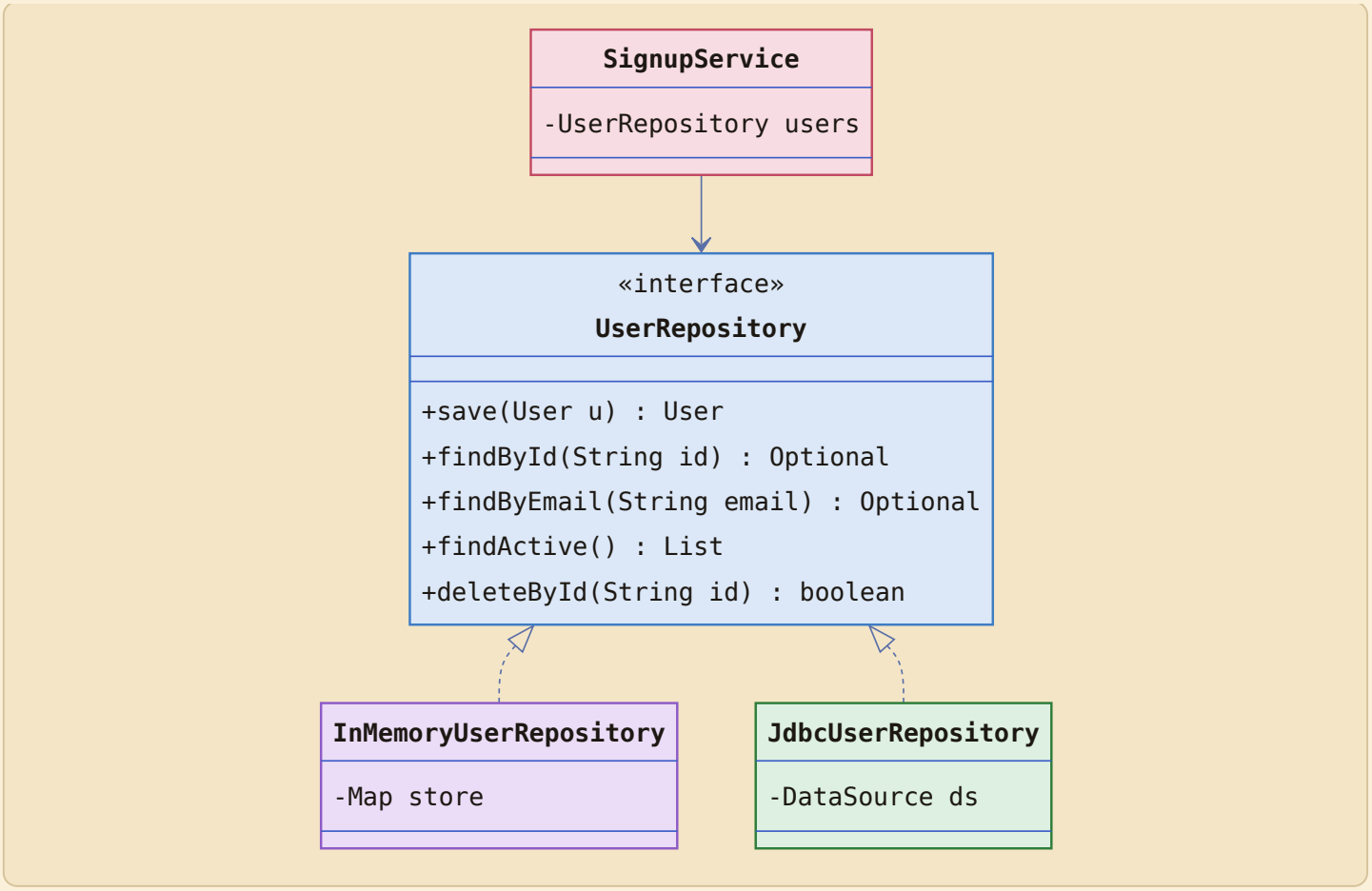
Analogy: a library's front desk. You ask for "books by this author"; you don't need to know which shelf, basement or branch they come from.

The problem it solves

When SQL, connection handling and mapping code are scattered through business classes, the business rules become hard to read, impossible to unit-test without a database, and tied to one storage technology.

How it works

1. Define the interface in the **domain layer** (the code that holds business rules) (`UserRepository`: `save`, `findById`, `findByEmail`, ...). Return `Optional` for "maybe not found".
2. Implement it in the **infrastructure layer** (code that talks to databases, networks and files): `InMemoryUserRepository` for tests, a JDBC/JPA one (JPA, the Java Persistence API, maps objects to tables) for production.
3. Services depend on the **interface** (injected — see DI).



(`JdbcUserRepository` is shown to illustrate a production implementation; the demo code includes the in-memory one.)

Java example

```java
// File: code/lld/ch11/repository/User.java
package lld.ch11.repository;

/** A domain object. It knows nothing about storage. */
public record User(String id, String email, boolean active) {}
```

```java
// File: code/lld/ch11/repository/UserRepository.java
package lld.ch11.repository;

import java.util.List;
import java.util.Optional;

/**
 * REPOSITORY: a collection-like interface for aggregates, in DOMAIN language.
 * The domain depends on this; the storage details live in implementations.
 */
public interface UserRepository {
    User save(User user);
    Optional<User> findById(String id);
    Optional<User> findByEmail(String email);
    List<User> findActive();
    boolean deleteById(String id);
}
```

```java
// File: code/lld/ch11/repository/InMemoryUserRepository.java
package lld.ch11.repository;

import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

/** One implementation: in memory (tests, prototypes). A JDBC/JPA one would implement the same interface. */
public final class InMemoryUserRepository implements UserRepository {
    private final Map<String, User> store = new ConcurrentHashMap<>();

    public User save(User user) { store.put(user.id(), user); return user; }
    public Optional<User> findById(String id) { return Optional.ofNullable(store.get(id)); }

    public Optional<User> findByEmail(String email) {
        return store.values().stream().filter(u -> u.email().equalsIgnoreCase(email)).findFirst();
    }

    public List<User> findActive() {
        return store.values().stream().filter(User::active)
                    .sorted(Comparator.comparing(User::id)).toList();
    }

    public boolean deleteById(String id) { return store.remove(id) != null; }
}
```

```java
// File: code/lld/ch11/repository/SignupService.java
package lld.ch11.repository;

/** Business logic talks to the repository interface only: no SQL, no maps, no files. */
public final class SignupService {
    private final UserRepository users;

    public SignupService(UserRepository users) { this.users = users; }

    public String signUp(String id, String email) {
        if (users.findByEmail(email).isPresent()) return "rejected: " + email + " already registered";
        users.save(new User(id, email, true));
        return "created " + id;
    }

    public void deactivate(String id) {
        users.findById(id).ifPresent(u -> users.save(new User(u.id(), u.email(), false)));
    }
}
```

```java
// File: code/lld/ch11/repository/Main.java
package lld.ch11.repository;

public class Main {
    public static void main(String[] args) {
        UserRepository repo = new InMemoryUserRepository();   // swap for a JDBC version in production
        SignupService service = new SignupService(repo);
        System.out.println("  " + service.signUp("u1", "asha@example.com"));
        System.out.println("  " + service.signUp("u2", "ben@example.com"));
        System.out.println("  " + service.signUp("u3", "ASHA@example.com"));
        service.deactivate("u2");
        System.out.println("  active users: " + repo.findActive());
        System.out.println("  findById(u9): " + repo.findById("u9"));
        System.out.println("  deleteById(u2): " + repo.deleteById("u2"));
    }
}
```

Output:

```
created u1
created u2
rejected: ASHA@example.com already registered
active users: [User[id=u1, email=asha@example.com, active=true]]
findById(u9): Optional.empty
deleteById(u2): true
```

Trade-offs, when not to use it, mistakes

Pros	Cons
Business logic is storage-agnostic and testable in memory.	Another layer; trivial CRUD (create, read, update, delete) apps may not need it.
One place for queries per aggregate.	Can leak storage concerns (lazy loading — fetching data only when first touched; transactions — changes that succeed or fail together) anyway.
Swap storage (SQL → NoSQL) behind the interface.	"Generic repository for everything" becomes a thin, useless wrapper.

- **Don't** create a repository per table; create one per **aggregate root** (DDD: the entry point object of a cluster, e.g. `Order` with its `OrderLine`s).
- **Repository vs DAO:** a DAO (Data Access Object) is table/record-oriented and closer to the database; a repository is domain-oriented and behaves like a collection of aggregates. In small apps they often look the same.
- **Common mistakes:** returning `null` instead of `Optional`; exposing SQL types (`ResultSet`) in the interface; putting business rules in the repository; dozens of `findByXAndYOrZ` methods — use **Specification** (next).
- **Thread-safety note:** `InMemoryUserRepository` uses a `ConcurrentHashMap` (the JDK's thread-safe hash map), so each single call is safe, but `SignupService.signUp` is **check-then-act** (a race where the answer to the check can be stale by the time you act on it) (`findByEmail`, then `save`): two concurrent sign-ups with the same email can both pass the check. A real system enforces uniqueness with a database **unique constraint** (a rule that makes the database reject a duplicate value in a column) (or an atomic `putIfAbsent` keyed by email in memory).

Real-world examples

Spring Data `CrudRepository` / `JpaRepository` (derived query methods like `findByEmail` are generated from method names), Micronaut Data, Hibernate/JPA behind them; Android Room DAOs wrapped in repositories (Android architecture guidance).

In the interview

30-second answer: "A repository is a collection-like interface for aggregates in domain language — save, findById, findByEmail — that hides storage. Services depend on the interface, so business logic is testable with an in-memory implementation and storage can change. In machine-coding rounds I always put an in-memory repository behind an interface, so the design is ready for a database later."

Self-check. Q: What does a repository return when nothing is found? A: `Optional.empty()` (or an empty list). Q: One repository per table? A: No, per aggregate root.

TL;DR: A **domain-language collection interface** over storage; services use the interface; start in-memory, swap later.

11.7.4 Specification

What it is

A **Specification** is a **business rule packaged as an object** with one question — `isSatisfiedBy(candidate)` — that can be **combined** with `and`, `or` and `not`. Introduced by Eric Evans and Martin Fowler.

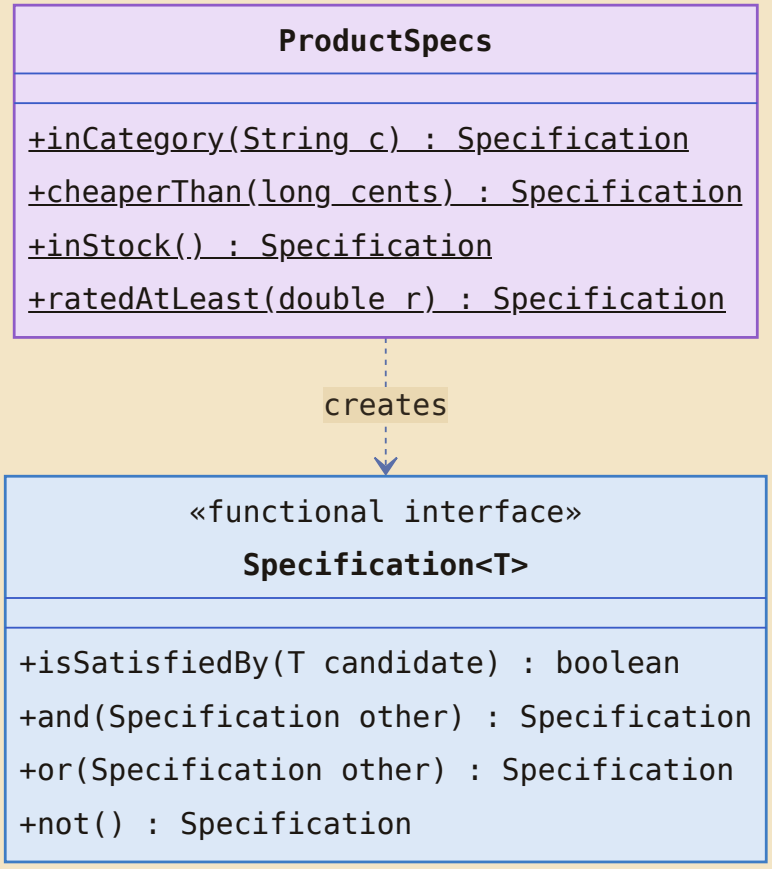
Analogy: the filter panel on a shopping site. "Electronics" ✓, "under ₹5,000" ✓, "in stock" ✓ — each is a separate rule, and the site combines the ones you tick.

The problem it solves

Rules like "bargain electronics = category electronics AND price < 50.00 AND in stock" end up duplicated in queries, validations and UI filters, or as an explosion of repository methods (`findByCategoryAndPriceLessThanAndStockGreaterThan...`). Specification gives each rule a **name**, makes it **reusable** in several places (selection, validation, building to order), and **composable**.

How it works

1. `Specification<T>` with `isSatisfiedBy(T)`; default methods `and`, `or`, `not` return new composite specifications (this is the **Composite** pattern + **Interpreter**-like evaluation).
2. Write small named factories (`inStock()`, `cheaperThan(...)`).
3. Combine them where needed; pass them to a repository or a stream filter.



Java example

```java
// File: code/lld/ch11/specification/Specification.java
package lld.ch11.specification;

/**
 * SPECIFICATION: a business rule as a small, named, combinable object.
 * isSatisfiedBy answers "does this candidate meet the rule?"
 */
@FunctionalInterface
public interface Specification<T> {
    boolean isSatisfiedBy(T candidate);

    default Specification<T> and(Specification<T> other) { return c -> isSatisfiedBy(c) && other.isSatisfiedBy(c); }
    default Specification<T> or(Specification<T> other)  { return c -> isSatisfiedBy(c) || other.isSatisfiedBy(c); }
    default Specification<T> not()                       { return c -> !isSatisfiedBy(c); }
}
```

```java
// File: code/lld/ch11/specification/Product.java
package lld.ch11.specification;

public record Product(String name, String category, long priceCents, int stock, double rating) {}
```

```java
// File: code/lld/ch11/specification/ProductSpecs.java
package lld.ch11.specification;

/** Named rules, in the language of the business. Each is tiny and testable alone. */
public final class ProductSpecs {
    private ProductSpecs() {}

    public static Specification<Product> inCategory(String c) { return p -> p.category().equals(c); }
    public static Specification<Product> cheaperThan(long cents) { return p -> p.priceCents() < cents; }
    public static Specification<Product> inStock() { return p -> p.stock() > 0; }
    public static Specification<Product> ratedAtLeast(double r) { return p -> p.rating() >= r; }
}
```

```java
// File: code/lld/ch11/specification/Main.java
package lld.ch11.specification;

import static lld.ch11.specification.ProductSpecs.*;

import java.util.List;

public class Main {
    static List<String> select(List<Product> catalog, Specification<Product> spec) {
        return catalog.stream().filter(spec::isSatisfiedBy).map(Product::name).toList();
    }

    public static void main(String[] args) {
        List<Product> catalog = List.of(
            new Product("usb-cable", "electronics", 499, 120, 4.1),
            new Product("headphones", "electronics", 7_999, 0, 4.6),
            new Product("keyboard", "electronics", 4_999, 8, 4.7),
            new Product("novel", "books", 1_299, 30, 4.8),
            new Product("cookbook", "books", 2_499, 0, 3.9));

        // The same rule objects are reused for querying AND for validation.
        Specification<Product> bargainElectronics = inCategory("electronics").and(cheaperThan(5_000)).and(inStock());
        Specification<Product> recommendable = ratedAtLeast(4.5).and(inStock());
        Specification<Product> needsRestock = inStock().not();

        System.out.println("  bargain electronics: " + select(catalog, bargainElectronics));
        System.out.println("  recommendable:       " + select(catalog, recommendable));
        System.out.println("  needs restock:       " + select(catalog, needsRestock));
        System.out.println("  books or top-rated:  " + select(catalog, inCategory("books").or(ratedAtLeast(4.7))));
        System.out.println("  is 'novel' recommendable? " + recommendable.isSatisfiedBy(catalog.get(3)));
    }
}
```

Output:

```
  bargain electronics: [usb-cable, keyboard]
  recommendable:       [keyboard, novel]
  needs restock:       [headphones, cookbook]
  books or top-rated:  [keyboard, novel, cookbook]
  is 'novel' recommendable? true
```

Trade-offs, when not to use it, mistakes

Pros	Cons
Rules are named, reusable and testable one by one.	In-memory specs don't translate to SQL automatically — you need a query-building version (e.g. JPA Criteria) for large data.
Combining rules replaces a method explosion.	Overkill for one or two fixed filters.
The same rule serves querying and validation.	Deeply nested combinations can be hard to debug.

- Java's own `Predicate<T>` (the JDK interface for a yes/no test on a value, with `and`, `or`, `negate`) is already a specification; a named `Specification` type adds domain meaning and a place for SQL translation.
- **Common mistake:** filtering millions of rows in memory — push specifications down to the database.

Real-world examples

Spring Data JPA `Specification<T>` (`where(...).and(...)`, `JpaSpecificationExecutor.findAll(spec)`), e-commerce search filters, eligibility rules (loan, coupon, delivery-slot eligibility), access-control policies.

In the interview

> **30-second answer:** "A specification is a named business rule object with `isSatisfiedBy`, combinable with and/or/not. It removes duplicated conditions and the explosion of repository finder methods, and the same rule can filter a query and validate an object. In Java, `Predicate` is the building block; Spring Data JPA has a `Specification` that translates to SQL."

Self-check. Q: What three combinators does a specification usually have? A: `and`, `or`, `not`. Q: Which pattern does combining specifications resemble? A: Composite (and Interpreter for evaluation).

> **TL;DR: Rules as small combinable objects**; reuse them for querying and validation.

11.7.5 MVC — Model–View–Controller (the Head First "compound pattern")

What it is

MVC splits an interactive application into three roles:

- **Model** — the data and business rules. Knows nothing about screens.
- **View** — displays the model and receives user input.
- **Controller** — interprets user input and tells the model what to change.

It was created by Trygve Reenskaug for Smalltalk at Xerox PARC in the late 1970s. *Head First Design Patterns* calls MVC a **compound pattern** — several patterns working together:

Pattern inside MVC	Where
Observer	The model notifies views (and sometimes controllers) when its state changes; views re-render.
Strategy	The view delegates "what should this input do?" to a controller object; swap the controller to change behaviour (e.g. read-only mode).
Composite	The view is a tree of nested components (panels containing labels, lists, buttons) rendered uniformly.

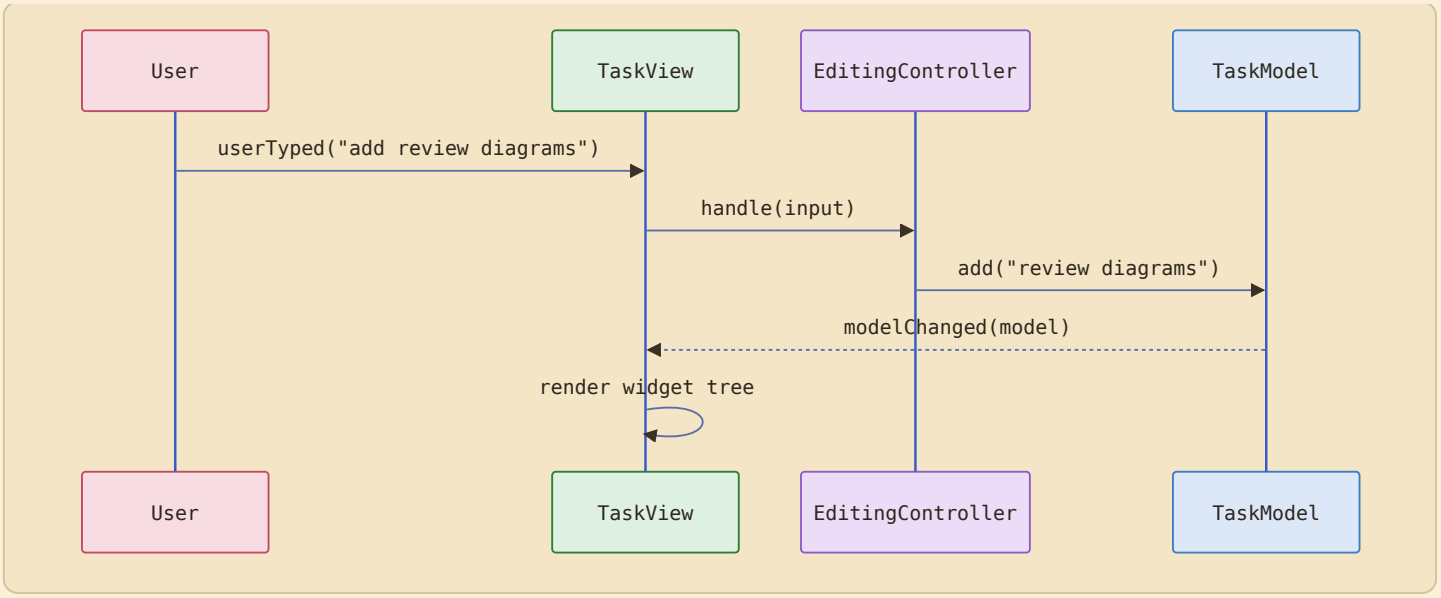
Analogy: a restaurant. The kitchen and pantry are the **model**; the dining-room menu and plates are the **view**; the waiter who takes your order and passes it to the kitchen is the **controller**.

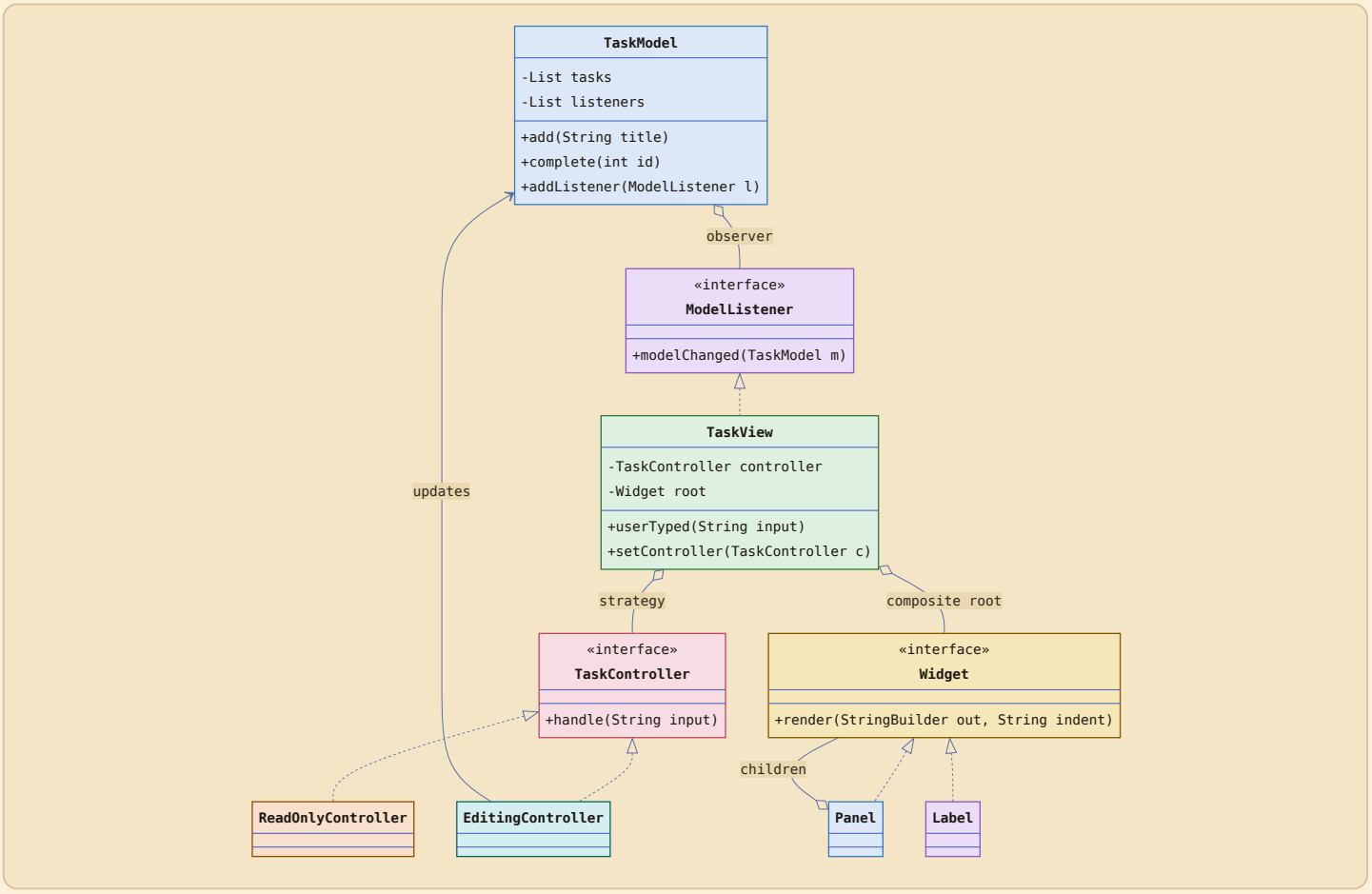
The problem it solves

When UI code, input handling and business rules are mixed, you cannot change the screen without risking the rules, cannot show the same data in two ways, and cannot test the rules without a UI.

How it works

1. The user acts on the **view** (`userTyped("add ...")`).
2. The view hands the input to its **controller** (strategy).
3. The controller calls the **model** (`model.add(title)`); it does **not** redraw anything.
4. The model changes and **notifies its listeners** (observer).
5. The view re-renders its **composite** widget tree from the model.





Java example

A console to-do app. `TaskModel` knows only `ModelListener`s. `TaskView` implements `ModelListener`, renders a **composite** of `Widget.Panel`/`Widget.Label`, and forwards input to a swappable `TaskController`. Switching to `ReadOnlyController` changes behaviour without touching the view or model.

```
// File: code/lld/ch11/mvc/Task.java
package lld.ch11.mvc;

public record Task(int id, String title, boolean done) {}
```

```java
// File: code/lld/ch11/mvc/TaskModel.java
package lld.ch11.mvc;

import java.util.ArrayList;
import java.util.List;

/** MODEL: state + business rules. Knows NOTHING about views or controllers, only listeners. */
public final class TaskModel {
    private final List<Task> tasks = new ArrayList<>();
    private final List<ModelListener> listeners = new ArrayList<>();
    private int nextId = 1;

    public void addListener(ModelListener l) { listeners.add(l); }

    public List<Task> tasks() { return List.copyOf(tasks); }

    public void add(String title) {
        if (title.isBlank()) throw new IllegalArgumentException("title must not be blank");
        tasks.add(new Task(nextId++, title, false));
        fireChanged();
    }

    public void complete(int id) {
        for (int i = 0; i < tasks.size(); i++) {
            Task t = tasks.get(i);
            if (t.id() == id) {
                tasks.set(i, new Task(t.id(), t.title(), true));
                fireChanged();
                return;
            }
        }
        throw new IllegalArgumentException("no task " + id);
    }

    private void fireChanged() { listeners.forEach(l -> l.modelChanged(this)); }
}
```

```java
// File: code/lld/ch11/mvc/ModelListener.java
package lld.ch11.mvc;

/** OBSERVER interface between model and views. */
public interface ModelListener {
    void modelChanged(TaskModel model);
}
```

```java
// File: code/lld/ch11/mvc/Widget.java
package lld.ch11.mvc;

import java.util.List;
import java.util.function.Supplier;

/** COMPOSITE: the view is a tree of widgets rendered the same way. */
public interface Widget {
    void render(StringBuilder out, String indent);

    record Label(Supplier<String> text) implements Widget {
        public void render(StringBuilder out, String indent) { out.append(indent).append(text.get()).append('\n'); }
    }

    record Panel(String title, List<Widget> children) implements Widget {
        public void render(StringBuilder out, String indent) {
            out.append(indent).append("+ ").append(title).append('\n');
            children.forEach(c -> c.render(out, indent + "  "));
        }
    }
}
```

```java
// File: code/lld/ch11/mvc/TaskController.java
package lld.ch11.mvc;

/** CONTROLLER as a STRATEGY: the view delegates "what does this input mean?" to it. */
public interface TaskController {
    void handle(String input);
}
```

```java
// File: code/lld/ch11/mvc/EditingController.java
package lld.ch11.mvc;

/** Interprets user input and updates the MODEL. It does not redraw anything itself. */
public final class EditingController implements TaskController {
    private final TaskModel model;
    private final TaskView view;

    public EditingController(TaskModel model, TaskView view) {
        this.model = model;
        this.view = view;
    }

    public void handle(String input) {
        try {
            if (input.startsWith("add ")) model.add(input.substring(4));
            else if (input.startsWith("done ")) model.complete(Integer.parseInt(input.substring(5).trim()));
            else view.showMessage("unknown command: " + input);
        } catch (IllegalArgumentException e) {        // NumberFormatException is a subclass
            view.showMessage("error: " + e.getMessage());
        }
    }
}
```

```java
// File: code/lld/ch11/mvc/ReadOnlyController.java
package lld.ch11.mvc;

/** Alternative strategy: same view, different behaviour (e.g. a viewer role). */
public final class ReadOnlyController implements TaskController {
    private final TaskView view;

    public ReadOnlyController(TaskView view) { this.view = view; }

    public void handle(String input) { view.showMessage("read-only mode, ignored: " + input); }
}
```

```java
// File: code/lld/ch11/mvc/TaskView.java
package lld.ch11.mvc;

import java.util.List;
import java.util.stream.Collectors;

/** VIEW: observes the model, renders it (composite), forwards user input to its controller (strategy). */
public final class TaskView implements ModelListener {
    private TaskController controller;
    private final Widget root;
    private TaskModel lastSeen;

    public TaskView() {
        root = new Widget.Panel("My Tasks", List.of(
            new Widget.Label(() -> lastSeen == null ? "(empty)" : lastSeen.tasks().stream()
                .map(t -> (t.done() ? "[x] " : "[ ] ") + t.id() + ". " + t.title())
                .collect(Collectors.joining("\n    "))),
            new Widget.Panel("Summary", List.of(
                new Widget.Label(() -> lastSeen == null ? "0 open" :
                    lastSeen.tasks().stream().filter(t -> !t.done()).count() + " open")))));
    }

    public void setController(TaskController controller) { this.controller = controller; }

    /** A user action arrives at the view... and is handed straight to the controller. */
    public void userTyped(String input) {
        System.out.println("> " + input);
        controller.handle(input);
    }

    public void showMessage(String msg) { System.out.println("  (" + msg + ")"); }

    @Override
    public void modelChanged(TaskModel model) {     // Observer callback: re-render from the model
        lastSeen = model;
        StringBuilder out = new StringBuilder();
        root.render(out, "  ");
        System.out.print(out);
    }
}
```

```java
// File: code/lld/ch11/mvc/Main.java
package lld.ch11.mvc;

public class Main {
    public static void main(String[] args) {
        TaskModel model = new TaskModel();
        TaskView view = new TaskView();
        model.addListener(view);                                  // Observer: model -> view
        view.setController(new EditingController(model, view));  // Strategy: view -> controller

        view.userTyped("add write chapter 11");
        view.userTyped("add review diagrams");
        view.userTyped("done 1");
        view.userTyped("done 42");

        System.out.println("-- switching the view to a read-only controller --");
        view.setController(new ReadOnlyController(view));
        view.userTyped("add sneaky task");
    }
}
```

Output:

```
> add write chapter 11
  + My Tasks
    [ ] 1. write chapter 11
    + Summary
      1 open
> add review diagrams
  + My Tasks
    [ ] 1. write chapter 11
    [ ] 2. review diagrams
    + Summary
      2 open
> done 1
  + My Tasks
    [x] 1. write chapter 11
    [ ] 2. review diagrams
    + Summary
      1 open
> done 42
  (error: no task 42)
-- switching the view to a read-only controller --
> add sneaky task
  (read-only mode, ignored: add sneaky task)
```

Trade-offs, variants and mistakes

Pros	Cons
Separation of concerns; business rules testable without UI.	More classes and indirection for tiny apps.
Several views on one model (list, chart, widget).	Update flows via observers can be hard to trace.
Swap controllers/views independently.	Controllers tend to get "fat" if business logic leaks into them.

- **Web MVC is different.** In **Spring MVC** (a "Model 2" design), a **front controller** (`DispatcherServlet` — a *servlet* is a Java class that handles HTTP requests inside a web server) routes each HTTP request to a `@Controller` method, which fills a model and returns a view name; the view renders once per request. The view does **not** observe the model — there is no long-lived observer link over HTTP.
- **Relatives: MVP** (Model–View–Presenter: a passive view, the presenter updates it) and **MVVM** (Model–View–ViewModel: the view binds to observable properties of a view-model — Android Jetpack `ViewModel` + `LiveData`/`StateFlow`, WPF). MVVM's view-model also plays a **mediator** role.
- **Common mistakes:** business logic in controllers ("fat controller"); views changing the model directly; models that import UI classes.

Real-world examples

Spring MVC, Ruby on Rails, Django (which calls it MTV), ASP.NET MVC, Swing (each component has a model, e.g. `JTable` + `TableModel` with `TableModelListener`), iOS UIKit (`UIViewController`), Android (MVVM).

In the interview

30-second answer: "MVC separates the model — data and rules — from the view that shows it and the controller that interprets input. Head First calls it a compound pattern: the model is an Observer subject for the views, the controller is a Strategy the view delegates to, and the view is a Composite of widgets. On the web, like Spring MVC, it becomes request-driven with a front controller, and the view doesn't observe the model."

Self-check. Q: Which three patterns make up MVC? A: Observer, Strategy, Composite. Q: Does the controller redraw the view? A: No — it updates the model, and the model's notification makes the view redraw.

TL;DR: Model (state + rules, observable) → **View** (composite, observes) → **Controller** (strategy that interprets input).

11.7.6 Event Sourcing (lite)

What it is

Event Sourcing stores **every change as an immutable event** ("ItemAdded", "ItemRemoved", "CheckedOut") in an **append-only log** (a log you can only add to at the end, never edit), and derives the current state by **replaying** those events — instead of storing only the latest state and overwriting it. It is included here because it **borrows ideas from Command, Memento and Observer at system level** (events look like recorded commands, snapshots are mementos, projections observe the log), and it shows up in design interviews about carts, wallets, orders and audit trails.

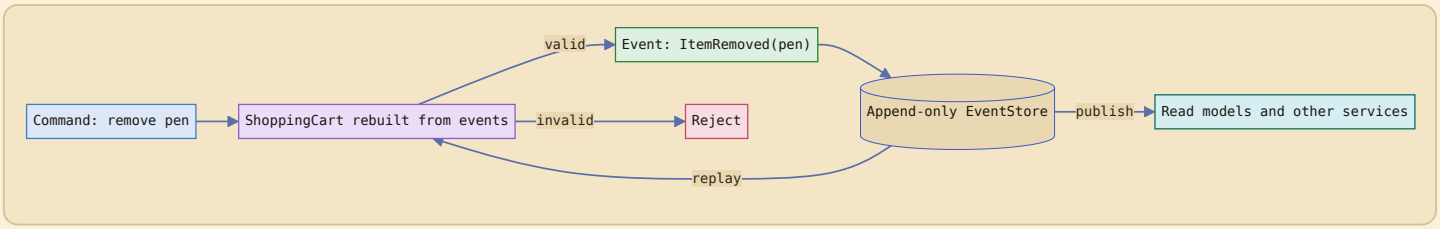
Analogy: a bank passbook. The balance is not the source of truth; the list of transactions is. The balance can always be recomputed from it.

The problem it solves

With "update in place", history is lost: you can't answer "what did the cart look like before checkout?", audit who changed what, or rebuild a **read model** (a copy of the data shaped for fast queries) after a bug. With events, the full history is kept, and you can **time-travel**, **audit**, and **build new views** later by replaying.

How it works

1. **Command side:** load the aggregate by replaying its events; **validate** the command against current state; if valid, **emit an event** (a fact, past tense).
2. **Append** the event to the store — never update or delete events.
3. **Apply**: a **pure function** (its output depends only on its inputs, and it changes nothing else) updates state from an event (no validation, no side effects — events are facts).
4. **Read side:** replay (all or the first *n* events) to get state at any point; publish events to **observers** that build read models (this split is **CQRS** — Command Query Responsibility Segregation).
5. For long streams, save periodic **snapshots** (Memento!) and replay only newer events.



Java example

A **sealed** `CartEvent` hierarchy; `ShoppingCart.apply` is an exhaustive pattern-matching `switch` over it (a `switch` *statement* with patterns must be exhaustive in Java 21, so a new event type that `apply` forgets is a compile error); `EventStore` is an append-only in-memory log whose methods are `synchronized`, so appends and reads from several threads are safe. Prices are `BigDecimal`.

```java
// File: code/lld/ch11/eventsourcing/CartEvent.java
package lld.ch11.eventsourcing;

import java.math.BigDecimal;

/** Facts that HAPPENED (past tense). Immutable; never updated or deleted. */
public sealed interface CartEvent permits CartEvent.ItemAdded, CartEvent.ItemRemoved, CartEvent.CheckedOut {
    record ItemAdded(String sku, int qty, BigDecimal unitPrice) implements CartEvent {}
    record ItemRemoved(String sku) implements CartEvent {}
    record CheckedOut() implements CartEvent {}
}
```

```java
// File: code/lld/ch11/eventsourcing/EventStore.java
package lld.ch11.eventsourcing;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

/** Append-only log per stream (cart id). The log IS the source of truth. */
public final class EventStore {
    private final Map<String, List<CartEvent>> streams = new ConcurrentHashMap<>();

    public synchronized void append(String streamId, CartEvent e) {
        streams.computeIfAbsent(streamId, k -> new ArrayList<>()).add(e);
    }

    public synchronized List<CartEvent> read(String streamId) {
        return List.copyOf(streams.getOrDefault(streamId, List.of()));
    }
}
```

```java
// File: code/lld/ch11/eventsourcing/ShoppingCart.java
package lld.ch11.eventsourcing;

import java.math.BigDecimal;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/** Aggregate whose current state is DERIVED by replaying events. */
public final class ShoppingCart {
    private final Map<String, BigDecimal> lineTotals = new LinkedHashMap<>();
    private boolean checkedOut;

    /** Rebuild state from history (optionally only the first n events: "time travel"). */
    public static ShoppingCart replay(List<CartEvent> history, int n) {
        ShoppingCart cart = new ShoppingCart();
        history.stream().limit(n).forEach(cart::apply);
        return cart;
    }

    /** Pure state transition: no validation, no side effects. Events are facts. */
    private void apply(CartEvent e) {
        switch (e) {
            case CartEvent.ItemAdded a -> lineTotals.merge(a.sku(), a.unitPrice().multiply(BigDecimal.valueOf(a.qty())),
BigDecimal::add);
            case CartEvent.ItemRemoved r -> lineTotals.remove(r.sku());
            case CartEvent.CheckedOut c -> checkedOut = true;
        }
    }

    /** Command side: validate against current state, THEN emit an event. */
    public CartEvent decideRemove(String sku) {
        if (checkedOut) throw new IllegalStateException("cart already checked out");
        if (!lineTotals.containsKey(sku)) throw new IllegalArgumentException(sku + " not in cart");
        return new CartEvent.ItemRemoved(sku);
    }

    @Override
    public String toString() {
        BigDecimal total = lineTotals.values().stream().reduce(new BigDecimal("0.00"), BigDecimal::add);
        return lineTotals.keySet() + " total=" + total + (checkedOut ? " CHECKED-OUT" : "");
    }
}
```

```java
// File: code/lld/ch11/eventsourcing/Main.java
package lld.ch11.eventsourcing;

import java.math.BigDecimal;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        EventStore store = new EventStore();
        String cartId = "cart-7";
        store.append(cartId, new CartEvent.ItemAdded("book", 2, new BigDecimal("12.99")));
        store.append(cartId, new CartEvent.ItemAdded("pen", 5, new BigDecimal("0.99")));

        ShoppingCart current = ShoppingCart.replay(store.read(cartId), Integer.MAX_VALUE);
        store.append(cartId, current.decideRemove("pen"));      // validated, then recorded
        store.append(cartId, new CartEvent.CheckedOut());

        List<CartEvent> history = store.read(cartId);
        System.out.println("event log:");
        history.forEach(e -> System.out.println("  " + e));

        System.out.println("state after each event (replay = time travel):");
        for (int n = 0; n <= history.size(); n++) {
            System.out.println("  after " + n + ": " + ShoppingCart.replay(history, n));
        }

        try {
            ShoppingCart.replay(history, history.size()).decideRemove("book");
        } catch (IllegalStateException e) {
            System.out.println("command rejected: " + e.getMessage());
        }
    }
}
```

Output:

```
event log:
  ItemAdded[sku=book, qty=2, unitPrice=12.99]
  ItemAdded[sku=pen, qty=5, unitPrice=0.99]
  ItemRemoved[sku=pen]
  CheckedOut[]
state after each event (replay = time travel):
  after 0: [] total=0.00
  after 1: [book] total=25.98
  after 2: [book, pen] total=30.93
  after 3: [book] total=25.98
  after 4: [book] total=25.98 CHECKED-OUT
command rejected: cart already checked out
```

Trade-offs, when not to use it, mistakes

Pros	Cons
Complete audit trail and time travel.	More complex than CRUD; a learning curve for the team.
Rebuild or add read models at any time.	**Event schema evolution** (versioning old events) is hard.
Natural fit with event-driven architectures.	Reads need projections (read models built from events); eventual consistency (the read side catches up after a short delay) between write and read sides.

- **Don't use it** for simple CRUD without audit or history needs.
- **Common mistakes:** putting validation in `apply` (replay of old facts may then fail); events named as commands (`RemoveItem` instead of `ItemRemoved`); mutable events; storing derived data inside events; no snapshots for very long streams; **appending without an expected-version check** — two commands decided on the same old state can both append. Real event stores take an `expectedVersion` on append and reject the second writer (optimistic concurrency). The toy `EventStore` here omits it.

Real-world examples

Bank ledgers and payment systems (balances derived from ledger entries); Git (the version-control system — its history of commits); **Axon Framework** and **EventStoreDB**; Kafka used as an event log with long or unlimited **retention** (how long stored messages are kept) (not *compacted* topics — compaction keeps only the latest record per key, which throws away the history event sourcing needs); shopping carts and order histories where "what happened when" matters.

In the interview

> **30-second answer:** "Event sourcing stores every change as an immutable, past-tense event in an append-only log and derives state by replaying. Commands are validated against the replayed state and then produce events; `apply` just folds events into state. You get audit and time travel, and can build new read models by replaying; the costs are complexity, schema evolution and eventual consistency. Snapshots keep replay fast — that's Memento plus a command log."

Self-check. Q: Where does validation happen? A: On the command side, before emitting the event — never in `apply`. Q: How do you keep replay fast? A: Periodic snapshots.

> **TL;DR: Store facts, not state**; rebuild by replay; validate commands, never events; snapshot long streams.

11.8 Choose the right behavioral pattern

Start from the **symptom** in the code or requirement, not from the pattern name.

If you see / need...	Use	Not to be confused with
A request must pass several independent checks; any may reject it; different routes need different checks	**Chain of Responsibility**	Decorator (every layer always runs)
Undo/redo, queue, schedule, retry, or log *requested actions*	**Command**	Strategy (how, not what)
Evaluate user-written rules/expressions in a small language	**Interpreter** (+ Specification)	Visitor (for many operations over the AST)
Walk a collection/tree/paged API without exposing structure	**Iterator** (`Iterable`)	Visitor (what to do at each node)
Many objects depend on each other's changes and the *order* of reactions matters	**Mediator**	Observer (no coordination)
Roll back to an earlier state without exposing internals	**Memento**	Command undo (store actions instead)
One change must notify an open-ended set of independent listeners	**Observer** (or a message broker for durability)	Mediator
Three or more methods repeat one `switch` over a status; which actions are legal depends on that status	**State**	Strategy (chosen from outside)
Several interchangeable algorithms for one job, chosen by config/user/run time	**Strategy** (often a lambda)	State, Template Method
Several classes share a long fixed flow and differ in a few steps	**Template Method** (or template + callbacks)	Strategy
Many new operations over a stable set of types	**Visitor**, or **sealed types + pattern matching** in Java 21	Iterator, Strategy
`null` checks everywhere for an optional collaborator	**Null Object** (or `Optional` for lookups)	—
Classes create their own collaborators; hard to test	**Dependency Injection**	Service Locator (anti-pattern)
Business logic mixed with storage code	**Repository**	DAO (table-oriented)
The same business rule duplicated in queries and validation; finder-method explosion	**Specification**	—
UI, input handling and business rules tangled together	**MVC / MVVM**	—
Need full history, audit, time travel of changes	**Event Sourcing** (+ snapshots)	Plain audit log

Three questions that settle the famous look-alikes:

1. **Do the alternatives reference each other?** Yes → **State**. No → **Strategy** (or Bridge if chosen once at wiring).
2. **Can one participant stop the flow?** Yes → **Chain of Responsibility**. Every participant always runs → **Decorator** / pipeline. Everyone is told, nobody stops anyone → **Observer**.
3. **Does a central object decide what happens next?** Yes → **Mediator**. Each listener decides for itself → **Observer**.

In the interview (chapter-level)

"Which patterns have you used in production?" Pick concrete ones and name the *trade-off*: "Strategy for payment methods, chosen by a map injected by Spring; Observer via Spring events for sending emails after an order, moved to Kafka when we needed retries; State for the order lifecycle, with the DB update guarded by `WHERE status = ?` to avoid races."

"When would you avoid patterns?" When only two branches exist and a third is not expected; when a lambda or an enum does the job; when the indirection costs more readability than it buys flexibility. Patterns are a vocabulary for trade-offs, not a checklist.

Self-check (whole chapter)

1. **Q:** An order moves placed → accepted → preparing → delivered and "cancel" is only legal in early stages. Pattern? **A:** State (enum or class per state).

2. **Q:** Checkout must support card, UPI, wallet, and new methods next quarter. Pattern? **A:** Strategy, selected by a factory/DI map.
3. **Q:** After an order is placed, send email, update analytics and notify the restaurant; more reactions will come. Pattern? **A:** Observer in-process, or pub/sub via a broker for reliability.
4. **Q:** A compiler needs type-checking, optimisation and pretty-printing passes over a fixed AST. Pattern? **A:** Visitor, or sealed interfaces + pattern matching in Java 21.
5. **Q:** Three report generators share "load → filter → format → export" but differ in format. Pattern? **A:** Template Method (or a fixed pipeline taking a formatting strategy).
6. **Q:** What three GoF patterns make up MVC? **A:** Observer, Strategy, Composite.

TL;DR

- **Observer** broadcasts; **Mediator** coordinates; **CoR** lets someone stop the flow.
- **State** vs **Strategy**: same diagram — states know each other and advance themselves; strategies are chosen from outside.
- **Template Method** = inheritance, one step varies; **Strategy** = composition, whole algorithm varies.
- **Visitor** = cheap new operations, expensive new types; Java 21 **sealed + switch** usually replaces it.
- Beyond GoF: **Null Object**, **DI**, **Repository**, **Specification**, **MVC**, **Event Sourcing** — the everyday glue of real systems and machine-coding answers.

Chapter 12 — Concurrency & Multithreading for LLD (Java)

> **Why this chapter exists.** In a low-level design (LLD) interview you will often hear: *"Now two users click **Book** on the same seat at the same moment. What happens?"* If your design has no answer, it is incomplete. This chapter teaches you, from zero, the ideas and Java tools you need to answer that question well. Chapter 13 then applies them to the classic concurrency interview problems, and chapters 14–21 use them inside full case studies.

How to read this chapter. Every concept follows the same path: *What it is → The problem it solves → How it works → Java example → Trade-offs → When not to use it → Common mistakes → Real-world examples → In the interview → Self-check → TL;DR.* All code is Java 21, standard library only, and lives in `code/lld/ch12/<topic>/Main.java`. Every `Main` compiles and runs; the real output is shown under each listing.

Run any example from `chapters/12-concurrency-concepts/code`:

```
javac --release 21 -d /tmp/lld_12 $(find . -name '*.java')
java -cp /tmp/lld_12 lld.ch12.threads.Main
```

Map of the chapter

Part	Sections	Code package
A. Foundations	12.1 Concurrency vs parallelism · 12.2 Processes vs threads · 12.3 Thread lifecycle · 12.4 Creating threads	`threads`
B. What goes wrong	12.5 Race conditions & critical sections · 12.6 Java Memory Model (visibility, happens-before, `volatile`)	`race`
C. Locks	12.7 Mutex / `synchronized` · 12.8 `ReentrantLock` · 12.9 Try-lock & timed locking · 12.10 Read-write locks & `StampedLock` · 12.11 Coarse vs fine-grained locking	`locks`
D. Coordination	12.12 Semaphores · 12.13 Condition variables · 12.14 `CountDownLatch` & `CyclicBarrier`	`coordination`
E. Lock-free	12.15 Compare-and-swap & atomics · 12.16 Concurrent collections	`atomics`, `collections`
F. Liveness failures	12.17 Deadlock · 12.18 Livelock · 12.19 Starvation	`deadlock`
G. Patterns	12.20 Signaling · 12.21 Thread pool · 12.22 Producer-consumer · 12.23 Reader-writer · 12.24 Immutability & thread confinement	`patterns`
H. Interviews	12.25 Concurrency in LLD interviews (parking spots, seat booking, inventory)	`lldcases`
	12.26 Choosing the right tool · 12.27 Chapter self-check · 12.28 Chapter TL;DR	

Words you will see everywhere (defined once here):

Term	Meaning in one line
Thread	One independent sequence of instructions running inside a program.
Shared state	Data (fields, objects) that more than one thread can read or write.
Thread-safe	A class that behaves correctly no matter how threads interleave, with no extra locking by callers.
Atomic	Happens as one indivisible step: other threads see either "before" or "after", never "halfway".
Lock / mutex	A token only one thread can hold at a time. Code that needs the token waits for it.
Critical section	A piece of code that touches shared state and must not run in two threads at once.
Contention	Many threads wanting the same lock at the same time, so they queue up.
Blocking	A thread stops and waits (for a lock, I/O, a signal) instead of using the CPU.
Liveness	"Something good eventually happens" — the program keeps making progress (no deadlock, livelock, starvation).
Safety	"Nothing bad ever happens" — no corrupted data, no double booking.

Part A — Foundations

12.1 Concurrency vs parallelism

What it is

- **Concurrency** is about *dealing with* many things at once: the program is structured as several tasks whose lifetimes overlap. They may take turns on one **CPU core** (one independent processing unit inside the processor; a modern CPU has several).
- **Parallelism** is about *doing* many things at once: tasks literally execute at the same instant on different CPU cores.

> **Analogy.** One chef cooking three dishes, switching between them while each simmers, is *concurrent*. Three chefs each cooking one dish at the same time is *parallel*. A restaurant with three chefs who also switch between dishes is both.

The problem it solves

Programs wait a lot — for disks, networks, databases, users. Concurrency lets the program do useful work while one task waits (better *responsiveness* and *throughput* — how much work gets finished per second; see System Design Fundamentals). Parallelism lets CPU-heavy work (sorting, image processing) finish sooner by using all cores (better *speed-up*).

How it works

1. The operating system (OS) *scheduler* (the part of the OS that decides which thread runs next) gives each runnable thread a short time slice on a core.
2. When the slice ends, or the thread blocks (for **I/O** — input/output such as disk or network reads — a lock, `sleep`), the scheduler performs a **context switch**: it saves that thread's **registers** (the CPU's few, fastest storage slots) and loads another's.
3. On one core, switching fast creates the *illusion* of simultaneity → concurrency.
4. On many cores, several threads really run at the same instant → parallelism.

	Concurrency	Parallelism
Core idea	Structure: many tasks in progress	Execution: many tasks running at the same instant
Needs multiple cores?	No	Yes
Main benefit	Responsiveness, overlapping waiting (I/O-bound work: mostly waiting on disk/network)	Speed on CPU-bound work (mostly computing)
Java tools	Threads, executors, virtual threads, `CompletableFuture`	Fork/Join, parallel streams, `Arrays.parallelSort`
Typical example	A web server handling 10,000 connections	Sorting 100 million numbers

Java example

Section 12.4's `Main` shows both: 10,000 tasks that each *sleep* 50 ms finish in well under a second on virtual threads (concurrency hides waiting). Chapter 13's multi-threaded merge sort uses **Fork/Join** (the JDK framework that splits a task into subtasks and runs them on all cores) to split CPU work over cores (parallelism).

Trade-offs

Concurrency adds complexity (races, deadlocks) and some overhead (context switches). Parallelism has a limit: **Amdahl's law** says that if a fraction *s* of the work is inherently sequential, the speed-up with *N* cores can never exceed 1 / (s + (1 − s)/N). With 10% sequential work, even infinite cores give at most 10× speed-up.

When not to use it

Small, fast, CPU-light tasks: splitting them over threads costs more than it saves. A single-threaded design is also a valid choice when it removes all locking (Redis — an in-memory key-value database — executes commands on one main thread for this reason; Node.js, a server-side JavaScript runtime, runs JavaScript on one **event-loop** thread: a single loop that handles ready events one at a time and never blocks).

Common mistakes

- Saying "concurrent" when you mean "parallel" (and vice versa) in an interview.
- Expecting more threads to always mean more speed. Past the core count, CPU-bound work gets *slower* from switching.

Real-world examples

- **Concurrency:** Tomcat and Jetty (Java web servers) serve many HTTP requests with a **pool** of threads (a fixed set of reusable worker threads); Netty (a Java networking library) and Node.js serve thousands of connections with a few event-loop threads.
- **Parallelism:** `Arrays.parallelSort`, parallel streams, and Spark/Hadoop (distributed data-processing frameworks) jobs splitting data across cores and machines.

In the interview

> **30-second answer:** "Concurrency is about structure — several tasks in progress, possibly interleaved on one core. Parallelism is about execution — tasks running at the same instant on several cores. Concurrency helps I/O-bound work; parallelism helps CPU-bound work."

Follow-ups:

- *Can you have concurrency without parallelism?* Yes — a single-core machine running several threads by time-slicing.
- *Can you have parallelism without concurrency?* Yes, in a narrow sense: SIMD (one instruction on many data items) runs in parallel inside a single task.
- *How many threads for a CPU-bound pool?* About the number of cores (`Runtime.getRuntime().availableProcessors()`). For I/O-bound work, more — roughly cores × (1 + wait time / compute time).

Self-check

- **Q:** A chat server with 50,000 idle connections — concurrency or parallelism problem? **A:** Concurrency: most connections are waiting, not computing.

- **Q:** What limits the speed-up of parallel code? **A:** The sequential part (Amdahl's law), plus coordination overhead.

TL;DR

Concurrency = dealing with many things (structure). Parallelism = doing many things at once (execution). I/O-bound → concurrency; CPU-bound → parallelism.

12.2 Processes vs threads

What it is

- A **process** is a running program with its *own* memory space (**heap** — the memory area where objects live — code, open files). Two processes cannot see each other's memory.
- A **thread** is a unit of execution *inside* a process. All threads of a process share the process's heap memory, but each thread has its own **stack** (local variables, call frames) and **program counter** (the address of the next instruction to run).

> **Analogy.** A process is a house; threads are people living in it. People in the same house share the kitchen (heap) — fast to share, but they can bump into each other. People in different houses (processes) must phone each other (inter-process communication) — slower, but no bumping.

The problem it solves

Threads let one program do several things at once while sharing data cheaply. Processes give strong isolation: one crashing does not corrupt the others.

How it works

1. The OS creates a process with a private **virtual address space** (the process's own view of memory, which the OS maps onto real RAM).
2. The process starts with one thread (`main` in Java).
3. The program can create more threads; they share heap objects, and each gets its own stack.
4. Communication: threads just read/write shared objects (with synchronization); processes use **pipes** (one-way byte channels between processes), **sockets** (network-style connection endpoints), shared-memory segments or message queues.

	Process	Thread
Memory	Private address space	Shares the process's heap; own stack
Creation cost	High (new address space)	Lower (platform thread ≈ an OS thread with ~1 MB stack reserved by default on 64-bit JVMs — Java Virtual Machines, the programs that run Java bytecode); virtual threads are much cheaper still
Communication	IPC: sockets, pipes, files	Shared objects — fast, but needs synchronization
Failure isolation	Strong: one crash does not kill others	Weak: an unhandled error can bring down the whole process (e.g. `OutOfMemoryError`)
Context switch	Heavier (address-space switch)	Lighter
Java	`ProcessBuilder`, `Runtime.exec`	`Thread`, executors

Java example

```java
// A new PROCESS (separate memory):
Process p = new ProcessBuilder("ls", "-l").inheritIO().start();
p.waitFor();

// A new THREAD (shares this JVM's heap):
Thread t = new Thread(() -> System.out.println("same heap, different stack"));
t.start();
t.join();
```

Trade-offs

Threads: fast sharing, but every shared mutable field is a potential bug. Processes: safe isolation, but communication is slower and needs **serialization** (turning objects into bytes so they can be sent).

When not to use it

Do not reach for threads when isolation matters more than speed (running untrusted plugins, running a crash-prone **native library** — machine code, e.g. written in C, called from Java); use separate processes or **containers** (lightweight isolated environments such as Docker).

Common mistakes

- Thinking local variables are shared between threads. They are not: each thread has its own stack. Only objects reachable from shared references are shared.
- Creating one platform thread per request without limit — each has a large stack and an OS thread behind it. Use a pool, or virtual threads.

Real-world examples

- **Google Chrome** runs tabs in separate *processes* so one crashing page does not take down the browser.
- **PostgreSQL** (a popular SQL database) uses one process per connection; **MySQL** uses one thread per connection.
- A **JVM** is one process; your application's threads, **garbage-collector** threads (the GC frees memory that nothing uses any more) and **JIT-compiler** threads (Just-In-Time compiler: turns frequently run bytecode into fast machine code while the program runs) all live inside it.

In the interview

30-second answer: "A process is an isolated program with its own memory; a thread is a path of execution inside a process that shares its heap but has its own stack. Threads are cheaper and share data easily but need synchronization; processes are isolated and safer but communicate through IPC."

Follow-ups:

- *What does a thread own privately?* Its stack (locals, call frames), program counter, and `ThreadLocal` values (variables that hold a separate copy per thread).
- *Why are threads cheaper to switch?* They share the address space, so the CPU's memory mappings stay the same.

Self-check

- **Q:** Two threads each declare `int x = 0;` inside a method and increment it. Race? **A:** No — locals live on each thread's own stack.
- **Q:** Two threads increment the same object's field. Race? **A:** Yes, unless synchronized.

TL;DR

Process = isolated house with private memory. Thread = a worker inside it sharing the heap, owning only its stack. Share cheaply, synchronize carefully.

12.3 Thread lifecycle & states

What it is

A Java thread moves through well-defined states. `Thread.getState()` returns one of the six values of the enum `java.lang.Thread.State`:

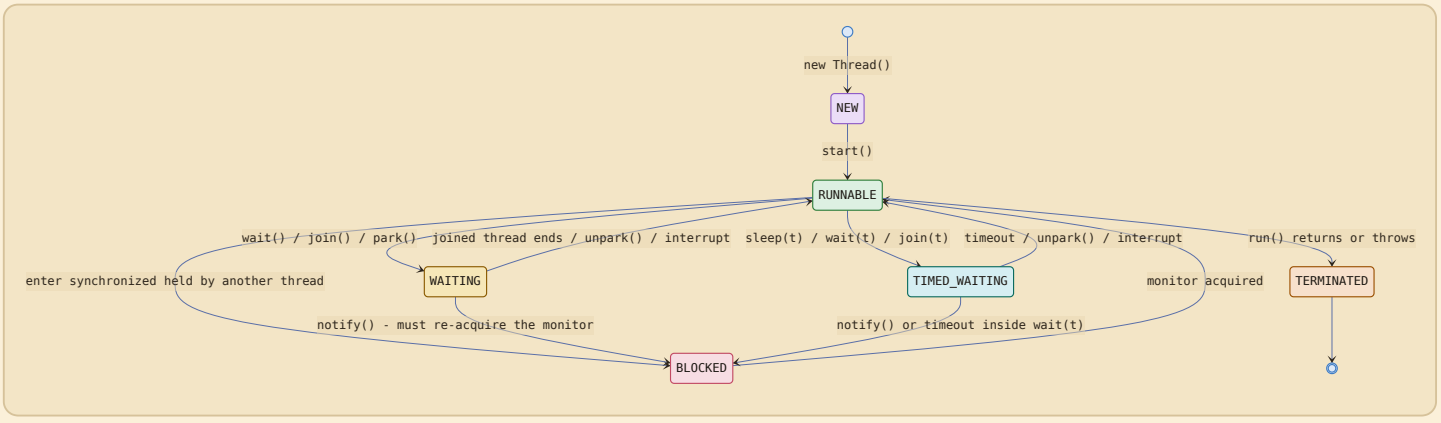
State	Meaning
`NEW`	Created with `new Thread(...)` but `start()` not called yet.
`RUNNABLE`	Started and either running on a CPU or ready to run (waiting only for the scheduler). Java does not distinguish "running" from "ready". A thread blocked in most I/O calls also shows as `RUNNABLE`.
`BLOCKED`	Waiting to *enter* a `synchronized` block/method whose monitor (the object's built-in lock, §12.7) another thread holds.
`WAITING`	Waiting with no time limit for another thread to act: `Object.wait()`, `Thread.join()`, `LockSupport.park()` (the low-level JDK call that suspends a thread; `ReentrantLock`, `Condition`, `CountDownLatch` use it).
`TIMED_WAITING`	Same, but with a timeout: `Thread.sleep(ms)`, `wait(ms)`, `join(ms)`, `parkNanos`, `tryLock(timeout)`.
`TERMINATED`	`run()` has returned or thrown. A thread cannot be restarted.

Analogy. A customer at a bank: `NEW` = has a ticket but hasn't entered; `RUNNABLE` = at the counter or next in line; `BLOCKED` = the only pen is being used, waiting to grab it; `WAITING` = told "wait until we call you"; `TIMED_WAITING` = "come back in 10 minutes"; `TERMINATED` = left the bank.

The problem it solves

Knowing the states lets you read **thread dumps** (a snapshot of every thread's state and call stack, taken with `jstack` or `jcmd <pid> Thread.print`, where pid is the process ID) and diagnose hangs: many threads `BLOCKED` on the same monitor → lock contention or **deadlock** (threads waiting on each other forever, §12.17); many `WAITING` on a queue → idle pool.

How it works



1. `new Thread(task)` → `NEW`.
2. `start()` → `RUNNABLE` (calling `start()` twice throws `IllegalThreadStateException`).
3. Trying to enter a busy `synchronized` block → `BLOCKED` until the monitor is free.
4. Calling `wait()`, `join()` or parking → `WAITING`; the timed versions → `TIMED_WAITING`.
5. A thread that wakes from `wait()` must first re-acquire the monitor, so it may pass through `BLOCKED` before returning to `RUNNABLE`.
6. When `run()` ends → `TERMINATED`.

Interruption. `t.interrupt()` sets a flag on `t`. If `t` is in `sleep`, `wait`, `join` (or other interruptible blocking calls), it wakes up with `InterruptedException` and the flag is cleared. Interruption is *cooperative*: the thread decides how to react. `Thread.stop()` is unsafe and no longer works on modern JDKs — never rely on it.

Java example

See `threadStates()` in the next section's `Main.java`: it drives one thread into each state and prints `getState()` (output lines under "Thread.State tour").

Trade-offs

Not applicable (it's a model, not a choice) — but note the `RUNNABLE` state hides whether the thread is *actually* on a CPU or stuck in native I/O.

When not to use it

Never use `getState()` for program logic (e.g. "if thread is WAITING then…"); states change between the check and your action. Use it only for monitoring and debugging.

Common mistakes

- Calling `run()` instead of `start()`: the code runs on the *current* thread; no new thread is created.
- Swallowing `InterruptedException` silently. Either rethrow it or restore the flag with `Thread.currentThread().interrupt()`.
- Thinking `BLOCKED` and `WAITING` are the same. `BLOCKED` = waiting for a *monitor*; `WAITING` = waiting for a *signal*.

Real-world examples

Production incident triage: a `jstack` dump showing 200 Tomcat worker threads `BLOCKED` on one `synchronized` logger or connection-pool method is a classic sign of a coarse lock bottleneck.

In the interview

> **30-second answer:** "Java threads have six states: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED. BLOCKED means waiting for a monitor lock; WAITING/TIMED_WAITING mean waiting for another thread's action, optionally with a timeout."

Follow-ups:

- *Is there a RUNNING state?* Not in Java's enum — running and ready are both `RUNNABLE`.
- *What state is a thread in during `ReentrantLock.lock()` when the lock is taken?* `WAITING` (it parks via `LockSupport`), not `BLOCKED` — `BLOCKED` is only for intrinsic monitors.
- *Can you restart a terminated thread?* No; create a new one (or submit the task to a pool).

Self-check

- **Q:** State of a thread inside `Thread.sleep(1000)`? **A:** `TIMED_WAITING`.
- **Q:** State of a thread inside `latch.await()`? **A:** `WAITING`.

TL;DR

NEW → RUNNABLE ↔ (BLOCKED | WAITING | TIMED_WAITING) → TERMINATED. BLOCKED = waiting for a monitor; WAITING = waiting for a signal. Use `start()`, never `run()`, and respect interrupts.

12.4 Creating threads: `Thread`, `Runnable`, `Callable`, `ExecutorService`, virtual threads

What it is

Five ways to run code concurrently in Java:

Way	What you write	Returns a value?	Use when
Extend `Thread`	`class T extends Thread { run() }`	No	Almost never — mixes the task with the worker
`Runnable`	`() -> { ... }` passed to `new Thread(r)` or a pool	No	A fire-and-forget task
`Callable<V>`	`() -> value` (may throw checked exceptions — ones the compiler forces you to catch or declare)	Yes, via `Future<V>`	You need a result or an exception back
`ExecutorService`	`pool.submit(task)`	`Future<V>`	Default choice: reuse threads, limit concurrency
Virtual threads (Java 21)	`Thread.ofVirtual().start(r)` / `Executors.newVirtualThreadPerTaskExecutor()`	via `Future`	Very many *blocking* (I/O-bound) tasks

> **Analogy.** A `Runnable` is a job description. A `Thread` is a worker. An `ExecutorService` is a staffing agency that keeps a team of workers and hands them job descriptions. A `Future` is the receipt you use to collect the finished job. Virtual threads are like freelancers who cost almost nothing to hire, so you hire one per job.

The problem it solves

Creating an OS thread per task is expensive and unbounded. Executors separate **what** to run (task) from **how** it runs (pooling, queueing, scheduling). Virtual threads remove the cost problem for blocking code, so you can go back to a simple "one thread per request" style.

How it works

1. Write the task as a `Runnable` (no result) or `Callable<V>` (result/exception).
2. Submit it to an `ExecutorService` (e.g. `Executors.newFixedThreadPool(n)`); you get a `Future<V>`.
3. `future.get()` blocks until the result is ready (or rethrows the task's exception wrapped in `ExecutionException`).
4. When done, `shutdown()` the pool (stop accepting tasks) and `awaitTermination(...)`; or, since Java 19, use **try-with-resources** (a `try (...)` block that closes its resource automatically at the end) — `ExecutorService.close()` waits for tasks to finish.
5. **Virtual threads** (final in Java 21, JEP 444 — a JEP, JDK Enhancement Proposal, is the numbered design document for a Java feature) are lightweight threads managed by the JVM. Many virtual threads run on a small pool of OS "carrier" threads (a `ForkJoinPool` sized to the number of cores by default). When a virtual thread blocks on I/O, `sleep`, or a `java.util.concurrent` lock, the JVM *unmounts* it and the carrier runs another virtual thread. The exception is **pinning**: a virtual thread that cannot unmount keeps its carrier busy while it blocks. On Java 21–23 this happens when it blocks inside a `synchronized` block/method (including `Object.wait()`) or inside a native method; JDK 24 (JEP 491) removed the `synchronized` case, leaving mainly native/foreign-function calls (and blocking inside a class initializer) as pinning causes.

Java example

```java
// File: code/lld/ch12/threads/Main.java
package lld.ch12.threads;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Callable;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;
import java.util.stream.IntStream;

/** Five ways to run code on another thread, plus a tour of every Thread.State. */
public class Main {

    // Way 1: subclass Thread (works, but couples "the task" with "the worker").
    static class GreeterThread extends Thread {
        GreeterThread() { super("greeter-thread"); }
        @Override public void run() {
            System.out.println("1) extends Thread   -> hello from " + getName());
        }
    }

    public static void main(String[] args) throws Exception {
        creatingThreads();
        virtualThreads();
        threadStates();
    }

    static void creatingThreads() throws Exception {
        System.out.println("=== Creating threads ===");
        Thread t1 = new GreeterThread();
        t1.start();          // start() creates a new call stack; run() would run on *this* thread
        t1.join();           // wait for it to finish so output order is predictable

        // Way 2: Runnable = a task with no result. Preferred over subclassing Thread.
        Runnable task = () -> System.out.println("2) Runnable         -> hello from " + Thread.currentThread().getName());
        Thread t2 = new Thread(task, "runnable-thread");
        t2.start();
        t2.join();

        // Way 3 + 4: Callable (returns a value / throws) submitted to an ExecutorService (a thread pool).
        ExecutorService pool = Executors.newFixedThreadPool(2);
        try {
            Callable<Integer> sumTask = () -> IntStream.rangeClosed(1, 100).sum();
            Future<Integer> future = pool.submit(sumTask);   // returns immediately
            System.out.println("3) Callable+Future  -> sum 1..100 = " + future.get()); // get() blocks until done

            List<Callable<String>> jobs = List.of(() -> "A", () -> "B", () -> "C");
            List<String> results = new ArrayList<>();
            for (Future<String> f : pool.invokeAll(jobs)) results.add(f.get());   // invokeAll keeps input order
            System.out.println("4) ExecutorService  -> invokeAll results " + results);
        } finally {
            pool.shutdown();                                  // stop accepting work
            pool.awaitTermination(5, TimeUnit.SECONDS);       // wait for running tasks
        }
    }

    static void virtualThreads() throws Exception {
        System.out.println("=== Virtual threads (Java 21) ===");
        Thread vt = Thread.ofVirtual().name("virtual-1").start(
                () -> System.out.println("5) virtual thread   -> isVirtual=" + Thread.currentThread().isVirtual()));
        vt.join();

        // 10,000 blocking tasks: cheap with virtual threads (each sleep parks the virtual thread,
        // freeing its carrier OS thread for other work).
        long start = System.nanoTime();
        int completed;
        try (ExecutorService vexec = Executors.newVirtualThreadPerTaskExecutor()) {
            List<Future<Integer>> fs = new ArrayList<>();
            for (int i = 0; i < 10_000; i++) {
                fs.add(vexec.submit(() -> { Thread.sleep(50); return 1; }));
```

```
            }
            int sum = 0;
            for (Future<Integer> f : fs) sum += f.get();
            completed = sum;
        } // close() waits for all tasks
        long ms = (System.nanoTime() - start) / 1_000_000;
        System.out.println("10,000 tasks that each sleep 50 ms completed: " + completed
                + " (well under 10,000 x 50 ms = 500 s: " + (ms < 60_000) + ")");
    }

    static void threadStates() throws Exception {
        System.out.println("=== Thread.State tour ===");
        Object monitor = new Object();

        Thread sleeper = new Thread(() -> {
            try { Thread.sleep(10_000); } catch (InterruptedException e) { /* woken early: fine */ }
        }, "sleeper");
        System.out.println("after new Thread()            : " + sleeper.getState());      // NEW
        sleeper.start();
        awaitState(sleeper, Thread.State.TIMED_WAITING);
        System.out.println("inside Thread.sleep(10s)      : " + sleeper.getState());      // TIMED_WAITING
        sleeper.interrupt();
        sleeper.join();
        System.out.println("after run() returns           : " + sleeper.getState());      // TERMINATED

        CountDownLatch gate = new CountDownLatch(1);
        Thread waiter = new Thread(() -> {
            try { gate.await(); } catch (InterruptedException ignored) { }
        }, "waiter");
        waiter.start();
        awaitState(waiter, Thread.State.WAITING);
        System.out.println("inside latch.await()          : " + waiter.getState());       // WAITING
        gate.countDown();
        waiter.join();

        Thread blocked;
        synchronized (monitor) {             // main holds the monitor...
            blocked = new Thread(() -> { synchronized (monitor) { /* got it */ } }, "blocked");
            blocked.start();
            awaitState(blocked, Thread.State.BLOCKED);
            System.out.println("waiting to enter synchronized : " + blocked.getState()); // BLOCKED
        }                                    // ...released here, so 'blocked' can proceed
        blocked.join();

        java.util.concurrent.atomic.AtomicBoolean stop = new java.util.concurrent.atomic.AtomicBoolean();
        Thread spinner = new Thread(() -> { while (!stop.get()) Thread.onSpinWait(); }, "spinner");
        spinner.start();
        System.out.println("busy running on a CPU         : " + spinner.getState());      // RUNNABLE
        stop.set(true);
        spinner.join();
    }

    /** Poll until the thread reaches the wanted state (max ~5 s). */
    static void awaitState(Thread t, Thread.State wanted) throws InterruptedException {
        long deadline = System.nanoTime() + TimeUnit.SECONDS.toNanos(5);
        while (t.getState() != wanted && System.nanoTime() < deadline) Thread.sleep(1);
    }
}
```

Explaining each part:

- `GreeterThread` subclasses `Thread` — shown only for completeness. Prefer passing a `Runnable`.
- `t1.start()` creates the new call stack; `t1.join()` waits for it to finish, which also guarantees that everything `t1` wrote is visible to `main` afterwards (a *happens-before* edge, section 12.6).
- `pool.submit(sumTask)` returns a `Future` immediately; `future.get()` blocks.
- `invokeAll` runs a list of `Callable`s and returns futures **in the same order** as the input list.
- `shutdown()` + `awaitTermination()` is the classic clean shutdown. `newVirtualThreadPerTaskExecutor()` in try-with-resources uses `close()`, which waits for all tasks.
- 10,000 sleeping tasks finish quickly: each `sleep` parks its virtual thread instead of holding an OS thread.
- `threadStates()` drives threads into each state (NEW, TIMED_WAITING, TERMINATED, WAITING, BLOCKED, RUNNABLE). `awaitState` polls, because a state change is not instant after `start()`.

Output:

```
=== Creating threads ===
1) extends Thread   -> hello from greeter-thread
2) Runnable         -> hello from runnable-thread
3) Callable+Future  -> sum 1..100 = 5050
4) ExecutorService  -> invokeAll results [A, B, C]
=== Virtual threads (Java 21) ===
5) virtual thread   -> isVirtual=true
10,000 tasks that each sleep 50 ms completed: 10000 (well under 10,000 x 50 ms = 500 s: true)
=== Thread.State tour ===
after new Thread()            : NEW
inside Thread.sleep(10s)      : TIMED_WAITING
after run() returns           : TERMINATED
inside latch.await()          : WAITING
waiting to enter synchronized : BLOCKED
busy running on a CPU         : RUNNABLE
```

Trade-offs

	Platform thread pool	**Virtual threads**
Cost per thread	High (OS thread + large stack)	Very low (heap-allocated stack that grows as needed)
How many	Hundreds to a few thousand	Millions
Best for	CPU-bound work (size ≈ cores)	Blocking I/O-bound work
Limits concurrency?	Yes, by pool size	No — add a `Semaphore` if a downstream resource needs a limit
Pooling	Yes	No — never pool virtual threads; create one per task

When not to use it

- Do not use virtual threads for CPU-heavy computation — they give no extra cores; use a fixed pool or Fork/Join.
- On Java 21–23, a virtual thread that blocks *inside a* `synchronized` *block* stays **pinned** to its carrier thread, reducing scalability; prefer `ReentrantLock` around blocking calls on those versions. (JDK 24's JEP 491 removed most of this pinning.)
- Do not use `Executors.newCachedThreadPool()` for unbounded incoming traffic — it can create unlimited platform threads.

Common mistakes

- Forgetting `shutdown()`: non-daemon pool threads keep the JVM alive after `main` ends (a **daemon** thread is a background thread that does not keep the JVM running; ordinary threads do).
- Ignoring the `Future` of a submitted `Runnable`: exceptions thrown inside are captured in the `Future` and silently lost if you never call `get()`.
- Using `Executors.newFixedThreadPool` for untrusted load: its queue is **unbounded** (`LinkedBlockingQueue`), so a burst can exhaust memory. Build a `ThreadPoolExecutor` with a bounded queue (section 12.21).

Real-world examples

- `Executors` factory methods are the textbook **Factory** pattern in the JDK.
- **Spring** (the most widely used Java application framework): its `@Async` runs methods on a `TaskExecutor` (commonly a `ThreadPoolTaskExecutor`).
- Spring Boot 3.2+ can run request handling on virtual threads with `spring.threads.virtual.enabled=true`; Tomcat and Jetty can be configured to use virtual threads too.

In the interview

30-second answer: "I never manage raw threads in business code. I describe work as `Runnable` / `Callable` and submit it to an `ExecutorService` sized for the workload: about cores for CPU-bound work, bounded queue with a rejection policy for incoming traffic. For many blocking I/O tasks on Java 21, I use virtual threads — one per task — and protect scarce resources with a semaphore." (A **semaphore** is a counter of permits that caps how many threads may proceed at once — §12.12.)

Follow-ups:

- *`Runnable` vs `Callable`?* `Callable` returns a value and may throw checked exceptions; `Runnable` does neither.
- *`submit` vs `execute`?* `execute(Runnable)` returns nothing and uncaught exceptions go to the thread's handler; `submit` returns a `Future` that captures the exception.
- *What does `CompletableFuture` add?* Non-blocking composition: `thenApply`, `thenCombine`, `allOf`, error handling with `exceptionally`, without calling `get()` at each step.

Self-check

- **Q:** What happens if you call `thread.run()`? **A:** The code runs synchronously on the current thread.
- **Q:** Why not pool virtual threads? **A:** They are cheap to create; pooling adds nothing and hides the one-thread-per-task model. Limit concurrency with a semaphore instead.

TL;DR

Tasks = `Runnable`/`Callable`. Run them with an `ExecutorService`, get results via `Future`. Java 21 virtual threads make one-thread-per-blocking-task cheap. Always shut pools down.

Part B — What goes wrong

12.5 Race conditions & critical sections

What it is

- A **race condition** is a bug where the result depends on the unlucky timing (interleaving) of threads.
- A **critical section** is the code that touches shared mutable state and must therefore run in only one thread at a time.
- A **data race** (the precise Java Memory Model term) is when two threads access the same variable, at least one writes, and there is no happens-before ordering between them (section 12.6).

> **Analogy.** Two people share a bank account with ₹100. Both check the balance at the same moment, both see ₹100, both withdraw ₹80. The bank gave out ₹160 from ₹100. The "check the balance, then withdraw" steps were the critical section.

The problem it solves

Understanding races tells you *which* code needs protection and *why* it goes wrong. The two classic shapes:

Shape	Example	Why it breaks
Read-modify-write	`count++`	It is really `tmp = count; tmp = tmp + 1; count = tmp`. Two threads read the same old value; one update is lost.
Check-then-act	`if (!map.containsKey(k)) map.put(k, v);` or `if (seat.isFree()) seat.book(me);`	The condition can become false between the check and the act.

How it works

1. Find all **shared mutable state** (fields reachable by more than one thread and changed after construction).
2. Find every **invariant** (a rule about the data that must always hold true) involving it (e.g. "a seat has at most one owner", "balance ≥ 0").
3. Mark the code that reads and then updates that state as a **critical section**.
4. Make each critical section **atomic** — with a lock (`synchronized`, `ReentrantLock`), with a single atomic operation (`AtomicInteger.incrementAndGet`, `ConcurrentHashMap.putIfAbsent`), or by removing the sharing (immutability — objects that never change; confinement — data only one thread touches; see §12.24).

Java example

```java
// File: code/lld/ch12/race/Main.java
package lld.ch12.race;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

/** Race conditions, three fixes, and the visibility problem solved with volatile. */
public class Main {

    interface Counter { void increment(); int get(); }

    /** BROKEN: count++ is read-modify-write (3 steps), so concurrent increments get lost. */
    static class UnsafeCounter implements Counter {
        private int count;
        public void increment() { count++; }
        public int get() { return count; }
    }

    /** Fix 1: a mutex (the object's intrinsic lock) makes the critical section atomic. */
    static class SynchronizedCounter implements Counter {
        private int count;
        public synchronized void increment() { count++; }
        public synchronized int get() { return count; }   // reads need the lock too (visibility)
    }

    /** Fix 2: a lock-free atomic built on compare-and-swap. */
    static class AtomicCounter implements Counter {
        private final AtomicInteger count = new AtomicInteger();
        public void increment() { count.incrementAndGet(); }
        public int get() { return count.get(); }
    }

    static final int THREADS = 4, PER_THREAD = 250_000;

    static int hammer(Counter c) throws InterruptedException {
        List<Thread> ts = new ArrayList<>();
        for (int i = 0; i < THREADS; i++) {
            Thread t = new Thread(() -> { for (int j = 0; j < PER_THREAD; j++) c.increment(); });
            ts.add(t);
            t.start();
        }
        for (Thread t : ts) t.join();   // join() also creates a happens-before edge: we see their writes
        return c.get();
    }

    // ---------- Visibility ----------
    static class Worker implements Runnable {
        private volatile boolean running = true;   // without volatile the loop may never see 'false'
        private long iterations;
        public void run() { while (running) iterations++; }
        void stop() { running = false; }
    }

    public static void main(String[] args) throws Exception {
        int expected = THREADS * PER_THREAD;
        System.out.println("expected                : " + expected);
        int unsafe = hammer(new UnsafeCounter());
        System.out.println("unsafe counter          : " + (unsafe == expected ? "correct this run (luck!)" : "LOST " +
(expected - unsafe) + " updates"));
        System.out.println("synchronized counter    : " + hammer(new SynchronizedCounter()));
        System.out.println("atomic counter          : " + hammer(new AtomicCounter()));

        // Check-then-act race, shown safely: two threads both "see" an empty seat.
        // Here we fix it with a single atomic operation instead of check + act.
        AtomicInteger seatOwner = new AtomicInteger(0);          // 0 = free
        Thread a = new Thread(() -> seatOwner.compareAndSet(0, 1));
        Thread b = new Thread(() -> seatOwner.compareAndSet(0, 2));
        a.start(); b.start(); a.join(); b.join();
        System.out.println("seat has exactly one owner: " + (seatOwner.get() == 1 || seatOwner.get() == 2));

        Worker w = new Worker();
        Thread t = new Thread(w);
```

```
        t.start();
        Thread.sleep(50);
        w.stop();                       // volatile write ...
        t.join(2_000);                  // ... is guaranteed to become visible to the worker
        System.out.println("volatile flag stopped worker: " + !t.isAlive());
    }
}
```

Explaining each part:

- `UnsafeCounter.increment()` does `count++` with no protection → updates get lost.
- `SynchronizedCounter` wraps the critical section in the object's intrinsic lock. Note that `get()` is synchronized too — that is for *visibility* (next section).
- `AtomicCounter` uses `incrementAndGet()`, a single hardware-backed atomic step (compare-and-swap, section 12.15).
- The seat example fixes a check-then-act race with one atomic call: `compareAndSet(0, id)` only succeeds if the seat is still free.
- `Worker` uses a `volatile` flag so the stop request is guaranteed to be seen (section 12.6).

Output (the number of lost updates changes every run — that unpredictability *is* the race):

```
expected                : 1000000
unsafe counter          : LOST 680044 updates
synchronized counter    : 1000000
atomic counter          : 1000000
seat has exactly one owner: true
volatile flag stopped worker: true
```

Trade-offs

Fix	Pros	Cons
Lock (`synchronized` / `ReentrantLock`)	Protects any multi-step, multi-variable invariant	Threads wait; risk of deadlock if nested
Atomic variable / atomic collection method	Fast, no blocking, no deadlock	Only covers one variable or one map entry
Immutability / confinement	No synchronization at all	Must redesign so state isn't shared or changed

When not to use it

Don't lock data that is never shared (method locals, objects confined to one thread) or never changed after safe construction (immutable objects).

Common mistakes

- Protecting writes but not reads ("reads are harmless") — a reader can see stale or half-updated state.
- Using two *separate* thread-safe calls and assuming the combination is atomic: `if (!map.containsKey(k)) map.put(k,v)` on a `ConcurrentHashMap` is still a race. Use `putIfAbsent` / `computeIfAbsent`.
- Locking on different objects in different methods — each lock protects nothing against the other.
- Believing a race is fixed because tests pass. Races show up rarely and under load.

Real-world examples

- Double-booking in ticketing systems, overselling limited-stock items in **flash sales** (short, heavily promoted sales of limited stock), duplicate payment processing when a user double-clicks "Pay" — all check-then-act races, solved with atomic operations, locks, or database constraints (unique indexes that reject duplicate values, `SELECT ... FOR UPDATE` — a SQL read that locks the rows it returns — and optimistic version columns: a version number checked on every update, so a write based on stale data fails).

In the interview

30-second answer: "A race condition is when correctness depends on thread timing. The two common shapes are read-modify-write, like `count++`, and check-then-act, like 'if seat is free then book it'. I find the

shared mutable state and its invariants, and make each critical section atomic — with a lock, an atomic operation like `putIfAbsent` or `compareAndSet`, or by removing sharing."

Follow-ups:

- *Is `volatile int count; count++` safe?* No. `volatile` gives visibility, not atomicity; the three steps can still interleave.
- *Is `ConcurrentHashMap` enough to make my class thread-safe?* Only if every invariant lives in single map operations. Multi-key invariants still need a lock or careful design.

Self-check

- **Q:** Why does `count++` lose updates? **A:** It is read, add, write — two threads can read the same value before either writes.
- **Q:** Name the race in `if (cache.get(k) == null) cache.put(k, load(k));`. **A:** Check-then-act; two threads may both load and put. Use `computeIfAbsent`.

TL;DR

Race = result depends on timing. Watch for read-modify-write and check-then-act. Make every critical section atomic, including reads.

12.6 Java Memory Model basics: visibility, happens-before, `volatile`

What it is

The **Java Memory Model (JMM)** (Java Language Specification, chapter 17) defines *when a write by one thread is guaranteed to be seen by a read in another thread*. Without the right synchronization, a thread may see a stale value — or never see the update at all.

- **Visibility**: will thread B *see* what thread A wrote?
- **Reordering**: compilers, the JIT, and CPUs may reorder instructions for speed as long as a *single* thread can't tell the difference. Other threads can.
- **Happens-before (HB)**: a guarantee that if action X *happens-before* action Y, then X's effects are visible to Y and X appears ordered before Y.
- `volatile`: a field modifier that makes reads/writes of that field visible to all threads and orders them (no caching in registers, no harmful reordering around it).

Analogy. Each thread works from its own notepad (**CPU caches** — small, fast memories next to each core — and registers) and only sometimes copies to the shared whiteboard (main memory). Happens-before rules are the moments when the notepad is guaranteed to be synced with the whiteboard.

The problem it solves

The `Worker` loop `while (running) iterations++;` with a *non-volatile* `running` may never stop: the JIT is allowed to read `running` once and turn the loop into `while (true)`. Visibility bugs like this are invisible in code review unless you know the JMM.

How it works

The key **happens-before rules** (you can quote these in an interview):

Rule	Meaning
Program order	Within one thread, each action HB the next one.
Monitor lock	An **unlock** of a monitor HB every later **lock** of the *same* monitor.
Volatile	A **write** to a volatile field HB every later **read** of that field.
Thread start	`t.start()` HB every action in thread `t`.
Thread join	Every action in `t` HB another thread's successful return from `t.join()`.
Transitivity	If A HB B and B HB C, then A HB C.
Final fields	Strictly a separate guarantee (JLS §17.5), not a happens-before edge: once a constructor finishes without leaking `this`, any thread that obtains a reference to the object sees the correct values of its `final` fields (and of objects reachable through them, as of the end of the constructor) — even if the reference was shared without synchronization.
`java.util.concurrent`	E.g. putting into a `BlockingQueue` HB taking it out; `countDown()` HB a return from `await()`; `Future` completion HB `get()` returning.

Steps to apply them:

1. For each shared variable, ask: "Is every write-then-read pair connected by a happens-before edge?"
2. If they are protected by the same lock → yes (monitor rule).
3. If the variable is a single independent flag or reference → `volatile` is enough.
4. If it is a compound update (`x++`, check-then-act) → `volatile` is **not** enough; use a lock or atomic.

Safe publication. An object is *safely published* when the reference to it is made visible through an HB edge: stored in a `volatile` field (or an `AtomicReference`), stored in a `final` field of a properly constructed object, created by a static initializer, put into a concurrent collection, or written under a lock that the reader also takes. Otherwise a reader might see the reference but not the fully constructed object.

Double-checked locking (lazy Singleton — a class with exactly one shared instance, created only on first use) is only correct if the instance field is `volatile`:

```
class Config {
    private static volatile Config instance;          // volatile is essential
    static Config get() {
        Config c = instance;
        if (c == null) {
            synchronized (Config.class) {
                c = instance;
                if (c == null) instance = c = new Config();
            }
        }
        return c;
    }
}
```

(Simpler and just as lazy: the *holder idiom* — `private static class Holder { static final Config I = new Config(); }` — class initialization is thread-safe by the JLS.)

Java example

`Worker` in the race example above: `private volatile boolean running`. The write `running = false` in `main` HB the worker's next read of `running`, so the loop exits. The output line `volatile flag stopped worker: true` confirms it.

Trade-offs

Tool	Visibility	Atomic compound ops	Blocking	Cost
`volatile`	Yes	No	No	Low
`synchronized` / `Lock`	Yes (for code under the same lock)	Yes	Yes	Medium
`Atomic*`	Yes	Yes, for one variable	No	Low–medium
`final` (immutable)	Yes, after construction	N/A — never changes	No	Free

When not to use it

Don't use `volatile` for anything that combines several steps or several variables — use a lock or an atomic class.

Common mistakes

- "It works on my laptop" — x86 CPUs (Intel/AMD) reorder less than ARM ones (e.g. Apple Silicon, AWS Graviton). Code without proper HB edges may fail only on other hardware or after **JIT warm-up** (once the JIT has compiled and optimised the hot code).
- Leaking `this` from a constructor (registering a listener, starting a thread in the constructor) — other threads may see a half-built object, and final-field guarantees are lost.
- Thinking `synchronized` on the writer alone is enough. The reader must use the *same* lock.

Real-world examples

- `volatile boolean running` shutdown flags in server loops and schedulers.
- `ConcurrentHashMap`, `AtomicInteger`, `CopyOnWriteArrayList` internally use volatile reads/writes and CAS (compare-and-swap, §12.15) to give HB guarantees without locks for readers.
- The JDK's `String` is immutable with `final` fields, so strings can be shared across threads freely.

In the interview

> **30-second answer:** "The Java Memory Model says a thread is only guaranteed to see another thread's write if there's a happens-before edge — unlock-then-lock of the same monitor, a volatile write-then-read, thread start/join, or a concurrent-collection handoff. `volatile` gives visibility and ordering for a single variable but not atomicity; for compound actions I use locks or atomics."

Follow-ups:

- *Why must the double-checked-locking field be volatile?* Without it, the write of the reference can be reordered before the constructor's writes, so another thread might see a non-null but half-initialized object.
- *Does `volatile` make `long`/`double` writes atomic?* Yes — non-volatile 64-bit `long`/`double` writes may be split into two 32-bit halves by the JLS (rare on modern 64-bit JVMs, but allowed); volatile ones cannot.

Self-check

- **Q:** Thread A sets a non-volatile `ready = true` after preparing data; thread B spins on `ready`. Two possible bugs? **A:** B may never see `ready` become true, or may see `ready` true but stale data (reordering).
- **Q:** Which JDK classes give HB for you? **A:** Locks, `Atomic*`, concurrent collections, `BlockingQueue`, `CountDownLatch`, `Future`, `ExecutorService.submit`.

TL;DR

Writes are only guaranteed visible across threads through happens-before edges. `volatile` = visibility + ordering for one variable, not atomicity. Locks = visibility + atomicity. Publish objects safely.

Part C — Locks

12.7 Mutex (mutual exclusion) and `synchronized`

What it is

A **mutex** (mutual-exclusion lock) is a lock that at most one thread can hold. In Java every object has a built-in mutex called its **intrinsic lock** or **monitor**. The `synchronized` keyword acquires it:

```
public synchronized void deposit(int amt) { balance += amt; }   // locks 'this'
public static synchronized void log() { ... }                    // locks the Class object
void transfer() { synchronized (lock) { ... } }                  // locks 'lock'
```

> **Analogy.** A single-occupancy restroom with one key on a hook. Take the key to enter; others wait; hang it back when you leave.

The problem it solves

Makes a critical section **atomic** (no interleaving) and **visible** (the next thread to take the lock sees everything done before the unlock).

How it works

1. Thread reaches `synchronized (obj)`. If nobody holds `obj`'s monitor, it takes it and continues.
2. If another thread holds it, this thread goes to `BLOCKED` until the monitor is free.
3. On leaving the block — normally *or by exception* — the monitor is released automatically.
4. Intrinsic locks are **reentrant**: the holder can enter other `synchronized` code on the same object again without blocking itself.

Java example

`SynchronizedCounter` in section 12.5, and `Mailbox` / `FooBarMonitor` style classes later. A tip: lock on a **private final** object, not on `this`, when you don't want outside code to be able to take your lock:

```
private final Object lock = new Object();
void add(Item i) { synchronized (lock) { items.add(i); } }
```

Trade-offs

Pros	Cons
Simplest syntax; auto-release even on exceptions	No timeout, no `tryLock`, not interruptible while waiting
JVM-optimised (uncontended locking is cheap)	One wait-set (the threads parked in `wait()`) per object (only one "condition")
Reentrant	No fairness option (fair = longest-waiting thread goes first); on Java 21–23 pins virtual threads when blocking inside

When not to use it

When you need a timeout, interruptible waiting, fairness, several condition queues, or read/write separation — use `ReentrantLock` / `ReentrantReadWriteLock`.

Common mistakes

- Synchronizing on a *changing* reference (`synchronized (list)` after `list = new ArrayList<>()`) — different threads lock different objects.
- Synchronizing on **boxed values** (objects such as `Integer` that wrap a primitive) or string literals (`synchronized ("LOCK")`, `synchronized (Integer.valueOf(1))`) — these may be shared JVM-wide, causing mysterious contention or deadlocks.
- Doing slow I/O while holding the lock.

Real-world examples

`Vector`, `Hashtable`, `StringBuffer` and `Collections.synchronizedList(...)` wrap every method in `synchronized` — thread-safe per call but slow under contention, which is why `ArrayList` / `HashMap` / `StringBuilder` plus concurrent collections replaced them.

In the interview

> **30-second answer:** "A mutex lets one thread at a time into a critical section. In Java, `synchronized` uses the object's monitor: it's reentrant, released automatically, and gives visibility. I lock on a private final object and keep the section short."

Follow-ups:

- *What does a static synchronized method lock?* The `Class` object, e.g. `Counter.class` — a different lock from instance methods.
- *Is `synchronized` slow?* Uncontended, it's cheap. Under heavy contention, any lock is slow — reduce sharing or use finer locks.

Self-check

- **Q:** Two synchronized instance methods on *different* objects — do they block each other? **A:** No, different monitors.
- **Q:** Does an exception inside `synchronized` release the lock? **A:** Yes.

TL;DR

Mutex = one holder at a time. `synchronized` is Java's built-in, reentrant, auto-releasing mutex. Lock a private final object; keep the section short.

12.8 `ReentrantLock` and reentrancy

What it is

`java.util.concurrent.locks.ReentrantLock` is an explicit mutex with the same basic meaning as `synchronized` plus extra features. **Reentrant** means the thread holding the lock can acquire it again (a hold count goes up) and must release it the same number of times.

> **Analogy.** A room key that remembers who has it: if you already hold it, walking into an inner door of the same room doesn't make you wait for yourself.

The problem it solves

- Without reentrancy, a locked method calling another locked method of the same object would **deadlock with itself**.
- Compared with `synchronized`, `ReentrantLock` adds: `tryLock()` / `tryLock(timeout)`, `lockInterruptibly()`, optional **fairness** (`new ReentrantLock(true)`), multiple `Condition`s per lock, and queries like `isLocked()`, `getHoldCount()`.

How it works

1. `lock.lock()` — acquire (hold count 0 → 1), or wait if another thread holds it.
2. Same thread calls `lock.lock()` again → hold count 2; no waiting.
3. Each `unlock()` decrements; at 0 the lock is released and a waiting thread may proceed.
4. **Always** release in `finally`:

```
lock.lock();
try {
    // critical section
} finally {
    lock.unlock();
}
```

Java example

```java
// File: code/lld/ch12/locks/Main.java
package lld.ch12.locks;

import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.BrokenBarrierException;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.CyclicBarrier;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.TimeoutException;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReadWriteLock;
import java.util.concurrent.locks.ReentrantLock;
import java.util.concurrent.locks.ReentrantReadWriteLock;
import java.util.concurrent.locks.StampedLock;

/** synchronized vs ReentrantLock, reentrancy, tryLock, ReadWriteLock, StampedLock. */
public class Main {

    // ---------- Reentrancy ----------
    static class Account {
        private final ReentrantLock lock = new ReentrantLock();
        private int balance = 100;

        void withdraw(int amount) {
            lock.lock();
            try {
                if (canWithdraw(amount)) balance -= amount;   // calls another locked method
            } finally {
                lock.unlock();                                // ALWAYS unlock in finally
            }
        }
        boolean canWithdraw(int amount) {
            lock.lock();                                      // same thread re-acquires: no self-deadlock
            try {
                System.out.println("hold count inside nested call = " + lock.getHoldCount());
                return balance >= amount;
            } finally {
                lock.unlock();
            }
        }
        int balance() { lock.lock(); try { return balance; } finally { lock.unlock(); } }
    }

    // ---------- Read-write lock: many readers OR one writer ----------
    static class ConfigStore {
        private final Map<String, String> map = new HashMap<>();
        private final ReadWriteLock rw = new ReentrantReadWriteLock();
        private final Lock r = rw.readLock(), w = rw.writeLock();

        String get(String k) { r.lock(); try { return map.get(k); } finally { r.unlock(); } }
        void put(String k, String v) { w.lock(); try { map.put(k, v); } finally { w.unlock(); } }

        /** Used by the demo: prove several readers are inside the read lock at once. */
        <T> T readWhile(java.util.concurrent.Callable<T> action) throws Exception {
            r.lock(); try { return action.call(); } finally { r.unlock(); }
        }
    }

    // ---------- StampedLock optimistic read ----------
    static class Point {
        private final StampedLock sl = new StampedLock();
        private double x, y;

        void move(double dx, double dy) {
            long stamp = sl.writeLock();
            try { x += dx; y += dy; } finally { sl.unlockWrite(stamp); }
        }
        double distanceFromOrigin() {
            long stamp = sl.tryOptimisticRead();   // no locking at all, just a version number
            double cx = x, cy = y;                 // copy fields to locals
            if (!sl.validate(stamp)) {             // a writer slipped in: fall back to a real read lock
                stamp = sl.readLock();
```

```
                try { cx = x; cy = y; } finally { sl.unlockRead(stamp); }
            }
            return Math.hypot(cx, cy);
        }
    }

    public static void main(String[] args) throws Exception {
        System.out.println("=== Reentrancy ===");
        Account acc = new Account();
        acc.withdraw(30);
        System.out.println("balance = " + acc.balance());

        System.out.println("=== tryLock with timeout ===");
        ReentrantLock printer = new ReentrantLock();
        CountDownLatch mainHolds = new CountDownLatch(1);
        printer.lock();                          // main grabs the printer and keeps it
        try {
            Thread impatient = new Thread(() -> {
                try {
                    mainHolds.await();
                    if (printer.tryLock(100, TimeUnit.MILLISECONDS)) {
                        try { System.out.println("worker: got the printer"); } finally { printer.unlock(); }
                    } else {
                        System.out.println("worker: printer busy for 100 ms, doing something else instead");
                    }
                } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            });
            impatient.start();
            mainHolds.countDown();
            impatient.join();
        } finally {
            printer.unlock();
        }
        System.out.println("tryLock() now (free)  = " + printer.tryLock());
        printer.unlock();

        System.out.println("=== ReadWriteLock: readers share ===");
        ConfigStore store = new ConfigStore();
        store.put("mode", "fast");
        int readers = 3;
        CyclicBarrier allReadersInside = new CyclicBarrier(readers);
        Thread[] rs = new Thread[readers];
        boolean[] ok = new boolean[readers];
        for (int i = 0; i < readers; i++) {
            final int id = i;
            rs[i] = new Thread(() -> {
                try {
                    ok[id] = store.readWhile(() -> {
                        // If the read lock were exclusive, the barrier would time out.
                        allReadersInside.await(2, TimeUnit.SECONDS);
                        return "fast".equals(store.get("mode"));   // read lock is reentrant too
                    });
                } catch (TimeoutException | BrokenBarrierException e) {
                    ok[id] = false;
                } catch (Exception e) { throw new RuntimeException(e); }
            });
            rs[i].start();
        }
        for (Thread t : rs) t.join();
        System.out.println(readers + " readers held the read lock at the same time: " + (ok[0] && ok[1] && ok[2]));

        System.out.println("=== StampedLock optimistic read ===");
        Point p = new Point();
        p.move(3, 4);
        System.out.println("distance = " + p.distanceFromOrigin());
    }
}
```

Explaining each part:

- `Account.withdraw` locks, then calls `canWithdraw`, which locks again: `getHoldCount()` prints 2 — reentrancy in action.
- "tryLock with timeout": `main` holds `printer`; the worker waits at most 100 ms, then does something else (section 12.9).

- `ConfigStore` uses a `ReentrantReadWriteLock`. Three readers all wait at a `CyclicBarrier` *while holding the read lock*; if the read lock were exclusive, the barrier would time out. It prints `true` (section 12.10).
- `Point` uses `StampedLock.tryOptimisticRead()`: read without locking, then `validate(stamp)`; fall back to a real read lock if a writer interfered (section 12.10).

Output:

```
=== Reentrancy ===
hold count inside nested call = 2
balance = 70
=== tryLock with timeout ===
worker: printer busy for 100 ms, doing something else instead
tryLock() now (free)  = true
=== ReadWriteLock: readers share ===
3 readers held the read lock at the same time: true
=== StampedLock optimistic read ===
distance = 5.0
```

Trade-offs

Feature	`synchronized`	`ReentrantLock`
Release	Automatic	Manual (`finally`) — easy to forget
Try / timeout	No	`tryLock()`, `tryLock(t, unit)`
Interruptible wait	No	`lockInterruptibly()`
Fairness	No	Optional (`true` = longest waiter first; lower throughput)
Conditions	One wait set	Many `Condition` objects
Reentrant	Yes	Yes

When not to use it

If you need none of the extra features, `synchronized` is shorter and can't be left locked by mistake.

Common mistakes

- `lock()` *inside* the `try` — if `lock()` threw, `finally` would `unlock()` a lock you don't hold (`IllegalMonitorStateException`). Call `lock()` just before `try`.
- Forgetting `unlock()` on an early `return` or exception path — the lock stays held forever.
- Turning on fairness "to be safe" — fair locks are noticeably slower; use them only when starvation is a real problem.

Real-world examples

`ArrayBlockingQueue`, `LinkedBlockingQueue` and `ThreadPoolExecutor` use `ReentrantLock` internally (`CopyOnWriteArrayList` did up to Java 8; since Java 9 it uses `synchronized` on an internal lock object). `ReentrantLock`, `ReentrantReadWriteLock`, `Semaphore` and `CountDownLatch` are all built on `AbstractQueuedSynchronizer` (AQS — the JDK's shared framework for building locks and synchronizers); `CyclicBarrier` is built on a `ReentrantLock` + `Condition`.

In the interview

30-second answer: "`ReentrantLock` is an explicit mutex with the same semantics as `synchronized` plus try-lock with timeout, interruptible acquisition, optional fairness, and multiple conditions. Reentrant means the owner can re-acquire it; the hold count must drop to zero to release. I always unlock in `finally`."

Follow-ups:

- *Why would a non-reentrant lock deadlock?* A method holding it that calls another method needing the same lock would wait for itself forever.
- *Is `StampedLock` reentrant?* No — e.g. a thread that holds its write lock and calls `writeLock()` or `readLock()` again blocks forever on itself. It also has no owner and no `Condition`s.

Self-check

- **Q:** A thread calls `lock()` 3 times and `unlock()` 2 times. Is the lock free? **A:** No; hold count is 1.

- **Q:** What does `new ReentrantLock(true)` change? **A:** Waiting threads get the lock in roughly FIFO (first in, first out) order (fair), at a throughput cost.

TL;DR

`ReentrantLock` = `synchronized` + tryLock/timeout/interruptible/fairness/conditions. Owner can re-enter. `lock(); try { … } finally { unlock(); }` always.

12.9 Try-lock & timed locking

What it is

- `tryLock()` — take the lock **only if it is free right now**; return `true`/`false` immediately.
- `tryLock(timeout, unit)` — wait up to the timeout; return `false` if still unavailable. Interruptible.

> **Analogy.** Knocking on a meeting room: if it's busy you don't stand there forever — you wait five minutes, then book another room.

The problem it solves

Plain `lock()` waits forever. That causes hung requests and makes deadlocks permanent. Try-lock lets a thread **give up and do something else** — fail fast, retry later, return "busy" to a user — and it breaks the *hold and wait* condition of deadlock (section 12.17).

How it works

1. Call `tryLock(timeout)`.
2. If it returns `true`: `try { work } finally { unlock(); }`.
3. If it returns `false`: take the fallback path (return an error, retry with **back-off** — waiting a short while before trying again — or release other locks you hold).
4. When acquiring several locks: if any `tryLock` fails, **release everything you already hold**, back off for a random short time, and retry.

Java example

In `locks/Main.java` above: the worker calls `printer.tryLock(100, TimeUnit.MILLISECONDS)` while `main` holds it, gets `false`, and prints the fallback message. In `deadlock/Main.java` (section 12.17), `transferTryLock` uses try-lock plus random back-off to avoid deadlock.

Trade-offs

Pros	Cons
No infinite waits; can report "busy"	Caller must handle failure paths
Breaks deadlock (hold-and-wait)	Retry loops without random back-off can **livelock**
Timeouts give predictable **latency** (time one operation takes)	Choosing the timeout is a judgement call

When not to use it

When failing is not acceptable and waiting is fine (e.g. a background **batch job** — one that processes data in bulk, with no user waiting — updating a counter) — plain `lock()` is simpler.

Common mistakes

- Calling `unlock()` when `tryLock` returned `false` → `IllegalMonitorStateException`.
- Retrying immediately in a tight loop — burns CPU and can livelock with another thread doing the same.
- Expecting untimed `tryLock()` to respect fairness — it *barges* (takes a free lock even if others are queued); `tryLock(0, unit)` respects fairness.

Real-world examples

- **Distributed locks** (locks shared by many servers, kept in an external store: Redis `SET key value NX PX 30000` — set only if absent, expire after 30,000 ms — Redisson's (a Java Redis client) `tryLock(wait, lease, unit)`,

ZooKeeper (a coordination service for clusters) recipes) follow the same idea: try for a bounded time, give up, or let the lock expire.

- Seat-hold features on ticketing sites: if the seat is locked by someone else's checkout, you're told immediately rather than made to wait.

In the interview

> **30-second answer:** "`tryLock` with a timeout lets a thread give up instead of waiting forever. I use it when latency matters or when I need several locks: if I can't get all of them, I release what I hold, back off randomly and retry — which prevents deadlock."

Follow-ups:

- *Why random back-off?* Two threads retrying on the same schedule can keep colliding forever (livelock); randomness breaks the symmetry.

Self-check

- **Q:** What does `tryLock(0, SECONDS)` do? **A:** Tries once without waiting but, unlike `tryLock()`, respects a fair lock's queue.

TL;DR

Try-lock = acquire only if available (optionally within a timeout). Handle `false`, release everything on failure, back off randomly.

12.10 Read-write locks: `ReadWriteLock` and `StampedLock`

What it is

A **read-write lock** is a pair of locks: a shared **read lock** (many readers at once) and an exclusive **write lock** (one writer, no readers). Java provides `ReentrantReadWriteLock` and, since Java 8, `StampedLock` (adds cheap *optimistic reads*).

> **Analogy.** A museum exhibit: any number of visitors can look at it together, but when a curator needs to rearrange it, the room is cleared and only the curator is inside.

The problem it solves

With a plain mutex, readers block each other even though reading doesn't change anything. For **read-heavy** data (configuration, **caches** — fast copies of data kept for quick reads — price catalogues) a read-write lock lets reads proceed in parallel.

How it works

1. Reader: `readLock().lock()` — succeeds if no writer holds the write lock. Many readers can hold it at once.
2. Writer: `writeLock().lock()` — waits until there are **no readers and no writer**, then holds exclusively.
3. `ReentrantReadWriteLock` rules: a writer may **downgrade** (acquire the read lock while holding the write lock, then release the write lock). A reader may **not upgrade** (asking for the write lock while holding the read lock deadlocks — it waits for all readers, including itself).
4. `StampedLock` optimistic read: `long s = tryOptimisticRead()` (no locking) → read fields into locals → `validate(s)`. If a writer acquired the lock in between, `validate` returns `false` and you retry with a real `readLock()`.

Java example

`ConfigStore` and `Point` in `locks/Main.java` above. The output line `3 readers held the read lock at the same time: true` proves readers share the lock.

Trade-offs

	`synchronized` / `ReentrantLock`	`ReentrantReadWriteLock`	`StampedLock`
Parallel readers	No	Yes	Yes (and optimistic reads take no lock at all)
Reentrant	Yes	Yes	**No**
Conditions	Yes	Write lock only	No
Overhead	Lowest	Higher bookkeeping	Low for optimistic reads; more complex code
Best when	Short sections or write-heavy	Reads ≫ writes and reads are not trivially short	Very read-heavy, tiny critical sections (e.g. coordinates)

When not to use it

- Write-heavy or very short critical sections — the extra bookkeeping of a RW lock can make it *slower* than a plain mutex.
- When a copy-on-write snapshot is simpler: keep an immutable map in a `volatile` field and replace it on write (readers never lock; see 12.23).

Common mistakes

- Trying to upgrade read → write in `ReentrantReadWriteLock` (self-deadlock).
- Modifying state while holding only the *read* lock.
- Using data read under an optimistic stamp before calling `validate`, or reading fields directly instead of copying them to locals first.
- Writer starvation in non-fair mode under a constant stream of readers (construct it with `new ReentrantReadWriteLock(true)` if it matters; `StampedLock` gives no fairness guarantee either).

Real-world examples

- In-memory caches and configuration holders; routing tables that are read on every request and updated rarely.
- `ConcurrentHashMap` goes further: reads take no lock at all.

In the interview

> **30-second answer:** "A read-write lock lets many readers in together but gives writers exclusive access. It helps when reads vastly outnumber writes. `ReentrantReadWriteLock` allows downgrade but not upgrade; `StampedLock` adds optimistic reads that take no lock and are validated afterwards, but it's not reentrant."

Follow-ups:

- *How would you design a thread-safe config service?* Either a RW lock around a `HashMap`, or an immutable `Map` in a `volatile` field replaced atomically on reload (readers never block).

Self-check

- **Q:** Two readers and one writer arrive; which can run together? **A:** The two readers; the writer runs alone.
- **Q:** Why copy fields to locals in an optimistic read? **A:** So that after `validate` succeeds you use a consistent snapshot, not fields that might change after the check.

TL;DR

Many readers OR one writer. Use for read-heavy shared data. No upgrade in `ReentrantReadWriteLock`; `StampedLock` = optimistic, non-reentrant.

12.11 Coarse-grained vs fine-grained locking

What it is

- **Coarse-grained locking**: one lock protects a large amount of state (e.g. the whole parking lot).

- **Fine-grained locking**: many locks, each protecting a small piece (e.g. one lock per floor, per spot, or per **hash bucket** — one slot of the internal array a hash map spreads its entries over).

> **Analogy.** One key for an entire office building (coarse) vs a key per room (fine). One key is simple but everyone queues at the front door; room keys let people work in parallel but you must manage many keys.

The problem it solves

Choosing the right granularity balances **simplicity and safety** against **throughput under contention**.

How it works

1. Start coarse: one lock around the whole object. Measure.
2. If contention is the bottleneck, split state into **independent** parts whose invariants don't cross — each gets its own lock (**lock splitting**).
3. For a big collection, use **lock striping**: N locks; item *i* uses lock `hash(i) % N` (Java 7's `ConcurrentHashMap` had 16 segments by default; chapter 13 builds a striped map).
4. When an operation needs several fine locks (e.g. booking seats A1 and A2), always acquire them in a **global order** to avoid deadlock.
5. Keep critical sections short: do slow work (I/O, computation) *outside* the lock.

	Coarse-grained	Fine-grained
Simplicity	High	Lower — more locks, ordering rules
Concurrency	Low: one thread at a time	High: threads on different parts run together
Deadlock risk	Low (one lock)	Higher (many locks) — needs lock ordering
Overhead	One lock	Memory and bookkeeping for many locks
Cross-part invariants	Easy	Hard — may need to take several locks

Java example

- Coarse: `SynchronizedCounter` (12.5).
- Fine: `Show` in `lldcases/Main.java` (12.25) has **one lock per seat**, taken in sorted order; `StripedHashMap` in chapter 13 has one lock per stripe.

Trade-offs

See table above. Rule of thumb: *correct first, then fine-grained only where **profiling** (measuring where a program really spends its time) shows contention*.

When not to use it

Don't split locks when an invariant spans the parts (e.g. "total seats booked ≤ capacity" across all rows) unless you also handle the cross-part check.

Common mistakes

- Premature fine-graining → complexity and deadlocks without real benefit.
- Locking fine-grained parts in inconsistent order.
- "Fine-grained" locks that still funnel through a shared counter everyone updates (the counter becomes the **bottleneck** — the one slow point that limits the whole system — use `LongAdder`, a counter that spreads updates over several cells, §12.15).

Real-world examples

- `ConcurrentHashMap` (Java 8+) locks only the head node of one bin for writes — extremely fine-grained.
- Databases use row-level locks (fine) vs table-level locks (coarse).
- Python's GIL (Global Interpreter Lock) is a famous coarse lock: simple, but one thread runs Python **bytecode** (the compiled instructions the interpreter executes) at a time.

In the interview

> **30-second answer:** "I start with one lock for correctness, then refine where contention matters: per-floor or per-spot locks for a parking lot, per-show or per-seat locks for booking, striping for maps. Finer locks raise throughput but need a consistent lock order to avoid deadlock."

Follow-ups:

- *Per-show or per-seat locking for BookMyShow (an Indian movie-ticketing site)?* Per-show is simple and usually enough (contention is per show). Per-seat allows more parallelism but multi-seat bookings must lock seats in sorted order.

Self-check

- **Q:** What is lock striping? **A:** Using a fixed array of N locks and mapping each item to one by hash, to get fine-grained locking with bounded lock count.

TL;DR

Coarse = simple, low throughput. Fine = parallel, complex, deadlock-prone. Start coarse, refine with measurements, and order your locks.

Part D — Coordination

12.12 Semaphores

What it is

A **semaphore** holds a number of **permits**. `acquire()` takes one (waiting if none are left); `release()` gives one back. `java.util.concurrent.Semaphore` implements it. A semaphore with 1 permit behaves like a mutex, *except* it has no owner — any thread may release it.

> **Analogy.** A car park with 3 spaces and a display showing free spaces. Cars enter only if the number is above zero; leaving increases it.

The problem it solves

- **Limiting concurrency**: at most N threads use a resource (DB connections, **API** calls — calls to another service's programming interface — GPU slots).
- **Signaling** between threads: a semaphore with 0 permits acts as "wait until someone signals" (used heavily in chapter 13).

How it works

1. `new Semaphore(n)` — n permits. The default is **non-fair**: a thread calling `acquire()` may *barge* ahead of threads already waiting. `new Semaphore(n, true)` is **fair**: permits go to waiting threads in first-in-first-out order (at some throughput cost). Even on a fair semaphore, the untimed `tryAcquire()` barges; `tryAcquire(0, TimeUnit.SECONDS)` respects fairness.
2. `acquire()` — if permits > 0, decrement and continue; else wait.
3. `release()` — increment and wake a waiter.
4. Also: `tryAcquire()`, `tryAcquire(timeout, unit)`, `acquire(k)` for several permits.
5. Always `release()` in `finally` after a successful `acquire()`.

Java example

```
// File: code/lld/ch12/coordination/Main.java
package lld.ch12.coordination;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.CyclicBarrier;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Semaphore;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

/** Semaphore, wait/notify, Condition, CountDownLatch, CyclicBarrier. */
public class Main {

    // ---------- Condition variable with the built-in monitor (wait / notifyAll) ----------
    /** A one-slot mailbox: put waits while full, take waits while empty. */
    static class Mailbox<T> {
        private T item;                                  // guarded by 'this'
        public synchronized void put(T x) throws InterruptedException {
            while (item != null) wait();                 // ALWAYS wait in a loop (spurious wakeups)
            item = x;
            notifyAll();                                 // wake the taker(s)
        }
        public synchronized T take() throws InterruptedException {
            while (item == null) wait();
            T x = item;
            item = null;
            notifyAll();                                 // wake the putter(s)
            return x;
        }
    }

    // ---------- Condition variable with ReentrantLock + Condition ----------
    /** Two threads take turns: PING, PONG, PING, PONG ... */
    static class TurnTable {
        private final ReentrantLock lock = new ReentrantLock();
        private final Condition turnChanged = lock.newCondition();
        private boolean pingsTurn = true;

        void play(boolean iAmPing, String word, int rounds, List<String> out) throws InterruptedException {
            for (int i = 0; i < rounds; i++) {
                lock.lock();
                try {
                    while (pingsTurn != iAmPing) turnChanged.await();   // releases lock while waiting
                    out.add(word);
                    pingsTurn = !pingsTurn;
                    turnChanged.signalAll();
                } finally {
                    lock.unlock();
                }
            }
        }
    }

    public static void main(String[] args) throws Exception {
        semaphoreDemo();
        waitNotifyDemo();
        conditionDemo();
        latchDemo();
        barrierDemo();
    }

    static void semaphoreDemo() throws InterruptedException {
        System.out.println("=== Semaphore: at most 3 concurrent DB connections ===");
        Semaphore connections = new Semaphore(3);
        AtomicInteger inUse = new AtomicInteger(), maxSeen = new AtomicInteger();
        ExecutorService pool = Executors.newFixedThreadPool(10);
        for (int i = 0; i < 10; i++) {
            pool.submit(() -> {
```

```
                try {
                    connections.acquire();                      // blocks when 0 permits left
                    try {
                        int now = inUse.incrementAndGet();
                        maxSeen.accumulateAndGet(now, Math::max);
                        Thread.sleep(30);                       // "run a query"
                        inUse.decrementAndGet();
                    } finally {
                        connections.release();                  // give the permit back
                    }
                } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            });
        }
        pool.shutdown();
        pool.awaitTermination(5, TimeUnit.SECONDS);
        System.out.println("10 queries done, never more than 3 at once: " + (maxSeen.get() <= 3));
    }

    static void waitNotifyDemo() throws InterruptedException {
        System.out.println("=== wait/notifyAll: one-slot mailbox ===");
        Mailbox<String> box = new Mailbox<>();
        List<String> received = new ArrayList<>();
        Thread consumer = new Thread(() -> {
            try { for (int i = 0; i < 3; i++) received.add(box.take()); }
            catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });
        consumer.start();
        for (String m : List.of("m1", "m2", "m3")) box.put(m);
        consumer.join();
        System.out.println("received in order: " + received);
    }

    static void conditionDemo() throws InterruptedException {
        System.out.println("=== Lock + Condition: strict turn-taking ===");
        TurnTable table = new TurnTable();
        List<String> out = java.util.Collections.synchronizedList(new ArrayList<>());
        Thread ping = new Thread(() -> { try { table.play(true, "PING", 3, out); } catch (InterruptedException ignored) {
} });
        Thread pong = new Thread(() -> { try { table.play(false, "PONG", 3, out); } catch (InterruptedException ignored) {
} });
        pong.start(); ping.start();             // start order does not matter
        ping.join(); pong.join();
        System.out.println(String.join(" ", out));
    }

    static void latchDemo() throws InterruptedException {
        System.out.println("=== CountDownLatch: wait for 3 services to start ===");
        CountDownLatch ready = new CountDownLatch(3);
        for (String svc : List.of("db", "cache", "search")) {
            new Thread(() -> { /* ...warm up... */ ready.countDown(); }, svc).start();
        }
        boolean allUp = ready.await(2, TimeUnit.SECONDS);   // one-shot: cannot be reset
        System.out.println("all services ready: " + allUp + " (count now " + ready.getCount() + ")");
    }

    static void barrierDemo() throws Exception {
        System.out.println("=== CyclicBarrier: 3 workers, 2 phases ===");
        AtomicInteger phase = new AtomicInteger();
        // The barrier action runs once per trip, by the last thread to arrive.
        CyclicBarrier barrier = new CyclicBarrier(3,
                () -> System.out.println("phase " + phase.incrementAndGet() + " complete: all 3 workers arrived"));
        List<Thread> ws = new ArrayList<>();
        for (int i = 0; i < 3; i++) {
            Thread t = new Thread(() -> {
                try {
                    for (int p = 0; p < 2; p++) {
                        Thread.sleep((long) (Math.random() * 20)); // uneven work
                        barrier.await();                           // reusable: works for every phase
                    }
                } catch (Exception e) { throw new RuntimeException(e); }
            });
            ws.add(t);
            t.start();
        }
        for (Thread t : ws) t.join();
```

```
    }
}
```

Explaining each part:

- `semaphoreDemo`: 10 tasks, 3 permits; `maxSeen` records the peak number of concurrent users and never exceeds 3.
- `Mailbox` is a condition variable built on `wait/notifyAll` (section 12.13).
- `TurnTable` is a condition variable built on `ReentrantLock` + `Condition` (section 12.13).
- `latchDemo` / `barrierDemo`: section 12.14.

Output:

```
=== Semaphore: at most 3 concurrent DB connections ===
10 queries done, never more than 3 at once: true
=== wait/notifyAll: one-slot mailbox ===
received in order: [m1, m2, m3]
=== Lock + Condition: strict turn-taking ===
PING PONG PING PONG PING PONG
=== CountDownLatch: wait for 3 services to start ===
all services ready: true (count now 0)
=== CyclicBarrier: 3 workers, 2 phases ===
phase 1 complete: all 3 workers arrived
phase 2 complete: all 3 workers arrived
```

Trade-offs

Pros	Cons
Simple way to cap concurrency	No owner: a bug can release more than acquired (permits grow)
Works as a signal (0 permits)	Doesn't protect data by itself — only counts

When not to use it

Don't use a semaphore as a mutex around shared data when a lock would do — locks have ownership checks and reentrancy.

Common mistakes

- Releasing in a path where `acquire` never succeeded (e.g. `acquire` interrupted) → permit leak upward.
- Forgetting to release on exceptions → permits slowly disappear, system stalls.

Real-world examples

- **Bulkhead pattern** (isolating resources per dependency, like watertight compartments in a ship, so one failure can't sink everything): Resilience4j's `SemaphoreBulkhead` (and Hystrix's — Netflix's older fault-tolerance library — semaphore isolation) cap concurrent calls to a dependency.
- **Connection pools** (sets of open database connections reused across requests) bound the number of connections handed out (HikariCP, a popular JDBC pool, enforces its maximum pool size).
- With virtual threads, a `Semaphore` is the recommended way to limit calls to a slow **downstream** service (one that your code calls).

In the interview

30-second answer: "A semaphore is a permit counter: acquire takes a permit or waits, release returns one. I use it to cap concurrent access to a limited resource — a bulkhead — or with zero initial permits as a signal from one thread to another. Unlike a lock it has no owner."

Follow-ups:

- *Binary semaphore vs mutex?* Both allow one holder, but a mutex has an owner (only the holder may unlock, and it's usually reentrant); a binary semaphore can be released by any thread and isn't reentrant.

Self-check

- **Q:** `new Semaphore(0)`: what does `acquire()` do? **A:** Waits until some thread calls `release()` — a signal.

TL;DR

Semaphore = N permits. Cap concurrency or signal. No ownership, so release carefully in `finally`.

12.13 Condition variables: `wait/notify` and `Condition`

What it is

A **condition variable** lets a thread *sleep until some condition about shared state becomes true*, releasing the lock while it sleeps, and lets another thread *wake it* after changing that state.

- Built-in: `Object.wait()`, `notify()`, `notifyAll()` — used inside `synchronized` on the same object.
- Explicit: `Condition c = lock.newCondition(); c.await(); c.signal(); c.signalAll();` — used while holding that `ReentrantLock`.

> **Analogy.** A restaurant pager: you (the thread) hand back your table (release the lock) and wait. When the table is ready, staff buzz the pager (notify). You then come back and check that the table really is free (re-check the condition), because someone might have taken it.

The problem it solves

Waiting for a state change without **busy-waiting** (spinning in a loop wasting CPU) and without missing the moment it happens.

How it works

The canonical pattern — memorize it:

```
synchronized (lock) {                // or: lock.lock(); try {
    while (!conditionHolds()) {      // ALWAYS a while loop, never if
        lock.wait();                 //   cond.await();  releases the lock while waiting
    }
    doAction();                      // condition is true AND we hold the lock
    lock.notifyAll();                //   cond.signal()/signalAll() if we changed state others wait for
}                                    // } finally { lock.unlock(); }
```

1. Acquire the lock.
2. While the condition is false, call `wait()` / `await()` — this **atomically releases the lock** and sleeps.
3. When woken, the thread **re-acquires the lock** before returning, then re-checks the condition (loop).
4. After changing state, call `notifyAll()` / `signal()` so waiters re-check.

Why the `while` loop? (a) **Spurious wakeups** — the JLS and the `Condition` docs allow a thread to wake without being notified. (b) Another thread may have grabbed the state between the notify and your re-acquisition. (c) With `notifyAll`, several threads wake but only one can succeed.

`notify` vs `notifyAll`: `notify()` wakes one arbitrary waiter. If different kinds of threads (producers and consumers) wait on the same monitor, `notify()` can wake the wrong kind and everyone sleeps forever. Use `notifyAll()` with a single monitor, or use separate `Condition`s (`notFull`, `notEmpty`) and targeted `signal()`.

Java example

`Mailbox` (monitor version) and `TurnTable` (Lock + Condition) in `coordination/Main.java` above. Output: `received in order: [m1, m2, m3]` and `PING PONG PING PONG PING PONG`.

Trade-offs

	`wait/notify`	`Lock + Condition`
Conditions per lock	One wait set	Many (e.g. `notFull`, `notEmpty`)
Targeted wake-up	No — use `notifyAll`	Yes — `signal()` on the right condition
Timed wait	`wait(ms)`	`await(t, unit)`, `awaitNanos` (returns time left)
Verbosity	Less	More (manual unlock)

When not to use it

Most of the time you don't need raw condition variables: `BlockingQueue`, `Semaphore`, `CountDownLatch`, `CompletableFuture` already solve the common waiting problems. Write your own only for custom coordination (interview problems, custom synchronizers).

Common mistakes

- `if` instead of `while` around `wait()`.
- Calling `wait()`/`notify()` without holding that object's monitor → `IllegalMonitorStateException`.
- Changing state *without* notifying → waiters sleep forever (a "lost wakeup").
- Calling `wait()` on one object while synchronized on another.

Real-world examples

`ArrayBlockingQueue` uses one `ReentrantLock` with two `Condition`s, `notEmpty` and `notFull` — exactly the pattern built by hand in chapter 13.

In the interview

> **30-second answer:** "A condition variable lets a thread release a lock and sleep until another thread changes the state and signals it. I always wait in a while loop that re-checks the condition, because of spurious wakeups and competing threads. With one monitor I use notifyAll; with ReentrantLock I use separate Conditions and targeted signal."

Follow-ups:

- *Why must `wait()` be called while holding the lock?* So the check-then-sleep is atomic; otherwise a notify could happen between your check and your wait and be lost.

Self-check

- **Q:** What does `wait()` do with the monitor? **A:** Releases it while sleeping and re-acquires it before returning.

TL;DR

Condition variable = "release lock, sleep until signalled, re-acquire, re-check". Always `while`, prefer `notifyAll` or separate `Condition`s.

12.14 `CountDownLatch` and `CyclicBarrier`

What it is

- **`CountDownLatch(n)`**: a one-time gate. Threads call `await()` and wait until `countDown()` has been called *n* times. It cannot be reset.
- **`CyclicBarrier(n, action)`**: a reusable meeting point. Each of *n* threads calls `await()`; when the *n*th arrives, the optional barrier action runs once, all are released, and the barrier resets for the next round.

> **Analogy.** Latch = a race starter who waits for all 3 judges to say "ready" and then fires the pistol once. Barrier = hikers who agree to wait for each other at every checkpoint before continuing.

The problem it solves

Coordinating *phases*: "start only after all services are up" (latch), "all workers finish step 1 before anyone starts step 2" (barrier), and "release all test threads at the same instant" (latch with count 1 — used throughout this book to maximise contention in tests).

How it works

	CountDownLatch	CyclicBarrier
Who waits	Any thread(s) calling `await()`	The *n* participating threads themselves
Who counts	Anyone calling `countDown()` (need not wait)	Arrival at `await()`
Reusable	No	Yes (auto-reset after each trip)
Extra	Count reaching 0 is permanent: later `await()` calls return at once. One thread may call `countDown()` several times	Barrier action runs once per trip; if one party is interrupted/times out, the barrier **breaks** and others get `BrokenBarrierException` (`reset()` repairs it)
Happens-before	Actions before `countDown()` HB actions after the matching `await()` returns	Actions before `await()` HB the barrier action, which HB actions after the other parties return from `await()`

`Phaser` (Java 7) generalises both: parties can register/deregister dynamically across many phases.

Java example

`latchDemo` and `barrierDemo` in `coordination/Main.java` above. Output shows `all services ready: true (count now 0)` and `phase 1 complete…`, `phase 2 complete…`.

Trade-offs

Latch: dead simple, one-shot. Barrier: reusable but fragile — one failing party breaks it for all.

When not to use it

For "wait for these N tasks and collect results", `ExecutorService.invokeAll` or `CompletableFuture.allOf(...)` is usually clearer.

Common mistakes

- Waiting on a latch forever because a worker threw before `countDown()` — put `countDown()` in `finally`, and use `await(timeout)`.
- Using a barrier with fewer threads than parties → everyone waits forever.

Real-world examples

- Test harnesses start N threads and release them together with a `CountDownLatch(1)` to maximise races (JCStress — OpenJDK's harness for testing concurrency correctness — uses stronger tools, but the idea is the same).
- Application start-up: wait for caches, DB pools, and message consumers to be ready before accepting traffic.

In the interview

> **30-second answer:** "`CountDownLatch` is a one-shot gate that opens after N countDowns; `CyclicBarrier` is a reusable rendezvous where N threads wait for each other each phase, optionally running an action when all arrive."

Follow-ups:

- *Can a latch be reset?* No — create a new one or use `CyclicBarrier`/`Phaser`.

Self-check

- **Q:** Which one would you use so 3 worker threads sync at the end of every simulation step? **A:** `CyclicBarrier(3)`.

TL;DR

Latch = one-shot "wait for N events". Barrier = reusable "wait for N threads". Put `countDown` in `finally`; use timeouts.

Part E — Lock-free

12.15 Compare-and-swap (CAS) & atomics

What it is

Compare-and-swap is a single CPU instruction (e.g. `LOCK CMPXCHG` on x86; LL/SC — load-linked/store-conditional — or CAS instructions on ARM): "if the value at this address is still `expected`, replace it with `new` and report success; otherwise change nothing and report failure." Java exposes it through `java.util.concurrent.atomic` classes: `AtomicInteger`, `AtomicLong`, `AtomicBoolean`, `AtomicReference`, `AtomicIntegerArray`, `LongAdder`, and through `VarHandle`.

> **Analogy.** Editing a shared wiki page: you save only if the page version is still the one you started from. If someone saved first, your save is rejected; you reload and try again.

The problem it solves

Atomic updates **without locks** (**lock-free**: no thread ever blocks another, and some thread always makes progress): no blocking, no deadlock, and usually faster under low-to-moderate contention. It is how lock-free data structures and the JDK's own concurrency utilities are built.

How it works

The **CAS retry loop**:

1. Read the current value.
2. Compute the new value from it.
3. `compareAndSet(current, new)`. If it succeeds, done.
4. If it fails, another thread changed the value — go back to step 1.

Helpers do the loop for you: `incrementAndGet`, `addAndGet`, `updateAndGet(fn)`, `accumulateAndGet(x, fn)` (the function may run more than once, so it must be side-effect-free).

`LongAdder` spreads updates over several internal cells when contended and sums them on `sum()` — much faster than `AtomicLong` for hot counters (metrics), but `sum()` is not an atomic snapshot.

ABA problem. A value goes A → B → A. A thread that read A earlier sees A again and its CAS succeeds, although the state changed in between. This matters for pointer-based structures (e.g. reusing nodes in a lock-free stack). Fix: attach a version — `AtomicStampedReference` (value + int stamp) or `AtomicMarkableReference`. (In Java, garbage collection prevents the classic node-reuse ABA for freshly allocated nodes, which is why the Treiber stack below is safe.)

Java example

```java
// File: code/lld/ch12/atomics/Main.java
package lld.ch12.atomics;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicReference;
import java.util.concurrent.atomic.AtomicStampedReference;
import java.util.concurrent.atomic.LongAdder;

/** Compare-and-swap (CAS), a lock-free stack, LongAdder, and the ABA problem. */
public class Main {

    /** Increment written by hand as a CAS retry loop (this is what incrementAndGet does inside). */
    static int casIncrement(AtomicInteger a) {
        while (true) {
            int current = a.get();                    // 1. read
            int next = current + 1;                   // 2. compute
            if (a.compareAndSet(current, next)) {     // 3. write only if nobody changed it meanwhile
                return next;
            }                                         // 4. someone won the race: retry
        }
    }

    /** Treiber stack: a classic lock-free data structure built on one AtomicReference. */
    static class LockFreeStack<T> {
        private record Node<T>(T value, Node<T> next) { }
        private final AtomicReference<Node<T>> head = new AtomicReference<>();

        void push(T v) {
            Node<T> oldHead, newHead;
            do {
                oldHead = head.get();
                newHead = new Node<>(v, oldHead);
            } while (!head.compareAndSet(oldHead, newHead));
        }
        T pop() {
            Node<T> oldHead;
            do {
                oldHead = head.get();
                if (oldHead == null) return null;
            } while (!head.compareAndSet(oldHead, oldHead.next()));
            return oldHead.value();
        }
    }

    static void runThreads(int n, Runnable r) throws InterruptedException {
        List<Thread> ts = new ArrayList<>();
        for (int i = 0; i < n; i++) { Thread t = new Thread(r); ts.add(t); t.start(); }
        for (Thread t : ts) t.join();
    }

    public static void main(String[] args) throws Exception {
        AtomicInteger counter = new AtomicInteger();
        runThreads(4, () -> { for (int i = 0; i < 100_000; i++) casIncrement(counter); });
        System.out.println("hand-written CAS loop counter = " + counter.get());

        AtomicInteger highWater = new AtomicInteger();
        runThreads(4, () -> {
            for (int i = 0; i < 1_000; i++) highWater.accumulateAndGet(i, Math::max);  // atomic "max"
        });
        System.out.println("atomic max                    = " + highWater.get());

        LockFreeStack<Integer> stack = new LockFreeStack<>();
        runThreads(4, () -> { for (int i = 0; i < 10_000; i++) stack.push(i); });
        int popped = 0;
        while (stack.pop() != null) popped++;
        System.out.println("lock-free stack popped        = " + popped);

        LongAdder hits = new LongAdder();             // striped counter: fast under heavy contention
        runThreads(8, () -> { for (int i = 0; i < 100_000; i++) hits.increment(); });
        System.out.println("LongAdder sum                 = " + hits.sum());
```

```
        // ABA: value goes A -> B -> A; a plain CAS cannot tell anything happened.
        AtomicReference<String> ref = new AtomicReference<>("A");
        String seen = ref.get();
        ref.set("B"); ref.set("A");                    // another thread's changes
        System.out.println("plain CAS after A->B->A succeeds   : " + ref.compareAndSet(seen, "C"));

        AtomicStampedReference<String> sref = new AtomicStampedReference<>("A", 0);
        int[] stampHolder = new int[1];
        String seen2 = sref.get(stampHolder);
        int seenStamp = stampHolder[0];
        sref.set("B", seenStamp + 1); sref.set("A", seenStamp + 2);
        System.out.println("stamped CAS after A->B->A succeeds : " + sref.compareAndSet(seen2, "C", seenStamp, seenStamp +
1));
    }
}
```

Explaining each part:

- `casIncrement` is the textbook retry loop — this is what `incrementAndGet` does conceptually.
- `accumulateAndGet(i, Math::max)` atomically tracks a maximum.
- `LockFreeStack` (the *Treiber stack*): push/pop swing the `head` reference with CAS; a failed CAS means another thread pushed/popped first — retry.
- `LongAdder` counts 800,000 increments from 8 threads with little contention.
- The ABA lines show a plain CAS succeeding after A→B→A, while the stamped CAS fails because the stamp moved from 0 to 2.

Output:

```
hand-written CAS loop counter = 400000
atomic max                    = 999
lock-free stack popped        = 40000
LongAdder sum                 = 800000
plain CAS after A->B->A succeeds   : true
stamped CAS after A->B->A succeeds : false
```

Trade-offs

	Locks	CAS / atomics
Blocking	Yes	No (threads retry)
Deadlock	Possible	Impossible (no locks)
Scope	Any number of variables	One variable/reference per CAS
Under heavy contention	Threads sleep (saves CPU)	Many failed retries (wastes CPU) — use `LongAdder` or striping
Difficulty	Moderate	Hard for anything beyond counters/flags/one reference

When not to use it

Invariants spanning several variables (move money between two accounts) — a single CAS can't cover them; use a lock, or put all fields into one immutable object and CAS the reference (as the lock-free token bucket in chapter 13 does).

Common mistakes

- Separate `get()` then `set()` on an atomic → still a race. Use `compareAndSet`/`updateAndGet`.
- Side effects inside `updateAndGet` lambdas (they may run several times).
- Using `AtomicLong` for a very hot metrics counter where `LongAdder` would scale better.

Real-world examples

- `ConcurrentHashMap` inserts into empty bins with CAS; `ConcurrentLinkedQueue` is a lock-free (Michael–Scott) queue; `AbstractQueuedSynchronizer` (behind `ReentrantLock`) uses CAS on its state.
- Optimistic locking in databases and ORMs (Object-Relational Mappers — libraries that map objects to database tables; JPA `@Version`, `UPDATE ... WHERE version = ?`) is CAS at the database level.
- LMAX Disruptor (a high-performance inter-thread messaging library) uses CAS-based sequences for its lock-free **ring buffer** (a fixed-size array used as a circular queue).

In the interview

> **30-second answer:** "CAS atomically updates a value only if it still equals what I read; otherwise I retry. Java's atomic classes wrap it, so counters, flags and single references can be updated without locks and without deadlock. It covers one variable at a time, can suffer ABA — fixed with a stamped reference — and under heavy contention I prefer `LongAdder`."

Follow-ups:

- *Is lock-free always faster?* No. Under heavy contention retries waste CPU; a lock that parks threads can be better.
- *How would you do a lock-free update of two fields?* Put them in an immutable object and CAS an `AtomicReference` to it.

Self-check

- **Q:** What does a failed `compareAndSet` mean? **A:** Another thread changed the value after you read it; re-read and retry.
- **Q:** What fixes ABA? **A:** A version stamp: `AtomicStampedReference`.

TL;DR

CAS = "update only if unchanged", retried in a loop. Atomics give lock-free, deadlock-free updates to one variable. Watch ABA; use `LongAdder` for hot counters.

12.16 Concurrent collections

What it is

Thread-safe collections in `java.util.concurrent` designed for high concurrency (unlike the old fully-`synchronized` `Vector` / `Hashtable`):

Need	Use	Key behaviour
Map	`ConcurrentHashMap`	Lock-free reads; writes lock only one bin; atomic `putIfAbsent`, `computeIfAbsent`, `compute`, `merge`; **no null keys/values**; *weakly consistent* iterators (see below)
Sorted map/set	`ConcurrentSkipListMap` / `Set`	Lock-free, sorted, O(log n) (cost grows very slowly with size)
List read ≫ write	`CopyOnWriteArrayList` / `Set`	Each write copies the whole array; iteration uses a snapshot (never throws CME, `ConcurrentModificationException`)
Queue (non-blocking)	`ConcurrentLinkedQueue` / `Deque`	Lock-free; `poll()` returns null when empty
Queue (blocking)	`ArrayBlockingQueue` (bounded), `LinkedBlockingQueue` (optionally bounded), `PriorityBlockingQueue`, `DelayQueue`, `SynchronousQueue` (no capacity: hand-off), `LinkedTransferQueue`	`put` waits when full, `take` waits when empty; `offer` / `poll` with timeouts

Two `ConcurrentHashMap` guarantees worth quoting exactly:

- **`computeIfAbsent` / `compute` / `merge` are atomic.** The Javadoc says the whole call is performed atomically, and for `computeIfAbsent` the mapping function is invoked **exactly once if the key is absent, otherwise not at all** — so two racing threads can never both create a value for the same key. Some updates by other threads (at least those hitting the same bin) block while the function runs, so keep it short and never touch the same map inside it. (The default `ConcurrentMap.computeIfAbsent` used by some other maps, e.g. `ConcurrentSkipListMap`, may call the function more than once and discard results — only the final put is atomic.)
- **Iterators are weakly consistent.** They never throw `ConcurrentModificationException`, may proceed while other threads modify the map, return each element at most once, and reflect the map's state at some point at or

after the iterator was created — they *may or may not* show later updates. `size()` / `isEmpty()` are likewise only estimates while updates are in flight.

> **Analogy.** A supermarket with many tills (ConcurrentHashMap's bins) instead of one till for the whole store (`Hashtable`).

The problem it solves

Wrapping a `HashMap` in one lock **serialises** every access (forces accesses to happen one at a time). Concurrent collections let many threads read and write different parts at the same time and offer **atomic compound operations** so you don't need external locks for common patterns.

How it works

1. Pick the collection by access pattern (table above).
2. Replace check-then-act sequences with a single atomic method:
   - `if (!m.containsKey(k)) m.put(k, v)` → `m.putIfAbsent(k, v)`
   - `m.put(k, m.get(k) + 1)` → `m.merge(k, 1, Integer::sum)`
   - lazy creation → `m.computeIfAbsent(k, key -> new ...)`
3. For hand-offs between threads, use a `BlockingQueue` (producer-consumer, 12.22).
4. Remember *compound operations across several calls are still not atomic.*

Java example

```java
// File: code/lld/ch12/collections/Main.java
package lld.ch12.collections;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.TimeUnit;

/** ConcurrentHashMap atomic compound ops, CopyOnWriteArrayList snapshots, BlockingQueue. */
public class Main {

    public static void main(String[] args) throws Exception {
        // ---------- ConcurrentHashMap ----------
        ConcurrentHashMap<String, Integer> wordCount = new ConcurrentHashMap<>();
        String[] words = {"apple", "banana", "apple", "cherry", "banana", "apple"};
        List<Thread> ts = new ArrayList<>();
        for (int t = 0; t < 4; t++) {
            Thread th = new Thread(() -> {
                for (int rep = 0; rep < 1_000; rep++)
                    for (String w : words) wordCount.merge(w, 1, Integer::sum);   // atomic per key
            });
            ts.add(th); th.start();
        }
        for (Thread th : ts) th.join();
        System.out.println("word counts (4 threads x 1000 reps): " + new TreeMap<>(wordCount));

        // computeIfAbsent: create-once per key, even under races (e.g. one lock/queue per user).
        ConcurrentHashMap<String, List<String>> ordersByUser = new ConcurrentHashMap<>();
        ordersByUser.computeIfAbsent("alice", k -> new CopyOnWriteArrayList<>()).add("order-1");
        ordersByUser.computeIfAbsent("alice", k -> new CopyOnWriteArrayList<>()).add("order-2");
        System.out.println("alice's orders: " + ordersByUser.get("alice"));

        // WRONG pattern (check-then-act across two calls) vs RIGHT (one atomic call):
        //   if (!map.containsKey(k)) map.put(k, v);   // two threads can both put
        //   map.putIfAbsent(k, v);                    // atomic
        Map<String, String> seats = new ConcurrentHashMap<>();
        System.out.println("first putIfAbsent returns : " + seats.putIfAbsent("A1", "bob"));
        System.out.println("second putIfAbsent returns: " + seats.putIfAbsent("A1", "eve") + "  (seat already bob's)");

        // ---------- CopyOnWriteArrayList ----------
        CopyOnWriteArrayList<String> listeners = new CopyOnWriteArrayList<>(List.of("L1", "L2"));
        int seen = 0;
        for (String l : listeners) {          // iterates over a snapshot
            listeners.add(l + "-new");        // no ConcurrentModificationException
            seen++;
        }
        System.out.println("iterated " + seen + " items; list now " + listeners);

        // ---------- BlockingQueue ----------
        BlockingQueue<Integer> q = new ArrayBlockingQueue<>(2);   // bounded = back-pressure
        System.out.println("offer 1: " + q.offer(1) + ", offer 2: " + q.offer(2) + ", offer 3: " + q.offer(3) + "
(full)");
        System.out.println("take -> " + q.take() + ", poll -> " + q.poll() + ", poll(50ms) on empty -> "
                + q.poll(50, TimeUnit.MILLISECONDS));
    }
}
```

Explaining each part:

- `merge(w, 1, Integer::sum)` is an atomic per-key increment; totals are exact.
- `computeIfAbsent` creates each per-user list once, even under races.
- `putIfAbsent` returns `null` for the winner and the existing value for losers — perfect for "claim this seat".
- Iterating a `CopyOnWriteArrayList` while adding to it sees the old snapshot (2 items) and throws no exception.
- `ArrayBlockingQueue(2)`: `offer` returns `false` when full; `poll(50ms)` returns `null` when empty.

Output:

```
word counts (4 threads x 1000 reps): {apple=12000, banana=8000, cherry=4000}
alice's orders: [order-1, order-2]
first putIfAbsent returns : null
second putIfAbsent returns: bob  (seat already bob's)
iterated 2 items; list now [L1, L2, L1-new, L2-new]
offer 1: true, offer 2: true, offer 3: false (full)
take -> 1, poll -> 2, poll(50ms) on empty -> null
```

Trade-offs

Collection	Great for	Bad for
`ConcurrentHashMap`	General shared maps, caches, counters	Multi-key atomic operations; `size()` is only an estimate under concurrent updates
`CopyOnWriteArrayList`	Listener lists, rarely-changed config	Frequent writes (O(n) copy each)
`Collections.synchronizedMap`	Legacy code, low contention	High contention; iteration still needs manual `synchronized(map)`
`ArrayBlockingQueue`	Bounded hand-off with back-pressure	Very high throughput (single lock for put/take)
`LinkedBlockingQueue`	Higher throughput (separate put/take locks)	Default constructor is **unbounded** (capacity `Integer.MAX_VALUE`)

When not to use it

If an object is confined to one thread, a plain `HashMap`/`ArrayList` is faster. If you need a transaction across several keys, a single lock (or a database) is clearer.

Common mistakes

- Doing slow work or calling back into the same map inside `computeIfAbsent`/`compute` — the function runs while the bin is locked; recursive updates can throw `IllegalStateException` or deadlock.
- Putting `null` in a `ConcurrentHashMap` → `NullPointerException`.
- Unbounded queues that silently grow until `OutOfMemoryError`.

Real-world examples

- Caffeine's (a Java caching library) cache is built on `ConcurrentHashMap`; Guava's (Google's core Java library) `LocalCache` uses its own segmented design derived from Java 7's `ConcurrentHashMap`.
- Guava's `EventBus` keeps each event type's subscribers in a `CopyOnWriteArraySet`; listener lists in general are the textbook copy-on-write use case.
- Log4j2's async **appenders** (the components that write log events to a file or other destination) and many logging frameworks hand events from app threads to a writer thread through queues.

In the interview

30-second answer: "I use `ConcurrentHashMap` for shared maps and its atomic methods — `putIfAbsent`, `computeIfAbsent`, `merge` — instead of check-then-act. `CopyOnWriteArrayList` for read-mostly lists like listeners. `BlockingQueue`, preferably bounded, to hand work between threads."

Follow-ups:

- *How does Java 8+ `ConcurrentHashMap` achieve concurrency?* Reads are lock-free (volatile reads). Writes CAS into empty bins, otherwise `synchronized` on the bin's first node; long bins become **red-black trees** (self-balancing sorted binary trees); resizing is done cooperatively by several threads. (Java 7 used 16 lock-striped segments by default.)
- *Why no nulls?* `get(k) == null` must unambiguously mean "absent"; with concurrency you can't follow up with `containsKey` atomically.

Self-check

- **Q:** Atomic way to count words in a shared map? **A:** `map.merge(word, 1, Integer::sum)`.

- **Q:** Why doesn't iterating a `CopyOnWriteArrayList` throw CME? **A:** It iterates an immutable snapshot of the array.

TL;DR

`ConcurrentHashMap` + atomic methods for maps, COW lists for read-mostly lists, bounded `BlockingQueue` for hand-offs. Single calls are atomic; sequences of calls are not.

Part F — Liveness failures

12.17 Deadlock

What it is

A **deadlock** is when two or more threads each wait forever for a resource another one holds. Nobody can proceed.

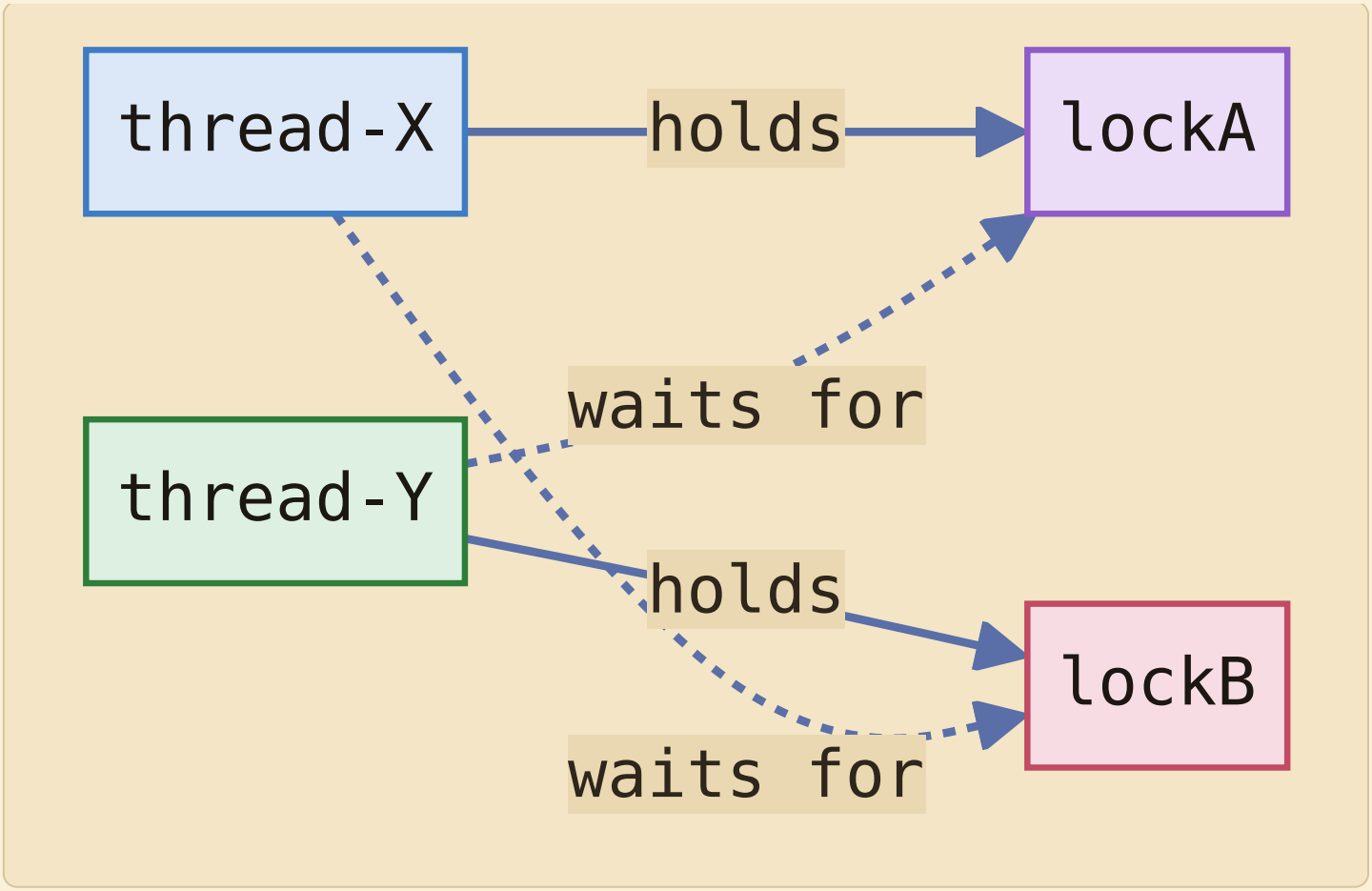
> **Analogy.** Two cars meet on a one-lane bridge from opposite ends. Each waits for the other to back up. Neither does.

The problem it solves

(Understanding deadlock lets you *design it out*.) A deadlock requires all four **Coffman conditions** (Coffman et al., 1971) at once:

#	Condition	Meaning	How to break it
1	**Mutual exclusion**	A resource can be held by only one thread	Use lock-free/immutable data, or shareable resources (read locks)
2	**Hold and wait**	A thread holds one resource while waiting for another	Acquire all resources at once, or use `tryLock` and release everything on failure
3	**No preemption**	Resources can't be taken away; only the holder releases	Timeouts (`tryLock(timeout)`), lock **leases** (locks that expire automatically after a set time)
4	**Circular wait**	A cycle: T1 waits for T2's lock, T2 waits for T1's	**Global lock ordering** (always lock lower id first)

Break **any one** condition and deadlock is impossible.



How it works

1. **Prevention by lock ordering (most common):** define a total order over locks (account id, seat id, `System.identityHashCode` with a tie-breaker lock) and always acquire in that order.
2. **Prevention by try-lock:** `tryLock(timeout)` each lock; if one fails, release all held locks, wait a random short time, retry.
3. **Avoid nested locks** where possible: call alien code (listeners, callbacks) *outside* your lock.
4. **Detection:** thread dumps (`jstack` / `jcmd`) print "Found one Java-level deadlock"; programmatically, `ThreadMXBean.findDeadlockedThreads()` (`ThreadMXBean` is the JDK's management interface for inspecting threads).
5. **Avoidance** (Banker's algorithm — grant a request only if the system can still finish every thread) exists in OS theory but is rarely used in application code.

Java example

```
// File: code/lld/ch12/deadlock/Main.java
package lld.ch12.deadlock;

import java.lang.management.ManagementFactory;
import java.lang.management.ThreadInfo;
import java.lang.management.ThreadMXBean;
import java.util.Arrays;
import java.util.concurrent.CyclicBarrier;
import java.util.concurrent.ThreadLocalRandom;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.locks.ReentrantLock;

/** Create a real deadlock, detect it, then fix it two ways: lock ordering and tryLock + back-off. */
public class Main {

    static class Account {
        final int id;
        final ReentrantLock lock = new ReentrantLock();
        int balance;
        Account(int id, int balance) { this.id = id; this.balance = balance; }
    }

    // ---------- Fix 1: global lock ordering (always lock the smaller id first) ----------
    static void transferOrdered(Account from, Account to, int amount) {
        Account first = from.id < to.id ? from : to;
        Account second = from.id < to.id ? to : from;
        first.lock.lock();
        try {
            second.lock.lock();
            try {
                if (from.balance >= amount) { from.balance -= amount; to.balance += amount; }
            } finally { second.lock.unlock(); }
        } finally { first.lock.unlock(); }
    }

    // ---------- Fix 2: tryLock with timeout + random back-off (breaks "hold and wait") ----------
    static void transferTryLock(Account from, Account to, int amount) throws InterruptedException {
        while (true) {
            if (from.lock.tryLock(1, TimeUnit.MILLISECONDS)) {
                try {
                    if (to.lock.tryLock(1, TimeUnit.MILLISECONDS)) {
                        try {
                            if (from.balance >= amount) { from.balance -= amount; to.balance += amount; }
                            return;
                        } finally { to.lock.unlock(); }
                    }
                } finally { from.lock.unlock(); }         // give up what we hold
            }
            // Random back-off: without it, both threads could retry in lock-step forever (LIVELOCK).
            Thread.sleep(0, ThreadLocalRandom.current().nextInt(1, 100_000));
        }
    }

    public static void main(String[] args) throws Exception {
        demonstrateDeadlock();

        Account a = new Account(1, 1_000), b = new Account(2, 1_000);
        Thread t1 = new Thread(() -> { for (int i = 0; i < 50_000; i++) transferOrdered(a, b, 1); });
        Thread t2 = new Thread(() -> { for (int i = 0; i < 50_000; i++) transferOrdered(b, a, 1); });
        t1.start(); t2.start(); t1.join(); t2.join();
        System.out.println("lock ordering: finished, total money = " + (a.balance + b.balance));

        Account c = new Account(3, 1_000), d = new Account(4, 1_000);
        Thread t3 = new Thread(() -> { try { for (int i = 0; i < 5_000; i++) transferTryLock(c, d, 1); } catch
(InterruptedException ignored) { } });
        Thread t4 = new Thread(() -> { try { for (int i = 0; i < 5_000; i++) transferTryLock(d, c, 1); } catch
(InterruptedException ignored) { } });
        t3.start(); t4.start(); t3.join(); t4.join();
        System.out.println("tryLock + back-off: finished, total money = " + (c.balance + d.balance));
    }

    /** Two threads lock in opposite order. A barrier forces the bad interleaving every time. */
    static void demonstrateDeadlock() throws Exception {
```

```
        Object lockA = new Object(), lockB = new Object();
        CyclicBarrier bothHoldOne = new CyclicBarrier(2);
        Thread x = new Thread(() -> {
            synchronized (lockA) { await(bothHoldOne); synchronized (lockB) { System.out.println("never printed"); } }
        }, "thread-X");
        Thread y = new Thread(() -> {
            synchronized (lockB) { await(bothHoldOne); synchronized (lockA) { System.out.println("never printed"); } }
        }, "thread-Y");
        x.setDaemon(true); y.setDaemon(true);   // daemon threads do not keep the JVM alive
        x.start(); y.start();

        ThreadMXBean mx = ManagementFactory.getThreadMXBean();
        long[] ids = null;
        for (int i = 0; i < 200 && ids == null; i++) { Thread.sleep(10); ids = mx.findDeadlockedThreads(); }
        String[] names = ids == null ? new String[0]
                : Arrays.stream(mx.getThreadInfo(ids)).map(ThreadInfo::getThreadName).sorted().toArray(String[]::new);
        System.out.println("deadlock detected between: " + Arrays.toString(names)
                + "  (states " + x.getState() + ", " + y.getState() + ")");
    }

    static void await(CyclicBarrier b) {
        try { b.await(); } catch (Exception e) { throw new RuntimeException(e); }
    }
}
```

Explaining each part:

- `demonstrateDeadlock` forces the deadly interleaving: each thread takes one lock, then both meet at a barrier, then each asks for the other's lock. `ThreadMXBean` finds the cycle; both threads show `BLOCKED`. They are daemon threads, so the JVM can still exit.
- `transferOrdered` always locks the account with the smaller id first → circular wait is impossible. 100,000 opposite-direction transfers finish and money is conserved.
- `transferTryLock` uses `tryLock(1 ms)` and releases on failure, with a random back-off (to avoid livelock, next section).

Output:

```
deadlock detected between: [thread-X, thread-Y]  (states BLOCKED, BLOCKED)
lock ordering: finished, total money = 2000
tryLock + back-off: finished, total money = 2000
```

Trade-offs

Strategy	Pros	Cons
Lock ordering	Simple, zero runtime cost	Needs a stable global order; hard across modules
Try-lock + back-off	Works without an order; bounded waits	Retry logic; possible livelock without randomness
Single coarse lock	No deadlock among your locks	Lower throughput
Lock-free / immutable	No locks at all	Harder to design

When not to use it

N/A — you always want to prevent deadlock. But don't add complicated detection/recovery in application code when simple lock ordering would prevent it.

Common mistakes

- Calling a listener or another object's synchronized method while holding your own lock (hidden nested locking).
- Ordering by `hashCode()` without handling equal hashes.
- Also *resource* deadlocks: a pool of 10 threads where each task submits a sub-task to the **same** pool and waits for it — when all 10 wait, nothing runs (thread-starvation deadlock).

Real-world examples

- Databases (MySQL InnoDB, PostgreSQL) detect deadlocks between **transactions** (groups of database operations that succeed or fail as one unit) and abort one as a victim — the "detect and recover" strategy.
- Bank transfer systems lock accounts in id order — the lock-ordering strategy.

In the interview

> **30-second answer:** "Deadlock needs four conditions: mutual exclusion, hold-and-wait, no preemption, and circular wait. I break circular wait with a global lock order — e.g. lock seats or accounts in ascending id — or break hold-and-wait with tryLock with timeout and release-and-retry. I avoid calling external code while holding a lock, and I'd detect it in production with thread dumps."

Follow-ups:

- *Two accounts with the same id order?* Same account → just one lock (transfer to self is a no-op or rejected).
- *How would you find a deadlock in production?* `jstack <pid>` or `jcmd <pid> Thread.print` shows it; monitoring can call `ThreadMXBean.findDeadlockedThreads()`.

Self-check

- **Q:** Which Coffman condition does lock ordering break? **A:** Circular wait.
- **Q:** Which does `tryLock` + release break? **A:** Hold and wait (and effectively no-preemption, by giving up).

TL;DR

Deadlock = cycle of waiting. Needs all 4 Coffman conditions. Break one: lock ordering (circular wait) or tryLock + release (hold and wait). Don't call alien code under a lock.

12.18 Livelock

What it is

A **livelock** is when threads are not blocked — they keep running and reacting to each other — but make **no progress**.

> **Analogy.** Two people in a corridor both step aside to let the other pass, in the same direction, again and again.

The problem it solves

(Recognising it.) Livelocks often appear *as a side effect of deadlock avoidance*: two threads that both `tryLock`, fail, release, and retry at the same rhythm can collide forever.

How it works (and how to fix it)

1. Thread 1 takes lock A, tries B, fails, releases A.
2. Thread 2 takes lock B, tries A, fails, releases B.
3. Both retry immediately → same collision, forever.
4. **Fixes:** random back-off (different waits break the symmetry), exponential back-off, a consistent lock order (then no retries are needed), or a priority/tie-breaker (e.g. the older transaction wins).

Java example

`transferTryLock` in `deadlock/Main.java` sleeps a **random** number of nanoseconds (`ThreadLocalRandom`) after a failed attempt. Remove that line and the two threads could, in principle, keep failing in lock-step.

Trade-offs

Random back-off makes progress very likely but adds latency; exponential back-off adapts to contention.

When not to use it

Don't design retry loops that react only to each other with no randomness or tie-breaker.

Common mistakes

- Fixed retry delays (`sleep(10)` for everyone).
- Retrying forever without a limit or alert.

Real-world examples

- **Ethernet (CSMA/CD)** (the classic wired-network rule: listen before sending, detect collisions) uses *binary exponential back-off* (doubling the maximum random wait after each collision) after collisions for exactly this reason.
- Retry policies in AWS SDKs (Amazon Web Services' client libraries) and Resilience4j (a Java fault-tolerance library) use exponential back-off **with jitter** (a small random amount added to each wait) to stop synchronized **retry storms** (crowds of clients all retrying at the same moment).

In the interview

> **30-second answer:** "Livelock is when threads keep reacting to each other without progress — like two people side-stepping in a corridor. It often comes from symmetric retry logic; I fix it with randomized or exponential back-off, or remove retries by using a lock order."

Self-check

- **Q:** Deadlock vs livelock — thread states? **A:** Deadlocked threads are blocked/waiting; livelocked threads are running (`RUNNABLE`) but achieving nothing.

TL;DR

Livelock = busy but no progress. Break symmetry with jittered back-off or a tie-breaker.

12.19 Starvation (and fairness)

What it is

Starvation is when a thread waits indefinitely because others keep getting the resource first, even though the system as a whole makes progress.

> **Analogy.** At a busy bar, a quiet customer never gets served because louder customers keep catching the bartender's eye.

The problem it solves

(Recognising it.) Causes: unfair (barging) locks under heavy contention, a writer waiting behind a constant stream of readers in a read-write lock, low thread priority, or a long-running task hogging a single-thread executor.

How it works (fixes)

1. Use **fair** locks/semaphores where it matters: `new ReentrantLock(true)`, `new Semaphore(n, true)`, `new ReentrantReadWriteLock(true)`. Fair = longest waiter goes next.
2. Keep critical sections short so the lock is free more often.
3. Separate pools for slow and fast work (bulkheads) so slow tasks can't starve fast ones.
4. Don't rely on `Thread.setPriority` — its effect is OS-dependent.

Java example

`Waiter` in chapter 13's Dining Philosophers uses `new Semaphore(n - 1, true)` so no philosopher waits forever for a seat.

Trade-offs

Fairness prevents starvation but reduces throughput (more context switches, no barging).

When not to use it

Don't make every lock fair "just in case"; use fairness when you've seen or can argue starvation risk.

Common mistakes

Confusing starvation with deadlock: in starvation *someone* progresses; in deadlock *nobody* does.

Real-world examples

OS schedulers use *aging* (raising the priority of long-waiting processes); Linux's schedulers (CFS, and its successor EEVDF since **kernel** — the core of the operating system — 6.6) aim to give every runnable task a fair share of CPU time.

In the interview

> **30-second answer:** "Starvation is when a thread never gets a resource because others always win — e.g. writers behind endless readers. Fair locks, short critical sections and separate pools fix it, at some throughput cost."

Self-check

- **Q:** One fix for writer starvation in `ReentrantReadWriteLock`? **A:** Construct it in fair mode (`new ReentrantReadWriteLock(true)`).

TL;DR

Starvation = some thread never gets a turn. Fairness, short sections, separate pools.

Part G — Concurrency patterns

The next five patterns share one runnable file.

```java
// File: code/lld/ch12/patterns/Main.java
package lld.ch12.patterns;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.RejectedExecutionException;
import java.util.concurrent.Semaphore;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;

/** Signaling, thread pool, producer-consumer, immutability, thread confinement. */
public class Main {

    // ---------- 1. Signaling ----------
    static void signaling() throws InterruptedException {
        System.out.println("=== Signaling: B must run after A ===");
        Semaphore aDone = new Semaphore(0);                  // 0 permits = "not signalled yet"
        List<String> log = java.util.Collections.synchronizedList(new ArrayList<>());
        Thread b = new Thread(() -> {
            try { aDone.acquire(); log.add("B: send email (after payment)"); }
            catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });
        Thread a = new Thread(() -> { log.add("A: charge payment"); aDone.release(); });
        b.start(); a.start();                                // B starts first but must wait
        a.join(); b.join();
        log.forEach(System.out::println);
    }

    // ---------- 2. Thread pool (hand-made, to see the moving parts) ----------
    static class SimpleThreadPool {
        private static final Runnable POISON = () -> { };
        private final BlockingQueue<Runnable> tasks = new LinkedBlockingQueue<>();
        private final List<Thread> workers = new ArrayList<>();
        private boolean shutdown;                            // guarded by 'this'

        SimpleThreadPool(int size) {
            for (int i = 0; i < size; i++) {
                Thread w = new Thread(() -> {
                    try {
                        while (true) {
                            Runnable r = tasks.take();          // idle workers block here
                            if (r == POISON) return;
                            try { r.run(); } catch (RuntimeException e) { System.err.println("task failed: " + e); }
                        }
                    } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
                }, "pool-worker-" + i);
                workers.add(w);
                w.start();
            }
        }
        // submit and the pill-adding part of shutdown share one lock, so "check the flag, then
        // enqueue" is atomic: no task can slip into the queue AFTER the poison pills and be lost.
        synchronized void submit(Runnable r) {
            if (shutdown) throw new RejectedExecutionException("pool is shut down");
            tasks.add(r);                                    // unbounded queue: add() never blocks
        }
        void shutdownAndWait() throws InterruptedException {
            synchronized (this) {
                if (shutdown) return;
                shutdown = true;
                for (int i = 0; i < workers.size(); i++) tasks.add(POISON);   // one pill per worker
            }
            for (Thread w : workers) w.join();           // wait OUTSIDE the lock
        }
    }

    static void threadPool() throws InterruptedException {
        System.out.println("=== Thread pool ===");
        SimpleThreadPool pool = new SimpleThreadPool(3);
```

```
        AtomicInteger done = new AtomicInteger();
        for (int i = 0; i < 20; i++) pool.submit(done::incrementAndGet);
        pool.shutdownAndWait();
        System.out.println("hand-made pool ran " + done.get() + " tasks on 3 threads");

        // Production version: bounded queue + explicit rejection policy.
        ThreadPoolExecutor exec = new ThreadPoolExecutor(
                2, 4,                                  // core and max threads
                30, TimeUnit.SECONDS,                  // idle time before extra threads die
                new ArrayBlockingQueue<>(100),         // bounded: never an OOM from a huge backlog
                new ThreadPoolExecutor.CallerRunsPolicy()); // when full, the submitter runs it (back-pressure)
        CompletableFuture<Integer> price = CompletableFuture.supplyAsync(() -> 100, exec);
        CompletableFuture<Integer> tax = CompletableFuture.supplyAsync(() -> 18, exec);
        System.out.println("ThreadPoolExecutor + CompletableFuture: total = " + price.thenCombine(tax,
Integer::sum).join());
        exec.shutdown();
        exec.awaitTermination(5, TimeUnit.SECONDS);
    }

    // ---------- 3. Producer-consumer ----------
    static void producerConsumer() throws InterruptedException {
        System.out.println("=== Producer-consumer ===");
        BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(10);   // producers block when full
        final int POISON = -1, CONSUMERS = 2;
        AtomicLong total = new AtomicLong();
        List<Thread> consumers = new ArrayList<>();
        for (int c = 0; c < CONSUMERS; c++) {
            Thread t = new Thread(() -> {
                try {
                    for (int x = queue.take(); x != POISON; x = queue.take()) total.addAndGet(x);
                } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            });
            consumers.add(t); t.start();
        }
        Thread producer = new Thread(() -> {
            try {
                for (int i = 1; i <= 1_000; i++) queue.put(i);
                for (int c = 0; c < CONSUMERS; c++) queue.put(POISON);   // tell each consumer to stop
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });
        producer.start();
        producer.join();
        for (Thread t : consumers) t.join();
        System.out.println("consumed sum of 1..1000 = " + total.get() + " (expected 500500)");
    }

    // ---------- 4. Immutability ----------
    /** Immutable value object: final fields, no setters, defensive copy. Safe to share freely. */
    record Order(String id, List<String> items) {
        Order { items = List.copyOf(items); }                  // caller's list can't change ours
        Order withItem(String item) {                           // "modify" = return a new object
            List<String> copy = new ArrayList<>(items); copy.add(item);
            return new Order(id, copy);
        }
    }

    // ---------- 5. Thread confinement ----------
    private static final ThreadLocal<StringBuilder> BUFFER = ThreadLocal.withInitial(StringBuilder::new);

    static String render(String name) {
        StringBuilder sb = BUFFER.get();       // each thread has its own builder: no sharing, no locks
        sb.setLength(0);
        return sb.append("Hello, ").append(name).toString();
    }

    public static void main(String[] args) throws Exception {
        signaling();
        threadPool();
        producerConsumer();

        System.out.println("=== Immutability ===");
        List<String> mutable = new ArrayList<>(List.of("pen"));
        Order o1 = new Order("o1", mutable);
        mutable.add("hacked");
        Order o2 = o1.withItem("book");
        System.out.println("o1 = " + o1.items() + ", o2 = " + o2.items());
```

```
        System.out.println("=== Thread confinement (ThreadLocal) ===");
        Thread t1 = new Thread(() -> System.out.println(render("T1")));
        t1.start(); t1.join();
        Thread t2 = new Thread(() -> System.out.println(render("T2")));
        t2.start(); t2.join();
    }
}
```

Output:

```
=== Signaling: B must run after A ===
A: charge payment
B: send email (after payment)
=== Thread pool ===
hand-made pool ran 20 tasks on 3 threads
ThreadPoolExecutor + CompletableFuture: total = 118
=== Producer-consumer ===
consumed sum of 1..1000 = 500500 (expected 500500)
=== Immutability ===
o1 = [pen], o2 = [pen, book]
=== Thread confinement (ThreadLocal) ===
Hello, T1
Hello, T2
```

12.20 Signaling pattern

What it is

One thread **notifies** another that an event happened, so the second can proceed. "B must run after A."

> **Analogy.** A relay race baton: the next runner can't start until the baton is handed over.

The problem it solves

Enforcing ordering between threads without busy-waiting (spinning on a flag).

How it works

1. Create a signal object in the "not yet" state: `new Semaphore(0)`, `new CountDownLatch(1)`, a `CompletableFuture`, or a flag + condition variable.
2. The waiting thread calls `acquire()` / `await()` / `join()`.
3. The signalling thread calls `release()` / `countDown()` / `complete(value)`.
4. The waiter wakes and continues. The signal also provides happens-before, so the waiter sees everything A did.

Tool	Reusable	Carries a value	Good for
`Semaphore(0)`	Yes (each release = one signal)	No	Ping-pong between threads (FooBar, chapter 13)
`CountDownLatch(1)`	No	No	One-time "ready/go"
`CompletableFuture`	No	Yes	"Here is the result"
Condition variable	Yes	Via shared state	Complex conditions

Java example

`signaling()` above: B starts first but waits on `aDone.acquire()`; A logs and calls `release()`. Output shows A's line before B's every time.

Trade-offs

Semaphores are compact but easy to mis-count; latches can't be reset; futures are clean but single-use.

When not to use it

If the "signal" is really "here's a work item", use a `BlockingQueue` (producer-consumer).

Common mistakes

- Signalling before the waiter starts waiting with a tool that *forgets* signals (plain `notify()` without a state flag = lost wakeup). Semaphores and latches remember.

Real-world examples

`CompletableFuture` chains in Spring WebFlux (Spring's non-blocking web framework)/Netty **callbacks** (functions handed to a library so it can call you back later); start-up gates; "order paid → send confirmation" flows inside one service.

In the interview

"Signaling means one thread waits for an event raised by another. I use a zero-permit semaphore for repeated hand-offs, a latch for one-time events, or a `CompletableFuture` when a result travels with the signal."

Self-check

Q: Why is a bare `notify()` risky for signaling? **A:** If it happens before the other thread waits, the signal is lost; always pair it with a state variable checked in a loop.

TL;DR

Signal = "you may go now". Semaphore(0), latch, or future. Make sure signals can't be lost.

12.21 Thread pool pattern

What it is

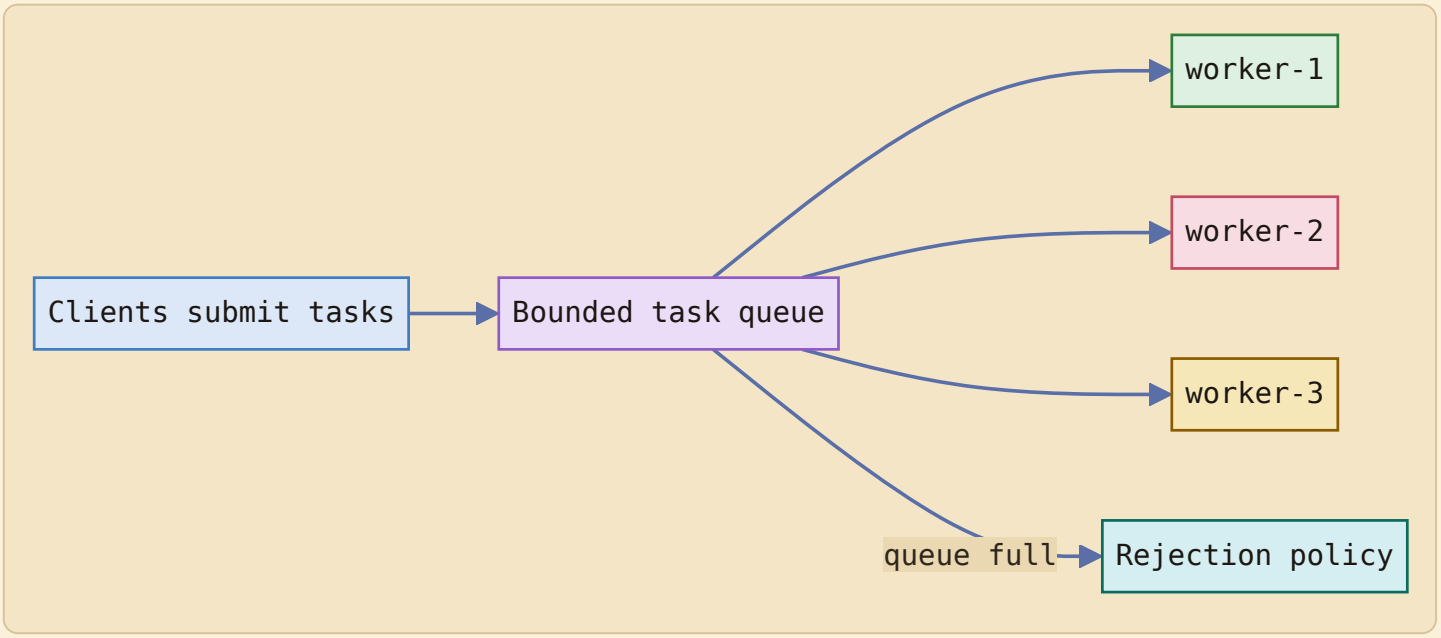
A fixed set of **worker threads** takes tasks from a shared **queue** and runs them, reusing threads instead of creating one per task.

> **Analogy.** A taxi rank: a few taxis serve a queue of passengers; taxis are reused, and if the queue gets too long, the rank manager turns people away.

The problem it solves

Creating/destroying threads per task is expensive and unbounded concurrency can overwhelm CPU, memory and downstream systems. A pool bounds concurrency and **amortises** thread creation (spreads its one-time cost over many tasks).

How it works



1. Workers loop: `task = queue.take(); task.run();`.

2. `submit` puts the task in the queue.
3. `ThreadPoolExecutor` rules: while fewer than **core** threads run, a new thread is created per task; then tasks go into the **queue**; if the queue is full, extra threads are created up to **max**; if still full, the **rejection policy** runs (`AbortPolicy` throws — default; `CallerRunsPolicy` runs it on the submitting thread, slowing producers; `DiscardPolicy`; `DiscardOldestPolicy`).
4. Shutdown: stop accepting, let workers drain the queue, then exit (the hand-made pool uses one "**poison pill**" — a special marker task meaning "stop now" — per worker). The "is it shut down? then enqueue" check in `submit` is a check-then-act, so `submit` and the pill-adding step of shutdown hold the same lock — otherwise a task could be enqueued *behind* the pills and silently never run.

Sizing: CPU-bound ≈ number of cores; I/O-bound ≈ cores × (1 + wait/compute) — or virtual threads.

Java example

`SimpleThreadPool` above (hand-made, to see the mechanics) and a production `ThreadPoolExecutor` with a bounded `ArrayBlockingQueue(100)` and `CallerRunsPolicy`, combined with `CompletableFuture.thenCombine`.

Trade-offs

Choice	Effect
Bounded queue	Protects memory; needs a rejection policy
Unbounded queue (`newFixedThreadPool`)	Never rejects, may run out of memory, and max-pool-size is never used
`CallerRunsPolicy`	Natural back-pressure: submitter slows down
Many pools (bulkheads)	Isolation between workloads; more threads overall

When not to use it

For huge numbers of blocking tasks on Java 21, virtual-thread-per-task is simpler. For fork/join-style recursive CPU work, use `ForkJoinPool` (**work stealing**: an idle thread takes queued tasks from a busy thread's queue).

Common mistakes

- Tasks that wait for other tasks in the same bounded pool (thread-starvation deadlock).
- Swallowed exceptions in `submit`ted tasks.
- Forgetting `shutdown()`.
- Using `ThreadLocal`s in pooled threads without clearing them (values leak between tasks).

Real-world examples

Tomcat's connector thread pool (`maxThreads`, default 200), Spring's `ThreadPoolTaskExecutor`, database connection pools (same idea for connections), `ForkJoinPool.commonPool()` behind parallel streams and `CompletableFuture.supplyAsync` without an executor.

In the interview

"A thread pool reuses a bounded set of workers fed by a queue. I size it by workload type, use a bounded queue with an explicit rejection policy for back-pressure, and give slow dependencies their own pool as a bulkhead."

Self-check

Q: In `ThreadPoolExecutor`, when are threads beyond `corePoolSize` created? **A:** Only when the queue is full (and fewer than max threads exist).

TL;DR

Workers + queue + rejection policy. Bound everything. Size by CPU vs I/O.

12.22 Producer-consumer pattern

What it is

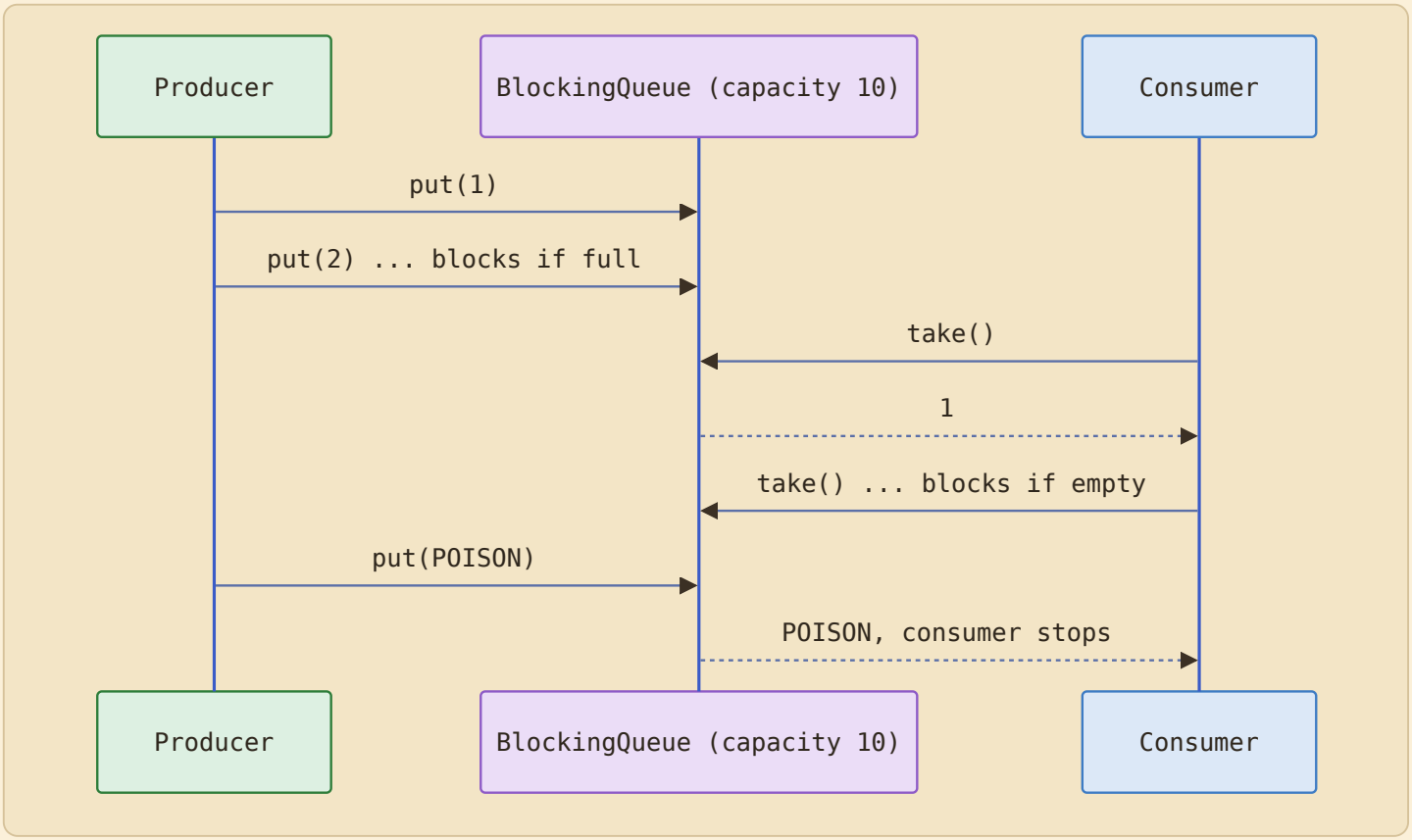
Producers create work items and put them into a shared **buffer**; **consumers** take items out and process them. The buffer decouples their speeds.

> **Analogy.** A restaurant kitchen's order rail: waiters (producers) clip tickets on the rail, cooks (consumers) take them off. If the rail is full, waiters wait; if empty, cooks wait.

The problem it solves

Producers and consumers run at different rates. A bounded buffer smooths bursts and applies **back-pressure** (producers slow down when consumers fall behind), without either side knowing about the other.

How it works



1. Share a bounded `BlockingQueue`.
2. Producers `put` (waits when full); consumers `take` (waits when empty).
3. Shutdown: each consumer must learn when to stop — a **poison pill** per consumer, or interrupting consumers, or a "done" flag plus `poll(timeout)`.

Java example

`producerConsumer()` above: 1 producer, 2 consumers, queue capacity 10, two poison pills. Sum is exactly 500500. Chapter 13 builds the blocking queue itself two ways.

Trade-offs

Pros	Cons
Decouples rates, enables back-pressure	Adds latency (queueing)
Easy to scale consumers	Ordering across multiple consumers isn't preserved
Clear separation of concerns	Need a shutdown protocol

When not to use it

When the caller needs the result immediately and synchronously, a direct call is simpler.

Common mistakes

- Unbounded buffer → no back-pressure → memory blow-up.
- One poison pill for N consumers → N−1 consumers hang forever.
- Busy-polling with `poll()` in a loop instead of blocking `take()`.

Real-world examples

Apache **Kafka** (a distributed, durable message log) is producer-consumer across machines (topics are the buffer; **consumer groups** — sets of consumers that split a topic's messages between them — scale consumers). Inside the Kafka Java producer, application threads append records to an in-memory accumulator and a background sender thread ships batches. Logging frameworks' async appenders, and `ThreadPoolExecutor` itself (submitters produce, workers consume).

In the interview

"Producer-consumer decouples producers from consumers with a bounded blocking queue: put waits when full, take waits when empty, giving natural back-pressure. I stop consumers with one poison pill each, and scale consumers independently."

Self-check

Q: Why bounded? **A:** So a slow consumer pushes back on producers instead of letting the queue grow until memory runs out.

TL;DR

Bounded `BlockingQueue` between producers and consumers. Back-pressure for free. One poison pill per consumer.

12.23 Reader-writer pattern

What it is

A design where **many readers may access shared data at the same time, but a writer needs exclusive access**.

> **Analogy.** A shared noticeboard: many people read it at once; when someone replaces a notice, readers step back for a moment.

The problem it solves

Read-mostly data (config, catalogues, routing tables) shouldn't be serialised through one mutex.

How it works

Two classic implementations:

1. **Read-write lock** (12.10): `readLock` for readers, `writeLock` for writers. Choose fair mode if writers could starve.
2. **Copy-on-write snapshot**: keep an immutable object in a `volatile` field. Readers just read the field (no lock). A writer builds a *new* immutable copy and swaps the reference (use a lock or `AtomicReference.updateAndGet` if several writers may race).

```
private volatile Map<String, String> config = Map.of();       // immutable snapshot
String get(String k) { return config.get(k); }                  // readers: no lock
synchronized void put(String k, String v) {                      // writers: serialized
    Map<String, String> copy = new HashMap<>(config);
    copy.put(k, v);
    config = Map.copyOf(copy);                                   // publish new snapshot
}
```

	RW lock	Copy-on-write snapshot
Reader cost	Acquire shared lock	One volatile read
Writer cost	Exclusive lock, in-place update	Copy whole structure
Readers see	Latest data	A consistent snapshot (maybe slightly old)
Best for	Frequent-ish writes, big data	Rare writes, small-to-medium data

Java example

`ConfigStore` (RW lock) in `locks/Main.java`; the snapshot code just above; `CopyOnWriteArrayList` in `collections/Main.java`.

Trade-offs

See table. Also: RW lock readers must not upgrade to write.

When not to use it

Write-heavy data — plain mutex or `ConcurrentHashMap` is better.

Common mistakes

Mutating the shared snapshot in place (breaks all readers' assumptions); upgrading read → write locks.

Real-world examples

Feature-flag and configuration clients that reload periodically and swap an immutable snapshot; Linux kernel RCU (read-copy-update) is the same idea at OS level.

In the interview

"For read-heavy data I use either a read-write lock or, if writes are rare, an immutable snapshot in a volatile field that writers replace — readers never block."

Self-check

Q: What guarantees readers see a fully built new snapshot? **A:** The `volatile` write of the reference happens-before later volatile reads, and the snapshot is immutable.

TL;DR

Many readers, one writer. RW lock, or immutable snapshot + volatile swap.

12.24 Immutability & thread confinement

What it is

Two ways to be thread-safe **without any synchronization**:

- **Immutability**: an object whose state can't change after construction (all fields `final`, no setters, no leaking mutable internals). Examples: `String`, `Integer`, `LocalDate`, `BigDecimal`, records with immutable components, `List.of(...)`.
- **Thread confinement**: data that only one thread ever touches. Local variables (stack confinement), `ThreadLocal<T>` (one copy per thread), or a design where one thread owns a structure (event loop, **actor** — an object with private state that processes messages from its mailbox one at a time).

> **Analogy.** Immutability: a printed book — everyone can read it at once, nobody can change it. Confinement: a personal notebook — you write freely because nobody else reads it.

The problem it solves

Races need *shared mutable* state. Remove "mutable" (immutability) or remove "shared" (confinement) and races are impossible.

How it works

Immutable class checklist:

1. Make the class `final` (or a `record`) so subclasses can't add mutability.
2. Make all fields `private final`.
3. No setters; "modifiers" return new objects (`withItem(...)`).
4. **Defensive-copy** (copy, so the caller's later changes can't reach you) mutable inputs in the constructor (`List.copyOf`) and don't expose mutable internals.
5. Don't let `this` escape during construction.

Confinement options:

1. Keep data in local variables.
2. `ThreadLocal.withInitial(...)` for per-thread helpers (formatters, buffers). Call `remove()` when used in pooled threads.
3. Single-writer design: one thread owns the state; others send it messages through a queue.

Java example

`Order` record above: the compact constructor does `List.copyOf(items)`, so the caller's later `mutable.add("hacked")` doesn't affect `o1`; `withItem` returns a new `Order`. `BUFFER` is a `ThreadLocal<StringBuilder>` — each thread gets its own builder.

Trade-offs

	Immutability	Confinement
Cost	New object per change (GC pressure — more garbage for the collector to clean)	Per-thread copies; ThreadLocal leaks if not cleaned
Sharing	Share freely	Not shareable by definition
Reasoning	Easiest possible	Easy, if confinement is really enforced

When not to use it

Very large objects changed at high frequency (copying costs too much) — use a concurrent structure instead.

Common mistakes

- `final List<String> items` is not immutable if the list itself is mutable and exposed.
- `ThreadLocal` in thread pools without `remove()` — values leak into the next task; with many virtual threads, heavy `ThreadLocal`s waste memory (*scoped values* are the alternative: a **preview** — an experimental feature you must opt into — in Java 21, final in Java 25 via JEP 506).
- Sharing `SimpleDateFormat` (mutable, not thread-safe) instead of using immutable `DateTimeFormatter`.

Real-world examples

- `java.time` types and `DateTimeFormatter` are immutable and thread-safe; `SimpleDateFormat` is not.
- **Netty** and **Vert.x** confine each connection to one event-loop thread, so handler code needs no locks.
- Redis executes commands on a single main thread (confinement at system level).
- Spring's `@Transactional` (marks a method to run inside a database transaction) stores the current transaction in `ThreadLocal`s (via `TransactionSynchronizationManager`).

In the interview

"The easiest thread-safety is none needed: make value objects immutable — final fields, defensive copies, 'with' methods — and confine mutable state to one thread with locals, ThreadLocal, or a single owner thread fed by a queue."

Self-check

Q: Is a class with `private final Date created` immutable? **A:** Not unless it copies the `Date` in and out — `Date` is mutable. Prefer `Instant`.

TL;DR

No shared mutable state → no races. Immutable objects share freely; confined objects aren't shared.

Part H — Concurrency in LLD interviews

12.25 How concurrency shows up in LLD interviews (parking, seat booking, inventory)

What it is

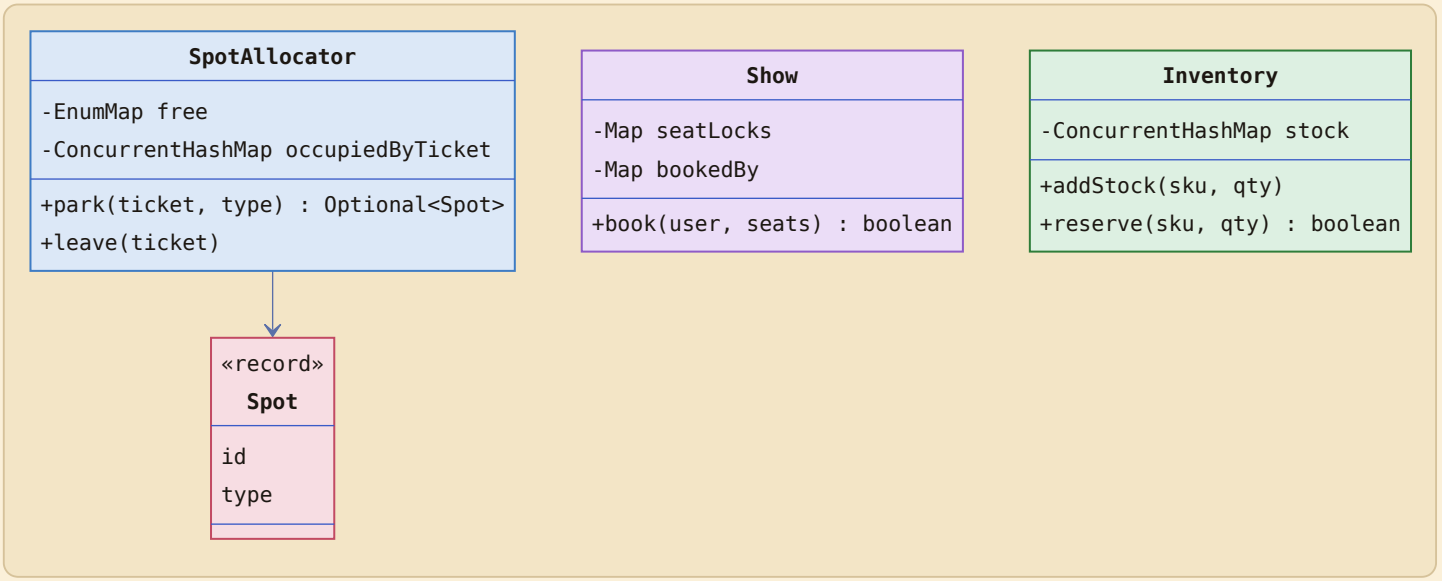
Most LLD case studies have a **contended resource**: a parking spot, a movie seat, a hotel room, a limited stock item, an elevator request, a wallet balance. The interviewer checks whether your design prevents double allocation and still performs.

The problem it solves

Showing you can turn a single-threaded class diagram into a **thread-safe design** with the smallest correct synchronization.

How it works — a 6-step recipe

1. **Name the shared mutable state** (free spots, seat → owner map, stock counts).
2. **State the invariant** ("a spot has at most one car", "stock ≥ 0", "a seat has at most one booking").
3. **Find the critical operation** (usually a check-then-act: *find free → assign*).
4. **Pick the smallest tool that makes it atomic:**
   - one item claimed → atomic op: `queue.poll()`, `map.putIfAbsent`, `AtomicBoolean.compareAndSet`, `AtomicInteger` CAS loop;
   - several items all-or-nothing → per-item locks acquired **in sorted order** (or one per-show/per-floor lock);
   - across processes/servers → database constraints/transactions, optimistic versioning, or a distributed lock.
5. **Choose granularity:** per floor/show/SKU (stock-keeping unit — one distinct product) locks rather than one global lock.
6. **Handle time:** temporary holds with expiry (seat held 10 minutes during payment), timeouts, and **idempotent** retries (safe to repeat, because doing the operation twice has the same effect as doing it once; see System Design Fundamentals).



Java example

```java
// File: code/lld/ch12/lldcases/Main.java
package lld.ch12.lldcases;

import java.util.ArrayList;
import java.util.EnumMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.TreeSet;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.locks.ReentrantLock;

/** Thread-safe building blocks for three interview favourites. */
public class Main {

    // ============================================================
    // 1. Parking lot: many gates allocate spots at the same time.
    // ============================================================
    enum SpotType { COMPACT, LARGE }
    record Spot(String id, SpotType type) { }

    static class SpotAllocator {
        // One lock-free queue of FREE spots per type. poll() is atomic: two gates can never get the same spot.
        private final Map<SpotType, ConcurrentLinkedQueue<Spot>> free = new EnumMap<>(SpotType.class);
        private final ConcurrentHashMap<String, Spot> occupiedByTicket = new ConcurrentHashMap<>();

        SpotAllocator(List<Spot> spots) {
            for (SpotType t : SpotType.values()) free.put(t, new ConcurrentLinkedQueue<>());
            spots.forEach(s -> free.get(s.type()).add(s));   // EnumMap itself is never modified after this
        }
        Optional<Spot> park(String ticket, SpotType type) {
            Spot s = free.get(type).poll();
            if (s != null) occupiedByTicket.put(ticket, s);
            return Optional.ofNullable(s);
        }
        void leave(String ticket) {
            Spot s = occupiedByTicket.remove(ticket);        // remove is atomic: double-exit is harmless
            if (s != null) free.get(s.type()).offer(s);
        }
        // ConcurrentLinkedQueue.size() walks the queue (O(n)) and is only an estimate while other
        // threads change it; fine for a report once all gates are idle, not for allocation logic.
        int freeCount(SpotType t) { return free.get(t).size(); }
    }

    // ============================================================
    // 2. Seat booking: all-or-nothing booking of several seats.
    // ============================================================
    static class Show {
        private final Map<String, ReentrantLock> seatLocks = new ConcurrentHashMap<>();
        private final Map<String, String> bookedBy = new ConcurrentHashMap<>();

        Show(Set<String> seatIds) { seatIds.forEach(id -> seatLocks.put(id, new ReentrantLock())); }

        /** Locks seats in sorted order (prevents deadlock), checks all, then books all. */
        boolean book(String user, Set<String> seats) {
            List<ReentrantLock> held = new ArrayList<>();
            try {
                for (String id : new TreeSet<>(seats)) {         // global order: A1 < A2 < A3
                    ReentrantLock l = seatLocks.get(id);
                    if (l == null) return false;                  // unknown seat
                    l.lock();
                    held.add(l);
                }
                for (String id : seats) if (bookedBy.containsKey(id)) return false;   // someone got one first
                for (String id : seats) bookedBy.put(id, user);
                return true;
            } finally {
```

```
                for (int i = held.size() - 1; i >= 0; i--) held.get(i).unlock();
            }
        }
        String owner(String seat) { return bookedBy.get(seat); }
    }

    // ======================================================================
    // 3. Inventory: never sell more than we have (no overselling).
    // ======================================================================
    static class Inventory {
        private final ConcurrentHashMap<String, AtomicInteger> stock = new ConcurrentHashMap<>();

        void addStock(String sku, int qty) {
            stock.computeIfAbsent(sku, k -> new AtomicInteger()).addAndGet(qty);
        }
        /** CAS loop: decrement only if enough stock remains. */
        boolean reserve(String sku, int qty) {
            AtomicInteger s = stock.get(sku);
            if (s == null) return false;
            while (true) {
                int cur = s.get();
                if (cur < qty) return false;
                if (s.compareAndSet(cur, cur - qty)) return true;
            }
        }
        int available(String sku) { AtomicInteger s = stock.get(sku); return s == null ? 0 : s.get(); }
    }

    // ----------------------------------------------------------------------
    /** Run 'n' tasks that all start at the same instant (maximises contention). */
    static void raceAll(int n, java.util.function.IntConsumer task) throws InterruptedException {
        CountDownLatch go = new CountDownLatch(1);
        try (ExecutorService ex = Executors.newFixedThreadPool(16)) {
            for (int i = 0; i < n; i++) {
                final int id = i;
                ex.submit(() -> { go.await(); task.accept(id); return null; });
            }
            go.countDown();
        } // try-with-resources close() waits for all tasks (Java 19+)
    }

    public static void main(String[] args) throws Exception {
        List<Spot> spots = new ArrayList<>();
        for (int i = 1; i <= 10; i++) spots.add(new Spot("C" + i, SpotType.COMPACT));
        SpotAllocator lot = new SpotAllocator(spots);
        AtomicInteger parked = new AtomicInteger();
        Set<String> handedOut = ConcurrentHashMap.newKeySet();
        raceAll(100, car -> lot.park("T" + car, SpotType.COMPACT).ifPresent(s -> { parked.incrementAndGet();
handedOut.add(s.id()); }));
        System.out.println("parking: 100 cars raced for 10 spots -> parked " + parked.get()
                + ", distinct spots " + handedOut.size() + ", free left " + lot.freeCount(SpotType.COMPACT));

        Show show = new Show(Set.of("A1", "A2", "A3", "A4"));
        AtomicInteger wins = new AtomicInteger();
        raceAll(50, u -> { if (show.book("user" + u, Set.of("A2"))) wins.incrementAndGet(); });
        System.out.println("booking: 50 users wanted seat A2 -> successful bookings " + wins.get());
        AtomicInteger groupWins = new AtomicInteger();
        raceAll(2, u -> { if (show.book(u == 0 ? "alice" : "bob", u == 0 ? Set.of("A3", "A4") : Set.of("A4", "A3")))
groupWins.incrementAndGet(); });
        System.out.println("booking: alice {A3,A4} vs bob {A4,A3} (opposite order) -> winners " + groupWins.get()
                + ", A3 and A4 same owner: " + show.owner("A3").equals(show.owner("A4")));

        Inventory inv = new Inventory();
        inv.addStock("iphone", 50);
        AtomicInteger sold = new AtomicInteger();
        raceAll(200, o -> { if (inv.reserve("iphone", 1)) sold.incrementAndGet(); });
        System.out.println("inventory: 200 buyers, 50 in stock -> sold " + sold.get() + ", left " +
inv.available("iphone"));
    }
}
```

Explaining each part:

- **Parking**: free spots per type live in a `ConcurrentLinkedQueue`. `poll()` atomically removes one spot, so two gates can never receive the same spot — no explicit lock. `leave` uses `ConcurrentHashMap.remove(ticket)`, so a

duplicated exit event can't return a spot twice.

- **Seat booking**: one `ReentrantLock` per seat. `book` locks the requested seats **in sorted order** (breaking circular wait), checks that *all* are free, then books them all — all-or-nothing. Alice asks for `{A3, A4}` and Bob for `{A4, A3}` at the same moment: exactly one wins, no deadlock, and both seats get the same owner.
- **Inventory**: an `AtomicInteger` per SKU and a CAS loop that decrements only if enough stock remains — 200 buyers, 50 items → exactly 50 sold, never negative.
- `raceAll` releases all tasks through a `CountDownLatch` at once to maximise contention.

Output:

```
parking: 100 cars raced for 10 spots -> parked 10, distinct spots 10, free left 0
booking: 50 users wanted seat A2 -> successful bookings 1
booking: alice {A3,A4} vs bob {A4,A3} (opposite order) -> winners 1, A3 and A4 same owner: true
inventory: 200 buyers, 50 in stock -> sold 50, left 0
```

Trade-offs

Resource	In-memory approach	Across many servers
Parking spot	Lock-free queue per type/floor, or per-floor lock	DB row update `WHERE status='FREE'` + check affected rows
Seat	Per-show or per-seat locks in sorted order; hold with TTL (time-to-live, an expiry)	Unique constraint on (show, seat); short-lived holds in Redis (`SET NX PX`)
Inventory	`AtomicInteger` CAS / `compute`	`UPDATE stock SET qty = qty - 1 WHERE sku = ? AND qty >= 1` ; reservations with expiry

When not to use it

Don't bolt `synchronized` onto every method of every class — interviewers read that as "doesn't know where the contention is". Protect the few operations that touch contended state.

Common mistakes

- A global `synchronized` on `ParkingLot.park()` — correct but serialises every gate.
- Checking availability with one call and booking with another (check-then-act).
- Locking seats in the order the user clicked them (deadlock between two multi-seat bookings).
- Forgetting that in-memory locks don't work across multiple app servers.

Real-world examples

- **BookMyShow / Ticketmaster-style** checkout: selected seats are *held* for a few minutes while you pay; if payment doesn't complete, the hold expires and the seats return to the pool — a lock with a lease.
- **Flash sales** (e.g. limited-stock phone launches) decrement stock atomically, often in Redis (`DECR` / Lua scripts, which Redis runs atomically) or with conditional SQL updates, to avoid overselling.
- **Uber-style dispatch**: offering one driver to one rider at a time is the same "claim a resource atomically" problem.

In the interview

> **30-second answer:** "The critical section is 'find a free X and assign it'. Inside one JVM I make that a single atomic step — a lock-free queue poll for spots, per-seat locks taken in sorted order for multi-seat booking, a CAS decrement for stock. Across servers I'd push it into the database with a conditional update or unique constraint, and add holds with expiry for seats during payment."

Follow-ups (with answers):

- *Two users book the same seat at the same millisecond?* Only one `putIfAbsent`/locked check-and-book succeeds; the other gets "seat unavailable".
- *How do you release a seat if payment fails or the user leaves?* Store a hold with an expiry time; a scheduler or lazy check frees expired holds (see the TTL cache in chapter 13).
- *Why not one global lock?* Correct but no parallelism: bookings for different shows would wait for each other.

- *What if the service runs on 10 machines?* JVM locks don't cross processes: use DB transactions/constraints, optimistic locking with a version column, or a distributed lock with a lease.

Self-check

- **Q:** Which Coffman condition does sorting seat ids break? **A:** Circular wait.
- **Q:** Why is `queue.poll()` safe for allocating spots? **A:** It's a single atomic remove; each spot can be removed by only one thread.

TL;DR

Find the contended resource and its invariant, make "check + claim" one atomic step with the smallest tool, keep locks fine-grained and ordered, and move to DB/distributed mechanisms when there's more than one server.

12.26 Choosing the right tool (cheat sheet)

I need to…	Use
Increment a shared counter	`AtomicInteger` / `AtomicLong`; `LongAdder` if very hot
Stop a loop from another thread	`volatile boolean` flag or `interrupt()`
Make a multi-step update atomic	`synchronized` or `ReentrantLock`
Give up if the lock is busy	`tryLock(timeout)`
Many readers, few writers	`ReentrantReadWriteLock`, `StampedLock`, or immutable snapshot in `volatile`
Limit concurrent access to N	`Semaphore(N)`
Wait for a condition on shared state	`Lock` + `Condition` (or `wait/notifyAll`) in a `while` loop
Wait for N events once	`CountDownLatch`
N threads meet each phase	`CyclicBarrier` / `Phaser`
Hand work between threads	Bounded `BlockingQueue`
Shared map	`ConcurrentHashMap` with `putIfAbsent` / `compute` / `merge`
Read-mostly list	`CopyOnWriteArrayList`
Run tasks	`ExecutorService` (bounded); virtual threads for blocking I/O
Split CPU work	`ForkJoinPool` / parallel streams
Combine async results	`CompletableFuture`
Avoid synchronization entirely	Immutability, thread confinement

12.27 Chapter self-check

1. **Q:** Concurrency vs parallelism in one line each? **A:** Concurrency = many tasks in progress (structure); parallelism = many tasks executing at the same instant (hardware).
2. **Q:** Name the six `Thread.State` values. **A:** NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.
3. **Q:** Why is `volatile` not enough for `count++`? **A:** It gives visibility, not atomicity of read-modify-write.
4. **Q:** Three happens-before rules? **A:** Unlock → later lock of the same monitor; volatile write → later read; `start()` → thread's actions; thread's actions → `join()` return.
5. **Q:** Why always `while` around `wait()`? **A:** Spurious wakeups and competing threads; the condition must be re-checked.
6. **Q:** Four Coffman conditions? **A:** Mutual exclusion, hold and wait, no preemption, circular wait.
7. **Q:** Deadlock vs livelock vs starvation? **A:** Nobody progresses and all wait / everybody runs but nobody progresses / some progress while one never does.
8. **Q:** `CountDownLatch` vs `CyclicBarrier`? **A:** One-shot count of events vs reusable meeting of N threads.
9. **Q:** What is ABA and its fix? **A:** Value changes A→B→A so CAS can't notice; add a version (`AtomicStampedReference`).

10. **Q:** How to allocate a parking spot to exactly one of many gates without a lock? **A:** Keep free spots in a concurrent queue and `poll()` — an atomic remove.

12.28 Chapter TL;DR

- Races come from **shared mutable state**; fix by making critical sections atomic, or by removing sharing/mutability.
- Visibility needs **happens-before** edges: locks, `volatile`, start/join, concurrent utilities.
- Toolbox: `synchronized` → `ReentrantLock` (try/timed/fair/conditions) → RW locks → semaphores → latches/barriers → atomics/CAS → concurrent collections.
- Liveness: prevent deadlock with **lock ordering** or **tryLock + back-off**; add jitter against livelock; fairness against starvation.
- Patterns: signaling, thread pool, producer-consumer, reader-writer, immutability/confinement.
- In LLD interviews: identify the contended resource, keep "check + claim" atomic, lock fine-grained and in order, and say what changes when there are many servers.

Chapter 13 — Classic Concurrency Problems

> **Goal.** Chapter 12 gave you the toolbox. This chapter uses it on the concurrency problems that interviewers actually ask — from LeetCode's (a popular coding-practice website) "Concurrency" section (FooBar, Zero-Even-Odd, FizzBuzz, H2O, Dining Philosophers) to "design a thread-safe X" **machine-coding** rounds (interviews where you write working code for a small system in an hour or two): TTL cache, concurrent hash map, blocking queue, Bloom filter, parallel merge sort, **rate limiter** (a component that caps how many requests a client may make per second).

How each problem is laid out: *Problem → Approach → Java solution (+ real output) → Why it's correct → Pitfalls → Real-world examples → In the interview → Self-check → TL;DR.*

All code is Java 21, standard library only, in `code/lld/ch13/<problem>/Main.java`. Run from `chapters/13-concurrency-problems/code`:

```
javac --release 21 -d /tmp/lld_13 $(find . -name '*.java')
java -cp /tmp/lld_13 lld.ch13.foobar.Main
```

Where the real order of events can vary between runs, the demo *verifies* the result and prints `true/false`, so the output shown here is what you get every time.

Key terms (all built in Chapter 12 — a one-line reminder each):

- **Thread-safe** — behaves correctly however threads interleave. A **race** is a bug where the result depends on the timing of threads.
- **Lock / monitor** — a token only one thread can hold at a time. Every Java object has a built-in one (its *monitor*), taken by `synchronized`; `ReentrantLock` is the explicit version.
- **Semaphore** — a counter of permits: `acquire()` takes one (waiting if none are left), `release()` gives one back.
- **Condition variable** (`wait/notifyAll`, or `Condition` with a `ReentrantLock`) — lets a thread sleep, releasing its lock, until another thread changes the shared state and wakes it.
- **Barrier** (`CyclicBarrier`) — a meeting point where N threads wait until all N have arrived, then all continue.
- **Atomic / CAS** — an *atomic* step happens all at once or not at all; *compare-and-swap* updates a value only if it still equals what you read. CAS is the basis of **lock-free** code (no locks; a thread that loses simply retries).
- **Happens-before** — the Java Memory Model rule that guarantees one thread's writes are visible to another (e.g. through the same lock, a `volatile` field, or a semaphore release → acquire).
- **Deadlock / livelock / starvation** — threads wait on each other forever / keep retrying but nobody progresses / one thread never gets its turn.

Problem map — which tool solves what

#	Problem	Core idea	Main tools
13.1	Print FooBar Alternately	Pass a "baton" between 2 threads	`Semaphore` ×2 (or monitor + turn flag)
13.2	Print Zero Even Odd	A dispatcher thread decides who goes next	`Semaphore` ×3
13.3	Fizz Buzz Multithreaded	4 threads, one shared counter, each waits for "its" numbers	`synchronized` + `wait/notifyAll`
13.4	Building H2O	Group threads into sets of 2 H + 1 O	`Semaphore` (slots) + `CyclicBarrier` (group)
13.5	Dining Philosophers	Avoid deadlock over shared forks	Lock ordering, or a "waiter" semaphore
13.6	Thread-safe cache with TTL	Expiring entries, no stampede	`ConcurrentHashMap`, `compute`, scheduled sweeper
13.7	Concurrent HashMap (simplified)	Lock striping (N locks, each guarding a slice of the data)	`ReentrantLock[]`, all-stripe resize
13.8	Bounded Blocking Queue	Wait when full / empty	`wait/notifyAll` **and** `Lock` + 2 `Condition`s
13.9	Concurrent Bloom Filter	Lock-free bit set	`AtomicLongArray` + atomic OR
13.10	Multi-threaded Merge Sort	Divide and conquer on cores	`ForkJoinPool`, `RecursiveAction`
13.11	Rate limiter (token bucket)	Atomic refill-and-take	`synchronized`, or CAS on an immutable state

Reusable mental checklist for any concurrency problem

1. What is the **shared state**? (a turn flag, a counter, a map, a buffer)
2. What **ordering or invariant** must hold? ("foo before bar", "never more than 2 H in a molecule", "never more than capacity items")
3. Who **waits**, for **what condition**, and who **signals** it?
4. Pick the primitive: turn-taking → semaphores; "wait until predicate" → condition variable; groups → barrier; mutual exclusion → lock; single-variable update → atomic.
5. Check **liveness** (the guarantee that every thread eventually makes progress): can anyone wait forever? Does every thread exit at the end?

13.1 Print FooBar Alternately (LeetCode 1115)

Problem

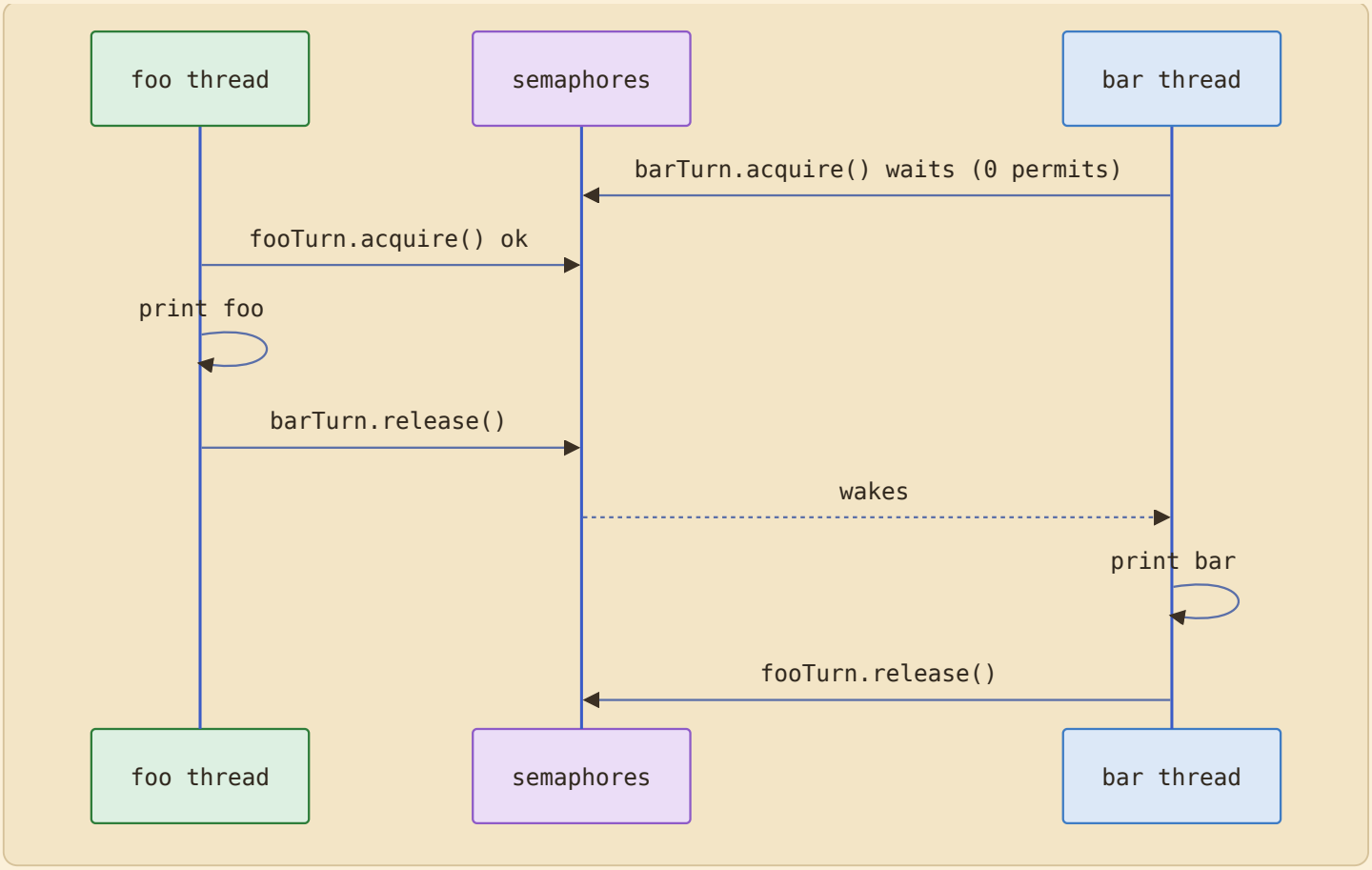
One `FooBar` object is shared by two threads. Thread A calls `foo()`, which prints `"foo"` *n* times; thread B calls `bar()`, which prints `"bar"` *n* times. Make the output exactly `"foobar"` repeated *n* times, whichever thread starts first.

Approach

This is pure **signaling** (12.20): a baton passes back and forth.

- `fooTurn = Semaphore(1)` — foo may go first.
- `barTurn = Semaphore(0)` — bar must wait.
- `foo`: acquire `fooTurn`, print, release `barTurn`. `bar`: acquire `barTurn`, print, release `fooTurn`.

Alternative: one monitor with a boolean `fooTurn`, `while (!fooTurn) wait();` then flip it and `notifyAll()`.


foo thread
semaphores
bar thread
barTurn.acquire() waits (0 permits)
fooTurn.acquire() ok
print foo
barTurn.release()
wakes
print bar
fooTurn.release()
foo thread
semaphores
bar thread

Java solution

```java
// File: code/lld/ch13/foobar/Main.java
package lld.ch13.foobar;

import java.util.concurrent.Semaphore;

/**
 * LeetCode 1115 - Print FooBar Alternately.
 * Thread A calls foo() n times, thread B calls bar() n times; output must be "foobar" repeated n times.
 */
public class Main {

    /** Solution 1: two semaphores used as a baton that is passed back and forth. */
    static class FooBarSemaphore {
        private final int n;
        private final Semaphore fooTurn = new Semaphore(1);   // foo may go first
        private final Semaphore barTurn = new Semaphore(0);   // bar must wait

        FooBarSemaphore(int n) { this.n = n; }

        void foo(Runnable printFoo) throws InterruptedException {
            for (int i = 0; i < n; i++) {
                fooTurn.acquire();       // wait for my turn
                printFoo.run();
                barTurn.release();       // hand the baton to bar
            }
        }
        void bar(Runnable printBar) throws InterruptedException {
            for (int i = 0; i < n; i++) {
                barTurn.acquire();
                printBar.run();
                fooTurn.release();
            }
        }
    }

    /** Solution 2: one monitor + a turn flag (wait/notifyAll). */
    static class FooBarMonitor {
        private final int n;
        private boolean fooTurn = true;                       // guarded by 'this'

        FooBarMonitor(int n) { this.n = n; }

        void foo(Runnable printFoo) throws InterruptedException {
            for (int i = 0; i < n; i++) {
                synchronized (this) {
                    while (!fooTurn) wait();
                    printFoo.run();
                    fooTurn = false;
                    notifyAll();
                }
            }
        }
        void bar(Runnable printBar) throws InterruptedException {
            for (int i = 0; i < n; i++) {
                synchronized (this) {
                    while (fooTurn) wait();
                    printBar.run();
                    fooTurn = true;
                    notifyAll();
                }
            }
        }
    }

    interface Body { void run() throws InterruptedException; }
    static Thread start(Body b) {
        Thread t = new Thread(() -> { try { b.run(); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); } });
        t.start();
        return t;
    }

    public static void main(String[] args) throws InterruptedException {
```

```
        StringBuffer out1 = new StringBuffer();                // StringBuffer is synchronized
        FooBarSemaphore s = new FooBarSemaphore(4);
        Thread b1 = start(() -> s.bar(() -> out1.append("bar")));   // start bar FIRST on purpose
        Thread a1 = start(() -> s.foo(() -> out1.append("foo")));
        a1.join(); b1.join();
        System.out.println("semaphores : " + out1);

        StringBuffer out2 = new StringBuffer();
        FooBarMonitor m = new FooBarMonitor(4);
        Thread b2 = start(() -> m.bar(() -> out2.append("bar")));
        Thread a2 = start(() -> m.foo(() -> out2.append("foo")));
        a2.join(); b2.join();
        System.out.println("monitor    : " + out2);
        System.out.println("both correct: " + (out1.toString().equals("foobar".repeat(4)) &&
out2.toString().equals("foobar".repeat(4))));
    }
}
```

Output:

```
semaphores : foobarfoobarfoobarfoobar
monitor    : foobarfoobarfoobarfoobar
both correct: true
```

Why it's correct

- **Invariant:** the permits in `fooTurn` + `barTurn` always total exactly 1 (each thread releases the *other* semaphore only after acquiring its own). So at most one thread can be printing, and the turns strictly alternate.
- Starting `bar` first doesn't matter: it blocks on `barTurn` (0 permits) until `foo` has printed.
- **Visibility:** a `release()` happens-before the `acquire()` it enables, so each thread sees the other's appends.
- **Termination:** both loops run exactly *n* times, and each acquire has a matching release.

Pitfalls

- Using one semaphore for both threads — a thread can re-acquire its own release and print twice.
- The monitor version with `if` instead of `while`, or `notify()` in more complex variants.
- **Busy-waiting** (spinning in a loop that keeps re-checking a flag) on a `volatile` flag (a field whose writes are immediately visible to other threads) works but burns a CPU core.

Real-world examples

Strict two-party hand-offs: ping-pong protocols, request/acknowledge stages, **double-buffering** (one buffer is filled while the other is read, then they swap) where a renderer and a producer alternate buffers.

In the interview

> **30-second answer:** "Two semaphores as a baton: foo's starts at 1, bar's at 0. Each thread acquires its own, prints, and releases the other's. The total permits are always one, so they alternate."

Follow-ups:

- *Three threads printing A, B, C in order?* Three semaphores in a ring: A(1), B(0), C(0); each releases the next.
- *Without semaphores?* A monitor with a turn variable and `wait/notifyAll`, or `Lock` + `Condition`.

Self-check

- **Q:** What happens if `barTurn` starts with 1? **A:** bar can print first → output may begin with "bar".

TL;DR

Two semaphores, one permit total, passed back and forth.

13.2 Print Zero Even Odd (LeetCode 1116)

Problem

Three threads share a `ZeroEvenOdd(n)` object. `zero()` may only print 0, `even()` only even numbers, `odd()` only odd numbers. Together they must print `0 1 0 2 0 3 … 0 n` — e.g. *n* = 5 → `0102030405`.

Approach

The zero thread acts as a **dispatcher**: after printing a 0 for position *i*, it releases `oddTurn` if *i* is odd, else `evenTurn`. The odd/even thread prints its number and hands control back via `zeroTurn`.

- `zeroTurn = 1`, `oddTurn = 0`, `evenTurn = 0`.
- zero loops *n* times; odd loops over 1, 3, 5…; even over 2, 4, 6….

Java solution

```java
// File: code/lld/ch13/zeroevenodd/Main.java
package lld.ch13.zeroevenodd;

import java.util.concurrent.Semaphore;
import java.util.function.IntConsumer;

/**
 * LeetCode 1116 - Print Zero Even Odd.
 * Three threads: zero() prints only 0s, even() only even numbers, odd() only odd numbers.
 * Together they must print 0 1 0 2 0 3 ... 0 n  (e.g. n=5 -> "0102030405").
 */
public class Main {

    static class ZeroEvenOdd {
        private final int n;
        private final Semaphore zeroTurn = new Semaphore(1);
        private final Semaphore oddTurn = new Semaphore(0);
        private final Semaphore evenTurn = new Semaphore(0);

        ZeroEvenOdd(int n) { this.n = n; }

        void zero(IntConsumer print) throws InterruptedException {
            for (int i = 1; i <= n; i++) {
                zeroTurn.acquire();
                print.accept(0);
                // The zero thread is the "dispatcher": it decides who prints next.
                if (i % 2 == 1) oddTurn.release(); else evenTurn.release();
            }
        }
        void odd(IntConsumer print) throws InterruptedException {
            for (int i = 1; i <= n; i += 2) {
                oddTurn.acquire();
                print.accept(i);
                zeroTurn.release();
            }
        }
        void even(IntConsumer print) throws InterruptedException {
            for (int i = 2; i <= n; i += 2) {
                evenTurn.acquire();
                print.accept(i);
                zeroTurn.release();
            }
        }
    }

    interface Body { void run() throws InterruptedException; }
    static Thread start(Body b) {
        Thread t = new Thread(() -> { try { b.run(); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); } });
        t.start();
        return t;
    }

    public static void main(String[] args) throws InterruptedException {
        for (int n : new int[] {1, 2, 5, 10}) {
            StringBuffer out = new StringBuffer();
            ZeroEvenOdd z = new ZeroEvenOdd(n);
            Thread e = start(() -> z.even(out::append));
            Thread o = start(() -> z.odd(out::append));
            Thread zt = start(() -> z.zero(out::append));
            zt.join(); o.join(); e.join();
            System.out.println("n=" + n + " -> " + out);
        }
    }
}
```

Output:

```
n=1 -> 01
n=2 -> 0102
n=5 -> 0102030405
n=10 -> 01020304050607080910
```

Why it's correct

- Exactly one semaphore holds a permit at any moment → exactly one thread may print.
- The zero thread's loop index *i* decides the parity of the next number, so the right thread is woken.
- Each thread's loop count matches the number of permits it will receive: zero gets *n*, odd gets ⌈n/2⌉, even gets ⌊n/2⌋. So nobody waits forever and everyone exits (e.g. *n* = 1: even's loop runs zero times).

Pitfalls

- Letting odd/even threads compute "the next number" from a shared counter without synchronization.
- Loop bounds that don't match the permits (e.g. even looping `n/2 + 1` times) → a thread waits forever and the program hangs.

Real-world examples

Dispatcher/worker designs where one coordinator routes each step to a specialised worker (e.g. an interpreter routing **opcodes** — the numeric codes of individual instructions — a scheduler assigning alternating tasks).

In the interview

> **30-second answer:** "Three semaphores. Zero starts with one permit. After each 0 it releases odd or even depending on the index, and they release zero back. One permit total in the system, so the order is forced."

Follow-up: *Could you use one lock + condition?* Yes: a shared `state` (ZERO/ODD/EVEN) and `current` number, each thread waits `while (state != mine)`, acts, updates state, `signalAll()`.

Self-check

- **Q:** For *n* = 4, how many times does `odd()` acquire? **A:** Twice (1 and 3).

TL;DR

Zero is the dispatcher; one permit circulates among three semaphores.

13.3 Fizz Buzz Multithreaded (LeetCode 1195)

Problem

Four threads share a `FizzBuzz(n)` object and must together output 1..n where: multiples of 15 → `fizzbuzz` (thread C), multiples of 3 only → `fizz` (thread A), multiples of 5 only → `buzz` (thread B), everything else → the number (thread D).

Approach

Keep one shared counter `current` guarded by the object's monitor. Each thread runs the same loop with its own predicate:

1. `while (current <= n && !mine(current)) wait();`
2. If `current > n` → `notifyAll()` and exit.
3. Print, `current++`, `notifyAll()` (the next number may belong to anyone).

This is the **condition variable** pattern (12.13) with a different predicate per thread. The helper `runWhile(predicate, action)` avoids writing the loop four times (DRY: Don't Repeat Yourself).

Java solution

```java
// File: code/lld/ch13/fizzbuzz/Main.java
package lld.ch13.fizzbuzz;

import java.util.ArrayList;
import java.util.List;
import java.util.function.IntConsumer;
import java.util.function.IntPredicate;

/**
 * LeetCode 1195 - Fizz Buzz Multithreaded.
 * Four threads share one counter 1..n: fizz() prints "fizz" (divisible by 3 only), buzz() "buzz"
 * (by 5 only), fizzbuzz() "fizzbuzz" (by 15), number() prints the number otherwise.
 */
public class Main {

    static class FizzBuzz {
        private final int n;
        private int current = 1;                            // guarded by 'this'

        FizzBuzz(int n) { this.n = n; }

        /** Shared loop: wait until 'current' is mine (or we're done), act, advance, wake everyone. */
        private void runWhile(IntPredicate mine, IntConsumer action) throws InterruptedException {
            while (true) {
                synchronized (this) {
                    while (current <= n && !mine.test(current)) wait();
                    if (current > n) { notifyAll(); return; }     // done: wake others so they can exit too
                    action.accept(current);
                    current++;
                    notifyAll();                                  // the next number may belong to anyone
                }
            }
        }

        void fizz(Runnable printFizz) throws InterruptedException {
            runWhile(i -> i % 3 == 0 && i % 5 != 0, i -> printFizz.run());
        }
        void buzz(Runnable printBuzz) throws InterruptedException {
            runWhile(i -> i % 5 == 0 && i % 3 != 0, i -> printBuzz.run());
        }
        void fizzbuzz(Runnable printFizzBuzz) throws InterruptedException {
            runWhile(i -> i % 15 == 0, i -> printFizzBuzz.run());
        }
        void number(IntConsumer printNumber) throws InterruptedException {
            runWhile(i -> i % 3 != 0 && i % 5 != 0, printNumber);
        }
    }

    interface Body { void run() throws InterruptedException; }
    static Thread start(Body b) {
        Thread t = new Thread(() -> { try { b.run(); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); } });
        t.start();
        return t;
    }

    public static void main(String[] args) throws InterruptedException {
        int n = 15;
        List<String> out = new ArrayList<>();          // only touched inside the FizzBuzz monitor
        FizzBuzz fb = new FizzBuzz(n);
        List<Thread> ts = List.of(
                start(() -> fb.fizz(() -> out.add("fizz"))),
                start(() -> fb.buzz(() -> out.add("buzz"))),
                start(() -> fb.fizzbuzz(() -> out.add("fizzbuzz"))),
                start(() -> fb.number(i -> out.add(String.valueOf(i)))));
        for (Thread t : ts) t.join();                  // join() makes the list's contents visible to main
        System.out.println(String.join(", ", out));
    }
}
```

Output:

```
1, 2, fizz, 4, buzz, fizz, 7, 8, fizz, buzz, 11, fizz, 13, 14, fizzbuzz
```

Why it's correct

- Every number satisfies **exactly one** predicate (they partition 1..n), so exactly one thread can act on each value.
- `current` is read and written only inside `synchronized (this)` → no races, full visibility.
- `notifyAll()` (not `notify()`): four different predicates wait on the same monitor; `notify()` could wake a thread whose predicate is false, which then waits again — and the right thread is never woken → hang.
- **Termination:** when `current` exceeds *n*, every waiting thread is woken (by the last `notifyAll`) and sees `current > n` in its loop condition, so all four exit.
- The output list is only modified inside the monitor, and `join()` makes it visible to `main`.

Pitfalls

- `notify()` instead of `notifyAll()` → lost wake-ups, random hangs.
- Forgetting the `current > n` exit check inside the wait loop → threads wait forever after the last number.
- Overlapping predicates (e.g. fizz = `i % 3 == 0` without excluding 15) → two threads compete for 15.

Real-world examples

"Workers wait until a shared state matches their role" — e.g. pipeline stages each waiting for a job to reach their stage in a shared state machine.

In the interview

> **30-second answer:** "One monitor, one shared counter. Each thread waits in a while loop until the counter matches its predicate or passes n, prints, increments and notifies all. The predicates partition the numbers, so exactly one thread acts per number."

Follow-up: *Semaphore version?* Possible: the number thread acts as dispatcher and releases the fizz/buzz/fizzbuzz semaphore for the next value — more efficient (no **thundering herd** — many threads woken at once when only one can proceed) but more code.

Self-check

- **Q:** Why `notifyAll`? **A:** Threads wait for *different* conditions on the same monitor; only waking all guarantees the right one wakes.

TL;DR

Shared counter + per-thread predicate + `while`-wait + `notifyAll`.

13.4 Building H2O (LeetCode 1117)

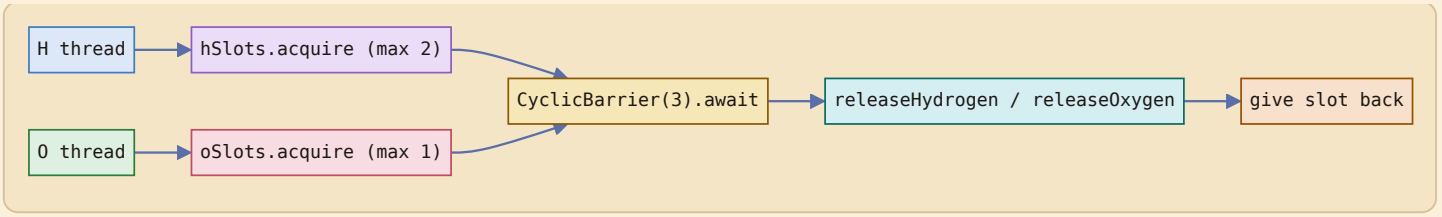
Problem

Hydrogen and oxygen threads arrive in any order. Each must pass a barrier in groups of three that form one water molecule: exactly **2 H + 1 O**. Threads from one molecule must all pass before any thread of the next molecule passes.

Approach

Combine two tools:

- **Slots** — `hSlots = Semaphore(2)`, `oSlots = Semaphore(1)`: at most 2 H and 1 O can be "in" the current molecule.
- **Group** — `CyclicBarrier(3)`: the three atoms wait for each other, then all go.
- A thread returns its slot permit **after** it has been released (printed), so the next molecule can only form once this one is finished.


H thread
hSlots.acquire (max 2)
O thread
oSlots.acquire (max 1)
CyclicBarrier(3).await
releaseHydrogen / releaseOxygen
give slot back

Java solution

```java
// File: code/lld/ch13/h2o/Main.java
package lld.ch13.h2o;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.BrokenBarrierException;
import java.util.concurrent.CyclicBarrier;
import java.util.concurrent.Semaphore;

/**
 * LeetCode 1117 - Building H2O.
 * Hydrogen and oxygen threads arrive in any order. They must leave in groups of three:
 * exactly 2 H + 1 O per group, and a group must fully leave before the next one starts.
 */
public class Main {

    static class H2O {
        private final Semaphore hSlots = new Semaphore(2);    // at most 2 H in the current molecule
        private final Semaphore oSlots = new Semaphore(1);    // at most 1 O in the current molecule
        private final CyclicBarrier bond = new CyclicBarrier(3);   // wait until all 3 atoms are present

        void hydrogen(Runnable releaseHydrogen) throws InterruptedException {
            hSlots.acquire();
            try {
                bond.await();
            } catch (BrokenBarrierException e) { throw new IllegalStateException(e); }
            releaseHydrogen.run();
            hSlots.release();                                 // free my slot only after I have "left"
        }
        void oxygen(Runnable releaseOxygen) throws InterruptedException {
            oSlots.acquire();
            try {
                bond.await();
            } catch (BrokenBarrierException e) { throw new IllegalStateException(e); }
            releaseOxygen.run();
            oSlots.release();
        }
    }

    public static void main(String[] args) throws InterruptedException {
        // 16 H + 8 O = 8 molecules, arriving in a scrambled (but reproducible) order.
        List<Character> atoms = new ArrayList<>();
        for (int i = 0; i < 8; i++) { atoms.add('H'); atoms.add('H'); atoms.add('O'); }
        java.util.Collections.shuffle(atoms, new java.util.Random(7));
        StringBuilder in = new StringBuilder();
        atoms.forEach(in::append);
        String input = in.toString();
        System.out.println("arrival order   : " + input);
        StringBuffer out = new StringBuffer();
        H2O h2o = new H2O();
        List<Thread> ts = new ArrayList<>();
        for (char c : input.toCharArray()) {
            Thread t = new Thread(() -> {
                try {
                    if (c == 'H') h2o.hydrogen(() -> out.append('H'));
                    else h2o.oxygen(() -> out.append('O'));
                } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            });
            ts.add(t);
            t.start();
        }
        for (Thread t : ts) t.join();

        // Output order varies run to run (HHO, HOH, OHH are all fine), so we verify it instead.
        String s = out.toString();
        boolean valid = s.length() == input.length();
        for (int i = 0; valid && i < s.length(); i += 3) {
            String g = s.substring(i, i + 3);
            valid = g.chars().filter(ch -> ch == 'H').count() == 2 && g.indexOf('O') >= 0;
        }
        System.out.println("molecules formed: " + s.length() / 3 + ", every group of 3 has 2 H + 1 O: " + valid);
    }
}
```

Output:

```
arrival order   : OHOOOHHHHOHHHHHOOHHHHHOH
molecules formed: 8, every group of 3 has 2 H + 1 O: true
```

Why it's correct

- **At most 2 H and 1 O** can reach the barrier in each round because of the slot semaphores; the barrier needs 3 parties, so each trip is exactly `H, H, O`.
- **No mixing between molecules:** a new molecule needs all three slots free again. Each slot is freed only after its thread printed. So the next group can't trip the barrier (and print) until all three of the previous group have printed.
- The barrier resets automatically (it's *cyclic*) for the next molecule.
- The demo checks every consecutive group of 3 in the output instead of printing it, because `HHO`, `HOH` and `OHH` are all valid orders.

Pitfalls

- Releasing the slot **before** printing → a new H might print before the previous O, mixing molecules.
- Using only semaphores without the barrier → the 2 H and 1 O aren't forced to leave "together".
- If the thread counts aren't exactly 2:1, the last threads wait forever (the problem guarantees they are).

Real-world examples

Batching: gather exactly N items of certain types before dispatching (e.g. matchmaking 2 players + 1 server slot, assembling an order that needs parts from different producers).

In the interview

> **30-second answer:** "Two semaphores cap the molecule at 2 H and 1 O; a CyclicBarrier of 3 makes the three wait for each other; each atom returns its slot only after it's released, so molecules never interleave."

Follow-up: *Without a barrier?* Use a lock with counters of waiting H and O; when 2 H + 1 O are available, the thread completing the set signals exactly those. More code, same idea.

Self-check

- **Q:** Why is `hSlots` initialised to 2, not 1? **A:** A molecule needs two hydrogens simultaneously inside the barrier.

TL;DR

Slots (semaphores) decide *who* can join; the barrier decides *when* they leave; free slots only after leaving.

13.5 Dining Philosophers (LeetCode 1226)

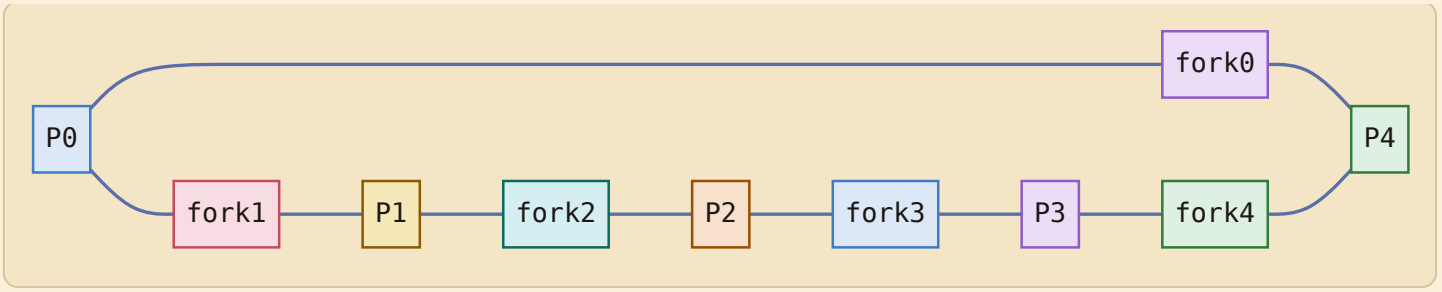
Problem

Five philosophers sit at a round table with one fork between each pair (5 forks). To eat, a philosopher needs **both** adjacent forks. If everyone picks up their left fork first, everyone may hold one fork and wait forever for the right one — **deadlock** (all four **Coffman conditions** hold — mutual exclusion, hold-and-wait, no preemption and circular wait; 12.17).

Approach

Break a Coffman condition:

1. **Resource ordering** (breaks circular wait): number the forks 0–4; always pick up the **lower-numbered** fork first. Philosopher 4 (forks 4 and 0) picks fork 0 first, so the cycle is impossible.
2. **Waiter / limited seats** (also breaks circular wait): a semaphore lets at most **n − 1** philosophers try at once — with 4 people and 5 forks, at least one can get both forks. Make the semaphore **fair** to prevent starvation.
3. (Other known solutions: `tryLock` both forks and put down on failure with random **back-off** (waiting a random short time before retrying); the Chandy–Misra algorithm for distributed settings.)


fork0
P0
P4
fork1
P1
fork2
P2
fork3
P3
fork4

Java solution

```java
// File: code/lld/ch13/philosophers/Main.java
package lld.ch13.philosophers;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Semaphore;
import java.util.concurrent.atomic.AtomicIntegerArray;
import java.util.concurrent.locks.ReentrantLock;

/**
 * Dining Philosophers (LeetCode 1226 variant): 5 philosophers, 5 forks, each needs both
 * neighbouring forks to eat. Naive "everyone picks up the left fork first" can deadlock.
 */
public class Main {

    interface Table { void eat(int p, Runnable eatAction) throws InterruptedException; }

    /** Fix 1 - resource ordering: always pick up the LOWER-numbered fork first (breaks circular wait). */
    static class OrderedForks implements Table {
        private final ReentrantLock[] forks;
        OrderedForks(int n) { forks = new ReentrantLock[n]; for (int i = 0; i < n; i++) forks[i] = new ReentrantLock(); }

        public void eat(int p, Runnable eatAction) {
            int left = p, right = (p + 1) % forks.length;
            ReentrantLock first = forks[Math.min(left, right)], second = forks[Math.max(left, right)];
            first.lock();
            try {
                second.lock();
                try { eatAction.run(); } finally { second.unlock(); }
            } finally { first.unlock(); }
        }
    }

    /** Fix 2 - a "waiter" lets at most n-1 philosophers try at once (breaks circular wait too). */
    static class Waiter implements Table {
        private final ReentrantLock[] forks;
        private final Semaphore seats;
        Waiter(int n) {
            forks = new ReentrantLock[n];
            for (int i = 0; i < n; i++) forks[i] = new ReentrantLock();
            seats = new Semaphore(n - 1, true);                // fair: nobody starves waiting for a seat
        }
        public void eat(int p, Runnable eatAction) throws InterruptedException {
            seats.acquire();
            try {
                ReentrantLock left = forks[p], right = forks[(p + 1) % forks.length];
                left.lock();
                try {
                    right.lock();
                    try { eatAction.run(); } finally { right.unlock(); }
                } finally { left.unlock(); }
            } finally { seats.release(); }
        }
    }

    static String dine(Table table, int n, int meals) throws InterruptedException {
        AtomicIntegerArray eaten = new AtomicIntegerArray(n);
        List<Thread> ts = new ArrayList<>();
        for (int p = 0; p < n; p++) {
            final int id = p;
            Thread t = new Thread(() -> {
                try {
                    for (int m = 0; m < meals; m++) {
                        Thread.onSpinWait();                                // "think"
                        table.eat(id, () -> eaten.incrementAndGet(id));
                    }
                } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            });
            ts.add(t); t.start();
        }
        for (Thread t : ts) t.join();
        return eaten.toString();
    }
```

```
    public static void main(String[] args) throws InterruptedException {
        System.out.println("resource ordering, meals per philosopher: " + dine(new OrderedForks(5), 5, 10_000));
        System.out.println("waiter (4 seats),  meals per philosopher: " + dine(new Waiter(5), 5, 10_000));
    }
}
```

Output:

```
resource ordering, meals per philosopher: [10000, 10000, 10000, 10000, 10000]
waiter (4 seats),  meals per philosopher: [10000, 10000, 10000, 10000, 10000]
```

Why it's correct

- **Ordering:** every philosopher acquires forks in increasing index order. A cycle of waiting would need someone to hold a higher fork while waiting for a lower one — impossible. No circular wait → no deadlock.
- **Waiter:** with at most 4 philosophers competing for 5 forks, by the **pigeonhole principle** (if there are more items than boxes, some box must get two) at least one of them can get both forks, eat, and release — so progress is always possible. The fair semaphore gives waiting philosophers seats in FIFO (first in, first out) order.
- Forks are `ReentrantLock`s released in `finally`, so an exception can't leave a fork held.
- 5 × 10,000 meals complete with each philosopher eating exactly 10,000 times.

Pitfalls

- "Everyone picks left then right" → deadlock.
- `tryLock` both forks and immediately retry without random back-off → livelock.
- Unfair strategies can starve one philosopher in theory (neighbours keep winning). The fair semaphore makes *seat* access first-come-first-served; for a strict no-starvation guarantee on the forks as well, make the fork locks fair too (`new ReentrantLock(true)`), at a throughput cost.

Real-world examples

Any time a task needs **two or more locks** at once: bank transfers (two accounts), moving an item between two inventories, database transactions updating several rows — solved by ordering (account id), or by the DB's deadlock detection aborting one transaction.

In the interview

30-second answer: "Naively everyone takes the left fork and waits for the right — circular wait, deadlock. I break circular wait by always taking the lower-numbered fork first, or by a waiter semaphore allowing only n−1 philosophers to try; fairness prevents starvation."

Follow-ups:

- *Which Coffman condition does each fix break?* Both break circular wait; the tryLock variant breaks hold-and-wait.
- *Maximum parallelism?* With 5 forks, at most 2 philosophers can eat at the same time.

Self-check

- **Q:** With resource ordering, which philosopher behaves "differently"? **A:** The one between forks 4 and 0 — they pick fork 0 (right) first.

TL;DR

Break circular wait: global fork order, or allow only n−1 diners.

13.6 Thread-safe cache with TTL

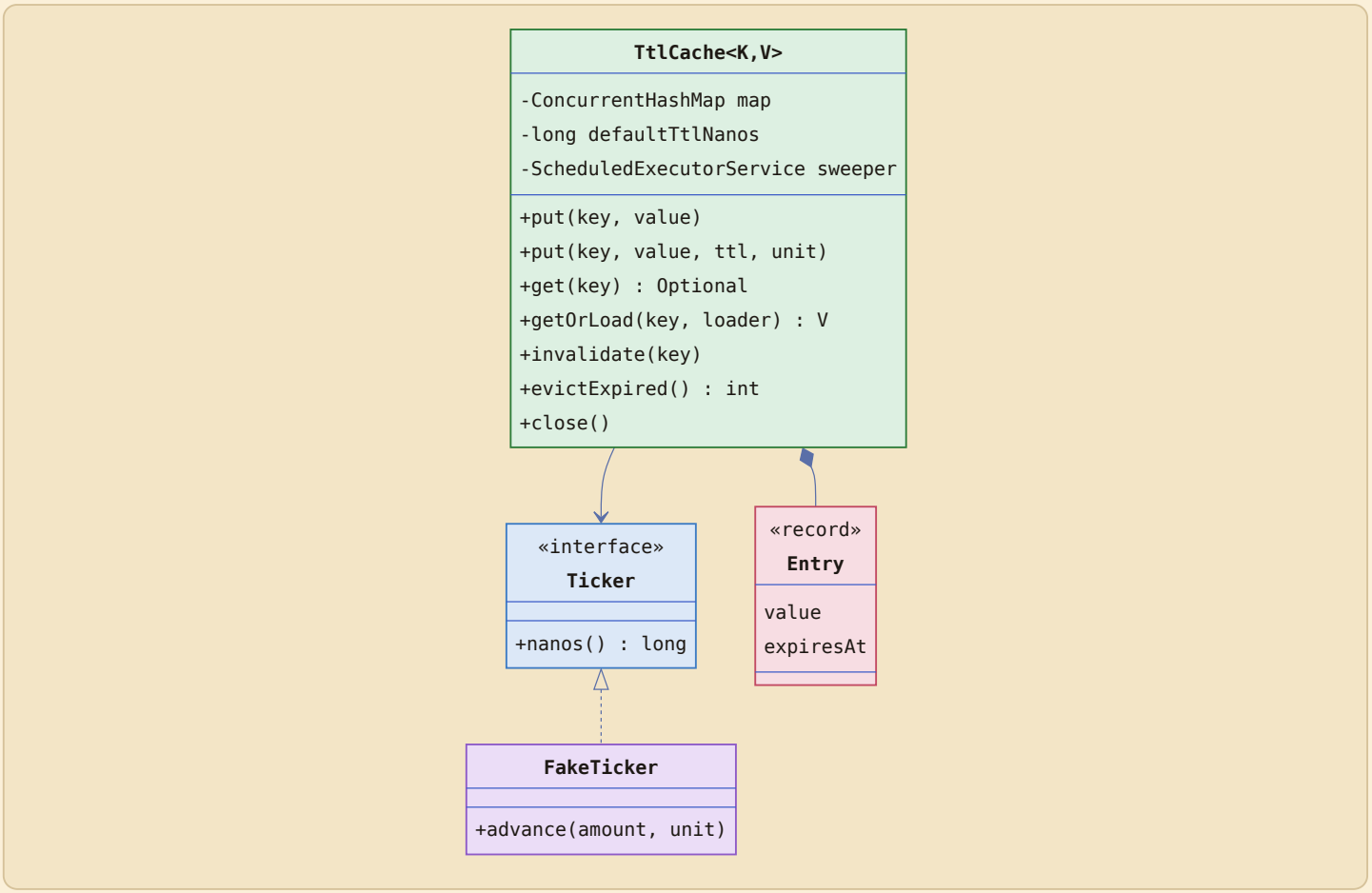
Problem

Design an in-memory key-value **cache** (a fast store that keeps copies of data for quick reads; see System Design Fundamentals) used by many threads where each entry expires after a **time-to-live** (TTL). Requirements: `put(k,`

`v)` (with default or per-entry TTL), `get(k)` returns nothing for expired entries, `getOrLoad(k, loader)` loads on a miss without a **cache stampede** (many threads loading the same key at once), expired entries are eventually removed from memory, and it's testable without sleeping.

Approach

- Store `Entry(value, expiresAt)` records (immutable) in a `ConcurrentHashMap`.
- **Lazy expiry**: `get` checks `expiresAt`; if expired, removes it with the **conditional** `remove(key, entry)` so it never deletes a fresher value another thread just put.
- **Active expiry**: a **daemon** (background thread that does not keep the JVM — Java Virtual Machine — running) `ScheduledExecutorService` (a JDK thread pool that runs tasks on a timer) periodically calls `evictExpired()` so keys nobody reads again don't leak memory.
- **No stampede**: `getOrLoad` uses `map.compute(...)`, which runs the function atomically per key; concurrent callers for the same key wait and the loader runs once.
- **Testability**: time comes from an injected `Ticker` (a `FakeTicker` in tests) — a small application of **Dependency Inversion** (depending on an interface rather than a concrete class, so the real clock can be swapped for a fake one).



Java solution

```
// File: code/lld/ch13/ttlcache/Main.java
package lld.ch13.ttlcache;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;
import java.util.function.Function;

/** Thread-safe key-value cache where every entry expires after a time-to-live (TTL). */
public class Main {

    /** Time source. Injected so tests can move time forward without sleeping. */
    @FunctionalInterface interface Ticker { long nanos(); }

    static final class FakeTicker implements Ticker {
        private final AtomicLong now = new AtomicLong();
        public long nanos() { return now.get(); }
        void advance(long amount, TimeUnit unit) { now.addAndGet(unit.toNanos(amount)); }
    }

    static final class TtlCache<K, V> implements AutoCloseable {
        private record Entry<V>(V value, long expiresAt) { }

        private final ConcurrentHashMap<K, Entry<V>> map = new ConcurrentHashMap<>();
        private final long defaultTtlNanos;
        private final Ticker ticker;
        private final ScheduledExecutorService sweeper;       // null = lazy expiry only

        TtlCache(long ttl, TimeUnit unit, Ticker ticker, long sweepEveryMillis) {
            this.defaultTtlNanos = unit.toNanos(ttl);
            this.ticker = ticker;
            if (sweepEveryMillis > 0) {
                sweeper = Executors.newSingleThreadScheduledExecutor(r -> {
                    Thread t = new Thread(r, "ttl-sweeper");
                    t.setDaemon(true);                          // never blocks JVM shutdown
                    return t;
                });
                // 'this' escapes to the sweeper thread before the constructor ends (a pattern 12.6 warns
                // about). It is safe HERE only because every field evictExpired() uses (map, ticker) is
                // already assigned and handing a task to an executor is a happens-before edge. In
                // production code prefer a static factory that starts the sweeper after construction.
                sweeper.scheduleAtFixedRate(this::evictExpired, sweepEveryMillis, sweepEveryMillis,
TimeUnit.MILLISECONDS);
            } else {
                sweeper = null;
            }
        }

        private boolean expired(Entry<V> e) { return ticker.nanos() - e.expiresAt() >= 0; }   // overflow-safe compare

        void put(K key, V value) { put(key, value, defaultTtlNanos, TimeUnit.NANOSECONDS); }

        void put(K key, V value, long ttl, TimeUnit unit) {
            Objects.requireNonNull(key); Objects.requireNonNull(value);
            map.put(key, new Entry<>(value, ticker.nanos() + unit.toNanos(ttl)));
        }

        Optional<V> get(K key) {
            Entry<V> e = map.get(key);
            if (e == null) return Optional.empty();
            if (expired(e)) {
                map.remove(key, e);   // conditional remove: never deletes a FRESH value another thread just put
                return Optional.empty();
            }
```

```
            return Optional.of(e.value());
        }

        /**
         * Read-through: if missing/expired, load it. compute() locks the key's bin, so concurrent callers
         * for the SAME key wait and the loader runs once (no "cache stampede").
         * The loader must be fast and must not touch this cache.
         */
        V getOrLoad(K key, Function<? super K, ? extends V> loader) {
            return map.compute(key, (k, old) ->
                    old != null && !expired(old) ? old : new Entry<>(loader.apply(k), ticker.nanos() + defaultTtlNanos)
            ).value();
        }

        void invalidate(K key) { map.remove(key); }

        /** Active expiry: drop every expired entry. Safe to run concurrently with readers and writers. */
        int evictExpired() {
            int removed = 0;
            for (Map.Entry<K, Entry<V>> me : map.entrySet()) {
                if (expired(me.getValue()) && map.remove(me.getKey(), me.getValue())) removed++;
            }
            return removed;
        }

        int rawSize() { return map.size(); }   // includes not-yet-swept expired entries

        @Override public void close() { if (sweeper != null) sweeper.shutdownNow(); }
    }

    public static void main(String[] args) throws Exception {
        FakeTicker clock = new FakeTicker();
        try (TtlCache<String, String> cache = new TtlCache<>(100, TimeUnit.MILLISECONDS, clock, 0)) {
            cache.put("session:42", "alice");
            cache.put("otp:alice", "913377", 30, TimeUnit.MILLISECONDS);   // per-entry TTL
            clock.advance(50, TimeUnit.MILLISECONDS);
            System.out.println("t=50ms  session=" + cache.get("session:42").orElse("<expired>")
                    + ", otp=" + cache.get("otp:alice").orElse("<expired>"));
            clock.advance(60, TimeUnit.MILLISECONDS);
            System.out.println("t=110ms session=" + cache.get("session:42").orElse("<expired>"));

            for (int i = 0; i < 5; i++) cache.put("k" + i, "v" + i);
            clock.advance(200, TimeUnit.MILLISECONDS);
            System.out.println("before sweep raw size=" + cache.rawSize() + ", swept=" + cache.evictExpired()
                    + ", after=" + cache.rawSize());

            // Stampede test: 16 threads miss the same key at the same moment.
            AtomicInteger loads = new AtomicInteger();
            CountDownLatch go = new CountDownLatch(1);
            List<Thread> ts = new ArrayList<>();
            List<String> results = java.util.Collections.synchronizedList(new ArrayList<>());
            for (int i = 0; i < 16; i++) {
                Thread t = new Thread(() -> {
                    try { go.await(); } catch (InterruptedException e) { return; }
                    results.add(cache.getOrLoad("user:7", k -> { loads.incrementAndGet(); return "profile-of-" + k; }));
                });
                ts.add(t); t.start();
            }
            go.countDown();
            for (Thread t : ts) t.join();
            System.out.println("16 concurrent misses -> loader ran " + loads.get() + " time(s), all got same value: "
                    + results.stream().allMatch("profile-of-user:7"::equals));
        }

        // Background sweeper with the real clock.
        try (TtlCache<Integer, Integer> real = new TtlCache<>(30, TimeUnit.MILLISECONDS, System::nanoTime, 10)) {
            for (int i = 0; i < 1_000; i++) real.put(i, i * i);
            System.out.println("real clock: size right after puts=" + real.rawSize());
            Thread.sleep(200);
            System.out.println("real clock: size after 200 ms (sweeper ran)=" + real.rawSize());
        }
    }
}
```

Output:

```
t=50ms  session=alice, otp=<expired>
t=110ms session=<expired>
before sweep raw size=5, swept=5, after=0
16 concurrent misses -> loader ran 1 time(s), all got same value: true
real clock: size right after puts=1000
real clock: size after 200 ms (sweeper ran)=0
```

Why it's correct

- All shared state is inside one `ConcurrentHashMap`; entries are immutable records, so a reader sees either the old or the new entry, never a half-written one.
- `remove(key, e)` removes only if the mapping is still *that exact entry*. If another thread replaced it with a fresh one after our read, the fresh one survives.
- `compute` executes the remapping function atomically for that key, so 16 simultaneous misses cause one load (the output shows `loader ran 1 time(s)`).
- Expiry uses `now - expiresAt >= 0` rather than `now >= expiresAt`, which stays correct even if `System.nanoTime()` values **overflow** (wrap around after passing the largest value a `long` can hold) (the JDK recommends comparing nano times by subtraction).
- The sweeper is a **daemon** thread and is shut down in `close()`, so it never keeps the JVM alive.
- The constructor hands `this::evictExpired` to the sweeper before it returns (`this` escapes). That is normally a bug (12.6), and it is safe here only because the fields the sweeper uses are already set and submitting to an executor creates a happens-before edge; a **static factory method** (a static method that builds and returns the object) that starts the sweeper after construction is the cleaner production choice.

Pitfalls

- Using `System.currentTimeMillis()` for TTL: wall-clock time can jump (NTP adjustments — the Network Time Protocol corrects the system clock). `System.nanoTime()` is **monotonic** (it only ever moves forward) for measuring durations.
- A slow loader inside `compute` blocks other keys that share the same hash bin, and a loader that touches the same map can throw or deadlock. For slow loads, store a `CompletableFuture<V>` (a JDK placeholder for a value that an asynchronous task will fill in later) in the map instead (`computeIfAbsent(k, key -> supplyAsync(...))`) so the bin lock is held only briefly.
- Only lazy expiry → memory leak for keys never read again. Only active expiry → a `get` could return a stale value between sweeps. Use both.
- No size limit: a real cache also needs a maximum size and an **eviction policy** (the rule for which entry to throw out when full: LRU = least recently used, LFU = least frequently used).

Real-world examples

- **Caffeine** (the de-facto Java caching library) offers `expireAfterWrite`/`expireAfterAccess`, size-based eviction, and a loading cache that prevents stampedes — the production version of this design. Guava's (Google's core Java library) `CacheBuilder` is its predecessor.
- **Redis** `EXPIRE`/`SET … EX` uses the same combination: lazy expiry on access plus periodic active sampling of keys with TTLs.
- Sessions, OTPs (one-time passwords), DNS caches (DNS is the internet's lookup from names to IP addresses), and seat holds in booking systems (a hold is a TTL entry).

In the interview

> **30-second answer:** "A ConcurrentHashMap of immutable entries holding value and expiry time. Get checks expiry lazily and removes with a conditional remove; a daemon scheduled sweeper removes expired keys nobody reads. getOrLoad uses compute so concurrent misses load once. Time is injected for testing."

Follow-ups (with answers):

- *Add max size with LRU?* Combine with an access-ordered `LinkedHashMap` under a lock, or a `ConcurrentHashMap` plus a concurrent linked **deque** (double-ended queue) of keys updated on access — or just use Caffeine (Window-TinyLFU).
- *Refresh without blocking readers?* "Refresh-ahead": serve the old value while one thread reloads asynchronously when the entry is close to expiring.

- *Distributed?* Redis/Memcached (in-memory cache servers shared by many machines) with TTLs; keep a small local cache in front for hot keys.

Self-check

- **Q:** Why `remove(key, e)` and not `remove(key)` on expiry? **A:** To avoid deleting a new value written between our read and our remove.

TL;DR

CHM (`ConcurrentHashMap`) + immutable (value, expiry) entries; lazy + scheduled expiry; `compute` for **single-flight** loads (one load per key, shared by every waiting caller); injected clock.

13.7 Design a (simplified) Concurrent HashMap — lock striping

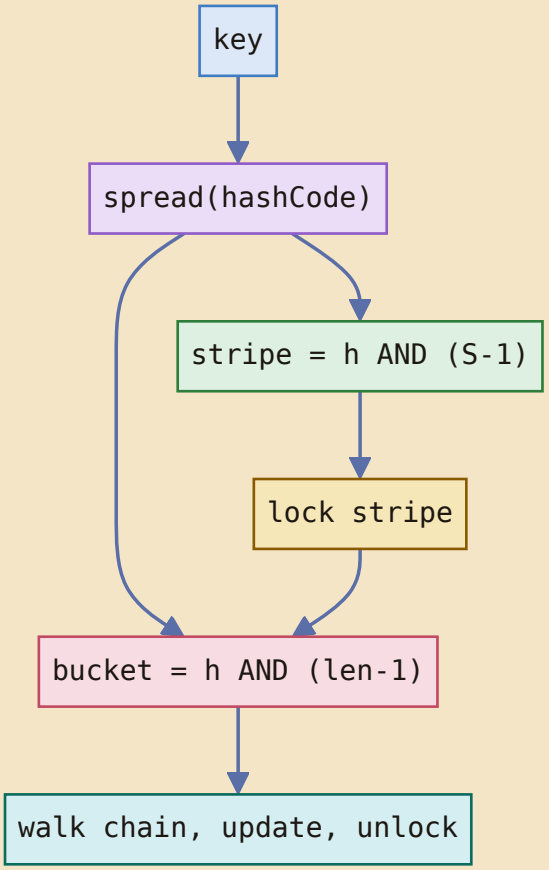
Problem

Implement a hash map that many threads can use at once with `get`, `put`, `remove`, `compute` and automatic resizing — faster than one global lock around a `HashMap`.

Approach

Lock striping (12.11):

- Buckets: an array of linked lists (**separate chaining**: each bucket holds a list of all entries whose hash lands there).
- Stripes: a fixed array of `S` locks (power of two). Key with hash *h* uses stripe `h & (S − 1)` and bucket `h & (tableLength − 1)`.
- Because the stripe depends only on the hash — **not** on the table length — a key's stripe never changes after a resize.
- Every operation locks only its stripe; threads on different stripes run in parallel.
- **Resize** (when size > 0.75 × buckets): acquire **all** stripe locks in index order (a global order → no deadlock), re-check the condition, **rehash** (recompute every key's bucket) into a table twice as big, publish via a `volatile` field, release in reverse.
- Size tracked with an `AtomicInteger`.


key
spread(hashCode)
stripe = h AND (S-1)
lock stripe
bucket = h AND (len-1)
walk chain, update, unlock

Java solution

```java
// File: code/lld/ch13/stripedmap/Main.java
package lld.ch13.stripedmap;

import java.util.ArrayList;
import java.util.List;
import java.util.Objects;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.locks.ReentrantLock;
import java.util.function.BiFunction;

/**
 * A simplified concurrent hash map using LOCK STRIPING: the buckets are guarded by a small array
 * of locks, so threads touching different stripes never block each other.
 * (Java 7's ConcurrentHashMap used this idea with "segments".)
 */
public class Main {

    static final class StripedHashMap<K, V> {
        private static final class Node<K, V> {
            final K key; final int hash; V value; Node<K, V> next;
            Node(K key, int hash, V value, Node<K, V> next) { this.key = key; this.hash = hash; this.value = value;
this.next = next; }
        }

        private volatile Node<K, V>[] table;                // replaced (never mutated in place) only while ALL locks are
held
        private final ReentrantLock[] stripes;
        private final AtomicInteger size = new AtomicInteger();
        private static final float LOAD_FACTOR = 0.75f;

        @SuppressWarnings({"unchecked", "rawtypes"})
        StripedHashMap(int initialBuckets, int stripeCount) {
            if (Integer.bitCount(stripeCount) != 1 || Integer.bitCount(initialBuckets) != 1)
                throw new IllegalArgumentException("sizes must be powers of two");
            if (stripeCount > initialBuckets)                // guarantees: every bucket maps to exactly one stripe
                throw new IllegalArgumentException("need stripes <= buckets");
            table = (Node<K, V>[]) new Node[initialBuckets];
            stripes = new ReentrantLock[stripeCount];
            for (int i = 0; i < stripeCount; i++) stripes[i] = new ReentrantLock();
        }

        private static int spread(Object key) {
            int h = key.hashCode();
            return (h ^ (h >>> 16)) & 0x7fffffff;            // mix high bits in, keep it non-negative
        }
        // A key's stripe depends only on its hash, NOT on the table size, so it never changes on resize.
        private ReentrantLock stripeFor(int hash) { return stripes[hash & (stripes.length - 1)]; }
        private static int indexFor(int hash, int len) { return hash & (len - 1); }

        V get(K key) {
            Objects.requireNonNull(key);
            int h = spread(key);
            ReentrantLock lock = stripeFor(h);
            lock.lock();
            try {
                Node<K, V>[] t = table;                          // stable while we hold a stripe lock
                for (Node<K, V> n = t[indexFor(h, t.length)]; n != null; n = n.next)
                    if (n.hash == h && n.key.equals(key)) return n.value;
                return null;
            } finally { lock.unlock(); }
        }

        /** Returns the previous value (or null). */
        V put(K key, V value) {
            Objects.requireNonNull(value);
            return doCompute(key, (k, old) -> value).oldValue();
        }

        V remove(K key) {
            Objects.requireNonNull(key);
            int h = spread(key);
            ReentrantLock lock = stripeFor(h);
            lock.lock();
```

```
            try {
                Node<K, V>[] t = table;
                int i = indexFor(h, t.length);
                for (Node<K, V> prev = null, n = t[i]; n != null; prev = n, n = n.next) {
                    if (n.hash == h && n.key.equals(key)) {
                        if (prev == null) t[i] = n.next; else prev.next = n.next;
                        size.decrementAndGet();
                        return n.value;
                    }
                }
                return null;
            } finally { lock.unlock(); }
        }

        /** Atomic read-modify-write for one key; returns the new value. Returning null from remap removes the key. */
        V compute(K key, BiFunction<? super K, ? super V, ? extends V> remap) {
            return doCompute(key, remap).newValue();
        }

        private record Result<V>(V oldValue, V newValue) { }

        private Result<V> doCompute(K key, BiFunction<? super K, ? super V, ? extends V> remap) {
            Objects.requireNonNull(key);
            int h = spread(key);
            ReentrantLock lock = stripeFor(h);
            boolean grew = false;
            V old, nv;
            lock.lock();
            try {
                Node<K, V>[] t = table;
                int i = indexFor(h, t.length);
                Node<K, V> prev = null, n = t[i];
                while (n != null && !(n.hash == h && n.key.equals(key))) { prev = n; n = n.next; }
                old = n == null ? null : n.value;
                nv = remap.apply(key, old);
                if (n != null && nv != null) {
                    n.value = nv;                                   // update in place
                } else if (n != null) {                             // remap said "remove"
                    if (prev == null) t[i] = n.next; else prev.next = n.next;
                    size.decrementAndGet();
                } else if (nv != null) {                            // insert at bucket head
                    t[i] = new Node<>(key, h, nv, t[i]);
                    grew = size.incrementAndGet() > t.length * LOAD_FACTOR;
                }
            } finally { lock.unlock(); }
            if (grew) resize();                                     // never resize while holding one stripe
            return new Result<>(old, nv);
        }

        /** Stop-the-world resize: take every stripe lock IN ORDER (no deadlock), rehash, publish. */
        @SuppressWarnings({"unchecked", "rawtypes"})
        private void resize() {
            for (ReentrantLock l : stripes) l.lock();
            try {
                Node<K, V>[] old = table;
                if (size.get() <= old.length * LOAD_FACTOR) return;  // another thread already resized
                Node<K, V>[] nt = (Node<K, V>[]) new Node[old.length * 2];
                for (Node<K, V> head : old) {
                    for (Node<K, V> n = head; n != null; ) {
                        Node<K, V> next = n.next;
                        int j = indexFor(n.hash, nt.length);
                        n.next = nt[j]; nt[j] = n;
                        n = next;
                    }
                }
                table = nt;
            } finally {
                for (int i = stripes.length - 1; i >= 0; i--) stripes[i].unlock();
            }
        }

        int size() { return size.get(); }
        int capacity() { return table.length; }
    }

    static void runThreads(int n, java.util.function.IntConsumer body) throws InterruptedException {
```

```
        List<Thread> ts = new ArrayList<>();
        for (int i = 0; i < n; i++) { final int id = i; Thread t = new Thread(() -> body.accept(id)); ts.add(t);
t.start(); }
        for (Thread t : ts) t.join();
    }

    public static void main(String[] args) throws InterruptedException {
        StripedHashMap<String, Integer> map = new StripedHashMap<>(16, 16);
        map.put("a", 1);
        System.out.println("put a=1, put a=2 returns old=" + map.put("a", 2) + ", get a=" + map.get("a")
                + ", remove a=" + map.remove("a") + ", get a=" + map.get("a"));

        final int THREADS = 8, PER = 20_000;
        runThreads(THREADS, id -> { for (int i = 0; i < PER; i++) map.put("t" + id + "-" + i, i); });
        int missing = 0;
        for (int id = 0; id < THREADS; id++) for (int i = 0; i < PER; i++) if (!Integer.valueOf(i).equals(map.get("t" + id
+ "-" + i))) missing++;
        System.out.println("8 threads x 20,000 puts -> size=" + map.size() + ", missing=" + missing
                + ", table grew from 16 to " + map.capacity() + " buckets");

        StripedHashMap<String, Integer> counters = new StripedHashMap<>(16, 8);
        runThreads(THREADS, id -> {
            for (int i = 0; i < 10_000; i++) counters.compute("hot-" + (i % 10), (k, v) -> v == null ? 1 : v + 1);
        });
        int total = 0;
        for (int i = 0; i < 10; i++) total += counters.get("hot-" + i);
        System.out.println("atomic compute on 10 hot keys: each=" + counters.get("hot-0") + ", total=" + total + "
(expected 80000)");

        runThreads(THREADS, id -> { for (int i = 0; i < PER; i++) map.remove("t" + id + "-" + i); });
        System.out.println("after concurrent removes size=" + map.size());
    }
}
```

Output:

```
put a=1, put a=2 returns old=1, get a=2, remove a=2, get a=null
8 threads x 20,000 puts -> size=160000, missing=0, table grew from 16 to 262144 buckets
atomic compute on 10 hot keys: each=8000, total=80000 (expected 80000)
after concurrent removes size=0
```

Why it's correct

- **Every bucket belongs to exactly one stripe.** Both the stripe count S and the table length are powers of two and $S \le$ table length, so the stripe index (`h & (S-1)`) is just the low bits of the bucket index (`h & (len-1)`). All keys in one bucket therefore share one stripe lock, and two threads can never modify the same chain at the same time. Two operations on the same key obviously share it too.
- **Resize safety:** resize holds *every* stripe lock, so no other operation is in progress; it re-checks the load condition (another thread may have resized first) and publishes the new table with a `volatile` write. Any later operation acquires a stripe lock that the resizer released → happens-before → it sees the new table and moved nodes.
- **No deadlock:** normal operations hold one lock; resize acquires all locks in ascending order and is started only *after* the caller released its own stripe.
- **Atomic compute:** read-modify-write of one key happens entirely under its stripe lock (the demo's 80,000 increments on 10 hot keys are exact).

Pitfalls

- Deriving the stripe from the *bucket index of the current table* — after a resize a key could move to a different lock while another thread holds the old one.
- Resizing while holding one stripe lock and then taking all locks → deadlock with another resizer. Release first, then resize.
- Reading `table` without a lock or `volatile` → a stale table after resize.
- Allowing `null` keys/values makes `get() == null` ambiguous (the JDK's `ConcurrentHashMap` forbids them too).

How the real `ConcurrentHashMap` goes further

Aspect	This design	JDK 8+ `ConcurrentHashMap`
Reads	Lock the stripe	Lock-free (volatile reads of nodes)
Write locking	One of S stripe locks	CAS into empty bins; `synchronized` on the bin's first node otherwise
Long chains	Linked list	Converted to a red-black tree beyond 8 nodes (when table ≥ 64)
Resize	Stop-the-world (all locks; everything else pauses)	Incremental: threads help transfer bins concurrently
Size	`AtomicInteger`	`LongAdder`-style counter cells

Real-world examples

Java 7's `ConcurrentHashMap` used **segments** (16 by default) — the same striping idea. Guava's `Striped<Lock>` offers striped locks for your own data; Caffeine uses striped buffers to record reads.

In the interview

> **30-second answer:** "An array of buckets plus a small array of locks; each key maps to a stripe by its hash bits, so different stripes work in parallel. Every bucket belongs to exactly one stripe. Resize takes all stripe locks in order, rehashes and publishes the new table through a volatile field."

Follow-ups:

- *How to make `get` lock-free?* Make `Node.value` and `next` volatile, publish nodes safely, and never mutate a chain in a way a reader could see half-done (the JDK copies/reorders carefully during transfer).
- *How many stripes?* Around the number of threads expected to write concurrently (JDK 7 default 16); more stripes = less contention but more memory.

Self-check

- **Q:** Why must stripe count divide the table length? **A:** So every bucket maps to exactly one stripe (stripe index = low bits of bucket index).

TL;DR

N locks for M buckets, stripe by hash bits, resize under all locks in order, publish via volatile.

13.8 Thread-safe bounded Blocking Queue (two ways)

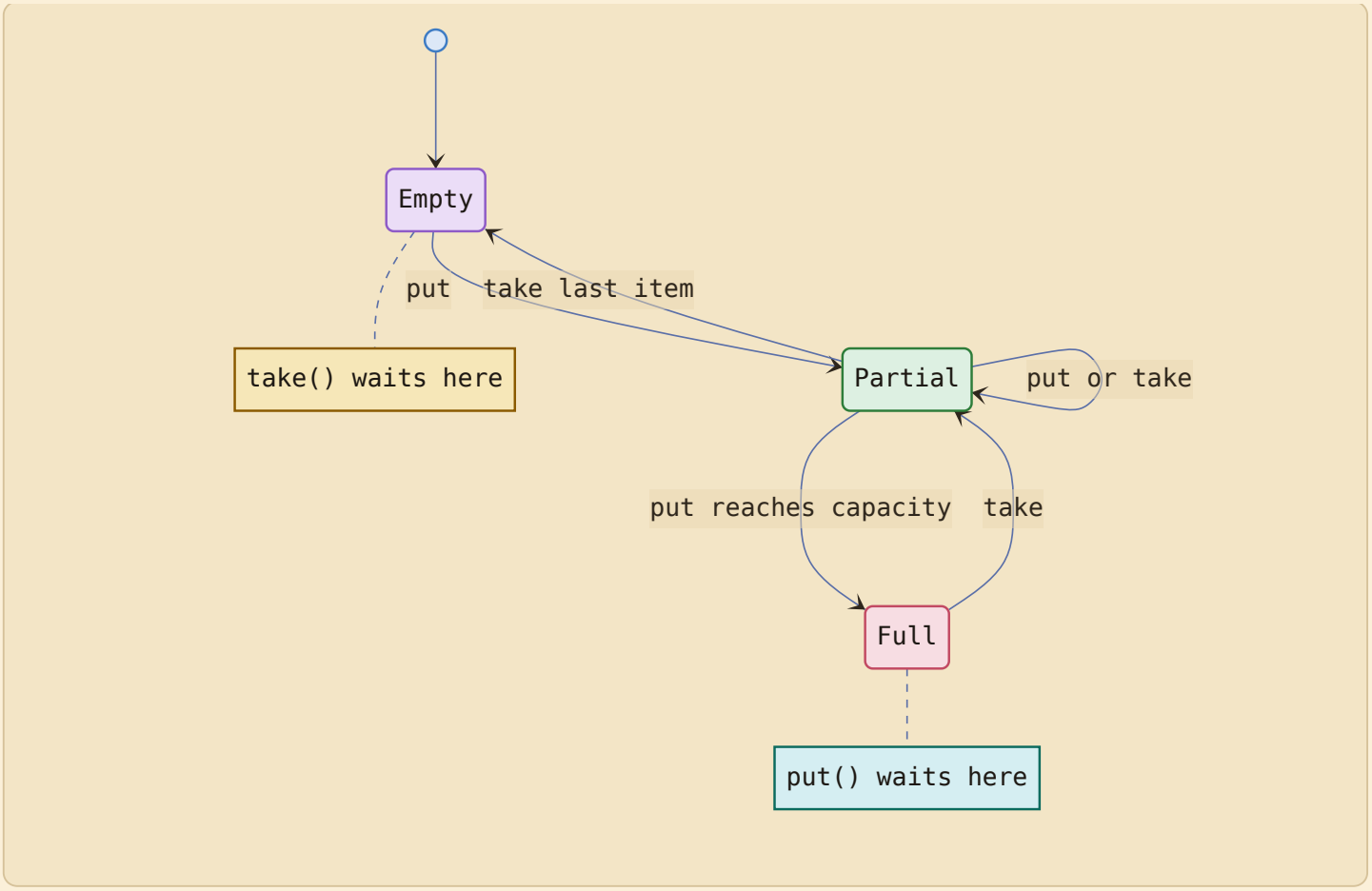
Problem

Implement a fixed-capacity FIFO queue for producer-consumer use: `put` blocks while full, `take` blocks while empty, plus `offer(item, timeout)` and `poll(timeout)` that give up after a timeout.

Approach

Storage: a **circular array** (an array whose end wraps around to the start; `head`, `tail`, `count`). Two synchronisation designs:

	Way 1: monitor	Way 2: `ReentrantLock` + 2 `Condition`s
Lock	`synchronized` methods	`lock.lockInterruptibly()`
Wait	`while (full) wait();`	`while (full) notFull.await();`
Wake	`notifyAll()` — producers and consumers share one wait set	`notEmpty.signal()` after enqueue, `notFull.signal()` after dequeue
Timed wait	`TimeUnit.timedWait(this, nanos)` + deadline loop	`awaitNanos(nanos)` returns the time left
Efficiency	Wakes everyone (thundering herd)	Wakes exactly one thread of the right kind


Empty
put
take last item
take() waits here
Partial
put or take
put reaches capacity
take
Full
put() waits here

Java solution

```java
// File: code/lld/ch13/blockingqueue/Main.java
package lld.ch13.blockingqueue;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicLong;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

/** A bounded blocking queue built two ways: monitor (wait/notifyAll) and Lock + two Conditions. */
public class Main {

    interface BoundedBlockingQueue<T> {
        void put(T item) throws InterruptedException;          // blocks while full
        T take() throws InterruptedException;                  // blocks while empty
        boolean offer(T item, long timeout, TimeUnit unit) throws InterruptedException; // false on timeout
        T poll(long timeout, TimeUnit unit) throws InterruptedException;                // null on timeout
        int size();
    }

    // ---------------------------------------------------------------------
    // Way 1: intrinsic lock. Producers AND consumers wait on the SAME monitor,
    // so we must use notifyAll() (notify() could wake the wrong kind of thread).
    // ---------------------------------------------------------------------
    static final class MonitorQueue<T> implements BoundedBlockingQueue<T> {
        private final Object[] items;
        private int head, tail, count;                         // all guarded by 'this'

        MonitorQueue(int capacity) { items = new Object[capacity]; }

        public synchronized void put(T item) throws InterruptedException {
            while (count == items.length) wait();
            enqueue(item);
            notifyAll();
        }
        public synchronized T take() throws InterruptedException {
            while (count == 0) wait();
            T x = dequeue();
            notifyAll();
            return x;
        }
        public synchronized boolean offer(T item, long timeout, TimeUnit unit) throws InterruptedException {
            long deadline = System.nanoTime() + unit.toNanos(timeout);
            while (count == items.length) {
                long left = deadline - System.nanoTime();
                if (left <= 0) return false;
                TimeUnit.NANOSECONDS.timedWait(this, left);      // wait(millis, nanos) under the hood
            }
            enqueue(item);
            notifyAll();
            return true;
        }
        public synchronized T poll(long timeout, TimeUnit unit) throws InterruptedException {
            long deadline = System.nanoTime() + unit.toNanos(timeout);
            while (count == 0) {
                long left = deadline - System.nanoTime();
                if (left <= 0) return null;
                TimeUnit.NANOSECONDS.timedWait(this, left);
            }
            T x = dequeue();
            notifyAll();
            return x;
        }
        public synchronized int size() { return count; }

        private void enqueue(T item) {
            if (item == null) throw new NullPointerException();  // null is our "timed out" signal
            items[tail] = item;
            tail = (tail + 1) % items.length;                    // circular buffer
            count++;
        }
        @SuppressWarnings("unchecked")
```

```
        private T dequeue() {
            T x = (T) items[head];
            items[head] = null;                                  // let GC reclaim it
            head = (head + 1) % items.length;
            count--;
            return x;
        }
    }

    // ------------------------------------------------------------------
    // Way 2: ReentrantLock with TWO condition queues. Producers wait on notFull,
    // consumers on notEmpty, so a targeted signal() is enough.
    // (This is how java.util.concurrent.ArrayBlockingQueue is built.)
    // ------------------------------------------------------------------
    static final class ConditionQueue<T> implements BoundedBlockingQueue<T> {
        private final Object[] items;
        private int head, tail, count;
        private final ReentrantLock lock = new ReentrantLock();
        private final Condition notFull = lock.newCondition();
        private final Condition notEmpty = lock.newCondition();

        ConditionQueue(int capacity) { items = new Object[capacity]; }

        public void put(T item) throws InterruptedException {
            if (item == null) throw new NullPointerException();
            lock.lockInterruptibly();
            try {
                while (count == items.length) notFull.await();
                enqueue(item);
            } finally { lock.unlock(); }
        }
        public T take() throws InterruptedException {
            lock.lockInterruptibly();
            try {
                while (count == 0) notEmpty.await();
                return dequeue();
            } finally { lock.unlock(); }
        }
        public boolean offer(T item, long timeout, TimeUnit unit) throws InterruptedException {
            if (item == null) throw new NullPointerException();
            long nanos = unit.toNanos(timeout);
            lock.lockInterruptibly();
            try {
                while (count == items.length) {
                    if (nanos <= 0) return false;
                    nanos = notFull.awaitNanos(nanos);           // returns the time still left
                }
                enqueue(item);
                return true;
            } finally { lock.unlock(); }
        }
        public T poll(long timeout, TimeUnit unit) throws InterruptedException {
            long nanos = unit.toNanos(timeout);
            lock.lockInterruptibly();
            try {
                while (count == 0) {
                    if (nanos <= 0) return null;
                    nanos = notEmpty.awaitNanos(nanos);
                }
                return dequeue();
            } finally { lock.unlock(); }
        }
        public int size() { lock.lock(); try { return count; } finally { lock.unlock(); } }

        private void enqueue(T item) {
            items[tail] = item;
            tail = (tail + 1) % items.length;
            count++;
            notEmpty.signal();                                   // wake ONE waiting consumer
        }
        @SuppressWarnings("unchecked")
        private T dequeue() {
            T x = (T) items[head];
            items[head] = null;
            head = (head + 1) % items.length;
            count--;
```

```
            notFull.signal();                                   // wake ONE waiting producer
            return x;
        }
    }

    // ------------------------------------------------------------------
    static void stressTest(String name, BoundedBlockingQueue<Integer> q) throws InterruptedException {
        final int PRODUCERS = 3, CONSUMERS = 3, PER_PRODUCER = 20_000;
        AtomicLong consumedSum = new AtomicLong(), consumedCount = new AtomicLong();
        List<Thread> ts = new ArrayList<>();
        for (int p = 0; p < PRODUCERS; p++) {
            Thread t = new Thread(() -> {
                try { for (int i = 1; i <= PER_PRODUCER; i++) q.put(i); }
                catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            });
            ts.add(t);
        }
        int perConsumer = PRODUCERS * PER_PRODUCER / CONSUMERS;
        for (int c = 0; c < CONSUMERS; c++) {
            Thread t = new Thread(() -> {
                try { for (int i = 0; i < perConsumer; i++) { consumedSum.addAndGet(q.take());
consumedCount.incrementAndGet(); } }
                catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            });
            ts.add(t);
        }
        ts.forEach(Thread::start);
        for (Thread t : ts) t.join();
        long expected = (long) PRODUCERS * PER_PRODUCER * (PER_PRODUCER + 1) / 2;
        System.out.println(name + ": consumed " + consumedCount.get() + " items, sum ok=" + (consumedSum.get() ==
expected)
                + ", left in queue=" + q.size());

        System.out.println(name + ": poll(30ms) on empty -> " + q.poll(30, TimeUnit.MILLISECONDS));
        q.put(1); q.put(2);                                     // capacity is 2 in this demo
        System.out.println(name + ": offer(30ms) on full -> " + q.offer(3, 30, TimeUnit.MILLISECONDS));
    }

    public static void main(String[] args) throws InterruptedException {
        stressTest("wait/notifyAll  ", new MonitorQueue<>(2));
        stressTest("Lock+Conditions ", new ConditionQueue<>(2));
    }
}
```

Output:

```
wait/notifyAll  : consumed 60000 items, sum ok=true, left in queue=0
wait/notifyAll  : poll(30ms) on empty -> null
wait/notifyAll  : offer(30ms) on full -> false
Lock+Conditions : consumed 60000 items, sum ok=true, left in queue=0
Lock+Conditions : poll(30ms) on empty -> null
Lock+Conditions : offer(30ms) on full -> false
```

Why it's correct

- All fields are accessed only while holding the single lock → no races; the lock gives visibility.
- **Waiting in `while` loops** handles spurious wake-ups and "someone else took the item first".
- **Way 1** must use `notifyAll()`: with one wait set, `notify()` might wake a producer when a consumer was needed (or vice versa), and the right thread might never wake → deadlock-like hang.
- **Way 2** can use `signal()` because producers only wait on `notFull` and consumers only on `notEmpty`; every enqueue makes exactly one more consumer able to proceed, every dequeue exactly one producer.
- Timed versions track the *remaining* time so repeated spurious wakeups can't extend the total wait.
- Stress test: 3 producers × 20,000 items, 3 consumers, capacity 2 (maximum blocking) → all 60,000 items consumed, sum correct, queue empty.

Pitfalls

- `if` instead of `while`.
- `notify()` with a single monitor shared by producers and consumers.
- Forgetting `items[head] = null` after dequeue → the array keeps references alive (memory leak for large objects).

- Accepting `null` items when `null` means "timed out" in `poll`.
- Using `lock()` instead of `lockInterruptibly()` makes cancellation harder (the JDK's queues use `lockInterruptibly` in blocking methods).

Real-world examples

`java.util.concurrent.ArrayBlockingQueue` is built exactly like Way 2 (one lock, `notEmpty`/`notFull`). `LinkedBlockingQueue` uses **two** locks (`putLock`, `takeLock`) so a producer and a consumer can work at the same time. Thread pools, logging frameworks, and Kafka's producer internals all rely on blocking hand-off.

In the interview

> **30-second answer:** "A circular array with head, tail and count guarded by one lock. put waits while count == capacity, take waits while count == 0, both in while loops. With synchronized I must use notifyAll because both sides share one wait set; with ReentrantLock I use two Conditions, notFull and notEmpty, and signal just the right side."

Follow-ups:

- *Higher throughput?* Two-lock queue (linked list, separate put/take locks, atomic count) like `LinkedBlockingQueue`, or a lock-free ring buffer (LMAX Disruptor, a high-performance Java messaging library) for single producer/consumer.
- *Unbounded?* Remove the "full" condition — but then there's no back-pressure.

Self-check

- **Q:** Why is `signal()` safe in Way 2 but `notify()` unsafe in Way 1? **A:** Separate condition queues guarantee the woken thread is of the right kind.

TL;DR

Circular buffer + one lock + "wait while full/empty" in loops. Monitor → `notifyAll`; Lock → `notFull`/`notEmpty` + `signal`.

13.9 Concurrent Bloom Filter

Problem

Design a thread-safe **Bloom filter** — a compact probabilistic set that answers *"definitely not present"* or *"probably present"* — that many threads can add to and query at the same time.

Approach

How a Bloom filter works (first-time explanation):

1. Keep an array of m bits, all 0.
2. To **add** an item, compute k hash positions and set those k bits to 1.
3. To **query**, compute the same k positions: if **any** bit is 0 the item was **definitely never added**; if all are 1 it was **probably** added (the bits might have been set by other items → a *false positive*).
4. No false negatives, no deletion (clearing a bit could erase other items; a *counting* Bloom filter with small counters supports deletes).

Sizing for n expected items and target false-positive rate p:

- bits: $m = \lceil -n \cdot \ln p / (\ln 2)^2 \rceil$
- hashes: $k = \text{round}((m / n) \cdot \ln 2)$

For n = 80,000 and p = 1% → $m \approx$ 766,805 bits (≈ 93.6 KiB — kibibytes of 1,024 bytes; the demo prints 93 because it uses integer division) and k = 7 — about 9.6 bits per item, whatever the item size.

Double hashing (Kirsch–Mitzenmacher): compute two hashes $h1$, $h2$ once and use positions $h1 + i \cdot h2$ for i = 0..k−1 — as good as k independent hashes in practice.

Thread safety without locks:

- Bits live in an `AtomicLongArray` (64 bits per word).

- Adding a bit = **atomic OR** via `getAndAccumulate(word, mask, (a, b) -> a | b)` (a CAS loop inside). Two threads setting different bits in the same word both succeed.
- Queries are plain volatile reads of words. Bits only ever go 0 → 1, so a concurrent query can at worst miss an item whose `add` hasn't finished — which is fine, because that add hadn't happened yet. After `add` returns, every later query sees its bits (happens-before of atomic operations).

Java solution

```java
// File: code/lld/ch13/bloomfilter/Main.java
package lld.ch13.bloomfilter;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicLongArray;

/**
 * A thread-safe Bloom filter: a compact set that answers "definitely NOT present" or
 * "PROBABLY present". No false negatives; a tunable rate of false positives.
 * Lock-free: bits live in an AtomicLongArray and are set with an atomic OR.
 */
public class Main {

    static final class ConcurrentBloomFilter {
        private final AtomicLongArray words;     // m bits packed into 64-bit words
        private final int numBits;               // m
        private final int numHashes;             // k

        /** Size the filter for 'expectedItems' at the target false-positive probability. */
        ConcurrentBloomFilter(int expectedItems, double falsePositiveRate) {
            double ln2 = Math.log(2);
            long m = (long) Math.ceil(-expectedItems * Math.log(falsePositiveRate) / (ln2 * ln2));
            if (m > Integer.MAX_VALUE - 63) throw new IllegalArgumentException("too large");
            this.numBits = (int) m;
            this.numHashes = Math.max(1, (int) Math.round((double) m / expectedItems * ln2));
            this.words = new AtomicLongArray((numBits + 63) / 64);
        }

        /** Adds the item. Returns true if at least one bit changed (i.e. it was definitely new). */
        boolean add(String item) {
            long base = fnv1a64(item);
            long h1 = fmix64(base), h2 = fmix64(base ^ 0x9E3779B97F4A7C15L) | 1;   // h2 odd
            boolean changed = false;
            for (int i = 0; i < numHashes; i++) {
                int bit = (int) Math.floorMod(h1 + i * h2, (long) numBits);        // double hashing
                long mask = 1L << (bit & 63);
                long before = words.getAndAccumulate(bit >>> 6, mask, (cur, m) -> cur | m);  // atomic OR (CAS loop)
                if ((before & mask) == 0) changed = true;
            }
            return changed;
        }

        boolean mightContain(String item) {
            long base = fnv1a64(item);
            long h1 = fmix64(base), h2 = fmix64(base ^ 0x9E3779B97F4A7C15L) | 1;
            for (int i = 0; i < numHashes; i++) {
                int bit = (int) Math.floorMod(h1 + i * h2, (long) numBits);
                if ((words.get(bit >>> 6) & (1L << (bit & 63))) == 0) return false;   // definitely absent
            }
            return true;
        }

        int numBits() { return numBits; }
        int numHashes() { return numHashes; }

        // 64-bit FNV-1a over the string's chars, then a strong bit-mixer (MurmurHash3's finalizer).
        private static long fnv1a64(String s) {
            long h = 0xcbf29ce484222325L;
            for (int i = 0; i < s.length(); i++) { h ^= s.charAt(i); h *= 0x100000001b3L; }
            return h;
        }
        private static long fmix64(long k) {
            k ^= k >>> 33; k *= 0xff51afd7ed558ccdL;
            k ^= k >>> 33; k *= 0xc4ceb9fe1a85ec53L;
            k ^= k >>> 33;
            return k;
        }
    }

    public static void main(String[] args) throws InterruptedException {
        final int THREADS = 8, PER_THREAD = 10_000, N = THREADS * PER_THREAD;
        ConcurrentBloomFilter bf = new ConcurrentBloomFilter(N, 0.01);
```

```
        System.out.println("sized for n=" + N + ", p=1%: m=" + bf.numBits() + " bits (" + bf.numBits() / 8 / 1024
                + " KiB), k=" + bf.numHashes() + " hashes");

        List<Thread> ts = new ArrayList<>();
        for (int t = 0; t < THREADS; t++) {
            final int id = t;
            Thread th = new Thread(() -> { for (int i = 0; i < PER_THREAD; i++) bf.add("user-" + id + "-" + i); });
            ts.add(th); th.start();
        }
        for (Thread th : ts) th.join();

        int falseNegatives = 0;
        for (int t = 0; t < THREADS; t++) for (int i = 0; i < PER_THREAD; i++) if (!bf.mightContain("user-" + t + "-" +
i)) falseNegatives++;
        System.out.println("false negatives after 8-thread insert: " + falseNegatives);

        int probes = 100_000, falsePositives = 0;
        for (int i = 0; i < probes; i++) if (bf.mightContain("stranger-" + i)) falsePositives++;
        System.out.printf("false-positive rate on %,d unseen keys: %.2f%% (target 1%%)%n", probes, 100.0 * falsePositives
/ probes);
        System.out.println("add(\"user-0-0\") again changes bits? " + bf.add("user-0-0"));
    }
}
```

Output:

```
sized for n=80000, p=1%: m=766805 bits (93 KiB), k=7 hashes
false negatives after 8-thread insert: 0
false-positive rate on 100,000 unseen keys: 1.01% (target 1%)
add("user-0-0") again changes bits? false
```

Why it's correct

- **No lost updates:** a plain `words[i] |= mask` on a `long[]` is read-modify-write and two threads could overwrite each other's bits → false negatives. The atomic OR retries on conflict, so every bit set survives.
- **Monotonic state:** bits never go back to 0, so there's no ABA issue and no need for locks.
- **Deterministic result:** OR is commutative, so the final bit array is the same whatever the thread interleaving — the demo's false-positive rate (≈1%) is identical every run.
- The measured false-positive rate matches the formula's target.

Pitfalls

- Non-atomic bit setting → false negatives (the one thing a Bloom filter must never produce).
- Poor hash functions (e.g. using `String.hashCode()` directly with `i * 31` offsets) → clustered bits and much higher false-positive rates. Mix bits (here FNV-1a + MurmurHash3's finalizer — two fast, non-cryptographic hash functions).
- Adding far more than n items → false-positive rate climbs quickly; resize by creating a larger filter (or a "scalable" chain of filters).
- `Math.floorMod` on `long` to avoid negative indices.

Real-world examples

- **Apache Cassandra, HBase, RocksDB, LevelDB** (databases and storage engines): Bloom filters per **SSTable** (Sorted String Table — an immutable sorted data file on disk)/file to skip disk reads for keys that aren't there.
- **Google Guava** `BloomFilter` (its `put`/`mightContain` are thread-safe since Guava 23 using atomic long arrays).
- **Redis** (RedisBloom module / Redis Stack) offers `BF.ADD`/`BF.EXISTS`.
- Checking whether a username is already taken, or a URL already crawled, before hitting the database; Medium has described using Bloom filters to avoid recommending already-read articles.

In the interview

30-second answer: "A bit array of size m with k hash positions per item from double hashing; add sets k bits, query checks them — any zero means definitely absent. For concurrency I store bits in an AtomicLongArray and set them with an atomic OR; since bits only go from 0 to 1, reads need no lock. m and k come from n and the target false-positive rate."

Follow-ups:

- *Deletions?* Counting Bloom filter (small counters instead of bits, atomic increments/decrements) or a Cuckoo filter (a Bloom-filter alternative that supports deletion).
- *Distributed?* Keep it in Redis (RedisBloom) or rebuild per node from a shared source.

Self-check

- **Q:** Can a Bloom filter say "absent" for an item that was added? **A:** No — no false negatives (assuming atomic, completed adds).
- **Q:** Bits per element for 1% FPR (false-positive rate)? **A:** About 9.6.

TL;DR

Bits + k hashes; "no" is certain, "yes" is probable. `AtomicLongArray` + atomic OR = lock-free and correct.

13.10 Multi-threaded Merge Sort (Fork/Join)

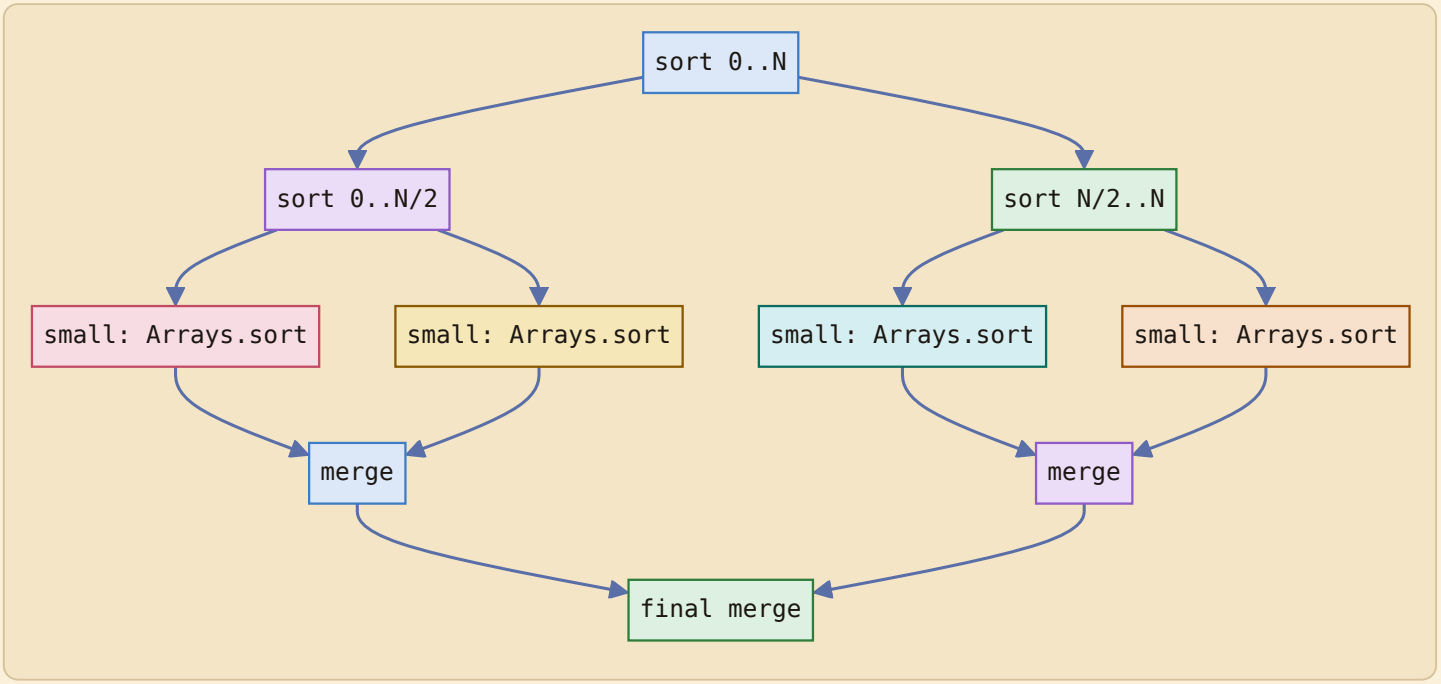
Problem

Sort a large `int[]` using all CPU cores.

Approach

Merge sort is naturally **divide and conquer**: sort the left half and right half *independently* (in parallel), then merge. Java's **Fork/Join framework** (`ForkJoinPool`, `RecursiveAction`/`RecursiveTask`) is built for this:

1. If the range is small (≤ 8,192 elements) → sort it sequentially (`Arrays.sort`); splitting further costs more than it saves.
2. Otherwise split at `mid = (lo + hi) >>> 1` and `invokeAll(left, right)` — forks both halves and waits for both.
3. Merge the two sorted halves using a **shared scratch buffer**; each task only uses its own `[lo, hi)` slice, so there's no sharing between concurrent tasks.
4. **Work stealing:** each pool thread has its own deque of tasks; idle threads steal tasks from the other end of busy threads' deques, keeping all cores busy.



Java solution

```java
// File: code/lld/ch13/mergesort/Main.java
package lld.ch13.mergesort;

import java.util.Arrays;
import java.util.Random;
import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.RecursiveAction;

/** Multi-threaded merge sort with the Fork/Join framework (divide and conquer + work stealing). */
public class Main {

    static final class ParallelMergeSort extends RecursiveAction {
        private static final long serialVersionUID = 1L;
        static final int THRESHOLD = 1 << 13;   // below ~8K elements, splitting costs more than it saves
        private final int[] a, buf;             // buf = one shared scratch array; tasks use disjoint ranges
        private final int lo, hi;               // sorts a[lo, hi)

        ParallelMergeSort(int[] a, int[] buf, int lo, int hi) { this.a = a; this.buf = buf; this.lo = lo; this.hi = hi; }

        @Override protected void compute() {
            if (hi - lo <= THRESHOLD) {
                Arrays.sort(a, lo, hi);                         // sequential base case
                return;
            }
            int mid = (lo + hi) >>> 1;                          // overflow-safe midpoint
            invokeAll(new ParallelMergeSort(a, buf, lo, mid),   // fork both halves, wait for both
                      new ParallelMergeSort(a, buf, mid, hi));
            merge(mid);
        }

        private void merge(int mid) {
            if (a[mid - 1] <= a[mid]) return;                   // already in order: skip
            System.arraycopy(a, lo, buf, lo, hi - lo);
            int i = lo, j = mid, k = lo;
            while (i < mid && j < hi) a[k++] = buf[i] <= buf[j] ? buf[i++] : buf[j++];   // <= keeps it stable
            while (i < mid) a[k++] = buf[i++];
            // right-half leftovers are already in place
        }
    }

    static void parallelSort(int[] a) {
        ForkJoinPool.commonPool().invoke(new ParallelMergeSort(a, new int[a.length], 0, a.length));
    }

    public static void main(String[] args) {
        int[] small = {38, 27, 43, 3, 9, 82, 10, 3};
        parallelSort(small);
        System.out.println("small: " + Arrays.toString(small));

        int n = 4_000_000;
        int[] data = new Random(42).ints(n).toArray();
        int[] expected = data.clone();
        Arrays.sort(expected);

        int[] mine = data.clone();
        parallelSort(mine);
        System.out.println(n + " random ints: matches Arrays.sort = " + Arrays.equals(mine, expected));

        int[] jdk = data.clone();
        Arrays.parallelSort(jdk);                               // the JDK's own parallel sort
        System.out.println("Arrays.parallelSort agrees        = " + Arrays.equals(jdk, expected));
    }
}
```

Output:

```
small: [3, 3, 9, 10, 27, 38, 43, 82]
4000000 random ints: matches Arrays.sort = true
Arrays.parallelSort agrees        = true
```

Why it's correct

- The two subtasks work on **disjoint** ranges of both `a` and `buf` → no data race between them.
- `invokeAll` returns only when both subtasks completed; Fork/Join's `join` establishes happens-before, so the parent's merge sees the children's writes.
- The merge takes from the left run on ties (`<=`), so it's **stable** (equal elements keep their original order; matters for objects, not for ints).
- The early exit `a[mid-1] <= a[mid]` skips merging already-ordered halves.
- The result is compared with `Arrays.sort` on 4 million random ints (seeded, so reproducible).

Pitfalls

- No threshold → millions of tiny tasks; overhead dominates.
- Allocating a new buffer in every merge → heavy GC (garbage collection — the JVM's automatic freeing of unused memory); allocate once.
- Calling `fork()` on both halves and `join()`ing in the wrong order, or doing blocking I/O inside Fork/Join tasks (it's meant for CPU work).
- `(lo + hi) / 2` can overflow for huge arrays; `>>> 1` avoids it.
- Expecting speed-up on tiny arrays or on a single core (Amdahl's law: the part that must run sequentially caps the total speed-up — here the final merge).

Real-world examples

- `Arrays.parallelSort` (Java 8+) splits the array, sorts the pieces in parallel on the common Fork/Join pool and merges them (the demo cross-checks against it).
- Parallel streams (`list.parallelStream()`) run on `ForkJoinPool.commonPool()`.
- Big-data sorts (Hadoop MapReduce's — a model that maps over data in parallel, then combines the results — sort/shuffle, Spark's `sortBy`) apply the same divide-sort-merge idea across machines.

In the interview

> **30-second answer:** "A RecursiveAction splits the range in half until it's below a threshold, sorts small pieces sequentially, then merges using one preallocated buffer. invokeAll runs both halves in parallel on the ForkJoinPool, whose work stealing keeps cores busy. Subtasks touch disjoint ranges, so no locks are needed."

Follow-ups:

- *`RecursiveAction` vs `RecursiveTask`?* `RecursiveTask<V>` returns a value (e.g. parallel sum); `RecursiveAction` returns nothing (in-place sort).
- *Can the merge be parallel too?* Yes — split the merge by binary-searching the median of one run in the other (parallel merge), which is how fully parallel merge sorts scale further.

Self-check

- **Q:** Why is a shared `buf` safe? **A:** Concurrent tasks use non-overlapping index ranges of it.

TL;DR

Split until small, sort sequentially, merge; Fork/Join + work stealing; disjoint ranges = no locks.

13.11 Rate limiter — token bucket (thread-safe)

Problem

Limit how often clients may call an **API** (Application Programming Interface — here, a web service that clients send requests to): allow short **bursts** up to *B* requests, and a sustained rate of *R* requests per second. Must be correct under many concurrent requests, per client (user id / API key — a secret string that identifies the caller / IP address).

Approach

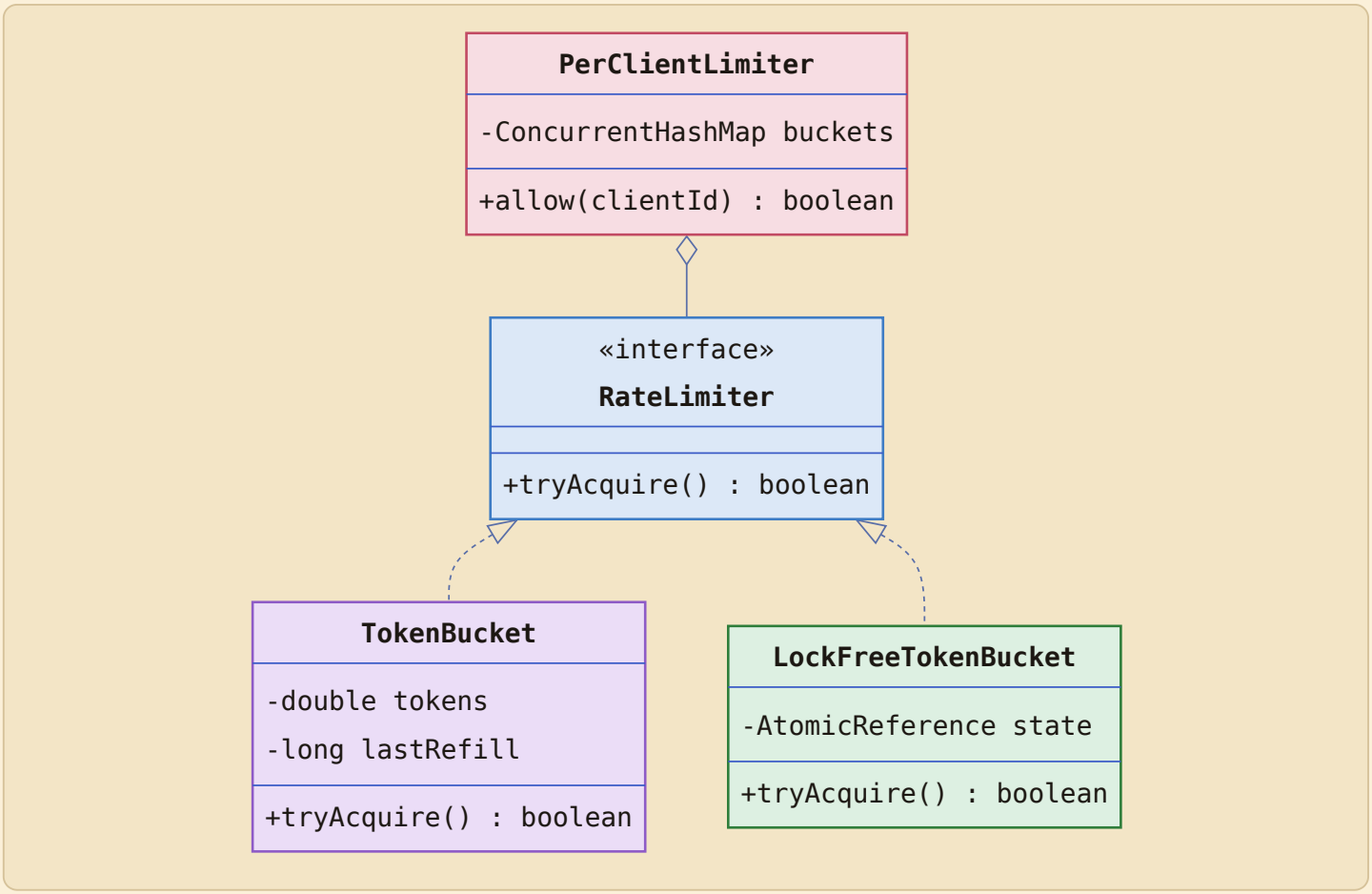
Token bucket (first-time explanation):

1. A bucket holds up to *capacity* tokens (the allowed burst) and starts full.
2. Tokens drip in at *refillRate* per second, never exceeding capacity.
3. Each request takes one token. No token → reject (HTTP 429 "Too Many Requests"; HTTP is the web's request/response protocol — see System Design Fundamentals).
4. Implement refill **lazily**: store `tokens` and `lastRefill`; on each request add `elapsed × rate` tokens. No background thread needed.

Thread safety: "refill + check + take" must be **one atomic step** — otherwise two threads can both see 1 token and both take it.

- Version 1: `synchronized tryAcquire()` — simple and correct.
- Version 2: **lock-free** — put `(tokens, lastRefill)` into one immutable `State` record inside an `AtomicReference` and CAS the whole state (the "several fields → one immutable object → one CAS" trick from 12.15).
- Per client: `ConcurrentHashMap.computeIfAbsent(clientId, …)` creates exactly one bucket per client.
- Time comes from an injected `Ticker` so the demo is deterministic.

Algorithm	Bursts	Memory	Notes
Token bucket	Allows up to capacity	2 numbers	Most common for APIs
Leaky bucket	Smooths to constant output rate	Queue	Good for shaping traffic (evening out bursts)
Fixed window counter	Allows 2× burst at window edges	1 counter	Simplest
Sliding window log	Exact	One timestamp per request	Memory-heavy
Sliding window counter	Approximate, smooth	2 counters	Good compromise



Java solution

```java
// File: code/lld/ch13/ratelimiter/Main.java
package lld.ch13.ratelimiter;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;
import java.util.concurrent.atomic.AtomicReference;
import java.util.function.Supplier;

/** Thread-safe token-bucket rate limiter: a lock-based version, a lock-free version, and per-user limits. */
public class Main {

    @FunctionalInterface interface Ticker { long nanos(); }

    static final class FakeTicker implements Ticker {
        private final AtomicLong now = new AtomicLong();
        public long nanos() { return now.get(); }
        void advance(long amount, TimeUnit unit) { now.addAndGet(unit.toNanos(amount)); }
    }

    interface RateLimiter { boolean tryAcquire(); }

    /**
     * Bucket holds up to 'capacity' tokens (the allowed burst). Tokens drip in at 'refillPerSecond'.
     * Each request takes one token; no token -> rejected. Refill is computed lazily from elapsed time.
     */
    static final class TokenBucket implements RateLimiter {
        private final double capacity, tokensPerNano;
        private final Ticker ticker;
        private double tokens;          // guarded by 'this'
        private long lastRefill;        // guarded by 'this'

        TokenBucket(int capacity, double refillPerSecond, Ticker ticker) {
            this.capacity = capacity;
            this.tokensPerNano = refillPerSecond / 1e9;
            this.ticker = ticker;
            this.tokens = capacity;     // start full
            this.lastRefill = ticker.nanos();
        }

        public synchronized boolean tryAcquire() {
            long now = ticker.nanos();
            tokens = Math.min(capacity, tokens + (now - lastRefill) * tokensPerNano);   // refill + acquire are ONE atomic
step
            lastRefill = now;
            if (tokens >= 1) { tokens -= 1; return true; }
            return false;
        }
    }

    /** Same algorithm, lock-free: the whole state is one immutable record swapped with CAS. */
    static final class LockFreeTokenBucket implements RateLimiter {
        private record State(double tokens, long lastRefill) { }
        private final double capacity, tokensPerNano;
        private final Ticker ticker;
        private final AtomicReference<State> state;

        LockFreeTokenBucket(int capacity, double refillPerSecond, Ticker ticker) {
            this.capacity = capacity;
            this.tokensPerNano = refillPerSecond / 1e9;
            this.ticker = ticker;
            this.state = new AtomicReference<>(new State(capacity, ticker.nanos()));
        }

        public boolean tryAcquire() {
            while (true) {
                State cur = state.get();
                long now = Math.max(ticker.nanos(), cur.lastRefill());   // never move time backwards
                double refilled = Math.min(capacity, cur.tokens() + (now - cur.lastRefill()) * tokensPerNano);
```

```
                boolean ok = refilled >= 1;
                State next = new State(ok ? refilled - 1 : refilled, now);
                if (state.compareAndSet(cur, next)) return ok;           // lost the race? recompute and retry
            }
        }
    }

    /** One bucket per client key (user id, API key, IP). computeIfAbsent creates each bucket exactly once. */
    static final class PerClientLimiter {
        private final ConcurrentHashMap<String, RateLimiter> buckets = new ConcurrentHashMap<>();
        private final Supplier<RateLimiter> factory;
        PerClientLimiter(Supplier<RateLimiter> factory) { this.factory = factory; }
        boolean allow(String clientId) { return buckets.computeIfAbsent(clientId, k -> factory.get()).tryAcquire(); }
    }

    static String burst(RateLimiter rl, int requests) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < requests; i++) sb.append(rl.tryAcquire() ? 'Y' : 'n');
        return sb.toString();
    }

    static int concurrentBurst(RateLimiter rl, int threads) throws InterruptedException {
        AtomicInteger allowed = new AtomicInteger();
        CountDownLatch go = new CountDownLatch(1);
        List<Thread> ts = new ArrayList<>();
        for (int i = 0; i < threads; i++) {
            Thread t = new Thread(() -> {
                try { go.await(); } catch (InterruptedException e) { return; }
                if (rl.tryAcquire()) allowed.incrementAndGet();
            });
            ts.add(t); t.start();
        }
        go.countDown();
        for (Thread t : ts) t.join();
        return allowed.get();
    }

    public static void main(String[] args) throws InterruptedException {
        for (String kind : List.of("synchronized", "lock-free")) {
            FakeTicker clock = new FakeTicker();
            RateLimiter rl = kind.equals("synchronized") ? new TokenBucket(5, 2, clock) : new LockFreeTokenBucket(5, 2,
clock);
            System.out.println("[" + kind + "] capacity 5, refill 2/s");
            System.out.println("  t=0.0s  7 requests : " + burst(rl, 7));
            clock.advance(1, TimeUnit.SECONDS);
            System.out.println("  t=1.0s  3 requests : " + burst(rl, 3));
            clock.advance(250, TimeUnit.MILLISECONDS);
            System.out.println("  t=1.25s 1 request  : " + burst(rl, 1) + "  (only 0.5 token so far)");
            clock.advance(250, TimeUnit.MILLISECONDS);
            System.out.println("  t=1.5s  1 request  : " + burst(rl, 1));
            clock.advance(1, TimeUnit.HOURS);
            System.out.println("  after 1 idle hour  : allowed " + concurrentBurst(rl, 100) + " of 100 concurrent requests
(burst capped at 5)");
        }

        FakeTicker clock = new FakeTicker();
        PerClientLimiter api = new PerClientLimiter(() -> new TokenBucket(3, 1, clock));
        StringBuilder alice = new StringBuilder(), bob = new StringBuilder();
        for (int i = 0; i < 4; i++) { alice.append(api.allow("alice") ? 'Y' : 'n'); }
        for (int i = 0; i < 2; i++) { bob.append(api.allow("bob") ? 'Y' : 'n'); }
        System.out.println("per-client (3 burst): alice " + alice + ", bob " + bob + "  (alice's spam doesn't hurt bob)");
    }
}
```

Output:

```
[synchronized] capacity 5, refill 2/s
  t=0.0s  7 requests : YYYYYnn
  t=1.0s  3 requests : YYn
  t=1.25s 1 request  : n  (only 0.5 token so far)
  t=1.5s  1 request  : Y
  after 1 idle hour  : allowed 5 of 100 concurrent requests (burst capped at 5)
[lock-free] capacity 5, refill 2/s
  t=0.0s  7 requests : YYYYYnn
  t=1.0s  3 requests : YYn
  t=1.25s 1 request  : n  (only 0.5 token so far)
  t=1.5s  1 request  : Y
  after 1 idle hour  : allowed 5 of 100 concurrent requests (burst capped at 5)
per-client (3 burst): alice YYYn, bob YY  (alice's spam doesn't hurt bob)
```

Why it's correct

- **Atomic decision:** in `TokenBucket`, refill, check and decrement happen inside one `synchronized` method. In `LockFreeTokenBucket`, the new state is computed from one snapshot and installed with `compareAndSet`; if another thread changed the state first, the CAS fails and we recompute — so each token is granted at most once.
- **Burst cap:** tokens are clamped to `capacity`, so after an idle hour only 5 of 100 simultaneous requests pass (as the output shows for both versions).
- **Rate:** at 2 tokens/s, after 1 s exactly 2 new tokens exist; at +0.25 s only 0.5 token → rejected; at +0.5 s → 1 token → allowed.
- **Per-client isolation:** `computeIfAbsent` is atomic, so two concurrent first requests from the same client can't create two buckets (which would double their limit).
- The lock-free version clamps `now` with `Math.max(..., lastRefill)` so a thread that read the clock slightly earlier can't move time backwards.

Pitfalls

- Separate `refill()` and `take()` calls → race; two threads spend the same token.
- A background thread per bucket to add tokens → thousands of threads/timers; lazy refill needs none.
- Using `currentTimeMillis()` (wall clock can jump) instead of `nanoTime()`.
- An unbounded map of buckets → memory leak for one-off clients; evict idle buckets (e.g. with the TTL cache from 13.6).
- In a multi-server deployment an in-memory limiter only limits per server. For a global limit, keep the bucket in **Redis** (an in-memory data store shared by all servers) and update it atomically with a Lua script (a small script that Redis runs as one uninterruptible step) (or use a sliding-window counter in Redis).

Real-world examples

- **Stripe** has described using token buckets (backed by Redis) for its API rate limits.
- **Amazon API Gateway** (AWS's managed front door for APIs) **throttling** (deliberately rejecting or delaying excess requests) is configured with a *rate* and a *burst* — token-bucket parameters.
- **Guava** `RateLimiter`, **Bucket4j** (token bucket for Java, with Redis/JCache — the standard Java caching API — back-ends), and **Resilience4j** `RateLimiter` are production Java libraries; NGINX's (a popular web server and reverse proxy) `limit_req` uses a leaky-bucket approach.

In the interview

30-second answer: "Token bucket: capacity for bursts, refill rate for the average. I refill lazily from elapsed nanoTime on each request, and make refill-check-take one atomic step — synchronized, or lock-free by CASing an immutable state record. A ConcurrentHashMap gives one bucket per client via computeIfAbsent; across servers I'd move the bucket into Redis with a Lua script."

Follow-ups (with answers):

- *How do you tell the client when to retry?* Compute `(1 − tokens) / rate` seconds and return it in a `Retry-After` header (a response header that tells the client how many seconds to wait).
- *Different limits per plan (free vs paid)?* The factory creates buckets with capacity/rate from the client's plan (Strategy pattern).
- *Blocking instead of rejecting?* Compute the wait time and sleep/park until a token will exist (Guava's `acquire()` does this).

Self-check

- **Q:** Capacity 5, rate 2/s, bucket empty at t=0. How many requests pass at t=3s? **A:** min(5, 3 × 2) = 5.
- **Q:** Why one immutable `State` in the lock-free version? **A:** `tokens` and `lastRefill` must change together; one CAS on one reference updates both atomically.

TL;DR

Bucket of tokens: capacity = burst, rate = average. Lazy refill; refill+take atomic (lock or CAS on immutable state); one bucket per client; Redis for multi-server.

13.12 Chapter summary

Problem	Key primitive	The one invariant to state in the interview
FooBar	2 semaphores	Exactly one permit exists across both
Zero Even Odd	3 semaphores	Zero dispatches; one permit circulates
FizzBuzz	monitor + `notifyAll`	Predicates partition 1..n
H2O	2 semaphores + barrier	≤2 H and ≤1 O per round; slot freed after release
Dining Philosophers	ordered locks / n−1 semaphore	No circular wait
TTL cache	CHM + `compute` + conditional remove	Never return or keep expired data; load once per key
Striped map	lock array	Each bucket belongs to exactly one stripe
Blocking queue	lock + 2 conditions	0 ≤ count ≤ capacity; wait in loops
Bloom filter	atomic OR	Bits only go 0 → 1; no false negatives
Merge sort	Fork/Join	Subtasks touch disjoint ranges
Rate limiter	lock or CAS on immutable state	Refill + take is one atomic step

13.13 Chapter self-check

1. **Q:** In FooBar, why doesn't it matter which thread starts first? **A:** The one without a permit blocks until the other releases it.
2. **Q:** In FizzBuzz, what bug appears with `notify()`? **A:** The wrong thread may be woken and go back to sleep; the right one never wakes → hang.
3. **Q:** In H2O, why release the slot after printing? **A:** So the next molecule cannot form (and print) until this one has fully left.
4. **Q:** Two ways to prevent Dining Philosophers deadlock? **A:** Always take the lower-numbered fork first; or allow at most n−1 philosophers to try at once.
5. **Q:** Why does the TTL cache use `remove(key, entry)`? **A:** To avoid deleting a fresh value written concurrently.
6. **Q:** Why must the striped map's stripe depend only on the hash? **A:** So a key's lock doesn't change when the table resizes.
7. **Q:** When is `signal()` safe instead of `signalAll()`? **A:** When each condition queue holds only threads waiting for the same thing and one state change enables exactly one waiter.
8. **Q:** What would a non-atomic OR do to a Bloom filter? **A:** Lose bits → false negatives.
9. **Q:** Why a threshold in parallel merge sort? **A:** Small tasks cost more to schedule than to sort.
10. **Q:** How do you make a token bucket global across 10 servers? **A:** Store it in Redis and update it atomically with a Lua script.

13.14 Chapter TL;DR

- **Ordering puzzles** (FooBar, ZeroEvenOdd, H2O): semaphores as batons, barriers for groups; keep "exactly one permit" style invariants.
- **Predicate waiting** (FizzBuzz, blocking queue): lock + `while (!condition) wait/await` + `notifyAll` or targeted `signal`.

- **Multi-lock problems** (philosophers): break circular wait with ordering or limiting.
- **Thread-safe data structures** (cache, map, queue, Bloom filter, rate limiter): immutable entries, atomic compound operations, striping, CAS on immutable state, and injected clocks for testable time.
- **Parallel algorithms** (merge sort): Fork/Join with thresholds and disjoint data.